

MSP430Family

Семейство микроконтроллеров MSP430

Рекомендации по применению



C12 Семейство микроконтроллеров MSP430.

Рекомендации по применению: Пер. с англ. — М.:

Серия «Библиотека Компэла». ЗАО «Компэл», 2005. — 544 с.

ISBN 5-98730-002-9

Данное издание представляет собой сборник статей сотрудников компании Texas Instruments, посвященных различным аспектам применения микроконтроллеров семейства MSP430. В сборник вошли переводы наиболее интересных, с точки зрения издательства, статей, опубликованные в период с октября 1997 года по декабрь 2005 года.

Рекомендации по применению несомненно окажут помощь отечественным инженерам-разработчикам в их профессиональной работе по созданию новых электронных устройств на базе популярных микроконтроллеров компании Texas Instruments.



ISBN 5-98730-002-9

© ЗАО «Компэл», 2005

MSP430Family

Содержание



К ЧИТАТЕЛЮ.....	13
I. ОБЩИЕ ВОПРОСЫ.....	16
Глава 1. Выбор микроконтроллера со сверхнизким потреблением.....	16
Введение.....	16
Среднее значение потребляемого тока.....	16
Режимы пониженного потребления.....	17
Системы синхронизация.....	18
Прерывания.....	19
Периферийные устройства.....	20
Защита от кратковременного пропадания напряжения питания.....	21
Ток утечки выводов.....	21
Эффективность обработки.....	21
Заключение.....	22
Глава 2. Сравнение микроконтроллеров семейства MSP430 с другими микроконтроллерами.....	24
Влияние аппаратного умножителя и полной оптимизации компилятора.....	27
Приложение.....	28
Тестовые прикладные задачи.....	28
Компилятор.....	29
Исходные тексты тестовых прикладных задач.....	30
8-bit Math.c.....	30
8-bit 2-dim Matrix.c.....	30
8-bit Switch Case.c.....	31
16-bit Math.c.....	33
16-bit 2-dim Matrix.c.....	34
16-bit Switch Case.c.....	35
32-bit Math.c.....	36
Floating-point Math.c.....	37
FIR Filter.c.....	38
Matrix Multiplication.c.....	39
Глава 3. Подключение EEPROM к MSP430 с помощью интерфейса I²C.....	42
1. Пример подключения.....	42
2. Исходный код для MSP430.....	43
2.1. Запись Байта (Byte Write).....	44
2.2. Чтение Текущего Адреса (Current Address Read).....	45
2.3. Чтение Произвольного Адреса (Random Address Read).....	46
2.4. Опрос Подтверждения (Acknowledge Polling).....	46
3. Ссылки:.....	46
Глава 4. MSP430F21x1 Обзор архитектуры.....	48
1. Введение.....	48
2. Обновления.....	49
2.1. Микропотребляющий модуль сброса Brownout Reset.....	49
2.2. Повышение тактовой частоты.....	49
2.3. Улучшенная встроенная Flash-память.....	51
2.4. Улучшенная защита программной памяти.....	52
2.5. Улучшения, касающиеся сторожевого (Watchdog) таймера.....	53

2.6. Дополнительные возможности	54
3. Архитектура ядра MSP430	54
4. Ресурсы для отладки	57
5. Заключение	57
II. Использование внутренних функций, периферийных блоков микроконтроллеров	60
Глава 1. Отличительные особенности программатора на основе загрузочного сектора MSP430	60
1. Введение	60
2. Стандартная функция сброса (RESET) и активизация BSL	61
2.1. Микроконтроллеры с флеш-памятью MSP430 в 20- и 28-выводном корпусе с выводами JTAG, объединёнными с портами общего назначения	61
2.2. Микроконтроллеры с флеш-памятью MSP430 с обособленными выводами JTAG	62
3. Протокол УАПП (UART)	63
4. Последовательность синхронизации	63
5. Команды	64
5.1. Незащищённые команды	64
5.2. Команды, защищённые паролем	64
6. Процесс программирования	64
7. Формат посылки	65
7.1. Структура данных	65
7.2. Контрольная сумма	65
7.3. Команды – детальное описание	67
7.3.1. Общие положения	67
7.3.2. Принять блок данных (RX Data Block)	67
7.3.3. Принять пароль (RX Password)	68
7.3.4. Полное стирание (Mass Erase)	68
7.3.5. Стереть сегмент (Erase Segment)	69
7.3.6. Проверка стирания (Erase Check)	69
7.3.7. Сменить скорость обмена (Change Baud Rate)	70
7.3.8. Загрузить программный счётчик (Load PC)	71
7.3.9. Передать блок данных (TX Data Block)	71
7.3.10. Передать версию BSL (TX BSL Version)	72
8. Загружаемые BSL	72
9. Выход из BSL	74
10. Защита паролем	74
11. Предохранитель защиты данных (Code Protection Fuse)	74
12. Внутренние настройки и ресурсы BSL	75
12.1. Идентификатор чипа и версия BSL	75
12.2. Вектора	75
12.3. Статус после инициализации	75
12.4. Распределение памяти и используемые ресурсы	76
13. Отличительные особенности версии BSL 1.10	77
14. Ссылки:	78
Приложение А. Различия между версиями BSL в различных микроконтроллерах	79

Глава 2. Реализация часов реального времени на MSP430	86
1. Введение	86
1.1. Применение часов реального времени	87
1.2. Задающий генератор MSP430	87
2. Цифровая коррекция фазового сдвига	87
2.1. Формирование синхроимпульсов	88
2.2. Выбор Таймера	88
2.3. Внешний интерфейс	88
2.4. Описание принципиальной схемы	89
2.5. Потребляемый ток	89
2.6. Проблемы обеспечения точности	90
2.7. Расширяемость	90
3. Обсуждение прикладной программы	91
4. Заключение	92
Ссылки:	92
Приложение A: пример программы	92
A.1 Использование сторожевого таймера – файл RTC11xWD.s43	92
A.2 Использование таймера Timer_A – файл RTC11xTA.s43 File	96
Глава 3. Использование широтно-импульсной модуляции таймера В в качестве ЦАП	102
1. Введение	102
2. Теория функционирования	102
2.1. Разрешение	103
2.2. Частота	105
2.3. Используемые ресурсы MSP430	105
2.4. Принципиальная схема и осциллограммы	106
2.5. Требования к фильтрам	108
2.6. Суммирование постоянного и переменного сигналов	109
3. Описание программы	110
3.1. Калибровка генератора DCO	111
Ссылки:	113
Приложение: Листинг программы	113
Глава 4. Техника самопрограммирования флэш-памяти MSP430	122
1. Введение	122
2. Состав Flash-модуля	123
3. Стирание и программирование Flash-модуля	124
3.1. Программирование Flash памяти с копированием прикладной программы в ОЗУ	124
3.2. Прямое перепрограммирование Flash памяти	126
4. Демонстрационная программа	127
5. Заключение	128
Ссылки:	129
Приложение A: файл flash_ram.s43	129
Приложение B: файл flash_idle.c	133
Приложение C: файл flash_var.s43	135

III. ИНТЕРФЕЙС С ВНЕШНИМИ УСТРОЙСТВАМИ	138
Глава 1. Подключение ADS1100 к MSP430F413	138
Описание аппаратной части	138
Описание программы	138
Краткий обзор	138
Работа программы	139
Библиотечная функция ведущего I ² C интерфейса	140
· void I2CSetup(void)	141
· unsigned int I2CRead16(unsigned char Addr)	141
· void I2CWrite8(unsigned char Addr, unsigned char Data)	141
Глава 2. Подключение 3-вольтовых микроконтроллеров MSP430 к 5-вольтовым схемам	144
1. Введение	144
2. Определения	145
2.1. Значения из спецификаций MSP430	145
2.2. Определения для внешней системы	146
3. Входные преобразователи	146
3.1. Входные преобразователи на резистивных делителях	146
3.2. Входной преобразователь на транзисторе	148
3.3. Входной преобразователь на операционном усилителе	150
3.4. Входной преобразователь на микросхеме ULN2003A	150
3.5. Входные преобразователи на интегральных микросхемах	151
3.6. Входные преобразователи аналоговых сигналов	152
4. Выходные преобразователи	153
4.1. Выходной преобразователь на транзисторе	153
4.2. Выходной преобразователь для входов КМОП – TTL	154
4.3. Согласование со входами микросхемы ULN2003	154
4.4. Выходной преобразователь на операционном усилителе	155
4.5. Выходные преобразователи на интегральных микросхемах	155
5. Двухнаправленные преобразователи	156
5.1. Простой двухнаправленный преобразователь на ОУ	156
5.2. Двухнаправленный преобразователь на интегральных микросхемах	158
6. Источники питания	159
7. Заключение	160
Ссылки:	160
Глава 3. Подключение MSP430 и температурного датчика TMP100 с интерфейсом I²C	162
1. Введение	162
2. Теория функционирования	163
3. Аппаратный интерфейс	165
4. Работа системы	165
5. Заключение	166
Ссылки:	166
Глава 4. Реализация функции UART с использованием таймера A3	168
Введение	168

Описание UART, реализованного на модуле таймера Timer_A микроконтроллера MSP430x11x (1)	169
Вычисление скорости передачи данных	171
Программная загрузка	171
Демонстрационная схема	172
Демонстрационная управляющая программа 11x1_uart1.s43	172
Демонстрационная управляющая программа 11x1_uart2.s43 – использование DCO для задания скорости обмена данными	173
Демонстрационная управляющая программа 11x1_uart3.s43 – использование 32 кГц кварцевого резонатора для задания скорости обмена данными	174
Ссылки:	174
Глава 5. Подключение MSP430 к EEPROM серии 24Cxx с помощью I²C-интерфейса	176
Введение	176
Теория функционирования	176
Демонстрационная схема	178
Прикладная программа	178
Ссылки:	178
Глава 6. Реализация шины SMBus на базе микроконтроллера MSP430	180
1. Введение	180
2. Аппаратная часть	182
3. Реализация ведущего устройства шины SMBus	183
4. Примеры реализации программы ведущего устройства шины SMBus	184
4.1. Поиск устройств на шине	184
4.2. Чтение данных смарт-батареи	185
5. Реализация ведомого устройства шины SMBus	185
6. Заключение	186
Ссылки:	186
Приложение А. Программа ведущего устройства шины SMBus	186
Приложение В. Программа поиска адресов	209
Приложение С. Программа работы со смарт-батареей	219
Приложение D. Программа ведомого устройства шины SMBus	254
Приложение E. Содержимое файла ASCII.txt	260
Глава 7. IrDA SIR кодер-декодер на MSP430C112	262
1. Введение	262
2. Технические требования к IrDA интерфейсу на физическом уровне	262
3. MSP430x112 – микроконтроллер со сверхнизким потреблением и минимальными функциональными возможностями	264
4. IrDA кодер-декодер на микроконтроллере MSP430x112	264
4.1. Краткий обзор модуля	264
4.2. Управляющая программа MSP430x112	265
4.2.1. Общие вопросы	265
4.2.2. IrDA-SIR кодер (передатчик)	266
4.2.3. IrDA-SIR декодер (приемник)	268
4.3. Установки	270

4.3.1. Выбор скорости передачи данных	270
4.3.2. Интерфейс пользователя	271
5. Описание схемы	272
6. Заключение	274
Ссылки:	274
Приложение А: исходный текст программы	274
IV. ПРИМЕРЫ КОНКРЕТНЫХ ПРИМЕНЕНИЙ	290
Глава 1. Использование TPS3619 совместно с микроконтроллерами MSP430 может уменьшить системное энергопотребление	290
Глава 2. Ультразвуковое измерение дальности на MSP430	294
1. Теория работы	294
2. Описание схемы	295
3. Программа Ultrasonic.s43	299
3.1. Инициализация Init_Device	299
3.2. Основной цикл Mainloop	299
3.3. Математические вычисления Math_calc	300
3.4. Обработчик прерывания базового таймера BT_ISR	301
3.5. Индикация Display	301
3.6. Задержка Delay	301
4. Заключение	301
Ссылки:	302
Приложение: Листинг программы	302
Глава 3. Декодирование ТВ ИК ДУ-сигналов используя Таймер А3	314
1. Введение	314
2. Выбор рабочей частоты	316
3. Демонстрационная схема	316
4. Программа декодирования	317
5. Протокол RC5	318
Программа 11x1_rc5.243	321
Протокол SIRC	323
Программа 11x1_SRC.s43	321
6. Программа UART	323
Ссылки:	323
Приложение А: программа 11x1_SIRC.s43.txt	323
Приложение В: программа 11x1_RC5.s43.txt	332
Глава 4. Компенсация фазового сдвига и калибровка токового трансформатора	342
1. Введение	342
2. Цифровая коррекция фазового сдвига	343
2.1. Описание задачи	343
2.2. FIR фильтр	344
3. Калибровка СТ	345
3.1. Эмуляция поп-UPF нагрузки	345
3.2. Вычисление ошибки	346
3.3. Установка таблицы параметров КИХ	346

3.4. СТ с большим фазовым сдвигом	347
3.5. Изменение частоты измеряемого сигнала	348
4. Заключение	348
Ссылки:	348
Глава 5. Твердотельный диктофон с использованием MSP430	350
1. Введение	350
2. Аппаратная часть	351
2.1. Краткий обзор системы	351
2.2. Аналоговая аппаратная часть	351
2.3. Цифровая аппаратная часть	354
3. Программное обеспечение	355
4. Демонстрационная программа F149VoiceDemo.s43	357
Ссылки:	364
Глава 6. ШИМ-управление двигателем постоянного тока	366
1. Введение	366
2. Теория функционирования	366
3. Демонстрационная управляющая программа	368
Ссылки:	368
Глава 7. Милливольтметр сигма-дельта типа на MSP430F11x1	370
1. Введение	370
2. Теория функционирования	371
2.1. Расчет номиналов элементов ФНЧ 1 битного ЦАП	372
2.2. Используемые ресурсы MSP430F11x1	373
2.3. Требующиеся внешние компоненты	373
2.4. Снижение системной ошибки	373
3. Описание программы fet_intADC.s43	375
3.1. Программное обеспечение для основного 12 битного АЦП	376
3.2. Использование стандартных процедур компании TI	377
4. Заключение	378
Ссылки:	386
Глава 8. Недорогой однофазный счетчик электроэнергии на MSP430C11x	388
1. Введение	388
2. Аналого-цифровое преобразование	389
2.1. Одно-градиентный АЦП	389
2.2. Формирование пилообразного сигнала	390
2.2.1. Период пилообразного сигнала	390
2.2.2. Практический пример	391
2.2.3. Измерение временных параметров пилообразного сигнала	392
2.3. Двухдиапазонный АЦП для канала измерения тока	393
2.4. Автоматическое переключение коэффициента усиления	395
3. Защита от внешнего вмешательства, предотвращающая воровство электроэнергии	395
4. Потоки данных в системе	396
4.1. Детектирование кадров	397
4.2. Исключение постоянной составляющей	398

4.3. Блок выбора канала тока	399
4.4. Регулятор усиления	399
4.5. Фильтр нижних частот	400
5. Преобразователь энергия-импульс	400
6. Калибровка	402
7. Заключение	402
Ссылки:	402
Глава 9. Цифровой термометр на базе MSP430	404
1. Введение	404
1.1. Краткое описание микроконтроллеров со сверхнизким энергопотреблением семейства MSP430	404
1.2. Описание схемы	405
2. Описание работы	406
3. Возможности таймера – порта Timer Port	407
4. Заключение	408
Ссылки:	408
Приложение: Листинг программы	408
Глава 10. FSK модуляция и демодуляция с использованием микроконтроллера MSP430	420
1. Введение	420
2. Теория демодуляции	420
2.1. Выбор частоты дискретизации	421
2.2. Управление оконечными устройствами	421
2.3. Демодуляция FSK	421
2.4. Синхронизация бита	422
3. Теории модуляции	422
3.1. Выбор частоты дискретизации	423
3.2. Построение таблицы значений	423
3.3. FSK модуляция	423
4. Преобразование данных	423
4.1. Аналого-цифровое преобразование	423
4.2. Цифро-аналоговое преобразование	424
5. Потребляемая мощность	424
6. Программное обеспечение	424
6.1. FSK приемник	425
6.2. FSK передатчик	425
7. Типовая схема	425
7.1. Использование MSP430C325 в качестве центрального процессора	425
7.2. Пример телефонного интерфейса	425
8. Заключение	426
Ссылки:	427
Приложение А: подпрограмма FSK приемника	427
Приложение В: подпрограмма FSK передатчика	441
Глава 11. Программный встроенный модем на MSP430	454
1. Введение	454

2. Использование модема стандарта V.21 на телефонных сетях общего пользования (ТФС ОП).....	454
3. Описания аппаратной части	457
3.1. Описание схемы демонстрационной платы	457
3.2. Отладочная плата LITELINK III (DAA)	458
3.3. Подключение к DAA	458
4. Описание программы	461
4.1. Модуль детектора вызывного сигнала	461
4.2. Модуль V.21	464
4.3. Демонстрационное применение программного модема	473
5. Используемые аппаратные ресурсы MSP430	474
6. Заключение	474
Ссылки:	474
Приложение А. Схема электрическая принципиальная программного модема	475
Приложение В. Перечень элементов программного модема	476
Глава 12. Генерация и распознавание DTMF-сигналов	478
Часть I.....	478
1. Введение	478
2. Спецификация сигналов DTMF	478
3. Генерация сигналов DTMF	480
3.1. Генерация при помощи прямоугольных сигналов	480
3.2. Программное обеспечение для генерации меандра	481
3.2.1. Генерация прямоугольных сигналов с использованием 8-битного таймера и таймера – порта Timer Port	482
3.2.2. Генерация прямоугольных сигналов с использованием таймера Timer_A	490
3.3. Аппаратная часть для генерации сигналов DTMF	498
4. Результаты исследований сигналов DTMF	503
5. Заключение	505
Ссылки:	506
Часть II.....	507
1. Введение	507
2. Прием сигналов DTMF с помощью цифровых фильтров-резонаторов	507
3. Основы цифровых фильтров.....	507
3.1. Свойства цифровых фильтров-резонаторов	508
3.2. Структура используемых цифровых фильтров-резонаторов	509
4. Представление чисел и арифметика.....	512
5. Расчет 8-ми цифровых фильтров-резонаторов и оптимизация коэффициентов	514
6. Проверка расчета фильтров с помощью программ математического моделирования	517
7. Программное обеспечение для реализации цифровых фильтров	519
8. Программа распознавания сигналов DTMF	521
9. Аппаратная часть.....	539
10. Измерения и результаты	541
11. Заключение	542
Ссылки:	542

К ЧИТАТЕЛЮ



Развитие отечественной электроники невозможно без современной элементной базы, отлаженных технологий производства, но в первую очередь без уникальных технических решений. Инженеры-разработчики — не побоюсь избитого штампа — локомотив отечественного производства электронной техники создают эти технические решения. Именно разработчикам адресована книга, которую вы, уважаемый читатель, держите сейчас в руках.

Данная книга представляет собой сборник статей, написанных инженерами компании Texas Instruments, посвященных различным аспектам применения микроконтроллеров семейства MSP430. Опыт инженеров одного из лидеров мирового производства высокоинтеллектуальных электронных компонентов без сомнения будет полезен отечественным разработчикам. На сайте компании Texas Instruments <http://www.ti.com> представлено огромное количество интересной технической информации, однако языковой барьер порой мешает русскоязычным инженерам в полном объеме пользоваться этим ресурсом.

Этот факт был одним из побуждающих мотивов, для начала публикаций по данной тематике.

«Рекомендации по применению микроконтроллеров MSP430» — это уже второе издание, посвященное микроконтроллерам данного семейства, выпущенное нашей компанией. Первая книга «Семейство микроконтроллеров MSP430x1xx — руководство пользователя» вышла из печати в сентябре 2004 года и вызвала живой интерес со стороны инженерно-технических сотрудников различных предприятий, разрабатывающих и производящих радиоэлектронную аппаратуру, преподавателей и студентов ряда технических университетов. Такой значительный интерес к микроконтроллерам MSP430 обусловлен уникальным набором качеств, свойственных данному семейству: сверхнизкое потребление энергии, высокая надежность и низкая цена, при одновременном обеспечении широкого набора функциональных возможностей.

Учитывая эту популярность, мы планируем регулярно публиковать материалы, посвященные микроконтроллерам данного семейства. Практически готова к изданию книга, посвященная микроконтроллерам MSP430x4xx, а в ближайших планах описание контроллеров новейшей серии MSP430x2xx.

Ваши замечания и предложения по
материалам данной книги просим
присылать по адресу:
E-mail: TI@compel.ru

Бренд-менеджер
по продукции Texas Instruments
компании «Компэл» – Илья Фурман.

I. Общие вопросы

Глава 1.

Выбор микроконтроллера со сверхнизким потреблением

Выбор микроконтроллера со сверхнизким потреблением

Mike Mitchell [slaa207.pdf (Англ.)]

В данном примере применения описывается, как сравнить микроконтроллеры со сверхнизким потреблением. Будут обсуждены основные различия между популярными микроконтроллерами со сверхнизким потреблением, а также рассмотрено как интерпретировать характеристики и технические требования и соотнести их с техническими требованиями к устройству.

Введение

Для современных устройств с автономным питанием потребление управляющего микроконтроллера становится все более важным параметром. Большинство поставщиков предлагают ряд микроконтроллеров со сверхнизким потреблением, но выбор наилучшего для Вашего устройства решения более сложен, чем простое просматривание и сравнение первой страницы технического описания. Для выбора наиболее оптимального с точки зрения потребления решения необходимо очень подробно сравнить характеристики микроконтроллеров во всех режимах работы (включая режимы пониженного потребления), рассмотреть способы синхронизации, характеристики обработки событий, встроенные периферийные устройства, наличие функции обнаружения и защиты от кратковременного пропадания напряжения питания, величину тока утечки и эффективность вычислительного ядра.

Среднее значение потребляемого тока

В низкопотребляющих устройствах среднее значение потребляемого тока определяет срок службы аккумулятора. Например, если для питания устройства используется источник питания с емкостью 400 мАч, то, для того чтобы проработать без замены или подзарядки источника питания один год, система должна иметь среднее значение потребляемого тока менее $400 \text{ мАч} / 8760 \text{ ч} = 45.7 \text{ мкА}$. На рисунке 1 показано, что для выполнения этого условия система может потреблять большее значение тока в течение коротких интервалов, а в остальное время должно иметь меньшее потребление.

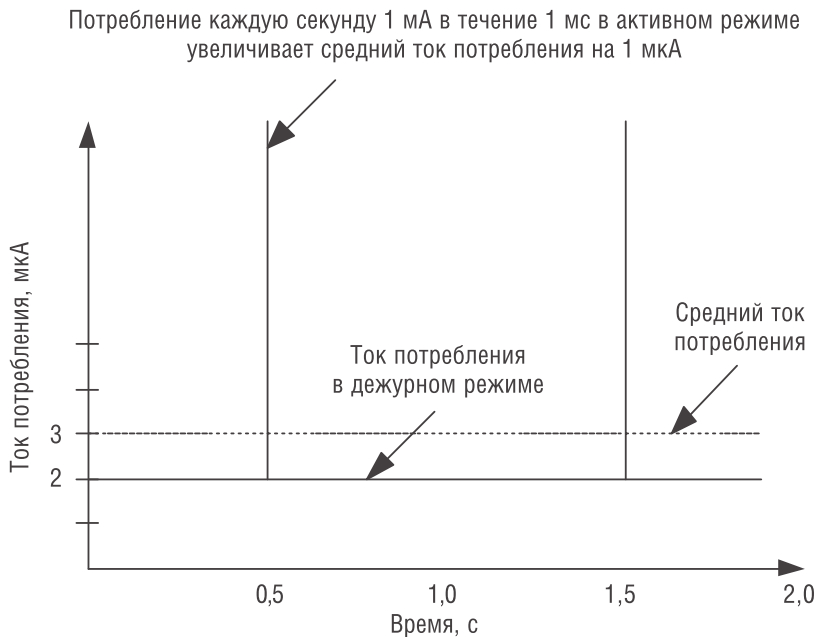


Рис. 1. Среднее значение потребляемого тока

Режимы пониженного потребления

Наличие режимов пониженного потребления – самая важная характеристика, дающая возможность микроконтроллерам удовлетворить требованиям к среднему значению потребляемого тока. Низкопотребляющие микроконтроллеры имеют режимы пониженного потребления, в которых они, тем не менее, способны выполнять различные функции. Например, микроконтроллеры семейства MSP430 имеют пять режимов пониженного потребления. В режиме Low-power mode zero (LPM0) отключается вычислительное ядро, но все остальные функции остаются активными. В режимах LPM1 и LPM2 в список отключенных функций добавляются различные функции синхронизации. LPM3 – наиболее часто используемый режим пониженного потребления, в котором остается активным только низкочастотный автогенератор и все периферийные устройства, использующие для работы синхросигналы этого автогенератора. LPM3 часто применяется для работы в прерывистом режиме, когда низкочастотный генератор применяется в качестве часов реального времени ввиду

того, что он работает от 32768 Гц кварцевого резонатора и потребляет менее 1 мкА. В режиме LPM4 выключаются все встроенные задающие генераторы, что приводит к автоматическому отключению всех синхронных периферийных устройств. В этом режиме аналоговые периферийные устройства могут оставаться в активном режиме, но если отключить и их, то микроконтроллер (даже осуществляя регенерацию ОЗУ) будет потреблять всего 100 нА.

Системы синхронизация

Во многом потребление микроконтроллера зависит от системы синхронизации. Устройства могут входить в режим пониженного потребления и выходить из него от нескольких раз в секунду до нескольких сотен раз в секунду. Способность быстро входить в такие режимы и выходить из них, а также быстро обрабатывать данные после активизации, является критическим параметром, так как после «пробуждения» микроконтроллер некоторое время простаивает, ожидая установления стабильных синхроимпульсов и потребляя довольно большой ток. Большинство низкопотребляющих микроконтроллеров содержат так называемые «мгновенные» (“instant-on”) автогенераторы, которые выходят на стабильный режим менее чем за 10-20 нс. Но важно понять, какие генераторы являются «мгновенными», а какие – нет. Некоторые микроконтроллеры имеют две ступени синхронизации при пробуждении: сначала синхронизация обеспечивается от низкочастотного (обычно 32768 Гц) задающего генератора, в то время как высокочастотный генератор выходит на стабильный режим генерации только через несколько миллисекунд или даже дольше. В таких микроконтроллерах вычислительное ядро может начать работу уже через 15 мкс, но с более низкой или нестабильной частотой. Если вычислительное ядро должно выполнить несколько команд, например 25, то потребуется приблизительно 763 мкс при частоте 32768 Гц. Вычислительное ядро потребляет на низкой частоте значительно меньше, но и производительность его также существенно ниже. С другой стороны, вычислительное ядро может работать на высокой, но неправильной или нестабильной частоте. В этом случае время обработки мало, но приложение не может выполнить точный расчет временных параметров. Если требуется точная синхронизация, то вычислительное ядро должно ожидать установления стабильных синхросигналов. Микроконтроллеры семейства MSP430 имеют время выхода высокочастотного задающего генератора в стабильный режим менее 6 мкс (а часто даже меньше), что позволяет выполнить те же 25 команд всего за 9 мкс (6 мкс пробуждение + 25 команд * 125 мкс). На рисунке 2 показаны временные диаграммы запуска «мгновенного» 8 МГц генератора, которому для выхода в режим стабильной генерации требуется всего 292 нс.

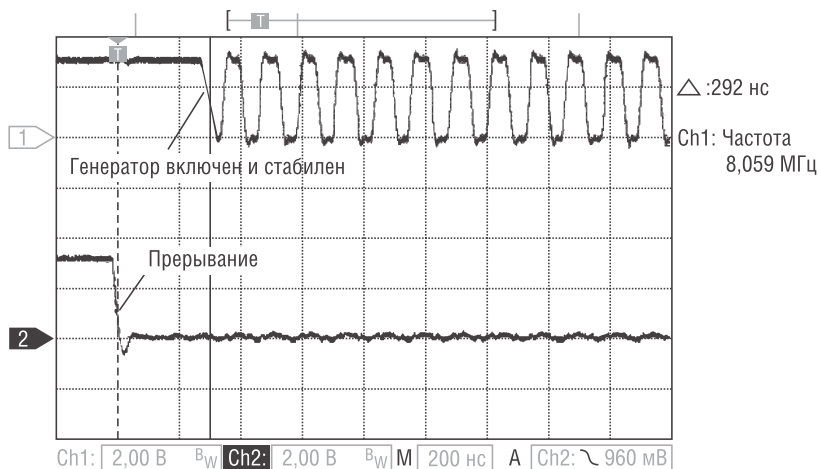


Рис. 1. Временные диаграммы запуска высокочастотного «мгновенного» задающего генератора микроконтроллеров семейства MSP430

Кроме того, если система синхронизации микроконтроллера способна формировать несколько синхросигналов, то периферийные устройства могут работать при остановленном вычислительном ядре. Например, АЦП микроконтроллера требуются высокочастотные синхроимпульсы. Если система синхронизации микроконтроллера способна формировать несколько синхронизирующих последовательностей, то при работе АЦП вычислительное ядро может быть остановлено, что позволит снизить потребление системы в целом.

Прерывания

Система обработки событий идет рука об руку с системой синхронизации. Прерывания способны вывести микроконтроллер из режима пониженного потребления, поэтому, чем больше имеется прерываний, тем больше гибкости микроконтроллер имеет для обеспечения снижения общего потребления. Опрос состояний выводов и периферийных устройств приведет к повышению потребления, так как это потребует дополнительного времени нахождения микроконтроллера в активном режиме. Хороший низкопотребляющий микроконтроллер должен иметь развитую систему прерываний для обработки внешних событий. Микроконтроллеры семейства MSP430 имеют обработчик прерывания от 16 линий портов ввода-вывода общего назначения и от всех периферийных устройств. Некоторые периферийные устройства, как например таймеры Timer_A

и Timer_B и АЦП ADC12, для обеспечения большей гибкости имеют несколько прерываний.

Наибольшую выгоду от развитой системы прерывания получают устройства с кнопками управления или клавиатурой для ввода данных. Не имеющие развитой системы прерывания микроконтроллеры должны периодически опрашивать клавиатуру или кнопки. Мало того, что это требует времени, так это ведет еще и повышению потребления. Микроконтроллеры с развитой системой прерывания могут бездействовать все время, пока не будет нажата какая-нибудь кнопка.

Периферийные устройства

Потребление и система управления питанием периферийных устройств также должны учитываться при выборе микроконтроллера. Некоторые низкопотребляющие микроконтроллеры изготавливаются на основе архитектур 20 – 30 летней давности, которые не были рассчитаны на обеспечение низкого потребления. Микроконтроллеры семейства MSP430 изначально разрабатывались как низкопотребляющие, и поэтому их периферийные модули также имеют низкое потребление. Об этом свидетельствует наличие возможности индивидуального их включения – выключения или, что еще более важно, автоматического включения – выключения периферийных модулей. АЦП ADC12 микроконтроллеров семейства MSP430 – пример таких интеллектуальных периферийных устройств. Если оно не выполняет преобразования, то оно не потребляет тока. При отсутствии преобразования оно автоматически блокирует свой встроенный задающий генератор и цифровую схему обработки. Перед началом преобразования модуль автоматически включается или перезапускается.

Кроме того, периферийные устройства могут управляться прерываниями от других периферийных устройств. Например, АЦП ADC12 микроконтроллеров семейства MSP430 может быть запущен таймером Timer_A или Timer_B. Это позволяет синхронизировать выборки АЦП и выполнять их без использования вычислительной мощности ядра. АЦП ADC12 может активизировать вычислительное ядро уже после окончания преобразования.

Некоторые микроконтроллеры семейства MSP430 имеют функцию прямого доступа к памяти (DMA), которая позволяет автоматически обрабатывать данные без вмешательства вычислительного ядра. Использование контроллера DMA позволяет не только увеличить скорость обработки данных, но, что еще более важно, и снизить потребляемую мощность. Использование DMA для автоматической записи данных, например в ОЗУ, позволяет вычислительному ядру быть отключенным, а АЦП в это время работать и выполнять преобразования. Вычислительное ядро активизируется уже после того, как все необходимые преобразования выполнены.

Управление синхронизацией периферийных модулей – другая важная характеристика, которая позволяет снизить средний ток потребления системы. I2C модуль микроконтроллеров семейства MSP430 имеет непосредственное управление системой синхронизации. Это означает, что, если I2C модулю нужны синхросигналы, то он автоматически активизирует задающий генератор. Это также позволяет вычислительному ядру оставаться отключенным. При этом имеется возможность декодировать I2C адрес даже при отключенном вычислительном ядре.

Защита от кратковременного пропадания напряжения питания

Многие микроконтроллеры имеют встроенную защиту от кратковременного пропадания напряжения питания, которая перезапускает его в тех случаях, когда напряжение питания снижается ниже нормального уровня. У большинства микроконтроллеров эта схема защиты потребляет от 10 до 70 мкА. Некоторые микроконтроллеры с целью снижения потребления имеют возможность отключения схемы защиты от кратковременного пропадания напряжения питания, но для обеспечения эффективности работы этой схемы необходимо, чтобы она работала все время, ведь кратковременные пропадания напряжения питания невозможно предсказать. Поэтому ток потребления такой схемы защиты добавляется к минимальному потребляемому системой току.

Схема защиты от кратковременного пропадания напряжения питания в микроконтроллерах семейства MSP430 не может быть отключена, при этом она не потребляет дополнительного тока. Эта информация содержится во всех технических описаниях этих микроконтроллеров.

Ток утечки выводов

Иногда при выборе низкопотребляющего микроконтроллера ток утечки выводов не учитывается, но для большинства приложений он очень важен. Большинство низкопотребляющих микроконтроллеров имеет ток утечки выводов до 1 мкА. Таким образом, суммарный ток утечки микроконтроллера, имеющего 20 выводов, может достигнуть 20 мкА! Микроконтроллеры семейства MSP430 имеют ток утечки выводов не более 50 нА, поэтому ток утечки 20 выводного микроконтроллера этого семейства не превысит 1 мкА.

Эффективность обработки

Наконец, часто неправильно истолковывается эффективность обработки микроконтроллеров. Часто думают, что 16-разрядные микроконтроллеры требуют в два раза больше памяти, чем 8-разрядные микроконтроллеры, но 16-

разрядным микроконтроллерам может потребоваться намного меньше памяти программы, а, кроме того, они обычно выполняют задачи быстрее. Например, в системах, где имеется 10- битный АЦП или требуется выполнение 16-разрядной математики, вычислительное ядро 8-разрядных микроконтроллеров сильно загружено. Далее, многие 8-разрядные микроконтроллеры имеют только один рабочий регистр (аккумулятор), при помощи которого и производятся все вычисления. В таблице 1 показаны команды, которые необходимо выполнить микроконтроллерам семейства MSP430 и обычному 8-разрядному микроконтроллеру для перемещения результата преобразования 10- битного АЦП. Работая на частоте 1 МГц, микроконтроллеры семейства MSP430 выполняют это перемещение за 6 мкс, а 8- разрядный микроконтроллер – за 24 мкс.

Таблица 1. Сравнение эффективности обработки 16- и 8-разрядных микроконтроллеров

16-разрядный микроконтроллер	8-разрядный микроконтроллер
<code>mov.w &ADC10MEM, &RAM</code>	<code>movf ADRESH, W</code> <code>movwf RAML</code> <code>bsf 0x20</code> <code>movlf ADCHRESL, W</code> <code>bcf 0x20</code> <code>movwf RAMH</code>

Заключение

При выборе низкопотребляющего микроконтроллера надо изучить все пункты его технического описания. Но усилия, затраченные на это, позволят продлить срок службы аккумулятора и удовлетворить технические требования к току потребления системы.

I. Общие вопросы

Глава 2.

Сравнение микроконтроллеров семейства MSP430 с другими микроконтроллерами

Сравнение микроконтроллеров семейства MSP430 с другими микроконтроллерами

Greg Morton [slaa205.pdf (Англ.)]

В данном примере применения приведены результаты сравнения микроконтроллеров семейства MSP430 с микроконтроллерами других производителей. Для разработки кода различных простейших операций и выполнения его в режиме симулятора была использована платформа разработчика Embedded Workbench™ компании IAR Systems. Для выяснения различных параметров микроконтроллеров эти различные часто используемые приложения были выполнены на каждом микроконтроллере. Для всех рассмотренных применений были посчитаны суммарные размер кода программы и количество командных циклов, требуемых для выполнения. На рисунке 1 и рисунке 2 приведены диаграммы сравнения размеров кодов программ и количество командных циклов для каждого исследованного микроконтроллера.

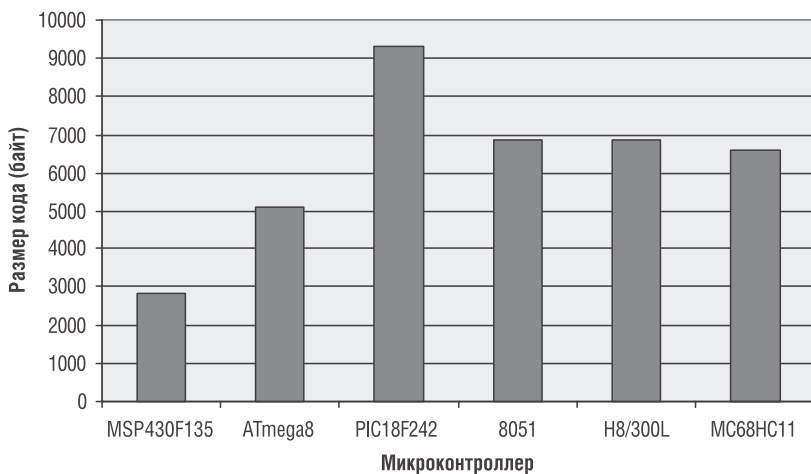


Рис. 1. Диаграмма сравнения суммарных размера кодов программ

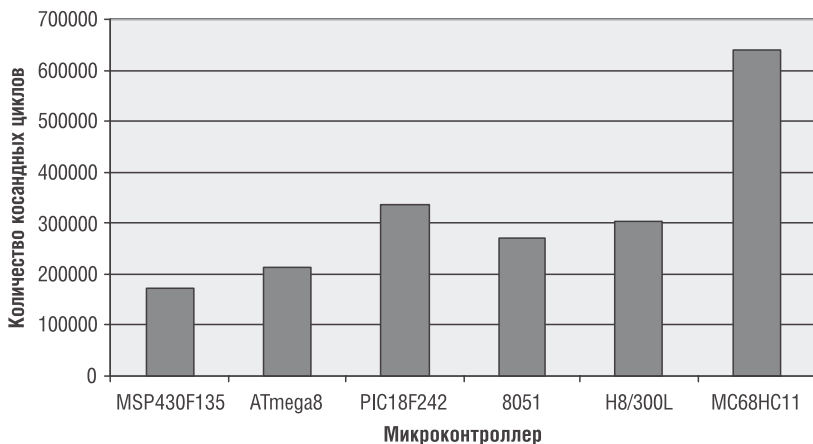


Рис. 2. Диаграмма сравнения суммарного количества командных циклов

В таблице 1 приведены коэффициенты деления частоты задающего генератора для формирования сигналов синхронизации вычислительного ядра исследованных микроконтроллеров. Полное время выполнения любого кода можно рассчитать как произведение количества командных циклов и коэффициента деления.

Таблица 1. Коэффициенты деления частоты задающего генератора для формирования сигналов синхронизации вычислительного ядра

Микроконтроллер	Коэффициент деления
MSP430F135	1
ATmega8	1
PIC18F242	4
8051	12
H8/300L	2
MC68HC11	4

В таблице 2 показаны суммарные размер кода и количество командных циклов, приведенные к соответствующим суммарным размеру кода и количеству циклов микроконтроллера MSP430.

Таблица 2. Приведенные результаты

Микроконтроллер	Приведенный суммарный размер кода	Приведенное суммарное количество командных циклов
MSP430F135	1.00	1.00
ATmega8	1.82	1.26
PIC18F242	3.31	1.98
8051	2.45	1.58
H8/300L	2.45	1.78
MC68HC11	2.36	3.76

Таблица 3 содержит размеры кодов программ (в байтах) и количества командных циклов исследованных прикладных программ для каждого микроконтроллера.

Таблица 3. Размер кодов и количество командных циклов

Применение	MSP430F135	ATmega8	PIC18F242	8051	H8/300L	MC68HC11
	байт циклов					
8- битная математика	172	116	386	141	354	285
	299	157	318	112	680	387
8- битный коммутатор	180	342	404	209	362	387
	50	131	109	84	388	214
16- битная математика	172	174	598	361	564	315
	343	319	625	426	802	508
16- битная матрица	156	570	846	825	450	490
	5784	24426	27021	29468	15280	23164
16- битный коммутатор	178	388	572	326	404	405
	49	144	163	120	398	230
32- битная математика	250	316	960	723	876	962
	792	782	1818	2937	1756	1446
Операции с плавающей точкой	662	1042	1778	1420	1450	1429
	1207	1601	1599	2487	2458	4664
Фильтр с КИХ	668	1292	2146	1915	1588	1470
	152193	164793	248655	206806	245588	567139
Перемножение матриц	252	510	936	345	462	499
	6633	16027	36190	9454	26750	26874
Итого:	2808	5114	9302	6880	6866	6622
	170249	213680	336543	269638	303198	640038

Влияние аппаратного умножителя и полной оптимизации компилятора

Для того, чтобы показать влияние аппаратного умножителя и полной оптимизации компилятора MSP430, было произведено сравнение кодов применений для MSP430F135 и MSP430F149. MSP430F149 содержит аппаратный умножитель, а MSP430F135 – нет. Для создания и выполнения исследуемых приложений был использован последний C компилятор версии 3.10A компании IAR для микроконтроллеров семейства MSP430. В таблице 4 приведены результаты тестов.

Таблица 4. Влияние аппаратного умножителя и полной оптимизации компилятора

Применение	MSP430F135 Оптимизация: нет Аппаратный умножитель: нет	MSP430F149 Оптимизация: нет Аппаратный умножитель: да	MSP430F149 Оптимизация: полная Аппаратный умножитель: да
	байт циклов		
8- битная математика	172 299	156 268	136 249
8- битная 2 Dim матрица	118 2899	118 2899	124 1110
8- битный выборочный коммутатор	180 50	180 50	178 49
16- битная математика	172 343	156 268	134 248
16- битная 2 Dim матрица	156 5784	156 5784	112 1174
16- битный выборочный коммутатор	178 49	178 49	176 48
32- битная математика	250 792	248 607	222 583
Операции с плавающей точкой	662 1207	702 1012	678 993
Фильтр с КИХ	688 152193	728 143694	732 137161
Перемножение матриц	252 6633	236 5399	166 2847
Итого:	2828 170249	2858 160030	2658 144462

Приложение

Тестовые прикладные задачи

Для сравнения параметров различных микроконтроллеров для каждого из них в режиме симуляции были выполнены приведенные ниже тестовые прикладные задачи. Исходные тексты программ этих прикладных задач приведен ниже.

8-bit_math.c – исходный файл, содержащий три математических функции. Одна функция выполняет сложение двух 8- битных чисел, одна выполняет умножение и одна – деление. Вызов этих функций осуществляется из функции «main()».

16-bit_math.c – исходный файл, содержащий три математических функции. Одна функция выполняет сложение двух 16- битных чисел, одна выполняет умножение и одна – деление. Вызов этих функций осуществляется из функции «main()».

32-bit_math.c – исходный файл, содержащий три математических функции. Одна функция выполняет сложение двух 32- битных чисел, одна выполняет умножение и одна – деление. Вызов этих функций осуществляется из функции «main()».

floating_point_math.c – исходный файл, содержащий три математических функции. Одна функция выполняет сложение двух чисел с плавающей запятой, одна выполняет умножение и одна – деление. Вызов этих функций осуществляется из функции «main()».

8-bit_switch_case.c – исходный файл с одной функцией, содержащей инструкцию коммутатора, имеющего 16 вариантов выбора. В зависимости от 8- битного значения выбирается один из вариантов выбора. Функция «main()» вызывает функцию «switch» с выбранным входным параметром.

16-bit_switch_case.c – исходный файл с одной функцией, содержащей инструкцию коммутатора, имеющего 16 вариантов выбора. В зависимости от 16- битного значения выбирается один из вариантов выбора. Функция «main()» вызывает функцию «switch» с выбранным входным параметром.

8-bit_2-dim_matrix.c – исходный файл, содержащий 3 двумерных массива 8- битных значений, один из которых инициализирован. Функция «main()» копирует первый массив во второй, а потом второй – в третий.

16-bit_2-dim_matrix.c – исходный файл, содержащий 3 двумерных массива 16- битных значений, один из которых инициализирован. Функция «main()» копирует первый массив во второй, а потом второй – в третий.

fir_filter.c – исходный файл, содержащий код, который вычисляет ответ на выходе фильтра 17-ого от имитированного входного воздействия АЦП.

matrix_multiplication.c – исходный файл, содержащий код, который перемножает матрицу 3x4 и матрицу 4x5.

Компилятор

Для формирования тестовых прикладных задач использовался «С» компилятор, встроенный в интегрированную среду разработки (IDE) Embedded Workbench компании IAR Systems. Демонстрационные версии IDE для всех микроконтроллеров были получены на сайте компании IAR Systems по адресу <http://www.iar.com>. В таблице 5 приведены версии программы, использованные для разработки прикладных задач для каждого конкретного микроконтроллера. Для каждого прикладного решения был посчитан размер программы. Все прикладные задачи были сформированы при установленном значении оптимизатора компилятора «none». Это было сделано для того, чтобы исключить влияние оптимизатора компилятора на результаты тестов. Например, при включенном оптимизаторе компилятор для микроконтроллеров семейства 8051 был способен вычислить результаты для некоторых математических функций во время компиляции. Алгоритм оптимизации был настолько сложен, что способен был определить, что входные значения математических не изменяются. Это позволило компилятору вычислить математическую функцию и заменить ее результатом вычисления.

Все прикладные задачи были выполнены в режиме симуляции. При этом регистрировалось количество команд, требуемых для выполнения каждой прикладной задачи.

Таблица 5. Используемые версии С-компилятора

Микроконтроллер	Версия «С» компилятора компании IAR
MSP430F135	2.21B
Atmel ATmega8	3.10C
Microchip PIC18F242	6.10A
Generic 8051	2.12A
Renesas H8/300L	4.20A
Motorola MC68HC11	4.45A

Исходные тексты тестовых прикладных задач

Ниже приведены исходные тексты на «С» тестовых прикладных задач, использованных для тестирования, описанного в данном документе.

8-bit Math.c

```

/*****
* Имя: 8-bit Math
* Назначение: тестовая прикладная задача 8-битных
* математических функций.
*****/
typedef unsigned char UInt8;
UInt8 add(UInt8 a, UInt8 b)
{
    return (a+b);
}
UInt8 mul(UInt8 a, UInt8 b)
{
    return (a*b);
}
UInt8 div(UInt8 a, UInt8 b)
{
    return (a/b);
}
void main(void)
{
    volatile UInt8 result[4];
    result[0]=12;
    result[1]=3;
    result[2]=add(result[0], result[1]);
    result[1]=mul(result[0], result[2]);
    result[3]=div(result[1], result[2]);
    return;
}

```

8-bit 2-dim Matrix.c

```

/*****
* Имя: 8-bit 2-dim Matrix
* Назначение: тестовая прикладная программа
* копирования 8-битных значений.
*****/
typedef unsigned char UInt8;
const UInt8 m1[16][4]={

```

```

        {0x12, 0x56, 0x90, 0x34},
        {0x78, 0x12, 0x56, 0x90},
        {0x34, 0x78, 0x12, 0x56},
        {0x90, 0x34, 0x78, 0x12},
        {0x12, 0x56, 0x90, 0x34},
        {0x78, 0x12, 0x56, 0x90},
        {0x34, 0x78, 0x12, 0x56},
        {0x90, 0x34, 0x78, 0x12},
        {0x12, 0x56, 0x90, 0x34},
        {0x78, 0x12, 0x56, 0x90},
        {0x34, 0x78, 0x12, 0x56},
        {0x90, 0x34, 0x78, 0x12},
        {0x12, 0x56, 0x90, 0x34},
        {0x78, 0x12, 0x56, 0x90},
        {0x34, 0x78, 0x12, 0x56},
        {0x90, 0x34, 0x78, 0x12}
};
void main (void)
{
    int i, j;
    volatile UInt8 m2[16][4], m3[16][4];
    for(i=0; i<16; i++)
    {
        for(j=0; j<4; j++)
        {
            m2[i][j]=m1[i][j];
            m3[i][j]=m2[i][j];
        }
    }
    return;
}

```

8-bit Switch Case.c

```

/*****
* Имя: 8-bit Switch Case
* Назначение: Тестовая прикладная задача,
* осуществляющая коммутацию по полученному 8-битному
* значению.
*****/
typedef unsigned char UInt8;
UInt8 switch_case(UInt8 a)
{

```

```
UInt8 output;
switch (a)
{
    case 0x01:
        output=0x01;
        break;
    case 0x02:
        output=0x02;
        break;
    case 0x03:
        output=0x03;
        break;
    case 0x04:
        output=0x04;
        break;
    case 0x05:
        output=0x05;
        break;
    case 0x06:
        output=0x06;
        break;
    case 0x07:
        output=0x07;
        break;
    case 0x08:
        output=0x08;
        break;
    case 0x09:
        output=0x09;
        break;
    case 0x0a:
        output=0x0a;
        break;
    case 0x0b:
        output=0x0b;
        break;
    case 0x0c:
        output=0x0c;
        break;
    case 0x0d:
        output=0x0d;
        break;
```



```

        case 0x0e:
            output=0x0e;
            break;
        case 0x0f:
            output=0x0f;
            break;
        case 0x10:
            output=0x10;
            break;
    } /* конец коммутации*/
    return (output);
}
void main(void)
{
    volatile UInt8 result;
    result=switch_case(0x10);
    return;
}

```

16-bit Math.c

```

/*****
* Имя: 16-bit Math
* Назначение: Тестовая прикладная задача,
* осуществляющая вычисление 16-битных математических
* функций.
*****/
typedef unsigned short UInt16;
UInt16 add(UInt16 a, UInt16 b)
{
    return (a+b);
}
UInt16 mul(UInt16 a, UInt16 b)
{
    return (a*b);
}
UInt16 div(UInt16 a, UInt16 b)
{
    return (a/b);
}
void main(void)
{
    volatile UInt16 result[4];
    result[0]=231;
}

```

```

        result[1]=12;
        result[2]=add(result[0], result[1]);
        result[1]=mul(result[0], result[2]);
        result[3]=div(result[1], result[2]);
        return;
}

```

16-bit 2-dim Matrix.c

```

/*****
* Имя: 16-bit 2-dim Matrix
* Назначение: Тестовая прикладная задача копирования
* 16-битных значений.
*****/
typedef unsigned short UInt16;
const UInt16 m1[16][4]={
    {0x1234, 0x5678, 0x9012, 0x3456},
    {0x7890, 0x1234, 0x5678, 0x9012},
    {0x3456, 0x7890, 0x1234, 0x5678},
    {0x9012, 0x3456, 0x7890, 0x1234},
    {0x1234, 0x5678, 0x9012, 0x3456},
    {0x7890, 0x1234, 0x5678, 0x9012},
    {0x3456, 0x7890, 0x1234, 0x5678},
    {0x9012, 0x3456, 0x7890, 0x1234},
    {0x1234, 0x5678, 0x9012, 0x3456},
    {0x7890, 0x1234, 0x5678, 0x9012},
    {0x3456, 0x7890, 0x1234, 0x5678},
    {0x9012, 0x3456, 0x7890, 0x1234},
    {0x1234, 0x5678, 0x9012, 0x3456},
    {0x7890, 0x1234, 0x5678, 0x9012},
    {0x3456, 0x7890, 0x1234, 0x5678},
    {0x9012, 0x3456, 0x7890, 0x1234}
};
void main(void)
{
    int i, j;
    volatile UInt16 m2[16][4], m3[16][4];
    for(i=0; i<16; i++)
    {
        for(j=0; j<4; j++)
        {
            m2[i][j] = m1[i][j];
            m3[i][j] = m2[i][j];
        }
    }
}

```

```
    }  
    return;  
}
```

16-bit Switch Case.c

```
/*  
* Имя: 16-bit Switch Case  
* Назначение: Тестовая прикладная задача,  
* осуществляющая коммутацию по полученному 16-битному  
* значению.  
*/  
typedef unsigned short UInt16;  
UInt16 switch_case(UInt16 a)  
{  
    UInt16 output;  
    switch (a)  
    {  
        case 0x0001:  
            output=0x0001;  
            break;  
        case 0x0002:  
            output=0x0002;  
            break;  
        case 0x0003:  
            output=0x0003;  
            break;  
        case 0x0004:  
            output=0x0004;  
            break;  
        case 0x0005:  
            output=0x0005;  
            break;  
        case 0x0006:  
            output=0x0006;  
            break;  
        case 0x0007:  
            output=0x0007;  
            break;  
        case 0x0008:  
            output=0x0008;  
            break;  
        case 0x0009:  
            output=0x0009;  
    }  
}
```

```

        break;
        case 0x000a:
            output=0x000a;
            break;
        case 0x000b:
            output=0x000b;
            break;
        case 0x000c:
            output=0x000c;
            break;
        case 0x000d:
            output=0x000d;
            break;
        case 0x000e:
            output=0x000e;
            break;
        case 0x000f:
            output=0x000f;
            break;
        case 0x0010:
            output=0x0010;
            break;
        }/* конец коммутации*/
    return (output);
}
void main(void)
{
    volatile UInt16 result;
    result=switch_case(0x0010);
    return;
}

```

32-bit Math.c

```

/*****
* Имя: 32-bit Math
* Назначение: Тестовая прикладная задача вычисления
* 32-битных математических функций.
*****/
#include <math.h>
typedef unsigned long UInt32;
UInt32 add(UInt32 a, UInt32 b)
{

```

```

        return (a+b);
    }
    UInt32 mul(UInt32 a, UInt32 b)
    {
        return (a*b);
    }
    UInt32 div(UInt32 a, UInt32 b)
    {
        return (a/b);
    }
    void main(void)
    {
        volatile UInt32 result[4];
        result[0]=43125;
        result[1]=14567;
        result[2]=add(result[0], result[1]);
        result[1]=mul(result[0], result[2]);
        result[3]=div(result[1], result[2]);
        return;
    }

```

Floating-point Math.c

```

/*****
* Имя : Floating-point Math
* Назначение: Тестовая прикладная задача выполнения
* математических функций с плавающей запятой.
*****/
float add(float a, float b)
{
    return (a+b);
}
float mul(float a, float b)
{
    return (a*b);
}
float div(float a, float b)
{
    return (a/b);
}
void main(void)
{
    volatile float result[4];

```

```

        result[0]=54.567;
        result[1]=14346.67;
        result[2]=add(result[0], result[1]);
        result[1]=mul(result[0], result[2]);
        result[3]=div(result[1], result[2]);
        return;
}

```

FIR Filter.c

```

/*****
* Имя : FIR Filter
* Назначение: Тестовая прикладная задача фильтра
* с КИХ. Входным значением для фильтра является массив
* из 51 16-битного значения.
* Фильтр имеет 17-ый порядок
*****/
#ifdef MSP430
#include <msp430x13x.h>
#endif
#include <math.h>
#define FIR_LENGTH 17
const float COEFF[FIR_LENGTH]=
{
    -0.000091552734,
    0.000305175781,
    0.004608154297,
    0.003356933594,
    -0.025939941406,
    -0.044006347656,
    0.063079833984,
    0.290313720703,
    0.416748046875,
    0.290313720703,
    0.063079833984,
    -0.044006347656,
    -0.025939941406,
    0.003356933594,
    0.004608154297,
    0.000305175781,
    -0.000091552734
};
/* Текущий массив симулирует входные значения от АЦП*/

```

```

const unsigned int INPUT[]=
{
    0x0400, 0x0800, 0x0C00, 0x1000, 0x1400, 0x1800,
    0x1C00, 0x2000,
    0x2400, 0x2000, 0x1C00, 0x1800, 0x1400, 0x1000,
    0x0C00, 0x0800,
    0x0400, 0x0400, 0x0800, 0x0C00, 0x1000, 0x1400,
    0x1800, 0x1C00,
    0x2000, 0x2400, 0x2000, 0x1C00, 0x1800, 0x1400,
    0x1000, 0x0C00,
    0x0800, 0x0400, 0x0400, 0x0800, 0x0C00, 0x1000,
    0x1400, 0x1800,
    0x1C00, 0x2000, 0x2400, 0x2000, 0x1C00, 0x1800,
    0x1400, 0x1000, 0x0C00, 0x0800, 0x0400
};

void main(void)
{
    int i, y; /* Счетчик циклов*/
    volatile float OUTPUT[36];
#ifdef MSP430
    WDTCTL=WDTPW+WDTHOLD; /*Остановка
    сторожевого таймера*/
#endif
    for(y=0; y<36; y++)
    {
        for(i=0; i<FIR_LENGTH/2; i++)
        {
            OUTPUT[y]=COEFF[i]*(INPUT[y+16-i]+
            INPUT[y+i]);
        };
        OUTPUT[y]=OUTPUT[y]+(INPUT[y+16-i]*
        COEFF[i]);
    }
    return;
}

```

Matrix Multiplication.c

```

/*****
* Имя: Matrix Multiplication
* Назначение: Тестовая прикладная программа
* перемножения матриц 3x4 и 4x5.
* матрицы содержат 16-битные значения.
*****/

```

```
typedef unsigned short UInt16;
const UInt16 m1[3][4]={
    {0x01, 0x02, 0x03, 0x04},
    {0x05, 0x06, 0x07, 0x08},
    {0x09, 0x0A, 0x0B, 0x0C}
};
const UInt16 m2[4][5]={
    {0x01, 0x02, 0x03, 0x04, 0x05},
    {0x06, 0x07, 0x08, 0x09, 0x0A},
    {0x0B, 0x0C, 0x0D, 0x0E, 0x0F},
    {0x10, 0x11, 0x12, 0x13, 0x14}
};
void main(void)
{
    int m, n, p;
    volatile UInt16 m3[3][5];
    for(m=0; m<3; m++)
    {
        for(p=0; p<5; p++)
        {
            m3[m][p]=0;
            for(n=0; n<4; n++)
            {
                m3[m][p]+=m1[m][n]*m2[n][p];
            }
        }
    }
    return;
}
```


I. Общие вопросы

Глава 3. *Подключение EEPROM к MSP430 с помощью интерфейса I²C*

Подключение EEPROM к модулю I²C микроконтроллеров MSP430

Christian Hernitscheck

В этом примере применения описана организация обмена по протоколу I²C между аппаратным модулем I²C микроконтроллера MSP430F169 и внешней EEPROM-памяти (24xx65). Приведены примеры реализации процедур Запись Байта (Byte Write), Чтение Текущего Адреса (Current Address Read), Чтение Произвольного Адреса (Random Address Read), и Опрос Подтверждения (Acknowledge Polling).

1. Пример подключения

Схема, изображённая на рис. 1, показывает пример подключения памяти EEPROM к аппаратному модулю I²C микроконтроллера MSP430F169.

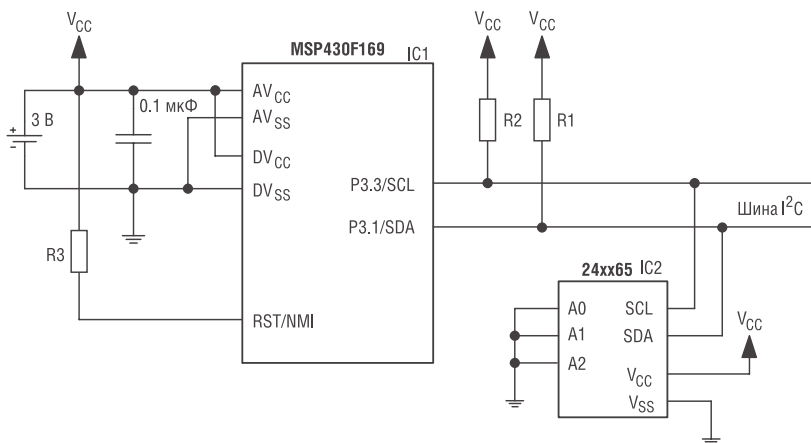


Рис. 1. Подключение памяти EEPROM типа 24xx65 к MSP430F169 по шине I²C

Пользователь выбирает I²C-адрес подключенных EEPROM на шине при помощи адресных выводов (A0, A1, and A2). Наличие таких выводов позволяет подключить на шину несколько устройств одновременно. Логические уровни на адресных выводах определяют блок адресов, занимаемых устройством в общем адресном пространстве. Конкретное устройство выбирается программно, передачей соответствующих бит (A0, A1, и A2) в контрольном байте.

2. Исходный код для MSP430

Этот пример программы показывает использование аппаратного модуля I²C микроконтроллера MSP430F169 для работы с EEPROM памятью по шине I²C. В зависимости от объема микросхемы памяти EEPROM адресация может видоизменяться. Существуют разновидности микросхем памяти EEPROM, которые используют только один байт для адресации (объем памяти 256 Байт или менее). В приведенном ниже примере для адресации используется два байта. Показаны примеры на языке C и на ассемблере, функции их идентичны. Подробное описание модуля I²C микроконтроллеров MSP430 приведено в [2].

Файлы "I2Croutines.c" и "I2Croutines.s43" могут быть использованы в качестве библиотек. Они содержат следующие функции:

- InitI2C()
Инициализация аппаратного модуля I2C микроконтроллера MSP430.
- EEPROM_ByteWrite(<address>, <data>)
Выполнение команды Запись Байта (Byte Write). Перед запуском указывается адрес и данные, которые следует записать.
- EEPROM_AckPolling();
Опрос Подтверждения (Acknowledge Polling) используется для определения окончания процесса записи в EEPROM. Функцию следует вызывать после записи данных в EEPROM.
- Result = EEPROM_RandomRead(<address>)
Команда Чтение по Произвольному Адресу (Random Read) позволяет читать содержимое ячейки памяти EEPROM с указанным адресом.
- Result = EEPROM_CurrentAddressRead()
Чтение по Текущему Адресу (Current Address Read), используется внутренних указатель адреса EEPROM. После выполнения операций чтения и записи внутренний указатель автоматически инкрементируется.

В данном примере показано использование функций из файлов "I2Croutines.c" и "I2Croutines.s43":

```
#include <msp430x16x.h>
...
/*---- внешние функции файла «I2Croutines.c» -----*/
extern void InitI2C(void);
extern void EEPROM_ByteWrite(unsigned int Address,
unsigned char Data);
extern unsigned char EEPROM_RandomRead(unsigned int
Address);
extern unsigned char EEPROM_CurrentAddressRead(void);
```

```
extern void EEPROM_AckPolling(void);
...
/*----- тело программы-----*/
void main(void)
{ ... // Инициализация
InitI2C(); // Инициализация модуля I2C
_EINT(); // Разрешение прерываний
...
EEPROM_ByteWrite(0x0000, 0x12);
EEPROM_AckPolling(); // Ожидание завершения
// цикла записи EEPROM

EEPROM_ByteWrite(0x0001, 0x34);
EEPROM_AckPolling(); // Ожидание завершения
// цикла записи EEPROM

EEPROM_ByteWrite(0x0002, 0x56);
EEPROM_AckPolling(); // Ожидание завершения
// цикла записи EEPROM

...
Data[0] = EEPROM_RandomRead(0x0000); // Чтение
// по адресу 0x0000
Data[1] = EEPROM_CurrentAddressRead(); // Чтение
// по адресу 0x0001
Data[2] = EEPROM_CurrentAddressRead(); // Чтение
// по адресу 0x0002
```

2.1. Запись Байта (Byte Write)

На рис. 2 показан протокол Записи Байта (Byte Write). Обратите внимание, что в состоянии STOP обнуляется бит MST в управляющем регистре UOCTL микроконтроллера MSP430. Перед началом следующей процедуры (например, Записи Байта), модуль I²C требует реконфигурации в ведущий режим (master mode) установкой бита MST.

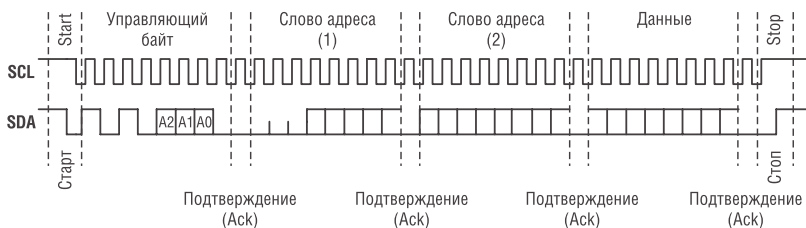


Рис. 2. Запись Байта (Byte Write)

Также следует обратить внимание на особенность формирования управляющего байта I²C. Модуль I²C микроконтроллера MSP430 отправляет адрес ведомого устройства (slave), который определён в управляющем регистре I2CSA. Обратите внимание, что адрес ведомого устройства расположен в битах 1...7 управляющего байта. Для формирования управляющего байта EEPROM адрес, находящийся в регистре I2CSA должен быть сдвинут на один бит влево (рис. 3). Например, чтобы получить управляющий байт 0xA0 следует определить адрес ведомого устройства (в регистре I2CSA) 0x50. Бит R/W, также расположенный в управляющем байте (бит 0), управляется автоматически при выборе режима передачи либо приёма. Режим выбирается с помощью бита I2CTRX в управляющем регистре I2CTCTL.



Рис. 3. Преобразование содержимого регистра I2CSA в управляющий байт I²C

2.2. Чтение Текущего Адреса (Current Address Read)

На рис. 3 показано формирование адреса ведомого устройства в управляющем байте. Старшие четыре бита представляют собой фиксированный адрес (1010). Младшими битами адреса выбирается ведомое устройство. Ведущее устройство (master) может адресовать различные микросхемы EEPROM на одной шине I²C путём выбора бит A0, A1, и A2 (адрес устройства EEPROM). На рис. 4 показана операция чтения по текущему адресу (Current Address Read). Перед началом обмена модуль I²C микроконтроллера MSP430 должен быть переключен. Следует выбрать ведущий режим (master) и разрешить приём.

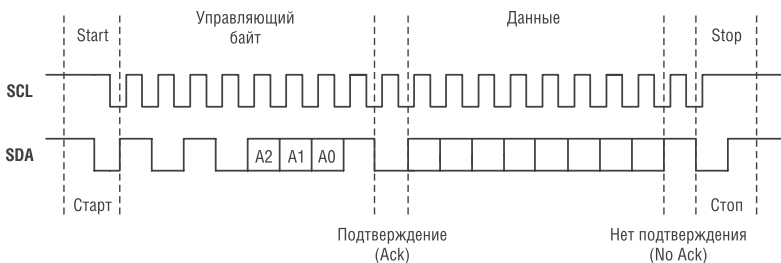


Рис. 4. Чтение Текущего Адреса (Current Address Read)

Формирование управляющего байта осуществляется так же, как описано в параграфе Запись Байта (Byte Write). В примере принимается только один байт. Для определения момента завершения обмена по I²C и приёма байта используется поллинг.

2.3. Чтение Произвольного Адреса (Random Address Read)

На рис. 5 показана реализация протокола Чтение Произвольного Адреса (Random Address Read). Протокол представляет своего рода смесь протоколов Запись Байта (Byte Write) и Чтение Текущего Адреса (Current Address Read).

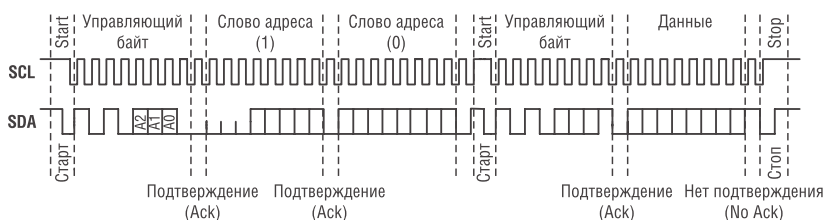


Рис. 5. Чтение Произвольного Адреса (Random Address Read)

Прежде всего, должен быть выставлен адресный счётчик памяти EEPROM. Это осуществляется при помощи операции Запись Байта (Byte Write). При этом в EEPROM предаётся новое значение адреса. После этого выполняется перезапуск и осуществляется операция Чтение Текущего Адреса (Current Address Read).

2.4. Опрос Подтверждения (Acknowledge Polling)

После того, как была принята команда записи и MSP430 сгенерировал условие останова (stop), в EEPROM запускается процедура записи с собственной системой тактирования. Время, необходимое для завершения этой процедуры, зависит от типа используемой EEPROM. В то время, как EEPROM находится в состоянии записи, она не будет выдавать Подтверждение (Acknowledge). Эта особенность может быть использована для определения момента завершения процесса записи.

3. Ссылки:

1. MSP430F169 Data Sheet (SLAS368)
2. Руководство пользователя MSP430x1xx (MSP430x1xx Family User's Guide) (SLAU049)

I. Общие вопросы

Глава 4.
MSP430F21x1
Обзор архитектуры

MSP430F21x1 Обзор архитектуры

Mark Buccini [slaa217.pdf (Англ.)]

1. Введение

Семейство микроконтроллеров (МК) MSP430 сочетает в себе низкое энергопотребление и весьма гибкую архитектуру, что делает его незаменимым для широкого круга применений, где требуется особо низкий потребляемый ток. Такими применениями являются счётчики электроэнергии, переносные измерительные устройства и бытовые приборы, другие устройства, работающие длительное время без смены батарей. Главные преимущества семейства MSP430:

- Особо низкое энергопотребление
- Комплексная система обработки смешанных сигналов (SoC) на одном кристалле
- Использование современных технологий программирования

Со времени появления первых устройств серии MSP430, более 30,000 разработчиков по всему миру выбрали данную архитектуру, создав за это время огромное количество разнообразных устройств, от простых, таких, как термостаты и счётчики электроэнергии, до изменивших жизнь миллионов диабетиков во всём мире переносных измерителей уровня глюкозы в крови. Высокая популярность семейства обусловила необходимость дальнейшего развития серии. Требования к разработкам постоянно растут и зачастую являются противоречивыми, например повышение производительности сочетается с понижением энергопотребления, снижением цены и работой в жёстких климатических условиях.

Для обеспечения таких требований к разработкам была разработана новая серия MSP430F2xx, в которой были учтены требования снижения цены и повышения производительности для работы как в устройствах с пониженным потреблением, так и в приборах общего применения. Потребляя менее 1 микроампера в дежурном режиме и обладая при этом способностью переключиться в режим синхронной работы со скоростью 16 миллионов инструкций в секунду (MIPS) за время менее одной микросекунды, новое семейство MSP430F2xx обеспечивает удвоенную производительность при половинном энергопотреблении по сравнению с более ранней серией MSP430F1xx.

2. Обновления

2.1. Микропотребляющий модуль сброса *Brownout Reset*

Все устройства серии MSP430F2xx содержат микропотребляющий модуль сброса по понижению питания (BOR). Модуль BOR определяет понижение питающего напряжения ниже уровня, безопасного для выполнения программы и переключает при этом условия микроконтроллер в состояние сброса (reset). Эта функция предотвращает непредсказуемое поведение устройства. Кратковременное понижение питающего напряжения – весьма сложная диагностируемая проблема. Например, она может возникать при простой операции замены батарей. При этом могут появляться случайные кратковременные броски питающего напряжения. Микропотребляющий модуль сброса по понижению питания BOR, присутствующий в серии MSP430F2xx, впервые появился в семействе MSP430F4xx и также присутствует в новых устройствах серии MSP430F1xx.

- Наличие модуля BOR повышает устойчивость работы устройства в тяжёлых эксплуатационных условиях, где может присутствовать нестабильность питающего напряжения.
- Функция BOR в MSP430 постоянно включена. Аналогичные решения других фирм потребляют ток величиной более 10 мкА, что ограничивает их применение в микропотребляющих устройствах.

2.2. Повышение тактовой частоты

Для обеспечения требований повышенной производительности наряду с низким энергопотреблением, новая улучшенная система тактирования (BCS+) семейства MSP430F2xx (так же, как и в других устройствах серии MSP430), как правило, использует два тактовых сигнала. Низкочастотный вспомогательный источник тактирования (ACLK) работает непосредственно от обычного часового кварца частотой 32768 Гц без дополнительных внешних элементов и используется для постоянно включенных низкочастотных периферийных модулей. Высокочастотный сигнал тактирования генерируется непосредственно внутри МК, включен постоянно и имеет цифровой контроль (DCO). Генератор DCO используется процессором (CPU) и остальными периферийными модулями только по необходимости. Для обеспечения низкого энергопотребления управление генератором DCO может осуществляться пользователем по прерываниям, основное же время прибор может проводить в режиме standby,

переключаясь в режим максимальной производительности на требуемое время лишь по необходимости. Система тактирования BCS+ совместима с более ранними системами тактирования семейства MSP430F1xx, но при этом включает в себя новые функции, позволяющие ещё более понизить потребление энергии, повысить помехоустойчивость и снизить стоимость прибора.

- Пониженное потребление в режиме standby – в этом режиме (также называемом real-time clock mode) энергопотребление снижено до <1 микроампера при напряжении питания 3В, по сравнению с »1.6 микроамперами в семействе MSP430F1xx. Этот режим имеет название «режим пониженного энергопотребления 3» (LPM3), активными при этом являются низкочастотный генератор ACLK, таймер, ОЗУ и прерывания. Пониженный потребляемый ток в режиме LPM3 является большим преимуществом в измерительных, промышленных и медицинских приборах, в которых основное время приходится на дежурный режим standby в ожидании использования. Например, в некоторых термоста-тах или распределителях достаточно измерять температуру с частотой не выше одного раза в несколько секунд. Таким образом, прибор находится в режиме LPM3 на протяжении более 99% рабочего времени. Уменьшение тока потребления в режиме LPM3 имеет в этом случае определяющее влияние на суммарный ток потребления и время работы от комплекта батарей.
- Встроенные конденсаторы с программируемой ёмкостью – добавление этой функции позволяет использовать более широкий диапазон кварцевых резонаторов без необходимости применения внешних компонентов, стабилизирующих генерацию. В семействе MSP430F1xx возможно только использование встроенных конденсаторов ёмкостью 12 пФ.
- Устойчивый к сбоям кварцевый генератор – сбой в генерации вызывает немаскируемое прерывание и запускает встроенный генератор для бесперебойной работы. Эта функция доступна без дополнительного энергопотребления, как в режиме высокочастотного генератора, так и в низкочастотном режиме. Устойчивый к сбоям кварцевый генератор обеспечивает более надёжное функционирование прибора в тяжёлых условиях эксплуатации. Ранее, в семействе MSP430F1xx эта функция была доступна только в высокочастотном режиме.
- Встроенный фильтр (de-glitch) на входе генератора – эта функция снижает уровень высокочастотных помех, наводимых на вход генератора, повышая устойчивость системы.

- Выбор диапазона кварцевого резонатора в высокочастотном режиме – эта функция обеспечивает повышенную стабильность и позволяет использовать более дешёвые резонаторы с высоким эквивалентным сопротивлением (ESR).
- Улучшенный встроенный генератор с цифровым управлением (DCO) – обновлённый модуль DCO, входящий в состав BCS+ с гарантированным временем запуска менее 1 микросекунды, погрешностью не более $\pm 2.5\%$ и рабочей частотой до 16 МГц во всём диапазоне рабочих температур и напряжений. Повышенная точность генератора DCO позволяет во многих случаях избавиться от внешнего кварцевого резонатора и снизить, таким образом, стоимость системы. Следует отметить, что генератор DCO запускается непосредственно на запрограммированной частоте, не требуя дополнительного времени для стабилизации. Это важно в задачах, где требуется моментальный доступ к генератору с известной и стабильной частотой, таких, как обмен данными по УАПП (UART). Без обеспечения стабильности, быстрый запуск тактового генератора в таких задачах не имеет смысла.

2.3. Улучшенная встроенная Flash-память

Ввиду того, что стоимость Flash-памяти в последнее время вплотную приблизилось к стоимости однократной ПЗУ (ROM) а также благодаря высокой гибкости, присущей системам с внутрисхемным программированием (ISP), Flash-память де-факто стала стандартом в наиболее современных МК. В большинстве устройств, основанных на микроконтроллерах серии MSP430, в настоящее время чипы с flash-памятью используется как для прототипов, так и для массовой продукции.

Изменения в программу прибора могут вноситься и проверяться от дня начала разработки вплоть до дня его непосредственного выпуска. Также имеется возможность для обновления прошивок непосредственно на месте эксплуатации и самим заказчиком. Использование внутрисхемного программирования Flash-памяти наряду со встроенным эмулятором позволяет отлаживать систему на конкретном чипе, запаянном в конкретное устройство, работающее при определённых эксплуатационных условиях.

Вся Flash-память микроконтроллеров MSP430 работает во всём диапазоне рабочих частот, питающих напряжений и токов потребления, присущих данным устройствам. В семействе MSP430F2xx применена улучшенная Flash-технология со следующими новыми особенностями:

- Минимальное напряжение программирования/стирания понижено с 2.7 В до 2.2В – это изменение на 0.5В имеет важное значение в тех случаях, когда прибор питается от двух щелочных (alkaline) батарей. Питающее напряжение в этом случае имеет диапазон от 3.3В до 1.8В. С учётом того, что кривая разряда таких батарей имеет характер, близкий к линейному, понижение напряжения программирования с 2.7В до 2.2В увеличивает почти на 50% время, в течение которого можно использовать ISP.
- Время программирования снижено до 17 мкс на байт, время полного стирания – до 20 мс. Ускоренный процесс записи/стирания вкупе с уменьшенным до 64 байт размером сектора и прерываемым алгоритмом ISP позволяют в большинстве случаев отказаться от использования внешней EEPROM. При этом снижается цена устройства и его размер.

2.4. Улучшенная защита программной памяти

Программатор на основе загрузочного сектора (BSL) даёт возможность доступа к ресурсам MSP430 с использованием стандартного протокола УАПП (UART). В основном он используется для программирования Flash-памяти. Доступ через BSL защищён с помощью 256-битного пароля, значением которого является содержимое шестнадцати 16-битных векторов прерываний. В незапрограммированном чипе это значение известно и равно 0xFFFF, но в этом случае устройство не содержит никакой полезной информации. После программирования кристалла, только программист знает содержание векторов прерываний, а, следовательно, и сам 256-битный пароль к загрузчику BSL. В серии MSP430F2xx защита программной памяти улучшена благодаря повышению противозлоупотребления алгоритма BSL, обладающего следующими новыми особенностями:

- Для уменьшения вероятности взлома, загрузчик BSL в серии MSP430F2xx производит полное стирание чипа после однократного приёма неверного 256-битного пароля. В более ранних версиях BSL в данном случае просто запрещался доступ.
- Для снижения вероятности случайного вызова программатора BSL из-за ошибки в программе, стирание и программирование средствами BSL возможно только при подаче корректной стартовой последовательности, после которой будет обнулён бит LOCK в контрольном регистре. Это делает случайное стирание программы посредством вызова программато-

ра BSL практически невозможным, т.к. flash-память может быть стёрта или запрограммирована только в том случае, когда BSL был запущен в результате подачи корректной последовательности, и после этого была подана правильная команда.

2.5. Улучшения, касающиеся сторожевого (Watchdog) таймера

Основной функцией существующего в предыдущих сериях семейства MSP430 сторожевого таймера (watchdog) является генерация системного сброса в случае превышения определённого временного интервала, в течение которого отсутствует требуемая программная обработка. Улучшенный сторожевой таймер (WDT+) в серии MSP430F2xx был впервые представлен в микросхеме электросчётчика MSP430FE42x. Он позволяет обработать большее количество программных проблем. Ниже приведен список улучшений, добавленных в модуль WDT+ для повышения надёжности системы:

- Добавлена функция устойчивого к сбоям тактирования, которая запрещает остановку источника тактирования модуля WDT+ в режиме сторожевого таймера. Например, если в качестве источника тактирования модуля WDT+ выбран неисправный кварцевый генератор, то функция устойчивого к сбоям тактирования автоматически переключит источник тактирования модуля WDT+ на встроенный генератор, обеспечивая устойчивое к сбоям функционирование системы.
- Невозможно программно отключить активный источник тактирования модуля WDT+. Преимущество такого решения видно на примере обработчика прерываний (ISR), в котором в стек сохраняется содержимое статусного регистра (SR). В том случае, если программа случайно нарушит содержимое сохранённого в стеке SR таким образом, что все источники тактирования будут запрещены, по возврату из прерывания сторожевой таймер будет навсегда остановлен. Модуль WDT+ исключает подобные ситуации.
- Запрещено выполнение программы с адреса периферийного модуля. Являясь представителем Фон-Неймановской архитектуры, MSP430 имеет общее адресное пространство для программной памяти и периферийных модулей. Несмотря на это, переход программы по адресам, зарезервированным за периферийными модулями (от 0x0000 до 0x01FF) не имеет никакого смысла. В случае случайного возникновения такой ситуации, сторожевой таймер WDT+ осуществит программный сброс.

2.6. Дополнительные возможности

Семейство MSP430F2xx также обладает рядом дополнительных возможностей. Выводы портов 1 и 2 теперь имеют возможность включения встроенных подтягивающих резисторов (pull up/down). Использование таких резисторов позволяет обойтись без дополнительных внешних элементов, уменьшая цену и размер прибора, что особенно важно для портативных устройств. Микроконтроллер MSP430F21x1 в 24-выводном корпусе QFN занимает на плате площадь 4×4 мм, т.е. примерно столько же, сколько всего 2 резистора типоминимала 1206. Таким образом, во многих случаях площадь, занимаемая внешними компонентами, значительно превышает площадь, занимаемую самим МК.

В 20/28-выводных устройствах серии MSP430F2xx выводы кварцевого генератора XIN/XOUT теперь совмещены с выводами порта P2.6 и P2.7, которые прежде были недоступны. Следовательно, в устройствах, не требующих внешнего кварцевого резонатора, становятся доступными два дополнительных вывода.

Модуль компаратора в семействе MSP430F2xx теперь снабжён входным мультиплексором, позволяющим расширить число измеряемых аналоговых сигналов.

3. Архитектура ядра MSP430

Обладая рядом новых дополнительных возможностей, семейство MSP430F2xx сохраняет совместимость с предыдущими устройствами серии MSP430 и использует такое же мощное 16-битное RISC-подобное ядро. Ядро MSP430 было спроектировано с учётом самых современных требований программирования. Разработчики стремились сделать его прозрачным – в том смысле, что даже максимально сложный код создаётся без значительных затрат времени на создание программных функций. Эффективность использования языка C на данный момент является ключевым требованием, т.к. всё большее количество разработчиков предпочитают использовать в работе только этот язык для обеспечения переносимости кода.

Почему такое внимание уделяется эффективности программы? Современное ядро, такое, как MSP430 является довольно компактным и занимает около 4 тысяч логических вентилей – это приблизительно эквивалентно 4-м килобайтам встроенной Flash-памяти. Таким образом, если встроенная память

занимает 32КБ и выше, что диктуется требованиями обслуживания возросшей сложности программы, то размер кристалла и его цена будут в основной мере определяться именно памятью.

Учитывая это обстоятельство, архитектура, обеспечивающая максимальную плотность кода (т.е. использующая минимальный объём программной памяти для конкретной функции) даёт в результате максимальное снижение стоимости чипа.

Так как семейство MSP430 в основном предназначено для низкопотребляющих устройств, архитектура, обеспечивающая эффективность программы также позволяет дополнительно снизить потребление энергии благодаря уменьшению числа тактов, затрачиваемых на определённую функцию. Семейство MSP430 обеспечивает наивысшую плотность кода благодаря двустороннему подходу – интеллектуальным периферийным модулям и современному RISC-подобному ядру.

- Интеллектуальные периферийные модули – все модули, входящие в состав MSP430 разработаны с учётом минимального программного обслуживания. Например, АЦП имеют автоматическое сканирование входов, аппаратный запуск преобразования и, зачастую, механизмы прямого доступа к памяти DMA. Такой набор аппаратных возможностей предоставляет ЦПУ возможность перенаправить ресурсы на специфические задачи, не отвлекаясь на перемещение данных. Это даёт возможность разработки более дешёвых систем, требующих меньших программных ресурсов и потребляющих меньше энергии.
- Современное RISC-подобное ядро, обладающее максимальной плотностью кода благодаря своим особенностям (которые будут рассмотрены ниже)

Для рассмотрения задачи получения максимальной эффективности кода на языке C, попробуем взглянуть на проблему со стороны компилятора. Итак, что же «любит» компилятор? Очевидно, это ортогональность команд и большое количество идентичных регистров, не имеющих ограничений на хранимые ресурсы. Также ему требуется стек для передачи параметров и хранения временных переменных. Архитектура MSP430 была разработана с учётом этих пожеланий и выглядит следующим образом:

- Полностью ортогональный набор команд – несмотря на то, что архитектура MSP430 содержит всего 27 инструкций, каждая из них может быть использована с любым типом адресации в пределах всего адресного

пространства. Эта особенность позволяет получить широкий набор функций при малом наборе команд, упростить и удешевить ЦПУ.

- Большое число регистров – кроме программного счётчика и указателя стека ядру доступны 12 идентичных 16-битных регистров общего назначения. Любой регистр может быть использован для хранения, как данных, так и указателя. При этом операции «регистр – регистр» выполняются за один такт. В этом принципиальное отличие семейства MSP430, от многих других МК, использующих единственный регистр или «аккумулятор».
- Отсутствие разбиения на страницы – 16-битная архитектура семейства MSP430 допускает непосредственную адресацию любой ячейки всего 64-килобайтного адресного пространства. В том числе, это касается доступа к встроенным периферийным модулям, таким, например, как 16-битные преобразователи данных. Эта особенность упрощает разработку программного обеспечения.
- Использование стека – указатель стека обладает теми же возможностями адресации, что и регистры общего назначения. При этом нет необходимости перемещения данных в базирующемся в ОЗУ стеке. Компилятор имеет возможность помещать параметры в стек и манипулировать ими в целях расширения возможностей ОЗУ.
- Использование переходов – анализ среднестатистической программы показывает, что около 10% кода составляют условные переходы. Для облегчения их реализации ядро поддерживает восемь однословных условных переходов с максимальной «дальностью» ± 512 .
- Примерно 20% среднестатистической программы обычно составляет простая операция чтение – модернизация – последующая запись операнда. В классической RISC-архитектуре это приводит к низкой плотности кода, т.к. операнд в этом случае обязательно проходит через регистр ЦПУ, используемый же регистр требуется сохранять и восстанавливать. Таким образом, использование трёх способов адресации (непосредственная, относительная и косвенная) обеспечивает минимизацию затрат на операции типа «память $\leftrightarrow$ память» при сохранении полного набора инструкций. Наличие таких методов адресации позволяет существенно сократить размер кода и повысить производительность.
- Работа с байтами и словами – любая инструкция может использовать как 16-битные, так и 8-битные данные. Это особенно удобно для обработки

таблиц, т.к. позволяет использовать наиболее удобный в каждом конкретном случае формат данных.

- Генератор констант – одним из факторов, влияющих на высокую эффективность программ для MSP430, является генератор констант, который аппаратно генерирует 6 наиболее часто используемых констант. Для этой цели автоматически используется один из регистров ЦПУ. Благодаря тому, что константы генерируются непосредственно в ядре, при этом не требуется чтение операнда из памяти, что экономит программную память и повышает скорость выполнения программы. Функция генератора констант является полностью автоматической и «прозрачной» для пользователя.

4. Ресурсы для отладки

Процесс разработки с использованием устройств семейства MSP430F2xx полностью совместим с ныне существующим семейством MSP430F1xx. Отладчик аппаратно включен в устройства, доступ к нему осуществляется при помощи стандартного интерфейса IEEE 1149 JTAG. Возможности встроенного отладчика MSP430 включают в себя полноскоростной и пошаговый режимы, аппаратные точки останова и контроль системы тактирования. При этом все эти функции осуществляет встроенная аппаратная логика, никаких дополнительных ресурсов не требуется. Тот факт, что отладка ведётся непосредственно на самом устройстве, встроенном в реальную систему исключает развитие событий по нежелательному сценарию «чёрт, но ведь оно же всё работало в эмуляторе!» ☺

Встроенный эмулятор является особенно важным в малогабаритных и «аналогово-насыщенных» приборах, где использование внешнего эмулятора, подключенного длинными кабелями, способно полностью исказить картину. Программирование по интерфейсу JTAG используется не только в процессе разработки, но и при программировании в процессе производства, а также может быть использовано для последующих программных апгрейдов.

5. Заключение

Семейство MSP430F2xx обеспечивает удвоенную производительность при половинном потреблении в режиме stand-by по сравнению с более ранними устройствами семейства MSP430F1. Кроме того, серия MSP430F2xx обладает

значительными улучшениями, позволяющими снизить общую стоимость прибора и повысить его функциональность, что делает микроконтроллеры данного семейства идеальным выбором как для уже существующих низкопотребляющих приборов на базе MSP430, так и в качестве отправной точки для множества новых разработок.

II. Использование внутренних функций, периферийных блоков микроконтроллеров

Глава 1.

Отличительные особенности программатора на основе загрузочного сектора MSP430

Отличительные особенности программатора на основе загрузочного сектора MSP430

Franz Graf [slaa089b.pdf (Англ.)]

Программатор на основе загрузочного сектора MSP430 (BSL) позволяет пользователю обращаться к внутренней памяти микроконтроллера MSP430 в процессе макетирования, производства или обслуживания. Для модификации доступны как программная (флеш) память, так и память данных (ОЗУ).

Для связи используется общеизвестный протокол УАПП (UART) с интерфейсом RS232, обеспечивающий гибкое использование как программного, так и аппаратного обеспечения.

Для использования программатора на основе загрузочного сектора следует подать специальную последовательность сигналов BSL на определённые выводы процессора. После этого подаётся определённая последовательность команд в соответствии с требуемым режимом. Режим загрузки может быть прерван переходом по определённому пользователем адресу либо осуществлением аппаратного сброса (reset).

Обращение к данным в памяти MSP430 через программатор на основе загрузочного сектора защищено от несанкционированного доступа пользовательским паролем.

1. Введение

Программатор на основе загрузочного сектора (BSL) представляет возможность программирования флеш-памяти микроконтроллера MSP430 в процессе разработки либо для обновлений. Активизация такого режима может быть осуществлена утилитой, передающей команды по широкоизвестному протоколу УАПП (UART). BSL даёт возможность пользователю контролировать функционирование MSP430 и осуществлять обмен данными с персональным компьютером либо другим устройством, поддерживающим протокол UART.

Для защиты программы BSL от случайного стирания она содержится в специальном, масочно запрограммированном на заводе загрузочном секторе ПЗУ. Программа BSL не может быть стёрта никакими методами. Код загрузчика тщательно оптимизирован под нужды BSL, доступ к нему осуществляется с помощью команд, описанных ниже в данном документе. В целях безопасности было обращено большое внимание на защиту пользовательского кода от несанкционированного считывания. Любая команда BSL, позволяющая непосредственное или опосредованное чтение данных защищена паролем.

Для активизации программатора на основе загрузочного сектора следует подать специальную последовательность сигналов BSL на определённые выводы процессора. После этого, передаётся символ синхронизации, сопровождаемый посылкой данных, определяющих выполнение желаемой операции.

2. Стандартная функция сброса (RESET) и активизация BSL

2.1. Микроконтроллеры с флеш-памятью MSP430 в 20- и 28-выводном корпусе с выводами JTAG, объединёнными с портами общего назначения

Формирование определённой последовательности сигналов на выводах -RST/NMI и TEST переводит MSP430 в режим выполнения программы с адреса сброса BSL, в отличие от обычного вектора сброса, расположенного по адресу FFFFh.

Если в приборе используется связь с ПК через УАПП (UART), эти 2 вывода могут управляться по линиям -DTR и -RTS последовательного порта (RS232) после соответствующего преобразования уровней. Детальное описание аппаратного обеспечения и сопутствующих вопросов дано в отдельном документе (см. Ссылки, раздел 14). Обычный вектор сброса по адресу FFFFh используется в том случае, когда вывод TEST находится в состоянии лог. «0» в то время, когда вывод RST/NMI переводится из лог. «0» в лог. «1» (стандартный метод, см. рис. 1).

Выполнение программы BSL начинается с того момента, как на вход TEST будет подано как минимум 2 положительных импульса и только в том случае, если вход TEST находился в состоянии лог. «1» во время фронта $0 \rightarrow 1$ на входе -RST/NMI (метод активизации BSL, см. рис. 2). Такая схема активизации повышает надёжность запуска BSL.

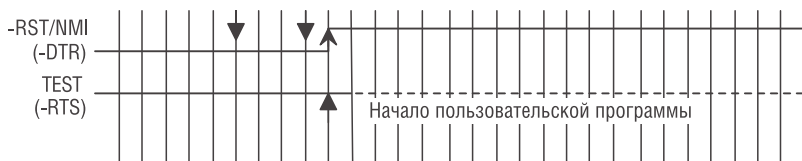


Рис. 1. Стандартная последовательность сигналов сброса (RESET)

Вход TEST обычно используется только для переключения выводов порта P1.7 – P1.4 между их обычными функциями и функциями интерфейса JTAG. Если по входу TEST принят второй положительный импульс во время того, как

вход -RST/NMI находится в состоянии лог. «0», сигнал на входе TEST переводится в лог. «0» внутренней схемой (прикладной режим).

Режим BSL не будет активизирован (с адреса вектора сброса BSL RESET) в том случае, если:

- Присутствует менее 2-х фронтов $0 \rightarrow 1$ на входе TEST во время лог. «0» на входе -RST/NMI
- Вход TEST находится в состоянии лог. «0» во время перехода $0 \rightarrow 1$ на входе -RST/NMI
- Ресурсы MSP430 уже контролируются через интерфейс JTAG
- Напряжение питания VCC «просело» ниже допустимого уровня, что вызвало сброс по включению (POR)
- Вход -RST/NMI сконфигурирован как NMI (установлен бит NMI)

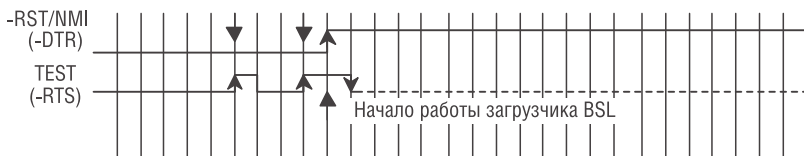


Рис. 2. Процедура активизации BSL с использованием выводов JTAG, объединённых с портами общего назначения

2.2. Микроконтроллеры с флеш-памятью MSP430 с обособленными выводами JTAG

Устройства с обособленными выводами интерфейса JTAG используют вывод TCK вместо вывода TEST.

Выполнение программы BSL начинается после того, как на входе TCK появятся как минимум два отрицательных перепада $1 \rightarrow 0$ и, кроме этого, вход TCK находится в состоянии лог. «0» во время перехода $0 \rightarrow 1$ на входе -RST/NMI (процедура активизации BSL, см. рис. 3). Такая схема активизации повышает надёжность запуска BSL.

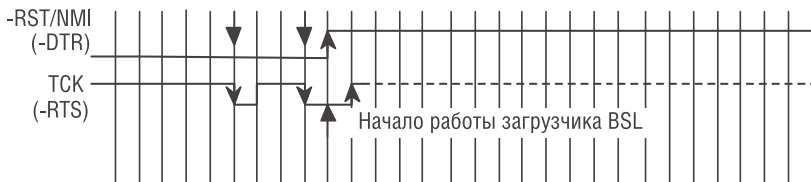


Рис. 3. Процедура активизации BSL с использованием обособленных выводов JTAG

Режим BSL не будет активизирован (с адреса вектора сброса BSL RESET) в том случае, если:

- Присутствует менее 2-х фронтов $1 \rightarrow 0$ на входе TCK во время лог. «0» на входе -RST/NMI
- Вход TCK находится в состоянии лог. «1» во время перехода $0 \rightarrow 1$ на входе -RST/NMI
- Ресурсы MSP430 уже контролируются через интерфейс JTAG
- Напряжение питания VCC «просело» ниже допустимого уровня, что вызвало сброс по включению (POR)
- Вход -RST/NMI сконфигурирован как NMI (установлен бит NMI)

3. Протокол УАПП (UART)

Используемый в этом случае протокол УАПП имеет следующие параметры:

- Скорость обмена 9600 бод, режим полудуплекс (один передатчик в любой момент времени).
- Стартовый бит, 8 бит данных (первым передаётся МЗР), бит контроля по чётности (even), 1 стоп-бит.
- Успешный приём подтверждается передачей соответствующего символа (acknowledge).

Примечание:

Использование скоростей обмена выше, чем 9600 бод может привести к сбоям в связи либо ошибкам записи флеш-памяти из-за несоблюдения временных параметров. Флеш-память в результате этого может быть подвергнута недопустимым нагрузкам, а операции стирания/записи не будут надёжно выполняться.

4. Последовательность синхронизации

Перед передачей любой из команд, следует отослать в BSL символ синхронизации (SYNC), равный 80h. Этот символ необходим для вычисления требуемых внутренних параметров, определяющих временные параметры УАПП и стирания/программирования флеш-памяти. Эта операция формирует опорное системное время BSL.

Далее, BSL отправляет символ подтверждения DATA_ACK = 90h, сообщаящий об успешном приёме.

Примечание:

Символ синхронизации не входит в последовательность данных Data Frame, описанную ниже.

5. Команды

Существуют два типа команд: требующие и не требующие пароля. Все команды, потенциально допускающие прямой или косвенный доступ к данным защищены паролем.

5.1. Незащищённые команды

- Принять пароль
- Полное стирание (стирает всю флеш-память, как основную, так и память данных)
- Считать версию BSL (команда поддерживается в BSL версии V1.60 или выше, либо в загрузаемой версии BL_150S_14x.txt)
- Сменить скорость обмена (команда поддерживается в BSL версии V1.60 или выше, либо в загрузаемой версии BL_150S_14x.txt)

5.2. Команды, защищённые паролем

- Принять блок данных для программирования флеш-памяти, ОЗУ либо периферийных модулей
- Передать блок данных
- Стереть сегмент
- Проверка стирания (команда поддерживается в BSL версии V1.60 или выше, либо в загрузаемой версии BL_150S_14x.txt)
- Загрузить программный счётчик и запустить пользовательскую программу

6. Процесс программирования

Процесс записи (команда «Принять блок данных» (RX data block)) во флеш-память, ОЗУ или в периферийные модули выполняется в режиме реального времени. Это означает, что байт либо слово данных обрабатывается непосредственно сразу после их принятия, а процесс записи завершается до того, как полностью принят следующий байт/слово. Таким образом, общее время записи определяется скоростью обмена, никакие способы буферизации не применяются.

Блоки данных, расположенные вне адресной области флеш-памяти, считаются загрузаемыми в ОЗУ или периферийные модули, поэтому биты данных контроля флеш-памяти игнорируются.

Примечание:

При потере связи по УАПП в результате помех на линии либо нарушении временных требований к передаче блоков данных, единственным способом восстановления обмена является повторная инициализация BSL.

7. Формат посылки

Формат посылки данных, используемый программатором, представляет собой стандартный последовательный протокол (SSP) фирмы TI для MSP430, дополненный предваряющей синхропоследовательностью (SS), как было описано выше. В таблице 1 BSL является приёмником.

Очевидная избыточность некоторых параметров является следствием адаптации под SSP а также служит для экономии объёма ПЗУ загрузчика.

Дополнительная информация по стандартному последовательному протоколу (SSP) содержится в документе «Применение программы начальной загрузки в микроконтроллеры MSP430 с флэш-памятью: программные и аппаратные предложения (версия В)», Application of Bootstrap Loader in MSP430 With Flash Hardware and Software Proposal (SLAA096).

7.1. Структура данных

- Первые 8 байт (от HDR до LH), которые являются обязательными (xx означает, что данные не являются значимыми).
- Необязательные байты данных от D1 до Dn.
- Два байта (CKL и CKH) контрольной суммы (являются необходимыми).

Подтверждение от BSL, необходимое для всех команд, кроме передачи блока данных и передачи версии BSL.

7.2. Контрольная сумма

16-битная (2 байта) контрольная сумма вычисляется по всем принятым/переданным байтам B1 ... Bn в посылке за исключением собственно байт контрольной суммы путём выполнения операции XOR над операндами-словами (2 байта) и инверсией результата.

Формула:

$$\text{CHECKSUM} = \text{INV}[(B1 + 256 \times B2) \text{ XOR } (B3 + 256 \times B4) \text{ XOR } \dots \text{ XOR } (Bn-1 + 256 \times Bn)]$$

или

$$\text{CKL} = \text{INV}[B1 \text{ XOR } B3 \text{ XOR } \dots \text{ XOR } Bn-1]; \text{ CKH} = \text{INV}[B2 \text{ XOR } B4 \text{ XOR } \dots \text{ XOR } Bn]$$

Таблица 1. Формат посылки команд BSL

Принятая команда BSL	H	D	R	C	M	D	L1	L2	AL	AH	LL	LH	D1	D2..Dn	CKL	CKH	ACK
Принять блок данных (RX data block)	80			12	n	n			AL	AH	n-4	0	D1	D2.. Dn-4	CKL	CKH	ACK
Принять пароль (RX Password)	80			10	24	24	xx	xx	xx	xx	xx	xx	D1	D2.. D20	CKL	CKH	ACK
Стереть сегмент (Erase segment)	80			16	04	04	04	04	AL	AH	02/04	A5	–	–	CKL	CKH	ACK
Полное стирание (Mass erase)	80			18	04	04	04	04	xx	xx	xx	xx	–	–	CKL	CKH	ACK
Проверка стирания (Erase check)	80			1C	04	04	04	04	AL	AH	LL	LH	–	–	CKL	CKH	ACK
Сменить скорость обмена (Change baud rate)	80			20	04	04	04	04	D1	D2	D3	xx	–	–	CKL	CKH	ACK
Передать блок данных (TX data block)	80			14	04	04	04	04	AL	AH	n	0	–	–	CKL	CKH	–
Ответ BSL	80			xx	xx	n	n	n	D1	D2..	...	...	...	...Dn	CKL	CKH	–
Передать версию BSL (TX BSL version)	80			1E	04	04	04	04	xx	xx	xx	xx	–	–	CKL	CKH	–
Ответ BSL	80			xx	10	10	10	10	D1	D2..	...	...	...	...D10	CKL	CKH	–

Примечание:

Все цифры в таблице – байты в шестнадцатеричном виде

ACK передаётся обратно от BSL.

Синхропоследовательность не входит в блок посылки.

Команды «Проверка стирания» и «Передать версию BSL» не входят в стандартный набор команд (V1.50 или выше).

Команда «Сменить скорость обмена» не входит в стандартный набор команд (V1.60 или выше либо загружаемый BL_150S_14x.txt).

Аббревиатуры:

HDR DATA_FRAME = 80h означает ожидается посылка данных в соответствии со стандартным последовательным протоколом (SSP) фирмы TI для MSP430.

CMD Идентификатор команды

L1, L2 Число байт в посылке от AL до Dn Ограничения: L1 = L2, L1 < 255, L1 чётное.

AL, AH Адрес начала блока либо адрес (проверки) записи либо адрес перехода, младший/старший байты

LL, LH Число байт данных (макс. 250), либо информация о стирании, младший/старший байты, либо длина блока проверки стирания (макс. FFFFh)

D1 ... Dn Байты данных

СКЛ, СКН 16-битная контрольная сумма, младший/старший байты

xx Данные могут быть любыми

-- Байты данных не принимаются и не передаются

АСК Символ подтверждения, возвращаемый BSL, может быть либо

DATA_ACK = 90h: Блок принят корректно, команда выполнена успешно, либо

DATA_NAK = A0h: Некорректный блок (например: ошибка контрольной суммы, $L1 \neq L2$), неизвестная, недопустимая либо выполненная с ошибками команда.

n Число байт в посылке от AL до Dn

7.3. Команды – детальное описание

См. Таблицу 1.

7.3.1. Общие положения

Следующие за байтом преамбулы HDR (80h) и идентификатором команды CMD, байты длины посылки L1 и L2 (которые должны быть равны между собой) содержат число байт, следующих за L2, за исключением байт контрольной суммы СКЛ и СКН.

Байты AL, AH, LL, LH, D1...Dn зависят от типа команды. Несмотря на это, байты контрольной суммы СКЛ (младший) и СКН (старший) являются обязательными.

Если пакет данных принят корректно и выполнение команды завершено успешно, BSL возвращает символ подтверждения DATA_ACK = 90h. При некорректно принятых посылках, операциях, не завершённых успешно, запрещённых и неопределённых командах возвращается символ DATA_NAK = A0h.

Примечание:

Версии BSL, ниже, чем V1.30 поддерживают только байтовые операции доступа. Из-за этого, периферийные модули с адресами 0100h–01FFh недоступны, т.к. ориентированы на по-словный доступ.

Начиная с версии V1.30 адреса 0000h–00FFh доступны побайтно; все остальные доступны по-словно.

7.3.2. Принять блок данных (RX Data Block)

Команда «Принять блок данных» используется для всех операций записи во флеш-память, ОЗУ либо регистры контроля периферийных модулей по адресам 0000h–01FFh. Команда защищена паролем.

16-битный чётный адрес начала блока задаётся байтами AL (младший) и AH (старший).

16-битная чётная длина блока определяется байтами LL (младший) и LH (старший). Ввиду того, что число собственно байт данных ограничено до 250, LH всегда равен 0.

Последующие байты данных завершаются байтами контрольной суммы SKL (младший) и SKH (старший). Если приём и программирование соответствующего блока данных были успешными, BSL передаёт символ подтверждения DATA_ACK. В противном случае BSL возвращает DATA_NAK.

Примечание:

Версии BSL V1.40 и выше поддерживают верификацию в режиме реального времени внутри MSP430 для адресов 0200h–FFFFh, что на 50% снижает время программирования/верификации.

В случае ошибки загружаемая версия программатора на основе загрузочного сектора BL_150S_14x.txt дополнительно содержит адрес первой некорректно записанной ячейки +3 в буфере ОЗУ по адресу 0200h (либо 021Eh для МК семейства F14x).

7.3.3. Принять пароль (RX Password)

Команда «Принять пароль» используется для снятия защиты с команд, защищённых паролем, выполняющих чтение, запись либо посекторное стирание памяти. Данная команда паролем не защищена.

Стартовый адрес, равно как длина блока, для данной команды не требуются, т.к. 32-байтовый пароль всегда расположен по адресам FFE0h–FFFFh. Байты данных от D1 до D20h содержат пароль, начинающийся с D1 по адресу FFE0h.

Если приём и верификация пароля выполнены успешно, BSL возвращает подтверждение DATA_ACK, а команды, защищённые паролем, разблокируются. В противном случае, BSL возвращает DATA_NAK.

Однократная разблокировка защищённых команд сохраняет своё действие вплоть до следующей инициализации сессии BSL.

7.3.4. Полное стирание (Mass Erase)

Команда «Полное стирание» производит стирание всей области флеш-памяти (основная память плюс информационная память, см. соответствующую документацию). Команда не защищена паролем.

Все параметры, указанные в Таблице 1, являются обязательными. После стирания BSL возвращает символ подтверждения DATA_ACK.

Полное стирание инициализирует область пароля со значением 32 байта 0FFh.

7.3.5. Стереть сегмент (Erase Segment)

Команда «Стереть сегмент» производит стирание определённых сегментов флеш-памяти и является защищённой паролем.

Байты адреса AL (младший) и AH (старший) определяют соответствующий сегмент. Корректным является любой чётный адрес в пределах сегмента, предназначенного к стиранию. По завершении операции BSL (V1.40 или ниже) возвращает символ подтверждения DATA_ACK.

BSL версий V1.60 или выше выполняет автоматическую проверку стирания соответствующего сегмента и возвращает DATA_NAK, если стирание не было выполнено успешно. В этом случае, адрес +1 первого нестёртого сегмента хранится в буфере в ОЗУ по адресу 0200h (или 021Eh для семейства F14x).

Стирание сегмента 0 очищает область пароля, таким образом, пароль принимает значение 32 байта 0FFh.

Задание LL = 0x04 и LH = 0xA5 приводит к полному стиранию основной памяти.

Выполнение такой команды должно быть повторено как минимум 12 раз, чтобы обеспечить общее время стирания не менее 200 мс. Последующая автоматическая проверка стирания не производится, следует дополнительно использовать команду «Проверка стирания» (Erase check).

Разбивка памяти микроконтроллеров MSP430Fxxxx по сегментам выглядит следующим образом:

0FE00h – 0FFFFh: Сегмент 0 основной памяти
 0FC00h – 0FDFFh: Сегмент 1
 0FA00h – 0FBFFh: Сегмент 2
 0F800h – 0F9FFh: Сегмент 3
 ...
 ...
 01200h – 013FFh: Сегмент n-1
 01100h – 011FFh: Сегмент n
 01080h – 010FFh: Сегмент A информационной памяти
 01000h – 0107Fh: Сегмент B информационной памяти

7.3.6. Проверка стирания (Erase Check)

Команда «Проверка стирания» проверяет стирание флеш-памяти в пределах заданного диапазона адресов. Команда защищена паролем.

16-битный стартовый адрес блока определяется байтами AL (младший) и AH (старший). 16-битная длина блока задаётся байтами LL (младший) и LH (старший). Оба могут быть как чётными, так и нечётными для обеспечения проверки границы чётности.

Если проверка стирания соответствующего блока выполнена успешно (все байты имеют значение 0FFh), BSL возвращает символ подтверждения DATA_ACK. В противном случае, BSL возвращает DATA_NAK и первый нестёртый адрес +1 сохраняется в буфере ОЗУ по адресу 0200h (или 021Eh для семейства F14x).

Примечание:

Эта команда не входит в набор стандартных команд. Она включена в BSL, начиная с версии V1.60 и в загружаемую версию BL_150S_14x.txt.

7.3.7. Сменить скорость обмена (Change Baud Rate)

Команда «Сменить скорость обмена» позволяет вести обмен на скоростях, выше, чем 9600 бод, являющейся скоростью обмена по умолчанию. Ускоренный обмен данными позволяет укоротить циклы программирования, что важно для устройств с большим объёмом флеш-памяти. Эта команда не защищена паролем.

Три байта от D1 до D3 определяют выбираемую скорость обмена. D1 и D2 определяют скорость процессора ($f \geq f_{min}$), D3 косвенно задаёт частоту генератора контроллера флеш-памяти ($f_{FTGmin} \leq f_{FTG} \leq f_{FTGmax}$) следующим образом:

D1:	F1xx:	Управляющий регистр основного модуля тактирования (Basic clock module) DCOCTL (DC0.2 ... DC0.0)
	F4xx:	Управляющий регистр умножителя частоты FLL+ SCF10 (D, FN_8 ... FN_2)
D2:	F1xx:	Управляющий регистр основного модуля тактирования BCSCCTL1 (XT20ff, Rsel.2 ... Rsel.0)
	F4xx:	Управляющий регистр умножителя частоты FLL+ SCF11 (NDC0)
D3:	0:	9600 бод
	1:	19200 бод
	2:	38400 бод

После приёма этой командной последовательности BSL возвращает символ подтверждения DATA_ACK на скорости обмена, соответствующей выбранной. Для программы обмена, работающей с BSL рекомендуется выдерживать паузу (примерно 10 мс) между сменой скорости обмена и последующим обменом данными для стабилизации системы тактирования.

Примечание:

Верхняя максимально допустимая скорость обмена зависит от различных параметров самой системы и окружающей среды, таких, как напряжение питания, диапазон температур и минимальной и максимальной рабочей частоты процессора. См. документацию на конкретный МК.

Примечание:

Эта команда входит в набор команд BSL, начиная с версии V1.60 и в загружаемую версию BL_150S_14x.txt.

Таблица 2. Рекомендации для MSP430F149 [F449]
 $(T_A = 25^\circ\text{C}, V_{CC} = 3.0\text{ В}, f_{max} = 6.7\text{ МГц})$ (см. [2])

Скорость обмена, бод	Частота процессора F_{min} (МГц) (см. [4])	D1 DCOCTL SCFIO (см. [1])	D2 BCSCCTL1 SCF1 (см. [1])	D3 (см. [1])	Время программирования/верификации 60КБ (сек.) (см. [3])
9600 (начальная)	1.05	80 (00)	85 (98)	00 (00)	78+3.7 (0.0)
19200	2.1	E0 (00)	86 (B0)	01 (01)	39+3.7(2.4)
38400	4.2	E0 (00)	87 (C8)	02 (02)	20+3.7 (2.4)

[1] D1...D3 – байты в шестнадцатеричном виде

[2] Значения в скобках () приведены для процессора MSP430F449.

[3] Дополнительные 3.7 (2.4) секунды связаны с загрузкой, верификацией и запуском загружаемого BSL.

[4] Минимальная частота процессора меньше, чем в стандартном BSL ПЗУ (см. Статус после инициализации, глава 12.3).

7.3.8. Загрузить программный счётчик (Load PC)

Команда «Загрузить программный счётчик» непосредственно загружает в программный счётчик (регистр R0) любое значение в пределах всего адресного пространства. Команда защищена паролем.

После приёма данной командной посылки, BSL возвращает символ подтверждения DATA_ACK.

После этого, выбранный адрес загружается в программный счётчик. Начинается выполнение программы с этого адреса, а сессия BSL прерывается.

Следует учитывать, что защита паролем в это время уже не действует.

7.3.9. Передать блок данных (TX Data Block)

Команда «Передать блок данных» используется для осуществления операции чтения из флеш-памяти, ОЗУ, либо регистров контроля периферийных модулей по адресам 0000h–01FFh. Команда защищена паролем.

16-битный стартовый адрес блока определяется байтами AL (младший) и AH (старший). 16-битная длина блока задаётся байтами LL (младший) и LH

(старший). Из-за того, что число байт данных ограничено до 250, байт LH всегда равен 0. В конце посылки передаются байты контрольной суммы CKL (младший) и CKH (старший).

В ответ на команду BSL возвращает запрошенный блок данных. После передачи HDR, незначащего CMD, L1 и L2, BSL передаёт байты данных от D1 до Dn, завершающиеся байтами контрольной суммы CKL (младший) и CKH (старший). Символ подтверждения в данном случае не требуется.

7.3.10. Передать версию BSL (TX BSL Version)

Команда «Передать версию BSL» возвращает пользователю информацию об идентификаторе чипа и программную версию загрузчика BSL. Пароль для данной команды не требуется.

Значения AL, AH, LL, и LH могут быть любыми, но, всё равно, должны быть переданы для обеспечения требований протокола. Посылка завершается байтами контрольной суммы CKL (младший) и CKH (старший).

После этого, BSL возвращает блок данных длиной 16 байт. Сначала передаётся HDR, потом незначащие параметры CMD, L1 и L2, после чего BSL пересылает байты данных от D1 до D16 (десятичных), завершается посылка байтами контрольной суммы CKL (младший) и CKH (старший). Символ подтверждения в данном случае не требуется.

Байты D1, D2 и D11, D12 (десятичные) содержат специальную информацию:

D1: Тип семейства (старший байт)

D2: Тип семейства (младший байт)

D11: Версия BSL (старший байт)

D12: Версия BSL (младший байт)

Остальные 12 байт содержат информацию для служебного пользования.

8. Загружаемые BSL

Для повышения функциональных возможностей BSL, иногда бывает удобно загрузить более свежую версию BSL в ОЗУ устройства с целью использования дополнительных возможностей. Это осуществляется путём передачи следующих команд BSL:

- Принять пароль (RX password) (снять защиту паролем для нижеследующих команд)
- Принять блок данных (RX data block) (код загружаемого BSL, стартовый адрес $\geq 220h$)
- Передать блок данных (TX data block) (для верификации)
- Принять блок данных (RX data block) (передать стартовый адрес)

- Загрузить программный счётчик (Load program counter PC) (записать стартовый адрес загружаемого BSL)
- Принять пароль (RX password) (снять защиту паролем для загруженного BSL)
- Выполнить любую команду (используя загруженный BSL)

Доступны следующие версии загружаемых BSL:

- **BL_150S_14x.txt** это полный BSL для семейств F14x/F13x с прошитым BSL версии 1.10. Поддерживает все функции загрузчика BSL версии V1.60. Ввиду того, что размер кода превышает 1 КБ, этот загрузчик можно использовать только с МК семейств F1x8 и F1x9. Адрес буфера ошибок для команд «Передать блок» («RX Block»), «Стереть сегмент» («Erase Segment»), и «Проверка стирания» («Erase Check») – 021Eh. BL_150S_14x.txt также может использоваться взамен PATCH.txt.
- **BS_150S_14x.txt** это урезанный BSL с неполным набором команд для семейств F14x/F13x с прошитой версией загрузчика BSL 1.10. Так как размер кода не превышает 512Б, этот загрузчик можно использовать совместно с любыми устройствами от F1x4 до F1x9. Поддерживаются следующие команды загрузчика BSL версии V1.60: «Сменить скорость обмена» (Change Baudrate), «Принять блок» (RX Block) (с верификацией в режиме реального времени), «Проверка стирания» (Erase Check) и «загрузить программный счётчик» (Load PC). Если требуется команда «Передать блок» (TX Block) (перенаправляемая к BSL, прошитому в ПЗУ), (например, для передачи адресов ошибок или верификации) загрузчик BSL должен быть по новой запущен из ОЗУ с помощью команды «Загрузить программный счётчик» (Load PC). Адрес буфера ошибок для команд «Передать блок» («RX Block») и «Проверка стирания» («Erase Check») – 021Eh. BS_150S_14x.txt также может быть использован в качестве частичной замены для PATCH.txt.

Примечание:

Пароль не требуется, команда «Принять пароль» отсутствует!

Более подробная информация по загрузке различных типов BSL, см. пример применения «Применение программы начальной загрузки в микроконтроллеры MSP430 с флэш-памятью: программные и аппаратные предложения (версия B)» (Application of Bootstrap Loader in MSP430 With Flash Hardware and Software Proposal) (SLAA096).

Программное обеспечение третьих фирм обычно использует загружаемые BSL для обеспечения максимальной функциональности, в частности, использования верификации в режиме реального времени, и для повышения скорости при работе с соответствующими устройствами.

9. Выход из BSL

Существуют 2 способа выхода из режима BSL:

- Микроконтроллер продолжает работу с определённого адреса, задаваемого командой «Загрузить программный счётчик». Следует учитывать, что защита паролем в это время уже не действует.
- Выполнить стандартную процедуру RESET (см. Рис. 1), что приведёт к выполнению микроконтроллером программы со стандартного адреса вектора сброса 0FFFFh.

10. Защита паролем

Защита паролем запрещает выполнение любой команды, потенциально допускающей непосредственный или косвенный доступ к данным. Без предварительного приёма корректного пароля после активизации BSL, могут быть выполнены только незащищённые паролем команды, такие, как «Полное стирание» и «Принять пароль» (опционально – «Передать версию BSL» и «Сменить скорость обмена»).

Выполнение команды «Принять пароль», сопровождаемое передачей корректного пароля разблокирует выполнение остальных команд.

Действие разблокировки сохраняется вплоть до следующей инициализации сессии BSL. Операция сброса ($-RST/NMI = 0 \rightarrow 1$, $VCC = 0 \rightarrow 1$) не оказывает влияния на функции защиты.

Пароль представляет собой значение 16-ти векторов прерываний, расположенных по адресам от FFE0h до FFFFh (256 бит), начиная с первого байта по адресу FFE0h. После выполнения процедуры «Полное стирание» и в незапрограммированных чипах значение пароля состоит из всех лог. «1»

Примечание:

Пользователь должен обращать внимание на тот факт, что пароль будет изменён в случае изменения векторов прерываний и инициализации следующей сессии BSL. Кроме этого, настоятельно рекомендуется инициализировать неиспользуемые вектора прерываний для повышения защиты данных.

11. Предохранитель защиты данных (Code Protection Fuse)

После того, как предохранитель JTAG (code protection fuse) пережжён, последующий доступ к функции JTAG/test невозможен. Единственным способом доступа чтения/записи к памяти является использование программатора на основе загрузочного сектора с передачей корректного пароля.

При этом, пережигание предохранителя JTAG посредством BSL невозможно. Если требуется пережигание предохранителя, следует использовать программирование через интерфейс JTAG.

12. Внутренние настройки и ресурсы BSL

Нижеследующие параграфы описывают внутренние настройки и ресурсы BSL. Ввиду того, что одинаковые МК могут существовать с различными прошитыми версиями BSL, для программы, работающей с BSL, требуется знать соответствующие настройки и ресурсы, которые могут зависеть как от типа устройства (например, назначение выводов RX/TX) так и от версии BSL (например, побайтовый/пословный доступ). Различные возможные комбинации параметров будут рассмотрены ниже.

12.1. Идентификатор чипа и версия BSL

Старшие 16 байт в ПЗУ загрузчика (по адресам 0FF0h–0FFh) содержат информацию об устройстве и версии загрузчика BSL в двоично–десятичном виде. Это справедливо для всех видов МК и всех версий BSL:

- 0FF0h–0FF1h: Идентификатор чипа (например, F413h для МК серии F41x).
- 0FFAh–0FFBh: Номер версии BSL (например, 0130h для версии BSL V1.30).

См. также таблицу соответствия между семействами MSP430 и версией BSL в Приложении А.

12.2. Вектора

Стартовая часть ПЗУ загрузчика содержит вектора программного вызова:

- 0C00h: Вектор для «холодного» запуска (мнемоника: BR &0C00h) \
- 0C02h: Вектор для «горячего» запуска (мнемоника: BR &0C02h). Версия V1.30 или выше. \
- 0C04h: Вектор(а) для будущего использования. Таблица может быть расширена. \

12.3. Статус после инициализации

После инициализации BSL, в силу вступают следующие настройки:

- Остановить сторожевой таймер Watchdog Timer
- Запретить все прерывания (GIE=0)
- В версии V1.10: указатель стека не изменяется, кроме тех случаев, когда он указывает на несуществующую область памяти. В этом случае, он будет инициализирован по адресу 021Ah.

- В версиях V1.30 и выше: указатель стека не изменяется в случае программного вызова BSL по адресу «горячего» запуска. В случае запуска BSL при помощи аппаратной активизации либо программного вызова по адресу «холодного» старта, указатель стека инициализируется по адресу 0220h

- F1xx:

Базовый модуль тактирования настраивается следующим образом (минимальная частота при этом равна 1.5МГц):

BCSCTL1 = 85h (RSEL = 5, XT2Off = 1)

DCOCTL = 80h (DCO = 4, MOD = 0)

BCSCTL2 = 00h только в случае «холодного» запуска

SR: SCG1 = 00h (SMCLK вкл) только в случае «холодного» запуска

F4xx:

Умножитель частоты FLL и модуль тактирования настраивается следующим образом (минимальная частота при этом равна 1.5МГц):

SCFIO = 00h (D = 0, FN_x = 0)

SCFI1 = 98h (N_DCO)

SCFQCTL: (M = 0)

SR: SCG0 = 1 (петля захвата FLL отключена)

FLL_CTL1 = 00h только в случае «холодного» запуска

- Программный УАПП (SW-UART): Timer_A функционирует в непрерывном режиме (continuous mode), источник тактирования – MCLK (Div = 1)

CCRO используется для сравнения

CCTL0 используется для поллинга CCIFG0

- Вывод TX сконфигурирован как выход, на выходе лог. «1» для состояния готовности RS232
- Вывод RX сконфигурирован как вход
- Защищенные паролем команды заблокированы (только в случае «холодного» запуска)

После системной инициализации, BSL готов к работе и находится в ожидании первой синхропоследовательности (SS), сопровождаемой посылкой данных, содержащих первую команду BSL.

12.4. Распределение памяти и используемые ресурсы

- Программный код загрузчика BSL расположен в загрузочном ПЗУ по адресам 0C00h–0FEFh.
- По адресам 0FF0h–0FFFh расположен идентификатор чипа
- Переменные BSL расположены в ОЗУ по адресам:

0200h–0213h (V1.10)

0200h–0219h (V1.30 и выше).

- Стек загрузчика BSL расположен в ОЗУ по адресам:
0214h–0219h (V1.10)
021Ah–021Fh (V1.30 или выше, только в случае «холодного» запуска).
- Используемые регистры:
R5–R9 (V1.30 и ниже)
R5–R10 (V1.40)
R5–R11 (V1.60).

Содержимое регистров не сохраняется.

- F1xx:

- Используемые регистры базового модуля тактирования:
DCOCTL по адресу 056h
BCSCTL1 по адресу 057h

- F4xx:

- Используемые регистры умножителя частоты FLL и модуля тактирования:
SCFI0 по адресу 050h
SCFI1 по адресу 051h
SCFQCTL по адресу 052h
- Используемые управляющие регистры таймера Timer_A:
TACTL по адресу 0160h
CCTL0 по адресу 0162h
TAR по адресу 0170h
CCR0 по адресу 0172h
- Влияния на обработку прерываний не оказывает.

13. Отличительные особенности версии BSL 1.10

Первая официальная версия V1.10 загрузчика BSL требует небольшого загружаемого дополнения PATCH.TXT, для корректного выполнения команды «Принять блок данных» (RX block). Ввиду этого, после запуска BSL следует выполнить следующие действия:

- Отправить команду «Принять пароль» (RX password) и разблокировать выполнение последующих команд
- Отправить команду «Загрузить программный счётчик» (Load program counter (PC)) со значением 0C22h (инициализировать указатель стека на безопасный адрес)

- Повторить команду «Принять пароль» (RX password) (unlock password protection for subsequent commands)
- Отправить команду «Принять блок данных» (RX data block) (код загружаемого дополнения, стартовый адрес 0220h)
- Отправить команду «Передать блок данных» (TX data block) (код загружаемого дополнения для верификации)
- С данного момента команды «Принять блок данных» (RX data block) и «Передать блок данных» (TX data block) могут быть использованы с одним условием: перед их выполнением следует установить программный счётчик на стартовый адрес обновления.
- Отправить команду «Загрузить программный счётчик» (Load program counter (PC)) со значением 0220h стартового адреса загружаемого обновления
- Отправить команду «Принять блок данных» (RX data block) (код, записываемый по любому адресу) – или –
- Отправить команду «Передать блок данных» (TX data block) (с любого адреса)
- Более подробная информация по загрузке обновления или других загрузчиков BSL содержится в примере применения «Применение программы начальной загрузки в микроконтроллеры MSP430 с флэш-памятью: программные и аппаратные предложения (версия В)», (Application of Bootstrap Loader in MSP430 With Flash Hardware and Software Proposal) (SLAA096).

14. Ссылки:

1. «Руководство пользователя MSP430x1xx» (MSP430x1xx Family User's Guide) (SLAU049)
2. «Руководство пользователя MSP430x4xx» (MSP430x4xx Family User's Guide) (SLAU056)
3. «Применение программы начальной загрузки в микроконтроллеры MSP430 с флэш-памятью: программные и аппаратные предложения (версия В)», Application of Bootstrap Loader in MSP430 With Flash Hardware and Software Proposal (SLAA096)

Список третьих фирм, предлагающих программные и аппаратные решения для работы с BSL, расположен на сайте, посвящённом микроконтроллеру MSP430 по адресу <http://www.ti.com/sc/msp430> (> Design Resources > Third Party > Tools).

Приложение А.

Различия между версиями BSL в различных микроконтроллерах

В таблицах данного раздела собрана информация о соответствии различных версий BSL разным микроконтроллерам серии MSP430 и используемых ими программных и аппаратных ресурсах.

Таблица 3. Версия BSL 1.10 в микроконтроллерах серий F13х, F14х(1), F11х, и F11х1

Тип микроконтроллера		F13x F14x(1)	F11x (устаревшие) F11x1 (устаревшие)
Версия BSL		1.10	
Адрес вектора BSL	«Холодный» запуск	0C00h	
	«Горячий» запуск	—	
Адрес идентификатора чипа		0FF0h	
Идентификатор чипа		F149h	F112h
Адрес версии BSL		0FFAh	
Версия BSL		0110h	
Номинальное время полного стирания, мс		17,2	
Доступ чтения/записи по адресам 0000h-FFFFh		побайтный	
Верификация в режиме реального времени (во время записи)		отсутствует	
Инициализация указателя стека	Недопустимый адрес SP	021Ah	
	Допустимый адрес SP	не изменяется	
Ресурсы, используемые BSL			
Вывод передатчика(TX)/вывод приёмника(RX)		P1.1/P2.2	
Используемый диапазон ОЗУ/стека		0200h-0219h	
Рабочие регистры		R5-R9	
Изменяемые регистры управления системным тактированием		BCSCTL1, DCOCTL	
Изменяемые регистры управления таймером Timer_A		TACTL, TAR, CCTLO, CCRO	
Подготовка к программному вызову		mov #00h, &CCTL0 bic.b #02h, &P1SEL bic.b #04h, &P2SEL bic.b #32h, &IE1 mov.b #00h, &BCSCTL2 mov #00h, SRbr &0C00h	

Тип микроконтроллера		F13x F14x(1)	F11x (устаревшие) F11x1 (устаревшие)
Версия BSL		1.10	
Комментарий 1	Обязательная доработка	Загрузить файл PATCH.TXT для исправления ошибки в ПЗУ. См. разделы «Отличительные особенности версии BSL 1.10» и «Загружаемые BSL»	
Комментарий 2	Опциональная доработка для F148 и F149: использовать загружаемый BSL (требуется > 1 кБ ОЗУ)	Загрузить файл BL_150S_14x.txt, что позволит использовать все возможности версии V1.60 и корректно использовать команду «Стереть сегмент» (Erase segment) См. раздел «Загружаемые BSL»	
Комментарий 3	Опциональная доработка для F1x4...F1x9: использовать маленький загружаемый BSL (требуется < 512 Б ОЗУ)	Загрузить файл BS_150S_14x.txt, что позволит использовать часть возможностей версии V1.60. См. раздел «Загружаемые BSL»	

Таблица 4. Версия BSL 1.30 в микроконтроллерах серий F41x, F11x, и F11x1

Тип микроконтроллера		F41x	F11x (устаревшие) F11x1A
Версия BSL		1.30	
Адрес вектора BSL	«Холодный» запуск	0C00h	
	«Горячий» запуск	0C02h	
Адрес идентификатора чипа		0FF0h	
Идентификатор чипа		F413h	F112h
Адрес версии BSL		0FFAh	
Версия BSL		0130h	
Номинальное время полного стирания, мс		206,4	
Доступ чтения/записи по адресам	0000h-00FFh	побайтный	
	0100h-FFFEh	пословный	
Верификация в режиме реального времени (во время записи)		отсутствует	
Инициализация указателя стека	«Холодный» запуск	0220h	
	«Горячий» запуск	не изменяется	

Тип микроконтроллера	F41x	F11x (устаревшие) F11x1A
Версия BSL	1.30	
Ресурсы, используемые BSL		
Вывод передатчика (TX)	P1.0	P1.1
Вывод приёмника (RX)	P1.1	P2.2
Используемый диапазон ОЗУ/стека	0200h-021Fh	
Рабочие регистры	R5-R9	
Изменяемые регистры управления системным тактированием	SCFIO, SCF11, SCFQCTL	BCSCTL1, DCOCTL
Изменяемые регистры управления таймером Timer_A	TACTL, TAR, CCTL0, CCR0	
Подготовка к программному вызову	mov #00h, &CCTL0 mov.b #00h, &FLLCTL1 br &0C00h	mov #00h, &CCTL0 mov.b #00h, &BCSCTL2 mov #00h, SR br &0C00h

Таблица 5. Версия BSL 1.40 в микроконтроллерах серий F12x

Тип микроконтроллера	F122, F123x	
Версия BSL	1.40	
Адрес вектора BSL	«Холодный» запуск	0C00h
	«Горячий» запуск	0C02h
Адрес идентификатора чипа	0FF0h	
Идентификатор чипа	F123h	
Адрес версии BSL	0FFAh	
Версия BSL	0140h	
Номинальное время полного стирания, мс	206,4	
Доступ чтения/записи по адресам	0000h-00FFh	побайтный
	0100h-FFFEh	пословный
Верификация в режиме реального времени (во время записи)	Для диапазона адресов 0200h – FFFEh	
Инициализация указателя стека	«Холодный» запуск	0220h
	«Горячий» запуск	не изменяется
Ресурсы, используемые BSL		
Вывод передатчика (TX)	P1.1	
Вывод приёмника (RX)	P2.2	

Тип микроконтроллера	F122, F123x
Версия BSL	1.40
Используемый диапазон ОЗУ/стека	0200h-021Fh
Рабочие регистры	R5-R10
Изменяемые регистры управления системным тактированием	BCSCTL1, DCOCTL
Изменяемые регистры управления таймером Timer_A	TACTL, TAR, CCTLO, CCR0
Подготовка к программному вызову	mov.b #00h, &BCSCTL2 mov #00h, SR br &0C00h

Таблица 6. Версия BSL 1.60 в микроконтроллерах серий F11x2, F12x2, F43x, F44x, FE42x, FW42x, FG43x

Тип микроконтроллера		F1122, F1132	F1222, F1232	F43x, F44x	FE42x, FW42x	FG43x
Версия BSL		1.60				
Адрес вектора BSL	«Холодный» запуск	0C00h				
	«Горячий» запуск	0C02h				
Адрес идентификатора чипа		0FF0h				
Идентификатор чипа		1132h	1232h	F449h	F427h	F439h
Адрес версии BSL		0FFAh				
Версия BSL		0160h				
Номинальное время полного стирания, мс		206,4				
Доступ чтения/записи по адресам	0000h-00FFh	побайтный				
	0100h-FFFEh	пословный				
Верификация в режиме реального времени (во время записи)		Для диапазона адресов 0200h – FFFEh				
Команда «Проверка стирания» (Erase check)		Присутствует				
Команда «Стирание сегмента» (Erase segment)		С верификацией стирания, адрес буфера ошибок 0200h				
Команда «Передача идентификатора» (TX identification)		Присутствует				
Команда «Сменить скорость обмена» (Change Baud Rate)		Присутствует				
Инициализация указателя стека	«Холодный» запуск	0220h				
	«Горячий» запуск	не изменяется				

Тип микроконтроллера		F1122, F1132	F1222, F1232	F43x, F44x	FE42x, FW42x	FG43x
Версия BSL		1.60				
Ресурсы, используемые BSL						
Вывод передатчика (TX)		P1.1			P1.0	
Вывод приёмника (RX)		P2.2			P1.1	
Используемый диапазон ОЗУ/стека		0200h-021Fh				
Рабочие регистры		R5-R12				
Изменяемые регистры управления системным тактированием		BCSCTL1, DCOCTL			SCFI0, SCFI1, SCFQCTL	
Изменяемые регистры управления таймером Timer_A		TACTL, TAR, CCTLO, CCR0				
Подготовка к программному вызову		mov.b #00h, &BCSCTL2 mov #00h, SR br &0C00h			mov.b #00h, &FLLCTL1 br &0C00h	
Комментарий	Адреса от 1000h до 11FFh верифицируются совместно (3 сегмента)	Используйте также команду «Проверка стирания» (Erase check)				

Таблица 7. Версия BSL 1.61 в микроконтроллерах серий F15х, F16х, F161х

Тип микроконтроллера		F15х, F16х	Предварительная информация для F1610, F1611, F1612
Версия BSL		1.61	
Адрес вектора BSL	«Холодный» запуск	0C00h	
	«Горячий» запуск	0C02h	
Адрес идентификатора чипа		0FF0h	
Идентификатор чипа		F169h	F16Ch
Адрес версии BSL		0FFAh	
Версия BSL		0161h	
Номинальное время полного стирания, мс		206,4	
Доступ чтения/записи по адресам	0000h-00FFh	побайтный	
	0100h-FFFFh	пословный	
Верификация в режиме реального времени (во время записи)		Для диапазона адресов 0200h – FFFh	
Команда «Проверка стирания» (Erase check)		Присутствует	
Команда «Стирание сегмента» (Erase segment)		С верификацией стирания, адрес буфера ошибок 0200h	

Тип микроконтроллера		F15x, F16x	Предварительная информация для F1610, F1611, F1612
Версия BSL		1.61	
Команда «Передача идентификатора» (TX identification)		Присутствует	
Команда «Сменить скорость обмена» (Change Baud Rate)		Присутствует	
Инициализация указателя стека	«Холодный» запуск	0220h	
	«Горячий» запуск	не изменяется	
Ресурсы, используемые BSL			
Вывод передатчика (TX)		P1.1	
Вывод приёмника (RX)		P2.2	
Используемый диапазон ОЗУ/стека		0200h-021Fh	
Рабочие регистры		R5-R12	
Изменяемые регистры управления системным тактированием		BCSCTL1, DCOCTL	
Изменяемые регистры управления таймером Timer_A		TACTL, TAR, CCTLO, CCRO	
Подготовка к программному вызову		mov.b #00h, &BCSCTL2 mov #00h, SR br &0C00h	
Комментарий	Адреса от 1000h до 11Ffh верифицируются совместно (3 сегмента)	Используйте также команду «Проверка стирания» (Erase check)	

II. Использование внутренних функций, периферийных блоков микроконтроллеров

Глава 2.

*Реализация часов реального времени
на MSP430*

Реализация часов реального времени на MSP430

Mike Mitchell [slaa076a.pdf (Англ.)]

В данном примере применения описывается создание часов реального времени (RTC) на микроконтроллере MSP430 при помощи встроенного таймера Timer_A или сторожевого таймера. Семейство микроконтроллеров MSP430 производства компании Texas Instruments является семейством 16-разрядных RISC микроконтроллеров со сверхнизким потреблением, усовершенствованной архитектурой и богатым набором периферийных устройств. В примере применения дается краткий обзор микроконтроллеров семейства MSP430 и более подробно обсуждается применение их в качестве часов реального времени. В данном описании приведены примеры схемной реализации, два примера кода прикладной программы и рассмотрены общие вопросы по проблемам обеспечения точности и стабильности.

1. Введение

Семейство микроконтроллеров MSP430 производства компании Texas Instruments является семейством 16-разрядных RISC микроконтроллеров со сверхнизким потреблением, усовершенствованной архитектурой и богатым набором периферийных устройств. Архитектура микропроцессоров реализована на основе новаторской схемы тактирования и передовых конструкторских решений, таких как высоко ортогональная структура, делающая прибор очень мощным и гибким. Микроконтроллеры MSP430x1xx потребляют менее 350 мкА при работе на частоте 1 МГц при 3 В питании и могут пробудиться из режима «сна», в котором потребляют менее 2 мкА, менее чем за 6 мкс. Такие низкое потребление и малое время выхода из режима «сна» позволяют изготавливать на основе этих микроконтроллеров устройства, работающие от автономного источника питания.

Кроме того, микроконтроллеры семейства MSP430 имеет обширный набор встроенных периферийных модулей и большой объем встроенной памяти, что позволяет изготавливать на их основе реальные «системы на кристалле». Микроконтроллеры содержат такие периферийные модули как 12- и 14-битные АЦП, градиентные АЦП, мультиплицируемые таймеры (некоторые с регистром захвата/сравнения и способностью формировать ШИМ сигнал), драйвер ЖКИ, встроенный задающий генератор, H/W перемножитель, USART, сторожевой таймер, GPIO и т.д.

Подробное описание микроконтроллеров семейства MSP430 можно найти в интернете по адресу <http://www.ti.com>.

1.1. Применение часов реального времени

Часы реального времени (RTC) используются в различных устройствах – от наручных часов до систем регистрации событий. В данном примере применения описывается способ реализации часов реального времени на микроконтроллере MSP430F1121. В некоторых приложениях MSP430, работающий в режиме часов реального времени может заменить аппаратные RTC, что позволит упростить разработку системы и снизить ее стоимость. В данном описании показаны примеры реализации часов реального времени на основе таймера Timer_A и сторожевого таймера; однако, можно использовать и любой другой таймер MSP430.

Все микроконтроллеры семейства MSP430 способны выполнять функции часов реального времени. Дополнительные повышение степени системной интеграции, снижение потребления и стоимости могут быть достигнуты и за счет выполнения микроконтроллером MSP430 функций других периферийных модулей. Например, микроконтроллеры '33x серии могут выполнять функции часов реального времени, супервизора напряжения питания системы, драйвера ЖКИ и UART, объединяя все эти четыре функции в одном приборе со сверх малым потреблением.

1.2. Задающий генератор MSP430

Встроенный задающий генератор – одна из обычно неправильно истолковываемых пользователями MSP430 тем. Основной причиной заблуждения является непонимание того, как вырабатываются стабильные системные синхрои импульсы. Все микроконтроллеры семейства MSP430 содержат и RC-генератор с цифровым управлением, и кварцевый генератор. RC-генератор обычно используется для синхронизации вычислительного ядра, а кварцевый генератор – для синхронизации периферийных устройств. В описываемых часах реального времени кварцевый генератор используется в качестве генератора тактовых импульсов для таймера/счетчика, который является времязадающим (таймер Timer_A или сторожевой таймер). Поэтому здесь не возникает проблем нестабильности, обычно присущих RC-генераторам.

2. Цифровая коррекция фазового сдвига

Общая идея реализации RTC весьма проста. Часы реального времени состоят из таймера/счетчика, вырабатывающего 1-секундные прерывания и небольшой программы обработки этих прерываний. Вычислительное ядро может бездействовать или выполнять другие функции между прерываниями. Данный пример применения и прикладная программа разработаны для мик-

роконтроллера MSP430F1121. В нижеследующих разделах дается полное описание этого устройства.

2.1. Формирование синхриимпульсов

Для формирования синхриимпульсов в RTC используется LFXT1 автогенератор, работающий в режиме LF с 32768 Гц кварцевым резонатором. Выход генератора LFXT1 подключен к формирователю ACLK. В свою очередь ACLK используется в качестве задающего генератора для таймера/счетчика, который является основой часов реального времени.

DCO вырабатывает сигнал для синхронизации вычислительного ядра и MCLK. Фактически вычислительное ядро работает асинхронно с периферийным таймером/счетчиком. Точность RTC не изменяется до тех пор, пока вычислительное ядро способно обслуживать каждое прерывание до возникновения следующего. Для более полного ознакомления с основным синхронизирующим модулем обратитесь к руководству пользователя по микроконтроллерам семейства MSP430x1xx (доступному в интернете по адресу <http://www.ti.com/sc/msp430>) и техническому описанию микроконтроллеров семейства MSP430x11x1.

2.2. Выбор Таймера

Микроконтроллер MPS430F1121 содержит два таймера: сторожевой таймер и таймер Timer_A. В данном примере применения описываются часы реального времени, реализованные на основе обоих таймеров. В обоих случаях таймер настроен таким образом, чтобы работать непрерывно и формировать прерывания точно в 1-секундных интервалах. Так как таймер настроен на работу от ACLK, а генератор тактовых импульсов работает от 32768 Гц кварцевого резонатора, то таймер просто досчитывает до 32767 и обнуляется, вырабатывая при переполнении запрос на прерывания. Для выполнения функции часов реального времени вычислительному ядру необходимо только обработать это прерывание.

Любой таймер, способный вырабатывать 1-секундные прерывания может использоваться с прикладной программой, приведенной в данном примере применения. Кроме того, программа может быть легко переделана для работы с любым другим таймером, формирующим периодические прерывания. Например, если таймер формирует прерывания каждые 0.5 секунды, то вычислительное ядро для формирования 1-секундных прерываний может использовать только каждое второе прерывание, и так далее.

2.3. Внешний интерфейс

Приведенные в приложении прикладные программы RTC не обеспечивают внешнего интерфейса. Если использовать вместо оговоренного любой другой

микроконтроллер семейства MSP430, то имеется возможность выбора обслуживаемого интерфейса: I2C, параллельный, UART, последовательный и т.д. Для микроконтроллеров семейства MSP430 имеются программные модули, позволяющие реализовать эти интерфейсы. Для изготовления законченных часов реального времени на базе описываемого устройства необходимо просто добавить в приведенный код прикладной программы код программы обслуживания нужного интерфейса. Более детальные обсуждения некоторых из возможных интерфейсов могут быть найдены в MSP430 FamilyApplication Report Book и других примерах применения.

2.4. Описание принципиальной схемы

На рисунке 1 приведена принципиальная схема часов реального времени. Обратите внимание, что единственный требуемый внешний компонент – 32768 Гц кварцевый резонатор.

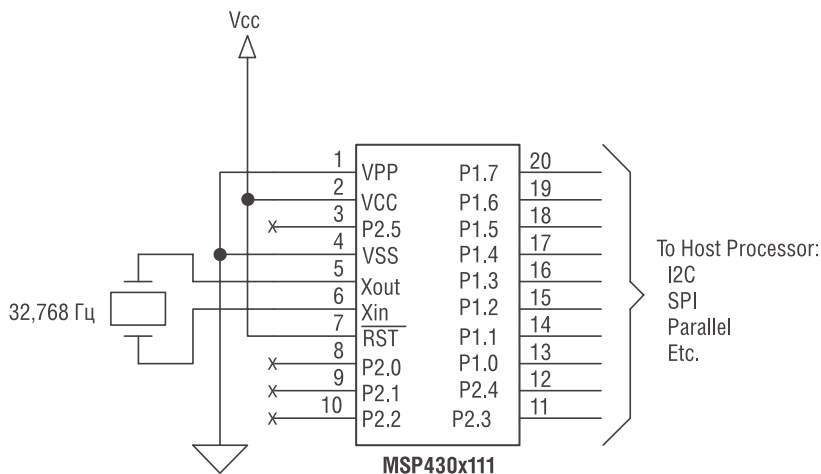


Рис. 1. Принципиальная схема часов реального времени

2.5. Потребляемый ток

Типовой ток потребления микроконтроллера MSP430F11x1 при работе в активном режиме на частоте 1 МГц от 3 В источника питания – 300 мкА. Типовой ток потребления в третьем режиме пониженного потребления (режиме сна) – 1.6 мкА при 3 В питании. Микроконтроллер выходит из третьего режима пониженного потребления менее чем за 6 мкс, а подпрограмма синхронизации выпол-

няется примерно за 130 мкс. Учитывая сверхнизкий ток потребления и малое время восстановления работоспособности, микроконтроллер MSP430F11x1, работающий в качестве RTC потребляет очень маленький средний ток, а поэтому способен работать длительное время от автономного источника питания.

2.6. Проблемы обеспечения точности

Точность RTC зависит от точности используемого кварцевого резонатора. Это позволяет выбрать такой кварцевый резонатор, который обеспечит требуемую точность.

2.6.1. Выбор точности кварцевого резонатора

Точность кварцевого резонатора определяется, прежде всего, следующими двумя параметрами: стабильностью резонансной частоты и емкостью нагрузки.

Влияние стабильности очевидно: чем выше стабильность резонансной частоты кварцевого резонатора, тем выше будет точность RTC. Например, RTC, использующие кварцевый резонатор со стабильностью резонансной частоты 30 ppm, будут более точными, чем часы, использующие кварцевый резонатор со стабильностью резонансной частоты 200 ppm.

Емкость нагрузки также влияет на точность хода RTC. Емкость нагрузки кристалла — это емкость, требующаяся резонатору, а не собственная емкость резонатора. Для работы на указанной частоте резонатору необходима емкостная нагрузка. 32768 Гц автогенератор всех микроконтроллеров семейства MSP430 содержит встроенные нагрузочные конденсаторы с номинальной емкостью 12 пФ. Так как эти конденсаторы включены последовательно, то суммарная емкость нагрузки получается 6 пФ.

Кроме того, емкость, сформированная автогенератором, увеличивается за счет существующих паразитных емкостей печатной платы и выводов. Например, 2 пФ паразитная емкость не является редкостью на печатной плате. Добавление 2 пФ к 6 пФ увеличивает полную емкостную нагрузку для резонатора до 8 пФ. В некоторых случаях, в зависимости от технических параметров резонатора и паразитных емкостей, в схему должны быть добавлены параллельные конденсаторы для обеспечения требуемой емкостной нагрузки.

2.7. Расширяемость

MSP430x11x — недорогие микроконтроллеры со сверхмалым потреблением, которые могут идеально заменить аппаратно реализованные часы реального времени. Основным преимуществом от использования MSP430 в качестве RTC является расширяемость функций, выполняемых MSP430 по сравнению с обыкновенными часами реального времени. Все микроконтрол-

леры MSP430x11x содержат 16-разрядное вычислительное ядро с RISC архитектурой, 16-разрядный сторожевой таймер, 16-разрядный таймер Timer_A (с тремя регистрами захвата/сравнения и аналоговым компаратором), минимум 128 байтное ОЗУ, минимум 2 Кбайтное ПЗУ и минимум 14 линий портов ввода-вывода общего назначения (GPIO). Не все эти функциональные возможности доступны одновременно у одного микроконтроллера, но они позволяют обеспечить устройству большую гибкость, чем дают стандартные аппаратно реализованные RTC.

Кроме того, модуль таймера Timer_A способен выполнять аналого-цифровое преобразование, формировать сигнал с ШИМ и работать в режиме UART с пропускной способностью до 115 200 бод. Сторожевой таймер способен работать как простой таймер, а все выводы GPIO и все периферийные устройства имеют широкие возможности по формированию прерываний.

3. Обсуждение прикладной программы

Код прикладной программы часов реального времени приведен в двух вариантах в приложении А. В каждом примере есть подпрограмма инициации, основная ветвь, подпрограмма расчета временных показателей (часов, минут и секунд) и подпрограмма обработки прерывания, отслеживающая 1-секундные прерывания от таймера.

Подпрограмма инициации производит настройку модулей MSP430. Модуль таймера настраивается на непрерывный счет от 0 до 32767 и формирование прерывания при переполнении. Также подпрограмма производит установку необходимых параметров основного модуля синхронизации.

Основная ветвь – программа, выполняемая каждый раз после формирования прерывания, и инициализирующая режим сна микроконтроллера. Даже тогда, когда вычислительное ядро переходит в режим ожидания, таймер все равно продолжает работать.

Подпрограмма обслуживания прерывания (interrupt service routine – ISR) таймера обрабатывает биты регистра состояния (status register – SR), которые записываются в стек до выполнения подпрограммы обслуживания прерывания. Это позволяет вычислительному ядру находиться в активном режиме, а не в режиме сна, по выходу из подпрограммы обслуживания прерывания.

Подпрограмма расчета временных параметров выполняется каждый раз после 1-секундного прерывания таймера. Она пересчитывает значения секунд, минут и часов в двоично-десятичной форме. В описании встроенных периферийных модулей микроконтроллеров семейства MSP430 можно найти более сложную подпрограмму, которая высчитывает и значения дней, месяцев и лет (учитывая и високосные годы).

4. Заключение

Микроконтроллеры семейства MSP430 могут работать в качестве часов реального времени с дополнительными функциональными возможностями, что дает дополнительную гибкость устройству и повышает степень интеграции на системном уровне.

Ссылки:

1. MSP430x1xx Family User' Guide, literature number SLAU049
2. MSP430x11x1 data sheet, literature number SLAS241C
3. MSP430 Family Application Report Book, literature number SLAA024

Приложение А: пример программы

А.1 Использование сторожевого таймера – файл RTC11xWD.s43

```

;*****
; ЭТА ПРОГРАММА ПОСТАВЛЯЕТСЯ В ВИДЕ «КАК ЕСТЬ». КОМПА-
НИИ TI НЕ ДАЕТ НИКАКИХ ГАРАНТИЙ
; И НЕ НЕСЕТ НИКАКОЙ ОТВЕТСТВЕННОСТИ ЗА ВОЗМОЖНЫЕ НЕ-
; ТОЧНОСТИ РАБОТЫ ИЛИ НЕПРАВИЛЬНОСТЬ РАБОТЫ ПРОГРАММЫ.
; ПРИ КОММЕРЧЕСКОМ ИСПОЛЬЗОВАНИИ ДАННОЙ ПРОГРАММЫ
; КОМПАНИЯ TI ОТКАЗЫВАЕТСЯ ОТ АВТОРСТВА И СНИМАЕТ
; С СЕБЯ ВСЯКУЮ ОТВЕТСТВЕННОСТЬ.
;
; КОМПАНИЯ TI ОТКАЗЫВАЕТСЯ ОТ ЛЮБЫХ ГАРАНТИЙ В ОТНОШЕ-
; НИ ТОГО, ЧТО ПРОГРАММА НЕ МОЖЕТ ПРИНЕСТИ ПРЯМЫЕ ИЛИ
; КОСВЕННЫЕ УБЫТКИ ИЛИ ПОТЕРЮ ДОВЕРИЯ ИЛИ ПРИБЫЛИ
; ПРИ ИСПОЛЬЗОВАНИИ ЕЕ ТЕОРИИ ИЛИ АЛГОРИТМОВ.
; ОТВЕТСТВЕННОСТЬ ПРИ КОММЕРЧЕСКОМ ИСПОЛЬЗОВАНИИ
; ДАННОЙ ПРОГРАММЫ ПОЛНОСТЬЮ ЛЕЖИТ НА ЛИЦЕ, ИСПОЛЬЗО-
ВАВШЕМ ДАННУЮ ПРОГРАММУ
;
; Если не указано иное, программа, созданная компанией
; Texas Instruments поставляется на условиях «свободно
; распространяемого программного обеспечения».
; Вы можете свободно изменять программу без всяких ог-
раничений
; Передавать программу третьим лицам можно только при
; доведении до их сведений этих условий и при
; его согласии с этими условиями.
; Вы должны приводить лицензию в каждой копии или
; частичной копии.

```

```

;
; Вы признаете и соглашаетесь, что Программа содержит
; защищенный авторским правом материал, коммерческую
; тайну и другую частную информацию компании TI
; и защищена в соответствии с законами об авторском
; праве, международным соглашением об авторском праве
; и законом о коммерческой тайне, а так же законом об
; охране интеллектуальной собственности. Для защиты
; авторских прав компании TI Вы соглашаетесь
; не декомпилировать, перерабатывать, или
; преобразовывать программу к удобочитаемому виду.
; Вы соглашаетесь ни в коем случае не изменять и не
; удалять внесенную в программу информацию
; об авторском праве. Компания TI оставляет за собой
; все права, не оговоренные в данной лицензии. Кроме
; того, что оговорено в данной лицензии, не может
; рассматриваться при возможном судебном
; разбирательстве.
;
; Вы не можете использовать программу для микроконт-
роллеров, которые не производит компания TI.
; *****
; RTC, использующие сторожевой таймер
; Описание: Эта программа демонстрирует возможность
; реализации часов реального времени.
; В основной программе MSP430 переводится в режим
; LPM3.
; Сторожевой таймер формирует прерывание каждую
; секунду и переводит MSP430 в активный режим, который
; позволяет выполнить основную программу.
; Основная программа вызывает подпрограмму часов Clock
; и вновь переводит микроконтроллер в режим LPM3.
; Эта программа написана для микроконтроллера
; MSP430F1121, но легко может быть переработана для
; использования в любом микроконтроллере семейства
; MSP430.
; Для хранения информации о часах, минутах и секундах
; используются три регистра.
; *****
#include <msp430x11x1.h> ; включить файл заголовков
; переменные RTC
#define          SEC    R13

```

```

#define      MIN      R14
#define      HR       R15
;-----
;          Программа RESET
RSEG CODE
;-----
RESET MOV    #02FEh,SP      ; Инициализация указателя
                               ; стека
CALL  #Setup                ; Подготовка ЖКИ и основного
                               ; таймера

; Основная программа
Mainloop BIS  #LPM3,SR      ; Установка битов SR
                               ; для режима LPM3
CALL  #Clock                ; обновление состояния часов
JMP   Mainloop              ; окончание основной программы
;-----
; Clock: обновление состояния переменных секунд,
; минут и часов (SEC, MIN и HR)
;
; Автор: Lutz Bierl.
;
; Эта подпрограмма, которая считает количество часов,
; минут и секунд.
; Она может использоваться с любым периферийным
; счетчиком любого микроконтроллера семейства MSP430,
; имеющим период переполнения 1 сек.
;
; Эта подпрограмма весьма специфическая.
; Она рассчитывает секунды, минуты и часы только
; в двоично-десятичном коде. При необходимости могут
; быть рассчитаны значения и в шестнадцатеричном коде.
;
; Среди примеров применения MSP430 имеются описания
; реализации часов реального времени, которые
; также рассчитывают и год с учетом коррекции
; високосного года.
;
;-----
Clock SETC      ; установка бита переноса
DADC.b          SEC      ; увеличение десятичного значения
                               ; секунд
CMP.b #060h,SEC      ; окончилась минута?

```

```

JLO          Clockend      ; нет, возврат
CLR.b        SEC           ; Да, очистка секунд
DADC.b        MIN          ; увеличение десятичного
                        ; значения минут
CMP.b #060h,MIN           ; прошло 60 минут?
JLO          Clockend      ; Нет, возврат
CLR.b        MIN          ; Да, очистка минут
DADC.b        HR           ; увеличение десятичного
                        ; значения часов
CMP.b #024h,HR           ; прошло 24 часа?
JLO          Clockend      ; Нет, возврат
CLR.b        HR           ; Да, очистка часов
Clockend     RET           ;
;-----
; Setup: Настройка модулей и управляющих регистров
;-----
Setup BIS.b #BIT0,&IE1    ; Разрешение прерывания
                        ; от сторожевого таймера
MOV #WDTPW+WDTTMSEL+WDTCNTCL+WDTSSEL,&WDTCTL
                        ; Остановка сторожевого
                        ; таймера
                        ; Установка интервала
                        ; режима таймера и установка
                        ; интервала прерывания
                        ; 1 секунда с ACLK.
ClearRTC      MOV.b #00h,SEC ; Очистка секунд (SEC)
MOV.b #00h,MIN ; Очистка минут (MIN)
MOV.b #00h,HR ; Очистка часов (HR)
EINT          ; Разрешение
                ; прерываний
RET           ; установки выполнены
;-----
; Подпрограмма обработки прерывания от сторожевого
; таймера:
; ЦП просто переходит в активное состояние в RETI
; путем манипулирования битами SR в переменной SR,
; которая помещена в стек.
; Флаг прерывания очищается автоматически
;-----
WDINT BIC #LPM3,0(SP) ; Очистка SR LPM3 битов
                        ; в верхней части стека
RETI ;

```

```

;-----
RSEG INTVEC ; вектора прерываний
;-----
DW RESET ;
DW RESET ;
DW RESET ;
DW RESET ;
DW RESET ;
DW RESET ;
DW RESET ;
DW RESET ;
DW RESET ; Таймер Timer_A (CCIFG0)
DW WDINT ; Сторожевой таймер
DW RESET ;
DW RESET ;
DW RESET ;
DW RESET ; NMI, повреждение автогенератора
DW RESET ; POR, внешний сброс, сторожевой таймер
END

```

A.2 Использование таймера Timer_A – файл RTC11xTA.s43 File

```

;*****
; ЭТА ПРОГРАММА ПОСТАВЛЯЕТСЯ В ВИДЕ «КАК ЕСТЬ». КОМ-
; ПАНИЯ TI НЕ ДАЕТ НИКАКИХ ГАРАНТИЙ И НЕ НЕСЕТ НИКАКОЙ
; ОТВЕТСТВЕННОСТИ ЗА ВОЗМОЖНЫЕ НЕТОЧНОСТИ РАБОТЫ ИЛИ
; НЕПРАВИЛЬНОСТЬ РАБОТЫ ПРОГРАММЫ.
; ПРИ КОММЕРЧЕСКОМ ИСПОЛЬЗОВАНИИ ДАННОЙ ПРОГРАММЫ
; КОМПАНИЯ TI ОТКАЗЫВАЕТСЯ ОТ АВТОРСТВА И СНИМАЕТ
; С СЕБЯ ВСЯКУЮ ОТВЕТСТВЕННОСТЬ.
;
; КОМПАНИЯ TI ОТКАЗЫВАЕТСЯ ОТ ЛЮБЫХ ГАРАНТИЙ
; В ОТНОШЕНИ ТОГО, ЧТО ПРОГРАММА НЕ МОЖЕТ ПРИНЕСТИ
; ПРЯМЫЕ ИЛИ КОСВЕННЫЕ УБЫТКИ ИЛИ ПОТЕРЮ ДОВЕРИЯ
; ИЛИ ПРИБЫЛИ ПРИ ИСПОЛЬЗОВАНИИ ЕЕ ТЕОРИИ ИЛИ
; АЛГОРИТМОВ.
; ОТВЕТСТВЕННОСТЬ ПРИ КОММЕРЧЕСКОМ ИСПОЛЬЗОВАНИИ
; ДАННОЙ ПРОГРАММЫ ПОЛНОСТЬЮ ЛЕЖИТ НА ЛИЦЕ,
; ИСПОЛЬЗОВАВШЕМ ДАННУЮ ПРОГРАММУ
;
; Если не указано иное, программа, созданная компанией
; Texas Instruments поставляется на условиях «свободно
; распространяемого программного обеспечения».

```


; Вы можете свободно изменять программу без всяких
; ограничений
; Передавать программу третьим лицам можно только при
; доведении до их сведений этих условий и при
; его согласия с этими условиями.
; Вы должны приводить лицензию в каждой копии или
; частичной копии.
;
; Вы признаете и соглашаетесь, что Программа содержит
; защищенный авторским правом материал, коммерческую
; тайну и другую частную информацию компании TI
; и защищена в соответствии с законами об авторском
; праве, международным соглашением об авторском праве
; и законом о коммерческой тайне, а так же законом
; об охране интеллектуальной собственности. Для защиты
; авторских прав компании TI
; Вы соглашаетесь не декомпилировать, перерабатывать,
; или преобразовывать программу к удобочитаемому виду.
; Вы соглашаетесь ни в коем случае не изменять и не
; удалять внесенную в программу информацию об
; авторском праве. Компания TI оставляет за собой
; все права, не оговоренные в данной лицензии. Кроме
; того, что оговорено в данной лицензии, не может
; рассматриваться при возможном судебном
; разбирательстве.
; Вы не можете использовать программу для
; микроконтроллеров, которые не производит
; компания TI.
;
; *****
; RTC, использующие таймер TIMER_A
; Описание: Эта программа демонстрирует возможность
; реализации часов реального времени.
; В основной программе MSP430 переводится в режим
; LPM3.
; Таймер Timer_A формирует прерывание раз в секунду,
; по которому MSP430 переходит в активный режим
; и оканчивает выполнение основной программы
; Основная программа вызывает подпрограммы Clock,
; а затем переводит микроконтроллер в режим LPM3.
;
; Эта программа написана для микроконтроллера
; MSP430F1121, но легко может

```

; быть переработана для использования в любом
; микроконтроллере семейства MSP430.
;
; Для хранения информации о часах, минутах и секундах
используются три регистра.
;*****
;
#include <msp430x11x1.h> ; включить файл заголовков
                        ; определения переменных RTC

#define      SEC      R13
#define      MIN      R14
#define      HR       R15
;-----
; Программа RESET
RSEG  CODE
;-----
RESET MOV #02FEh,SP      ; Инициализация указателя
                        ; стека
CALL  #Setup            ; Подготовка ЖКИ и основного
                        ; таймера

; Основная программа
Mainloop  BIS  #LPM3,SR   ; установка битов SR
                        ; для режима LPM3
CALL  #Clock             ; обновление часов
JMP    Mainloop          ; окончание основной
                        ; программы
;-----
; Clock: Обновление переменных секунд, минут и часов
; (SEC, MIN и HR)
;
; Автор Lutz Bierl.
;
; Эта подпрограмма, которая считает количество часов,
; минут и секунд.
; Она может использоваться с любым периферийным
; счетчиком любого микроконтроллера семейства MSP430,
; имеющим период переполнения 1 сек.
;
; Эта подпрограмма весьма специфическая.
; Она рассчитывает секунды, минуты и часы только
; в двоично-десятичном коде. При необходимости могут
; быть рассчитаны

```

```

; значения и в шестнадцатеричном коде.
;
; Среди примеров применения MSP430 имеются описания
; реализации часов реального времени, которые
; также рассчитывают и год с учетом коррекции
; високосного года.
;
;-----
Clock SETC                ; Установка бита
                        ; переноса
DADC.b      SEC          ; увеличение
                        ; десятичного значения секунд
CMP.b #060h,SEC          ; Прошла одна минута?
JLO         Clockend     ; Нет, возврат
CLR.b      SEC          ; Да, очистка секунд
DADC.b      MIN         ; Увеличение десятичного
                        ; значения минут
CMP.b #060h,MIN         ; Прошло 60 минут?
JLO         Clockend     ; Нет, возврат
CLR.b      MIN         ; Да, очистка минут
DADC.b      HR          ; Увеличение десятичного
                        ; значения часов
CMP.b #024h,HR          ; Прошло 24 часа?
JLO         Clockend     ; Нет, возврат
CLR.b      HR          ; Да, очистка часов
Clockend RET ;
;-----
; Setup: Настойка модулей и управляющих регистров
;-----
Setup MOV    #WDTPW+WDTHOLD,&WDTCTL ; Остановка
                        ; сторожевого
                        ; таймера
setupTA      MOV    #TASSEL0+TACLR,&TACTL ; ACLK -
                        ; источник для
                        ; Timer_A.
BIS    #CCIE,&CCTL0      ; Разрешение
                        ; прерывания от CCR0.
MOV    #07FFFh,&CCR0      ; загрузка в CCR0
                        ; значения 32,767.
BIS    #MC0, &TACTL      ; запуск TA в режиме
                        ; "up to CCR0"
ClearRTC    MOV.b #00h,SEC ; Очистка переменной

```

```

SEC
MOV.b #00h,MIN          ; Очистка переменной MIN
MOV.b #00h,HR           ; Очистка переменной HR
EINT                    ; Разрешение прерываний
RET ;

;-----
; Подпрограмма обработки прерывания от таймера Timer_A:
; ЦП просто переходит в активное состояние в RETI
; путем манипулирования битами SR
; в переменной SR, которая помещена в стек.
; Флаг прерывания CCR0 очищается автоматически
;-----
CCR0INT      BIC #LPM3,0(SP) ; Очистка битов SR
                                   ; LPM3 переход в верх ]
                                   ; указателя стека
RETI
;-----
RSEG INTVEC ; Вектора прерываний
;-----
DW    RESET      ;
DW    RESET      ;
DW    RESET      ;
DW    RESET      ;
DW    RESET      ;
DW    RESET      ;
DW    RESET      ;
DW    RESET      ;
DW    RESET      ;
DW    CCR0INT     ; Таймер Timer_A (CCIFG0)
DW    RESET      ; Сторожевой таймер
DW    RESET      ;
DW    RESET      ;
DW    RESET      ; NMI, повреждение автогенератора
DW    RESET      ; POR, внешний сброс, сторожевой
                                   ; таймер
END

```

II. Использование внутренних функций, периферийных блоков микроконтроллеров

Глава 3.

Использование широтно-импульсной модуляции таймера В в качестве ЦАП

Использование широтно-импульсной модуляции таймера В в качестве ЦАП

Mike Mitchel [slaa116.pdf (Англ.)]

В этом примере применения описан способ одновременной генерации синусоидального, пилообразного и сигнала с постоянным уровнем с использованием сигналов широтно-импульсной модуляции (ШИМ), получаемых с помощью таймера Timer_B семейства микроконтроллеров со сверхнизким энергопотреблением MSP430. Сигналы ШИМ обычно используются для получения аналоговых сигналов во встраиваемых применениях. Ниже будет рассмотрен способ получения, как переменных, так и постоянных напряжений при помощи выходов ШИМ.

В данном примере использован таймер Timer_B микроконтроллера MSP430F149, аналогичным образом можно также использовать таймер Timer_A.

1. Введение

Во многих встраиваемых применениях на базе микроконтроллеров требуется генерация аналоговых сигналов. В некоторых случаях для этих целей используются встроенные либо внешние цифро-аналоговые преобразователи (ЦАП).

Тем не менее, сигналы ШИМ также могут быть использованы для получения требуемых аналоговых сигналов. С их помощью можно синтезировать аналоговые сигналы, как с переменным, так и с постоянным уровнем. В этой статье обсуждается использование ШИМ-таймера в качестве ЦАП и приводится пример одновременной генерации синусоидального сигнала, пилообразного сигнала и сигнала с постоянным уровнем, а также суммирование синусоидального сигнала с постоянным уровнем для создания смещения. В данном примере использован таймер Timer_B, аналогичным образом можно также использовать таймер Timer_A.

2. Теория функционирования

Сигналы ШИМ представляют собой цифровые сигналы с постоянной частотой и меняющимся коэффициентом заполнения. Пример таких сигналов приведен на рис. 1. Если коэффициент заполнения ШИМ-сигнала изменяется со временем, то, отфильтровав его, получим на выходе аналоговый сигнал. Блок-схема ЦАП на базе ШИМ с использованием такого способа приведена ниже на рис. 2. В примере программы в конце этой статьи таймер Timer_B микроконтроллера MSP430F149 используется для одновременной генерации

синусоидального и пилообразного сигналов с разными частотами и сигнала с постоянным уровнем. ЦАПы, основанные на ШИМ также позволяют синтезировать другие сигналы. Некоторые аудиопроцессоры фирмы Texas Instruments Inc. также используют сигналы ШИМ для генерации речевых сигналов.

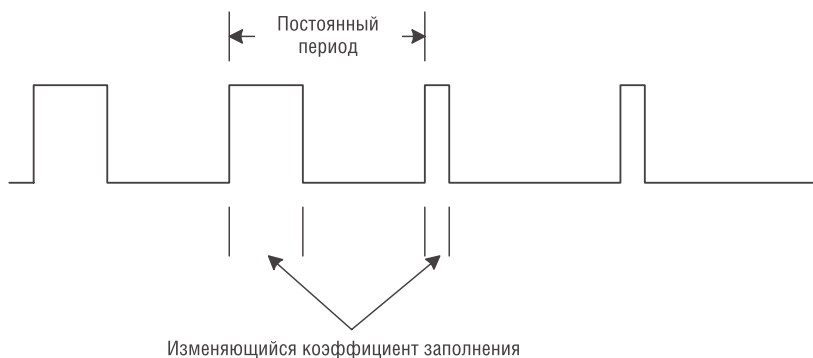


Рис. 1. ШИМ-сигнал

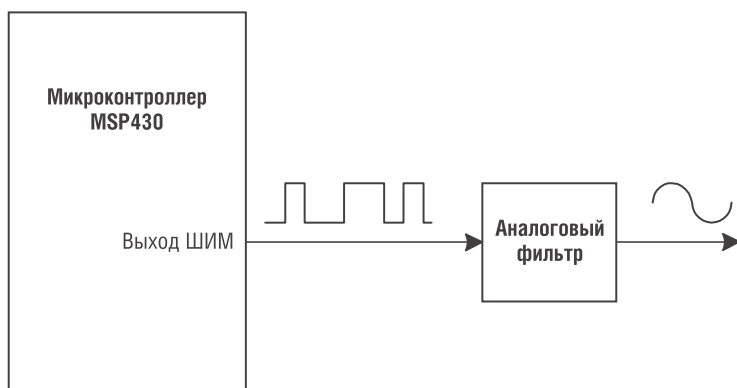


Рис. 2. Блок-диаграмма ЦАП на базе ШИМ

2.1. Разрешение

Разрешение ЦАП на базе ШИМ, построенного на таймере Timer_B эквивалентно длине счётчика, которая, как правило, представлена значением в ре-

гистре CCR0. МЗР ЦАП на базе ШИМ представляет собой единичный отсчёт, а разрешение соответствует общему числу отсчётов:

$$R_{counts} = L_{counts}$$

Где R_{counts} – разрешение (в отсчётах),

L_{counts} – длина счётчика (в отсчётах)

Например, в этой статье рассматривается 8-битный ЦАП, таким образом, длина счётчика 8 бит или 256 отсчётов.

В общем виде, разрешение ЦАП на базе ШИМ, построенного на таймере и фильтре эквивалентно разрешению ШИМ-сигнала, используемого ЦАП-ом. Таким образом, разрешение ШИМ-сигнала зависит от длины счётчика и наименьшего возможного изменения коэффициента заполнения в ШИМ-счётчике. Математически это можно записать как:

$$R_{COUNTS} = \frac{L}{C}$$

Где R_{counts} – разрешение (в отсчётах),

L – длина счётчика (в отсчётах),

C – наименьшее возможное изменение коэффициента заполнения (в отсчётах)

Разрешение, выраженное в количестве бит, будет выглядеть следующим образом:

$$R_{BITS} = \log_2(R_{COUNTS}) = \log_2\left(\frac{L}{C}\right) = \frac{\ln\left(\frac{L}{C}\right)}{\ln(2)}$$

или

$$R_{BITS} = \log_2(R_{COUNTS}) = \frac{\ln(R_{COUNTS})}{\ln(2)}$$

Например, если счётчик ШИМ имеет длину 512 отсчётов, а коэффициент заполнения может изменяться как минимум на 2 отсчёта, результирующее разрешение в отсчётах ЦАП на базе ШИМ будет составлять:

$$R_{COUNTS} = \frac{L}{C} = \frac{512}{2} = 256$$

Разрешение в битах составляет:

$$R_{BITS} = \log_2(256) = \frac{\ln(256)}{\ln(2)} = 8 \text{ bits}$$

2.2. Частота

Частота, требуемая для формирования выходного сигнала ШИМ эквивалентна частоте обновления ЦАП, а каждое изменение коэффициента заполнения ШИМ эквивалентно одному отсчёту ЦАП. Требуемая для ШИМ-таймера частота зависит от требуемой частоты ШИМ-сигнала и необходимого разрешения:

$$F_{\text{clock}} = F_{\text{PWM}} \times 2^n$$

где F_{clock} – требуемая частота тактирования ШИМ-таймера,

F_{PWM} – частота ШИМ-сигнала, соответствующая частоте обновления ЦАП,

n – требуемое разрешение ЦАП в битах

В этой статье описан способ построения 8-битного ЦАП на базе ШИМ и одновременная генерация с его помощью синусоидального сигнала частотой 250Гц и пилообразного сигнала частотой 125Гц. Требуемая частота для данного примера – 8кГц (32 отсчёта на каждый период синуса (передискретизация 16х), и 64 отсчёта на каждый период пила (передискретизация 32х). Это соответствует требуемой частоте сигнала ШИМ в 8кГц и требуемой частоте тактирования ШИМ в 2.048МГц.

Как правило, наилучшие результаты получаются в том случае, когда частота ШИМ намного превышает требуемую частоту синусоидального сигнала, либо требуемую полосу синтезируемых частот. Это обусловлено тем, что чем больше частота ШИМ, тем меньше порядок требуемого фильтра и проще реализация подходящего фильтра.

2.3. Используемые ресурсы MSP430

Пример программы, приведенный в конце этой статьи, демонстрирует одновременный синтез синусоидального сигнала частотой 250Гц, пилообразного сигнала частотой 125Гц и постоянное значение с уровнем 2/3 V_{cc} , используя для этого таймер Timer_B и внешние фильтры. Аналогичным образом можно также использовать таймер Timer_A.

Таймер Timer_B используется в 16-битном режиме и сконфигурирован для работы в режиме инкремента, но при этом счётчик считает до значения, записанного в регистр захвата-сравнения 0 (CCR0), после чего сбрасывается в 0. В регистр CCR0 загружается значение 255, таким образом, эффективная разрядность счётчика составляет 8 бит. Регистр CCR1 и выход TB1 используются для формирования синусоиды. CCR2 и TB2 используются для пилообразного сигнала, а CCR3 и TB3 – для постоянного значения. Для всех выходов выбран выходной режим 7 «сброс/установка» (reset/set mode).

В этом режиме каждый выход сбрасывается в «0» в тот момент, когда значение счётчика достигает значения в соответствующем регистре CCRx и

устанавливается в «1», когда значение счётчика равно значению в регистре CCR0. Это обеспечивает на соответствующих выходах импульсы положительной полярности с длительностью, эквивалентной значению в соответствующих регистрах CCRx. В качестве источника тактирования таймера Timer_B выбрана частота SMCLK.

Также в примере использованы:

- Кварцевый резонатор частотой 32768 Гц
- Встроенный генератор с цифровым управлением (DCO), работающий на частоте 2.048 МГц
- Частоты SMCLK и MCLK, равные 2.048 МГц
- Timer_A для калибровки генератора DCO
- Два регистра ЦПУ

2.4. Принципиальная схема и осциллограммы

Полная принципиальная схема устройства приведена на рис. 3. Генерируемые ей переменные сигналы показаны на рис. 4. Сигнал с постоянным уровнем показан на рис. 5 совместно с сигналом ШИМ.

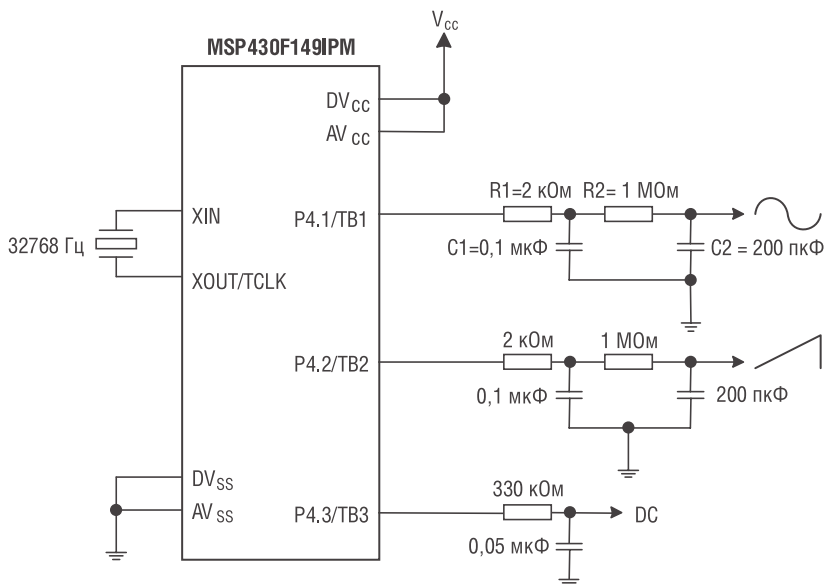


Рис. 3. Принципиальная схема

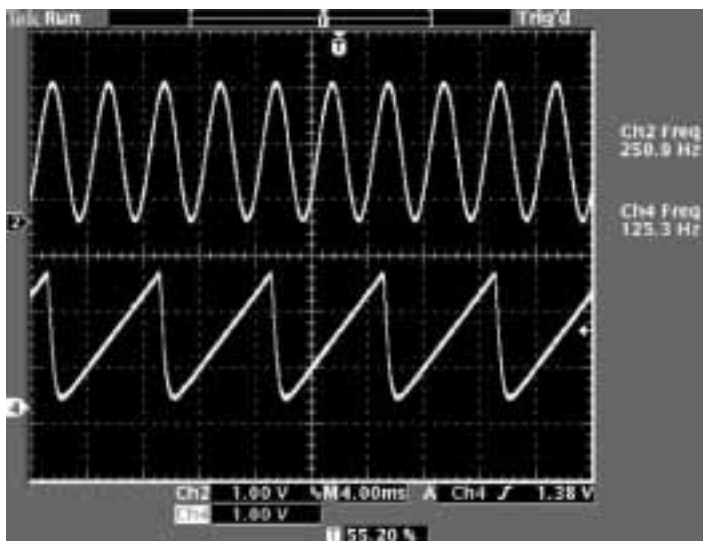


Рис. 4. Осциллограммы сигналов

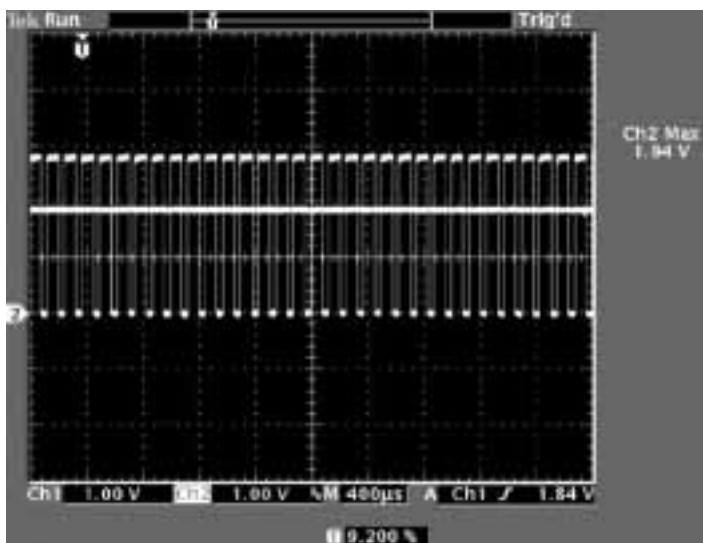


Рис. 5. Сигнал ШИМ и постоянный сигнал

Синусоидальный сигнал, формируемый в этом примере, использует 32 отсчёта на период. Значения отсчётов хранятся в таблице в начале программы. Для определения текущей ячейки используется указатель, в конце каждого периода ШИМ в регистр сравнения-захвата ШИМ-таймера записывается новое значение.

Для генерации пилообразного сигнала в этом примере табличные значения не используются. «Пила» формируется обычным инкрементом коэффициента заполнения на каждом периоде ШИМ до тех пор, пока не будет достигнуто максимальное значение. После этого инкремент начинается снова с минимального значения. Такое последовательное увеличение коэффициента заполнения ШИМ-сигнала после его фильтрации формирует на выходе пилообразный сигнал.

Сигнал с постоянным уровнем в данном примере формируется записью неизменного значения коэффициента заполнения ШИМ-сигнала. Постоянный уровень прямо пропорционален значению коэффициента заполнения ШИМ-сигнала. Таким образом, раз значение коэффициента заполнения ШИМ-сигнала на выходе ТВЗ не изменяется, после фильтрации RC-цепью получим на выходе постоянное значение.

2.5. Требования к фильтрам

Фильтры, используемые для восстановления сигналов, показаны выше на рис. 3. Фильтры для переменных сигналов представляют собой простые двух-полюсные каскадированные RC-цепочки. Этот выбор обусловлен простотой реализации и нежелательностью использования активных элементов в приборах с низким энергопотреблением. Однако, при этом требуется более высокая частота дискретизации, чем при использовании фильтров более высокого порядка. Для фильтров, показанных на рис. 3 рекомендуется передискретизация для ЦАП не менее, чем 16х.

Частота среза фильтра вычисляется следующим образом:

$$F_c = \frac{1}{2\pi RC}$$

Где $R1C1 = R2C2 = RC$

Наилучшие характеристики фильтра получаются при $R2 \gg R1$. Кроме этого, выбор частоты среза, слишком близкой к границе полосы сигнала, приведёт к существенному ослаблению. Для уменьшения такого ослабления, вносимого фильтром, следует выбирать частоту среза выше границы полосы сигнала, но намного ниже частоты ШИМ-сигнала.

В отличие от фильтров переменных сигналов, фильтр для постоянного сигнала используется только для хранения заряда. Поэтому в этом случае применена простая однополюсная RC-цепь.

2.6. Суммирование постоянного и переменного сигналов

Сигналы, синтезируемые ЦАП-ом на базе ШИМ, можно сложить. Схема на рис. 6 демонстрирует формирование смещённого синусоидального сигнала добавлением постоянного смещения в $2/3V_{cc}$ к синусоидальному сигналу. Кроме этого, синусоидальный сигнал ослаблен примерно на $2/3$, чтобы обеспечить возможность его смещения по постоянному напряжению без перегрузки суммирующего усилителя. В результате получаем синусоидальный сигнал амплитудой $1V_{pp}$ с постоянным смещением примерно в 2 В, как показано на рис. 7. На заднем плане также показан ШИМ-сигнал.

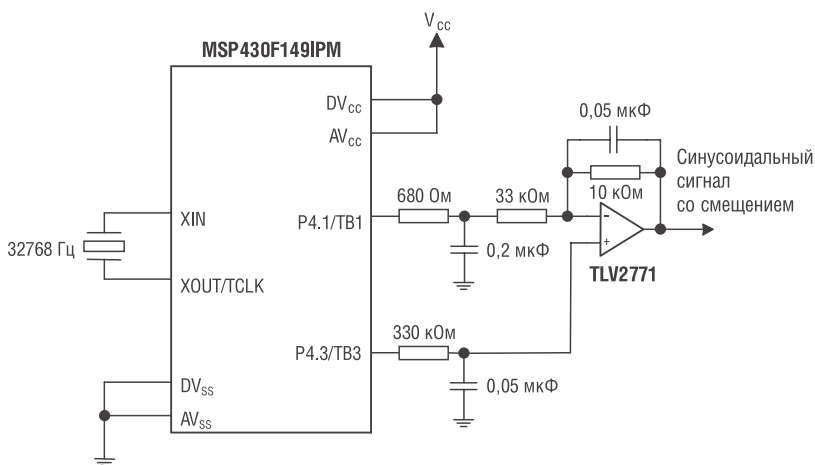


Рис. 6. Суммирующая схема

Сложение сигналов таким образом, как описано в примере выше, позволяет легко регулировать постоянную и переменную составляющую по отдельности через ШИМ-выходы. Например, синусоидальная составляющая может оставаться постоянной, а постоянная составляющая при этом изменяться в большую или меньшую сторону простым изменением коэффициента заполнения ШИМ-сигнала, отвечающего за постоянный сигнал. Обратите внимание на то, что фильтр, используемый для синусоидального сигнала, немного изменён. Так как в схему был добавлен суммирующий усилитель для добавления смеще-

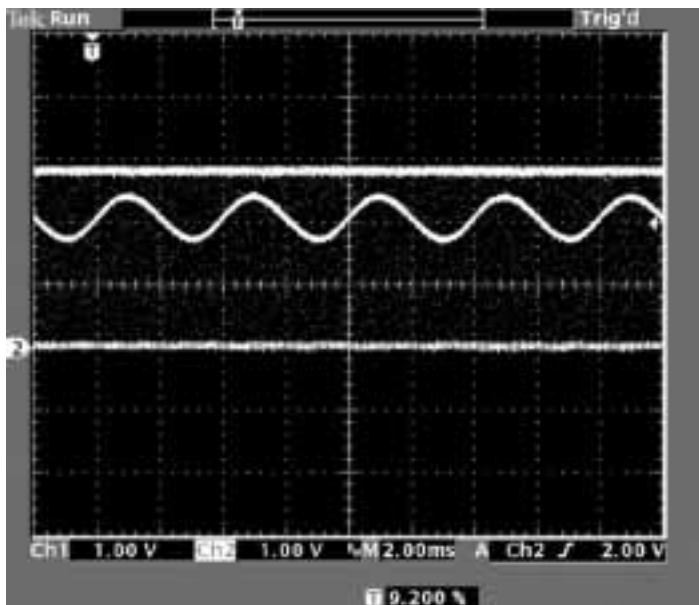


Рис. 7. Синусоидальный сигнал со смещением

ния к синусоидальному сигналу, часть элементов фильтра была совмещена со схемой усилителя. Для получения аналогичных результатов также можно использовать другие схемы фильтров и активных сумматоров.

3. Описание программы

Полный листинг программы приведен в Приложении А. Процесс выполнения программы проиллюстрирован на рис. 8.

После сброса (reset) производится останов сторожевого таймера watchdog, конфигурация выходных портов и системы тактирования. Далее формируется программная задержка для стабилизации кварцевого резонатора 32768Гц, что необходимо для калибровки генератора DCO. Далее вызывается подпрограмма стабилизации генератора DCO. После её завершения, таймер Timer_B, регистры CCR1 и CCR2 конфигурируются для генерации сигналов ШИМ и осуществляется запуск таймера.

В завершение, MSP430 переводится в режим пониженного энергопотребления 0 (LPM0) для сохранения энергии. ЦПУ «просыпается» для обработки каждого прерывания CCIFG0, после чего возвращается в режим LPM0.

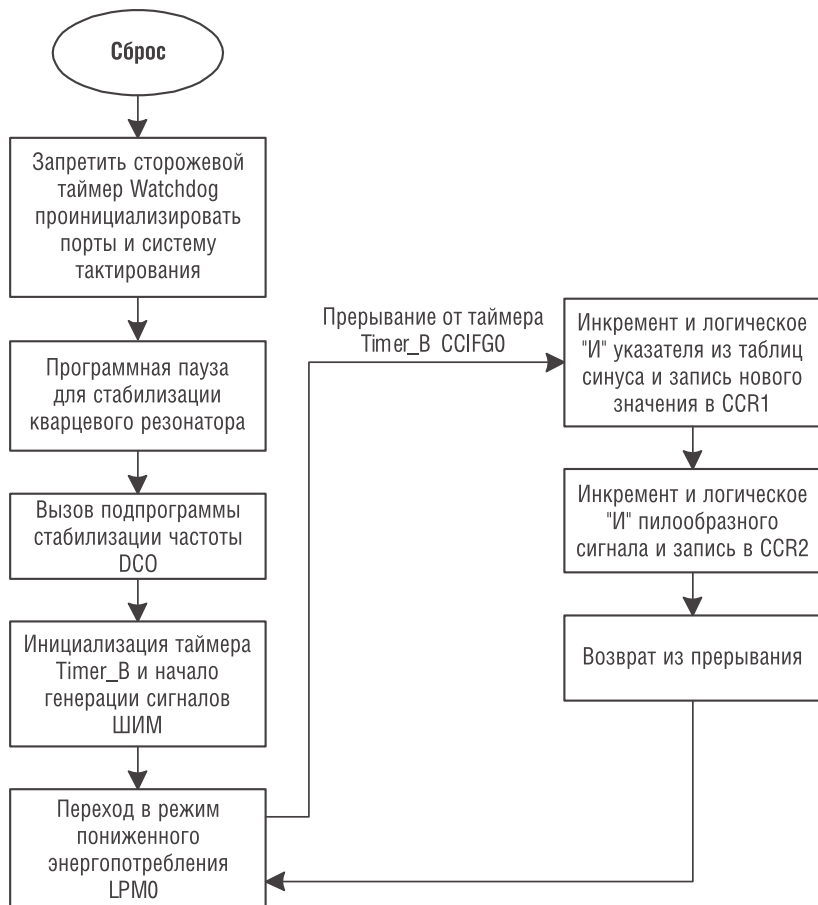


Рис. 8. Процесс выполнения программы

3.1. Калибровка генератора DCO

Генератор DCO микроконтроллера MSP430 представляет собой генератор RC типа и обладает схожими с RC-генератором характеристиками, в частности такими, как неточная установка частоты и её уход. Тем не менее, цифровое управление этим генератором позволяет настроить его на стабильную частоту, используя источник тактирования со стабильным известным значением, такой, как кварцевый резонатор 32768Гц. Некоторые микроконтроллеры семейства

MSP430 имеют встроенный генератор с автоподстройкой частоты Frequency Lock Loop (FLL), позволяющий осуществить это автоматически, однако, устройства подсемейства MSP430x1xx такого модуля не имеют. За более подробными сведениями о генераторе DCO и системе тактирования MSP430 обратитесь к разделу «Система тактирования» (Clock system) соответствующего руководства пользователя.

Ввиду того, что микроконтроллеры MSP430x1xx не имеют встроенного FLL, генератор DCO должен быть программно откалиброван для того, чтобы источник тактирования имел стабильную и известную частоту. Программное обеспечение, приведенное в этом примере, реализует стабилизатор частоты FLL для калибровки генератора DCO. Осуществляется это путём сравнения числа тактов генератора DCO за один период генератора 32768Гц. Точнее говоря, в данном примере частота 32768Гц (ACLK) делится на 4, поэтому программный FLL подсчитывает количество тактов генератора DCO за 1/4 периода ACLK. После этого, подсчитанное число сравнивается с номинальным значением и значение DCO увеличивается либо уменьшается на одну ступень в зависимости от результата счёта, после чего счёт повторяется снова. Процедура повторяется до тех пор, пока не будет достигнуто требуемое количество тактов.

По выполнении этого условия частота генератора DCO является откалиброванной к требуемому значению.

В этом примере программный FLL выполняется только один раз, т.к. устройство не подвергается атмосферному воздействию, колебаниям питающего напряжения Vcc или длительному использованию. Однако, в реальных условиях эксплуатации, подпрограмма FLL должна выполняться периодически для отстройки генератора DCO от вышеперечисленных воздействий и традиционного ухода частоты генераторов RC-типа.

За дополнительной информацией по программной реализации FLL обращайтесь к примеру применения «Управление частотой генератора с цифровым управлением (DCO) микроконтроллеров MSP430x11x» («Controlling the DCO Frequency of the MSP430x11x» literature number SLAA074).

Альтернативой программной калибровке генератора DCO может служить использование внешних кварцевых резонаторов с требуемой для ШИМ-таймера частотой. Это применимо для микроконтроллеров семейств 13x и 14x благодаря наличию высокочастотного кварцевого XT-генератора. При использовании такого метода, SMCLK должна тактироваться от XT-генератора, а не от генератора DCO. См. главу 7 руководства пользователя MSP430x1xx Family User's Guide, literature number SLAU049, по вопросам использования системы тактирования микроконтроллеров семейства 1xx. Также см. документацию на конкретные устройства по вопросам максимальных рабочих частот.

Ссылки:

1. MSP430x13x/14x data sheet (SLAS272)
2. Руководство пользователя MSP430x1xx (MSP430x1xx Family User's Guide (SLAU049))
3. MSP430x3xx Family User's Guide (SLAU012)
4. Controlling the DCO of the MSP430x11x (SLAA074)

Приложение: Листинг программы

```

;*****
; Программа реализации ЦАП на базе ШИМ
; Генерирует синусоиду частотой 250Гц с использованием
; ШИМ на базе таймера Timer_B.
; Описание: программа демонстрирует использование
; ШИМ-таймера совместно с внешними фильтрами для
; реализации ЦАП. Показан способ формирования
; синусоиды частотой 250Гц, пилообразного сигнала
; частотой 125 Гц и сигнала с постоянным уровнем
; с использованием таймера Timer_B.
; Возможно аналогичное использование таймера Timer_A.
; В таблице sine table holds the хранятся отсчёты
; значений синусоиды. Для формирования «пилы»
; используется простой инкремент значения ШИМ.
; Постоянный уровень формируется зарядом конденсатора
; RC-цепи при помощи выхода ШИМ. Значение напряжения
; при этом прямо пропорционально коэффициенту
; заполнения сигнала ШИМ. После инициализации,
; ЦПУ переводится в режим пониженного
; энергопотребления LPM0. Он остаётся в этом режиме
; до прихода прерывания CCIFG0 от таймера Timer_B.
; В подпрограмме обработки прерывания таймера
; Timer_B ISR в регистр CCR1 загружается следующее
; значение для синусоиды, значение «пилы»
; инкрементируется и загружается в регистр CCR2.
; По возвращении из прерывания ЦПУ снова переходит
; в режим пониженного энергопотребления LPM0.
;*****
#include <MSP430X14x.H> ;Включить в проект файл
                        ;со стандартными
                        ;определениями
Delta      EQU      250 ;Delta = требуемая частота
                        ;DCO/8192

```

```

; Требуемая частота генератора DCO = 2.048МГц
; Это значение используется в программном
; стабилизаторе FLL для калибровки частоты генератора
; DCO с использованием кварцевого резонатора
; с частотой 32768Гц в качестве опорного. Более
; подробная информация по стабилизации частоты
; генератора DCO и программе стабилизатора FLL
; см. пример применения под названием «Управление
; частотой генератора с цифровым управлением (DCO)
; микроконтроллеров MSP430x11x»
; ( «Controlling the DCO frequency of the
; MSP430x11x» Literature number SLAA074)

;*****
RSEG CODE
;*****
Sine_Tab      DW      255      ;Таблица синуса, отсчёты
DW      254      ;десятичные значения
DW      246      ;загружаемые в регистр TBCCR1
;для изменения
DW      234      ;коэффициента заполнения ШИМ.
DW      219      ;Вместо байт требуется
;использовать слова
DW      199      ;т.к. в регистры TB
DW      177      ;требуется записывать слово
;целиком
DW      153      ;Значение '0' использовать
;нельзя
DW      128      ;иначе таймер будет глючить.
DW      103
DW      79
DW      57
DW      37
DW      22
DW      10
DW      2
DW      1
DW      2
DW      10
DW      22
DW      37
DW      57
DW      79

```

DW 103
 DW 128
 DW 153
 DW 177
 DW 199
 DW 219
 DW 234
 DW 246
 DW 255;

```
***** Здесь начинается программа *****
RESET      mov    #02FEh,SP      ; Инициализация
                                   ; указателя стека
StopWDT     mov    #WDTPW+WDTHOLD,&WDTCTL ; Остано-
                                   ; вить сторожевой
                                   ; таймер WDT
SetupP4     bis.b  #00Eh,&P4SEL ; Выбрать TB1, TB2
                                   ; и TB3 вместо портов
                                   bis.b  #00Eh,&P4DIR ; P4.x, и назначить их
                                   ; выходами
SetupBC     mov.b  #0A6h,&BCSCTL1 ; ACLK делится на 4.
                                   ; RSEL=6, MCLK и SMCLK
                                   ; не делятся,
                                   ; источником MCLK
                                   ; и SMCLK является DCO
                                   ; генератор XT2
                                   ; выключен.
                                   ; ПРИМЕЧАНИЕ: для
                                   ; определения значения
                                   ; Rsel для требуемой
                                   ; частоты DCO,
                                   ; обратитесь к таблице
                                   ; DCO в документации
                                   call    #Delay ; Пауза для стабилиза-
                                   ; ции кварца.
                                   ; Необходима из-за
                                   ; того, что
                                   ; кварцевый резонатор
                                   ; 32768Гц используется
                                   ; в качестве опорного
                                   ; для стабилизации
                                   ; частоты
                                   ; генератора DCO.
```

```

; Поэтому резонатор
; на 32768
; должен быть
; стабильным.
call  #SW_FLL      ; Вызов подпрограммы
                   ; стабилизации частоты
                   ; генератора DCO.
call  #TB_SETUP    ; Инициализация
                   ; таймера Timer_B для
                   ; генерации сигналов
                   ; ШИМ
clr   R15           ; R15 и R14 использу-
                   ; ются как указатели
clr   R14           ; на таблицу синусов
                   ; и для хранения
                   ; значения «пилы»
                   ; после стабилизации
                   ; DCO
eint                     ; Разрешение
                   ; прерываний
bis   #LPM0,SR        ; «усыпить» ЦПУ.
                   ; Завершение программы
                   ; остаётся обработчик
                   ; прерывания SCIFG0
                   ; где обновляются
                   ; значения ШИМ

;*****
Delay                                ; Программная пауза
                                   ; для стабилизации
                                   ; кварца
;*****

mov   #0004h,R15
L1    mov   #0FFFFh,R14 ; ~ 1 секунда .
L2    dec   R14          ;
      jnz   L2           ;
      dec   R15          ;
      jnz   L1           ;
      ret                                ;
;*****
;*****
SW_FLL ; Подпрограмма стабилизации частоты DCO.
       ; Кварцевый генератор 32768Гц используется

```

```

; в качестве опорной частоты для стабилизации
; и подстройки генератора DCO до требуемой частоты
; 2.048 МГц. Это требуется в устройствах где нужна
; фиксированная частота DCO и для тех моделей MSP430
; где нет аппаратного модуля FLL. См. руководства
; пользователя MSP430x3xx и MSP430x1xx Family
; User's Guides (literature number SLAU012 и
; SLAU049 соответственно) по вопросам, касающимся
; системы тактирования MSP430
;
; Процедура основана на подсчёте тактов DCO
; за период ACLK (точнее, за 1/4 периода ACLK
; т.к. частота ACLK поделена на 4). Таймер Timer_A
; используется для подсчёта тактов DCO
; Полученное значение сравнивается с номинальным
; (Delta). Если число слишком большое, DCO декременти-
; руется. Если число слишком маленькое, DCO инкремен-
; тируется.
; Далее сравнение повторяется.
;
; Процесс продолжается до тех пор, пока значения не
; сравняются. После этого DCO генерирует на требуемой
; частоте
; См. пример применения «Управление частотой
; генератора с цифровым управлением (DCO)
; микроконтроллеров MSP430x11x»
; ( «Controlling the DCO frequency of the
; MSP430x11x» Literature number SLAA074) по
; вопросам, связанным с управлением генератором DCO.
; Эта процедура вызывается только один раз
; в качестве примера, в реальных условиях её требуется
; выполнять периодически, чтобы быть уверенным,
; что генератор DCO остаётся откалиброванным.
;*****
                clr     R15                ;
Setup_TA        mov     #TASSEL1+TACLR,&TACTL
                                   ; Источник
                                   ; тактирования
                                   ; TA - SMCLK
Setup_CC2        mov     #CCIS0+CM0+CAP,&CCTL2
                                   ; Определить
                                   ; CCR2,CAP,ACLK

```

```

bis    #MC1,&TACTL ; Запустить
                        ; таймер timer_A
                        ; в непрерывном
                        ; режиме
Test_DCO bit    #CCIFG,&CCTL2; Проверка флага
                        ; захвата
jz     Test_DCO      ;
bic    #CCIFG,&CCTL2; Очистка флага
                        ; захвата
                        ;
AdjDCO  mov    &CCR2,R14 ; В R14 число
                        ; тактов SMCLK
sub     R15,R14      ; в R14 разница
mov     &CCR2,R15    ; В R15 число
                        ; тактов SMCLK
cmp     #Delta,R14   ; Delta =
                        ; SMCLK/(32768/4)
jlo     IncDCO       ;
jeq     DoneFLL      ;
DecDCO  dec.b  &DCOCTL ;
jmp     Test_DCO     ;
IncDCO  inc.b  &DCOCTL ;
jmp     Test_DCO     ;
DoneFLL clr    &CCTL2 ; очистить CCR2
        clr    &TACTL ; остановить
                        ; таймер timer_A
        ret     ; возврат из
                        ; подпрограммы
;*****
TB_SETUP ; Подпрограмма инициализации таймера
        ; Timer_B для генерации ШИМ
;*****
mov     #TBSSEL1+TBCLR,&TBCTL
                        ; SMCLK – источ-
                        ; ник тактирова-
                        ; ния TB.
mov     #CCIE,&TBCCCTL0 ;Перевести CCR0
                        ; в режим сравнения,
                        ; разрешить прерывания
                        ; от него
mov     #0FFh,&TBCCR0; Записать 255d
                        ; в CCR0. Это

```

```

; соответствует
; периоду
; ШИМ-выходов
; в 256 отсчётов
; (8 бит). т.е.
; получаем
; 8-битный ЦАП.
mov    #02E0h,&TBCCTL1    ; Перевести CCRx
; в режим сравнения,
mov    #02E0h,&TBCCTL2    ;запретить
; прерывания, режим
; выхода '7', т.е.
mov    #02E0h,&TBCCTL3; сброс/установка.
; Сигнал EQU0
; устанавливает выход
; в «1», а EQU1 в «0».
; Установим условия
; сравнения для
; регистра как его
; равенство нулю
mov    #Sine_Tab,&TBCCR1
; Загрузить первый
; отсчёт в CCR1
mov    #01h,R14            ;Загрузить начальное
; значение «пилы»
; в R14.
mov    #0AAh,&TBCCR3 ;И значение для
; постоянного уровня
; которое соответству-
;ет ~ 2/3 Vcc т.к.
; #0AAh это примерно
; 2/3 от #0FFh.
bis    #MC0,&TBCTL ;Запустить таймер
; timer_B в режиме
; инкремента

ret

;*****
TB_ISR;Подпрограмма обработки прерываний таймера
;Timer_B: изменяет значения в регистрах
; CCR1 и CCR2 для изменения коэффициента
; заполнения ШИМ для синусоиды и «пилы».

```

```

; Значение в CCR3 не изменяется для
; формирования постоянного уровня
;*****
incd    R15          ; Инкремент указателя
                        ; в R15 to
                        ; для следующего
                        ; значения в таблице
                        ; синуса
                        ; Инкремент двойной,
                        ; т.к. значения в
                        ; таблице двухбайтовые
and     #03Fh,R15    ; Операция лог. «И»
                        ; с числом 03Fh даёт
                        ; счётчик по модулю 32
                        ; для указания на
                        ; значения в таблице
mov     Sine_Tab(R15),&TBCCR1
                        ; Загрузить следующее
                        ; значение синуса
                        ; в CCR1
add     #04h,R14     ; Инкрементировать
                        ; значение «пилы»
                        ; Изменение шага в R14
                        ; приведёт к изменению
                        ; частоты «пилы»
and     #0FFh,R14    ; Исключим лишние биты
mov     R14,&TBCCR2   ; Загрузим новое
                        ; значение пилы в CCR2
reti    ; возврат, прерывания
                        ; разрешены
;*****
COMMON INTVEC      ; Вектора прерываний MSP430x14x
;*****
ORG             TIMERB0_VECTOR
DW             TB_ISR          ; Прерывание от CCIFG0
ORG             RESET_VECTOR
DW             RESET           ; POR, внеш. Reset,
                                ; Watchdog
END

```


II. Использование внутренних функций, периферийных блоков микроконтроллеров

Глава 4.

*Техника самопрограммирования
флэш-памяти MSP430*

Техника самопрограммирования флэш-памяти MSP430

Anton Muehlhofer [slaa103.pdf (Англ.)]

Самопрограммируемость Flash становится все более и более важной функцией. Однако при выполнении программирования/стирания Flash памяти программы микроконтроллер не может одновременно выполнять прикладную программу, расположенную в том же массиве памяти. Таким образом, микроконтроллер только с одним встроенным массивом Flash памяти не может одновременно выполнять программу и изменять ее. Проблема может быть решена двумя способами:

1) Команды стирания/программирования Flash памяти перед выполнением должны копироваться в ОЗУ.

2) При выполнении стирания/программирования вычислительное ядро переходит в режим Idle. В данном примере применения описывается, как выполнить операции стирания/программирования Flash памяти, а также приводится необходимое программное обеспечение.

1. Введение

Самопрограммируемость Flash становится все более и более важной функцией. Однако при выполнении программирования/стирания Flash памяти программы микроконтроллер не может одновременно выполнять прикладную программу, расположенную в том же массиве памяти. Таким образом, микроконтроллер только с одним встроенным массивом Flash памяти не может одновременно выполнять программу и изменять ее.

Есть два подхода к решению этой задачи. Первый состоит в том, что команды стирания/программирования Flash памяти перед выполнением копируются в ОЗУ. Второй – в том, что при выполнении стирания/программирования вычислительное ядро переходит в режим Idle.

Все микроконтроллеры семейства MSP430 компании Texas Instruments имеют встроенный контроллер Flash памяти, который позволяет выполнять прикладную программу и изменять содержимое одного и того же модуля Flash памяти.

Этот пример применения описывает способ, который позволяет стирать/программировать Flash память, а также приводятся тексты программных модулей, необходимых для выполнения этих операций.

2. Состав Flash-модуля

Состав Flash модуль MSP430:

- Управляющая логика: автомат состояния и задающий генератор управляют стиранием/записью Flash памяти
- Логика защиты Flash: защита от некорректных действий при стирании/записи
- Программируемый источник напряжения: встроенный преобразователь напряжения, который вырабатывающий напряжение, необходимое для программирования Flash памяти

Три 16-разрядных управляющих регистра FCTL1, FCTL2 и FCTL3 полностью управляют Flash модулем (см. рис. 1). Более детальное описание этих регист-

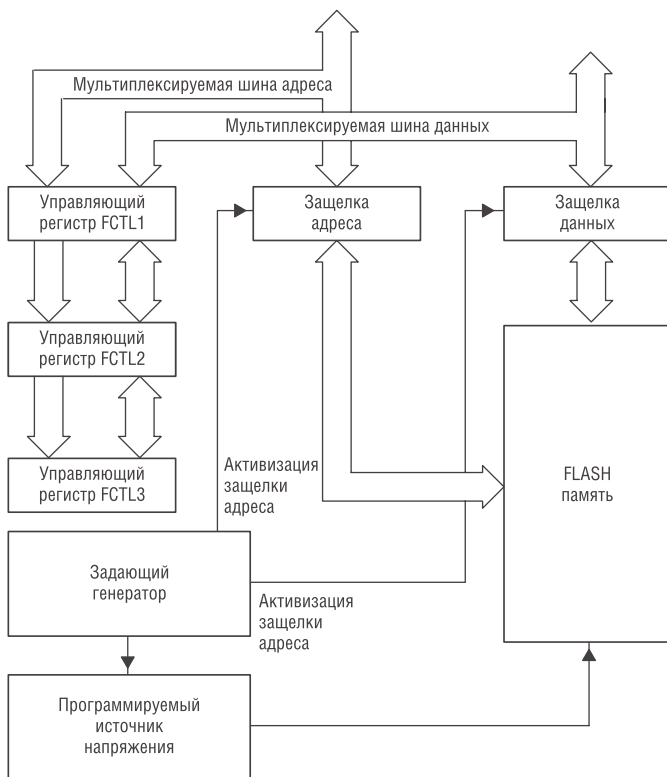


Рис. 1. Состав Flash модуля

ров можно найти в руководстве пользователя по микроконтроллерам семейства MSP430 (документ SLAU049).

3. Стирание и программирование Flash-модуля

Обычно вычислительное ядро считывает из Flash памяти данные или код прикладной программы. Однако, иногда данные во Flash памяти необходимо изменить при выполнении программы. В течение этой операции стирания-записи (то есть перепрограммирования) Flash памяти управление осуществляется от задающего генератора, имеющегося в модуле Flash. В это время вычислительное ядро не может получить доступа к Flash памяти и, следовательно, программа должна находиться в другом месте, например в ОЗУ, или вычислительное ядро должно перейти в Idle режим. После завершения программирования Flash памяти вычислительное ядро вновь берет управление над ней на себя. Оба эти метода имеют свои преимущества.

В настоящее время микроконтроллеры семейства MSP430 имеют один модуль Flash для хранения данных и прикладной программы. Это означает, что при программировании Flash памяти векторы прерываний недоступны и никакие запросы на прерывание не могут быть обработаны. Поэтому, при модификации Flash памяти все источники прерывания, вплоть до сторожевого таймера, должны быть отключены. Это можно сделать при помощи программы, приведенной на рисунке 2.

```
dint                                ; блокировка всех
                                   ; маскируемых
                                   ; прерываний
clr.b      &IE1                    ; блокировка NMI, ACCV
                                   ; и OF прерываний
mov        #5A80h, &WDTCTL        ; блокировка
                                   ; сторожевого таймера
```

Рис. 2. Программа блокировки всех прерываний и сторожевого таймера

3.1. Программирование Flash памяти с копированием прикладной программы в ОЗУ

При программировании Flash памяти ОЗУ – единственная память, из которой вычислительное ядро может выполнять прикладную программу. Пример программы, приведенный на рисунке 3, копирует функцию программирования Flash памяти в стек для выполнения прикладной программы в из ОЗУ. Когда Flash память становится снова доступной, счетчик команд указывает на Flash память и указатель стека восстанавливается. Бит Busy (бит 0 в FCTL3) указывает

доступность Flash памяти. В приложении А приведена прикладная программа flash_ram.s43, которая может выполнять три функции:

- стирание одного сегмента
- программирование одного байта
- программирование одного слова

Выполнение программы из ОЗУ позволяет вычислительному ядру работать в процессе изменения Flash памяти. Это позволяет MSP430 выполнять программирование Flash памяти и, например, принимать данные через UART. Конечно, в этой ситуации обнаружение принятых данных может быть реализовано только путем опроса флага принятия данных UART.

```

MODULE flash_ww
PUBLIC Flash_ww
RSEG CODE
#define _CPU_ 5 /* 5=MSP430F1121, 6=MSP430F149 device*/
#include <Std_def.s43>
;*****
; Запись во Flash память слова
;*****
; Запись 1 слова во Flash путем копирования небольшой
; функции (18 байт)
; в ОЗУ при помощи динамической памяти, расположенной
; в стеке.
; Определение функции:
void Flash_ww(int*Data_ptr, int word)
; скрэтч-регистр: R12 = int *Data_ptr
;                R14 - int слово
;                R13, R15 - общего назначения
; Предположение: При записи Flash не занята
(Flash_ww_start)
;*****
Flash_ww
dint                                ;защита стека от повреждения
clr.b &IE1                          ;блокировка NMI, ACCV и OF
                                   ;прерываний
mov    #5A80h, &WDTCTL;блокировка сторожевого
                                   ;таймера
mov    #Flash_ww_end, R13 ;определение конечного
                                   ;адреса и длины
mov    #Flash_ww_length, R15 ;функция, которая
                                   ;будет скопирована в ОЗУ

```

```

    mov    #0A500h, &FCTL3      ; LOCK = 0
copy    push @R13                ; копирование функции
                                   ; в ОЗУ

    decd   R13
    dec    R15
    jnz    copy
    mov    SP, R15
    mov    #0A540h, &FCTL1      ; WRT = 1
    call   R15                  ; вызов из ОЗУ функции
                                   ; записи Flash

    mov    #0A500h, &FCTL1      ; WRT = 0
    mov    #0A510h, &FCTL3      ; LOCK = 1
    add    #2*Flash_ww_length, SP ; обслуживание
                                   ; стека

    ret;

;*****
;***** Функция записи Flash памяти *****
Flash_ww_start
    mov    R14, 0 (R12)         ; запись байта во
                                   ; Flash память

wait_bf бит #1, &FCTL3         ; ожидание флага
                                   ; занятости

    jnz    wait_bf
Flash_ww_end
    ret

                                   ; вычисление номера
                                   ; слова Flash памяти,
                                   ; которое необходимо
                                   ; записать в ОЗУ

Flash_ww_length EQU (Flash_ww_end-Flash_ww_
start+2)/2
ENDMOD

```

Рис. 3. Программа копирования кода прикладной программы из Flash памяти в стек

3.2. Прямое перепрограммирование Flash памяти

Flash модуль MSP430 имеет уникальную функцию – самопрограммируемость. Эта функция позволяет перепрограммировать Flash память без копирования программы в другую память. Когда в течение перепрограммирования Flash памяти вычислительное ядро выбирает команды из нее, Flash модуль возвращает вычислительному ядру команду 3FFFh (JMP \$). Эта команда «зацикливает» вычислительное ядро до тех пор, пока перепрограммирование

Flash памяти не будет закончено. По окончании перепрограммирования Flash модуль возвращает следующую команду, и выполнение программы продолжается.

Как можно увидеть из кода программы, приведенной на рисунке 3, это – самый простой способ перепрограммировать Flash память микроконтроллеров семейства MSP430. Однако у этого метода имеется один недостаток – при перепрограммировании Flash памяти вычислительное ядро находится в режиме останова и никакая программа или подпрограмма обработки прерывания не может в это время выполняться. Кроме того, этот метод позволяет перепрограммировать Flash память только байтами или словами (7 бит в FCTL1 сброшен), а значит не может использоваться режим быстрой записи сегментами. Более полные примеры программ для стирания и записи Flash памяти при помощи этой функции приведены в приложении А (файл Flash_idle.c) и приложении В (файл Flash_test.c).

```

/  *****/
/* Flash _ww */
/* запись одного слова (16 бит) во Flash память*/
/  *****/
void Flash_ww(int *Data_ptr, int word)
{
    FCTL3 = 0x0A500; /* Lock = 0 */
    FCTL1 = 0x0A540; /* WRT = 1 */
    *Data_ptr=word; /* запись слова */
    FCTL1 = 0x0A500; /* WRT = 0 */
    FCTL3 = 0x0A510; /* Lock = 1 */
}

```

Рис. 4. Подпрограмма остановки вычислительного ядра при перепрограммировании Flash памяти

4. Демонстрационная программа

Было реализовано два метода перепрограммирования Flash памяти:

- программирование Flash памяти из ОЗУ;
- прямое перепрограммирование Flash памяти.

Файл Flash_ram.s43 содержит программу перепрограммирования Flash памяти из ОЗУ, которая написана на ассемблере и может быть вызвана из C компилятора. Файл Flash_idle.c содержит программу прямого перепрограммирования Flash памяти. Эти файлы содержат одинаковые функции:

- void Flash_wb(char *Data_ptr, char byte);
- void Flash_ww(int *Data_ptr, int word);
- void Flash_clr(int *Data_ptr);

Поскольку они имеют одинаковый формат вызова, то они взаимозаменяемы, что и показано на рисунке 5.

Демонстрационной программе необходимы три следующих файла:

- Flash _ram.s43 или Flash _idle.c – содержат функции перепрограммирования Flash памяти
- Flash _test.c – основная программа, которая вызывает функции перепрограммирования Flash памяти
- Flash _var.s43 – программа, показывающая как определить реальные адреса глобальных переменных языка C в области памяти.

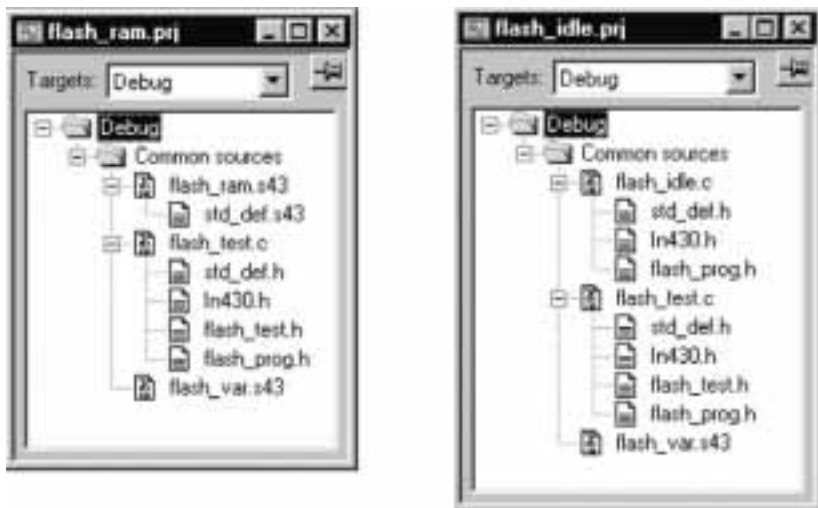


Рис. 5. Окна проектов программирования Flash памяти из ОЗУ и прямого программирования Flash памяти

5. Заключение

В данном примере применения описаны два метода перепрограммирования Flash памяти и приведены примеры их программной реализации. В том случае, когда необходимо обеспечить функционирование вычислительного ядра (например, прием данных при помощи UART), нужно использовать метод перепрограммирования из ОЗУ. При этом методе необходима дополнительная память. Более простой метод состоит в остановке вычислительного ядра при перепрограммировании Flash память. Длительность записи одного слова приблизительно равняется 80 мкс, а одного сегмента – 12 мс.

Ссылки:

1. MSP430x1xx Family User's Guide, SLAU049, 2000
2. MSP430x11x1 Datasheet, SLAS241C, June 2000
3. MSP430x13x, MSP430x14x Datasheet, SLAS272A, July 2000

Приложение A: файл flash_ram.s43

```
;*****
; flash_ram.c 2000-06-20
;
; Этот модуль содержит функции, необходимые для
; программирования Flash памяти при помощи программы,
; работающей из ОЗУ. При программировании Flash памяти
; вычислительное ядро остается активным и может
; использоваться, например, для принятия данных при
; помощи UART. Необходимая функция копирования
; содержится в определенной программе во Flash памяти
; или в функции стирания.
;
; Реализованные функции:
; -----
; void Flash_wb( char *Data_ptr, char byte );
; void Flash_ww( int *Data_ptr, int word );
; void Flash_clr( int *Data_ptr );
;
; Автор: Anton Muehlhofer, компания Texas Instruments
; Incorporated
;*****
MODULE          Flash_wb
PUBLIC          Flash_wb
RSEG            CODE
#define          _CPU_ 5 /* определение микроконтроллера:
                        5=MSP430F1121, 6=MSP430F149 */
#include <Std_def.s43> /* порты */

;*****
; Запись байта во Flash память
;*****
; Программирование 1 байта во Flash память путем копи-
; рования небольшой (18 байт) функции в ОЗУ динамичес-
; кой памяти, расположенной в стеке.
```

```
; Определение функции: void Flash_wb( char *Data_ptr,
; char byte)
; рабочий регистр: R12 = char *Data_ptr
;               R14 = символичный байт
;               R13,R15 = общего назначения
;*****
```

```
Flash_wb
    dint                ; предотвращение искажения
                        ; стека
    mov    #Flash_wb_end,R13 ; определение
                        ; конечного адреса и длины
    mov    #Flash_wb_length,R15; функции,
                        ; копируемой в ОЗУ
    mov    #0A500h,&FCTL3    ; LOCK = 0
copy    push @R13            ; копирование функции в ОЗУ
    decd   R13
    dec    R15
    jnz    copy
    mov    SP,R15
    mov    #0A540h,&FCTL1    ; WRT = 1
    call   R15              ; вызов из ОЗУ функции
                        ; записи Flash памяти
    mov    #0A500h,&FCTL1    ; WRT = 0
    mov    #0A510h,&FCTL3    ; LOCK = 1
    add    #2*Flash_wb_length,SP; Обслуживание стека
    ret
```

;----- Функция записи во Flash память -----

```
Flash_wb_start
    mov.b  R14,0(R12)        ; запись байта данных
                        ; во Flash память
wait_bf    bit  #1,&FCTL3    ; ожидание флага
                        ; индикации «свободно-
                        ; го» состояния
    jnz    wait_bf
Flash_wb_end
    ret
                        ; вычисление номера
                        ; слова, которое будет
                        ; записано во Flash
                        ; память
```

```
Flash_wb_length EQU (Flash_wb_end-Flash_wb_start+2)/2
ENDMOD
;=====
MODULE      flash_ww
PUBLIC      Flash_ww
RSEG        CODE
#define _CPU_ 5 /* определение микроконтроллера:
                5=MSP430F1121, 6=MSP430F149 */
#include <Std_def.s43>

;*****
; Запись слова во Flash память
;*****
; Программирование 1 слова во Flash память путем
; копирования небольшой (18 байт) функции в ОЗУ
; динамической памяти, расположенной в стеке.
; Определение функции: void Flash_ww( int *Data_ptr,
; int word)
; рабочий регистр: R12 = int *Data_ptr
;
; R14 = слово
;
; R13,R15 = общего назначения
;*****

Flash_ww
    dint                ; предотвращение повреждение стека
    mov    #Flash_ww_end,R13 ; определение
                                ; конечного адреса
                                ; и размера
    mov    #Flash_ww_length,R15 ; функции, которая
                                ; копируется в ОЗУ
    mov    #0A500h,&FCTL3      ; LOCK = 0
copy    push    @R13           ; копирование функции
                                ; в ОЗУ

    decd    R13
    dec     R15
    jnz     copy
    mov     SP,R15
    mov     #0A540h,&FCTL1      ; WRT = 1
    call    R15                ; вызов функции записи
                                ; во Flash память из
                                ; ОЗУ
    mov     #0A500h,&FCTL1      ; WRT = 0
    mov     #0A510h,&FCTL3      ; LOCK = 1
```

```

        add    #2*Flash_ww_length,SP ; Обслуживание
                                   ; стека
        ret
;----- Функция записи во Flash память -----
Flash_ww_start
        mov    R14,0(R12)      ; запись байта данных
                                   ; во Flash память
wait_bf bit    #1,&FCTL3      ; ожидание установки бита
                                   ; «свободного» состояния
        jnz    wait_bf
Flash_ww_end
        ret
                                   ; вычисление номера слова,
                                   ; которое будет записано
                                   ; во Flash память
Flash_ww_length EQU (Flash_ww_end-Flash_ww_start+2)/2
ENDMOD

;=====

MODULE      Flash_clr
PUBLIC      Flash_clr
RSEG        CODE
#define _CPU_ 5      /* определение типа микроконтролле-
                      ра:
                      5=MSP430F1121, 6=MSP430F149 */
#include <Std_def.s43>
;*****
; Очистка сегмента Flash памяти
;*****
; Очистка 1 сегмента Flash памяти путем копирования
; небольшой (x байт) функции в ОЗУ динамической
; памяти, расположенной в стеке.
; определение функци: void Flash_clr( int *Data_ptr )
; рабочий регистр: R12 = int *Data_ptr
;
;          R13,R15 = общего назначения
;*****
Flash_clr
        dint
                                   ; предотвращение
                                   ; повреждение стека
        mov    #Flash_ww_end,R13 ; определение конечно-
                                   ;го адреса и размера

```

```

        mov    #Flash_wv_length,R15; функции, которая
                                ; копируется в ОЗУ
        mov    #0A500h,&FCTL3      ; LOCK = 0
copy    push   @R13                ; копирование функции
                                ; в ОЗУ

        decd   R13
        dec    R15
        jnz    copy
        mov    SP,R15
        mov    #0A540h,&FCTL1      ; WRT = 1
        call   R15                 ; вызов функции записи
                                ; во Flash память

        mov    #0A500h,&FCTL1      ; ERASE = 0
        mov    #0A510h,&FCTL3      ; LOCK = 1
        add    #2*Flash_clr_length,SP; обслуживание
                                ; стека

        ret

;----- Функция очистки Flash памяти-----
Flash_clr_start
        mov    #0,0(R12)          ; стирание сегмента Flash
                                ; памяти
wait_bf bit    #1,&FCTL3          ; ожидание установки флага
                                ; «свободного» состояния
        jnz    wait_bf
Flash_clr_end
        ret
; вычисление номера слова, которое будет стерто во
Flash памяти
Flash_clr_length EQU (Flash_clr_end-Flash_clr_
start+2)/2
END

```

Приложение В: файл flash_idle.c

```

/*****
/* flash_idle.c 2000-06-20 */
/* */
/* Функции стирания и программирования Flash памяти */
/* */
/* Приведенные ниже функции используют алгоритм
/* непосредственного программирования Flash памяти. */

```

```

/* После запуска цикла записи или программирования,
/* вычислительное ядро будет ожидать */
/* до тех пор пока Flash память не станет снова
/* доступной для чтения, поэтому нет необходимости */
/* копировать программу в ОЗУ. Таким образом,
/* вычислительное ядро находится в */
/* режиме "idle". */
/* */
/* Замечание: Так как все вектора прерывания недоступ-
/* ны при программировании Flash*/
/* памяти, поэтому все прерывания должны быть запреще-
/* ны. */
/* */
/* Автор: Anton Muehlhofer, компания Texas
/* Instruments Incorporated */

/*****/
#define _CPU_ 5      /* Определение микроконтроллера:
5=MSP430F1121, 6=MSP430F149 */
#include <std_def.h>    /* порты */
#include <flash_prog.h> /* прототипы функций */

/*****/
/* Flash_wb */
/* Программирование 1 байта (8 бит) во Flash память */
/*****/
void Flash_wb(char *Data_ptr, char byte)
{
    FCTL3 = 0x0A500; /* Lock = 0 */
    FCTL1 = 0x0A540; /* WRT = 1 */
    *Data_ptr=byte; /* программирование слова
                     во Flash память*/
    FCTL1 = 0x0A500; /* WRT = 0 */
    FCTL3 = 0x0A510; /* Lock = 1 */
}

/*****/
/* Flash_ww */
/* программирование 1 слова (16 бит) во Flash память
*/
/*****/
void Flash_ww( int *Data_ptr, int word )
{

```

```

    FCTL3 = 0x0A500; /* Lock = 0 */
    FCTL1 = 0x0A540; /* WRT = 1 */
    *Data_ptr=word; /* программирование слова
                    /* во Flash память */
    FCTL1 = 0x0A500; /* WRT = 0 */
    FCTL3 = 0x0A510; /* Lock = 1 */
}

/*****
/* Flash_clr */
/* Стирание 1 сегмента Flash памяти */
*****/
void Flash_clr( int *Data_ptr)
{
    FCTL3 = 0x0A500; /* Lock = 0 */
    FCTL1 = 0x0A502; /* ERASE = 1 */
    *Data_ptr=0; /* стирание сегмента Flash
                /* памяти */
    FCTL1 = 0x0A500; /* ERASE = 0 */
    FCTL3 = 0x0A510; /* Lock = 1 */
}

```

Приложение C: файл flash_var.s43

```

; ****
; File: flash_var.s43 30. мая 2000
;
; определение двух однобайтных переменных, имеющих
; специфические адреса во Flash памяти, которые могут
; быть доступны в C как стандартные внешние символьные
; переменные (см. файл "flashd.h"):
;
; extern unsigned char SegA_last; /* абсолютный адрес
; 010FFh */
; extern unsigned char SegB_last; /* абсолютный адрес
; 0107Fh */
;
; Автор: Anton Muehlhofer, компания Texas Instruments
; Incorporated
; ****
; =====
; extern unsigned char SegA_last; /* абсолютный адрес
; 010FFh */

```

```

;=====
        NAME    SegA_last
        PUBLIC  SegA_last
        ASEG    010FFh
SegA_last    DS    0
        ENDMOD
;=====
; extern unsigned char SegB_last; /* абсолютный адрес
; 0107Fh */
;=====
        NAME    SegB_last
        PUBLIC  SegB_last
        ASEG    0107Fh
SegB_last    DS    0
        ENDMOD
        END

```


III. Интерфейс с внешними устройствами

Глава 1. *Подключение ADS1100 к MSP430F413*

Подключение ADS1100 к MSP430F413

Andreas Dannenberg [slaa206.pdf (Англ.)]

В данном примере применения описывается, как можно подключить дифференциальный 16-разрядный сигма-дельта АЦП ADS1100 к микроконтроллеру MSP430F413. Для примера осуществляется измерение и индикация на ЖКИ выходного сопротивления полномостового тензорезистивного датчика.

Описание аппаратной части

Микроконтроллер MSP430F413, используемый в этом примере применения – это недорогой микроконтроллер со встроенным драйвером ЖКИ. Он осуществляет считывание данных от АЦП и индикацию результата измерения на жидкокристаллическом дисплее. Для подключения к ведущему устройству ADS1100 имеет I²C интерфейс. Так как используемый микроконтроллер MSP430F413 не имеет аппаратного периферийного модуля I²C интерфейса, то для реализации его задействованы линии P2.0 и P2.1 второго порта ввода-вывода общего назначения. Выбор этих линий был сделан произвольно, и они могут быть переназначены путем изменения основного файла I2C_Master.h. К линиям I²C шины подключены 10 кОм подтягивающие резисторы. На рисунке 1 показана схема электрическая принципиальная описываемого устройства.

Питание мостового датчика осуществляется от выводов порта ввода-вывода. Это позволяет снизить потребление путем отключения питания мостового датчика после окончания измерения. Для реализации шины питания мостового датчика по три линии портов микроконтроллера были подключены параллельно.

3.5-цифровой статический ЖКИ марки VI-302 производства компании Varitronix используется для отображения результата измерения и подключается непосредственно к микроконтроллеру MSP430. Кнопки PB1 и PB2 подключены к линиям P1.6 и P1.7 и подтянуты к линии питания 100-кОм резисторами.

Описание программы

Краткий обзор

Демонстрационное программное обеспечение данного примера применения написано как на C, так и на ассемблере. Обе программы функционально идентичны и используют общую библиотеку ведущего I²C интерфейса, кото-

рая написана на ассемблере. Проект, сформированный только из программ, написанных на ассемблере, имеет размер приблизительно на 20% меньше, чем смешанный проект, сформированный из программы, написанной на С и библиотеки, написанной на ассемблере.

Таблица 1. Описание файлов

Имя файла	Описание
ADS1100_Demo.c	Демонстрационная программа считывания данных из ADS1100, написанная на С
ADS1100_Demo.s43	Демонстрационная программа считывания данных из ADS1100, написанная на ассемблере
I2C_Master.s43	Библиотека реализации ведущего I ² C интерфейса
I2C_Master.h	Основной файл процедуры ведущего I ² C интерфейса

Работа программы

После сброса при подаче питания происходит инициализация периферийных устройств MSP430. При этом происходит отключение сторожевого таймера, настройка нагрузочных конденсаторов генератора LFXT1 для работы с внешним часовым кварцевым резонатором и инициализация контроллера ЖКИ и основного таймера. В исходном тексте для хранения калибровочных данных резервируется два 16- битных слова в сегменте A Flash памяти MSP430. Если оба слова содержат одинаковое значение (например, как после сброса при включении питания – 0xFFFF), то активизируется режим калибровки. В противном случае микроконтроллер входит в режим измерения.

ADS1100 настроен на работу в непрерывном режиме преобразования с 16-разрядной точностью. Для усиления выходного напряжения дифференциального мостового датчика коэффициент усиления усилителя с программируемым коэффициентом усиления (УПКУ) АЦП установлен равным 8. Более подробное описание режима работы ADS1100 можно найти в техническом описании данного прибора. Запись конфигурационного байта в АЦП по I2C интерфейсу осуществляет библиотечная функция I2CWrite8().

Затем, MSP430 переходит в режим пониженного потребления LPM3, при этом прерывания разрешены. С этого времени программа управляется только по прерываниям. Разрешены два источника прерывания. Основным таймер ISR формирует сигнал прерывания каждые 0.25 сек. и, в основном, используется для того, чтобы получить и отобразить результаты измерения. Порт 1 ISR используется для отслеживания нажатия кнопки.

В режиме калибровки задаются две константы. Это нижний (CAL LO) и верхний (CAL HI) пороги диапазона измерений. При нажатии одной из кнопок результат преобразования АЦП ADS1100 считывается при помощи функции I2CRead16() и сохраняется в соответствующей временной переменной. По окончании калибровки обе эти переменные сохраняются в сегменте INFOA Flash памяти при помощи функции внутрисистемного самопрограммирования. После этого система переходит в режим измерения. В этом режиме результат преобразования ADS1100 считывается и сравнивается с предыдущим каждые 0.25 сек. Если значения отличаются, то состояние индикатора обновляется. Это позволяет избежать ненужных 32-разрядных целочисленных умножений и делений при неизменности измеряемого сигнала. Значение индицируемого значения рассчитывается по следующей формуле:

$$DisplayValue = \frac{CurrentADCValue - CalMin}{CalMax - CalMin} \times CAL_MIN_MAX_SPAN$$

Диапазон от CalMax до CalMin пересчитывается в значение от 0 до CAL_MIN_MAX_SPAN. По умолчанию CAL_MIN_MAX_SPAN установлен равным 1,000. При помощи кнопки PB2 можно в любое время перейти в режим калибровки.

Нажатие кнопки PB1 вызывает остановку преобразования, отключает ЖКИ и переводит MSP430 в режим пониженного потребления LPM3. В этом режиме устройство потребляет менее 1 мкА, при этом 32 кГц генератор остается в рабочем состоянии. Если необходимо, то можно задействовать режим пониженного потребления LPM4. При повторном нажатии кнопки PB1 происходит переход в нормальный режим работы.

Библиотечная функция ведущего I²C интерфейса

Эта библиотека была создана для того, чтобы микроконтроллеры семейства MSP430, не имеющие аппаратного модуля I²C интерфейса, могли все же вести обмен данными с внешними устройствами по этому интерфейсу. Для передачи и приема данных по I²C интерфейсу эта библиотека использует технологию bit-bang. По стандарту I²C требуется выход с тремя состояниями. Для этого линии переводятся в режим входов, а выводы притягиваются к линии питания при помощи внешних подтягивающих резисторов. При этом только при необходимости формирования нуля на линии MSP430 переводит вывод в режим выхода и удерживает его в низком состоянии.

Для того чтобы иметь возможность правильно формировать временные параметры, библиотека написана на ассемблере. Если написать ее на C, то временные параметры будут зависеть от компилятора и настроек оптимизатора.

Когда микроконтроллер работает на заданной по умолчанию частоте 1 048 576 Гц, все временные параметры сгенерированного I²C интерфейса удовлетворяют требованиям к работе в нормальном режиме. Библиотека может использоваться любыми микроконтроллерами семейства MSP430.

Из основных программ, написанных на С или ассемблере, можно легко организовать вызов любых подпрограмм пользователя. Для упрощения интеграции в среду С существует основной файл. Имеется возможность вызова трех функций:

· ***void I2CSetup(void)***

Эта функция инициализирует выходы MSP430, предназначенные для ведения обмена данными по I²C интерфейсу. Выводы портов SDA и SCL, так же как и определения состояния портов PI2C_DIR, PI2C_OUT и PI2C_IN, могут быть изменены в основном файле I2C_Master.h. Для обслуживания линий интерфейса рекомендуется использовать 4 младшие линии портов. В этом случае для манипуляции битами вычислительное ядро MSP430 способно использовать характеристику постоянного генератора, что позволяет уменьшить размер кода и повысить быстродействие. Более подробно об этом можно прочитать в руководстве пользователя по 16-битным RISC микроконтроллерам семейства MSP430x4xx.

· ***unsigned int I2CRead16(unsigned char Addr)***

Эта функция считывает 16-разрядные данные из указанного внешнего подчиненного I²C устройства. LSB передаваемого адресного I²C байта устанавливается внутренне и служит для указания запрашиваемого устройства. Данные считываются с шины старшим значащим битом вперед.

· ***void I2CWrite8(unsigned char Addr, unsigned char Data)***

При вызове этой функции осуществляется передача 8-битных данных Data устройству с адресом Addr.

Кроме того, библиотека содержит дополнительные вспомогательные функции, позволяющие генерировать I²C условия старта, останова и подтверждения (ACK или nACK). Как было сказано выше, эти функции могут вызываться и различными пользовательскими функциями. Для удовлетворения специфических требований можно создать и свои варианты функций.

На рисунке 1 приведена схема электрическая принципиальная описанного устройства.

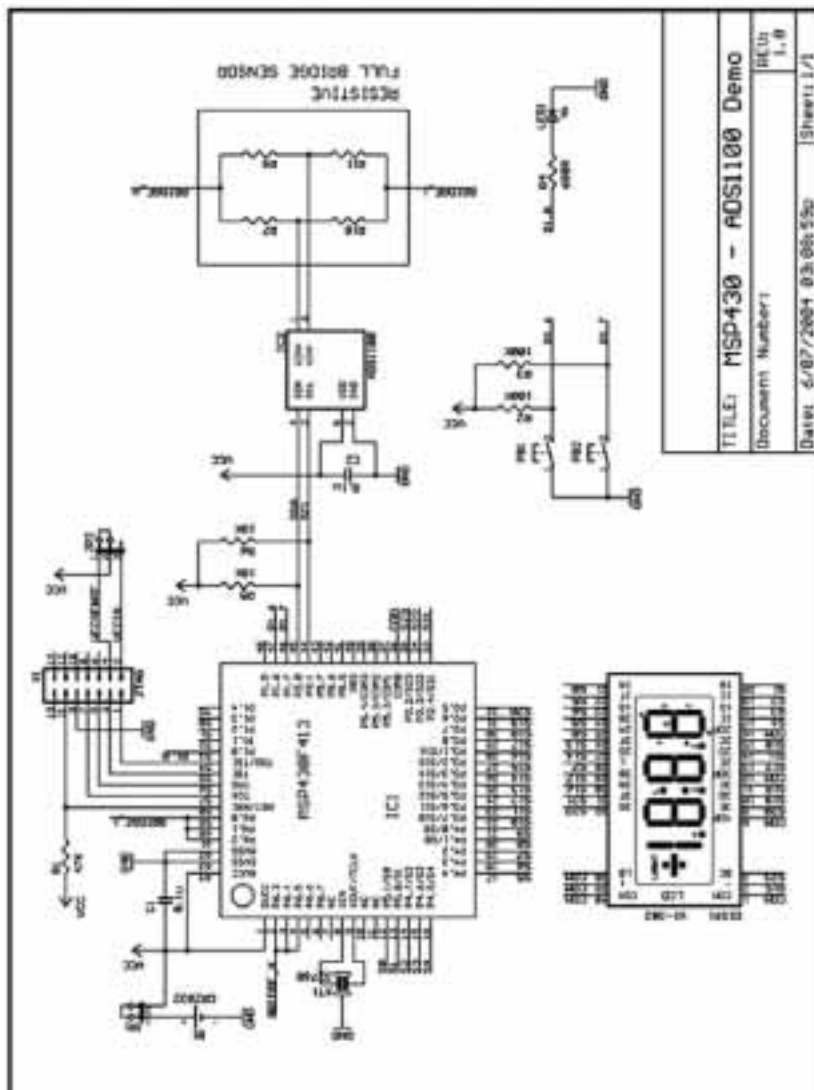


Рис. 1. Схема электрическая принципиальная

III. Интерфейс с внешними устройствами

Глава 2.

Подключение 3-вольтовых микроконтроллеров MSP430 к 5-вольтовым схемам

Подключение 3-вольтовых микроконтроллеров MSP430 к 5-вольтовым схемам

Lutz Bierl [slaa148.pdf (Англ.)]

В этом примере применения рассматривается подключение 3 вольтовых микроконтроллеров MSP430x1xx и MSP430x4xx к схемам с питанием 5В или выше. Описано согласование входов, выходов и двунаправленных линий. Там, где это необходимо, даются примеры расчётов. Также показаны примеры построения простых источников питания на оба напряжения.

1. Введение

Современные микроконтроллеры MSP430, например, семейства MSP430x1xx и MSP430x4xx работают в диапазоне питающих напряжений от 1,8 В до 3,6 В. Это обусловлено используемым технологическим процессом и имеет ряд преимуществ, в частности – микроконтроллеры этих серий потребляют меньший ток, чем 5-вольтовые MSP430C3xx.

Сложности возникают при необходимости сопряжения таких устройств с 5-вольтовыми или ещё более высоковольтными схемами. В этом примере применения показано, как согласовать 5-вольтовые интерфейсы со входами, выходами и двунаправленными линиями микроконтроллеров MSP430. На рис. 1 показаны примеры таких согласований. В этом примере рассматриваются устройства, помеченные на рисунке тёмно-серым цветом.

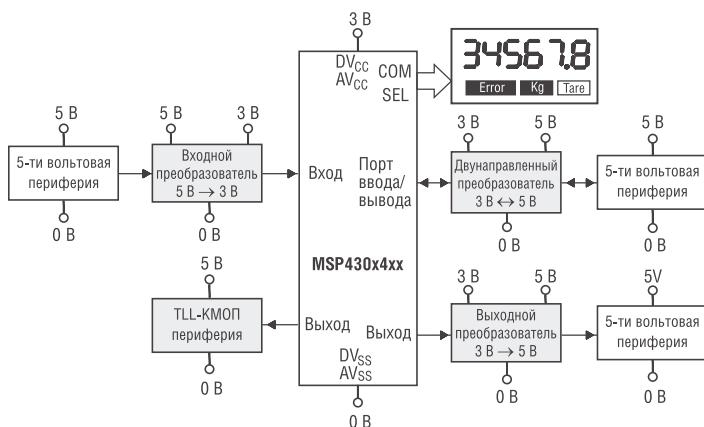


Рис. 1. Интерфейс между 3 В MSP430 и 5 В устройством

Примечание:

Далее по тексту термин *MSP430* означает микроконтроллеры семейств *MSP430x1xx* и *MSP430x4xx*.

Примечание:

Приведенные формулы для внешнего источника питания $V_{(sys)}$ также справедливы и могут быть использованы для напряжений выше, чем 5В. Их можно использовать для любых уровней внешнего напряжения, например, $V_{(sys)} = 12\text{ В}$.

В формулах будут использованы следующие замены для упрощения вычислений при малых значениях a_x (например, для начального разброса p).

$$\frac{1}{1+a_x} \approx (1-a_x) \quad \frac{1}{1-a_x} \approx (1+a_x) \quad \frac{1+a_x}{1-a_x} \approx (1+2a_x) \quad \frac{1-a_x}{1+a_x} \approx (1-2a_x)$$

При таких заменах ошибка будет ничтожной при $|a_x| < 0,1$.

2. Определения

2.1. Значения из спецификаций MSP430

Численные значения для расчётов наихудших случаев взяты из [4]. Если не указано иное, значения приведены при $DV_{CC} = 3\text{ В}$:

$DV_{CC(min)}$	Минимальный уровень питания цифровой части MSP430x4xx	1,8 В
$DV_{CC(max)}$	Максимальный уровень питания цифровой части MSP430x4xx	3,6 В
$V_{IT(max)}$	Максимальный порог уровня лог. «1» на входе порта MSP430	1,9 В
$V_{IT(min)}$	Минимальный порог уровня лог. «0» на входе порта MSP430	0,9 В
$V_{OH(min)}$	Минимальное напряжение лог. «1» на выходе порта при $I_O = -1,5\text{ мА}$	$DV_{CC} - 0,25\text{ В}$
$V_{OL(max)}$	Максимальное напряжение лог. «0» на выходе порта при $I_O = 1,5\text{ мА}$	$DV_{SS} + 0,25\text{ В}$
I_{lkg}	Ток утечки по входу MSP430	$\pm 50\text{ нА}$
	Абсолютный максимум тока через защитные диоды любого вывода MSP430 ($V_I < -0,3\text{ В}$ или $V_I > V_{CC} + 0,3\text{ В}$)	$\pm 2\text{ мА}$

Примечание:

Выходное сопротивление $r_{DS(on)}$ выходов MSP430x4xx во внимание не принимается, ввиду выбора высокоомных резисторов в расчётах. По сравнению со значениями используемых резисторов выходное сопротивление $r_{DS(on)}$ (не более 167 Ом) пренебрежительно мало.

2.2. Определения для внешней системы

$V_{(sys)}$	Напряжение питания внешней системы	[В]
$V_{(sysH)}$	Выходное напряжение лог. «1» внешней системы	[В]
$V_{(sysL)}$	Выходное напряжение лог. «0» внешней системы	[В]
$V_{(sys+)}$	Входное напряжение лог. «1» внешней системы	[В]
p	Начальный разброс интерфейсных резисторов	[%]
$DV_{CC(min)}$	Минимальное напряжение питания для MSP430 при $DV_{CC} = 3,0 \text{ В} \pm 10\%$ ($3,0 \text{ В} \times 0,9 = 2,7 \text{ В}$)	[В]

3. Входные преобразователи

Показанные входные преобразователи в первую очередь рассчитаны на согласование 5-ти и 3-вольтовых систем. Тем не менее, их также можно применять и при более высоких значениях внешнего питания, например, для согласования входа микроконтроллера MSP430 с 12-вольтовыми сигналами.

3.1. Входные преобразователи на резистивных делителях

Внешний цифровой уровень $V_{I(sys)}$ подаётся на MSP430. Формулы для наихудших случаев для расчёта резисторов $R1$ и $R2$ показанных на рис. 2:

$$\frac{R1}{R2} < \frac{V_{(sysH)} \min - V_{IT(max)}}{V_{IT(max)} \times (1+2p)} \text{ и } \frac{R1}{R2} > \frac{V_{(sysL)} \max - V_{IT(min)}}{V_{IT(min)} \times (1-2p)} \text{ и } R1 \parallel R2 < \frac{DV_{CC}}{I_{lkg}}$$

Первые два неравенства обеспечивают превышение входным напряжением $V_{I(430)}$ на входе MSP430 соответствующих наихудших пороговых значений. Третье неравенство обеспечивает отсутствие влияния тока утечки I_{lkg} входных диодов на напряжение $V_{I(430)}$. Для исключения протекания тока через защитные диоды входов MSP430, необходимо, чтобы выполнялось:

$$V_{(sysH)} \max \times \frac{R2 \max}{R1 \min + R2 \max} < DV_{CC(min)} + 0,3$$

и

$$V_{(sysL)} \min \times \frac{R2 \min}{R1 \max + R2 \min} > -0,3$$

ПРИМЕР: Входные напряжения от внешней системы: $V_{(sysH)} = 5,0 \text{ В} \pm 10\%$ и $V_{(sysL)} = 0,5 \text{ В} \pm 0,5 \text{ В}$. Начальный разброс резисторов $p = \pm 5\%$. Минимальное напряжение питания MSP430 в этом примере $DV_{CC(min)} = 2,7 \text{ В}$ ($3,0 \text{ В} - 10\%$). Учитывая приведенные выше значения спецификаций на входные пороговые уровни $V_{IT(max)}$ и $V_{IT(min)}$ и величину входного напряжения $V_{(sysH)min}$:

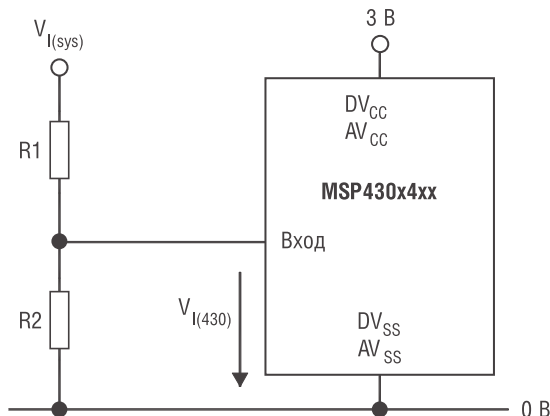


Рис. 2. Резистивный входной преобразователь 5В → MSP430

$$\frac{R1}{R2} < \frac{V_{(sysH)} \min - V_{IT} (max)}{V_{IT} (max) \times (1+2p)} = \frac{4,5В - 1,9В}{1,9В \times (1+0,1)} \rightarrow \frac{R1}{R2} < 1,244$$

Условие для входного напряжения низкого уровня $V_{(sysL)max}$:

$$\frac{R1}{R2} > \frac{V_{(sysL)} \max - V_{IT} (min)}{V_{IT} (min) \times (1-2p)} > \frac{1,0В - 0,9В}{0,9В \times (1-0,1)} \rightarrow \frac{R1}{R2} > 0,1234$$

Для обеспечения незначительности влияния тока утечки I_{lkg} :

$$R1 \parallel R2 \ll \frac{DV_{CC}}{|I_{lkg}|} = \frac{3В}{|\pm 50 \text{ nA}|} \rightarrow R1 \parallel R2 \ll 60 \text{ МОм}$$

Эти три расчётные формулы допускают достаточно широкий диапазон для выбора R1 и R2. Если выбрать $R1/R2 = 1,0$ и $R1 \parallel R2 = 600 \text{ кОм}$, то $R1 = 1,2 \text{ МОм}$ и $R2 = 1,2 \text{ МОм}$. Для предотвращения протекания тока через входные защитные диоды MSP430 необходимо, чтобы выполнялось:

$$V_{(sysH)} \max \times \frac{R2 \max}{R1 \min + R2 \max} < DV_{CC(min)} + 0,3$$

и

$$V_{(sysL)} \min \times \frac{R2 \min}{R1 \max + R2 \min} > -0,3$$

$$5,5 \text{ В} \times \frac{1,26 \text{ МОм}}{1,14 \text{ МОм} + 1,26 \text{ МОм}} < 2,7 \text{ В} + 0,3 \rightarrow 2,8875 \text{ В} < 3,0 \text{ В}$$

$$0,0 \text{ В} \times \frac{1,14 \text{ МОм}}{1,26 \text{ МОм} + 1,14 \text{ МОм}} > -0,3 \rightarrow +0,0 \text{ В} > -0,3 \text{ В}$$

Выполнение двух последних неравенств не имеет большой важности, если значение тока через вывод MSP430 гораздо меньше, чем $\pm 2 \text{ мА}$ (абсолютный максимум допустимого значения входного тока). Именно такой случай имеет место в нашем примере: $R1||R2 = 600 \text{ кОм}$.

Приведенные выше расчётные формулы справедливы для следующих выводов MSP430, настроенных на ввод:

- Все порты ввода/вывода (P1...P6)
- Входы кварцевого резонатора XIN и XT2IN: $V_{IL(X)} \max = 0,2 \times DV_{CC}$, $V_{IL(X)} \min = 0,8 \times DV_{CC}$
- Вход RST/NMI: $V_{IL} \max = DV_{SS} + 0,6 \text{ В}$, $V_{IH} \min = 0,8 \times DV_{CC}$
- Входы компаратора Comparator_A CA0 и CA1
- Входы UART/SPI URXDx, SOMIx, SIMOx, UCLK
- Входы таймера Timer_A TACLK, TA0...TA2
- Входы таймера Timer_B TBCLK, TB0...TB6
- Входы АЦП ADC12: время выборки $t_{(sample)}$ должно быть согласовано с импедансом резистивного делителя R1||R2. Дополнительная информация содержится в главе ADC12 в [2] или [3].

3.2. Входной преобразователь на транзисторе

Входной преобразователь на транзисторе представляет собой весьма простую схему, позволяющую согласовать различные внешние устройства с микроконтроллерами семейства MSP430. Пример реализации инвертирующего входного буфера приведен на рис. 3. Резистор RC может быть отключен при помощи выхода контроллера для снижения потребляемого тока в режиме пониженного энергопотребления 3.

Расчётные формулы для резисторов RC, RB1 и RB2 имеют следующий вид:

$$R_C < \frac{DV_{CC(min)} - V_{T(max)}}{(1 + \rho) \times (I_{kg} + I_{kg(T)})} \quad \frac{R_{B1}}{R_{B2}} > \left(\frac{V_{(sysL)} \max}{V_{BE(off)}} - 1 \right) \times (1 + 2\rho)$$

Третье неравенство обеспечивает включение транзистора при входном уровне $V_{(sysH)} \min$:

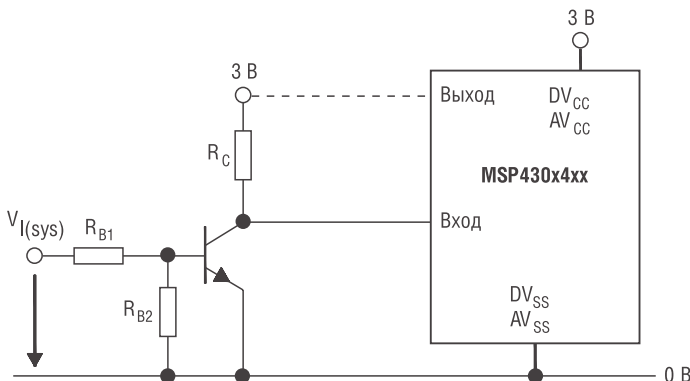


Рис. 3. Транзисторный входной преобразователь из 5 В окружения

$$R_{B1} < \frac{V_{(sysH)} \min - V_{BE(on)} \times \left(1 + \frac{R_{B1}}{R_{B2}} \times (1 + 2p)\right)}{DV_{CC(max)}} \times \beta \min \times R_C \min$$

Где $V_{BE(off)}$ – напряжение база-эмиттер транзистора, при котором он гарантированно выключается [В]

$V_{BE(on)}$ – напряжение база-эмиттер транзистора, при котором он гарантированно включается [В]

β – коэффициент усиления по току транзистора

$I_{kg(Tr)}$ – ток утечки транзистора [А]

Пример: Входное напряжение с уровнем $V_{I(sys)}$ подаётся на вход MSP430 с током утечки $I_{kg} = \pm 50$ нА. Минимальное напряжение лог. «1» $V_{(sysH)} \min = 4,5$ В, максимальное напряжение лог. «0» $V_{(sysL)} \max = 0,7$ В. Начальный разброс всех резисторов $p = \pm 5\%$. Напряжение питания $DV_{CC} = 3$ В $\pm 10\%$. Характеристики транзистора $V_{BE(on)} = 0,75$ В, $V_{BE(off)} = 0,2$ В, $\beta_{\min} = 100$, $I_{kg(Tr)} = 10$ нА.

Максимальное значение R_C равно:

$$R_C < \frac{DV_{CC(min)} - V_{IT(max)}}{(1+p) \times (I_{kg} + I_{kg(Tr)})} = \frac{2,7В - 1,9В}{(1+0,05) \times (50нА + 10нА)} = 12,7М\Omega$$

Минимальное соотношение номинальных значений R_{B1} и R_{B2} равно:

$$\frac{R_{B1}}{R_{B2}} > \left(\frac{V_{(sysL) \max}}{V_{BE(off)}} - 1 \right) \times (1 + 2p) = \left(\frac{0,7B}{0,2B} - 1 \right) \times (1 + 0,1) = 2,75$$

Максимально допустимое значение для R_{B1} :

$$R_{B1} < \frac{V_{(sysH) \min} - V_{BE(on)} \times \left(1 + \frac{R_{B1}}{R_{B2}} \times (1 + 2p) \right)}{DV_{CC(max)}} \times \beta \min \times R_c \min$$

$$R_{B1} < \frac{4,5B - 0,75B \times [1 + 2,75 \times (1 + 0,1)]}{DV_{CC(max)}} \times 100 \times 2MOM \times (1 - 0,05) = 85,3MOM$$

С учётом значения $R_{B1} = 39 MOM$ получаем для R_{B2} :

$$R_{B2} < \frac{R_{B1}}{2,75} = \frac{39MOM}{2,75} = 14,18MOM$$

3.3. Входной преобразователь на операционном усилителе

Операционные усилители являются оптимальным решением для входных преобразователей в тех случаях, когда они уже используются в системе (в качестве интегратора, компаратора, усилителя, ЦАП и т.д.). Для ОУ TLC27L4 необходимо ограничить входные напряжения до уровня не выше, чем $V_{DD} + 0,3 V$. Минимальное напряжение питания TLC27L4 $V_{CC(min)} = 3 V$.

Формулы для вычисления номиналов резисторов $R1$ и $R2$ по схеме, приведенной на рис. 4 при наихудших условиях:

$$\frac{R1}{R2} < \frac{V_{(sysH) \min}}{V_{ref(max)}} \times (1 - 2p) \quad \text{и} \quad \frac{R1}{R2} > \frac{V_{(sysL) \max}}{V_{ref(min)}} \times (1 - 2p) \quad \text{и} \quad R1 \parallel R2 \leq \frac{DV_{CC}}{II_{kg}}$$

$$\text{Где } V_{ref(max)} = DV_{CC(max)} \times \frac{R_{B2} \max}{R_{B1} \min + R_{B2} \max} \quad \text{и} \quad V_{ref(min)} = DV_{CC(min)} \times \frac{R_{B2} \min}{R_{B1} \max + R_{B2} \min}$$

Пример расчёта приведен в главе Входные преобразователи на резистивных делителях, параграф 3.1.

3.4. Входной преобразователь на микросхеме ULN2003A

В левой части рис. 7, три буфера микросхемы ULN2003A используются в качестве входного преобразователя для согласования с 5-вольтовыми и 12-

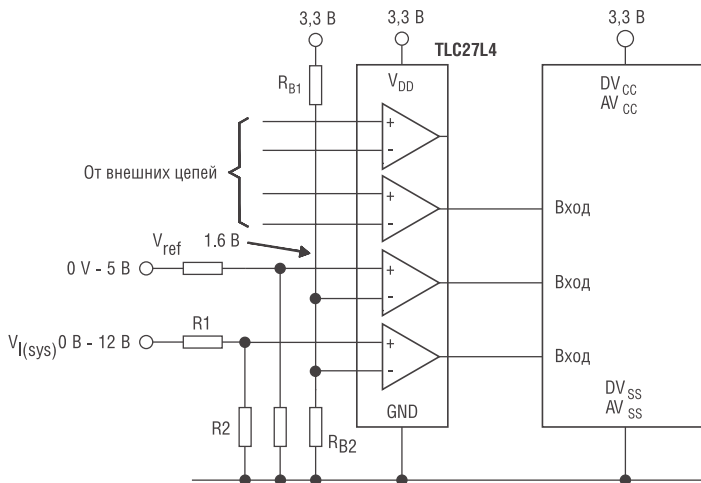


Рис. 4. Входной преобразователь на операционном усилителе

вольтовыми системами. Последовательный резистор RV ($p = \pm 5\%$) для входного сигнала амплитудой 12 В ($V_{(\text{sysH})} \text{ min} = 11 \text{ В}$) рассчитывается следующим образом:

$$R_v < \frac{V_{(syH)} \min - V_{l(on)} \min}{(1+p) \times I_{l(on)} \max} = \frac{1B - 2,4B}{1,05 \times 1,35mA} \rightarrow R_v \max < 6,06k\Omega$$

«Подтягивающий» резистор R_p на входе MSP430 вычисляется так:

$$R_p < \frac{V_{CC} - V_{IT(max)}}{(1+p) \times I_{CF(max)}} = \frac{3B - 1,9B}{1,05 \times 50\mu A} \rightarrow R_p \max < 20,9k\Omega$$

Для уменьшения потребляемого тока, резисторы R_p подключаются к DV_{cc} только по необходимости.

3.5. Входные преобразователи на интегральных микросхемах

Для преобразования 5-вольтовых сигналов в 3,3-вольтовые можно использовать любые КМОП-микросхемы, удовлетворяющие следующим условиям:

- Предназначена для работы с питающим напряжением 3,3 В или менее.
- Явно указана возможность работы с входными уровнями более 3,3 В.

Указанным требованиям соответствуют микросхемы семейств АНС и LVC. Их можно подключать к источнику питания 3,3 В микроконтроллера MSP430.

Примечание:

Обязательно проверьте, действительно ли выбранная вами микросхема допускает подачу входных сигналов с уровнем V_I , выше, чем уровень источника питания 3,3 В. В разделе предельные условия эксплуатации (absolute maximum ratings) документации должна присутствовать подобная запись:

Диапазон входных напряжений (Input voltage range) V_I от 0,5 В до 7 В

Для «обычных» же КМОП микросхем запись будет выглядеть так:

Диапазон входных напряжений (Input voltage range) от 0,5 В до $V_{CC} + 0.5$ В

или так:

Входной ток защитных диодов (Input clamp current) I_{IK} ($V_I < 0$ или $V_I > V_{CC}$) ± 20 мА

3.6. Входные преобразователи аналоговых сигналов

Для согласования входов АЦП ADC12 можно использовать такие же резистивные делители, как и рассмотренные выше для цифровых сигналов. На рис. 5 показано подключение 5-вольтового датчика тока Холла к входу A_x АЦП ADC12. Расчётные формулы для наихудших условий приведены в главе Входные преобразователи на резистивных делителях, параграф 3.1.

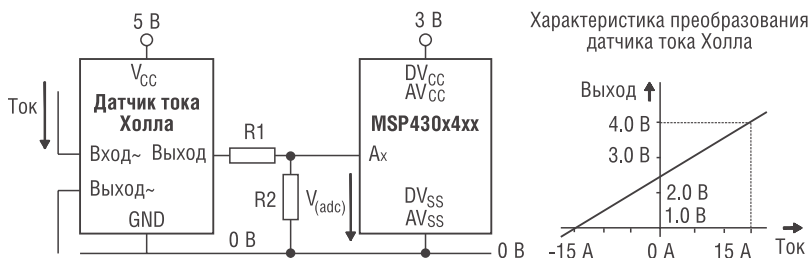


Рис. 5. Аналоговый входной преобразователь 5 В → 3 В для АЦП ADC12

Если внешняя схема не способна запитывать вытекающим током резистивный делитель, следует применять повторитель на ОУ, расположенный между выходом периферийного устройства и резистором R1. Время выборки АЦП ADC12 $t(\text{sample})$ должно быть согласовано с импедансом резистивного делителя $R1||R2$. Дополнительная информация содержится в главе ADC12 в [2] или [3].

4. Выходные преобразователи

Преобразователи не требуются для работы с ЖКИ и с пассивными датчиками. Их можно соответствующим образом подключать непосредственно к MSP430. См. [2].

4.1. Выходной преобразователь на транзисторе

Простой преобразователь для систем с рабочим напряжением более 3 В показан на рис. 6. Нагрузка транзистора R_L может быть самой разнообразной: резисторы, нагреватели, вентиляторы, реле и т.д. Резистор в цепи базы R_B вычисляется следующим образом:

$$R_B < \frac{R_L \min \times \beta_{\min} \times (V_{OH(\min)} - V_{BE(on)})}{(1 + p) \times V_{(sys) \max}}$$

Где $R_L \min$ – минимальное сопротивление нагрузки [Ом]

$\beta_{\min}$ – минимальный коэффициент усиления по току транзистора

$V_{(sys) \max}$ – максимальное напряжение питания внешней системы [В]

$V_{BE(on)}$ – напряжение включения база-эмиттер транзистора [В]

Благодаря низкому напряжению на выходе порта MSP430, резистора между базой транзистора и 0 В не требуется.

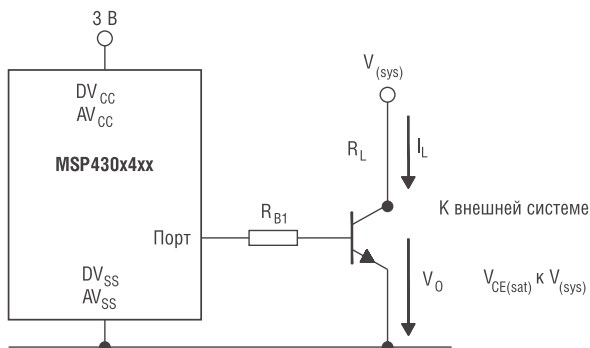


Рис. 6. Выходной преобразователь на транзисторе для согласования с 5-вольтовой системой

Пример: нагрузка $R_L = 100 \text{ Ом} \pm 3\%$ подключена к $V_{(sys)} = 5 \text{ В} \pm 10\%$. Начальный разброс резистора R_B $p = \pm 10\%$. Минимальное напряжение питания $DV_{CC(\min)} = 2,7 \text{ В}$ ($3,0 \text{ В} - 10\%$). Характеристики транзистора: $V_{BE(on)} = 0,7 \text{ В}$ и $\beta_{\min} = 100$.

$$R_b < \frac{1000 \times (1 - 0,03) \times 100 \times (2,7B - 0,25B - 0,7B)}{(1 + 0,1) \times 5B \times (1 + 0,1)} \rightarrow R_b < 2805,80 \Omega$$

4.2. Выходной преобразователь для входов КМОП – TTL

Все интегральные микросхемы со входами TTL-КМОП могут быть использованы в качестве выходных цепей для MSP430x1xx и MSP430x4xx. Входные уровни для этих ИС:

$V_{IH\min}$	Минимальное входное напряжение лог. «1»	2,0 В
$V_{IL\max}$	Максимальное входное напряжение лог. «0»	0,8 В

Оба напряжения находятся в пределах выходных напряжений портов MSP430: $DV_{CC} - 0,25$ В и $DV_{SS} + 0,25$ В при $DV_{CC} = 2,7$ В... 3,6 В. Никаких согласующих элементов в данном случае не требуется, TTL-КМОП микросхемы содержат встроенный преобразователь 3-В/5-В.

4.3. Согласование со входами микросхемы ULN2003

Для увеличения выходных токов или для управления несколькими (до семи) 5-вольтовыми выходами можно использовать выходной буфер ULN2003A. Его характеристики приведены ниже:

$I_L\max$	Максимальный выходной ток	500 мА
$V_L\max$	Максимальное выходное напряжение	50 В
$V_{I(on)}\max$	Максимальное входное напряжение ($I_L = 200$ мА)	2,4 В
$I_{I(on)}\max$	Максимальный входной ток ($V_I = 3,85$ В)	1,35 мА
$I_{CE}\max$	Максимальный ток утечки выхода ($V_{CE} = 50$ В)	50 мкА

В правой части рис. 7, ULN2003A используется в качестве выходного буфера для согласования с 5-вольтовыми и 12-вольтовыми периферийными устройствами. Защитные диоды на выходе ULN2003A используемые для 12-вольтовых периферийных устройств не влияют на 5-вольтовые сигналы.

Входной преобразователь в левой части рис. 7 описан в разделе Входной преобразователь на микросхеме ULN2003A, глава 3.4.

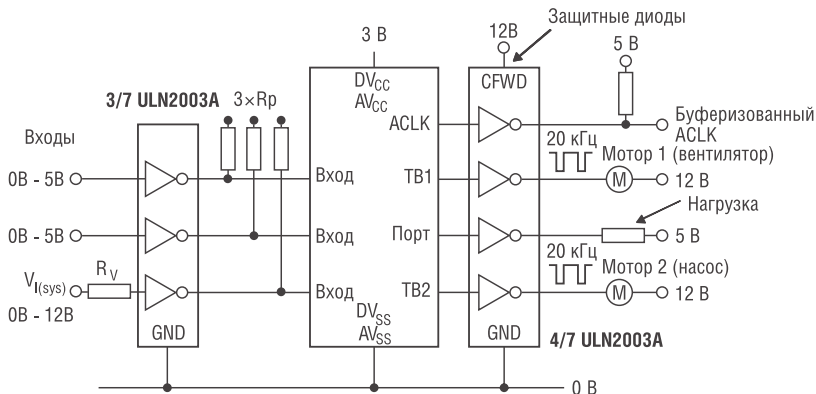


Рис. 7. Интерфейсы с сильноточными выходными буферами ULN2003

4.4. Выходной преобразователь на операционном усилителе

Согласование с системами, имеющими напряжение питания $V(\text{sys})$ до 16 В можно реализовать на базе счетверённого ОУ TLC27L4. Резистивные делители на инвертирующих входах TLC27L4 образуют напряжения ≈ 1.5 В. Номиналы резисторов R_{n1} и R_{n2} должны удовлетворять условиям:

$$R_{B1} \parallel R_{B2} \ll \frac{DV_{CC}}{\sum I_{Ikg(0p)}} = \frac{3B}{4 \times 0,7nA} = 1,07 \Omega M$$

4.5. Выходные преобразователи на интегральных микросхемах

В качестве выходных преобразователей 3-вольтовых с 5-вольтовыми схемами можно использовать практически любые TTL-совместимые ИС, например, такие, как семейства НСТ и АНСТ. То же самое справедливо и для биполярных ИС. Если источник питания 5 В может выключаться в то время, как 3-вольтовый источник остаётся включенным (например, в ситуации, изображённой на рис. 11), то для коммутации V_{cc} можно использовать только ИС, не содержащие встроенных защитных диодов. В противном случае, через защитный диод будет протекать ток с 3-вольтового источника питания на землю. Это означает, что в таких случаях можно использовать только семейство АНСТ или биполярные ИС.

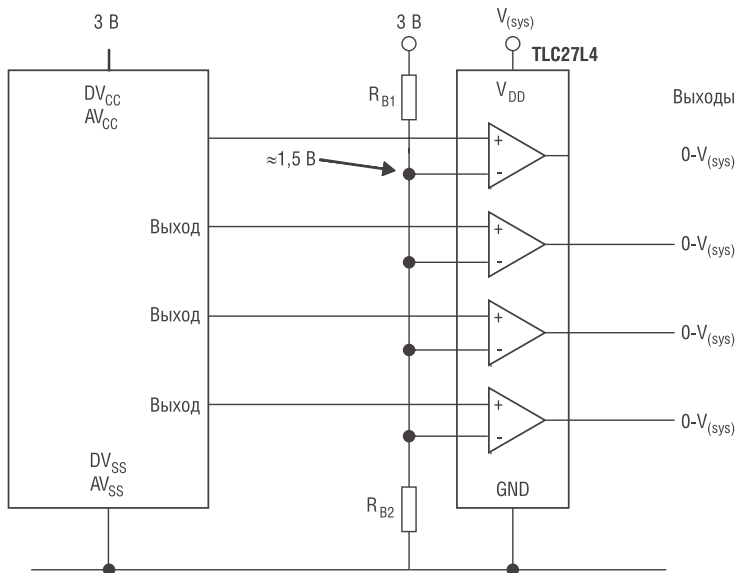


Рис. 8. Выходной преобразователь на операционном усилителе

5. Двухнаправленные преобразователи

5.1. Простой двухнаправленный преобразователь на ОУ

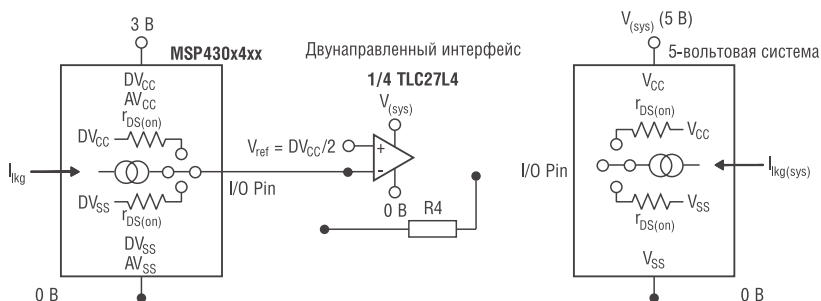


Рис. 9. Двухнаправленный интерфейс между 3-вольтовой и 5-вольтовой системами

Интерфейс, изображенный на рис. 9 можно использовать в случае, если требуется двунаправленный обмен между MSP430 и 5-вольтовыми системами. Операционный усилитель исполняет роль триггера: порт ввода-вывода, работающий в режиме выхода, контролирует состояние триггера. Второй порт ввода-вывода должен быть сконфигурирован как вход.

Формулы расчёта резисторов R3, R4 и R5 для наихудших случаев:

$$\frac{R4}{R5} < (1 - 2p) \times \left(\frac{V_{(sysH)} \min}{V_{ref(max)}} - 1 \right) \quad \frac{R4}{R5} > (1 + 2p) \times \left(\frac{V_{(sysH)} \max}{DV_{CC(min)}} - 1 \right)$$

Следующее неравенство гарантирует высокий уровень на входе MSP430 с учётом токов утечки MSP430 и операционного усилителя I_{lkg} и $I_{lkg(Op)}$:

$$R4 < \frac{V_{(sysH)} \min - V_{ref(max)} \times \left(1 + \frac{R4}{R5} \times (1 + 2p) \right)}{(I_{lkg} + I_{lkg(Op)}) \times (1 + p)}$$

Для V_{ref} – опорное напряжение для детектирования входного уровня [В]

I_{lkg} – ток утечки входа MSP430 [А]

$I_{lkg(Op)}$ – ток утечки входа ОУ [А]

$I_{lkg(sys)}$ – ток утечки входа 5-вольтовой системы [А]

Пример: двунаправленный интерфейс со следующими параметрами:

$V_{(sys)} = 5 \text{ В} \pm 10\%$, $V_{(sysH)} \min = 4 \text{ В}$, $V_{(sysH)} \max = 3,5 \text{ В}$, $I_{lkg(sys)} = \pm 1 \text{ мкА}$,

$I_{lkg(Op)} = \pm 700 \text{ нА}$, $I_{lkg} = \pm 50 \text{ нА}$, $V_{ref} = 1,5 \text{ В} \pm 5\%$.

Начальный разброс резистора $p = \pm 5\%$.

Минимальное напряжение питания $DV_{CC(min)} = 2,7 \text{ В}$ ($3,0 \text{ В} - 10\%$).

$$\frac{R4}{R5} < (1 - 2p) \times \left(\frac{V_{(sysH)} \min}{V_{ref(max)}} - 1 \right) = (1 - 2 \times 0,05) \times \left(\frac{4\text{В}}{1,575\text{В}} - 1 \right) = 1,386$$

$$\frac{R4}{R5} > (1 + 2p) \times \left(\frac{V_{(sysH)} \max}{DV_{CC(min)}} - 1 \right) = (1 + 2 \times 0,05) \times \left(\frac{5,5\text{В}}{2,7\text{В}} - 1 \right) = 1,14$$

$$\frac{R4}{R5} = \frac{1,38 + 1,14}{2} = 1,26$$

$$R4 < \frac{V_{(sysH)} \min - V_{ref(max)} \times \left(1 + \frac{R4}{R5} \times (1 + 2p)\right)}{(I_{kg} + I_{kg(Op)}) \times (1 + p)} = \frac{4,0B - 1,575B \times (1 + 1,26 \times (1 + 2 \times 0,05))}{(50nA + 700pA) \times (1 + 0,05)} = 4,55M\Omega$$

$$R5 = \frac{R4}{1,26} = \frac{2M\Omega}{1,26} = 1,59M\Omega$$

$$R3 < \frac{V_{(sys)} \min - V_{(sys+)} \max}{(1 + p) \times \left(I_{kg(sys)} + \frac{V_{(sys+)} \max - V_{ref(min)}}{R4 \times (1 + p)}\right)} = \frac{4,5B - 3,5B}{(1 + 0,05) \times \left(1\mu A + \frac{3,5B - 1,425B}{2M\Omega \times (1 + 0,05)}\right)} = 479k\Omega$$

Номиналы трёх резисторов равны: R3 = 330 кОм, R4 = 2 МОм, R5 = 1,6 МОм.

5.2. Двухнаправленный преобразователь на интегральных микросхемах

Для двухнаправленного интерфейса можно применять специализированные преобразователи уровней, такие, как SN74LVCC4245A. Эта ИМС представляет собой 8-битный двухнаправленный преобразователь 5-вольтовых уровней для всего порта MSP430. На рис. 10 показан пример использования этой ИС.

MSP430 определяет направление передачи при помощи вывода DIR. Шина A может быть при необходимости отключена от шины B при помощи вывода OE.

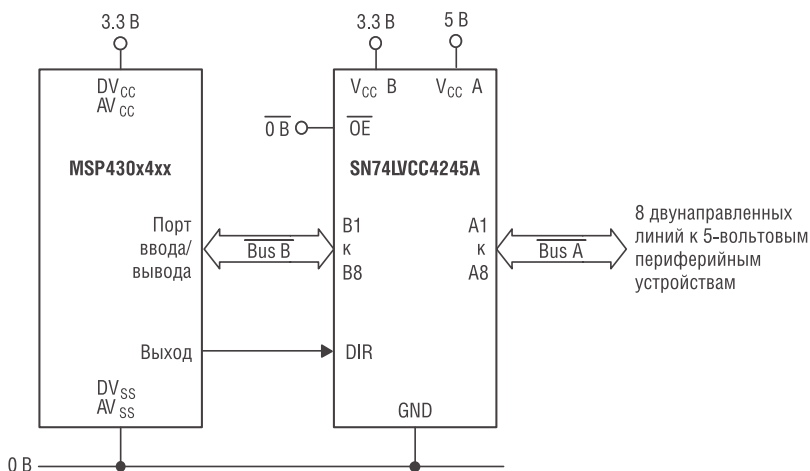


Рис. 10. Двухнаправленный преобразователь на интегральных микросхемах

6. Источники питания

Примечание:

Формулы для расчёта рассматриваемых ниже источников питания приведены в разделе Источники питания для систем на базе MSP430 (Power Supplies for MSP430 Systems) в [1]. Также в этом разделе рассмотрены другие типы источников питания, например трансформаторные ИП и приведены соответствующие формулы.

На рис. 11 показан конденсаторный источник питания на два выходных напряжения, $V_{CC1} = 3 \text{ В}$ и $V_{CC2} = 5 \text{ В}$. Если выходного тока TLC27L4s недостаточно, то можно использовать выходной буфер NPN, как нарисовано на рис. 12.

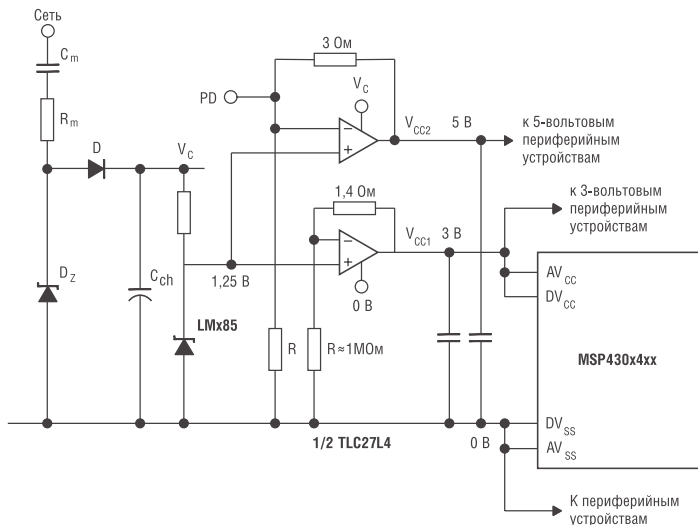


Рис. 11. Конденсаторный источник питания с двумя выходными напряжениями

Используя вывод выключения PD, MSP430 может отключать 5-вольтовый источник на время нахождения в режиме пониженного потребления (LPM3):

- Активный режим (Active mode): выход MSP430 подключенный к PD переключается в высокоимпедансное состояние
- Режим пониженного энергопотребления (LPM3): выход MSP430 подключенный к PD подключается к DVCC. На выходе 5-вольтового регулятора напряжение, близкое к 0.

На рис. 12 показан источник питания на два выходных напряжения $V_{cc1} = 3 \text{ В}$ и $V_{cc2} = 5 \text{ В}$. 3-вольтовый источник питания буферизован NPN-транзистором для увеличения тока.

III. Интерфейс с внешними устройствами

Глава 3.
*Подключение MSP430
и температурного датчика TMP100
с интерфейсом I²C*

Подключение MSP430 и температурного датчика TMP100 с интерфейсом I²C

Zack Albus [slaa151.pdf (Англ.)]

В данном примере применения описывается аппаратный и программный интерфейс между 16-разрядным микроконтроллером MSP430F413 и цифровым температурным датчиком TMP100. Система, построенная на основе этих микросхем производства компании Texas Instruments, способна работать от одной 3 В батарейки более 10 лет. В нижеследующих разделах описано как аппаратно соединить эти микросхемы и программно реализовать протокол обмена по двухпроводной I²C шине. Несмотря на то, что в данном примере применения используется температурный датчик TMP100, описанный интерфейс может быть применен для подключения к любому микроконтроллеру семейства MSP430 любого периферийного устройства с подчиненным I²C интерфейсом.

1. Введение

Во множестве систем требуется отслеживать и измерять температуру. Часто такие измерения должны выполняться удаленно и при использовании автономного источника питания, а, следовательно, для увеличения срока службы потреблять система должна минимально. В большинстве систем измерение температуры производится при помощи дискретных транзисторов, терморезисторов или термопар и «систем на кристалле» (SoC). TMP100 представляет собой SoC, а при объедине-

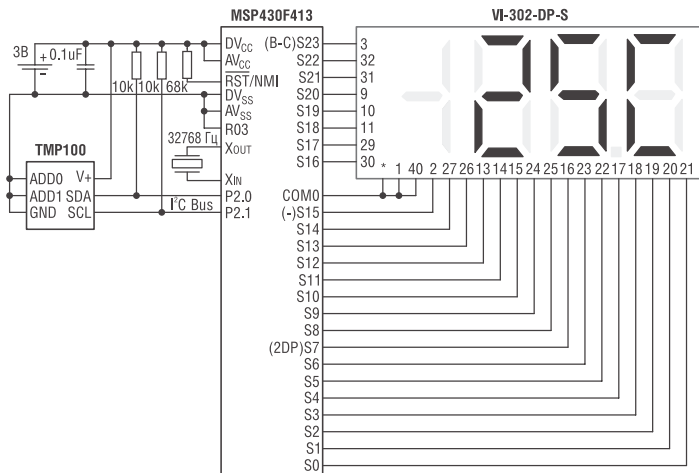


Рис. 1. Схема соединения MSP430F413 и TMP100

нии со сверхнизко потребляющим микроконтроллером MSP430, как это показано на рисунке 1, становится полнофункциональной системой измерения температуры, способной работать от одной батарейки более 10 лет.

2. Теория функционирования

Система измерения температуры MSP430-TMP100 предназначена для постоянного отображения температуры датчика TMP100. В режиме одновибратора TMP100 после каждого измерения переходит в дежурный режим, ток потребления в котором снижается до 0.1 мкА. Вместе с MSP430, который потребляет в режиме LPM3 всего 0,9 мкА, и 3.5 цифровым статическим ЖКИ получается система измерения температуры с разрешающей способностью 0.5°C и средним потреблением менее 2,45 мкА. Диаграмма потребления тока системой приведена на рисунке 2.

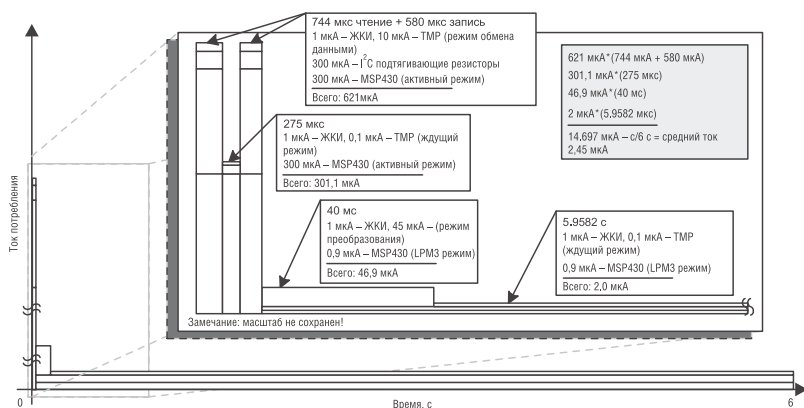


Рис. 2. Диаграмма потребления системой тока

Питание системы осуществляется от одной 3 В батарейки типа CR-2032, которая имеет емкость порядка 220 мА*часов. При среднем токе потребления, равном 2.45 мкА (см. рисунок 2), измерение и индикация температуры может осуществляться в течение более чем 10 лет:

$$220 \text{ мА} \cdot \text{ч} / 2.45 \text{ мкА} = 89\,796 \text{ часов} = 10.25 \text{ лет}$$

Для обеспечения этого пришлось позаботиться о минимальном количестве рабочих циклов MSP430. Время выхода из дежурного режима, равное 6 мкс, позволило MSP430 управлять цифровым температурным датчиком TMP100, работающем в режиме однократного преобразования. На рисунке 3 приведена блок-схема алгоритма работы MSP430 и цифрового температурного датчика.

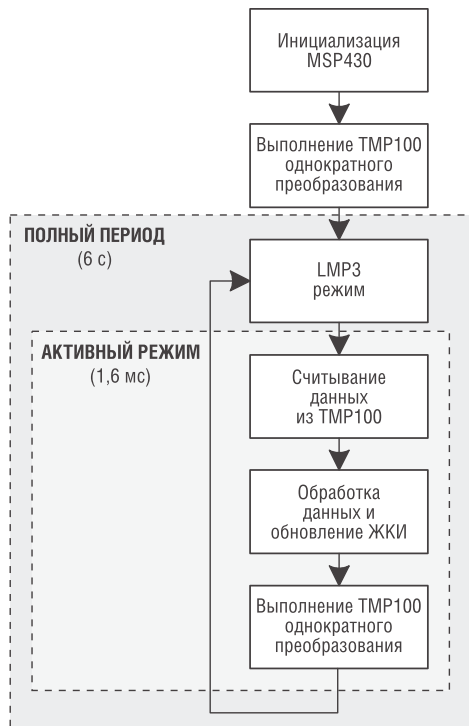


Рис. 3. Блок-схема алгоритма работы MSP430F413/TMP100

За счет снижения времени активности MSP430 в 1,6 мс цикле преобразования можно добиться уменьшения потребления. Малое время выхода из дежурного режима LPM3 позволяет MSP430 выполнять требуемые задачи и возвращаться в дежурный режим за очень короткий промежуток времени. В дежурном режиме микроконтроллер ожидает появления прерывания от таймера и по его возникновению начинает выполнять основной алгоритм. Дальнейшее описание работы системы дается в разрезе разделов, соответствующих модулям прикладной программы MSP430 на ассемблере. Обратите внимание, что дальнейшая оптимизация кода возможна при снижении количества вызываемых подпрограмм. Для упрощения понимания программа разбита на подпрограммы. К существующим подпрограммам, приведенным на рисунке 3, можно добавить еще пользовательские подпрограммы, например подпрограмму перевода градусов Цельсия в градусы Фаренгейта или подпрограмму обмена данными с периферийными устройствами.

3. Аппаратный интерфейс

Как показано на рисунке 1, системный интерфейс между MSP430 и TMP100 требует дополнительных дискретных компонентов. Питание системы осуществляется от 3 В Li+ батарейки, а в качестве фильтра питания рекомендуется использовать 0.1 мкФ конденсатор. Вход сброса RST рекомендуется подключить через 68 кОм резистор, а 32.768 кГц используется для формирования тактовых импульсов в дежурном режиме. Два дополнительных 10 кОм резистора используются в качестве подтягивающих резисторов линий I²C интерфейса.

Индикация измеренной температуры осуществляется при помощи 3.5 цифрового статического ЖКИ. Встроенный драйвер ЖКИ MSP430F413 позволяет обслуживать до 24 сегментов, а 4 выхода обслуживания общих линий расширяет максимально возможное количество обслуживаемых сегментов до 96. В данном примере применения использован 24 сегментный статический ЖКИ производства компании Varitonix. Хотя в данном примере применения использован микроконтроллер MSP430F413, но может быть использован и любой другой микроконтроллер семейства MSP430 серии 4xx.

4. Работа системы

I²C интерфейс с TMP100 реализован полностью программно и использует всего две линии ввода/вывода портов общего назначения MSP430. За счет наличия у TMP100 поддержки конфигурирования адреса, к одной I²C шине можно подключить до восьми таких приборов. В данном примере применения входы установки адреса подключены к общей линии, и поэтому семибитный адрес TMP100 имеет значение 1001000.

При любых запросах к TMP100 сначала необходимо послать адрес (7 бит + бит записи WR), а потом 8 битный указатель адреса. Указатель адреса определяет один из трех обновляемых микроконтроллером регистров. В этом примере применения конфигурационный регистр TMP100 обновляется в каждом цикле основной программы и устанавливает цифровой температурный датчик в режим однократного преобразования. Точно также осуществляется считывание данных.

Запрос на считывание данных из TMP100, как и в режиме записи, начинается с передачи 16 битных данных. Сначала передается адрес датчика, а потом 8 битный указатель регистра TMP100. Потом снова передается адрес датчика, после чего считываются данные. При второй передаче адреса бит RD/WR в байте адреса должен быть установлен, что указывает на предстоящее считывание данных.

Результат преобразования передается в виде 16 битных данных. Оба байта передаются старшим значащим битом вперед. Имеется возможность настройки представления результата преобразования в виде двоичного числа с количеством разрядов от 9 до 12. По умолчанию данные представляются в 9 битном формате, который обеспечивает разрешение 0.5°C/МЗБ. Этот формат и используется в опи-

сываемом примере применения. Остальные младшие значащие биты, передаваемые TMP100 считываются как ноль и могут игнорироваться.

Для универсального измерителя температуры разрешающая способность в 1°С вполне достаточна. В этом случае можно не считывать второй байт. Но тогда не осуществляется преобразования градусов Цельсия в градусы Фаренгейта с приемлемой точностью, зато снижается время работы системы в активном режиме.

После принятия данных от TMP100, MSP430 выполняет их преобразование и отображает измеренную температуру на ЖКИ. При этом осуществляется обработка СЗБ, несущего информацию о знаке измеренной температуры, и преобразование двоичного кода в двоично-десятичный, который и отображается на ЖКИ.

По окончании обработки данных MSP430 переходит в дежурный режим LPM3, ток потребления в котором снижается до 2.0 мкА. Через 6 секунд формируется прерывание по переполнению таймера, которое выводит микроконтроллер из дежурного режима и основная программа начинает выполняться с начала. Для генерации 2 секундного сигнала прерывания используется модуль основного таймера, который потребляет 3.55 мкА, что позволит системе работать от 3 В батарейки всего 7.5 лет. Для увеличения срока службы можно снизить частоту сигнала прерывания или использовать модуль Таймера А. В любом случае, частота прерывания полностью программируема и может быть отрегулирована в соответствии с требованиями к системе.

5. Заключение

Как было сказано раньше, измерение температуры может осуществляться различными способами. Способ, применяемый в конкретной задаче, зависит от физических ограничений, наложенных на систему. Часто портативность и низкое потребление являются основными ограничительными факторами. Описанная система, состоящая из MSP430 и TMP100, является эффективным решением такой задачи.

При помощи описанных аппаратных и программных решений можно реализовать законченный низкопотребляющий измеритель температуры. Эта система не просто измеряет температуру, а делает это так, что может работать от 3 В Li+ батарейки более 10 лет.

Ссылки:

1. MSP430F413 Device Datasheet, Literature Number SLAS340C
2. MSP430 4xx User's Guide, Literature Number SLAU056B
3. TMP100 Datasheet, Literature Number SBOS231B
4. "THE I2C-BUS SPECIFICATION VERSION 2.1 JANUARY 2000", Philips Semiconductors
5. VI-302 LCD Datasheet, Varitronix International Ltd

III. Интерфейс с внешними устройствами

Глава 4. *Реализация функции UART с использованием таймера A3*

Реализация функции UART с использованием таймера A3

Mark Buccini [slaa078a.pdf (Англ.)]

В данном примере применения описывается как при помощи модуля таймера timer_A3 реализовать функцию UART. Приведенные примеры предназначены для микроконтроллеров семейства MSP430x11x (1), но могут быть адаптированы для любых микроконтроллеров семейства MSP430, которые содержат модуль таймера timer_A. UART реализован на основе аппаратного модуля таймера timer_A3 и прикладной программы. Реализованный приемопередатчик является полудуплексным, управляемым событиями, поддерживает протокол 8N1 и работает на скоростях от 1200 до 115200 бод.

Введение

В микроконтроллере MSP430x11x (1) возможность ведения последовательной асинхронной передачи данных может быть реализована за счет использования встроенного модуля таймера timer_A3. В данном примере применения описывается несколько функций UART, демонстрирующих RS232 интерфейс между микроконтроллером с Flash памятью MSP430F1121 и персональным компьютером (см. рисунок 1). Также описывается как использовать аппаратный модуль таймера timer_A3, который обеспечивает автоматическое детекти-

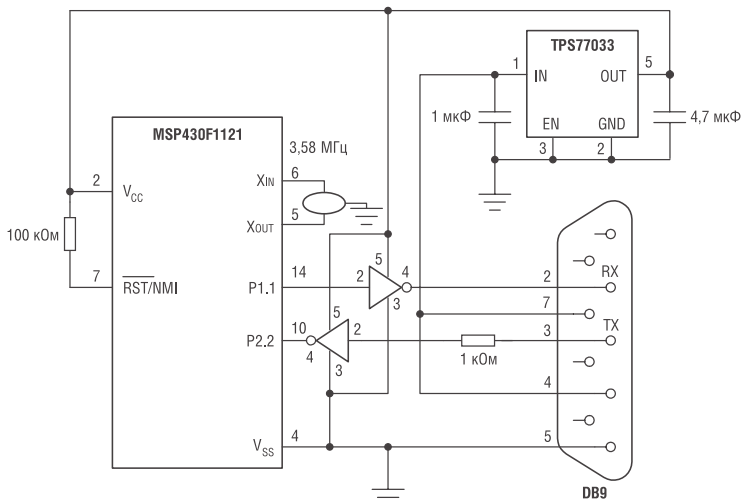


Рис. 1. Демонстрационная схема реализации UART на микроконтроллере MSP430x11x (1)

рование стартового условия, формирует заданную частоту передачи данных и захватывает биты данных. Аппаратный модуль таймера timer_A3 позволяет существенно сократить код прикладной программы и уменьшить загрузку вычислительного ядра микроконтроллера. Также параметры этого модуля позволяют UART работать в реальном времени в фоновом режиме.

Описание UART, реализованного на модуле таймера Timer_A микроконтроллера MSP430x11x (1)

Как показано в рисунке 1, MSP430F1121 ведет последовательный обмен данными с другой системой, в нашем случае с ПК, по RS232 интерфейсу. Обмен данными между системами ведется по трехпроводному кабелю, который содержит линии: передачи, приема и общую. При этом используется протокол 8N1: каждое сообщение содержит 8 информационных бит, не содержит бита проверки на четность и заканчивается одним стоп-битом. Для поддержки других протоколов нужно внести небольшие изменения в функционал UART. Описываемый UART использует нулевой регистр захвата-сравнения (CCR0), один из трех доступных, и таймер timer_A3. CCR0 используется для обнаружения старт-бита, задания скорости обмена данными и захвата данных. Два незадействованных регистра захвата-сравнения могут использоваться для других задач. Регистр CCR0 был выбран совершенно произвольно. Для реализации UART могут быть выбраны любые регистры CCRx. При помощи функции выбора периферийных задач линии P1.1 и P2.2 портов общего назначения подключены к CCR0 timer_A3. Линия P1.1 выбрана для передачи информации, а P2.2 – для приема. Периферийные задачи для выводов назначены при помощи регистров P1SEL и P2SEL. Поскольку вывод P1.1 используется для передачи, то он должен быть настроен на работу в качестве выхода. Осуществляется это установкой соответствующего бита регистра установка направленности линий портов (P1DIR). P2.2 должен работать как вход. По умолчанию у микроконтроллеров семейства MSP430 все выводы настроены на работу в режиме входа. Timer_A3 настроен на работу в непрерывном режиме, что позволяет использовать его одновременно и для других функций. Регистр R4 используется в качестве буфера приемопередатчика (RXTXData). Регистр R5 (BitCnt) используется в качестве регистра подстройки смещения. Регистры R4 и R5 были выбраны произвольно. Для выполнения этих функций могут быть использованы любые регистры вычислительного ядра или ячейки ОЗУ.

В режиме приема нулевой управляющий регистр захвата-сравнения (CCTL0) сначала устанавливает модуль CCR0 в режим захвата данных по спадающему фронту сигнала на входе P2.2. Спадающий фронт на этом входе указывает на начала старт-бита. После того, как UART установлен в режим приема данных, от вычислительного ядра не требуется никаких ресурсов для обеспечения готовности принятия кодового слова. Ресурсы вычислительного ядра будут востребованы только после

поступления старт-бита на вход P2.2. По спадающему фронту на входе P2.2 текущее значение работающего в свободном режиме таймера timer_A3 (TAR) сохраняется в регистре CCR0. Захват данных осуществляется аппаратными средствами таймера timer_A3 без использования программных средств. Также формируется прерывание для вычислительного ядра. Время задержки обработки прерывания не имеет значения, так как момент появления спадающего фронта точно зафиксирован в регистре CCR0. После обнаружения спадающего фронта старт-бита программное обеспечение перестраивает CCTLO таким образом, чтобы CCR0 сравнивал данные в середине первого информационного бита. Смещение на 1.5 бита осуществляется за счет увеличения содержимого CCR0 на соответствующее значение. Полученные данные защелкиваются в SCCI. SCCI – считываемая защелка регистра CCTLO. При работе UART SCCI синхронно с CCR0 защелкивает логический уровень, присутствующий на входе P2.2. UART принимает данные, защелкнутые в SCCI. Программное обеспечение не контролирует непосредственно состояние входа P2.2.

```
RX_Bit    bit.w    #SCCI, &CCTL0    ;получение бита,
                                     ;защелкнутого в SCCI
                                     ;сохранение принятого
                                     ;бита
rrc.b     RXTXData
```

После принятия первого бита содержимое CCR0 увеличивается на величину, позволяющую выполнить захват в середине следующего информационного бита. 8 информационных бит сохраняются программно в регистр RXTXData.

Задача передачи данных более простая, так как MSP430 сам решает, когда передавать данные, и при этом не требуется осуществлять поиск никаких фронтов сигналов. Данные, сохраненные в буферном регистре RXTXData, аппаратно сдвигаются на выход P1.1 при помощи регистра CCR0 таймера timer_A.

CCR0 перестраивается в режим сравнения для передачи следующего бита при помощи выхода режимно-управляемых битов в CCTLO. Режимно-управляющие биты определяют состояние выхода, которое будет установлено аппаратно в момент последующего сравнения. При этом отпадает необходимость программно отслеживать момент установки значения следующего выходного бита и выполнять саму установку. Программно осуществляется только загрузка значения следующего передаваемого бита:

```
TX_Bit    rra.b     RXTXData          ;смещение LSB для
                                     ;переноса
                                     ;переход если бит = 1
jc        TX_Mark
TX_Space   bis.w     #OUTMOD2, &CCTL0;область TX
ret;
TX_Comp    bic.w     #CCIE, &CCTL0    ;все биты приняты,
                                     ;блокировка прерывания
TX_Mark    bic.w     #OUTMOD2, &CCTL0;метка TX
ret;
```

Перед передачей каждого бита программа прокручивает регистр RXTXData для установки бита переноса. Выполняется соответствующий переход и подготавливается режим выхода CCTL0, после чего к CCR0 добавляется длительность одного бита.

Вычисление скорости передачи данных

CCR0 таймера Timer_A3 используется для формирования скорости передачи данных. В зависимости от требуемой скорости рассчитывается необходимый интервал длительности бита (Bitime). Bitime – величина, на которую должен увеличить свое состояние таймер timer_A3 за время, равное длительности одного бита (см. рисунок 2). Величина Bitime рассчитывается как отношение частоты увеличения таймера timer_A3 и требуемой скорости передачи данных.

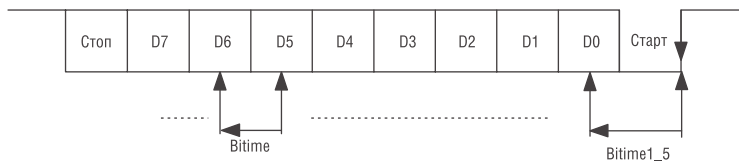


Рис. 2. Информационное слово в формате UART8N1

Timer_A3 имеет несколько доступных источников синхроимпульсов и делителей (см. техническое описание микроконтроллера). К возможным источникам синхросигнала таймера timer_A3 микроконтроллера MSP430x11x (1) относятся источник вспомогательных синхроимпульсов (ACLK), источник системных синхроимпульсов (SMCLK) и два источника внешних синхроимпульсов.

Пример: Рассмотрим формирование скорости передачи данных 9600 бод при помощи таймера timer_A3, синхронизирующегося от источника вспомогательных синхроимпульсов ACLK, который работает от 3.579545 МГц кварцевого резонатора:

$$\text{Bittime} = 3\,579\,545 / 9600 = 372.9 \sim 373$$

$$\text{Фактическая скорость передачи данных} = 3\,579\,545 / 373 = 9597$$

Значение 373 используется потому, что только целочисленное значение Bitime может быть добавлено к CCR0. Погрешность формирования скорости передачи данных 9600 бод при частоте синхроимпульсов 3.579545 МГц, возникающая вследствие округления Bitime, не превышает 0.03 % на бит.

Программная загрузка

При выборе программной или аппаратной реализации UART разработчики исходят из степени загруженности системы в том или ином случае. Описанная в данном примере применения реализация UART использует возможности ап-

паратного таймера timer_A3, что позволяет снизить загруженность вычислительного ядра. Модуль таймера Timer_A3 автоматически определяет спадающий фронт стартового бита и защелкивает содержимое информационных бит. Для приема или передачи одного бита требуется не более 26 программных циклов (включая циклы подпрограммы обработки прерывания). Таким образом, степень загруженности вычислительного ядра для обслуживания реализованного UART является функцией, зависящей от частоты основных синхросигналов и скорости передачи данных. При использовании частоты MCLK, равной 3.58 МГц, для ведения обмена данными на скорости 9600 бод загрузка составит:

Программная загрузка = (26 циклов вычислительного ядра) * (9600 бод) / 3 579 545 = 6.9 %

Демонстрационная схема

Демонстрационная схема, приведенная на рисунке 1, питается непосредственно от последовательного порта ПК, при этом 3.3 В напряжение питания микроконтроллера формируется стабилизатором с низким падением напряжения TPS76033. Для преобразования уровней сигналов интерфейса использованы два инвертора SN74AHC1G04 производства компании TI. Если необходимо реализовать интерфейс, полностью отвечающий стандарту RS232, то необходимо использовать низкопотребляющие 3 В преобразователи уровня типа MAX2331, которые также производятся компанией TI. Вход сброса подтянут к линии питания, а задающий генератор работает от 3.58 МГц керамического резонатора.

Демонстрационная управляющая программа 11x1_uart1.s43

Демонстрационная управляющая программа 11x1_uart1.s43 предназначена для использования в качестве управляющей программы микроконтроллера, присутствующего в схеме на рисунке 1. Это программа формирования эха принятого сигнала. Т.е. ПК передает данные в демонстрационную схему, которая после принятия возвращает их назад. При инициализации происходит настройка всех выводов и синхронизация всех генераторов с кварцевым генератором LFXT1. LFXT1 настроен на работу в высокочастотном (HF) режиме. Если, как в этом примере, для формирования MCLK используется ВЧ резонатор, то он должен быть стабильным или же в режиме защиты от сбоев будет использоваться DCOCLK для формирования MCLK. OSCFAULT может опрашиваться для гарантирования стабильности работы автогенератора перед выбором его для формирования MCLK.

```
SetupBC    bis.b    #XTS, &BCSCTL1; ACLK = LFXT1 ВЧ XTAL
SetupOsc   bic.b    #OIFG, &IFG1 ; сброс флага ошибки
                                   ; автогенератора

                mov.b    #0FFh, R15
SetupOsc1   dec.b    R15           ; формирование задержки
                                   ; для гарантирования
                                   ; запуска
```

```

jnz      SetupOsc1
bit.b    #OIFG, &IFG1 ;флаг ошибки
                        ;автогенератора
                        ;установлен?

jnz      SetupOsc;
bis.b    #SELM1+SELM0, &BCSCTL2;MCLK = LFXT1

```

Режим защиты от сбоев описан в руководстве по использованию микроконтроллеров семейства MSP430x1xx (документ SLAU049). Основная программа инициализирует UART при помощи подпрограммы RX_Ready, а затем переводит микроконтроллер в режим пониженного потребления LPM0, в котором вычислительное ядро отключено. В этом режиме активны только timer_A3 и ACLK. Даже при отключенном вычислительном ядре UART работает в режиме ожидания поступающих данных как фоновая подпрограмма обслуживания прерывания. После принятия приемопередатчиком информационного слова подпрограмма обслуживания прерывания UART активизирует вычислительное ядро и запускает основную программу. Подпрограмма передачи данных возвращает в ПК принятые данные, которые и отображаются на его мониторе. После этого цикл начинается сначала: микроконтроллер вновь переходит в режим ожидания поступления данных. Данная демонстрационная программа оптимизирована с точки зрения повышения быстродействия. В CCR0_ISR BitCnt используется для автоматического увеличения адреса косвенной адресации, что требуется для обработки передаваемых и принимаемых битов. Программе не требуется опрашивать флаги или уменьшать регистры для определения какой механизм нужно использовать. Автоматическое увеличение адреса используется совместно с поисковой таблицей для мгновенного определения адреса подпрограммы.

```

add.w    #Bitime, &CCR0;Bitime для
                        ;следующего бита
br       @BitCnt +     ;переход к соответствую-
                        ;щей подпрограмме

```

Преимущество автоматического увеличения адреса состоит в быстродействии и эффективности в условиях ограниченного количества доступных программных циклов.

Демонстрационная управляющая программа 11x1_uart2.s43 – использование DCO для задания скорости обмена данными

Демонстрационная программа 11x1_uart2.s43 показывает, как реализовать высокоскоростной UART на микроконтроллере семейства MSP430, работающем в режиме сверх низкого потребления и использующего только один 32 768 кГц часовой кварцевый резонатор. Для задания скорости обмена данными может использоваться встроенный в MSP430 задающий генератор с цифровой регулировкой (DCO). DCO – RC генератор с цифровой регулировкой, способный рабо-

тать в диапазоне частот от 100 кГц до 5 МГц и выше и имеющий время запуска менее 6 мкс. DCO позволяет вести обмен данными даже в режиме сверх низкого потребления: спадающий фронт старт-бита запускает отключенный DCO менее чем за 6 мкс. Демонстрационная программа 11x1_uart2.s43 позволяет вести обмен данными на скорости 9600 бод, используя DCO для задания скорости обмена данными. 32 768 Гц часовой кварцевый резонатор заменяет 3.58 МГц резонатор, использованный в схеме на рисунке 1. DCO работает на частоте 1 228 800 Гц и используется для формирования подсистемных синхроимпульсов (SMCLK), являясь источником синхросигнала для модуля таймера timer_A3. DCO настраивается на высокую частоту при помощи программной системы ФАПЧ. Программная ФАПЧ считает количество синхроимпульсов SMCLK (синхросигнал, формируемый DCO) за период сигнала ACLK (синхросигнал, формируемый 32 768 Гц кварцевым генератором). После этого осуществляется регулировка частоты DCO. при стабильных температуре и напряжении питания частота DCO остается неизменной. Программная ФАПЧ может быть запущена в любое время для подстройки частоты DCO. Подробнее о программной ФАПЧ можно прочитать в техническом описании микроконтроллеров семейства MSP430x11x1 (документ SLAS241). В данном примере основная программа выполняется в режиме LPM3, типовой ток потребления в котором менее 2 мкА, при этом UART готов к приему данных. Эта программа, также как и описанная выше, формирует эхо принятого сигнала. В данном примере для приема кодовой комбинации используется регистр BitCnt.

Демонстрационная управляющая программа 11x1_uart3. s43 – использование 32 кГц кварцевого резонатора для задания скорости обмена данными

В некоторых случаях источником синхросигнала для таймера timer_A3 должен быть 32 768 Гц часовой кварцевый резонатор. В этом случае также можно реализовать UART, но работающий на более низкой скорости, что позволит избежать ошибок принятия данных. Демонстрационная программа 11x1_uart3. s43 позволяет реализовать UART, работающий на скорости 2400 бод. Программная ФАПЧ для подстройки DCO не требуется, так как синхронизация UART осуществляется от 32 кГц кварцевого автогенератора. В активном режиме вычислительное ядро синхронизируется от высокочастотного DCO. Также в этом примере, регистры RXTXData и BitCnt содержатся в ОЗУ.

Ссылки:

1. MSP430x11x1 Mixed Signal Microcontroller, Data Sheet, Texas Instruments Literature Number SLAS241.
2. MSP430x1xx Family, User's Guide, Texas Instruments Literature Number SLAU049.

III. Интерфейс с внешними устройствами

Глава 5.
*Подключение MSP430
к EEPROM серии 24Схх
с помощью I²C-интерфейса*

Подключение I²C-интерфейса к ЭСПЗУ серии 24Схх

Brian Merritt [slaa115.pdf (Англ.)]

Данный пример применения описывает как, используя стандартную шину I²C, реализовать интерфейс между микроконтроллером семейства MSP430 и внешней EEPROM памятью. Приведенные схема включения и управляющая программа могут быть использованы для обеспечения интерфейса между любым микроконтроллером семейства MSP430 и EEPROM памятью или другим внешним устройством с I²C интерфейсом.

Введение

Микроконтроллеры семейства MSP430 имеют программируемую в схеме Flash или OTP память. Но многим приложениям требуется дополнительная внешняя EEPROM память. Такая потребность может возникнуть при большом размере прикладной программы или при необходимости хранения больших информационных массивов. У микроконтроллеров с OTP памятью или ROM памятью EEPROM может понадобиться для хранения обновленного программного обеспечения, калибровочных констант, серийных номеров или данных датчиков. Использование I²C шины – наиболее часто используемый метод ведения обмена данными между EEPROM памятью и микроконтроллерами.

Микроконтроллеры семейства MSP430 способны эмулировать выход с открытым коллектором, который необходим для реализации I²C интерфейса. В данном примере применения описывается, как реализовать такой интерфейс между микроконтроллером MSP430x11x1 и 512 байтной EEPROM памятью 24LC04. Для реализации интерфейса не требуется дополнительных специфических приборов, а только используются аппаратные периферийные модули MSP430. Прикладная программа демонстрационной схемы считывает и записывает случайные ячейки внешней EEPROM памяти.

Прикладная программа разработана для ведения обмена данными с внешней EEPROM памятью, но небольшие доработки позволят вести обмен данными и с другими приборами, имеющими I²C интерфейс. Для реализации I²C интерфейса задействовано всего 420 байт памяти программы. Остальная часть памяти программы используется для обеспечения интерфейса микроконтроллера с ПК (пользовательского интерфейса). Для этой подпрограммы требуется около 520 байт памяти программы. Для реализации I²C интерфейса пользовательский интерфейс не нужен, он реализован только для демонстрации работы устройства.

Теория функционирования

Используя две любые доступные линии портов ввода-вывода общего назначения, MSP430 способен эмулировать выходы с открытыми коллекторами для

управления линиями SDA и SCL интерфейса I²C. Это достигается за счет установки для этих выводов низкого выходного сигнала и использования прямой установки направленности выводов для задания логического уровня сигнала на линии. Всякий раз, когда необходимо установить на выходе высокий логический уровень, устанавливается бит направленности линии порта, который переводит линию порта в режим входа. А подтягивающий резистор сигнальной линии устанавливает на ней высокий логический уровень. Если требуется установить на линии низкий уровень, то вывод переводится в режим выхода. А так как предварительно в порты был записан ноль, то на линии установится низкий логический уровень.

Перед записью случайного байта MSP430 посылает EEPROM стартовое условие. После этого передается управляющий байт. Управляющий байт состоит из кода прибора, к которому идет обращение, адреса блока памяти и бита определения последующей операции (см. рисунок 1). После этого EEPROM подтверждает получение байта. В ответ MSP430 передает адрес ячейки памяти, в которую будут записываться данные. EEPROM снова передает подтверждение приема. После этого MSP430 передает данные, которые записываются в указанную ячейку памяти. После того, как EEPROM подтвердит прием данных, MSP430 посылает условие останова.

1	0	1	0	X (A2)	X (A1)	A0	R/W
4-битный код для 24LC04				адрес блока памяти			чтение/ запись

Примечание: Биты, помеченные "X" не имеют значения для семейства 24LC04

Рис. 1. Управляющий байт для 24LC04 EEPROM

Считывание произвольного байта несколько отличается от записи. Основное отличие состоит в том, что управляющий байт передается дважды. Сначала – для установки требуемого адреса, а потом – для инициализации чтения. В первом случае бит чтения/записи имеет значение логического нуля, а во втором – логической единицы.

Прикладная программа демонстрационной схемы разработана для обеспечения возможности считывания/записи 512 байт EEPROM памяти 24LC04, но микросхемы памяти с меньшим объемом также будут считываться при условии, что старшие биты будут игнорироваться. При небольших доработках этой программы можно осуществлять обмен данными и с микросхемами памяти большего объема.

Интерфейс пользователя обеспечивает связь демонстрационной схемы с терминальной программой, запущенной на ПК. Обмен данными ведется в ASCII

кодах. Терминальная программа должна быть настроена на ведение обмена данными со скоростью 9600 бод по протоколу 8N1.

Демонстрационная схема

Демонстрационная схема, приведенная на рисунке 2, питается непосредственно от RS232 последовательного порта персонального компьютера. Преобразование уровней сигналов выполняют два инвертора SN74AH1G04 производства компании TI. Если требуется полностью RS232-совместимый интерфейс, то можно использовать микросхему MAX3221, которая также выпускается компанией TI. Два резистора необходимы для привязки сигнальных линий к шине питания. TPS77033 – производимый компанией TI стабилизатор с низким падением напряжения, который формирует из сигналов последовательного порта напряжение питания микроконтроллера и памяти.

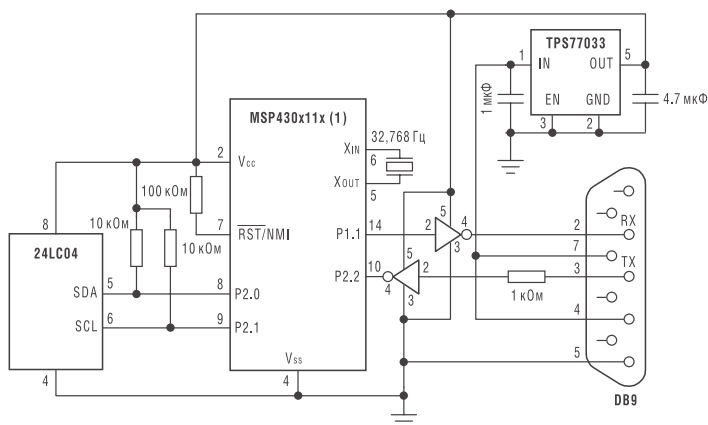


Рис. 2. Демонстрационная схема

Прикладная программа

Прикладная программа для MSP430 доступна в интернете по адресу <http://www.ti.com/sc/msp430>. Называется она I2C_EEmem. S43 и упакована в zip архив с таким же именем. Код программы написан на ассемблере в интегрированной среде разработки IAR Kickstart.

Ссылки:

1. MSP430x11x1 Data sheet (SLAS241C)
2. MSP430x1xx User's Guide (SLAU049)
3. 24xx Data sheets (various companies)

III. Интерфейс с внешними устройствами

Глава 6.
*Реализация шины SMBus на базе
микроконтроллера MSP430*

Реализация шины SMBus на базе микроконтроллера MSP430

Richard Baker [slla073.pdf (Англ.)]

Данный пример применения описывает программную реализацию шины системного управления (system management bus (SMBus)) на базе микроконтроллера MSP430. Рассмотрены протоколы ведущего (master) устройства, базирующегося на прерываниях ведомого (slave), а также примеры использования ведущего устройства. Шина SMBus является подвидом шины I²C и широко используется в смарт-батареях (smart batteries) и других системных устройствах.

1. Введение

SMBus это двухпроводной синхронный последовательный протокол, являющийся производным от I²C. Так же, как и I²C, он использует линии последовательного тактирования (SCL) и данных (SDA). Для исключения неопределённого состояния на линии используются подтягивающие (pullup) резисторы или источники тока, а устройства, подключенные к шине должны иметь выходы типа «открытый коллектор» либо «открытый сток». Это соответствует монтажному «И» всех устройств, что означает, что каждое из устройств может либо перевести шину в состояние лог. «0», либо освободить её. Так как подключённые к шине устройства могут быть запитаны от различных напряжений, они не должны переводить линию в состояние лог. «1».

Все устройства разделяются на ведущие (master) и ведомые (slave). Ведущее устройство может инициировать передачу и формирует тактовый сигнал. Ведомое принимает либо передаёт данные, но при этом процесс передачи инициируется ведущим. Устройства могут иметь как признаки ведущего и ведомого одновременно, так и быть исключительно одним либо другим. Например, ведомое устройство может в критических случаях становиться ведущим и передавать системе сообщения о неисправности. Алгоритм такого поведения целиком определяется конкретной областью применения.

Каждое устройство на шине имеет уникальный 7-битный адрес. Это позволяет подключать к линии до 128 устройств, однако некоторые адреса являются зарезервированными. Различные коммерчески доступные устройства имеют адреса, присвоенные координационным комитетом SMBus.

При приёме и передаче данных используются семь различных протоколов. Это «Быстрая команда» (Quick Command), «Отправить байт» (Send Byte), «Принять байт» (Receive Byte), «Записать Слово/Байт» (Write Byte/Word), «Прочитать

Слово/Байт» (Read Byte/Word), «Запрос состояния» (Process Call), и «Чтение/запись Блока» (Block Read/Write). Каждый протокол имеет собственный конкретный набор действий, но общая схема схожа для всех протоколов.

Передача всегда начинается с символа «Старт» (S). Этот символ передаётся переводом обеих линий в состояние лог. «1», последующим переводом линии SDA в состояние лог. «0» и, наконец, выставлением лог. «0» на шине SCL. После передачи стартовой последовательности, начинается передача 8-ми блоков, завершающаяся подтверждением (ACK). Подтверждение передаётся ведомым устройством путём перевода линии SDA в состояние лог. «0» во время передачи ведущим сигнала SCL. Если линия SDA не была переведена в такое состояние за время импульса на шине SCL, это состояние называется неподтверждением (NACK). Первый 8-битный блок является 7-битным адресом ведомого устройства вкупе с битом, определяющим направление передачи во всех протоколах, за исключением «Быстрой команды». В последнем же данный бит собственно является командой. После передачи адреса и направления, ведущий ожидает подтверждения от ведомого. При этом он формирует одиночный импульс на линии SCL и контролирует перевод ведомым в это время линии SDA в состояние лог. нуля. Если подтверждение не получено, ведущее устройство прекращает передачу и повторяет её позднее. Эта операция также может использоваться для определения присутствия ведомого на шине. После получения подтверждения, передаётся либо получается очередной блок из 8-ми бит, либо передаётся символ останова (P). Этот символ формируется переводом обеих линий в состояние лог. «0», последующим переводом линии SCL в состояние лог. «1» и завершается переводом в лог. «1» линии SDA. Остальные 8-битные блоки могут быть данными, командами либо следующей комбинацией адреса и бита чтения/записи. Точный формат каждого из протоколов можно найти в документе System Management Bus specification на сайте www.ti.com. Ввиду того, что на шине используется подключение устройств по принципу «монтажное «И»», она обладает механизмом арбитража. Если более одного ведущего устройства пытаются передавать данные, первый из них, «отпустивший» шину во время того, как оставшийся удерживает её в состоянии лог. «0» должен отдать арбитраж. Чтобы обеспечить эффективность такого метода, каждое ведущее устройство должно проверять состояние шины каждый раз после перевода её в состояние лог. «1». Если после этого шина удерживается в состоянии лог. «0», ведущее устройство должно прекратить передачу и повторить попытку позднее. В результате такого решения системы арбитража, устройства с меньшими адресами имеют приоритет над устройствами со старшими адресами. Если два ведущих пытаются начать передачу одновременно, тот из них, который передаёт старший адрес, первым освободит линию и потеряет, таким образом, приоритет.

Рабочими частотами шины являются 10 кГц и 100 кГц. Это аналогично шине I²C, за исключением того, что I²C не имеет ограничения скорости снизу, при этом устройства могут занимать шину на неограниченное время. SMBus имеет более жёсткие ограничения, ограничивая минимальную скорость. Несмотря на это, ведомые устройства не обязаны работать с той же частотой, что и ведущие. Единственным ограничением является минимальная скорость их работы в 10 кГц. Возможность работы с разной скоростью реализована следующим образом: если ведомому устройству требуется дополнительное время для завершения операции, оно принуждает ведущего к ожиданию удержанием линии SCL в состоянии лог. «0». Это называется «удлинением тактового сигнала» (clock-low extending). Ведущее устройство должно дождаться, пока ведомое «отпустит» линию SCL, но, если это займёт больше времени, чем разрешено спецификацией, произойдёт тайм-аут. Этим обеспечивается минимальная скорость шины. Дополнительную информацию по протоколу SMBus можно найти на форуме Smart Battery System Implementers Forum (SBS IF). Комитет SBS IF образован представителями фирм Benchmarq Microelectronics Inc., Duracell Inc., Energizer Power Systems, Intel Corporation, Linear Technology Corporation, Maxim Integrated Products, Mitsubishi Electric Corporation, National Semiconductor Corporation, Toshiba Battery Co., и Varta Batterie AG. Комитетом опубликована спецификация SMBus, доступная для скачивания на сайте в формате pdf.

2. Аппаратная часть

На рис. 1 приведен пример подключения шины SMBus. В этом случае MSP430 подключен к смарт-батарее. Батарея питает шину и MSP430, но это не является обязательным, необходимым условием является лишь общая «зем-

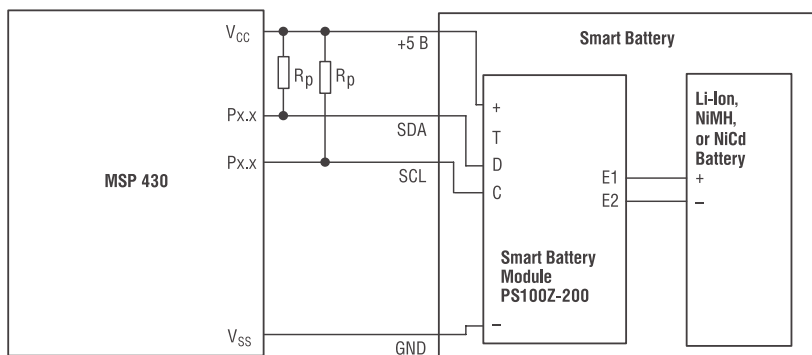


Рис. 1. Пример организации шины SMBus

ля». Подтягивающие (pull up) резисторы обозначены как RP. Для уменьшения потребляемого тока, значения RP выбраны максимально возможными в рамках требований спецификации. Существуют также специальные драйвера шины SMBus, используемые вместо подтягивающих резисторов. Такие устройства обеспечивают максимальный ток на фронте и минимальный при низком уровне на шине.

Примечание:

Неиспользованный вывод T это термистор. Он используется в блоках Li-Ion батарей для аварийных измерений в случае неисправности модуля батареи или шины SMBus.

3. Реализация ведущего устройства шины SMBus

Программа ведущего устройства шины SMBus для MSP430 реализует все существующие типы протоколов. Для экономии регистров, параметры передаются через стек. Максимальная глубина стека 40 байт. Память вне стека не затрагивается, кроме случая операций над блоками. Во время передачи блока его начало передаётся как параметр (блок растёт в сторону верхних адресов памяти). Во время записи блока его размер известен до начала передачи, во время приёма блока его размер передаётся ведомым устройством. При этом следует соблюдать осторожность, чтобы данные из блока не затёрли программную или оперативную память.

Программа состоит из двух основных частей: подпрограмм верхнего уровня и подпрограмм нижнего уровня. Низкоуровневые подпрограммы обрабатывают сдвиг бит, временные характеристики и работу с портами ввода-вывода. Протоколы SMBus, по существу, состоят из нескольких стандартных блоков, которые вызываются и повторяются в определенном порядке. Эти стандартные блоки обрабатываются подпрограммами нижнего уровня. Подпрограммы верхнего уровня обеспечивают пользовательский интерфейс и сохраняют регистры. На каждый протокол приходится одна такая подпрограмма, в них устанавливаются параметры, а затем вызываются подпрограммы нижнего уровня. Для обеспечения требования протокола SMBus к выходам («открытый коллектор»), выводы процессора переключаются со входа на выход и наоборот. В выходных защёлках портов всегда присутствует лог. «0». Это означает, что MSP430 может «освободить» линию переключением вывода на вход, либо «занять» её (т.е. выставить там лог. «0») путём переключения вывода на выход. Выбор требуемых выводов осуществляется «привязкой» их в начале программы к конкретным линиям SDA, SCL и DNC. SDA и SCL соответствуют линиям данных и тактирования шины SMBus. DNC является одновременно линиями SCL и SDA.

Соответствующий порт выбирается назначением OUT, DIR и IN. В зависимости от конкретного применения, могут использоваться не все протоколы. Ненужные подпрограммы могут быть «закомментированы», что уменьшит размер кода. Удаление подпрограмм высокого уровня является очевидным, при этом возможно также удаление некоторых неиспользуемых подпрограмм низкого уровня.

Так как SMBus поддерживает несколько ведущих устройств на шине (multimaster) и «горячее включение» (hot-plug), это приводит к появлению ошибок. Их причинами являются занятость шины, отсутствие ведомого устройства, конфликты между несколькими ведущими устройствами и помехи на шине. Программа включает обработку ошибок и флаг ошибки. При обнаружении ошибки передача прекращается, программа выставляет флаг ошибки и возвращается из подпрограммы верхнего уровня. Пользовательская программа должна проверять данный флаг для определения успешного (возвращает 1) или ошибочного (возвращает 0) завершения. Обработка события ошибки осуществляется пользовательской программой.

Основная программа включает в себя подпрограммы верхнего и нижнего уровней и располагается отдельно от пользовательской программы. Место, где должна располагаться пользовательская программа оставлено свободным. Для передачи данных по шине SMBus, просто вызывается соответствующая подпрограмма верхнего уровня.

Примеры передачи параметров приведены в комментариях к каждой подпрограмме. Исходные коды приведены в приложении А. Также в приложениях приводятся демонстрационные программы, иллюстрирующие различные протоколы и примеры работы с смарт-батареями.

4. Примеры реализации программы ведущего устройства шины SMBus

4.1. Поиск устройств на шине

Первый пример довольно прост. В нём производится поиск устройств, присутствующих на шине и отображение адреса последнего найденного устройства. Это весьма удобно во время проверки соединений шины. Первоначально пример был написан для поиска адресов устройств, он наглядно демонстрирует возможность уменьшения размера кода при помощи удаления неиспользуемых подпрограмм.

Ввиду простоты примера, была удалена значительная часть кода. Оставшийся код можно запускать из-под программы монитора отладочного модуля MSP430x33x (EVK part number MSP-EVK430x330).

4.2. Чтение данных смарт-батарей

Этот пример значительно объёмнее первого. В нём используются протоколы «Чтение слова», «Запись слова» и «Чтение блока» для связи со смарт-батареями. В примере считываются температура, напряжение, производитель, химическая разновидность и ёмкость батареи.

Также используется запись значения в регистр «Предупреждение об оставшейся ёмкости» (RemainingCapacityAlarm), записанное значение проверяется. Эти операции производятся циклически, результат отображается на ЖКИ. Пример демонстрирует использование более сложных протоколов и процесс обработки возвращаемых данных. В данной конфигурации программа протоколов SMBus запускается из EPROM отладочного модуля MSP430x33x (EVK part number MSP-EVK430x330), а пользовательская программа запускается из монитора этого модуля.

5. Реализация ведомого устройства шины SMBus

Эта программа представляет пример реализации ведомого устройства шины SMBus и построена на базе прерываний. Назначения при помощи оператора `eqi` в начале программы позволяют выбрать адрес ведомого устройства, выводы и порты. В программе реализован протокол «Отправка байта» (что для ведомого устройства означает передачу байта к нему от ведущего). В промежутке между прерываниями ведомое устройство может находиться в режиме пониженного потребления либо выполнять другие задачи. Так как прерывание происходит каждый раз при переходе линии SDA из состояния лог. «1» в лог. «0», ведомое устройство при загрузенности шины проводит значительную часть времени в подпрограмме обработки прерываний.

Чтобы исключить контроль шины ведомым устройством во время работы с другими ведомыми могут быть использованы внешние элементы, контролирующие шину и формирующие прерывания только при совпадении адреса ведомого устройства. Ещё одним недостатком использования прерываний без дополнительных внешних устройств является время обработки прерывания. Так как оно составляет несколько тактов, при этом существует вероятность пропуска стартовой последовательности. При реализации поллинга вероятность пропустить стартовую посылку снижается, но процессор постоянно занят отслеживанием состояния шины. Выбор оптимального решения зависит от конкретной реализации. Вариант с прерываниями позволяет процессору обслуживать другие задачи во время неактивного состояния шины, но при этом есть вероятность пропустить первую попытку установления связи. Вариант с поллингом не оставляет процессору возможности обрабатывать другие задачи

вне зависимости от состояния шины. Потребление также при этом возрастает, так как процессор не может быть переведен в режим пониженного энергопотребления во время неактивного состояния шины.

На базе данного протокола возможна реализация остальных. В нашем примере обрабатываются стартовые и стоповые последовательности, приём 7-битных блоков адреса и 8-битных блоков данных / команд. Использование этих процедур в требуемом порядке позволяет реализовать некоторые другие протоколы.

Регистры не сохраняются в стеке с целью обеспечения требований спецификации в области наиболее быстрой стартовой последовательности. Время между переходом линий SDA и SCL в состояние лог. «0» может составлять всего 4 мс. Если известно, что ведущее устройство выдаёт сигналы с большей длительностью, можно сохранять регистры в стеке в начале процедуры, но это может привести к пропуску стартовой посылки при работе с более быстрым ведущим устройством.

6. Заключение

Необходимость увеличения времени жизни батарей в переносных устройствах привела к разработке батарей со сложным химическим составом и повышению требований к технологиям мониторинга. Шина SMBus предоставляет удобный и чёткий интерфейс для мониторинга смарт-батарей, а сверхнизкое потребление микроконтроллера MSP430 делает его незаменимым для таких систем.

Ссылки:

1. SBS Implementers Forum, System Management Bus Specification, Version 1.1, 12/11/98.

Приложение А. Программа ведущего устройства шины SMBus

```

;*****
; Программа ведущего устройства шины SMBus, параметры
; передаются через стек
;*****
RAM_orig      .set    00240h      ; Стартовый адрес в ОЗУ
SP_orig       .set    005DEh      ; Указатель стека
EPROM         .set    08850h      ; Размещение EPROM
;--- Определения управляющих регистров-----
WDTCTL        .equ    0120h
WDTHold       .equ    80h
WDT_wrkey     .equ    05A00h

```

```

; PORT 0
P0IE      .equ    015h
P0DIR     .equ    012h
P0IN      .equ    010h
P0OUT     .equ    011h
; port 2
P2IN      .equ    028h
P2OUT     .equ    029h
P2IE      .equ    02Dh
P2SEL     .equ    02Eh
P2DIR     .equ    02Ah
SDA       .equ    020h
SCL       .equ    080h
DNC       .equ    0A0h
OUT       .equ    P2OU
TDIR      .equ    P2DIR
IN        .equ    P2IN
LCD1      .equ    031h
LCDM      .equ    030h
IE1       .equ    0h
IE2       .equ    01h
IFG1      .equ    02h
IFG2      .equ    03h
block     .equ    0550h
alcd      .equ    0B500h      ; Размещение таблицы
                                ; символов ASCII для ЖКИ
; *****
; Reset : инициализация процессора
; *****
                .sect  "MAIN", RAM_orig
RESET
                MOV    #SP_orig, SP ; Инициализация
                                ; указателя стека
; *****
; Начало пользовательской программы
; *****
; Здесь должна быть расположена пользовательская
; программа
; *****
; Завершение пользовательской программы
; *****
; *****
                .sect  "smbus", EPROM
; *****

```

```

; Подпрограммы верхнего уровня, п/п нижнего уровня
; вызываются индивидуально для реализации различных
; протоколов
; Протокол «Быстрая команда» 1
; =====
; Влияние на регистры: нет
; Влияние на память: стек, максимум 40 байт
; -----
; Описание      Шаг                                     Направление
; -----
;               старт (start)                             из
;               адрес (address)                             из
;               чтение/запись (R/W)                         из
;               подтверждение (acknowledge) в
;               стоп (stop)                                 из
;
; Параметры в стеке
; -----
; флаг ошибки error_flag      младшая область памяти
;                               low mem
; не используется      unused
; не используется      unused
; не используется      unused
; адрес                address      старшая область памяти
;                               high mem
;
; Пример использования
; -----
;               PUSH    #00h      ; место для кода ошибки
;               PUSH    #00Bh     ; поместить адрес в стек
;               SUB      #06h, SP  ; остальные параметры не
;                               ; используются
;               CALL     #qcsr
;               ADD      #08h, SP  ; вычесть объём
;                               ; параметров
;               POP      R9        ; читать из стека код
;                               ; ошибки
;               CMP      #00h, R9
;               JZ       error     ; ошибочное завершение –
;                               ; вызов обработчика ошибок
; *****
qcsr
                PUSH     SR
                PUSH     R7
                PUSH     R8

```

```

PUSH    R9
PUSH    R14
PUSH    R15
PUSH    R10
PUSH    R11
PUSH    R12
PUSH    R13
MOV.B   28(SP), R7      ; получить адрес
                        ; из стека
CALL    #sbit           ; отправить
                        ; стартовую посылку
                        ; и адрес
;CALL   #sendone        ; отправить «1» или
CALL    #sendzero       ; отправить «0»
CALL    #ack            ; ждём подтверждения
                        ; (ACK)
CALL    #pbit           ; отправить стоповую
                        ; посылку
MOV.B   #01h, 30(SP)    ; возвращает «1»
                        ; при успешном
                        ; завершении

POP     R13
POP     R12
POP     R11
POP     R10
POP     R15
POP     R14
POP     R9
POP     R8
POP     R7
POP     SR
RET

; *****
; Протокол «Отправить байт» 2
; =====
; Влияние на регистры: нет
; Влияние на память: стек, максимум 40 байт
; -----
; Описание      Шаг                               Направление
; -----
;               старт (start)                      из
;               адрес (address)                     из
;               чтение/запись (R/W)                из
;               подтверждение (acknowledge) в

```

```

;          команда (command)                      из
;          подтверждение (acknowledge) в
;          стоп (stop)                            из
;
; Параметры в стеке
; -----
; флаг ошибкиerror_flag                младшая область памяти
;                                     low mem
; не используется      unused
; не используется      unused
; данные               data
; адрес               address        старшая область памяти
;                                     high mem
;
; Пример использования
; -----
;          PUSH   #00h          ; место для кода ошибки
;          PUSH   #00Bh        ; поместить адрес в стек
;          PUSH   #055h        ; отправляемые данные
;          SUB     #04h, SP     ; остальные параметры не
;                               ; используются
;
;          CALL    #sbp
;          ADD     #08h, SP     ; вычесть объем парамет-
;                               ; ров
;          POP     R9           ; читать из стека код
;                               ; ошибки
;
;          CMP     #00h, R9
;          JZ      error        ; ошибочное завершение -
;                               ; вызов обработчика
;                               ; ошибок
;
sbp
          PUSH    SR
          PUSH    R7
          PUSH    R8
          PUSH    R9
          PUSH    R14
          PUSH    R15
          PUSH    R10
          PUSH    R11
          PUSH    R12
          PUSH    R13
          MOV.B   28(SP), R7    ; получить адрес из
                               ; стека
          MOV.B   26(SP), R8    ; получить данные из

```

```

; стека
CALL #sbit ; отправить стартовую
; посылку и адрес
CALL #sendzero ; отправить «0» для
; записи
CALL #ack ; ждём подтверждения
; (ACK)
CALL #sbyte ; отправить байт
; команды
CALL #ack ; ждём подтверждения
; (ACK)
CALL #pbit ; отправить стоповую
; посылку
MOV.B #01h, 30(SP) ; возвращает «1» при
; успешном завершении

POP R13
POP R12
POP R11
POP R10
POP R15
POP R14
POP R9
POP R8
POP R7
POP SR
RET

; *****
; Протокол «Принять байт» 3
; =====
; Влияние на регистры: нет
; Влияние на память: стек, максимум 40 байт
; -----
; Описание Шаг Направление
; -----
; старт (start) из
; адрес (address) из
; чтение/запись (R/W) из
; подтверждение (acknowledge) в
; данные (data) в
; неподтверждение (NACK) из
; стоп (stop) из
;
; Параметры в стеке
; -----

```

```

; флаг ошибкиerror_flag          младшая область памяти
                                   low mem

; данные                          data
; не используется               unused
; команда                       command
; адрес                          address      старшая область памяти
                                           high mem

; -----
; Пример использования
; -----
;
;          PUSH    #00h          ; место для кода ошибки
;
;          PUSH    #00Bh         ; поместить адрес в стек
;
;          PUSH    #08h         ; код команды
;
;          SUB     #04h, SP      ; зарезервировать место
;                               ; под принимаемые данные
;
;          CALL    #rcbp         ;
;          POP     R8            ; переместить данные
;                               ; в R8
;
;          ADD     #06h, SP      ; вычесть объём
;                               ; параметров
;
;          POP     R9            ; читать из стека код
;                               ; ошибки
;
;          CMP     #00h, R9      ;
;          JZ      error         ; ошибочное завершение -
;                               ; вызов обработчика
;                               ; ошибок
; *****
rbp
    PUSH    SR
    PUSH    R7
    PUSH    R8
    PUSH    R9
    PUSH    R14
    PUSH    R15
    PUSH    R10
    PUSH    R11
    PUSH    R12
    PUSH    R13
    MOV.B   26(SP), R8 ; параметр команды
    MOV.B   28(SP), R7 ; параметр адреса
    CALL    #sbit      ; отправить стартовую
                       ; посылку и адрес
    CALL    #sendzero  ; отправить «0» для
                       ; записи

```



```

CALL    #ack          ; ждём подтверждения
                        ; (ACK)
CALL    #sbyte         ; отправить код команды
CALL    #ack          ; ждём подтверждения
                        ; (ACK)
CALL    #rsbit         ; отправить повторяющие-
                        ; ся стартовую посылку
                        ; и адрес
CALL    #sendone       ; отправить «1» для
                        ; чтения
CALL    #ack          ; ждём подтверждения
                        ; (ACK)
CALL    #rbyte         ; принять байт данных
CALL    #nack          ; отправить неподтверж-
                        ; дение (NACK)
CALL    #pbit          ; отправить стоповую
                        ; посылку
MOV.B   R14, 22(SP) ; копировать байт данных
                        ; в стек
MOV.B   #01h, 30(SP) ; возвращает «1» при
                        ; успешном завершении

POP     R13
POP     R12
POP     R11
POP     R10
POP     R15
POP     R14
POP     R9
POP     R8
POP     R7
POP     SR
RET

; ***** ;
Протокол «Принять слово» 7
; =====
; Влияние на регистры: нет
; Влияние на память: стек, максимум 40 байт
; -----
; Описание      Шаг                               Направление
; -----
;               старт (start)                       из
;               адрес (address)                       из
;               чтение/запись (R/W)                   из
;               подтверждение (acknowledge) в

```

```

; команда (command)                                ИЗ
; подтверждение (acknowledge) в
; старт (start)                                    ИЗ
; адрес (address)                                  ИЗ
; чтение/запись (R/W)                              ИЗ
; подтверждение (acknowledge) в
; данные (data)                                    В
; подтверждение (acknowledge) в
; данные (data)                                    В
; подтверждение (acknowledge) в
; стоп (stop)                                      ИЗ

; Параметры в стеке
; -----
; флаг ошибки error_flag                          младшая область памяти
;                                                  low mem

; младший байт
; данных low data byte
; старший байт
; данных high data byte
; команда command
; адрес address старшая область памяти
; high mem

; -----
; Пример использования
; -----
; PUSH #00h ; место для кода ошибки
; PUSH #00Bh ; поместить адрес в стек
; PUSH #008h ; код команды "темпера-
;           ; тура батареи"
; PUSH #00h ; зарезервировать байт
;           ; под принимаемые данные
; PUSH #00h ; зарезервировать байт
;           ; под принимаемые данные
; CALL #rwp
; POP R10 ; переместить данные
;         ; в R10
; POP R11 ; переместить данные
;         ; в R11
; ADD #04h, SP ; освободить место,
;             ; занимаемое командой
;             ; и адресом
; POP R9 ; читать из стека код

```

```

;                                ; ошибки
;      CMP    #00h, R9
;      JZ     error              ; ошибочное завершение -
;                                ; вызов обработчика
;                                ; ошибок
; *****
rwp
      PUSH    SR
      PUSH    R7
      PUSH    R8
      PUSH    R9
      PUSH    R14
      PUSH    R15
      PUSH    R10
      PUSH    R11
      PUSH    R12
      PUSH    R13
      MOV.B   26(SP), R8 ; параметр команды
      MOV.B   28(SP), R7 ; параметр адреса
      CALL    #sbit      ; отправить стартовую
                        ; посылку и адрес
      CALL    #sendzero  ; отправить «0» для
                        ; записи
      CALL    #ack        ; ждём подтверждения
                        ; (ACK)
      CALL    #sbyte     ; отправить код команды
      CALL    #ack        ; ждём подтверждения
                        ; (ACK)
      CALL    #rsbit     ; отправить повторяющие-
                        ; ся стартовую посылку
                        ; и адрес
      CALL    #sendone   ; отправить «1» для
                        ; чтения
      CALL    #ack        ; ждём подтверждения
                        ; (ACK)
      CALL    #rbyte     ; принять младший байт
                        ; данных
      MOV.B   R14, 22(SP); сохранить младший байт
                        ; данных в стеке
      CALL    #sack      ; ждём подтверждения
                        ; (ACK)
      CALL    #rbyte     ; принять старший байт
                        ; данных
      CALL    #nack      ; отправить неподтверж-

```

```

; дение (NACK)
CALL #pbit ; отправить стоповую
; посылку
MOV.B R14, 24(SP) ; сохранить старший байт
; данных в стеке
MOV.B #01h, 30(SP) ; возвращает «1» при
; успешном завершении

POP R13
POP R12
POP R11
POP R10
POP R15
POP R14
POP R9
POP R8
POP R7
POP SR
RET

; *****;
Протокол «Запись блока» 8
; =====
; Влияние на регистры: нет
; Влияние на память: стек, максимум 40 байт, блок
; памяти
; Описание Шаг Направление
; -----
; старт (start) из
; адрес (address) из
; чтение/запись (R/W) из
; подтверждение (acknowledge) в
; команда (command) из
; подтверждение (acknowledge) в
; счётчик байт (byte count) из
; подтверждение (acknowledge) в
; данные (data) из
; подтверждение (acknowledge) в
;
; данные (data) в
; подтверждение (acknowledge) в
;
; данные и АСК повторяются n раз
; стоп (stop) из

; Параметры в стеке

```

```

; -----
; флаг ошибкиerror_flag          младшая область памяти
;                               low mem
; счётчик байт      byte count
; указатель блока   block pointer
; команда           command
; адрес             address      старшая область памяти
;                               high mem
; -----
; Пример использования
; -----
;           PUSH    #00h          ; место для кода ошибки
;           PUSH    #00Bh        ; поместить адрес в стек
;           PUSH    #0020h       ; код команды "темпера-
;                               ; тура батареи"
;           PUSH    #00550h      ; указатель на начало
;                               ; блока
;           PUSH    #00h         ; зарезервировать байт
;                               ; под счётчик
;           CALL    #blkw
;           POP     R10          ; переместить данные
;                               ; в R10
;           ADD     #06h,SP
;           POP     R9           ; читать из стека код
;                               ; ошибки
;           CMP     #00h, R9
;           JZ      error        ; ошибочное завершение -
;                               ; вызов обработчика
;                               ; ошибок
; *****
blkw
        PUSH    SR
        PUSH    R7
        PUSH    R8
        PUSH    R9
        PUSH    R14
        PUSH    R15
        PUSH    R10
        PUSH    R11
        PUSH    R12
        PUSH    R13
        MOV.B   26(SP), R8 ; параметр команды
        MOV.B   28(SP), R7 ; параметр адреса
        MOV     24(SP), R9 ; стартовый адрес блока

```

```

MOV.B #00h, R15 ; циклический байтовый
                  ; счётчик
CALL #sbit       ; отправить стартовую
                  ; посылку и адрес
CALL #sendzero   ; отправить «0» для
                  ; записи
CALL #ack         ; ждём подтверждения
                  ; (ACK)
CALL #sbyte      ; отправить код команды
CALL #ack         ; ждём подтверждения
                  ; (ACK)
MOV.B 22(SP), R8 ; копировать счётчик
                  ; байт
CALL #sbyte      ; отправить счётчик байт
CALL #ack         ; ждём подтверждения
                  ; (ACK)
blk_rep MOV.B 0(R9), R8 ; передать данные
                  ; из блока
CALL #sbyte      ; отправить байт данных
CALL #ack         ; ждём подтверждения
                  ; (ACK)
INC R15          ; инкремент счётчика
                  ; положения блока
INC R9           ; инкремент указателя
                  ; блока
CMP 22(SP), R15 ; проверка завершения
                  ; данных
JNZ blk_rep      ; повтор до завершения
CALL #pbit       ; отправить стоповую
                  ; посылку
MOV.B #01h, 30(SP); возвращает «1» при
                  ; успешном завершении

POP R13
POP R12
POP R11
POP R10
POP R15
POP R14
POP R9
POP R8
POP R7
POP SR
RET

```

```
;*****
```

```

; Протокол «Чтение блока» 9
; =====
; Влияние на регистры: нет
; Влияние на память: стек, максимум 40 байт, блок
; памяти
; Описание      Шаг                                     Направление
; -----
;      старт (start)                                     из
;      адрес (address)                                   из
;      запись (W)                                         из
;      подтверждение (acknowledge) в
;      команда (command)                                   из
;      подтверждение (acknowledge) в
;      старт (start)                                       из
;      адрес (address)                                   из
;      чтение (R)                                         из
;      подтверждение (acknowledge) в
;      счётчик байт (byte count)                         в
;      подтверждение (acknowledge) из
;      данные (data)                                       в
;      подтверждение (acknowledge) из
;
;      данные и ACK повторяются n раз
;      неподтверждение (NACK)                             из
;      стоп (stop)                                         из
;
; Параметры в стеке
; -----
; флаг ошибки error_flag      младшая область памяти
;                               low mem
;
; счётчик байт      byte count
; указатель блока   block pointer
; команда           command
; адрес             address      старшая область памяти
;                               high mem
;
; -----
; Пример использования
; -----
;      PUSH    #00h      ; место для кода ошибки
;      PUSH    #00Bh     ; поместить адрес в стек
;      PUSH    #0020h    ; код команды "темпера-
;                               ; тура батареи"
;
;      PUSH    #00550h   ; указатель на начало
;                               ; блока
;      PUSH    #00h      ; зарезервировать байт

```

```

;                                ; под счётчик
; CALL    #blkr
; POP     R10                    ; переместить счётчик
;                                ; в R10
; ADD     #06h, SP
; POP     R9                     ; читать из стека код
;                                ; ошибки
; CMP     #00h, R9
; JZ      error                  ; ошибочное завершение -
;                                ; вызов обработчика
;                                ; ошибок
; *****blkr
PUSH     SR
PUSH     R7
PUSH     R8
PUSH     R9
PUSH     R14
PUSH     R15
PUSH     R10
PUSH     R11
PUSH     R12
PUSH     R13
MOV.B    #00h, R15              ; циклический байтовый
;                                ; счётчик
MOV.B    26(SP), R8             ; параметр команды
MOV.B    28(SP), R7             ; параметр адреса
MOV      24(SP), R9             ; стартовый адрес блока
CALL     #sbit                  ; отправить стартовую
;                                ; посылку и адрес
CALL     #sendzero              ; отправить «0» для
;                                ; записи
CALL     #ack                   ; ждём подтверждения
;                                ; (ACK)
CALL     #sbyte                 ; отправить код команды
CALL     #ack                   ; ждём подтверждения
;                                ; (ACK)
CALL     #rsbit                 ; отправить повторяющие-
;                                ; ся стартовую посылку
;                                ; и адрес
CALL     #sendone               ; отправить «1» для
;                                ; чтения
CALL     #ack                   ; ждём подтверждения
;                                ; (ACK)
CALL     #rbyte

```



```

MOV.B R14, 22(SP); поместить счётчик
; в стек
CALL #sack      ; отправить подтвержде-
; ние (ACK)
rblk_rep CALL #rbyte ; принять байт данных
MOV.B R14, 0(R9); переместить данные
; из блока
INC R15         ; инкремент счётчика
; положения блока
INC R9         ; инкремент указателя
; блока
CMP 22(SP), R15; проверка завершения
; данных
JZ blk_done    ; повтор до завершения
CALL #sack     ; отправить подтвержде-
; ние (ACK)
CMP 22(SP), R15; проверка завершения
; данных
JNZ rblk_rep   ; повтор до завершения
blk_done CALL #nack ; отправить неподтверж-
; дение (NACK)
CALL #pbit     ; отправить стоповую
; посылку
MOV.B #01h, 30(SP); возвращает «1» при
; успешном завершении

POP R13
POP R12
POP R11
POP R10
POP R15
POP R14
POP R9
POP R8
POP R7
POP SR
RET

; *****
; Подпрограммы нижнего уровня, общие для всех
; протоколов
; *****
; *****
; отправка стартовой последовательности и адреса
; *****
sbit ; «отпустить» обе линии

```

```

BIC.B #DNC, DIR
; проверка освобождения шины
MOV #05h, R10 ; счётчик на 50 микросе-
; кунд

wait
MOV.B IN, R11 ; копировать состояние
; входов в R11
AND #DNC, R11 ; маска входов
CMP #DNC, R11 ; если линии SDA или SCL
; в лог. «0»то шина
; занята

JNZ busy
DEC R10 ; декремент счётчика
JNZ wait

rsbit
BIC.B #DNC, DIR ; дополнительное
; освобождение линии
; с целью использования
; этой же подпрограммы
; для команды повторяю-
; щегося старта
; отправка стартовой последовательности
BIS.B #SDA, DIR
BIS.B #SCL, DIR
; отправка адреса
MOV.B #07h, R13 ; счётчик на 7 бит
; адреса

ashift
MOV.B R7, R11 ; копировать адрес в R11
RLA.B R11 ; сдвиг влево, при этом
; СЗР 7-битного адреса
; будет находиться на
; 7-й позиции
MOV.B R11, R12 ; скопировать его для
; возможности маскирова-
; ния
; без потери данных
AND.B #080h, R12 ; маскировать все биты
; кроме СЗР
CMP.B #00h, R12 ; сравнить с 0
JNZ one
CALL #sendzero ; отправить «1»
JMP zero
one
CALL #sendone ; отправить «0»
zero
DEC R13 ; декремент счётчика
JNZ ashift ; если счётчик <7 бит,

```

```

; повторить сдвиг
RET
;*****
; подтверждение (ACK) не принято, отправить STOP и busy
;*****
sbusy      CALL    #pbit      ; отправить STOP и busy
           ADD     #02h, SP    ; удалить данные из
                               ; стека, так как
                               ; инструкция RET
                               ; не использовалась
           MOV.B   #00h, 28(SP) ; код ошибки
           POP     R13         ; восстановление
                               ; регистров
           POP     R12
           POP     R11
           POP     R10
           POP     R15
           POP     R14
           POP     R9
           POP     R8
           POP     R7
           POP     SR
           RET
;*****
; Ожидание подтверждения (ACK)
;*****
ack
           BIC.B   #SDA, DIR   ; выставить лог. «1»
                               ; на линии SDA
           BIC.B   #SCL, DIR   ; выставить лог. «1»
                               ; на линии SCL
           ; проверка удлинения тактового сигнала
           ; (clock low extending), чтобы не
           ; опережать ведомое устройство
           ;MOV    #008h, R11 ; счётчик на 750 мкс
           MOV     #02FFh, R11 ; удлинённый счётчик
                               ; для батареи
                               ; PS100Z-200
clkex      MOV.B   IN, R10     ; проверка шины
           DEC     R11
           JZ      busy        ; арбитраж – время
                               ; истекло
           AND.B   #SCL, R10   ; проверка линии SCL
           JZ      clkex

```

```

; ожидание подтверждения (ACK)
; (SDA переходит в лог. «0»)
MOV    #05F6h, R11 ; счётчик таймаута
; для NACK

wack

MOV.B  IN, R10      ; проверка шины
DEC    R11
JZ     sbusy        ; таймаут NACK-- отпра-
; вить бит Р и повторить
; попытку
AND.B  #SDA, R10    ; маскировать всё, кроме
; SDA
JNZ    wack         ; приём ACK или ожидание
; таймаута
BIS.B  #SCL, DIR    ; выставить лог. «0»
; на линии SCL

RET

; *****
; Отправка стоповой последовательности
; *****
pbit

BIS.B  #SDA, DIR    ; выставить лог. «0»
; на линии SDA
BIS.B  #SCL, DIR    ; выставить лог. «0»
; на линии SCL

NOP
BIC.B  #SCL, DIR    ; выставить лог. «1»
; на линии SCL

NOP
BIC.B  #SDA, DIR    ; выставить лог. «1»
; на линии SDA

RET

; *****
; Отправка байта (используются также sendzero
; и sendone)
; *****
sbyte
; отправка данных

MOV.B  #08h, R13    ; счётчик на 250 мкс
MOV.B  R8, R11      ; копировать данные
; в R11

dshift

MOV.B  R11, R12
AND.B  #080h, R12   ; маскировать все биты,

```

```

                                ; кроме СЗР
CMP.B #00h, R12                ; сравнить с 0
JNZ   on
CALL  #sendzero                ; отправить «1»
JMP   zer
on    CALL  #sendone            ; отправить «0»
zer

RLA.B R11                      ; сдвиг влево, отправля-
                                ; емый бит в СЗР
DEC   R13                      ; декремент счётчика
                                ; на 8
JNZ   dshift
RET

;*****
; Приём байта
;*****
rbyte

MOV.B #08h, R12                ; счётчик на 8 бит
                                ; данных
BIC.B #SDA, DIR                ; выставить лог. «1»
                                ; на линии SDA

rrep

BIS.B #SCL, DIR                ; выставить лог. «0»
                                ; на линии SCL
BIC.B #SCL, DIR                ; выставить лог. «1»
                                ; на линии SCL
MOV   #0035h, R13              ; удлинённый счётчик для
                                ; батареи PS100Z-200

; удлинение тактового сигнала
clkxtn
MOV.B IN, R10                  ; проверка шины
DEC   R13
JZ    busy                     ; выход по таймауту
AND.B #SCL, R10
JZ    clkxtn                   ; ожидание, если SCL
                                ; не «1»
MOV.B IN, R10                  ; проверка шины
RLA.B R11                      ; вход = МЗР
AND.B #SDA, R10                ; проверка SDA
JZ    inzero
BIS.B #01h, R11                ; установить «1» в МЗР
inzero
DEC   R12
JNZ   rrep                     ; продолжение для остав-
                                ; шейся части байта
MOV.B R11, R14                 ; копирование данных

```

```

        BIS.B #SCL, DIR ; выставить лог. «0»
                           ; на линии SCL
        RET

;*****
; Отправка неподтверждения (NACK)
;*****
nack
        BIC.B #SDA, DIR ; выставить лог. «1»
                           ; на линии SDA
        BIS.B #SCL, DIR ; выставить лог. «0»
                           ; на линии SCL
        BIC.B #SCL, DIR ; выставить лог. «1»
                           ; на линии SCL
        ; проверка удлинения тактового сигнала
        ; (clock low extending), чтобы не
        ; опережать ведомое устройство
        ; ждём установки «1» на линии SCL
clkcn  MOV.B IN, R10
        AND.B #SCL, R10 ; проверка линии SCL
        JZ    clkcn
        BIS.B #SCL, DIR ; «отпустить» SCL
        RET

;*****
; Отправка подтверждения (ACK)
;*****
sack    ; ждём, пока ведомый «отпустит» линию SDA
        MOV #000Fh, R11; счётчик на ?50 мкс
swaita  MOV.B IN, R10 ; проверка шины
        DEC R11
        JZ    busy
        AND.B #SDA, R10 ; проверка SCL
        JZ    swaita
        BIS.B #SDA, DIR ; перевести SDA в «0»
        BIC.B #SCL, DIR ; перевести SCL в «1»
        MOV #000Fh, R11; счётчик на ?50 мкс
swait   MOV.B IN, R10 ; проверка шины
        DEC R11
        JZ    busy
        AND.B #SCL, R10 ; проверка SCL
        JZ    swait
        BIS.B #SCL, DIR ; перевести SCL в «0»
        BIC.B #SDA, DIR ; «отпустить» SDA
        RET

```

```

;--- очистка ЖКИ-----
show_clr
        MOV    #15, r5        ; очистка дисплейной
                                ; памяти
show_clr1
        MOV.b  #0, LCD1-1(r5)
        DEC    r5
        JNZ    show_clr1
        RET

;*****
; Определения для ЖКИ
;*****
LCD_TYPE
;---ЖКИ модуля STK/EVK-----
a        .equ   01h
b        .equ   02h
c        .equ   10h
d        .equ   04h
e        .equ   80h
f        .equ   20h
g        .equ   08h
h        .equ   40h

;--- Определения символов-----
LCD_Tab  .byte  a+b+c+d+e+f ; отображает "0"
        .byte  b+c          ; отображает "1"
        .byte  a+b+d+e+g    ; отображает "2"
        .byte  a+b+c+d+g    ; отображает "3"
        .byte  b+c+f+g      ; отображает "4"
        .byte  a+c+d+f+g    ; отображает "5"
        .byte  a+c+d+e+f+g  ; отображает "6"
        .byte  a+b+c        ; отображает "7"
        .byte  a+b+c+d+e+f+g ; отображает "8"
        .byte  a+b+c+d+f+g  ; отображает "9"
        .byte  0            ; отображает ":"
                                ; пустой
        .byte  g            ; отображает ";" -
        .byte  a+d+e+f      ; отображает "<" [
        .byte  d+g          ; отображает "="
        .byte  a+b+c+d      ; отображает ">" ]
        .byte  a+b+e+g      ; отображает "?"
        .byte  a+b+d+e+f+g  ; отображает "@"
        .byte  a+b+c+e+f+g  ; отображает "A"
        .byte  c+d+e+f+g    ; отображает "B" b

```

```

.byte a+d+e+f      ; отображает "C"
.byte b+c+d+e+g    ; отображает "D" d
.byte a+d+e+f+g    ; отображает "E"
.byte a+e+f+g      ; отображает "F"
.byte a+b+c+d+f+g ; отображает "G"
.byte b+c+e+f+g    ; отображает "H"
.byte b+c          ; отображает "I"
.byte b+c+d+e      ; отображает "J"
.byte 0            ; отображает "K"
.byte d+e+f        ; отображает "L"
.byte a+b+c+e+f    ; отображает "M"
.byte c+e+g        ; отображает "N" n
.byte c+d+e+g      ; отображает "O" o
.byte a+b+e+f+g    ; отображает "P"
.byte 0            ; отображает "Q"
.byte e+g          ; отображает "R" r
.byte a+c+d+f+g    ; отображает "S"
.byte d+e+f+g      ; отображает "T" t
.byte c+d+e        ; отображает "U" u
.byte 0            ; отображает "V"
.byte 0            ; отображает "W"
.byte 0            ; отображает "X"
.byte b+c+d+f+g    ; отображает "Y"
.byte a+b+d+e+g    ; отображает "Z" 2

```

LCD_Tab_End

```

.even

```

```

;*****
; Вектора прерываний
;*****
;
;       .sect  "Int_Vect",0FFFFh-1 ; без программой
;                                   ; монитора
;       .word  RESET               ; POR, внеш.
;                                   ; Reset,
;                                   ; Watchdog
;
;       .end
;       .sect  "Int_Vect",05FFh-1  ; с программой
;                                   ; монитора
;       .word  RESET               ; POR, внеш.
;                                   ; Reset,
;                                   ; Watchdog
;
;       .end

```


Приложение В. Программа поиска адресов

```
;*****
; Программа поиска адресов
;*****
; Программа сканирует шину SMBus на предмет подключен-
; ных устройств.
; Начинает с адреса 0x00 и перебирает все 128 возмож-
; ных адресов.
; Текущий адрес при сканировании отображается на пра-
; вой части ЖКИ
; На левой части отображается адрес последнего из най-
; денных устройств.
; Когда устройство найдено, программа приостанавлива-
; ется, адрес переносится
; в левую часть ЖКИ, после чего выполнение продолжает-
; ся.
; Наличие устройства на шине проверяется посылкой
; стартовой последовательности, адреса устройства и
; ожидания подтверждения. Далее; тестируется следующий
; адрес.
;
; Программа поиска адресов является примером удаления
; неиспользуемых частей кода. т.к. ей не требуются
; некоторые протоколы, некоторые подпрограммы верхнего
; и нижнего уровней были удалены.
;*****
RAM_orig      .set    00240h      ; Стартовый адрес
SP_orig       .set    005DEh      ; указатель стека
;--- Определения управляющих регистров-----
WDTCTL        .equ    0120h
WDTHold        .equ    80h
WDT_wrkey      .equ    05A00h
;PORT 0
P0IE           .equ    015h
P0DIR          .equ    012h
P0IN           .equ    010h
P0OUT          .equ    011h
; port 2
P2IN           .equ    028h
P2OUT          .equ    029h
P2IE           .equ    02Dh
P2SEL          .equ    02Eh
P2DIR          .equ    02Ah
SDA            .equ    020h
```

```

SCL      .equ    080h
DNC      .equ    0A0h
OUT      .equ    P2OUT
DIR      .equ    P2DIR
IN       .equ    P2IN
LCD1     .equ    031h
LCDM     .equ    030h
IE1      .equ    0h
IE2      .equ    01h
IFG1     .equ    02h
IFG2     .equ    03h
address  .equ    0500h      ; Адрес в ОЗУ для
                           ; переменной сканирова-
                           ; ния
;*****
; Reset : Инициализация процессора
;*****
        .sect    "MAIN",RAM_orig
RESET
        MOV     #SP_orig,SP ; Инициализация указа-
                           ; теля стека
;*****
; Начало пользовательской программы
;*****
        CALL    #show_clr
; Сброс адреса
res      MOV.B   #00FFh, address      ; адрес начина-
                                       ; ется с 0xFF
; Начало цикла перебора адресов
retry
        INC.B   address      ; переход к следующему
                           ; адресу из 0xFF в 0x00
        CMP.B   #080h, address      ; адрес = 80?
        JZ      res          ; сброс адреса по
                           ; переполнению
; пауза
        MOV     #02Fh, R7      ; цикл паузы
dela2    MOV     #031Fh, R6
dela     DEC     R6
        JNZ     dela
        DEC     R7
        JNZ     dela2
; Отображение сканируемого адреса
        MOV     #0000h, R5      ; очистка (убедиться, что

```

```

; очищается старший
; байт)
MOV.B address, R5 ; R5 ? младший байт
AND #000Fh, R5 ; маска на старшую
; тетраду
MOV.B LCD_Tab(R5), LCD1+1
; отобразить младшую тетраду 0-F
MOV.B address, R5 ; R5 ? младший байт
AND #00F0h, R5 ; маска на младшую
; тетраду
RRA R5 ; сдвиг вправо
RRA R5 ; сдвиг вправо
RRA R5 ; сдвиг вправо
RRA R5 ; сдвиг вправо
MOV.B LCD_Tab(R5), LCD1+2
; отобразить старшую тетраду 0-F
PUSH #00h ; место для кода ошибки
PUSH address ; поместить адрес в стек
SUB #06, SP
CALL #hunt
POP R10 ; вернуть данные в R10
POP R11 ; вернуть данные в R11
ADD #04h, SP ; освободить место для
; адреса и команд
POP R9 ; вернуть код ошибки
CMP #00h, R9
JZ retry ; повтор при неудаче
MOV #0FFh, R7
del44 MOV #0B4Fh, R6
del43 DEC R6
JNZ del43
DEC R7
JNZ del44
MOV #0000h, R5 ; очистка (убедиться,
; что очищается старший
; байт)
MOV.B address, R5 ; R5 ? младший байт
AND #000Fh, R5 ; маска на старшую
; тетраду
MOV.B LCD_Tab(R5), LCD1+4
; отобразить младшую тетраду 0-F
MOV.B address, R5 ; R5 ? младший байт
AND #00F0h, R5 ; маска на младшую
; тетраду

```

```

RRA      R5          ; сдвиг вправо
RRA      R5          ; сдвиг вправо
RRA      R5          ; сдвиг вправо
RRA      R5          ; сдвиг вправо
MOV.B    LCD_Tab(R5), LCD1+5
; отобразить старшую тетраду 0-F
JMP      retry       ; повтор для следующей
                      ; го адреса

; *****
; Завершение пользовательской программы
; *****
; *****
; Подпрограммы верхнего уровня, п/п нижнего уровня
; вызываются индивидуально для реализации различных
; протоколов
; *****
; *****
; Поиск это модифицированная часть протокола «Чтение
; байта»
; *****

hunt     PUSH      SR
        PUSH      R7
        PUSH      R8
        PUSH      R9
        PUSH      R14
        PUSH      R15
        PUSH      R10
        PUSH      R11
        PUSH      R12
        PUSH      R13
        MOV.B     26(SP), R8    ; параметр команды
        MOV.B     28(SP), R7    ; параметр адреса
        CALL      #sbit        ; отправить стартовую
                                ; последовательность
                                ; и адрес
        CALL      #sendzero     ; отправить 0 для записи
        CALL      #ack          ; ждём подтверждения
                                ; (ACK)
        MOV.B     #01h, 30(SP) ; возвращает «1» при
                                ; успешном завершении

        POP       R13
        POP       R12

```

```

        POP    R11
        POP    R10
        POP    R15
        POP    R14
        POP    R9
        POP    R8
        POP    R7
        POP    SR
        RET

;*****
; Подпрограммы нижнего уровня, общие для всех
; протоколов
;*****
;*****
; отправка стартовой последовательности и адреса
;*****
sbit                                ; «отпустить» обе линии
        BIC.B  #DNC, DIR
; проверка освобождения шины
        MOV    #05h, R10      ; счётчик на 50 микросе-
                                ; кунд
wait
        MOV.B  IN, R11        ; копировать состояние
                                ; входов в R11
        AND    #DNC, R11      ; маска входов
        CMP    #DNC, R11      ; если линии SDA или SCL
                                ; в лог. «0»то шина
                                ; занята
        JNZ    busy
        DEC    R10            ; декремент счётчика
        JNZ    wait
rsbit    BIC.B  #DNC, DIR      ; дополнительное
                                ; освобождение линии
                                ; с целью использования
                                ; этой же подпрограммы
                                ; для команды повторяю-
                                ; щегося старта
; отправка стартовой последовательности
        BIS.B  #SDA, DIR
        BIS.B  #SCL, DIR
; отправка адреса
        MOV.B  #07h, R13      ; счётчик на 7 бит
                                ; адреса

```

```

ashift      MOV.B  R7, R11      ; копировать адрес в R11
            RLA.B  R11          ; сдвиг влево, при этом
                                ; СЗР 7-битного адреса
                                ; будет находиться
                                ; на 7-й позиции
            MOV.B  R11, R12     ; скопировать его для
                                ; возможности маскирова-
                                ; ния без потери данных
            AND.B  #080h, R12   ; маскировать все биты
                                ; кроме СЗР
            CMP.B  #00h, R12    ; сравнить с 0
            JNZ    one          ;
one          CALL  #sendzero    ; отправить «1»
            JMP    zero
zero        CALL  #sendone      ; отправить «0»
            DEC    R13          ; декремент счётчика
            JNZ    ashift       ; если счётчик <7 бит,
                                ; повторить сдвиг
            RET

;*****
; подтверждение (ACK) не принято, отправить STOP
; и busy
;*****
sbusy       CALL  #pbit         ; отправить STOP и busy
            ADD    #02h, SP     ; удалить данные из
                                ; стека, так как
                                ; инструкция RET
                                ; не использовалась
            MOV.B  #00h, 28(SP) ; код ошибки
            POP    R13          ; восстановление регист-
                                ; ров
            POP    R12
            POP    R11
            POP    R10
            POP    R15
            POP    R14
            POP    R9
            POP    R8
            POP    R7
            POP    SR
            RET

;*****
; Обработка занятости системы

```

```

;*****
busy3      ADD    #06h, SP    ; удаление лишних
                                ; данных процедур
                                ; sendone или sendzero
busy       ADD    #02h, SP    ; удалить данные
                                ; из стека, так как
                                ; инструкция RET
                                ; не использовалась
                                ; код ошибки
                                ; восстановление регист-
                                ; ров
                                POP    R12
                                POP    R11
                                POP    R10
                                POP    R15
                                POP    R14
                                POP    R9
                                POP    R8
                                POP    R7
                                POP    SR
                                RET

;*****
; Отправить «1»
;*****
sendone    PUSH   R10
           PUSH   R11
           BIC.B  #SDA, DIR    ; установить лог. «1»
                                ; на линии SDA
           BIC.B  #SCL, DIR    ; установить лог. «1»
                                ; на линии SCL

; проверка удлинения тактового сигнала
; (clock low extending), чтобы не опережать ведомое
; устройство
           MOV.B  #04h, R11    ; установить число
                                ; тактов для верного
                                ; времени
clkext     MOV.B  IN, R10      ; проверка шины
           DEC    R11
           JZ     busy3        ; арбитраж – время
                                ; истекло
           AND.B  #SCL, R10    ; проверка линии SCL
           JZ     clkext

; арбитраж: убедиться, что на линии SDA действительно
; лог. «1», иначе приоритет отдаётся другому ведущему

```

```

; устройству
MOV.B IN, R10      ; проверка шины
AND.B #SDA, R10    ; маскировать всё,
                   ; кроме SDA

JZ      busy3
BIS.B #SCL, DIR    ; освободить линию SCL
MOV.B IN, R10      ; определение повторяю-
                   ; щейся стартовой после
                   ; довательности

AND.B #SDA, R10
JZ      busy3
POP     R11
POP     R10
RET

; *****
; Отправить «0»
; *****
sendzero
PUSH    R10
PUSH    R11
BIS.B #SDA, DIR    ; выставить лог. «0»
                   ; на линии SDA
BIC.B #SCL, DIR    ; выставить лог. «1»
                   ; на линии SCL

; проверка удлинения тактового сигнала
; (clock low extending), чтобы не опережать ведомое
; устройство
MOV     #04h, R11  ; счётчик времени
                   ; перехода SCL в «1»
clke    MOV.B IN, R10 ; проверка шины
DEC     R11
JZ      busy3      ; арбитраж - время
                   ; истекло
AND.B #SCL, R10    ; проверка линии SCL
JZ      clke
BIS.B #SCL, DIR    ; освободить линию SCL
POP     R11
POP     R10
RET

; *****
; Ожидание подтверждения (ACK)
; *****
ack

```



```

        BIC.B #SDA, DIR ; выставить лог. «1»
                        ; на линии SDA
        BIC.B #SCL, DIR ; выставить лог. «1»
                        ; на линии SCL
; проверка удлинения тактового сигнала
; (clock low extending), чтобы не опережать ведомое
; устройство
        MOV #008h, R11 ; счётчик на 250 мкс
        MOV #02FFh, R11; удлинённый счётчик для
                        ; батареи PS100Z-200
clkex   MOV.B IN, R10   ; проверка шины
        DEC R11
        JZ busy        ; арбитраж - время
                        ; истекло
        AND.B #SCL, R10 ; проверка линии SCL
        JZ clkex
; ожидание подтверждения (ACK) (SDA переходит
; в лог. «0»)
        MOV #05F6h, R11; счётчик таймута
                        ; для NACK
wack    MOV.B IN, R10   ; проверка шины
        DEC R11
        JZ sbusy       ; таймаут NACK-- отпра-
                        ; вить бит Р и повторить
                        ; попытку
        AND.B #SDA, R10 ; маскировать всё,
                        ; кроме SDA
        JNZ wack       ; приём ACK или ожидание
                        ; таймута
        BIS.B #SCL, DIR ; выставить лог. «0»
                        ; на линии SCL
        RET
; *****
; Отправка стоповой последовательности
; *****
pbit    BIS.B #SDA, DIR ; выставить лог. «0»
                        ; на линии SDA
        BIS.B #SCL, DIR ; выставить лог. «0»
                        ; на линии SCL
        NOP
        BIC.B #SCL, DIR ; выставить лог. «1»
                        ; на линии SCL

```

```

NOP
BIC.B #SDA, DIR ; выставить лог. «1»
                  ; на линии SDA

RET

;--- очистка ЖКИ-----
show_clr
MOV #15, r5 ; очистка дисплейной
              ; памяти

show_clr1
MOV.b #0, LCD1-1(r5)
DEC r5
JNZ show_clr1
RET

;*****
; Определения для ЖКИ
;*****
LCD_TYPE
;---ЖКИ модуля STK/EVK-----
a .equ 01h
b .equ 02h
c .equ 10h
d .equ 04h
e .equ 80h
f .equ 20h
g .equ 08h
h .equ 40h
LCD_Tab .byte a+b+c+d+e+f ;отображает "0"
        .byte b+c ; отображает "1"
        .byte a+b+d+e+g ; отображает "2"
        .byte a+b+c+d+g ; отображает "3"
        .byte b+c+f+g ; отображает "4"
        .byte a+c+d+f+g ; отображает "5"
        .byte a+c+d+e+f+g ; отображает "6"
        .byte a+b+c ; отображает "7"
        .byte a+b+c+d+e+f+g ; отображает "8"
        .byte a+b+c+d+f+g ; отображает "9"
        .byte a+b+c+e+f+g ; отображает "A"
        .byte c+d+e+f+g; ;отображает "B" b
        .byte a+d+e+f ; отображает "C"
        .byte b+c+d+e+g ; отображает "D" d
        .byte a+d+e+f+g ; отображает "E"
        .byte a+e+f+g ; отображает "F"

LCD_Tab_End
        .even

```

```
;*****
; Вектора прерываний
;*****
        .sect  "Int_Vect",05FFh-1  ; с программой
                                   ; монитора
        .word  RESET                ; POR, внеш.
                                   ; Reset,
                                   ; Watchdog

        .end
```

Приложение С. Программа работы со смарт-батареях

```
;*****
; Программа SBData
; -----
; Читает температуру, напряжение, производителя,
; хим. конфигурацию, ёмкость, и предупреждение
; об оставшейся ёмкости из монитора батареи с шиной
; SMBus.
; Также производится запись в регистр «Предупреждение
; об оставшейся ёмкости» RemainingCapacityAlarm).
;
; В данном частном случае применялся батарейный
; смарт-модуль PowerSmart;PS100Z-200
; Другие данные из батареи можно читать, меняя команды.
; Контроллер смарт-батареи PowerSmart поддерживает
; стандарт SMBus V1.0 и спецификацию Smart Battery
; data
; Настоящая конфигурация программы написана для
; использования с пользовательской программой под
; управлением программы монитора на отладочной плате
; 337 EVK, подпрограммы SMBus находятся в EPROM. Также
; требуется Таблица ASCII символов для ЖКИ
; Здесь использован файл ASCII.txt. На него ссылается
; указатель alcd.
;*****
RAM_orig      .set    00240h      ; Стартовый адрес
SP_orig       .set    005DEh      ; указатель стека
EPROM         .set    08850h      ; Размещение EPROM

;--- Определения управляющих регистров-----
WDTCTL        .equ    0120h
WDTHold       .equ    80h
WDT_wrkey     .equ    05A00h
;PORT 0
```

```

P0IE      .equ    015h
P0DIR     .equ    012h
P0IN      .equ    010h
P0OUT     .equ    011h
; port 2
P2IN      .equ    028h
P2OUT     .equ    029h
P2IE      .equ    02Dh
P2SEL     .equ    02Eh
P2DIR     .equ    02Ah
SDA        .equ    020h
SCL        .equ    080h
DNC        .equ    0A0h
OUT        .equ    P2OUT
DIR        .equ    P2DIR
IN         .equ    P2IN
LCD1       .equ    031h
LCDM       .equ    030h
IE1        .equ    0h
IE2        .equ    01h
IFG1       .equ    02h
IFG2       .equ    03h
block      .equ    0550h
alcd       .equ    0B500h      ; Размещение таблицы
                                ; символов ASCII для ЖКИ
;*****
; Reset : Initialize processor
;*****
                .sect  "MAIN",RAM_orig
RESET
                MOV    #SP_orig,SP; Инициализация указателя стека
                                ;
;*****
; Начало пользовательской программы
;*****
repeat
                CMP.B  #0FFh, R8
                JNZ    skp
                INC.B  R7      ; инкремент старшего
                                ; байта значения
; для протокола записи слова
                MOV.B  #00h, R8
skp
                INC.B  R8      ; инкремент значения

```

```

; для протокола записи
; слова
skp0
; Чтение температуры
    CALL #show_clr
    PUSH #00h ; место для кода ошибки
    PUSH #000Bh ; поместить адрес в стек
    PUSH #008h ; поместить команду
                  ; чтения температуры
                  ; батареи
    PUSH #00h ; зарезервировать байт
                  ; для данных
    PUSH #00h ; зарезервировать байт
                  ; для данных
    CALL #rwp
    POP R10 ; поместить данные в R10
    POP R11 ; поместить данные в R11
    ADD #04h, SP ; освободить место
                  ; команды и адреса
    POP R9 ; читать код ошибки
    CMP #00h, R9
    JZ skp0 ; повтор при неудаче
    RLA R11
    RLA R11
    RLA R11
    RLA R11
    RLA R11
    RLA R11
    RLA R11
    RLA R11
    XOR R10, R11 ; объединить 2 байта
                  ; данных
    MOV R11, R12
    SUB #0AAAh, R12; перевод в градусы
                  ; Цельсия
    CALL #display ; вызов подпрограммы
                  ; отображения
    XOR.B #40h, LCD1+1
; добавить десятичную точку
    MOV.B alcd+'T', LCD1+5
    CALL #delay ; вызов подпрограммы
                  ; задержки
skp1
    CALL #show_clr ; чтение напряжения

```

```

PUSH    #00h           ; место для кода ошибки
PUSH    #000Bh         ; поместить адрес в стек
PUSH    #009h          ; поместить в стек
                        ; команду чтения напря-
                        ; жения батареи
PUSH    #00h           ; зарезервировать байт
                        ; для данных
PUSH    #00h           ; зарезервировать байт
                        ; для данных
CALL    #rwp
POP     R10             ; поместить данные в R10
POP     R11             ; поместить данные в R11
ADD     #04h, SP        ; освободить место
                        ; команды и адреса
POP     R9              ; читать код ошибки
CMP     #00h, R9
JZ      skip1           ; повтор при неудаче
RLA     R11
RLA     R11
RLA     R11
RLA     R11
RLA     R11
RLA     R11
RLA     R11
RLA     R11
XOR     R10, R11        ; объединить 2 байта
                        ; данных
MOV     R11, R12
CALL    #display        ; вызов подпрограммы
                        ; отображения
XOR.B   #40h, LCD1+3
; добавить десятичную точку
MOV.B   alcd+'U', LCD1+5
CALL    #delay          ; вызов подпрограммы
                        ; задержки
skip2    ; чтение названия производителя
CALL    #show_clr
PUSH    #00h           ; место для кода ошибки
PUSH    #000Bh         ; поместить адрес в стек
PUSH    #0020h         ; поместить в стек
                        ; команду чтения имени
                        ; производителя
PUSH    #block         ; зарезервировать байт
                        ; для данных

```

```

        PUSH    #00h          ; зарезервировать байт
                                ; для данных
        CALL    #blkcr
        POP     R10           ; поместить данные в R10
        ADD     #06h, SP
        POP     R9            ; читать код ошибки
        CMP     #00h, R9
        JZ      skip2         ; повтор при неудаче
        MOV.B   block, R5     ; копировать символ
        MOV.B   alcd(R5), LCD1+6
; отобразить символ
        MOV.B   block+1, R5 ; копировать символ
        MOV.B   alcd(R5), LCD1+5
; отобразить символ
        MOV.B   block+2, R5 ; копировать символ
        MOV.B   alcd(R5), LCD1+4
; отобразить символ
        MOV.B   block+3, R5 ; копировать символ
        MOV.B   alcd(R5), LCD1+3
; отобразить символ
        MOV.B   block+4, R5 ; копировать символ
        MOV.B   alcd(R5), LCD1+2
; отобразить символ
        MOV.B   block+5, R5 ; копировать символ
        MOV.B   alcd(R5), LCD1+1
; отобразить символ
        MOV.B   block+6, R5 ; копировать символ
        MOV.B   alcd(R5), LCD1+0
; отобразить символ
        CALL    #delay        ; вызов подпрограммы
                                ; задержки
skip3    ; чтение химической конфигурации
        CALL    #show_clr
        PUSH    #00h          ; место для кода ошибки
        PUSH    #000Bh        ; поместить адрес в стек
        PUSH    #0022h        ; поместить в стек
                                ; команду
; чтения химической конфигурации
        PUSH    #block        ; зарезервировать байт
                                ; для данных
        PUSH    #00h          ; зарезервировать байт
                                ; для данных
        CALL    #blkcr
        POP     R10           ; поместить данные в R10

```

```

        ADD    #06h, SP
        POP    R9          ; читать код ошибки
        CMP    #00h, R9
        JZ     skip3       ; повтор при неудаче
        MOV.B  block, R5   ; копировать символ
        MOV.B  alcd(R5), LCD1+6
; отобразить символ
        MOV.B  block+1, R5 ; копировать символ
        MOV.B  alcd(R5), LCD1+5
; отобразить символ
        MOV.B  block+2, R5 ; копировать символ
        MOV.B  alcd(R5), LCD1+4
; отобразить символ
        MOV.B  block+3, R5 ; копировать символ
        MOV.B  alcd(R5), LCD1+3
; отобразить символ
        CALL   #delay      ; вызов подпрограммы
                          ; задержки
skip4    ; чтение ёмкости батареи
        CALL   #show_clr
        PUSH   #00h        ; место для кода ошибки
        PUSH   #000Bh      ; поместить адрес в стек
        PUSH   #0018h      ; поместить в стек
                          ; команду чтения ёмкости
                          ; батареи
        PUSH   #00h        ; зарезервировать байт
                          ; для данных
        PUSH   #00h        ; зарезервировать байт
                          ; для данных
        CALL   #rwp
        POP    R10         ; поместить данные в R10
        POP    R11         ; поместить данные в R11
        ADD    #04h, SP    ; освободить место
                          ; команды и адреса
        POP    R9          ; читать код ошибки
        CMP    #00h, R9
        JZ     skip4       ; повтор при неудаче
        RLA    R11
        RLA    R11
        RLA    R11
        RLA    R11
        RLA    R11
        RLA    R11
        RLA    R11

```



```

        RLA      R11
        XOR      R10, R11
        MOV      R11, R12
        CALL     #display      ; вызов подпрограммы
                                ; отображения
        XOR.B    #40h, LCD1+3

; добавить десятичную точку
        MOV.B    alcd+'A', LCD1+1
        MOV.B    alcd+'H', LCD1+0
        CALL     #delay        ; вызов подпрограммы
                                ; задержки
skp5    ; запись предупреждения
        ; об оставшейся ёмкости
        (RemainingCapacityAlarm)
        CALL     #show_clr
        PUSH     #00h          ; место для кода ошибки
        PUSH     #000Bh        ; поместить адрес в стек
        PUSH     #001h        ; поместить в стек
                                ; команду
        PUSH     R8             ; зарезервировать байт
                                ; для данных
        PUSH     R7             ; зарезервировать байт
                                ; для данных
        CALL     #wwp
        POP      R11            ; поместить данные в R11
        POP      R10            ; поместить данные в R10
        ADD      #04h, SP       ; освободить место
                                ; команды и адреса
        POP      R9             ; читать код ошибки
        CMP      #00h, R9
        JZ       skp5          ; повтор при неудаче
        RLA      R11
        RLA      R11
        RLA      R11
        RLA      R11
        RLA      R11
        RLA      R11
        RLA      R11
        RLA      R11
        XOR      R10, R11
        MOV      R11, R12
        CALL     #display      ; вызов подпрограммы
                                ; отображения
        MOV.B    alcd+'S', LCD1+6

```

```

CALL    #delay      ; вызов подпрограммы
                        ; задержки
skp6    ; считывание оставшейся ёмкости
        ; (RemainingCapacityAlarm)
CALL    #show_clr
PUSH    #00h         ; место для кода ошибки
PUSH    #000Bh       ; поместить адрес в стек
PUSH    #001h        ; поместить в стек
                        ; команду
PUSH    #00h         ; зарезервировать байт
                        ; для данных
PUSH    #00h         ; зарезервировать байт
                        ; для данных
CALL    #rwp
POP      R10         ; поместить данные в R10
POP      R11         ; поместить данные в R11
ADD      #04h, SP    ; освободить место
                        ; команды и адреса
POP      R9          ; читать код ошибки
CMP      #00h, R9
JZ       skp6        ; повтор при неудаче
RLA      R11
RLA      R11
RLA      R11
RLA      R11
RLA      R11
RLA      R11
RLA      R11
RLA      R11
XOR      R10, R11
MOV      R11, R12
CALL     #display    ; вызов подпрограммы
                        ; отображения
MOV.B    alcd+'R', LCD1+6
CALL     #delay      ; вызов подпрограммы
                        ; задержки
JMP      repeat      ; завершение программы,
                        ; бесконечный цикл
;*****
; Окончание пользовательской программы
;*****
delay
PUSH     R6
PUSH     R7

```

```

MOV    #00DFh, R7 ; вложенный цикл
                ; задержки
dela4      MOV    #0D40h, R6
dela3      DEC    R6
                JNZ    dela3
                DEC    R7
                JNZ    dela4
                POP    R7
                POP    R6
                RET

display    ; подпрограмма отображения
                PUSH   R5
                PUSH   R12
                PUSH   R13
                PUSH   R14
                PUSH   R15
                MOV    #16, R15 ; процедура преобразова-
                ; ния hex ? BCD
                CLR    R14      ; входные значения
                ; в R12, выходные в R13
                ; и R14
                CLR    R13
ls1         RLA    R12
                DADD   R13, R13
                DADD   R14, R14
                DEC    R15
                JNZ    ls1
                MOV.B  R13, R5 ; поместить 2 младших
                ; BCD-разряда в R5
                AND    #000Fh, R5 ; маскировать всё, кроме
                ; младшей тетрады
                MOV.B  LCD_Tab(R5), LCD1+0
; Отобразить младшую тетраду
                MOV.B  R13, R5
; поместить 2 младших BCD-разряда в R5
                AND    #00F0h, R5 ; маскировать всё, кроме
                ; старшей тетрады
                RRA    R5 ; сдвиг вправо
                RRA    R5 ; сдвиг вправо
                RRA    R5 ; сдвиг вправо
                RRA    R5 ; сдвиг вправо
                MOV.B  LCD_Tab(R5), LCD1+1
; Отобразить старшую тетраду
                SWPB   R13 ; обмен байтами в R13

```

```

MOV.B R13, R5      ; поместить 2 старших
                   ; BCD-разряда в R5
AND    #000Fh, R5   ; маскировать всё, кроме
                   ; младшей тетрады
MOV.B LCD_Tab(R5), LCD1+2
; Отобразить младшую тетраду
MOV.B R13, R5      ; поместить 2 старших
                   ; BCD-разряда в R5
AND    #00F0h, R5   ; маскировать всё, кроме
                   ; старшей тетрады
RRA     R5           ; сдвиг вправо
RRA     R5           ; сдвиг вправо
RRA     R5           ; сдвиг вправо
RRA     R5           ; сдвиг вправо
MOV.B LCD_Tab(R5), LCD1+3
; Отобразить старшую тетраду
MOV.B R14, R5      ; поместить 2 младших
                   ; BCD-разряда в R5
AND    #000Fh, R5   ; маскировать всё, кроме
                   ; младшей тетрады
MOV.B LCD_Tab(R5), LCD1+4
; Отобразить младшую тетраду
MOV.B R14, R5      ; поместить 2 младших
                   ; BCD-разряда в R5
AND    #00F0h, R5   ; маскировать всё, кроме
                   ; старшей тетрады
RRA     R5           ; сдвиг вправо
RRA     R5           ; сдвиг вправо
RRA     R5           ; сдвиг вправо
RRA     R5           ; сдвиг вправо
MOV.B LCD_Tab(R5), LCD1+5
; Отобразить старшую тетраду
POP     R15
POP     R14
POP     R13
POP     R12
POP     R5
RET

.sect "smbus", EPROM
;*****
; Подпрограммы верхнего уровня, п/п нижнего уровня
; вызываются индивидуально для реализации различных
; протоколов

```

```

;*****
;*****
; Протокол «Быстрая команда» 1
; =====
; Влияние на регистры: нет
; Влияние на память: стек, максимум 40 байт
; -----
; Описание      Шаг                                     Направление
; -----
;               старт (start)                             из
;               адрес(address)                             из
;               чтение/запись (R/W)                         из
;               подтверждение (acknowledge) в ?
;               стоп (stop)                                 из
;
; Параметры в стеке
; -----
; флаг ошибки error_flag      младшая область памяти
;                               low mem
; не используется      unused
; не используется      unused
; не используется      unused
; адрес                  address      старшая область памяти
;                               high mem
;
; Пример использования
; -----
;               PUSH    #00h          ; место для кода ошибки
;               PUSH    #00Bh         ; поместить адрес в стек
;               SUB      #06h, SP      ; остальные параметры не
;                                       ; используются
;
;               CALL    #qscr
;               ADD      #08h, SP      ; вычесть объем парамет-
;                                       ; ров
;               POP      R9            ; читать из стека код
;                                       ; ошибки
;
;               CMP      #00h, R9
;               JZ       error         ; ошибочное завершение -
;                                       ; вызов обработчика
;                                       ; ошибок
;*****
qscr
        PUSH    SR
        PUSH    R7

```

```

        PUSH    R8
        PUSH    R9
        PUSH    R14
        PUSH    R15
        PUSH    R10
        PUSH    R11
        PUSH    R12
        PUSH    R13
        MOV.B   28(SP), R7 ; получить адрес из
                           ; стека
        CALL    #sbit      ; отправить стартовую
                           ; посылку и адрес
;        CALL    #sendone   ; отправить «1» или
        CALL    #sendzero  ; отправить «0»
        CALL    #ack       ; ждём подтверждения
                           ; (ACK)
        CALL    #pbit      ; отправить стоповую
                           ; посылку
        MOV.B   #01h, 30(SP); возвращает «1» при
                           ; успешном завершении

        POP     R13
        POP     R12
        POP     R11
        POP     R10
        POP     R15
        POP     R14
        POP     R9
        POP     R8
        POP     R7
        POP     SR
        RET
; *****
; Протокол «Записать байт» 4
; =====
; Влияние на регистры: нет
; Влияние на память: стек, максимум 40 байт
; -----
; Описание      Шлаг                                     Направление
; -----
;                старт (start)                             из
;                адрес (address)                           из
;                чтение/запись (R/W)                       из
;                подтверждение (acknowledge) в ?
;                команда (command)                         из

```

```

;           подтверждение(acknowledge) в ?
;           данные (data)                      из
;           подтверждение(acknowledge) в ?
;           стоп (stop)                        из
;
; Параметры в стеке
; -----
; флаг ошибкиerror_flag           младшая область памяти
;                                low mem
; не используется   unused
; данные            data
; команда           command
; адрес             address      старшая область памяти
;                                high mem
;
; Пример использования
; -----
;           PUSH   #00h           ; место для кода ошибки
;           PUSH   #00Bh          ; поместить адрес в стек
;           PUSH   #001h          ; поместить команду
;                                ; в стек
;           PUSH   #055h          ; отправляемые данные
;           SUB     #02h, SP       ; остальные параметры
;                                ; не используются
;           CALL    #wbp
;           ADD     #08h, SP       ; вычесть объём парамет-
;                                ; ров
;           POP     R9             ; читать из стека код
;                                ; ошибки
;           CMP     #00h, R9
;           JZ      error          ; ошибочное завершение -
;                                ; вызов обработчика
;                                ; ошибок
wbp
        PUSH     SR
        PUSH     R7
        PUSH     R8
        PUSH     R9
        PUSH     R14
        PUSH     R15
        PUSH     R10
        PUSH     R11
        PUSH     R12
        PUSH     R13

```

```

MOV.B 26(SP), R8 ; параметр команды
MOV.B 28(SP), R7 ; параметр адреса
CALL #sbit      ; отправить стартовую
                ; посылку и адрес
CALL #sendzero  ; отправить «0» для
                ; записи
CALL #ack       ; ждём подтверждения
                ; (ACK)
CALL #sbyte     ; отправить код команды
CALL #ack       ; ждём подтверждения
                ; (ACK)
MOV.B 24(SP), R8 ; отправка данных
CALL #sbyte     ; отправить байт данных
CALL #ack       ; ждём подтверждения
                ; (ACK)
CALL #pbit      ; отправить стоповую
                ; посылку
MOV.B #01h, 30(SP) ; возвращает «1»
                ; при успешном заверше-
                ; нии

POP R13
POP R12
POP R11
POP R10
POP R15
POP R14
POP R9
POP R8
POP R7
POP SR
RET

```

```

; *****
; Протокол «Записать слово» 5
; =====
; Влияние на регистры: нет
; Влияние на память: стек, максимум 40 байт
; -----
; Описание      Шаг                                     Направление
; -----
;               старт (start)                             из
;               адрес (address)                           из
;               чтение/запись (R/W)                       из
;               подтверждение (acknowledge) в ?
;               данные (data)                               из

```



```

;           подтверждение(acknowledge) в ?
;           данные (data)           из
;           подтверждение(acknowledge) в ?
;           стоп (stop)                     из
; Параметры в стеке
; -----
; флаг ошибкиerror_flag                младшая область памяти
;                                     low mem
; не используется      unused
; данные                data
; команда               command
; адрес                address          старшая область памяти
;                                     high mem
; -----
; Пример использования
; -----
;           PUSH    #00h                ; место для кода ошибки
;           PUSH    #00Bh              ; поместить адрес в стек
;           PUSH    #001h              ; код команды
;           PUSH    #0AAh              ; младший отправляемый
;                                     ; байт
;           PUSH    #055h              ; старший отправляемый
;                                     ; байт
;           CALL    #wwp
;           ADD     #08h, SP            ; освободить место,
;                                     ; занимаемое параметрами
;           POP     R9                  ; читать из стека код
;                                     ; ошибки
;           CMP     #00h, R9
;           JZ      error              ; ошибочное завершение -
;                                     ; вызов обработчика
;                                     ; ошибок
; *****
wwp
           PUSH    SR
           PUSH    R7
           PUSH    R8
           PUSH    R9
           PUSH    R14
           PUSH    R15
           PUSH    R10
           PUSH    R11
           PUSH    R12
           PUSH    R13

```

```

MOV.B 26(SP), R8 ; параметр команды
MOV.B 28(SP), R7 ; параметр адреса
CALL #sbit      ; отправить стартовую
                ; посылку и адрес
CALL #sendzero  ; отправить «0»
                ; для записи
CALL #ack       ; ждём подтверждения
                ; (ACK)
CALL #sbyte     ; отправить код команды
CALL #ack       ; ждём подтверждения
                ; (ACK)
MOV.B 24(SP), R8 ; данные для записи -
                ; младший байт
CALL #sbyte     ; отправить младший байт
                ; данных
CALL #ack       ; ждём подтверждения
                ; (ACK)
MOV.B 22(SP), R8 ; данные для записи -
                ; старший байт
CALL #sbyte     ; отправить старший байт
                ; данных
CALL #ack       ; ждём подтверждения
                ; (ACK)
CALL #pbit      ; отправить стоповую
                ; посылку
MOV.B #01h, 30(SP) ; возвращает «1» при
                ; успешном завершении

POP R13
POP R12
POP R11
POP R10
POP R15
POP R14
POP R9
POP R8
POP R7
POP SR
RET
; *****

; Протокол «Читать байт» 6
; =====
; Влияние на регистры: нет
; Влияние на память: стек, максимум 40 байт
; -----

```

; Описание	Шаг	Направление
; -----		
;	старт (start)	из
;	адрес (address)	из
;	чтение/запись (R/W)	из
;	подтверждение (acknowledge) в ?	
;	команда (command)	из
;	подтверждение (acknowledge) в ?	
;	старт (start)	из
;	адрес (address)	из
;	чтение/запись (R/W)	из
;	подтверждение (acknowledge) в ?	
;	данные (data)	в ?
;	неподтверждение (NACK)	из
;	стоп (stop)	из
;		
; Параметры в стеке		
; -----		
; флаг ошибки	error_flag	младшая область памяти low mem
;		
; младший байт		
; данных	data	
; не используется	unused	
; команда	command	
; адрес	address	старшая область памяти high mem
; -----		
; Пример использования		
; -----		
;	PUSH #00h	; место для кода ошибки
;	PUSH #000Bh	; поместить адрес в стек
;	PUSH #008h	; код команды чтения
;		
;	PUSH #00h	; температуры батареи
;		
;	PUSH #00h	; зарезервировать байт
;		
;	PUSH #00h	; для данных (не исполь-
;		
;	PUSH #00h	; зуется)
;		
;	PUSH #00h	; зарезервировать байт
;		
;	PUSH #00h	; для данных
;		
;	CALL #rbp	
;	POP R10	; переместить данные
;		
;	POP R10	; в R10
;		
;	ADD #06h, SP	; освободить место
;		
;	POP R9	; команды и адреса
;		
;	POP R9	; читать из стека код

```

;                                ; ошибки
;      CMP      #00h, R9
;      JZ       error           ; ошибочное завершение -
                                ; вызов обработчика
                                ; ошибок

;*****
rbp
    PUSH  SR
    PUSH  R7
    PUSH  R8
    PUSH  R9
    PUSH  R14
    PUSH  R15
    PUSH  R10
    PUSH  R11
    PUSH  R12
    PUSH  R13
    MOV.B 26(SP), R8 ; параметр команды
    MOV.B 28(SP), R7 ; параметр адреса
    CALL  #sbit      ; отправить стартовую
                    ; посылку и адрес
    CALL  #sendzero  ; отправить «0» для
                    ; записи
    CALL  #ack        ; ждём подтверждения
                    ; (ACK)
    CALL  #sbyte      ; отправить код команды
    CALL  #ack        ; ждём подтверждения
                    ; (ACK)
    CALL  #rsbit      ; отправить повторяющие-
                    ; ся стартовую посылку
                    ; и адрес
    CALL  #sendone    ; отправить «1» для
                    ; чтения
    CALL  #ack        ; ждём подтверждения
                    ; (ACK)
    CALL  #rbyte      ; принять байт данных
    CALL  #nack       ; отправить неподтверж-
                    ; дение (NACK)
    CALL  #pbit       ; отправить стоповую
                    ; посылку
    MOV.B R14, 22(SP) ; копировать байт данных
                    ; в стек
    MOV.B #01h, 30(SP); возвращает «1» при
                    ; успешном завершении

```

```

        POP      R13
        POP      R12
        POP      R11
        POP      R10
        POP      R15
        POP      R14
        POP      R9
        POP      R8
        POP      R7
        POP      SR
        RET

; *****
; Протокол «Читать слово» 7
; =====
; Влияние на регистры: нет
; Влияние на память: стек, максимум 40 байт
; -----
; Описание      Шаг                                Направление
; -----
;      старт (start)                                из
;      адрес (address)                              из
;      чтение/запись (R/W)                          из
;      подтверждение (acknowledge) в ?
;      команда (command)                            из
;      подтверждение (acknowledge) в ?
;      старт (start)                                из
;      адрес (address)                              из
;      чтение/запись (R/W)                          из
;      подтверждение (acknowledge) в ?
;      данные (data)                                в ?
;      подтверждение (acknowledge) в ?
;      данные (data)                                в ?
;      подтверждение (acknowledge) в ?
;      стоп (stop)                                  из
;
; Параметры в стеке
; -----
; флаг ошибки error_flag                младшая область памяти
;                                       low mem
; младший байт
; данных                                low data byte
; старший байт
; данных                                high data byte
; команда                                command

```

```

; адрес                      address      старшая область памяти
;                               high mem

; -----
; Пример использования
; -----
;
;          PUSH    #00h          ; место для кода ошибки
;          PUSH    #00Bh        ; поместить адрес в стек
;          PUSH    #008h        ; код команды "темпера-
;                               ; тура батареи"
;          PUSH    #00h          ; зарезервировать байт
;                               ; под принимаемые данные
;          PUSH    #00h          ; зарезервировать байт
;                               ; под принимаемые данные
;
;          CALL    #rwp
;          POP     R10           ; переместить данные
;                               ; в R10
;          POP     R11           ; переместить данные
;                               ; в R11
;          ADD     #04h, SP      ; освободить место,
;                               ; занимаемое командой
;                               ; и адресом
;          POP     R9            ; читать из стека код
;                               ; ошибки
;
;          CMP     #00h, R9
;          JZ      error        ; ошибочное завершение -
;                               ; вызов обработчика
;                               ; ошибок
; *****
rwp
          PUSH    SR
          PUSH    R7
          PUSH    R8
          PUSH    R9
          PUSH    R14
          PUSH    R15
          PUSH    R10
          PUSH    R11
          PUSH    R12
          PUSH    R13
          MOV.B   26(SP), R8     ; параметр команды
          MOV.B   28(SP), R7     ; параметр адреса
          CALL    #sbit          ; отправить стартовую
;                               ; посылку и адрес
          CALL    #sendzero      ; отправить «0» для

```

```

; записи
CALL #ack ; ждём подтверждения
; (ACK)
CALL #sbyte ; отправить код команды
CALL #ack ; ждём подтверждения
; (ACK)
CALL #rsbit ; отправить повторяющие-
; ся стартовую посылку
; и адрес
CALL #sendone ; отправить «1» для
; чтения
CALL #ack ; ждём подтверждения
; (ACK)
CALL #rbyte ; принять младший байт
; данных
MOV.B R14, 22(SP) ; сохранить младший байт
; данных в стеке
CALL #sack ; ждём подтверждения
; (ACK)
CALL #rbyte ; принять старший байт
; данных
CALL #nack ; отправить неподтверж-
; дение (NACK)
CALL #pbit ; отправить стоповую
; посылку
MOV.B R14, 24(SP) ; сохранить старший байт
; данных в стеке
MOV.B #01h, 30(SP) ; возвращает «1» при
; успешном завершении

POP R13
POP R12
POP R11
POP R10
POP R15
POP R14
POP R9
POP R8
POP R7
POP SR
RET

```

```

; *****
; Протокол «Запись блока» 8
; =====
; Влияние на регистры: нет

```

```

; Влияние на память: стек, максимум 40 байт,
; блок памяти

; Описание      Шаг                               Направление
; -----
;               старт (start)                      из
;               адрес (address)                     из
;               чтение/запись (R/W)                  из
;               подтверждение (acknowledge) в ?
;               команда (command)                   из
;               подтверждение (acknowledge) в ?
;               счётчик байт (byte count)            из
;               подтверждение (acknowledge) в ?
;               данные (data)                       из
;               подтверждение (acknowledge) в ?
;
;               данные (data)                       в ?
;               подтверждение (acknowledge) в ?
;
;               данные и ACK повторяются n раз
;               стоп (stop)                         из

; Параметры в стеке
; -----
; флаг ошибки error_flag                младшая область памяти
;                                       low mem
; счётчик байт      byte count
; указатель блока   block pointer
; команда           command
; адрес            address                старшая область памяти
;                                       high mem

; -----
; Пример использования
; -----
;               PUSH    #00h                ; место для кода ошибки
;               PUSH    #00Bh              ; поместить адрес в стек
;               PUSH    #0020h              ; код команды "темпера-
;                                       ; тура батареи"
;               PUSH    #00550h             ; указатель на начало
;                                       ; блока
;               PUSH    #00h                ; зарезервировать байт
;                                       ; под счётчик
;               CALL    #blkw
;               POP     R10                 ; переместить данные
;                                       ; в R10

```



```

;          ADD    #06h, SP
;          POP     R9          ; читать из стека код
;                               ; ошибки
;          CMP     #00h, R9
;          JZ      error      ; ошибочное завершение –
;                               ; вызов обработчика
;                               ; ошибок
;*****
blkw
        PUSH     SR
        PUSH     R7
        PUSH     R8
        PUSH     R9
        PUSH     R14
        PUSH     R15
        PUSH     R10
        PUSH     R11
        PUSH     R12
        PUSH     R13
        MOV.B    26(SP), R8    ; параметр команды
        MOV.B    28(SP), R7    ; параметр адреса
        MOV      24(SP), R9    ; стартовый адрес блока
        MOV.B    #00h, R15     ; циклический байтовый
                               ; счётчик
        CALL     #sbit         ; отправить стартовую
                               ; посылку и адрес
        CALL     #sendzero     ; отправить «0» для
                               ; записи
        CALL     #ack          ; ждём подтверждения
                               ; (ACK)
        CALL     #sbyte        ; отправить код команды
        CALL     #ack          ; ждём подтверждения
                               ; (ACK)
        MOV.B    22(SP), R8    ; копировать счётчик
                               ; байт
        CALL     #sbyte        ; отправить счётчик байт
        CALL     #ack          ; ждём подтверждения
                               ; (ACK)
blk_rep  MOV.B    0(R9), R8    ; передать данные
                               ; из блока
        CALL     #sbyte        ; отправить байт данных
        CALL     #ack          ; ждём подтверждения
                               ; (ACK)
        INC      R15          ; инкремент счётчика

```

```

; положения блока
INC    R9          ; инкремент указателя
; блока
CMP    22(SP), R15 ; проверка завершения
; данных
JNZ    blk_rep     ; повтор до завершения
CALL   #pbit       ; отправить стоповую
; посылку
MOV.B  #01h, 30(SP) ; возвращает «1» при
; успешном завершении

POP    R13
POP    R12
POP    R11
POP    R10
POP    R15
POP    R14
POP    R9
POP    R8
POP    R7
POP    SR
RET

```

```

; *****
; Протокол «Чтение блока» 9
; =====
; Влияние на регистры: нет
; Влияние на память: стек, максимум 40 байт,
; блок памяти
; Описание      Шаг                                     Направление
; -----
;      старт (start)                                     из
;      адрес (address)                                   из
;      запись (W)                                         из
;      подтверждение (acknowledge) в ?
;      команда (command)                                 из
;      подтверждение (acknowledge) в ?
;      старт (start)                                     из
;      адрес (address)                                   из
;      чтение (R)                                         из
;      подтверждение (acknowledge) в ?
;      счётчик байт (byte count)                         в ?
;      подтверждение (acknowledge) из
;      данные (data)                                     в ?
;      подтверждение (acknowledge) из
;

```

```

;           данные и ACK повторяются n раз
;           неподтверждение (NACK)           из
;           стоп (stop)                       из
; Параметры в стеке
; -----
; флаг ошибки error_flag                     младшая область памяти
;                                           low mem
; счётчик байт      byte count
; указатель блока   block pointer
; команда           command
; адрес            address                     старшая область памяти
;                                           high mem
; -----
; Пример использования
; -----
;           PUSH    #00h           ; место для кода ошибки
;           PUSH    #00Bh          ; поместить адрес в стек
;           PUSH    #0020h         ; код команды "темпера-
;                                   ; тура батареи"
;           PUSH    #00550h        ; указатель на начало
;                                   ; блока
;           PUSH    #00h           ; зарезервировать байт
;                                   ; под счётчик
;           CALL    #blkcr
;           POP     R10            ; переместить счётчик
;                                   ; в R10
;           ADD     #06h, SP
;           POP     R9             ; читать из стека код
;                                   ; ошибки
;           CMP     #00h, R9
;           JZ      error          ; ошибочное завершение -
;                                   ; вызов обработчика
;                                   ; ошибок
; *****
blkcr
           PUSH    SR
           PUSH    R7
           PUSH    R8
           PUSH    R9
           PUSH    R14
           PUSH    R15
           PUSH    R10
           PUSH    R11
           PUSH    R12

```

```

PUSH    R13
MOV.B   #00h, R15 ; циклический байтовый
                ; счётчик
MOV.B   26(SP), R8 ; параметр команды
MOV.B   28(SP), R7 ; параметр адреса
MOV     24(SP), R9 ; стартовый адрес блока
CALL    #sbit      ; отправить стартовую
                ; посылку и адрес
CALL    #sendzero  ; отправить «0» для
                ; записи
CALL    #ack        ; ждём подтверждения
                ; (ACK)
CALL    #sbyte      ; отправить код команды
CALL    #ack        ; ждём подтверждения
                ; (ACK)
CALL    #rsbit      ; отправить повторяющие-
                ; ся стартовую посылку
                ; и адрес
CALL    #sendone    ; отправить «1» для
                ; чтения
CALL    #ack        ; ждём подтверждения
                ; (ACK)
CALL    #rbyte
MOV.B   R14, 22(SP) ; поместить счётчик
                ; в стек
CALL    #sack       ; отправить подтвержде-
                ; ние (ACK)
rblk_rep CALL #rbyte ; принять байт данных
MOV.B   R14, 0(R9)  ; переместить данные
                ; из блока
INC     R15         ; инкремент счётчика
                ; положения блока
INC     R9          ; инкремент указателя
                ; блока
CMP     22(SP), R15 ; проверка завершения
                ; данных
JZ      blk_done    ; повтор до завершения
CALL    #sack       ; отправить подтвержде-
                ; ние (ACK)
CMP     22(SP), R15 ; проверка завершения
                ; данных
blk_done JNZ    rblk_rep ; повтор до завершения
CALL    #nack       ; отправить неподтверж-
                ; дение (NACK)

```

```

CALL    #pbit          ; отправить стоповую
                        ; послылку
MOV.B   #01h, 30(SP)   ; возвращает «1» при
                        ; успешном завершении

POP     R13
POP     R12
POP     R11
POP     R10
POP     R15
POP     R14
POP     R9
POP     R8
POP     R7
POP     SR
RET

; *****
; Подпрограммы нижнего уровня, общие для всех
; протоколов
; *****
; *****
; отправка стартовой последовательности и адреса
; *****
sbit          ; «отпустить» обе линии
        BIC.B #DNC, DIR
; проверка освобождения шины
        MOV    #05h, R10 ; счётчик на 50 микросе-
                        ; кунд

wait

        MOV.B  IN, R11   ; копировать состояние
                        ; входов в R11
        AND    #DNC, R11 ; маска входов
        CMP    #DNC, R11 ; если линии SDA или
                        ; SCL в лог. «0»то шина
                        ; занята
        JNZ    busy
        DEC    R10       ; декремент счётчика
        JNZ    wait

rsbit      BIC.B #DNC, DIR ; дополнительное
                        ; освобождение линии с
                        ; целью использования
                        ; этой же подпрограммы
                        ; для команды повторяю-
                        ; щегося старта
; отправка стартовой последовательности

```

```

        BIS.B  #SDA, DIR
        BIS.B  #SCL, DIR
; отправка адреса
        MOV.B  #07h, R13 ; счётчик на 7 бит
                           ; адреса
        MOV.B  R7, R11   ; копировать адрес в R11
ashift  RLA.B  R11       ; сдвиг влево, при этом
                           ; СЗР 7-битного
                           ; адреса будет находить-
                           ; ся на 7-й позиции
        MOV.B  R11, R12  ; скопировать его для
                           ; возможности маскирова-
                           ; ния без потери данных
        AND.B  #080h, R12 ; маскировать все биты
                           ; кроме СЗР
        CMP.B  #00h, R12 ; сравнить с 0
        JNZ    one
        CALL   #sendzero ; отправить «1»
        JMP    zero
one      CALL   #sendone  ; отправить «0»
zero     DEC    R13       ; декремент счётчика
        JNZ    ashift    ; если счётчик <7 бит,
                           ; повторить сдвиг

        RET

; *****
; подтверждение (ACK) не принято, отправить STOP
; и busy
; *****
sbusy    CALL   #pbit      ; отправить STOP и busy
        ADD    #02h, SP   ; удалить данные из сте-
                           ; ка, так как инструкция
                           ; RET не использовалась
        MOV.B  #00h, 28(SP); код ошибки
        POP    R13        ; восстановление регист-
                           ; ров
        POP    R12
        POP    R11
        POP    R10
        POP    R15
        POP    R14
        POP    R9
        POP    R8
        POP    R7
        POP    SR

```

```

RET
;*****
; Обработка состояния занятости (busy)
;*****
busy3      ADD    #06h, SP    ; удалить лишние данные
                                ; от подпрограмм
                                ; sendone или sendzero
busy       ADD    #02h, SP    ; удалить данные
                                ; из стека, так как
                                ; инструкция RET
                                ; не использовалась
MOV.B #00h, 28(SP) ; код ошибки
POP R13      ; восстановить регистры
POP R12
POP R11
POP R10
POP R15
POP R14
POP R9
POP R8
POP R7
POP SR
RET
;*****
; Отправить <1>
;*****
sendone    PUSH   R10
           PUSH   R11
           BIC.B  #SDA, DIR    ; установить лог. <1>
                                ; на линии SDA
           BIC.B  #SCL, DIR    ; установить лог. <1>
                                ; на линии SCL
; проверка удлинения тактового сигнала
; (clock low extending), чтобы не опережать ведомое
; устройство
           MOV.B  #04h, R11    ; установить число так-
                                ; тов для верного време-
                                ; ни
clkext     MOV.B  IN, R10      ; проверка шины
           DEC    R11
           JZ     busy3        ; арбитраж - время
                                ; истекло
           AND.B  #SCL, R10    ; проверка линии SCL
           JZ     clkext

```

```

; арбитраж: убедиться, что на линии SDA
; действительно лог. «1», иначе приоритет
; отдаётся другому ведущему устройству
MOV.B IN, R10      ; проверка шины
AND.B #SDA, R10    ; маскировать всё,
                   ; кроме SDA

JZ      busy3
BIS.B #SCL, DIR    ; освободить линию SCL
MOV.B IN, R10      ; определение повторяю-
                   ; щейся стартовой после-
                   ; довательности

AND.B #SDA, R10
JZ      busy3
POP     R11
POP     R10
RET

; *****
; Отправить «0»
; *****
sendzero
        PUSH    R10
        PUSH    R11
        BIS.B #SDA, DIR    ; выставить лог. «0»
                           ; на линии SDA
        BIC.B #SCL, DIR    ; выставить лог. «1»
                           ; на линии SCL

; проверка удлинения тактового сигнала (clock low
; extending), чтобы не опережать ведомое устройство
        MOV     #04h, R11  ; счётчик времени
                           ; перехода SCL в «1»
clke    MOV.B IN, R10      ; проверка шины
        DEC     R11
        JZ      busy3     ; арбитраж - время
                           ; истекло
        AND.B #SCL, R10   ; проверка линии SCL
        JZ      clke
        BIS.B #SCL, DIR   ; освободить линию SCL
        POP     R11
        POP     R10
        RET

; *****
; Ожидание подтверждения (ACK)
; *****
ack
```



```

        BIC.B #SDA, DIR ; выставить лог. «1»
                        ; на линии SDA
        BIC.B #SCL, DIR ; выставить лог. «1»
                        ; на линии SCL
; проверка удлинения тактового сигнала (clock low
; extending), чтобы не опережать ведомое устройство
        MOV #008h, R11 ; счётчик на 250 мкс
        MOV #02FFh, R11 ; удлинённый счётчик
                        ; для батареи PS100Z-200
clkex   MOV.B IN, R10   ; проверка шины
        DEC R11
        JZ busy        ; арбитраж – время
                        ; истекло
        AND.B #SCL, R10 ; проверка линии SCL
        JZ clkex
; ожидание подтверждения (ACK) (SDA переходит в лог.
; «0»)
        MOV #05F6h, R11 ; счётчик таймаута для
                        ; NACK
wack    MOV.B IN, R10   ; проверка шины
        DEC R11
        JZ sbusy       ; таймаут NACK-- отпра-
                        ; вить бит Р и повторить
                        ; попытку
        AND.B #SDA, R10 ; маскировать всё, кроме
                        ; SDA
        JNZ wack       ; приём ACK или ожидание
                        ; таймаута
        BIS.B #SCL, DIR ; выставить лог. «0»
                        ; на линии SCL
        RET
; *****
; Отправка стоповой последовательности
; *****
pbit    BIS.B #SDA, DIR ; выставить лог. «0»
                        ; на линии SDA
        BIS.B #SCL, DIR ; выставить лог. «0»
                        ; на линии SCL
        NOP
        BIC.B #SCL, DIR ; выставить лог. «1»
                        ; на линии SCL
        NOP

```

```

        BIC.B #SDA, DIR    ; выставить лог. «1»
                           ; на линии SDA
        RET
;*****
; Отправка байта (используются также sendzero
; и sendone)
;*****
sbyte
; отправка данных
        MOV.B #08h, R13    ; счётчик на 250 мкс
        MOV.B R8, R11      ; копировать данные
                           ; в R11
dshift
        MOV.B R11, R12
        AND.B #080h, R12   ; маскировать все биты,
                           ; кроме СЗР
        CMP.B #00h, R12    ; сравнить с 0
        JNZ    on
        CALL #sendzero     ; отправить «1»
        JMP    zer
on      CALL #sendone      ; отправить «0»
zer
        RLA.B R11          ; сдвиг влево, отправля-
                           ; емый бит в СЗР
        DEC    R13         ; декремент счётчика
                           ; на 8
        JNZ    dshift
        RET
;*****
; Приём байта
;*****
rbyte
        MOV.B #08h, R12    ; счётчик на 8 бит
                           ; данных
        BIC.B #SDA, DIR    ; выставить лог. «1»
                           ; на линии SDA
rrep
        BIS.B #SCL, DIR    ; выставить лог. «0»
                           ; на линии SCL
        BIC.B #SCL, DIR    ; выставить лог. «1»
                           ; на линии SCL
        MOV    #0035h, R13 ; удлинённый счётчик для
                           ; батареи PS100Z-200
                           ; удлинение тактового

```

```

clkxtn      MOV.B  IN, R10      ; сигнала
            DEC    R13          ; проверка шины
            JZ     busy         ; выход по таймауту
            AND.B  #SCL, R10
            JZ     clkxtn       ; ожидание, если SCL
                                ; не «1»
            MOV.B  IN, R10      ; проверка шины
            RLA.B  R11          ; move over for input
                                ; as LSB
            AND.B  #SDA, R10    ; проверка SDA
            JZ     inzero
inzero      BIS.B  #01h, R11    ; установить «1» в M3P
            DEC    R12
            JNZ    rrep         ; продолжение для остав-
                                ; шейся части байта
            MOV.B  R11, R14     ; копирование данных
            BIS.B  #SCL, DIR    ; выставить лог. «0»
                                ; на линии SCL
            RET

; *****
; Отправка неподтверждения (NACK)
; *****
nack
            BIC.B  #SDA, DIR    ; выставить лог. «1»
                                ; на линии SDA
            BIS.B  #SCL, DIR    ; выставить лог. «0»
                                ; на линии SCL
            BIC.B  #SCL, DIR    ; выставить лог. «1»
                                ; на линии SCL
            ; проверка удлинения тактового сигнала
            ; (clock low extending), чтобы не опере-
            ; жать ведомое устройство
            ; ждём установки «1» на линии SCL
clk         MOV.B  IN, R10
            AND.B  #SCL, R10    ; проверка линии SCL
            JZ     clk         ;
            BIS.B  #SCL, DIR    ; «отпустить» SCL
            RET

; *****
; Отправка подтверждения (ACK)
; *****
sack        ; ждём, пока ведомый «отпустит» линию SDA

```

```

MOV    #000Fh, R11 ; счётчик на ?50 мкс
swaita MOV.B IN, R10      ; проверка шины
DEC    R11
JZ     busy
AND.B  #SDA, R10      ; проверка SCL
JZ     swaita
BIS.B  #SDA, DIR      ; перевести SDA в «0»
BIC.B  #SCL, DIR      ; перевести SCL в «1»
MOV    #000Fh, R11 ; счётчик на ?50 мкс
swait  MOV.B IN, R10      ; проверка шины
DEC    R11
JZ     busy
AND.B  #SCL, R10      ; проверка SCL
JZ     swait
BIS.B  #SCL, DIR      ; перевести SCL в «0»
BIC.B  #SDA, DIR      ; «отпустить» SDA
RET

;--- очистка ЖКИ-----
show_clr MOV    #15, r5      ; очистка дисплейной
                                ; памяти

show_clr1 MOV.b  #0, LCD1-1(r5)
DEC      r5
JNZ      show_clr1
RET

;*****
; Определения для ЖКИ
;*****
LCD_TYPE

;---ЖКИ модуля STK/EVK-----
a      .equ  01h
b      .equ  02h
c      .equ  10h
d      .equ  04h
e      .equ  80h
f      .equ  20h
g      .equ  08h
h      .equ  40h

;--- Определения символов-----
LCD_Tab .byte  a+b+c+d+e+f ; отображает "0"
        .byte  b+c          ; отображает "1"
        .byte  a+b+d+e+g    ; отображает "2"

```

```

.byte a+b+c+d+g ; отображает "3"
.byte b+c+f+g ; отображает "4"
.byte a+c+d+f+g ; отображает "5"
.byte a+c+d+e+f+g ; отображает "6"
.byte a+b+c ; отображает "7"
.byte a+b+c+d+e+f+g ; отображает "8"
.byte a+b+c+d+f+g ; отображает "9"
.byte 0 ; отображает ":" пустой
.byte g ; отображает ";" -
.byte a+d+e+f ; отображает "<" [
.byte d+g ; отображает "="
.byte a+b+c+d ; отображает ">" ]
.byte a+b+e+g ; отображает "?"
.byte a+b+d+e+f+g ; отображает "@"
.byte a+b+c+e+f+g ; отображает "A"
.byte c+d+e+f+g ; отображает "B" b
.byte a+d+e+f ; отображает "C"
.byte b+c+d+e+g ; отображает "D" d
.byte a+d+e+f+g ; отображает "E"
.byte a+e+f+g ; отображает "F"
.byte a+b+c+d+f+g ; отображает "G"
.byte b+c+e+f+g ; отображает "H"
.byte b+c ; отображает "I"
.byte b+c+d+e ; отображает "J"
.byte 0 ; отображает "K"
.byte d+e+f ; отображает "L"
.byte a+b+c+e+f ; отображает "M"
.byte c+e+g ; отображает "N" n
.byte c+d+e+g ; отображает "O" o
.byte a+b+e+f+g ; отображает "P"
.byte 0 ; отображает "Q"
.byte e+g ; отображает "R" r
.byte a+c+d+f+g ; отображает "S"
.byte d+e+f+g ; отображает "T" t
.byte c+d+e ; отображает "U" u
.byte 0 ; отображает "V"
.byte 0 ; отображает "W"
.byte 0 ; отображает "X"
.byte b+c+d+f+g ; отображает "Y"
.byte a+b+d+e+g ; отображает "Z" 2

```

LCD_Tab_End

.even

```

;*****
; Вектора прерываний

```

```

;*****
; .sect "Int_Vect", 0FFFFh-1 ; без программы
;                               ; монитора
; .word RESET                ; POR, внеш.
;                               ; Reset,
;                               ; Watchdog
; .end
; .sect "Int_Vect", 05FFh-1 ; с программой
;                               ; монитора
; .word RESET                ; POR, внеш.
;                               ; Reset,
;                               ; Watchdog
; .end

```

Приложение D. Программа ведомого устройства шины SMBus

```

;*****
; Программа ведомого устройства шины SMBus
;*****
USER_END      .set    0FFFFh
Device        .set    325
;--- Определения управляющих регистров-----
SRE           .equ    0h
IE1           .equ    0h
IE2           .equ    01h
IFG1          .equ    02h
IFG2          .equ    03h
WDTCTL        .equ    0120h
WDTHold       .equ    80h
WDT_wrkey     .equ    05A00h
CPUOFF        .set    10h
OSCOFF        .set    20h
;Порт PORT 0
P0IE          .equ    015h
P0DIR         .equ    012h
P0IN          .equ    010h
P0OUT         .equ    011h
P0IES         .equ    014h
P0IFG         .equ    013h
;Порт port 2
P2IN          .equ    028h
P2OUT         .equ    029h
P2IE          .equ    02Dh
P2SEL         .equ    02Eh

```

```

P2DIR      .equ    02Ah
LCD1       .equ    031h
LCDM       .equ    030h
OUT        .equ    P0OUT
IN         .equ    P0IN
DIR        .equ    P0DIR
SDA        .equ    010h
SCL        .equ    020h
DNC        .equ    030h

; Расположение таблицы
Table      .equ    0300h

;*****
; Reset : инициализация процессора
;*****
        .sect    "onreset", 0FFFEh
        .word    INIT
        .sect    "intv", 0FFE0h
        .word    rbp
        .sect    "MAIN", 0C000h

INIT

        .if      Device = 325
MOV      #03C0h, SP
        .endif
        .if      Device = 337
MOV      #05D0h, SP
        .endif
MOV      # (WDTHold+WDT_wrkey), &WDTCTL
; Остановить сторожевой таймер Watchdog
; Timer

; Инициализация порта port2
BIC.B    #DNC, DIR    ; Биты 1 и 0 - входы
BIC.B    #DNC, OUT    ; установить лог. «1»
                        ; на линиях SDA и SCL
BIS.B    #SDA, P0IE   ; разрешить прерывания
                        ; по P0.4
BIS.B    #SDA, P0IES  ; прерывание по спаду
                        ; на P0.4

RESET
nready   ;шина не находится в состоянии готовности
        ;/ нет стартовой посылки
waddr    ; возврат сюда, если неверный адрес
        ; устройства
MOV.B    #000h, error ; сброс флага

```

```

; ошибки
MOV.B #055h, address ; адрес = 55h
MOV.B #09h, data ; данные = 09h
EINT ; общее разре-
; ние прерываний

; *****
; Начало пользовательской программы
; *****
repeat
    JMP repeat ; ожидание прерывания

; *****
; Завершение пользовательской программы
; *****
; *****
; приём байта (здесь обрабатывается прерывание)
; *****
rbp
    ; PUSH R11 ; если известно, что
    ; ; стартовая посылка
    ; ; длительная, можно
    ; ; добавить эти строки

    ; PUSH R12
    ; PUSH R13
    ; PUSH R14
    BIC.B #SDA, P0IFG ; очистить флаг
    ; прерывания

; *****
; проверка стартовой последовательности
; *****
wait2    MOV.B IN, R11 ; копировать состояние
    ; входов в R11
    DEC R9
    JZ done
    AND #DNC, R11 ; маскировать все биты,
    ; кроме входов
    CMP #SCL, R11 ; убедиться, что на
    ; линии SDA лог. «0»
    ; а на SCL лог. «1»
    JNZ wait2 ; если нет - ожидаем
wait3    MOV.B IN, R11 ; копировать состояние
    ; входов в R11
    DEC R9
    JZ done

```



```

        AND    #DNC, R11    ; маскировать все биты,
                             ; кроме входов
        JNZ    wait3        ; если нет - ожидаем
; *****
; Стартовая последовательность получена
; *****
        MOV    #007h, R13   ; счётчик на 7 адресных
                             ; бит
        MOV    #0000h, R12  ; регистр для сохране-
                             ; ния адреса
taddr    MOV.B  IN, R14      ; убедиться, что на
                             ; линии SCL лог. «0»
        DEC    R9
        JZ     done
        AND    #SCL, R14
        JNZ    taddr
addr     MOV.B  IN, R14      ; чтение порта
        DEC    R9
        JZ     done
        MOV.B  R14, R11     ; копировать данные
        AND    #SCL, R14    ; маскировать все биты,
                             ; кроме SCL
        JZ     addr         ; ждём лог. «1» на SCL
        AND    #SDA, R11    ; маскировать все биты,
                             ; кроме SDA
        JZ     szero        ; если «0» - просто
                             ; сдвиг
        RLA.B  R12           ; сдвиг влево
        BIS.B  #0001h, R12  ; если «1» установить
                             ; «1» в МЗР и сдвиг
        JMP    sone
szero    RLA.B  R12           ; арифметический сдвиг
                             ; влево ? 0
sone     DEC    R13          ; декремент счётчика
        JNZ    taddr        ; повтор, если менее
                             ; 7 бит
        MOV.B  address, R13 ; копировать адрес
                             ; устройства в R13
        CMP.B  R13, R12     ; и сравнить его с при-
                             ; нятым адресом
        JNZ    waddr        ; переход, если адрес
                             ; неверный
; *****

```

```

; адрес правильный, продолжаем
; *****
; чтение бита R/W (чтение/запись)
rwait2      MOV.B  IN, R11      ; ждём лог. «0» на SCL
            DEC    R9
            JZ     done
            AND.B  #SCL, R11    ; маскировать все биты,
                                ; кроме SCL
            JNZ    rwait2
rwait       MOV.B  IN, R11      ; ждём лог. «1» на SCL
            DEC    R9
            JZ     done
            AND.B  #SCL, R11    ; маскировать все биты,
                                ; кроме SCL
            JZ     rwait
            MOV.B  IN, R11      ; копировать данные
                                ; из порта
            AND.B  #SDA, R11    ; маскировать все биты,
                                ; кроме SDA
            JNZ    done         ; 0 если запись, пере-
                                ; ход, если 1 (чтение)
; *****
; Отправить бит подтверждения (ACK)
; *****
rwait3      MOV.B  IN, R11      ; ждём лог. «0» на SCL
            DEC    R9
            JZ     done
            AND.B  #SCL, R11    ; маскировать все биты,
                                ; кроме SCL
            JNZ    rwait3
            BIS.B  #SDA, DIR    ; перевести линию SDA
                                ; в лог. «0»
rwait4      MOV.B  IN, R11      ; ждём лог. «1» на SCL
            DEC    R9
            JZ     done
            AND.B  #SCL, R11    ; маскировать все биты,
                                ; кроме SCL
            JZ     rwait4
rwait5      DEC    R9
            JZ     done
            MOV.B  IN, R11      ; ждём лог. «0» на SCL
            AND.B  #SCL, R11    ; маскировать все биты,
                                ; кроме SCL
            JNZ    rwait5

```

```

        BIC.B #SDA, DIR ; «отпустить» линию SDA
;*****
; готов принять байт
;*****
        MOV     #008h, R13 ; счётчик на 8 бит
                           ; данных
        MOV     #0000h, R12 ; регистр для сохране-
                           ; ния адреса
tdat     MOV.B   IN, R14    ; убедиться, что на
                           ; линии SCL лог. «0»

        DEC     R9
        JZ      done
        AND     #SCL, R14
        JNZ     tdat
dat      MOV.B   IN, R14    ; читаем данные из порта
        MOV.B   R14, R11    ; копируем данные
        DEC     R9
        JZ      done
        AND     #SCL, R14 ; маскировать все биты,
                           ; кроме SCL
        JZ      dat        ; ждём лог. «1» на SCL
        AND     #SDA, R11 ; маскировать все биты,
                           ; кроме SDA
        JZ      szero      ; если «0» – просто
                           ; сдвиг

        RLA.B   R12
        BIS.B   #0001h, R12 ; если «1» установить
                           ; «1» в МЗР и сдвиг

        JMP     szero
szero    RLA.B   R12        ; арифметический сдвиг
                           ; влево
szero    JZ      done
        DEC     R13        ; декремент счётчика
        JNZ     tdat
        MOV.B   R12, data ; копируем в ОЗУ
;*****
; Отправить бит подтверждения (ACK)
;*****
rwait6   MOV.B   IN, R11    ; ждём лог. «0» на SCL
        DEC     R9
        JZ      done
        AND.B   #SCL, R11 ; маскировать все биты,
                           ; кроме SCL
        JNZ     rwait6

```

```

                                BIS.B  #SDA, DIR    ; перевести линию SDA
                                ; в лог. «0»
rwait7      MOV.B  IN, R11    ; ждём лог. «1» на SCL
                                DEC    R9
                                JZ     done
                                AND.B  #SCL, R11    ; маскировать все биты,
                                ; кроме SCL
                                JZ     rwait7
rwait8      MOV.B  IN, R11    ; ждём лог. «0» на SCL
                                DEC    R9
                                JZ     done
                                AND.B  #SCL, R11    ; маскировать все биты,
                                ; кроме SCL
                                JNZ     rwait8
done        BIC.B  #SDA, DIR  ; «отпустить» линию SDA

                                ; POP    R14          ; если известно, что
                                ;          ; стартовая посылка
                                ;          ; длительная, можно
                                ;          ; добавить эти строки

                                ; POP    R13
                                ; POP    R12
                                ; POP    R11
                                MOV     #0FFh, R9
RETI
; *****
; переменные
; *****
address     .equ    0248h      ; адрес устройства
data        .equ    0242h      ; отправляемые данные
datin       .equ    0244h      ; принимаемые данные
error       .equ    0246h      ; флаг ошибки

```

Приложение E. Содержимое файла ASCII.txt

@B500

```

00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B7 12 8F 1F 3A 3D BD 13 BF 3F 00 00 00 00 00 00
00 BB BC A5 9E AD A9 3F BA 12 96 00 A4 B3 98 9C
AB 00 88 3D AC 94 00 00 00 3E 8F 00 00 00 00 00
00 BB BC A5 9E AD A9 3F BA 12 96 00 A4 B3 98 9C
AB 00 88 3D AC 94 00 00 00 3E 8F 00
q

```

III. Интерфейс с внешними устройствами

Глава 7.
*IrDA SIR кодер-декодер
на MSP430C112*

IrDA SIR кодер-декодер на MSP430C112

Juergen Mayer [slaa044.pdf (Англ.)]

В данном примере применения описывается использование микроконтроллера MSP430x112 в качестве IrDA SIR кодер-декодера. Аппаратная часть обсуждается на уровне блок-схемы и принципиальной схемы IrDA модуля. Также обсуждаются вопросы программирования и проблемы пользовательского интерфейса. В приложении А приведена прикладная программа IrDA кодера-декодера.

1. Введение

Системы передачи данных по инфракрасному каналу связи появились в связи с необходимостью ведения обмена данными между переносными компьютерами и периферийным оборудованием. IrDA стандарт является дешевым и практическим протоколом передачи данных по инфракрасному каналу связи.

MSP430 – мощный микроконтроллер, способный выполнять различные прикладные программы, в том числе и реализовать последовательный протокол передачи данных IrDA. Микроконтроллер имеет 16- разрядную архитектуру и низкое потребление, что позволяет ему выполнять обмен данными на средних и высоких скоростях в системах с автономным питанием. В данном примере применения описывается использование микроконтроллера MSP430x112 в качестве IrDA кодера-декодера, реализованного на отладочной IrDA плате.

Модуль IrDA приемопередатчика может использоваться для обмена данными между компьютерами по RS232 порту. Для этого каждый компьютер должен иметь стандартный RS232 порт и некоторое обычное терминальное программное обеспечение, типа программы Hyperterminal, которая имеется в операционных системах Windows 3.11, Windows95 и WindowsNT.

Приведенная прикладная программа может использоваться и на других микроконтроллерах семейства MSP430, содержащих драйверы ЖКИ, аналоговые преобразователи и аппаратные умножители.

2. Технические требования к IrDA интерфейсу на физическом уровне

Технические требования к IrDA интерфейсу на физическом уровне предназначены для определения параметров полудуплексного инфракрасного канала связи, позволяющего вести обмен данными на расстояния до 1 М. Полный

стандарт требует ведения обмена данными на скоростях до 4 Мбит/с. Однако, в данном примере применения мы рассмотрим передачу данных на скоростях от 2.4 кбит/сек до 115.2 кбит/сек.

Последовательный инфракрасный интерфейс IrDA должен иметь минимальную пропускную способность 9.6 кбит/сек. Параметры скорости потока и длительности импульсов приведены в таблице 1.

Таблица 1. Технические требования к скорости потока и длительности импульса IrDA интерфейса

DATA RATE	BIT TIME	IrDA PULSE DURATION MINIMUM	IrDA PULSE DURATION NOMINAL	IrDA PULSE DURATION MAXIMUM
2.4 kbit/s	416 μ s	1.41 μ s	78.13 μ s	88.55 μ s
9.6 kbit/s	104 μ s	1.41 μ s	19.53 μ s	22.13 μ s
19.2 kbit/s	52.0 μ s	1.41 μ s	9.77 μ s	11.07 μ s
38.4 kbit/s	26.0 μ s	1.41 μ s	4.88 μ s	5.96 μ s
57.6 kbit/s	17.3 μ s	1.41 μ s	3.26 μ s	4.34 μ s
115.2 kbit/s	8.68 μ s	1.41 μ s	1.63 μ s	2.23 μ s

Минимальная длительность импульса должна быть та же самая, что и при передаче данных на скорости 115.2 кбит/сек. Причина состоит в том, что на физическом уровне IrDA интерфейса может быть реализовано два вида импульсной модуляции: с длительностью импульса бита 3/16 или с минимальной длительностью импульса 1.63 мс минус допуск 0.22 мс.

При передаче данных на скоростях до 115.2 кбит/сек нет никакой необходимости передавать последовательный инфракрасный интерактивный импульс (serial infrared interaction pulse – SIP), который гарантировал бы совместимость с более медленной ИК системой. Это приводит к упрощению структуры IrDA кадра при работе на скоростях до 115.2 кбит/сек.

Для скоростей передачи данных до 115.2 кбит/сек электрический сигнал к модулю кодер-декодера передается в виде последовательного битового потока. Логический 0 в битовом потоке представлен ИК импульсом с длительностью, указанной в таблице 1, а логическая 1 периодом бита без ИК импульса.

Битовый поток имеет кадровую организацию. Кадр, как показано в рисунке 1, начинается стартовым битом, затем следуют 8 информационных битов, а заканчивается кадр стоповым битом.

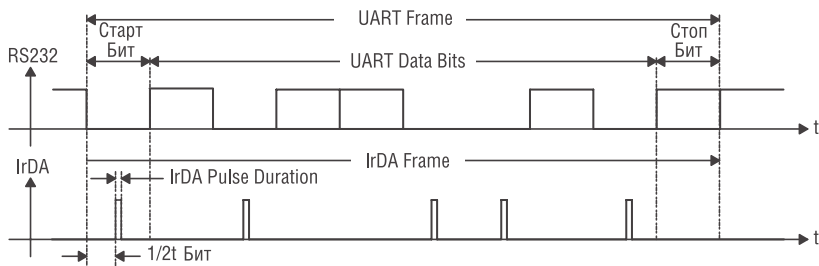


Рис. 1. IrDA и UART кадры

По IrDA стандарту ИК импульс начинается в центре битового периода и имеет длительность $3/16$ длительности битового периода или фиксированную длительность, эквивалентную $3/16$ длительности битового периода при скорости передачи 115.2 кбит/сек.

Использование фиксированной длительности рекомендуется для снижения потребления в системах с автономным питанием.

3. MSP430x112 – микроконтроллер со сверхнизким потреблением и минимальными функциональными возможностями

MSP430x112 – 16-разрядный микроконтроллер семейства MSP430, имеющий сверхнизкое потребление и минимальные функциональные возможности, разработанный для применения в устройствах с автономным питанием. Каждый член этого семейства имеет набор различных периферийных модулей. В отличие от большинства других членов семейства MSP430 этот микроконтроллер не имеет ни драйвера ЖКИ, ни АЦП.

MSP430x112 содержит мощный 16-разрядный таймер с тремя регистрами захвата/сравнения, четырнадцать линий портов ввода-вывода и новый модуль автогенератора.

Автогенератор способен работать без внешних компонентов с одним только низкочастотным (32 кГц) или высокочастотным (до 6 МГц) кварцевым резонатором или тактироваться от внешнего источника синхроимпульсов с частотой до 6 МГц.

4. IrDA кодер-декодер на микроконтроллере MSP430x112

4.1. Краткий обзор модуля

На рисунке 2 приведена структурная схем IrDA модуля приемопередатчика. Основой модуля является микроконтроллер MSP430x112, который выпол-

няет задачи кодирования и декодирования данных, а также обмена данными по RS232 интерфейсу. Микросхема TSLM1100 обеспечивает требуемый логический интерфейс между MSP430 и ИК каналом связи. Этот ИК приемопередатчик способен на физическом уровне обеспечивать двунаправленный полудуплексный IrDA интерфейс версии 1.1. С целью снижения потребления в модуле установлена длительность импульса 1.63 мкс.

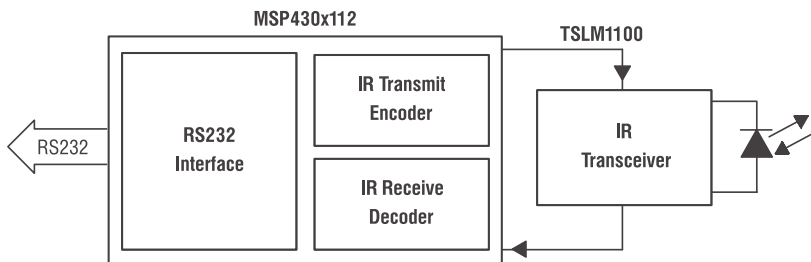


Рис. 2. Структурная схема IrDA модуля

Модуль может быть непосредственно подключен к обычному RS232 порту через разъем типа DB9S. Вследствие того, что IrDA интерфейс является полудуплексным, то после сброса модуль должен быть инициализирован для работы в качестве IrDA приемника или передатчика, что и описано в разделе 4.3.2.

4.2. Управляющая программа MSP430x112

В следующем разделе описаны некоторые проблемы программирования и детально обсуждены вопросы использования модуля таймера TIMER_A для кодирования и декодирования последовательного потока данных.

Законченный листинг прикладной программы для управляющего микроконтроллера MSP430x112 приведен в приложении А.

4.2.1. Общие вопросы

После сброса системы MSP430 должен быть инициализирован. При инициализации DCO модуляция отключается, а высокочастотный генератор инициализируется. Это позволяет подключать к выводам Xout и Xin кварцевый резонатор с резонансной частотой до 6 МГц. Высокочастотный генератор формирует синхроимпульсы, используемые для передачи последовательного потока данных. Для снижения стоимости можно использовать 32 кГц кварцевый резонатор совместно с программной системой ФАПЧ. Эта программная ФАПЧ управляет модуляционным DCO регистром и подстройкой частоты, сравнивая деленные синхроимпульсы с опорными 32 кГц колебаниями.

Далее производится инициализация портов ввода-вывода и опрос состояния переключателей. Состояние переключателей сохраняется в переменной BAUDRATE и используется для определения скорости передачи данных в обоих направлениях.

По RS232 интерфейсу передается строка инициализации. Так как состояние сигнала CTS на линии RS232 не проверяется, то между посылками необходимо формировать задержку. Это предотвращает переполнение входного буфера RS232 порта у некоторых компьютеров на высоких скоростях передачи данных.

В это время разрешено только прерывание по линии порта P1.0 и обеспечивается готовность принятия строки инициализации ^r или ^t. В зависимости от ответа пользователя MSP430 инициализирует все порты и прерывания для работы в режиме IrDA приемника или IrDA передатчика. Кроме того, микроконтроллер посылает подтверждение через линию порта P1.1. Это подтверждение отображается в терминальном окне.

4.2.2. IrDA-SIR кодер (передатчик)

Если пользователь вводит команду ^t, то MSP430 работает в качестве кодера между RS232 входным портом и ИК выходным портом. Таймер Timer_A и порты инициализированы следующим образом (см. также описание схемы в главе 5):

- RS232 вход (сигнале TXD) $\Rightarrow$ порт P2.5
- IrDA выход (TSLM1100) $\Rightarrow$ порт P2.4 (CCR2)

Длительность полупериода бита загружена в регистр CCR0, а таймер Timer_A установлен в режим увеличения/уменьшения (Up/Down). Второй модуль захвата/сравнения подключается к линии P2.4 и осуществляет захват регистра сравнения CCR2, в который загружено значение регистра CCR0 за минусом значения, соответствующего длительности ИК импульса 1.63 мкс.

При передаче второй управляющий регистр захвата/сравнения (CCTL2) установлен в режиме формирования сброса. Это означает, что каждый раз, когда в регистре таймера Timer_A (TAR) устанавливается значение, равное значению в регистре CCR2, то на линии P2.4 устанавливается низкий логический уровень, а когда устанавливается значение, равное значению в регистре CCR0, то на этой линии устанавливается высокий логический уровень.

Когда таймер Timer_A запускается, то он увеличивает свое значение до значения, равного CCR0, а потом ведет обратный отсчет, после чего опять запускается и снова увеличивает свое значение до CCR0. Пока таймер Timer_A работает, в каждый битовый период без вмешательства вычислительного ядра формируется импульс длительностью 1.63 мкс.

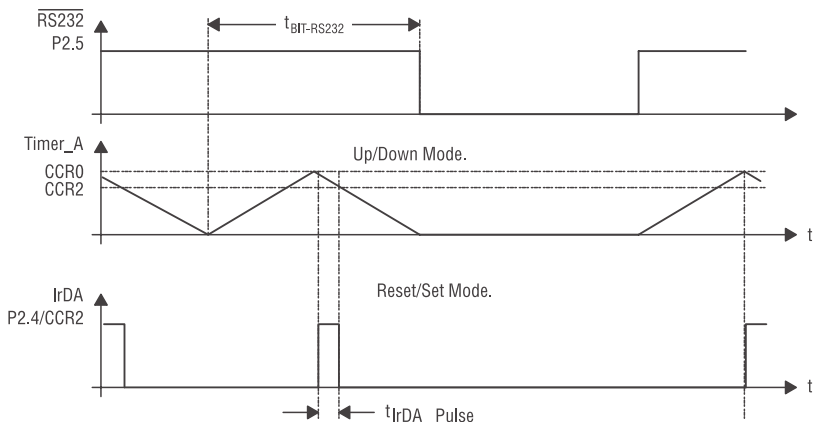


Рис. 3. Использование таймера Timer_A при RS232/IrDA преобразовании

Линейный драйвер RS232 интерфейса (HIN232) производит инвертирование потока данных, формирование логических уровней сигнала и подачу его на линию P2.5. Высокий уровень на линии P2.5 представляет собой логический 0 и наоборот. Согласно IrDA стандарту логический 0 на линии RS232 представляется ИК импульсом. Это означает, что нарастающий фронт на линии P2.5 должен запустить таймер Timer_A, а спадающий – остановить его. Это обеспечивается установкой бита разрешения прерывания для линии P2.5.

Каждое изменение состояния на линии RS232 вызывает прерывание, и MSP430 начинает выполнять подпрограмму обработки прерывания (interrupt service routine – ISR): Текст программы вверху 11 стр. описания

```
; *****
; Подпрограмма обработки прерывания передатчика
; TRANSMITTER RS232 (P2.5) → IrDA (P2.4)
; *****
TX_01    BIS      #04h, &TACTL          ;                    5
циклов

        XOR R14, &TACTL      ; запуск/остановка
                                ; таймера Timer_A
                                ; (режим передачи/
                                ; приема)                    4 цикла

        XOR.B R13, &P2IES    ; изменение фронта
                                ; выбора IR                  4 цикла

        BIC.B #0FFh, &P2IFG  ; очистка флагов
                                ; прерывания                  5 циклов

        RETI                  ;                    5 циклов
```

Через шесть циклов после установки прерывания (время, необходимого на сохранения состояния микроконтроллера), MSP430 запускает ISR. Для изменения состояния управляющего регистра TACTL таймер Timer_A должен быть остановлен.

Далее мы запускаем или останавливаем таймер путем установки необходимого значения управляющего бита в регистре TACTL. После этого в регистре P2IES изменяем бит, определяющий фронт формирования прерывания.

Полностью вся подпрограмма обработки прерывания выполняется за $23 + 6$ циклов. Поэтому, при работе на частоте 3.6864 МГц (период 271.2 нс) ISR выполняется за $29 \times 271.2 \text{ нс} = 7.865 \text{ мкс}$. При работе со скоростью передачи данных 115.2 кбит/сек время между формируемыми прерываниями минимальное и равняется 8.68 мкс. Вышеупомянутая подпрограмма обработки прерывания выполняется быстрее, позволяя обеспечивать передачу данных без потерь.

4.2.3. IrDA-SIR декодер (приемник)

Если пользователь вводит команду ^r, то MSP430 работает в качестве декодера между ИК входом и выходом RS232. Таймер Timer_A приостанавливается, а порты инициализируются следующим образом (см. также описание схемы в главе 5):

- IrDA вход (TSLM1100) $\Rightarrow$ порт P1.2
- RS232 выход (RXD) $\Rightarrow$ порт P1.1 (CCR0)

Длительность битового периода загружается в регистр периода CCR0, а Timer_A устанавливается в режим увеличения состояния до значения, равного содержимому регистра CCR0, после чего он сбрасывается и снова начинает считать до значения CCR0. Нулевой модуль захвата/сравнения подключается к порту P1.1.

При приеме регистр управления захватом/сравнением (CCTL0) находится в режиме установки. Это означает, что каждый раз, когда значение регистра таймера Timer_A (TAR) становится равным значению регистра CCR0, на порту P1.1 устанавливается высокий логический уровень.

Если мы запускаем таймер Timer_A, то он увеличивает свое значение, а когда его состояние сравнивается со значением, записанным в регистре CCR0, то на линии P1.1 устанавливается высокий уровень, после чего он перезапускается и снова считает до CCR0 снова. В течение всего времени работы таймер Timer_A постоянно устанавливает на порту P1.1 высокий уровень.

ИК приемопередатчик формирует на порту P1.2 инвертированный сигнал. Это означает, что спадающий фронт устанавливает на порту P1.1 низкий логический уровень и перезапускает таймер. После этого таймер Timer_A синхронизируется с ИК сигналом. Когда таймер достигает значения CCR0 (длительность

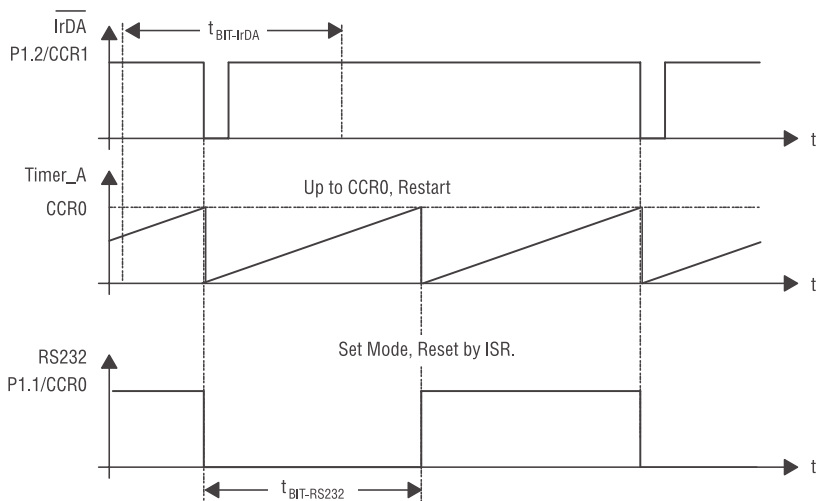


Рис. 4. Использование таймера Timer_A при IrDA/RS232 преобразовании

RS232 бита), он автоматически устанавливает на порту P1.1 высокий уровень и постоянно передает высокий уровень до тех пор, пока не будет сформирован новый спадающий фронт.

```

; *****
; Подпрограмма обработки прерывания приемника RECEIVER
; IrDA (P1.2) → RS232 (P1.1)
; *****
RX_01  CLR    &CCTL0                ; CC0 в режиме выхода/
                                           ; на выводе P1.1
                                           ; низкий уровень      5 циклов
      BIS    #04h, &TACTL            ; сброс таймера
                                           ; Timer_A          5 циклов
      MOV    #20h, &CCTL0            ; CCTL0 устанавливает
                                           ; на порту P1.1 низкий
                                           ; уровень и "режим → P1.1
                                           ; высокий"          5 циклов
      BIC    #01h, &CCTL1            ; сброс флага
                                           ; прерывания
                                           ; CCTL1!          5 циклов
      RETI                               ;
                                           ;                  5 циклов

```

Каждый спадающий фронт на порту P1.2 вызывает прерывание, и MSP430 начинает выполнять подпрограмму обработки прерывания.

Выполнение подпрограммы обработки прерывания начинается через шесть циклов после появления прерывания. Сначала на линии P1.1 устанавливается низкий логический уровень, а модуль CCR0 устанавливается в режим только вывода. После этого таймер Timer_A сбрасывается путем установки бита CLR в регистре TACTL, а флаг прерывания CCR1 сбрасывается.

ISR выполняется за $25 + 6$ циклов. Поэтому при работе на частоте 3.6864 МГц (период 271.2 нс) подпрограмма выполняется за $31 * 271.2 \text{ нс} = 8.407 \text{ мкс}$. При обмене данными со скоростью 115.2 кбит/сек минимальное время между прерываниями – 8.68 мкс. Вышеупомянутая подпрограмма обработки прерывания выполняется быстрее, позволяя обеспечивать прием данных без потерь.

Так как каждый спадающий фронт на порту P1.2 формирует прерывание и, следовательно, низкий логический сигнал на линии RS232, то свечение флуоресцентных ламп и помехи от других приборов могут формировать ошибочные биты. Время между спадающим и нарастающим фронтами может быть измерено при помощи второго модуля захвата/сравнения и сопоставлено с длительностью IrDA импульса, что позволит устранить ошибки. Для достижения этого модуль CCR2 должен быть установлен в режим захвата по нарастающему фронту. Так как CCR1 срабатывает по спадающему фронту, то значения этих регистров могут быть сравнены в подпрограмме обработки прерывания. Если длительность импульса меньше, чем 1.6 мкс – 0.22 мкс, то импульс признается ложным. Однако для этого требуется повысить рабочую частоту микроконтроллера или снизить скорость передачи данных.

4.3. Установки

4.3.1. Выбор скорости передачи данных

IrDA модуль может работать на шести различных скоростях передачи данных в пределах от 2.4 кбит/сек до 115.2 кбит/сек. Скорость передачи данных должна быть выбрана перед началом сеанса связи.

Таблица 2. Установка скорости передачи данных

DATA RATE	JUMPER		
	J3	J2	J1
2.4 kbit/s	L	L	L
2.4 kbit/s	L	L	H
19.2 kbit/s	L	H	L

DATA RATE	JUMPER		
	J3	J2	J1
38.4 kbit/s	L	H	H
57.6 kbit/s	H	L	L
115.2 kbit/s	H	L	H

Скорость передачи данных устанавливается тремя перемычками J1, J2 и J3 – смотри таблицу 3. Если положения перемычек изменяются при работе системы, то необходимо выполнить ее перезапуск. После этого модуль может быть подключен к RS232 порту компьютера, и питание может быть подано. Удостоверьтесь, что в терминальной программе установлена та же скорость передачи данных, что и в модуле.

4.3.2. Интерфейс пользователя

Обычная терминальная программа типа гипертерминала (Hyperterminal), которая имеется в операционных системах Windows 3.11, Windows95 и WindowsNT, может использоваться для ведения обмена данными с IrDA модулем.

Параметры настройки COM порта должны быть следующими:

Скорость (бит в секунду):	см. положение перемычек J1, J2 и J3 на IrDA модуле
Количество информационных разрядов:	7
Проверка:	на четность
Количество стоповых бит:	1
Управление потоком:	нет

Удостоверьтесь, что вы используете одни и те же параметры терминальных программ на обоих компьютерах.

Нажатие кнопки сброса (RESET) на IrDA модуле приведет к отображению в окне терминальной программы строки инициализации, как это показано на рисунке 5.

После этого следует определить, какой модуль будет работать в режиме передатчика, а какой в режиме приемника.

После набора команды ^r, модуль переходит в режим приема и выдает подтверждение (см. рисунок 6).



Рис. 5. Сообщение об инициализации IrDA модуля



Рис. 6. Сообщение об инициализации IrDA модуля

Для передачи данных от одного персонального компьютера в другой установите один IrDA модуль в режим передатчика, а другой – в режим приемника.

Для изменения направления передачи нажмите кнопку сброса RESET и повторите процедуру инициализации.

5. Описание схемы

На рисунке 7 показана принципиальная схема IrDA модуля.

При использовании кварцевого резонатора с резонансной частотой выше 3 МГц, MSP430x112 требуется 5 В питание. Так как RS232 драйвер (HIN232)

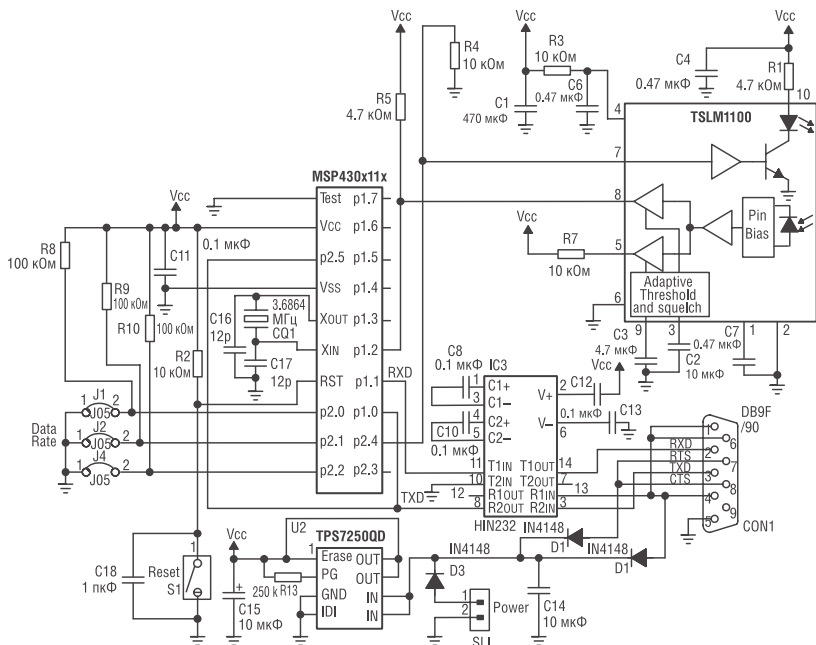


Рис. 7. Принципиальная схема IrDA модуля

имеет довольно высокое потребление, то не все персональные компьютеры способны обеспечить питание модуля по RS232 порту. Поэтому в некоторых случаях придется использовать внешний источник питания или аккумулятор. Для снижения потребляемой мощности следует выбирать наиболее низкую рабочую частоту.

Для снижения влияния шумов следует использовать дополнительный фильтр питания ИК модуля TSLM1100. Для повышения помехоустойчивости конденсаторы C6 и C7 должны быть размещены у самых выводов TSLM1100. Для получения более детальной информации см. техническое описание TSLM1100.

Так как TSLM1100 изготавливается в прозрачном пластиковом корпусе, рекомендуют защитить прибор от воздействия излучения, особенно от излучения флуоресцентных ламп и ИК пультов дистанционного управления.

6. Заключение

IrDA стандарт становится все более популярным в последнее время, так как все больше и больше приложений требуют обеспечения беспроводного канала связи. В данном примере применения показано, что низкое потребление и беспроводной канал связи могут идти рука об руку.

При использовании MSP430C112 становится возможным изготавливать устройства с автономным питанием, обеспечивающие передачу данных по беспроводным каналам связи со скоростью до 115.2 кбит/сек.

Программа управляющего микроконтроллера IrDA модуля, приведенная в приложении А, может быть легко адаптирована для выполнения специфических задач.

Ссылки:

1. Texas Instruments, Data Transmission Seminar 1998, SLLDE01C Texas Instruments 1998.
2. Infrared Data Association, Physical Layer Link Specification, Version 1.1, 17.10.1995.
3. Texas Instruments, Datasheet MSP430x110, Texas Instruments 1998.
4. Texas Instruments, MSP430 Family Architecture Guide and module Library, Texas Instruments 1996.
5. Texas Instruments, MSP430 Family Software User's Guide, Texas Instruments 1994.
6. Texas Instruments, Datasheet TSLM1100, Texas Instruments 1997.

Приложение А: исходный текст программы

```

;*****
; Программа IrDA SIR кодера - декодера на базе
; MSP430x112 (C) компании TEXAS INSTRUMENTS 1998
; Имя файла:   IrDA11X.asm
; Проект:      MSP430x112 IrDA Demo
; Автор:       Jurgen Mayer (Германское отделение
;              компании Texas Instruments)
; Описание:    IrDA SIR кодер - декодер physical,
;              версия 1.1
;              RX = RS232 → IrDA
;              TX = IrDA → RS232
;              RX/TX определяется
;              символами ^T( = RS232 → IrDA )
;              и ^R( = IrDA → RS232)

```

```

; Установки программы терминала:
;           1 старт бит/7 бит данных/1 стоп бит/
;           проверка на четность
; Последнее обновление:
;           версия 2.1 / 28.05.98
;           версия 2.2 / 23.07.98
;*****
SP_orig      .set 0300h          ; указатель стека
WDTCTL       .equ 0120h
WDTHold      .equ 080h
WDT_wrkey    .equ 05A00h
IE1          .set 000h
IFG1         .set 002h
P1IN         .set 020h
P1OUT        .set 021h
P1DIR        .set 022h
P1IFG        .set 023h
P1IES        .set 024h
P1IE         .set 025h
P1SEL        .set 026h
P2IN         .set 028h
P2OUT        .set 029h
P2DIR        .set 02Ah
P2IFG        .set 02Bh
P2IES        .set 02Ch
P2IE         .set 02Dh
P2SEL        .set 02Eh
TACTL        .set 160h          ; Таймер Timer A
CCTL0        .set 162h
CCTL1        .set 164h
CCTL2        .set 166h
TAR          .set 170h
CCR0         .set 172h
CCR1         .set 174h
CCR2         .set 176h
DCOCTL       .set 056h          ; установка частоты DCO
BCSCTL1      .set 057h
BCSCTL2      .set 058h
TRANSMIT     .set 028h          ; TX СИМВОЛ "^T"
RECEIVE      .set 048h          ; RX СИМВОЛ "^R"

```

```

;*****
; Основная программа
;*****
                .sect "ROM", 0F000h
RESET          MOV     #SP_orig, SP           ; инициализация
                                                ; указателя стека
                MOV     #(WDTHold+WDT_wrkey), &WDTCTL ; останов-
                                                ; ка сторожевого
                                                ; таймера

main_1         call    #init_sys
                CLR.B   STATUS_1
                CLR.B   STATUS_2
                CLR.W   BAUDRATE
                call    #init_Px
                call    #init_BAUDRATE
                MOV     BAUDRATE, R5           ; сохранение скорости
                                                ; обмена данными
                                                ; в регистре R5
                RRA     R5                     ; деленное на 2
                SUB     #02h, R5              ; настройка для синхро
                                                ; низации
                MOV     R5, R8                 ; копирование
                                                ; в регистр R8
                MOV     #0, R9                 ; текст старта...
                MOV     #28, R10               ; текст остановки...
                call    #TEXT_OUT              ; запись в ПК...
                call    #init_RXTX
                MOV.B   STATUS_1, STATUS_2
                EINT                          ; общее разрешение
                                                ; прерываний

main_2         MOV.B   &P2IN, R5              ; сканирование состоя-
                                                ; ния перемишки
                                                ; и пересылка его
                                                ; в регистр R5
                BIC     #0FFF8H, R5
                MOV.B   R5, STATUS_1           ; сохранение скорости
                                                ; обмена данными
                CMP.B   STATUS_1, STATUS_2 ; любые изменения
                JEQ     main_2                 ; нет
                DINT
                JMP     main_1                 ; restart

main_3         JMP     main_3

```

```

;*****
Сброс: инициализация процессора
;*****
init_sys
    MOV.B #00h,DCOCTL        ; отключение модуляции
                             ; DCO
    BIS.B #040h,BCSCTL1      ; 1-oe: включение
                             ; HF- автогенератора,
                             ; работающего
                             ; от внешнего кварце-
                             ; вого резонатора
    MOV.B #0C8h,BCSCTL2      ; 2-oe: LFXT1 ⇒ MCLK /
SMCLK
    CLR.B    IE1
    CLR.B    IFG1
    CLR      R5
    CLR      R6
    CLR      R7
    CLR      R8
    CLR      R9
    CLR      R10
    CLR      R11
    CLR      R12
    CLR      R13
    CLR      R14
    CLR      R15
    RET

;*****
; Инициализация генератора скорости обмена данными
; - сканирование перемычки
; - установка скорости ведения обмена данными
;*****
init_BAUDRATE
    PUSH    R5
    CLR     R5
    MOV.B   &P2IN,R5          ; сканирование состоя-
                             ; ния перемычки и
                             ; пересылка его
                             ; в регистр R5
    BIC     #0FFF8H,R5
    MOV.B   R5,STATUS_1       ; запоминание скорости
                             ; обмена данными

```

```

        RLA      R5
        ADD      #BAUD2400,R5
        MOV      @R5,BAUDRATE      ; загрузка циклов
                                      ; в переменную BAUDRATE

        POP      R5
        RET

;*****
; Инициализация - Px
; SW - BAUDRATE - P2.0/1/2 вход
; RX/TX - P1.0 = вход → выбор фронта IR - P1.0
; высокий низкий
; RX - P2.5 общий вход RS232
; P2.4 выход модуля IrDA
; TX - P1.2 общий порт ввода/вывода ↔ вход модуля
; IrDA
; P1.1 общий порт ввода/вывода ↔ вход модуля RS232
;*****
init_Px
        BIC.B    #0DH,&P1DIR      ; P1.0/2/3 вход
        BIS.B    #02H,&P1DIR      ; P1.1 выход
        BIC.B    #07H,&P1SEL      ; P1.0 общий порт
                                      ; ввода/вывода P1.1/2
                                      ; порты модуля при
                                      ; передаче
        BIS.B    #02H,&P1OUT      ; установка на выходе
                                      ; P1.1 высокого уровня
        BIC.B    #0FFH,&P1IFG     ; очистка флагов
                                      ; прерывания
        BIS.B    #001H,&P1IES     ; выбор спадающего
                                      ; фронта IR на выводе
                                      ; P1.0
        BIS.B    #001H,&P1IE      ; разрешение прерыва-
                                      ; ния от P1.0
        BIC.B    #10H,&P2OUT      ; установка на выводе
                                      ; P2.4 низкого уровня
        BIC.B    #027H,&P2DIR     ; P2.0/1/2/5 вход
        BIS.B    #010H,&P2DIR     ; P2.4 выход
        BIC.B    #27H,&P2SEL      ; P2.0/1/2/5 порты
                                      ; ввода-вывода общего
                                      ; назначения
        BIC.B    #0FFH,&P2IFG     ; очистка флагов
                                      ; прерываний

        RET

```

```

;*****
; Инициализация подпрограммы обработки прерывания
; - RX/TX
; - P1.0 - вход RS232
; - количество бит содержится в регистре R6
;*****
init_RXTX
    MOV    BAUDRATE,R5        ; сохранение скорости
                                ; обмена данных
                                ; в регистре R5
    RRA    R5                 ; деление на 2
    SUB    #02h,R5           ; настройка для
                                ; синхронизации
    MOV    R5,R8              ; копирование
                                ; в регистр R8
    MOV    #08h,R6            ; загрузка счетчика
                                ; битов в регистр R6 (n)
                                ; → см. подпрограмму
                                ; обработки прерывания
                                ; RX/TX
    MOV    #25,R9             ; текст начала...
    MOV    #79,R10            ; текст остановки...
    call   #TEXT_OUT          ; запись в ПК...
    RET

;*****
; подпрограмма обработки прерывания RX/TX
; - вход P1.0
; - скорость обмена данными в регистрах R5/R8
; - счетчик битов в регистре R6
; - входные данные (DATA_in) в регистре R7
;*****
RXTX_01 MOV    R8,R8          ; пустой цикл
        MOV    R8,R8          ; пустой цикл
RXTX_02 MOV    R8,R5          ; загрузка скорости
                                ; обмена данными
                                ; в регистр R5
                                ; пустой цикл
RXTX_03 MOV    R8,R8          ; пустой цикл
        DEC    R5
        JNZ    RXTX_03        ; линия выборки?
        RRC    &P1IN          ; передача на бита
                                ; на вывод P1.0 LSB
                                ; вперед!

```

```

        RLC      R7                ; передача в переменную
                                   ; DATA_in
        DEC      R6                ; уменьшение счетчика
                                   ; битов
        JNZ      RXTX_02          ; последний бит?
        CMP.B    #RECEIVE,R7      ; → Прием?
        JEQ      init_RX
        CMP.B    #TRANSMIT,R7    ; → Передача?
        JEQ      init_TX
        BIC.B    #0FFH,&P1IFG     ; очистка флагов
                                   ; прерывания
        CLR      R7                ; очистка старого
                                   ; значения переменной
                                   ; DATA_in

        call     #init_RXTX
        RETI
init_TX CALL     #init_TATX       ; начало передачи
                                   ; RS232 → IrDA
        MOV      #66,R9           ; начало текста...
        MOV      #79,R10          ; конец текста...
        call     #TEXT_OUT        ; запись в ПК...
        BIC.B    #0FFH,&P1IFG     ; очистка флага
                                   ; прерывания
        BIC.B    #001H,&P1IE      ; Interrupt disable
                                   ; P1.0
        RETI
init_RX MOV      #53,R9           ; начало текста...
        MOV      #63,R10          ; конец текста...
        call     #TEXT_OUT        ; запись в ПК...
        BIC.B    #0FFH,&P1IFG     ; очистка флага
                                   ; прерывания
        BIC.B    #001H,&P1IE      ; запрет прерывания
                                   ; от P1.0
        call     #init_TARX       ; начало передачи
                                   ; IrDA → RS232
        RETI
;*****
; Инициализация Timer_A - TRANSMITTER
; - режим TX (RS232 → IrDA)
; - P2.5 - вход RS232
; - P2.4 CCR2 - выход IrDA (TSLM1100)
;*****

```



```

init_TATX
    MOV    #0200h,&TACTL    ; Подготовка Timer_A
                                ; (MCLK, Timer
                                ; halted...)
    CLR    CCTL1            ; запрет прерывания
                                ; от CCTL1
    MOV    #0080h,&CCTL0    ; установка регистра
                                ; управления нулевого
                                ; модуля захвата/
                                ; сравнения
    MOV    BAUDRATE,&CCR0   ; в регистр нулевого
                                ; модуля захвата/
                                ; сравнения загружает-
                                ; ся период
    MOV    #00E0h,&CCTL2    ; установка регистра
                                ; управления второго
                                ; модуля захвата/
                                ; сравнения → рабочий
                                ; режим
    MOV    BAUDRATE,R15
    SUB    #006h,R15        ; Вычитание: импульс
                                ; цикла > для увеличе-
                                ; ния/уменьшения - пе-
                                ; реключение!!
    MOV    R15,&CCR2        ; регистр второго
                                ; модуля захвата/срав-
                                ; нения → длительность
                                ; импульса 6h
    MOV    #030H,R14       ; запуск/остановка
                                ; таймера Timer_A
                                ; (режим увеличения/
                                ; уменьшения)
    BIC.B  #0FFH,&P2IFG     ; очистка флагов преры-
                                ; ваний от портов 2.x
    BIS.B  #020H,&P2IES     ; фронт IR устанавли-
                                ; вает на выводе P2.5
                                ; высокий или низкий
                                ; сигнал
    BIS.B  #020H,&P2IE      ; разрешение прерыва-
                                ; ния от P2.5
    MOV    #020H,R13       ; изменение фронта IR
                                ; изменяет состояние
                                ; на выходе P2.5

```

```

        BIS.B #010H, &P2SEL      ; модуль порта P2.4
        RET

;*****
; Инициализация Timer_A - RECEIVER
; - режим RX (IrDA → RS232)
; - P1.2 CCR1 - вход IrDA (TSLM1100!)
; - P1.1 CCR0 - выход RS232
;*****
init_TARX
        MOV     #0200h, TACTL      ; Подготовка Timer_A
                                   ; (MCLK, Timer
                                   ; halted...)
        MOV     #0000h, &CCTL0     ; CCTL0 в режиме выхо-
                                   ; да для установки
                                   ; на P1.1 высокого
                                   ; уровня
        BIS     #00004h, &CCTL0    ; P1.1 высокий уровень
        BIS     #0020h, &CCTL0     ; CCTL0 в режиме уста-
                                   ; новки → P1.1 высокий
                                   ; уровень
        MOV     BAUDRATE, &CCR0    ; загрузка в CCR0
                                   ; скорости обмена
                                   ; данными
        RLA     CCR0               ; умножение на 2
        BIS.B   #06H, &P1SEL       ; содержимое модуля
                                   ; P1.1/2 → CC1!
        MOV     #8110h, CCTL1      ; → импульс нарастаю-
                                   ; щего фронта IrDA +
                                   ; прерывание
        BIS.B   #010H, TACTL       ; запуск таймера

Timer_A
                                   ; 000h = останов
                                   ; 010h = увеличение до
                                   ; CCR0 перезапуск 0
                                   ; 020h = увеличение до
                                   ; 65536
                                   ; 030h = режим добав-
                                   ; ления/вычитания

        RET

```

```

;*****
; Подпрограмма обработки прерывания передатчика
; RS232 (P2.5) → IrDA (P2.4)
;*****
TX_01    BIS    #04h,&TACTL
          XOR    R14,&TACTL          ; запуск/остановка
                                          ; таймера Timer_A
                                          ; режим добавления/
                                          ; уменьшения)
          XOR.B  R13,&P2IES          ; изменение фронта IR
                                          ; изменяет состояние
                                          ; P2.5
          BIC.B  #0FFh,&P2IFG        ; очистка флагов
                                          ; прерывания
          RETI

;*****
; Подпрограмма обработки прерывания приемника
; IrDA (P1.2) → RS232 (P1.1)
;*****
RX_01    CLR    &CCTL0              ; CC0 в режиме выхода/
                                          ; P1.1 низкий уровень
          BIS    #004h,&TACTL        ; сброс таймера Timer_A
          MOV    #0020h,&CCTL0       ; CCTL0 устанавливает
                                          ; на выводе P1.1 низ-
                                          ; кий уровень, а "ре-
                                          ; жим → P1.1 высокий
                                          ; уровень"
          BIC    #01h,&CCTL1         ; очистка флага преры-
                                          ; вания CCTL1!
          RETI

;*****
; Подпрограмма: запись строки из TEXT(R9) в программу
; терминала
; - выход P3.3 → см. подпрограмму инициализации.
; - начальная позиция строки в регистре R9 / конечная
; позиция строки в регистре R10
;*****
TEXT_OUT PUSH  R8
          PUSH  R7
          PUSH  R6
          PUSH  R5

```

	MOV	BAUDRATE, R5	; сохранение скорости
			; обмена данными
	RRA	R5	; в регистре R5
	SUB	#03h, R5	; деление на 2
			; подстройка к синхро-
			; низации
TEXT_02	BIC.B	#02H, &P1OUT	; запись стартового
			; бита → низкий уро-
			; вень
	MOV.B	TEXT(R9), R7	; загрузка байта
			; данных Data_byte
	MOV.B	#08H, R6	; счетчик битов
	MOV.B	#08H, R6	; только для....
	JMP	TEXT_03	;задержки
TEXT_03	MOV	R8, R5	; копирование в ре-
			; гистр R8
TEXT_04	MOV	R8, R8	; пустой цикл
	DEC	R5	
	JNZ	TEXT_04	; запись следующего
			; бита?
	RLA.B	R7	; разворот для передачи
	JC	TEXT_H	
	BIC.B	#02H, &P1OUT	; запись низкого
			; состояния бита
	DEC	R6	
	JNZ	TEXT_03	; следующий байт?
	JMP	TEXT_05	
TEXT_H	BIS.B	#02H, &P1OUT	; запись высокого
			; состояния бита
	DEC	R6	
	JNZ	TEXT_03	; следующий байт?
TEXT_05	BIS.B	#02H, &P1OUT	; запись высокого
			; состояния стоп бита
	MOV	#0F000h, R5	; задержка между
			; каждым символом
			; ...10 мс
TEXT_06	DEC	R5	
	JNZ	TEXT_06	
	INC	R9	
	CMP	R10, R9	; цифра x?
	JNZ	TEXT_02	; следующий байт
	POP	R5	
	POP	R6	

```

        POP      R7
        POP      R8
        RET

; *****
; Переменные программы
; - STATUS
; - BAUDRATE .WORD #0200h
; *****

.sect "RAM", 0200h
STATUS_1 .BYTE #00H
STATUS_2 .BYTE #00H
BAUDRATE .WORD #00H

; .sect "ROM" ;MCLK = 3.6864 МГц/t=271.2 нс
;BAUD2400 .WORD #0300h ;1536/2 циклов
;BAUD9600 .WORD #00C0h ;384/2 циклов
;BAUD1920 .WORD #0060h ;192/2 циклов
;BAUD3840 .WORD #0030h ;96/2 циклов
;BAUD5760 .WORD #0020h ;64/2 циклов
;BAUD1152 .WORD #0010h ;32/2 циклов
; .sect "ROM" ;MCLK = 4.194 МГц/t=238.4 нс
;BAUD2400 .WORD #036Bh ;1750/2 циклов
;BAUD9600 .WORD #00DAh ;436/2 циклов
;BAUD1920 .WORD #006Dh ;218/2 циклов
;BAUD3840 .WORD #0037h ;110/2 циклов
;BAUD5760 .WORD #0024h ;72/2 циклов
;BAUD1152 .WORD #0012h ;36/2 циклов
; sect "ROM" ;MCLK = 5.0000 МГц/t=200 нс
;BAUD2400 .WORD #0412h ;2083/2 циклов
;BAUD9600 .WORD #0104h ;520/2 циклов
;BAUD1920 .WORD #0082h ;260/2 циклов
;BAUD3840 .WORD #0041h ;130/2 циклов
;BAUD5760 .WORD #002Bh ;86/2 циклов
;BAUD1152 .WORD #0016h ;43/2 циклов
.sect "ROM" ;MCLK = 6.144 МГц/t=162.8 нс
BAUD2400 .WORD #0500h ;2560/2 циклов
BAUD9600 .WORD #0140h ;640/2 циклов
BAUD1920 .WORD #00A0h ;320/2 циклов
BAUD3840 .WORD #0050h ;160/2 циклов
BAUD5760 .WORD #0035h ;107/2 циклов
BAUD1152 .WORD #001Ah ;53/2 циклов
; .sect "ROM" ;MCLK = 7.3728 МГц/t=135.6 нс
;BAUD2400 .WORD #0600h ;3072/2 циклов
;BAUD9600 .WORD #0180h ;768/2 циклов

```

```

;BAUD1920 .WORD #00C0h ;384/2 ЦИКЛОВ
;BAUD3840 .WORD #0060h ;192/2 ЦИКЛОВ
;BAUD5760 .WORD #0040h ;128/2 ЦИКЛОВ
;BAUD1152 .WORD #0020h ;64/2 ЦИКЛОВ
TEXT      .BYTE 005h      ;" "
          .BYTE 0B1h      ;"CR1"
          .BYTE 050h      ;"CR2"
          .BYTE 02Bh      ;"T"
          .BYTE 0A6h      ;"e"
          .BYTE 01Eh      ;"x"
          .BYTE 087h      ;"a"
          .BYTE 0CFh      ;"s"
          .BYTE 005h      ;" "
          .BYTE 093h      ;"I"
          .BYTE 077h      ;"n"
          .BYTE 0CFh      ;"s"
          .BYTE 02Eh      ;"t"
          .BYTE 04Eh      ;"r"
          .BYTE 0AFh      ;"u"
          .BYTE 0B7h      ;"m"
          .BYTE 0A6h      ;"e"
          .BYTE 077h      ;"n"
          .BYTE 02Eh      ;"t"
          .BYTE 0CFh      ;"s"
          .BYTE 005h      ;" "
          .BYTE 08Dh      ;"1"
          .BYTE 09Ch      ;"9"
          .BYTE 09Ch      ;"9"
          .BYTE 01Dh      ;"8"
          .BYTE 005h      ;" "
          .BYTE 0B1h      ;"CR1"
          .BYTE 050h      ;"CR2"
TXT_28    .BYTE 0B2h      ;"M"
          .BYTE 0CAh      ;"S"
          .BYTE 00Ah      ;"P"
          .BYTE 02Dh      ;"4"
          .BYTE 0CCh      ;"3"
          .BYTE 00Ch      ;"0"
          .BYTE 01Eh      ;"x"
          .BYTE 08Dh      ;"1"
          .BYTE 08Dh      ;"1"

```

```

        .BYTE 01Eh      ; "x"
        .BYTE 005h      ; " "
        .BYTE 093h      ; "I"
        .BYTE 04Eh      ; "r"
        .BYTE 022h      ; "D"
        .BYTE 082h      ; "A"
        .BYTE 0B4h      ; "-"
        .BYTE 022h      ; "D"
        .BYTE 0A3h      ; "E"
        .BYTE 0B2h      ; "M"
        .BYTE 0F3h      ; "O"
        .BYTE 0B1h      ; "CR1"
        .BYTE 050h      ; "CR2"
TXT_50  .BYTE 07Bh      ; "^"
        .BYTE 04Eh      ; "r"
        .BYTE 0BDh      ; "="
TXT_53  .BYTE 04Bh      ; "R"
        .BYTE 0A3h      ; "E"
        .BYTE 0C3h      ; "C"
        .BYTE 0A3h      ; "E"
        .BYTE 093h      ; "I"
        .BYTE 06Ah      ; "V"
        .BYTE 0A3h      ; "E"
        .BYTE 04Bh      ; "R"
        .BYTE 0B1h      ; "CR1"
TXT_62  .BYTE 050h      ; "CR2"
        .BYTE 07Bh      ; "^"
        .BYTE 02Eh      ; "t"
        .BYTE 0BDh      ; "="
TXT_66  .BYTE 02Bh      ; "T"
        .BYTE 04Bh      ; "R"
        .BYTE 082h      ; "A"
        .BYTE 072h      ; "N"
        .BYTE 0CAh      ; "S"
        .BYTE 0B2h      ; "M"
        .BYTE 093h      ; "I"
        .BYTE 02Bh      ; "T"
        .BYTE 02Bh      ; "T"
        .BYTE 0A3h      ; "E"
        .BYTE 04Bh      ; "R"
        .BYTE 0B1h      ; "CR1"
TXT_78  .BYTE 050h      ; "CR2"

```

```

;*****
; Векторы прерываний
;*****
        .sect "Int_Vect", 0FFE0h
        .word RESET      ; от портов P0.2 - P0.7
        .word RESET      ; от основного таймера
        .word RXTX_01     ; от линий ввода/вывода
                           ; порта P1
        .word TX_01       ; от линий ввода/вывода
                           ; порта P2
        .word RESET      ; от таймера/порта
        .word RESET      ; нет источника
        .word RESET      ; нет источника
        .word RESET      ; нет источника
        .word RX_01       ; от таймера Timer_A
        .word RESET      ; от модуля Timer_A/CAP/CMP
        .word RESET      ; от сторожевого таймера
        .word RESET      ; нет источника
        .word RESET      ; от P0.1
        .word RESET      ; от P0.0
        .word RESET      ; от NMI
        .word RESET      ; от POR, внешнего сброса
                           ; и сторожевого таймера

```


IV. Примеры конкретных применений

Глава 1.

*Использование TPS3619 совместно
с микроконтроллерами MSP430
может уменьшить системное
энергопотребление*

Использование TPS3619 совместно с микроконтроллерами MSP430 может уменьшить системное энергопотребление

Dennis Hudgins [sbva014a.pdf (Англ.)]

Микроконтроллеры семейства MSP430 идеальны для применения в тех устройствах, для которых время работы без подзарядки аккумулятора является критическим параметром. Эти микроконтроллеры имеют дежурный режим «с сохранением содержимого ОЗУ», в котором ток потребления не превышает 0.1 мкА. В некоторых системах, имеющих преобразователь напряжения питания, может использоваться прибор TPS3619, который позволяет отключить этот преобразователь, что позволит снизить потребляемую системой мощность.

Для многих устройств с автономным питанием ток потребления в дежурном режиме является более критическим параметром, чем ток потребления в рабочем режиме. Это особенно справедливо для переносной контрольно-измерительной аппаратуры с автономным питанием типа цифровых термометров, измерителей содержания сахара в крови или тонометров. Эти устройства производят измерения и вычисления, результат которых отображается на ЖКИ или сохраняется в памяти. Для увеличения срока службы батареек или аккумуляторов между измерениями система находится в дежурном режиме или режиме пониженного потребления. Так как такие устройства большую часть времени находятся в дежурном режиме, то снижение потребления в этом режиме позволяет существенно увеличить срок службы аккумулятора. Так как микроконтроллер хранит данные в энергозависимом ОЗУ, то и в режиме бездействия должно обеспечиваться его питание. В системах, которые используют преобразователь напряжения питания, супервизор источника автономного питания (SVS), например TPS3619, позволяет в дежурном режиме отключать преобразователь напряжения питания. В дежурном режиме TPS3619 позволяет питать микроконтроллер MSP430 непосредственно от батарейки, а преобразователь напряжения питания отключать, что позволяет снизить общее потребление системы. На рисунке 1 показана типовая схема включения TPS3619 и преобразователя напряжения для питания микроконтроллеров семейства MSP430.

В схеме, приведенной на рисунке 1 использован преобразователь напряжения REG710, который при работе от двух щелочных/никель-кадмиевых (NiCad) батарей формирует 3.3 В напряжение. При необходимости можно использовать и другие преобразователи напряжения питания.

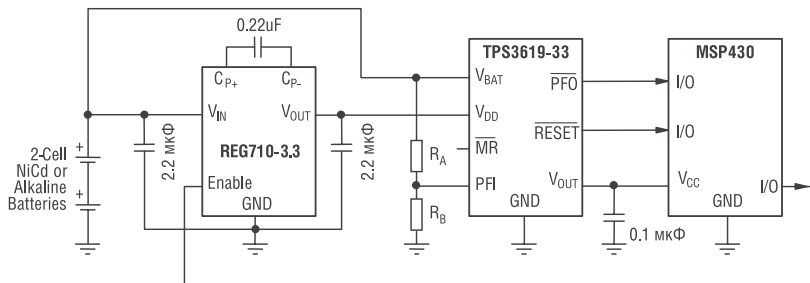


Рис. 1. Типовая схема включения TPS3619 и преобразователя напряжения для питания микроконтроллеров семейства MSP430

После окончания преобразования и сохранения данных MSP430 может отключить преобразователь REG710. После отключения REG710, напряжение на его выходе начинает уменьшаться. Когда оно достигает установленного порогового значения V_{IT} , то супервизор напряжения питания переключается на работу от напряжения поданного на его вход V_{BAT} , т.е. напрямую от батареек. В это время на выходе Reset устанавливается низкий уровень. Сигнал на этом выходе может использоваться для отслеживания режима работы SVS. Но этот сигнал не должен использоваться для сброса MSP430, так как микроконтроллер продолжает работать, питаясь непосредственно от батареек. Если же необходимо формировать сигнал сброса микроконтроллера, то вместо TPS3619 можно использовать TPS3606.

TPS3619 кроме того, что обеспечивает выбор источника питания, еще способен отслеживать снижения напряжения на входе PFI. Вход PFI подключен ко внутреннему компаратору TPS3619, который имеет порог срабатывания 1.15 В. Когда напряжение на входе PFI падает ниже 1.15 В, то на выходе $\overline{\text{PFO}}$ устанавливается низкий уровень. Резисторы R_A и R_B могут использоваться для установки произвольного порога срабатывания детектора разряда батареи, при этом не следует забывать, что MSP430 имеет минимальное рабочее напряжение 1.8 В. Формируемый этим детектором сигнал может использоваться для формирования предупреждения пользователя о разряде батарей или для перевода микроконтроллера в режим пониженного потребления. Для исключения влияния входного тока вывода PFI на точность резистивного делителя суммарное сопротивление резисторов должно быть не менее 1 МОм.

Функция отслеживания разряда батарей является вспомогательной. Следует учитывать, что резистивный делитель детектора также потребляет ток. При оговоренных условиях средний ток потребления через резисторы равняется приблизительно 2 мкА (2 В/1 МОм). Многие микроконтроллеры семейства MSP430 имеют встроенный детектор разряда батарей, который не потребляет постоянного тока, что ведет к снижению общего потребления системы.

При использовании TPS3619 в режиме пониженного потребления можно уменьшить ток потребления системы примерно на 65 мкА. В режиме работы от батарейки TPS3619 потребляет не более 0.5 мкА. В активном же режиме TPS3619-33 имеет типовой ток потребления всего 17 мкА.

IV. Примеры конкретных применений

Глава 2. *Ультразвуковое измерение дальности на MSP430*

Ультразвуковое измерение дальности на MSP430

Murugavel Raju [slaa136a.pdf (Англ.)]

В этом примере применения описан ультразвуковой дальномер на базе микропотребляющего МК MSP430F413. Прибор передаёт «пачку» волн ультразвуковой частоты по направлению объекта и принимает соответствующий отражённый сигнал. Встроенный в MSP430 аналоговый компаратор Comparator_A используется для определения момента принятия отражённого сигнала. Микроконтроллер с высокой точностью измеряет время прохождения ультразвуковой «пачкой» расстояния от излучателя до объекта и обратно. Принимая во внимание, что скорость звука при комнатной температуре равна 1100 футов/сек, MSP430 вычисляет расстояние между прибором и объектом и выводит значение на 2-цифровой ЖКИ со статическим управлением при помощи встроенного ЖК-драйвера. Расстояние отображается с точностью 1 дюйм. Минимально измеряемое расстояние – 8 дюймов, оно ограничено физическими характеристиками излучателя. Максимальное измеряемое расстояние – 99 дюймов. Амплитуда отражённого сигнала зависит от материала объекта, его формы и размера. Звукопоглощающие предметы, такие, как ковры и отражающие объекты площадью менее 2-х квадратных футов плохо отражают сигнал, для таких предметов максимально измеряемое расстояние меньше. Если величина отражённого сигнала меньше порога срабатывания Comparator_A, прибор перейдёт в режим перегрузки. На экране будет показано сообщение об ошибке E.

1. Теория работы

Прибор основан на эффекте отражения звуковых волн. Эти волны можно представить как продольные колебания давления в среде их распространения. Предметы, размеры которых превышают длину падающей звуковой волны, отражают её; отражённая волна называется эхом. Если скорость звука в среде известна и можно измерить время распространения волны от источника до объекта и обратно, то расстояние между излучателем и предметом может быть точно вычислено. На этом принципе измерения и основан данный прибор. Средой распространения звуковых волн в данном случае является воздух, а звуковые волны излучаются ультразвуковым диапазоне, т.к. он не воспринимается человеческим ухом. Принимая скорость звука в воздухе равной 1100 футов в секунду при комнатной температуре и обозначив время распространения звуковой волны от излучателя до объекта и обратно через t (сек), расстояние d вычисляется по формуле $d=1100 \times t$

X t (дюймов). Так как звуковые волны преодолевают удвоенное расстояние между излучателем и предметом, реальное расстояние между источником и объектом будет равно $d/2$.

2. Описание схемы

В данном приборе использованы керамические ультразвуковые излучатели на частоту 40 кГц. MSP430 подаёт на излучатель пачку из 12 импульсов частотой 40 кГц прямоугольной формы стабилизированной при помощи кварцевого резонатора и принимает «эхо» с помощью УЗ-приёмника. Таймер Timer_A в MSP430 сконфигурирован для подсчёта 40-килогерцовых импульсов от кварца, таким образом, временное разрешение измерения составляет 25 мкс, что более чем достаточно для данных целей. Тактовая частота для измерений получена при помощи кварцевого генератора, что обеспечивает её высокую стабильность. Отражённый сигнал, принятый приёмником усиливается операционным усилителем, выход которого подключен к входу компаратора Comparator_A. Компаратор

Comparator_A определяет наличие сигнала на входе и формирует сигнал захвата для таймера Timer_A, результат счёта при этом «защёлкивается» в регистре захвата-сравнения CCR1. Время осуществления «защёлкивания» в точности соответствует времени прихода импульсов отражённого сигнала. Запомненный в регистре результат счёта соответствует времени, затраченному пачкой ультразвуковых импульсов на преодоление расстояния от прибора до объекта и обратно. Расстояние в дюймах вычисляется микроконтроллером MSP430 исходя из измеренного времени и выводится на 2-х цифровой ЖКИ со статическим управлением. Сразу после этого МК переходит в режим пониженного потребления LPM3 для снижения потребляемой мощности. Таймер Basic Timer1 запрограммирован на формирование прерываний каждые 205 мс. Прерывание от Basic Timer1 переводит MSP430 в активный режим, при этом повторяются цикл измерения и вывод на ЖК. На рис.1 показана принципиальная схема прибора. МК MSP430F413 (U1) является главным элементом схемы. По ссылке [1] находится data sheet на данный МК. LCD1 – 2-х цифровой низковольтный ЖКИ со статическим управлением, управляемый встроенным драйвером ЖКИ. R03 подключен к VSS, а R13 и R23 оставлены неподключенными, таким образом организовано статическое управление ЖК. Кварцевый резонатор на 40 кГц X1 специально выбран для использования в составе низкочастотного кварцевого генератора для обеспечения резонансной частоты используемых в приборе ультразвуковых излучателей. R12 является «подтягивающим» резистором для вывода Reset, а за рабочим уровнем напряжения следит встроенный супервизор. Конденсатор C9 обеспечивает

фильтрацию питания и должен быть расположен как можно ближе к выводам питания. 14-выводный разъём J1 предназначен для подключения интерфейса JTAG к MSP430 для внутрисхемного программирования и отладки при помощи MSP430 flash emulation tool. Светодиод LED1 сигнализирует о циклах измерения. Вывод порта P1.5 сконфигурирован как выход частоты ACLK, требуемой для УЗ-излучателя.

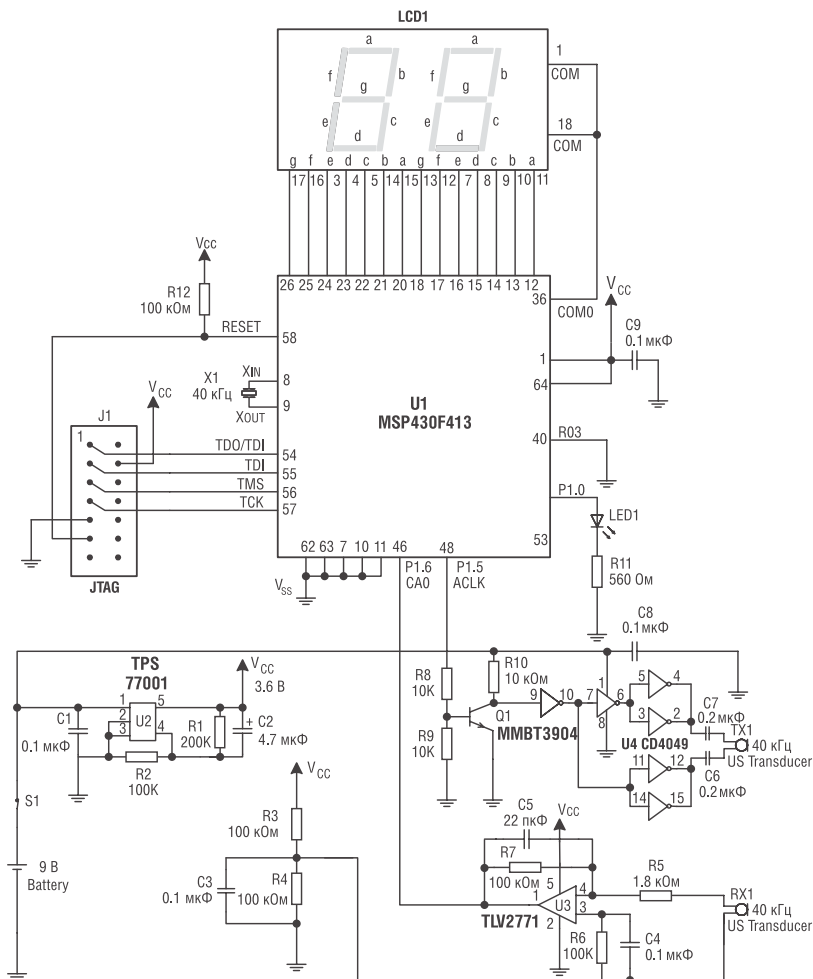


Рис. 1. Схема прибора

Цепь выходного драйвера раскачки излучателя запитана непосредственно от 9-вольтовой батареи и обеспечивает на выходе размах сигнала 18 VPP. Это напряжение получается при помощи мостовой схемы на 4-х инверторах U4-CD4049. Data sheet на CD4049 по ссылке [6]. Один из инверторов вращает фазу сигнала на 180° для одного из плеч драйвера, на второе плечо поступает неинвертированный сигнал. При таком построении драйвера на выходе обеспечивается размах сигнала 18 VPP, необходимый для излучателя. По два инвертора соединены в параллель для удвоения выходного тока. Конденсаторы C6 и C7 обеспечивают развязку излучателя по постоянному току. Т.к. CD4049 запитан от 9 В, а MSP430 от 3.6 В, уровень логического сигнала не согласован. Биполярный транзистор Q1 служит преобразователем уровня.

Операционный усилитель U3 это 5-выводный ОУ с высокой скоростью нарастания выходного сигнала TLV2771 производства TI, документация на который расположена по ссылке [5]. Этот усилитель имеет широкую полосу сигнала и обеспечивает высокое усиление на частоте 40кГц. ОУ включен по инвертирующей схеме. КУ устанавливается резисторами R7 и R5 и равен 55, C5 служит для частотной коррекции. R3 и R4 создают смещение на неинвертирующем входе ОУ для корректной работы с однополярным источником питания. Усиленный ультразвуковой сигнал является двухполярным относительно постоянного уровня в данной точке. Высокая добротность приёмника RX1 обеспечивает требуемую избирательность и ослабление частот, кроме 40 кГц. Выход ОУ подключен ко входу CA0 компаратора Comparator_A через вывод порта P1.6. опорное напряжение компаратора Comparator_A выбрано от внутреннего источника 0.5 VCC. Пока не принято ультразвуковое «эхо» уровень напряжения на входе CA0 несколько меньше, чем на опорном CA1. При приёме сигнала уровень на входе возрастает выше опорного, при этом переключается выход компаратора Comparator_A CAOUT. Резистором R3 осуществляется точная подстройка чувствительности и, соответственно, оптимального диапазона измерения.

MSP430 и усилитель сигнала U3-приёмника питаются от управляемого источника 3.6 В, подключенного к 9-вольтовой батарейке. Источник выполнен на стабилизаторе с низким падением напряжения на регулирующем элементе марки TPS77001. Данная м/сх выпускается фирмой TI, информацию можно найти по ссылке [4]. Резисторы R1 и R2 устанавливают напряжение на выходе стабилизатора на уровне 3.6 В. Конденсаторы C1 и C2 рекомендуются типовой схемой включения стабилизатора. Передатчик питается непосредственно от 9-вольтовой батареи. Выключатель S1 управляет питанием прибора.

На рис.2 приведена осциллограмма пачки из 12 импульсов частотой 40 кГц. Обратите внимание, что напряжение «от пика до пика» составляет 19.2 В. Синусоидальный «звон» на вершинах прямоугольных импульсов вызван резонансом излучателя.

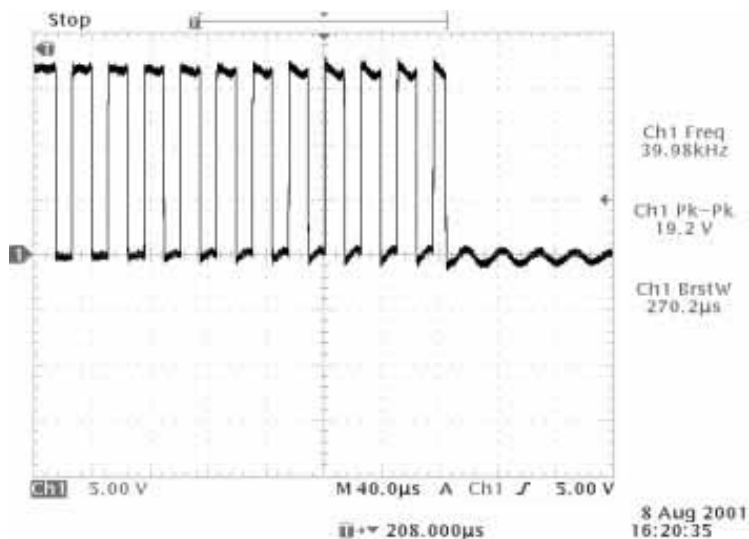


Рис. 2. Осциллограмма «пачки» 40-килогерцовых импульсов на выходе передатчика

На рис. 3 изображены осциллограммы для полного цикла измерения. Верхняя осциллограмма показывает пачку из 12 40-килогерцовых импульсов на

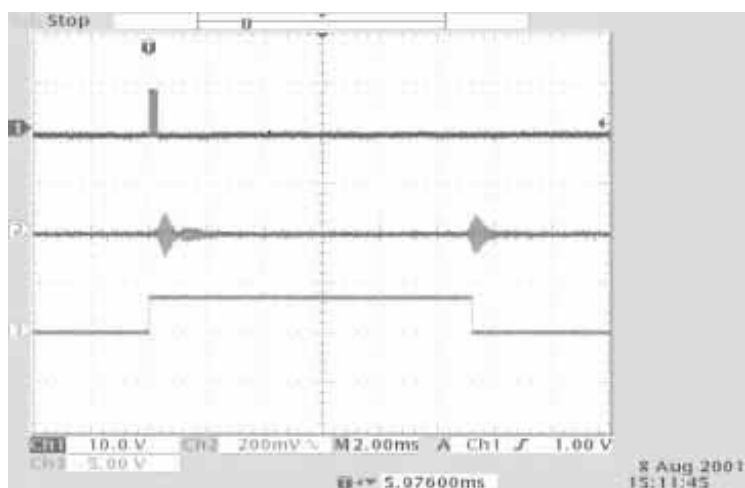


Рис. 3. Осциллограммы для 1-го измерительного цикла

выходе излучателя. На следующей осциллограмме показан усиленный выход приёмника (вывод 1 ОУ). Первая «пачка» – это сигнал, принятый непосредственно с передатчика и проигнорированный микроконтроллером. Следующая «пачка» – это отражённое эхо, этот сигнал используется в MSP430 для измерения. На последней осциллограмме показан измеренный микроконтроллером интервал времени. Он соответствует времени распространения ультразвукового сигнала от прибора до объекта и зависит от измеряемого расстояния.

3. Программа Ultrasonic.s43

3.1. Инициализация *Init_Device*

В этой подпрограмме инициализируется и конфигурируется периферия контроллера. Прежде всего запрещается сторожевой Watchdog Timer. Для стабилизации низкочастотного кварцевого генератора вводится программная задержка. Умножитель частоты FLL+ имеет значение 64, что даёт частоту MCLK 2.56 МГц. Вывод P1.0 сконфигурирован как выход для управления светодиодом. неиспользуемые выводы сконфигурированы как выходы, а вывод P1.5 настроен на вывод 40-килогерцовой буферизованной частоты ACLK. Базовый таймер Basic Timer1 включен и сконфигурирован таким образом, чтобы формировать частоту 150 Гц для ЖКИ и вызывать прерывания ЦПУ каждые 205 мс для проведения цикла измерений. Компаратор Comparator_A использует внутренне опорное напряжение 0.5 VCC, а биты CAPD установлены в «1», что отключает буфера на входах компаратора. Модуль ЖК контроллера включен и сконфигурирован в режиме статического управления для подключения 2-цифрового ЖКИ прибора. Дисплейная память очищена, т.е. по умолчанию ЖК показывает значение 00. Затем разрешаются прерывания от базового таймера Basic Timer1 и включается общее разрешение прерываний Global Interrupt Enable (GIE), что приводит к периодическим прерываниям ЦПУ.

3.2. Основной цикл *Mainloop*

В основном цикле на ЖК выводится новое значение, которое хранится в буфере DIGITS, после чего MSP430 переходит в режим пониженного потребления LPM3. MSP430 остаётся в этом режиме до тех пор, пока не возникнет прерывание от базового таймера Basic Timer1, после чего процессор перейдёт к выполнению обработчика прерываний базового таймера BT_ISR. При этом начинается цикл измерения. Таймер Timer_A работает в 16-битном режиме и тактируется частотой ACLK. Регистр CCR1 находится в режиме сравнения с числом 12, таким образом, на выходе P1.5 получаем «пачку» из 12 импульсов частотой 40 кГц. Далее формируется задержка из 36 тактов частоты ACLK для прекращения колебаний на выходе излучателя. Это осуществляется перево-

дом регистра CCR1 в режим сравнения с числом 36. Во время этой паузы, процессор MSP430 пребывает в режиме низкого потребления LPM0.

Теперь прибор готов к приёму отражённого сигнала УЗ-приёмником. Компаратор Comparator_A сконфигурирован на ожидание сигнала эха, после получения которого формируется прерывание для «захвата» значения таймера Timer_A в регистре захвата-сравнения CCR1. Это значение соответствует измеренному времени, в течение которого ультразвуковые волны преодолевали расстояние от излучателя до объекта и обратно. К полученному значению прибавляется 48 для компенсации времени формирования 12 импульсов «пачки» и 36 тактов ожидания установления передатчика. Скорректированное значение в регистре CCR1 представляет собой точный временной интервал между началом «пачки» и временем принятия эха. Далее вызывается подпрограмма математической обработки, вычисляющая значение расстояния в дюймах и возвращающая соответствующий результат. В случае перегрузки (объект вне зоны действия) эхо не будет получено и компаратор Comparator_A не сформирует сигнал прерывания. MSP430 останется в режиме пониженного энергопотребления LPM0 до следующего прерывания базового таймера. В этом случае проверяется бит CAIFG в регистре CCTL1 для установления того факта, что эхо не было получено. Чтобы отобразить данное состояние, в буфер DIGITS записывается значение 0xBE, что приведёт к индикации буквы E на экране ЖКИ. Далее программа возвращается к началу основного цикла Mainloop, обновляет индикацию на ЖКИ и переходит в режим пониженного потребления LPM3. Следующее прерывание базового таймера Basic Timer1 возвратит MSP430 в активный режим и выполнение программы продолжится.

3.3. Математические вычисления *Math_calc*

Подпрограмма Math_calc отвечает за выполнение математических вычислений, необходимых в приборе. Скорректированное 16-битное значение в регистре CCR1 сохраняется в переменной Result. Это значение представляет из себя время, в течение которого ультразвуковые волны преодолевали расстояние от излучателя до объекта и обратно. Так как таймер Timer_A считает время 25-микросекундными интервалами, то реальное время вычисляется как Result X 25 мкс. Принимая во внимание, что скорость звука при комнатной температуре составляет 1100 футов/сек, значение Result, полученное из таймера Timer_A соответствует 6-ти отсчётам на один дюйм расстояния. Таким образом, разделив значение Result на 6, получим ожидаемое значение расстояния в дюймах. Для обеспечения требуемой точности при использовании целочисленных вычислений в MSP430, 16-битное значение Result сначала умножается на 100, потом делится на 6. Умножение 16X16-бит осуществляется

подпрограммой Mul100. 32-битный результат умножения сохраняется в переменных htX100_msw и htX100_lsw. После этого 32-битный результат делится на 6 и полученное значение записывается в ячейку DIGITS. В этой переменной значение хранится в шестнадцатеричном виде. Подпрограмма hex2bcd конвертирует его в двоично-десятичный формат (BCD), при этом две последние цифры двоично-десятичного числа отбрасываются чтобы скомпенсировать предыдущее умножение на 100. Полученное двухразрядное число записывается обратно в ячейку DIGITS.

3.4. Обработчик прерывания базового таймера BT_ISR

Подпрограмма обработки прерываний базового таймера Basic Timer1 BT_ISR изменяет биты статусного регистра SR, находящегося в стеке таким образом, чтобы обеспечить функционирование MSP430 в активном режиме после возврата из подпрограммы обработки прерываний. Это требуется для того, чтобы MSP430 продолжил выполнение программы, следующей за инструкцией LPM3 в основном цикле Mainloop.

3.5. Индикация Display

Эта подпрограмма обновляет информацию на двухцифровом статическом ЖКИ используя информацию, содержащуюся в переменной DIGITS. Данные для линий сегментов статического ЖКИ хранятся в таблице LCD_Tab. Загрузка в память ЖК требуемых данных для сегментов осуществляется выборкой из таблицы LCD_Tab значения, соответствующего числу, хранящемуся в переменной DIGITS.

3.6. Задержка Delay

Эта подпрограмма обеспечивает программную задержку с требуемым 16-битным значением. Значения в регистрах не изменяются, т.к. для счёта используется переменная, расположенная сверху стека (TOS). После окончания задержки указатель стека (SP) инкрементируется, таким образом, перед возвратом из подпрограммы ему возвращается первоначальное значение.

4. Заключение

Такие периферийные устройства, как аналоговый компаратор Comparator_A, 16-битный таймер Timer_A с аппаратными регистрами захвата-сравнения, базовый таймер Basic Timer1 и драйвер ЖКИ значительно упрощают разработку ультразвукового дальномера и обеспечивают однокристальное решение. Средний потребляемый ток прибора 1.3 мА при измерении расстояния в 15 дюймов. Сюда входит собственное потребление LDO-стабилизатора U2, операционного усилителя U3, и КМОП-инвертера U4. Только ОУ потребляет 1 мА, таким обра-

зом, вся оставшаяся схема потребляет 300мкА. Ток светодиода во включенном состоянии – 5 мА. MSP430 потребляет в среднем 2.1 мкА с учётом постоянно включенного ЖКИ. Это стало возможным благодаря использованию преимуществ функций ультранизкого энергопотребления MSP430. Основное время MSP430 находится в режиме пониженного потребления LPM3, ресурсы ЦПУ использованы всего на 5.6%.

Т.к. скорость звука зависит от температуры, измерение расстояния будет иметь большую погрешность при температурах, отличающихся от комнатной. Для повышения точности измерения во всём диапазоне температур прибор может быть дополнен простым измерителем температуры на термисторе и соответствующей коррекцией показаний. Измеренное расстояние и температура также, по желанию, могут быть сохранены во встроенной флеш-памяти. Добавление дополнительного каскада усиления сигнала приёмника и использование мультиплексированного ЖКИ с любым желаемым количеством цифр позволит увеличить диапазон измерения.

Ссылки:

1. MSP430x41x Mixed Signal Microcontroller data sheet SLAS340
2. MSP430x4xx Family User's Guide, SLAU056
3. MSP430 Family Mixed-Signal Microcontrollers, application report SLAA024
4. TPS770xx Ultra Low-Power LDO Linear Regulators, data sheet SLVS210
5. TLV277x Family of High-Slew-Rate Operational Amplifiers, data sheet SLOS209
6. CD4049UB, CMOS Hex Inverting Buffer/Converter, data sheet SCHS046A

Приложение: Листинг программы

```
; *****
; *****
; Демо - программа ультразвукового дальномера на базе
; MSP430F413
; *****
#include <msp430x41x.h>      ;Стандартные определения
;Назначение регистров
; *****
#define DIGITS R11
#define Result R10
#define IRBT R9
#define IROP1 R4
#define IROP2L R5
#define IROP2M R6
```

```

#define IRACL R7
#define IRACM R8

;*****
;Определение переменных
;*****
        RSEG    UDATA0
        htX100_msw: DS 2          ; слово по адресам
                                   ; ОЗУ RAM 200h & 201h
        htX100_lsw: DS 2          ; 202h & 203h
;*****
        RSEG    CSTACK            ; Начало сегмента стека
        DS      0
        RSEG    CODE              ; Начало программного
                                   ; сегмента
RESET    mov.w   #SFE(CSTACK),SP  ; Указатель стека
        call    #Init_Device      ; Инициализация
                                   ; устройства
        mov.w   #0,DIGITS         ; Очистка буфера
                                   ; DIGITS

Mainloop
        bic.b   #CAON,&CACTL1     ; Отключить компаратор
                                   ; Comparator_A
        call    #Display          ; Вывести данные
                                   ; на ЖКИ
        bis.w   #LPM3,SR          ; Перейти в режим
                                   ; пониженного потреб-
                                   ; ления LPM3

;*** Отправить пачку импульсов и начать измерения ***
        clr.w   &CCTL1           ; Запретить CCTL1
        clr.w   &TACTL           ; Запретить таймер
                                   ; timer_A
        bis.b   #BIT0,&P1OUT      ; Включить светодиод

SetupTimerA
        mov.w   #TASSEL0+TACLR+MC1,&TACTL
                                   ; TACLK = ACLK, 16 бит
                                   ; режим up mode
        bis.b   #BIT5,&P1SEL      ; вывод ACLK на P1.5
        mov.w   #12,&CCR1         ; 12 импульсов часто-
                                   ; той 40KHz
        mov.w   #CCIE,&CCTL1     ; Прерывания в режиме
                                   ; сравнения

```

```

bis.w #LPM0,SR           ; Ждём прерывания от
                          ; CCR1
bic.b #BIT5,&P1SEL       ; Отключить ACLK на
                          ; порту P1.5

TimerCLR
bis.w #TACLR,&TACTL
mov.w #36,&CCR1           ; Пауза для установле-
                          ; ния передатчика
mov.w #CCIE,&CCTL1       ; Прерывания в режиме
                          ; сравнения
bis.w #LPM0,SR           ; Ждём прерывания от
                          ; CCR1
bis.b #CAON,&CACTL1      ; Включить компаратор
                          ; Comparator_A
bic.b #CAIFG,&CACTL1     ; Очистить флаг преры-
                          ; вания от Comparator_A
mov.w #CM0+CCIS0+SCS+CAP+CCIE,&CCTL1
                          ; Прерывание по фронту
                          ; от компаратора
push    &TAR             ; TOS = TAR в начале
                          ; измерения
bis.w #LPM0,SR           ; Ждём прерывания
                          ; от CCR1 (Эхо)
clr.w &CCTL1             ; Запрещаем CCTL1
bic.b #BIT0,&P1OUT       ; Выключаем светодиод
bit.b #CAIFG,&CACTL1     ; Проверяем, принято
                          ; ли эхо
jz      Next             ; если нет - перегрузка
mov.w &CCR1,Result       ; Result = TAR (CCR1)
sub.w @SP+,Result        ; Result = время
add.w #48,Result         ; Компенсируем время
                          ; на излучение пачки
                          ; импульсов и задержку
                          ; 36 циклов на уста-
                          ; новление передатчика

;***** Завершение измерения *****
call    #Math_calc       ; Вызов подпрограммы
                          ; математических
                          ; вычислений
swpb    DIGITS           ; Сдвиг на две цифры
                          ; для эмуляции деления
                          ; на 100

```



```

        jmp      Mainloop          ; Следующий цикл
                                   ; измерения
Next    mov.w    #0beh,DIGITS      ; Вывести на ЖКИ «Е»
                                   ; если не принято эхо

        jmp      Mainloop

;*****
Init_Device    ; Инициализация MSP430x41x
;*****
        mov.w    #WDTPW+WDTHOLD,&WDTCTL
                                   ; Остановить стороже-
                                   ; вой таймер WDT
        bis.b    #030h,&FLL_CTL0   ; Включить встроенные
                                   ; конденсаторы генера-
                                   ; тора
        call     #Delay            ; Задержка для стаби-
                                   ; лизации генератора
        mov.b    #03fh,&SCFQCTL    ; MCLK = 40КГцX64 =
                                   ; 2.56МГц
        call     #Delay            ; задержка для стаби-
                                   ; лизации умножителя
                                   ; частоты FLL
SetupP1 mov.b    #000h,&P1OUT       ; Очистить выходную
                                   ; защёлку порта P1
        bis.b    #0bfh,&P1DIR       ; Неиспользуемые
                                   ; выводы назначить
                                   ; выходами
        bis.b    #040h,&P1SEL       ; Вывод будет исполь-
                                   ; зован для компарато-
                                   ; ра Comp_A
SetupP2 mov.b    #000h,&P2OUT       ; Очистить выходную
                                   ; защёлку порта P2
        bis.b    #0ffh,&P2DIR       ; Неиспользуемые
                                   ; выводы назначить
                                   ; выходами
SetupP6 mov.b    #000h,&P6OUT       ; Очистить выходную
                                   ; защёлку порта P2
        bis.b    #0ffh,&P6DIR       ; Неиспользуемые
                                   ; выводы назначить
                                   ; выходами
SetupBT mov.b    #BTRFQ0+BTRFQ1+BTIP2+BTDIV,&BTCTL
                                   ; Разрешить базовый
                                   ; таймер BT с частотой

```

```

; обновления ЖК 150 Гц
; и прерываниями
; каждые 250 мс
SetupCA mov.b #CAPD6,&CAPD      ; Запретить буфер
; в компараторе
mov.b #P2CA0,&CACTL2           ; P1.6 – вход компара-
; тора
mov.b #CARSEL+CAREF1+CAON,&CACTL1 ; Включить компаратор,
; внутр. опоры 1/2Vcc
SetupLCD bis.b #LCDON+LCDSON+LCDSG0_7,LCDCTL
; Вкл. модуль ЖК
; в статическом режиме
ClearLCD mov #15,R15           ; Очистка 15 знакомест
; ЖКИ
mov.b #LCDMEM,R14
Clear1 mov.b #0,0(R14)         ; Запись нулей в дисп-
; лейную память
inc.b R14
dec R15                        ; Все сегменты очищены?
jnz Clear1                    ; Если нет – чистим
; дальше
bis.b #BTIE,&                  ; Разрешить прерывания
; от базового таймера
; Basic Timer
eint                           ; Общее разрешение
; прерываний
ret

;*****
BT_ISR      ;Обработчик прерываний базового таймера
;Basic Timer ;после RETI ЦПУ остаётся
;в активном режиме
;*****
bic #LPM3,0(SP)      ; Очистить биты LPM3
; в стеке TOS
reti                ; при возврате из пре-
; рывания
;*****
TAX_ISR      ;Общий обработчик прерывания для
; регистров захвата сравнения CCR1-4
; и переполнения таймера
;*****

```

```

        add.w  &TAIV,PC      ; Добавить смещение, соот-
                               ; ветствующее источнику пре-
                               ; рывания к программному
                               ; счётчику PC
        reti                ; CCR0 – не обрабатывается
        jmp    CCR1_ISR     ; по CCR1 вызывается обработка
        reti                ; CCR2 – не обрабатывается
        reti                ; CCR3 – не обрабатывается
        reti                ; CCR4 – не обрабатывается
TA_over reti                ; переполнение таймера Timer_A
CCR1_ISR bic.w #CCIFG,&CCTL1
        bic.w  #LPM0,0(SP)   ; По reti выход из
                               ; режима пониженного
                               ; потребления LPM0

        reti                ;

;*****
Display      ; Подпрограмма индикации значений
              ; DIGIT1 & DIGIT2
              ; ЦПУ использует регистры R15, R14, R13
              ; и R12, они не сохраняются
;*****
        mov.w  #LCDM1,R15   ; В R15 указатель на первую
                               ; позицию ЖКИ
        mov.b  DIGITS,R14   ; Значение копируется в R14
OutLCD mov.b  R14,R13       ; R14 копируется в R13
        rra.b  R13          ; Сдвиг вправо
        rra.b  R13          ; 4 раза
        rra.b  R13          ; для обмена
        rra.b  R13          ; тетрад местами
        and.b  #0Fh,R14     ; младшая тетрада → R14
        and.b  #0Fh,R13     ; старшая тетрада → R13
        mov.b  LCD_Tab(R14),R12
                               ; Вывод младшей тетрады
                               ; в 1-ю цифру ЖКИ
        mov.b  R12,0(R15)   ; Вывод сегментов a & b млад-
                               ; шей тетрады в ЖКИ

        rra.w  R12
        inc.b  R15
        mov.b  R12,0(R15)   ; Вывод сегментов c & d млад-
                               ; шей тетрады в ЖКИ

```

```

rra.w R12
inc.b R15
mov.b R12,0(R15) ; Вывод сегментов e & f
                  ; младшей тетрады в ЖКИ

rra.w R12
inc.b R15
mov.b R12,0(R15) ; Вывод сегментов g & h
                  ; младшей тетрады в ЖКИ

rra.w R12
inc.b R15
mov.b LCD_Tab(R13),R12 ; Вывод старшей
                        ; тетрады во 2-ю цифру
                        ; ЖКИ

mov.b R12,0(R15) ; Вывод сегментов a & b
                  ; старшей тетрады в ЖКИ

rra.w R12
inc.b R15
mov.b R12,0(R15) ; Вывод сегментов c & d
                  ; старшей тетрады в ЖКИ

rra.w R12
inc.b R15
mov.b R12,0(R15) ; Вывод сегментов e & f
                  ; старшей тетрады в ЖКИ

rra.w R12
inc.b R15
mov.b R12,0(R15) ; Вывод сегментов g & h
                  ; старшей тетрады в ЖКИ

rra.w R12
ret

;*****
; Определение ЖКИ
;*****
;Определение сегментов
a      equ    001h
b      equ    010h
c      equ    002h
d      equ    020h
e      equ    004h
f      equ    040h

```

```

g      equ    008h
h      equ    080h
Blank  equ    000h

LCD_Tab db    a+b+c+d+e+f      ; Отображает "0"
        db    b+c              ; Отображает "1"
        db    a+b+d+e+g       ; Отображает "2"
        db    a+b+c+d+g       ; Отображает "3"
        db    b+c+f+g         ; Отображает "4"
        db    a+c+d+f+g       ; Отображает "5"
        db    a+c+d+e+f+g     ; Отображает "6"
        db    a+b+c           ; Отображает "7"
        db    a+b+c+d+e+f+g   ; Отображает "8"
        db    a+b+c+d+f+g     ; Отображает "9"
        db    a+b+c+e+f+g     ; Отображает "A"
        db    Blank           ; Пустой экран
        db    a+d+e+f         ; Отображает "C"
        db    b+c+d+e+g       ; Отображает "D"
        db    a+d+e+f+g       ; Отображает "E"
        db    a+e+f+g         ; Отображает "F"

;*****
Delay   ; Программная задержка
;*****
        push   #0FFFFh        ; Задержка → TOS
DL1     dec.w  0(SP)           ; Декремент TOS
        jnz    DL1            ; Задержка выполнена?
        incd   SP              ; Очистка TOS
        ret                    ; Возврат из подпро-
                                ; граммы

;*****
Math_calc ; Подпрограмма математических вычислений
;*****
        mov.w  #0h, DIGITS     ; Очистить DIGIT
        cmp.w  #0h, Result     ; Проверка результата
                                ; счёта Result на 0
        jeq    calc_over       ; Если 0 – на выход
        call   #Mul100          ; Result умножается на
                                ; 100
        call   #Divide          ; Потом делится на #06d

```

```

        call    #Hex2bcd          ; Преобразование
                                   ; 16-битного двоичного
                                   ; числа в двоично-де-
                                   ; сятичное (BCD)
                                   ; Result xx.xx
calc_over ret                    ; Возврат из подпро-
                                   ; граммы

;*****
Mul100  ; Подпрограмма умножения на 100
        ; входные данные Result 16 бит и константа
        ; 64h (100d) 16 бит на выходе 32 бит
        ; переменные htX100_msw & htX100_lsw
;*****
        mov.w  #100,IROP1 ; Загрузить 1-ый множитель
                                   ; 100 в IROP1
mpyu    clr.w  htX100_lsw ; Очистить буфер для младшей
                                   ; части слова
        clr.w  htX100_msw ; И для старшей
macu    clr.w  IROP2M     ; Очистить 2-й множитель
L$002   bit.w  #1,IROP1   ; Проверка младшего бита
        jz     L$01       ; Если 0, не делать ничего
        add.w  Result,htX100_lsw ; Если 1, добавить
                                   ; Result к произведе-
                                   ; нию
        addc.w IROP2M,htX100_msw ;
L$01    rla.w  Result     ; Произведение X 2
        rlc.w  IROP2M     ;
        rrc.w  IROP1      ; Проверка следующего бита
        jnz    L$002      ; Если в carry "1",
                                   ; то завершаем
        ret

;*****
Divide  ; Подпрограмма деления 32-битного числа на
        ; 16-битное
        ; входные данные 32-бит htX100_msw & htX100_
        ; lsw и 16-битная константа #06, на выходе
        ; 16-битная переменная DIGIT
;*****
        clr.w  DIGITS     ; Очистка буфера для нового
                                   ; результата
        mov.w  #17,IRBT   ; Организация цикла

```

```

div1    cmp.w #06,htX100_msw    ; Сравнить делитель
                                           ; со старшим словом
                                           ; делимого
        jlo    div2            ; Если меньше - переход
                                           ; на div2
        sub.w #06,htX100_msw    ; Вычесть 6 из старше-
                                           ; го слова
div2    rlc.w DIGITS            ; Сдвинуть результат влево
                                           ; через бит переноса
        jc     div4            ; Если в carry "1", то завер-
                                           ; шаем
        dec.w IRBT             ; Декремент счётчика цикла
        jz     div3            ; Если счётчик = 0:завершение
        rla.w htX100_lsw       ; Делимое X 2
        rlc.w htX100_msw       ;
        jnc    div1            ; Если в carry «0» переход
                                           ; к div1
        sub.w #06,htX100_msw    ; Вычесть 6 из старше-
                                           ; го слова
        setc                                ; Установить carry в «1»
        jmp    div2            ; Повтор
div3    clrc                    ; Очистить carry
div4    ret                    ; Возврат из подпрограммы
;*****
Hex2bcd ; Подпрограмма перевода 16-битного шестнадцате
; ричного значения
; DIGITS, в двоично-десятичное (BCD) входные
; данные 16-бит шестнадцатеричные в выходные
; там же в BCD
;*****
        mov    #16,r9          ; в R9 число бит
        clr    r8              ; Обнулить R8
        clr    r7              ; Обнулить R7
L$1     rla     DIGITS          ; Арифметический сдвиг DIGITS
                                           ; влево
        dadd   r7,r7           ; Десятичное сложение
        dadd   r8,r8           ; source + carry →
                                           ; destination
        dec    r9              ; Декремент счётчика бит
        jnz    L$1            ; Все 16 бит обработаны?
        mov    r7,DIGITS      ; Результат → DIGITS
        ret                    ; Возврат из подпрограммы

```

```

;*****
COMMON INTVEC ; Вектора прерываний MSP430x41x
;*****

        ORG     BASICTIMER_VECTOR
BT_VEC  DW      BT_ISR      ; Вектор базового таймера
        ; Basic Timer

        ORG     TIMERA1_VECTOR ; Вектор таймера
        ; Timer_A

TIMA_VEC DW      TAX_ISR    ;
        ORG     RESET_VECTOR
RESET_VEC DW     RESET      ; POR, внешний Reset,
        ; сторожевой таймер Watchdog
;*****
        END

```


IV. Примеры конкретных применений

Глава 3. *Декодирование ТВ ИК ДУ-сигналов используя Таймер А3*

Декодирование ТВ ИК ДУ-сигналов используя Таймер A3

Mark Buccini/Stefan Schauer [slaa134.pdf (Англ.)]

В данном примере применения описывается использование модуля таймера Timer_A3 для декодирования сигналов телевизионных инфракрасных (ИК) систем дистанционного управления стандартов RC5 и SIRC. Декодер, описанный в этом документе, формирует прерывание и работает в фоновом режиме, используя специфические характеристики модуля таймера Timer_A3. При этом используется только очень малая часть ресурсов вычислительного ядра микроконтроллера MSP430. Специфическая аппаратная возможность побитного защелкивания таймера Timer_A3 позволяет декодировать принятый ИК сигнал в реальном масштабе времени, независимо и асинхронно с вычислительным ядром. Активность вычислительного ядра и, соответственно, потребление при этом остаются на минимальном уровне. Способность модуля Timer_A3 проводить декодирование данных позволяет выполнять при необходимости одновременно и другие задачи. Описываемый в данном примере применения декодер ориентирован для реализации на основе микроконтроллеров MSP430x11x (1) и MSP430x12x, но также может быть легко реализован и на любом другом микроконтроллере семейства MSP430, содержащем модуль таймера Timer_A3.

1. Введение

Дополнительная возможность декодирования устройствами на основе микроконтроллеров семейства MSP430 сигналов телевизионных ИК систем дистанционного управления – наиболее дешевый способ реализации беспроводного ИК канала связи. Способность декодировать ИК сигналы может быть добавлена в любое устройство на базе микроконтроллера MSP430x11x (1), при этом используется только один его таймер захвата/сравнения Timer_A3 и 200 байт памяти программы, плюс необходим дополнительный дешевый внешний ИК приемник. На рисунке 1 можно увидеть демонстрационную схему ИК декодера. Для демонстрации работы декодера поступившие на линию P1.0 порта ввода-вывода последовательные пакеты данных передаются в персональный компьютер, а так же при принятии сигнала «увеличение канала» («channel +») загорается светодиод.

В RC5 и SIRC протоколах передача последовательных пакетов данных осуществляется путем модуляции ИК несущей сигналом с частотой 40 кГц. Логическая единица передается ИК импульсами с частотой 40 кГц, а отсутствие этих импульсов интерпретируется как передача нуля. Модуляция ИК сигнала

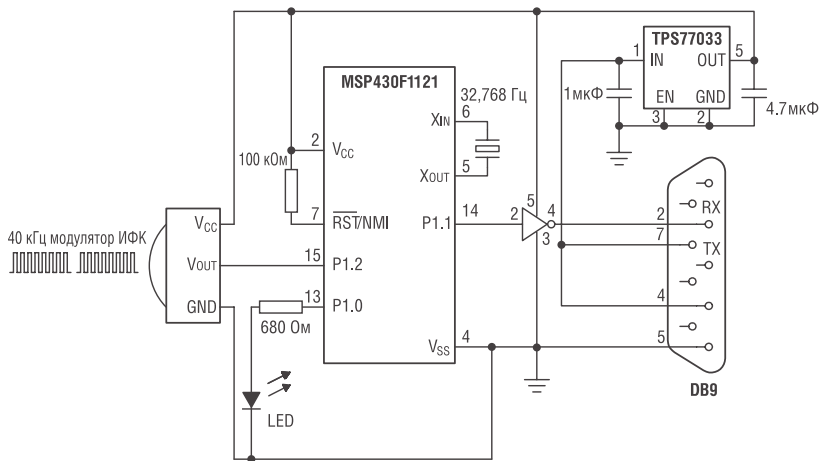


Рис. 1. Демонстрационная схема ИК декодера

частотой 40 кГц используется для обеспечения возможности фильтрации естественных шумов, возникающих при работе флуоресцентных ламп дневного света. Наиболее часто используется частота модуляции 40 кГц, но в некоторых других системах используется другая частота модуляции в диапазоне от 32 до 64 кГц. Для декодирования сигналов стандартов RC5 или SIRC сначала должна быть осуществлена демодуляция 40 кГц сигналов, а только потом – демодуляция информационного сигнала. В описываемом устройстве для преобразования ИК сигнала в электрический сигнал и усиления, фильтрации и демодулирования этого сигнала используется простейший трехвыводной 3 В ИК датчик TSOP1840SS3V производства компании VISHAY. На выходе этого ИК приемника принятый сигнал присутствует в виде логического сигнала, соответствующего переданному последовательному пакету данных. При отсутствии ИК сигнала с частотой модуляции 40 кГц на выходе ИК приемника присутствует высокий логический сигнал, а при наличии ИК сигнала с частотой модуляции 40 кГц на выходе присутствует сигнал с низким логическим выходом. Таким образом видно, что кроме демодуляции, ИК приемник осуществляет инверсию принятого сигнала. Сигнал с выхода ИК приемника подается непосредственно на вход порта P1.0 микроконтроллера MSP430x11x (1). Программно линия порта P1.2 настроена на работу в режиме захвата/сравнения таймера Timer_A3. Использование функции захвата/сравнения таймера Timer_A3 позволяет существенно упростить декодирование принятых данных. CCR1 выпол-

няет прием и передачу данных аппаратно без использования вычислительного ядра и других системных ресурсов операционной системы микроконтроллера. Для передачи принятых данных в персональный компьютер на базе регистра захвата/сравнения CCR0 реализован UART, который использует в качестве выхода линию порта P1.1.

2. Выбор рабочей частоты

Скорость передачи информационных пакетов в системах RC5 и SIRC относительно низкая (< 1 кбит / с) по сравнению с рабочей частотой MSP430. В демонстрационной схеме используется внешний кварцевый резонатор с частотой 32 768 Гц для формирования вспомогательных синхроимпульсов (ACLK) и тактирования модуля таймера Timer_A3. При такой частоте синхронизации Timer_A3 имеет разрешающую способность 30.5 мкс – более чем достаточная точность для декодирования сигналов стандартов RC5 или SIRC без ошибок. Встроенный задающий генератор с цифровой системой управления (DCO) вырабатывает основные синхроимпульсы (MCLK) с частотой порядка 1 МГц. Так как для ИК декодера на основе таймера Timer_A3 используется свой отдельный 32 768 Гц кварцевый резонатор, то быстродействие вычислительного ядра не является критическим параметром. Вычислительное ядро должно работать на такой частоте, которая позволит выполнять все требующиеся от него задачи. Использование медленного синхросигнала ACLK для ИК декодера и более высокочастотного DCO для тактирования вычислительного ядра позволяет совместить низкое потребление в дежурном режиме и высокое быстродействие при обработке пакетов. Если не синхронизировать MSP430 ЦП от низкочастотного часового кварцевого резонатора, то потребуются использовать более высокую рабочую частоту, что приведет к увеличению потребления. Вычислительное ядро должно работать на высокой частоте но в течение коротких интервалов времени, что и позволит найти оптимальное решение между быстродействием и потреблением.

3. Демонстрационная схема

Демонстрационная схема питается через 3.3 В стабилизатор с низким падением напряжения TPS76033 от последовательного порта персонального компьютера. Только один маломощный светодиод используется для индикации приема команды «увеличение канала» («channel +»). Передача данных от линии порта P1.1 осуществляется через инвертор SN74AHC1G04 производства компании TI. Если необходимо реализовать полный RS232 интерфейс, то можно использовать низкопотребляющий 3 В преобразователь уровней MAX3221. сразу после сброса запускается 32 768 Гц автогенератор, которо-

му не требуются даже внешние конденсаторы сдвига фазы, так как они уже встроены в MSP430.

4. Программа декодирования

Данный документ содержит два примера программы. Программа «11x1_rc5.s43» предназначена для декодирования сигналов RC5 протокола, а «11x1_sirc.s43» – для декодирования сигналов SIRC протокола. Основные части обеих программ небольшие и работают идентично. Только фоновое программное обеспечение ИК декодера уникально.

```

Mainloop      call    #IR_Ready    ; Вызов подпрограммы
                                   ; подготовки ИК деко-
                                   ; дера
                                   bis.w #LPM3, SR    ; Переход в режим
                                   ; LPM3, остановка,
                                   ; снижение потребления
                                   call    #TXIR_2_PC  ; Передатчик принял
                                   ; команду
                                   call    #LED_Displ ; Проверка на соот-
                                   ; ветствие коду кнопки
                                   ; «Channel +»
                                   jmp     Mainloop   ;

```

ИК декодер активизируется из основной части путем запуска подпрограммы IR_Ready. Затем в основной программе идет установка регистра состояния (SR), в результате чего микроконтроллер переходит в третий режим пониженного потребления (LPM3). В LPM3 режиме вычислительное ядро и DCO остановлены, но Timer_A3 работает от ACLK с полностью активной логикой прерываний регистра CCR1. Даже, несмотря на то, что система находится в LPM3 режиме, Timer_A3 декодера будет работать в фоновом режиме и вырабатывать управляющие прерывания. Архитектура MSP430 автоматически активизирует вычислительное ядро и DCO после обнаружения любого разрешенного прерывания. DCO запускается и выходит в рабочий режим менее, чем за 6 мкс. Это позволяет эффективно обрабатывать короткие пакетные события. Кроме того, после того, как произошло разрешенное прерывание, система автоматически сохраняет оригинальное состояние регистра SR в стеке и сбрасывает в нем биты активизации режимов пониженного потребления. После того, как программа обработки прерывания была обработана, команда reti (выход из прерывания) восстановит сохраненное значение регистра SR. Если это сохраненное в стеке значение не изменялось при выполнении подпрограммы обработки прерывания, то система вернется в то состояние, которое было до прерывания.

В данном примере применения после того, как оканчивается прием ИК пакета при помощи фоновой подпрограммы, прерывание от CCR1 возвращает вычислительное ядро в активный режим путем сброса битов активизации LPM3 режима в сохраненном значении регистра SR. Это очень удобный способ управления работой основной программы с обработкой событий по прерыванию. Полученный пакет ИК данных преобразуется в четыре двоичные кодовые комбинации в ASCII коде и передается в персональный компьютер по UART со скоростью 2400 бодов в формате 8N1 при помощи подпрограммы TXIR_2_PC. Четирем ASCII кодовым комбинациям предшествуют кодовые комбинации перевода корретки и перехода на новую строку. Активная стадия основной программы заканчивается вызовом подпрограммы LED_Disr. Если принята команда увеличения канала «channel +», то подпрограмма LED_Disr зажжет светодиод, подключенный к линии порта P1.0. После этого микроконтроллер переходит в режим LPM3, ожидая следующего пакета данных.

5. Протокол RC5

RC5 протокол основан на передаче данных при помощи манчестерского кода. Манчестерский код уникален тем, что данные передаются изменением состояния в середине бита. Логическая 1 интерпретируется MSP430 при переходе поступившего от ИК приемника инвертированного сигнала в середине бита из высокого состояния в низкое, а 0 – при переходе из низкого состояния в высокое. ИК пакет в стандарте RC5 состоит из 14 бит: два стартовых бита (S1, S0), один служебный бит (C), пять адресных бит (A4 – A0) и код команды из шести бит (C5 – C0). Пакет передается старшим битом вперед, т.е. первыми будут приняты стартовые биты. На рисунке 2 показан RC5 пакет на входе MSP430 после демодуляции и инверсии его ИК приемником. Стартовые биты всегда имеют

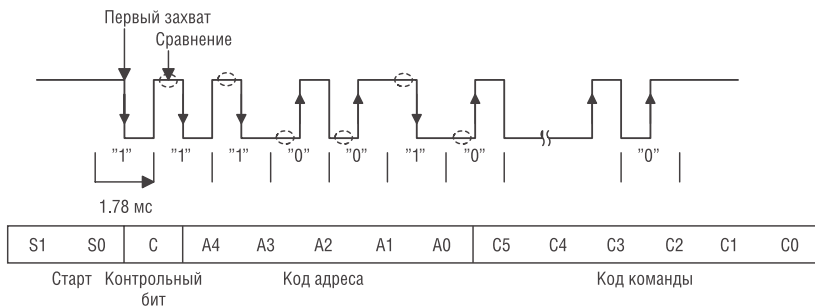


Рис. 2. Инвертированный пакет данных стандарта RC5, поступающий на вход MSP430

значение логической единицы. Контрольный бит изменяет свое значение при каждом нажатии кнопки на пульте дистанционного управления. Пять адресных бит предназначены для идентификации 32 различных типов устройств. Шесть командных бит предназначены для передачи 64 команд управления. Длительность бита в стандарте RC5 довольно высокая – 1.78 мс, причем половину этого периода передается высокий сигнал, а другую половину – низкий сигнал. Длительность всего 14 битного пакета данных равна приблизительно 25 мс.

Программа 11x1_rc5.243

Программа 11x1_RC5.s43 использует два регистра. Регистр IRData (R6) принимает ИК пакеты, а регистр IRBit (R7) используется как временный счетчик, отслеживающий биты ИК данных как они были получены. Выбор регистров R6 и R7 был сделан произвольно, вместо них могут использоваться любые другие регистры или ячейки памяти ОЗУ. В программе определены две константы: Bit_50 – константа, соответствующая половине длительности бита, и Bit_75 – константа, соответствующая трем четвертям длительности бита. Поскольку для формирования синхросигнала ACLK используется кварцевый резонатор с частотой 32 768 Гц, то константы BIT_50 и Bit_75 будут иметь следующие значения:

```
Bit_50 equ 29 ; 890 мкс при частоте ACLK  
; 32768 Гц  
Bit_75 equ 44 ; 1348 мкс при частоте ACLK  
; 32768 Гц
```

Подпрограмма IR_Ready позволяет CCR1 захватывать счетчик таймера Timer_A3 (TAR) и запрашивать прерывание по срезам сигнала на выходе ИК приемника. TAR захватывается по спадающему фронту и автоматически сохраняется в CCR1, при этом происходит формирование запроса на прерывание TA1_ISR. В Подпрограмма обработки прерывания TA1_ISR определяет было ли вызвано прерывание захватом или сравнением. Если прерывание было вызвано захватом, то BIT_75 (3/4 длины бита) добавляется непосредственно в CCR1, который хранит точное время спадающего фронта импульса на выходе ИК приемника в середине первого бита. Так как захват данных произошел в середине первого бита, то добавление 3/4 длительности бита приведет к смещению CCR1 в середину первой половины следующего бита. Теперь CCR1 настроен на работу в режиме сравнения данных. Следующее прерывание от CCR1 теперь установлено в середине первой половины второго стартового бита. В соответствии с настройками, в режиме сравнения CCR1 происходит сохранение присутствующего на входе P1.2 логического уровня в защелке

регистра захвата/сравнения (SCCI). SCCI обеспечивает очень важную аппаратную роль CCR1, позволяющую запоминать принятый логический уровень на входе P1.2 по синхросигналам, выработанным таймером Timer_A3 независимо от вычислительного ядра. В нашем примере применения вычислительное ядро практически бездействует, в то время как ИК декодер работает. Даже при отключенном вычислительном ядре логический уровень на входе P1.2 будет зафиксирован и сохранен в SCCI в моменты, когда CCR1 выполняет сравнение. Прикладная программа не считывает данные непосредственно со входа P1.2; вместо этого происходит считывание защелкнутых в SCCI данных. Программа постепенно возвратит полученные от SCCI данные и сохранит их в регистре памяти IRData.

```
RX_Cont bit.w *SCCI, *CCTL1      ; Carry = Data бит
                                   ; в SCCI
                                   rlc.w IRData      ; Carry → IRData
```

После возвращения каждого бита CCR1 перестраивается на захват и по нарастающему и по падающему фронту. Следующий захват данных произойдет точно в середине битового периода, что приведет к автоматической синхронизации с принимаемым потоком данных от ИК приемника. Для того чтобы гарантировать правильный прием данных, прерывание от CCR2 также разрешено и в него загружено значение половины длительности бита, сохраненного в CCR1. Если пакет принимается правильно, то следующее изменение принимаемого бита должно произойти через четверть длительности периода бита. При правильном приеме подпрограмма CCR1_ISR обработки прерывания от CCR1 производит сброс запроса на прерывание от CCR2. Если CCR1_ISR не отследит прерывание, то подпрограмма CCR2_ISR обработки прерывания от CCR2 сбросит декодер, предположив ошибку в принимаемом пакете. В нормальном режиме баланс отрицательных и положительных битов сохраняется, и принимаемые данные сохраняются в регистре IRData.

Протокол SIRC

В протоколе SIRC данные кодируются длительностью пакета. Длительность бита определяет его логическое значение. Стартовый бит имеет длительность 2.4 мс, логический ноль – 600 мкс, а логическая единица – 1.2 мс. Также все информационные биты, за исключением стартового бита, содержат 600 мкс синхроимпульс. Поэтому логический ноль передается за 1200 мкс, а логическая единица – за 1800 мкс. Полный пакет в протоколе SIRC состоит из стартового бита и 12 информационных бит. Эти 12 информационных бит состоят из семи-битного кода команды (C6 к C0) и пяти-битного кода устройства (D4 к D0). Пере-

дача данных в протоколе SIRC ведется младшим битом вперед. Первым после стартового бита принимается бит C0.

На рисунке 3 показан инвертированный ИК приемником пакет данных SIRC протокола на входе MSP430.

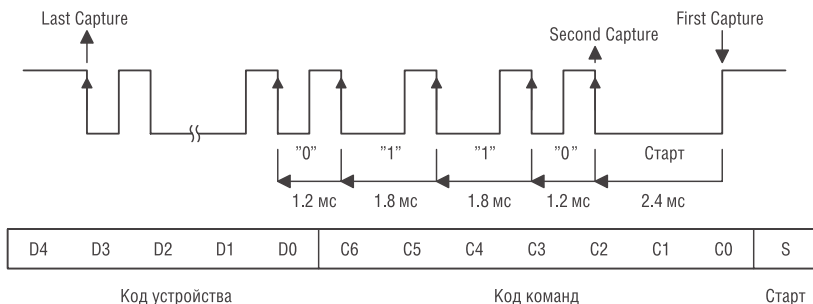


Рис. 3. Инвертированный пакет данных SIRC протокола на входе MSP430

Программа 11x1_SRC.s43

В программе 11x1_SRC.s43 задействовано три регистра: IRData (R6) принимает данные, IRBit (R7) используется для хранения данных в том виде, в котором они приняты, а IRLength (R8) используется для хранения длительности принятых информационных бит. Также определены три константы: IR_Mid – соответствует половине длительности бита относительно периода таймера Timer_A3, IR_Start – соответствует длительности приблизительно 2.3 мс (минимальная длительность правильного стартового бита), и IR_Start2 – соответствует приблизительно 2.5 мс (максимальная длительность правильного стартового бита).

```
IR_Mid equ 49 ; 1500 мкс при частоте ACLK 32768 Гц
IR_Start equ 75 ; 2300 мкс при частоте ACLK 32768 Гц
IR_Start2 equ 82 ; 2500 мкс при частоте ACLK 32768 Гц
```

Подпрограмма IR_Ready позволяет CCR1 захватывать по спадающему фронту данные от ИК приемника. Это позволяет MSP430 находиться в режиме пониженного потребления LPM3, при этом только ACLK и Timer_A3 остаются активными. По первому спадающему фронту, указывающему на начало стар-

тового бита, модуль CCR1 осуществляет захват содержимого TAR и запрашивает прерывание TA1_ISR. Состояние TAR захватывается аппаратно, вызова подпрограмм обработки прерывания при этом не требуется. После захвата CCR1 программно сохраняется в регистре IRLength. После первого спадающего фронта осуществляется изменение фронта захвата CCR1 на нарастающий фронт, что позволит отследить конец стартового бита. Длительность информационного бита вычисляется путем вычитания из содержимого регистра TAR, сохраненного в регистре CCR1, предыдущего значения IRLength. Если полученная длительность не лежит в пределах от 2.3 мс до 2.5 мс, то программа принимает решение о ложном захвате стартового бита. Правильный стартовый бит должен иметь длительность 2.4 мс. В подпрограмме обработки прерывания TA1_ISR IRBit используется для считывания в обратном порядке 12 информационных бит в том виде, в котором они получены. Каждый информационный бит рассчитывается путем сравнения его длительности с константой IR_Mid. Константа IR_Mid соответствует длительности 1500 мкс, которая является средним значением между длительностями бита информационной единицы и информационного нуля. Если длительность принятого бита больше IR_Mid, то принимается решение о том, что принята единица, а если меньше – то ноль. Аппаратная защелка модуля Timer_A3 гарантирует, что подпрограмма обработки прерывания считает правильное значение временного интервала. Программа не считывает непосредственно содержимое регистра модуля Timer_A3, а считывает захваченное в CCR1 значение таймера. Также для временного хранения содержимого CCR1 используется программный стек.

```

IR_ST_Test  push.w  &CCR1                ; Сохранение
                                           ; содержимого
                                           ; CCR1 в стеке
                sub.w  IRLength,0 (SP)      ; Последний
                                           ; захваченный
                                           ; временной
                                           ; интервал
IR_Bit      cmp.w    #IR_Mid,0 (SP)        ; C=1 если
                                           ; IR_RXed бит=1
IR_Shift    rrc.w    IRData                ; Carry → IRData

```

Аппаратная защелка CCR1 модуля таймера Timer_A3 позволяет одновременно с ИК декодером выполняться другим программам обработки событий в реальном времени. Подпрограмма ИК декодера выполняется в фоновом режиме. После проверки приема каждого бита данные помещаются в регистр IRData.

6. Программа UART

Функция UART реализована при помощи модуля захвата/сравнения CCR0 и использует два регистра: RXTXData (R4) и BitCnt (R5). Полное описание функционирования UART можно найти в других примерах применения. Отслеживание принятых данных при помощи персонального компьютера.

Для принятия пакетов данных, переданных описанным демонстрационным устройством, может быть использована любая терминальная программа. Содержимое 16-разрядного регистра IRData передается в формате с правым выравниванием в виде четырех ASCII символов. Программа терминала должна быть настроена на прием данных в формате 8N1 со скоростью 2400 бод.

Ссылки:

1. MSP430x11x1 Mixed Signal Microcontroller datasheet, Texas Instruments literature number SLAS241
2. MSP430x1xx Family Users Guide, Texas Instruments literature number SLAU049
3. Implementing a UART Function with Timer_A3, Texas Instruments literature number SLAA078
4. Photo Module for PCM Remote Control System, VISHAY literature number 82052

Приложение A: программа 11x1_SIRC.s43.txt

```
;*****
; ЭТА ПРОГРАММА ПОСТАВЛЯЕТСЯ В ВИДЕ «КАК ЕСТЬ».
; КОМПАНИЯ TI НЕ ДАЕТ НИКАКИХ ГАРАНТИЙ И НЕ НЕСЕТ
; НИКАКОЙ ОТВЕТСТВЕННОСТИ ЗА ВОЗМОЖНЫЕ НЕТОЧНОСТИ
; РАБОТЫ ИЛИ НЕПРАВИЛЬНОСТЬ РАБОТЫ ПРОГРАММЫ.
; ПРИ КОММЕРЧЕСКОМ ИСПОЛЬЗОВАНИИ ДАННОЙ ПРОГРАММЫ
; КОМПАНИЯ TI ОТКАЗЫВАЕТСЯ ОТ АВТОРСТВА И СНИМАЕТ
; С СЕБЯ ВСЯКУЮ ОТВЕТСТВЕННОСТЬ.
;
; КОМПАНИЯ TI ОТКАЗЫВАЕТСЯ ОТ ЛЮБЫХ ГАРАНТИЙ
; В ОТНОШЕНИИ ТОГО, ЧТО ПРОГРАММА НЕ МОЖЕТ ПРИНЕСТИ
; ПРЯМЫЕ ИЛИ КОСВЕННЫЕ УБЫТКИ ИЛИ ПОТЕРЮ ДОВЕРИЯ ИЛИ
; ПРИВЫЛИ ПРИ ИСПОЛЬЗОВАНИИ ЕЕ ТЕОРИИ ИЛИ АЛГОРИТМОВ.
; ОТВЕТСТВЕННОСТЬ ПРИ КОММЕРЧЕСКОМ ИСПОЛЬЗОВАНИИ
; ДАННОЙ ПРОГРАММЫ ПОЛНОСТЬЮ ЛЕЖИТ НА ЛИЦЕ,
; ИСПОЛЬЗОВАВШЕМ ДАННУЮ ПРОГРАММУ
;
; Если не указано иное, программа, созданная компанией
```

```

; Texas Instruments поставляется на условиях
; «свободно распространяемого программного обеспече-
; ния».
; Вы можете свободно изменять программу без всяких
; ограничений
; Передавать программу третьим лицам можно только при
; доведении до их сведений этих условий и при его
; согласии с этими условиями.
; Вы должны приводить лицензию в каждой копии или
; частичной копии.
;
; Вы признаете и соглашаетесь, что Программа содержит
; защищенный авторским правом материал, коммерческую
; тайну и другую частную информацию компании TI
; и защищена в соответствии с законами об авторском
; праве, международным соглашением об авторском праве
; и законом о коммерческой тайне, а так же законом
; об охране интеллектуальной собственности.
; Для защиты авторских прав компании TI
; Вы соглашаетесь не декомпилировать, перерабатывать,
; или преобразовывать программу к удобочитаемому виду.
; Вы соглашаетесь ни в коем случае не изменять
; и не удалять внесенную в программу информацию об
; авторском праве. Компания TI оставляет за собой все
; права, не оговоренные в данной лицензии. Кроме того,
; что оговорено в данной лицензии, не может рассматри-
; ваться при возможном судебном разбирательстве.
;
; Вы не можете использовать программу для микроконт-
; роллеров, которые не производит компания TI.
;

#include <msp430x11x1.h>
;*****
; MSP-FET430X110 Demo - SIRC декодер, передающий
; принятые команды ИК систем ДУ в ПК на скорости
; 2400 бод
;
; Описание: Декодирование 12-битных SIRC команд
; ИК систем ДУ при помощи таймера Timer_A. CCR1 тайме-
; ра Timer_A используется для декодирования ИК паке-
; тов, режим захвата используется для измерения
; длительности ИК бита.

```

```

; Принятые пакеты передаются в ПК при помощи модуля
; CCR0 таймера Timer_A, работающего в качестве UART.
; Пакеты передаются в виде 4 ASCII байтов, содержащих
; символы CR и LF. Если принята команда «channel+»,
; то на выводе P1.0 устанавливается высокий уровень,
; а если любая другая – то низкий.
; ИК данные принимаются младшим значащим битом вперед.
; Сначала передается старт-бит, а потом – 12 бит
; данных: D4-D3-D2-D1-D0-C6-C5-C4-C3-C2-C1-C0-Start
;
; Демонстрация с помощью ИК монитора – переданные
; ИК данные содержат символы CR, LF и 4 ASCII байта
;
;
;                                     MSP430F1121
;                                     -----
;                                     /|\  |                               XIN|-
;                                     |  |                               | 32 кГц
;                                     --|RST                             XOUT|-
;                                     |                               |
; IR Receiver -> |P1.2/CCR1   P1.0|--> LED0
;                | P1.1                               |--> 2400 8N1
;
; Длительности битов
; Start = 2.4 мс ~ 79 при ACLK = 32 кГц
; 1 = 1.2 мс низкий
; 2 = 0.6 мс низкий
; sync = 0.6 мс высокий
;
;
;               sync  snyc      snyc  snyc
;   +---+       +---+ +---+     +---+ +---+
;   |           | | | |         | | | |
;   |           | | | |         | | | |
;   +---+\\-----+  +---+  +-----+  +---+ +-----+
;               ^  0  ^  1      ^  0  ^  Start  ^
;
; Используемые регистры
#define RXTXData    R4
#define BitCnt      R5
#define IRData      R6
#define IRBit       R7
#define IRLength    R8
;

```

```

; Для того, чтобы скорость обмена данными по UART была
; 2400 бод необходимо, чтобы ACLK = 32768
Bitime_5      equ 06      ; .длина 5 бит + небольшая
                        ; подстройка.
Bitime        equ 014     ; при длине бита 427 мкс
                        ; скорость ~ 2341 бод
;
IR_Mid        equ 49      ; 1500 мкс при ACLK=32768 Гц
IR_Start     equ 75      ; 2300 мкс при ACLK=32768 Гц
IR_Start2    equ 82      ; 2500 мкс при ACLK=32768 Гц
;
LED0          equ 001h    ; LED0 на выводе P1.0
TXD           equ 002h    ; TXD подключается к выводу
                        ; P1.1
IRIN          equ 004h    ; Вход ИК данных - P1.2
Ch_up         equ 16      ;
Ch_dwn        equ 17      ;
LF            equ 0ah     ; Символ LF (перевод строки)
                        ; в ASCII коде
CR            equ 0dh     ; Символ CR (возврат каретки)
                        ; в ASCII коде
;

; Автор: М. Buccini
; Компания Texas Instruments, Inc
; Июль 2001
; *****
; -----
ORG 0F000h ; Начало программы
; -----
RESET      mov.w #0300h,SP ; Инициализация указателя
                        ; стека микроконтроллеров
                        ; серии 'x112x
          call #Init_Sys  ; Инициализация системных
                        ; периферийных устройств
;
Mainloop   call #IR_Ready ; Подготовка ИК декодера
          bis.w #LPM0,SR  ; Вход в режим LPMx, остано-
                        ; ка, снижение потребления
          call #TXIR_2_PC ; Передача принятой команды
          call #LED_Disb  ; Приняли команду
                        ; «Channel +/-»
          jmp Mainloop    ;

```

```

;-----
Init_Sys      ; Инициализация системных периферийных
              ; устройств
;-----
StopWDT mov.w #WDTPW+WDTHOLD,&WDTCTL ; Остановка
                                         ; сторожевого
                                         ; таймера
SetupTA mov.w #TASSEL0+MC1,&TACTL    ; ACLK,
                                         ; продолжение
SetupC0 mov.w #OUT,&CCTL0              ;
SetupP1 bis.b #IRIN+TXD,&P1SEL         ; P1.2 CCR1,
                                         ; P1.1 CCR0
                                         ; P1.0,
                                         ; TXD выходы
                                         ; P1.0, низкий,
                                         ; светодиод
                                         ; выключен
eint          ;
ret           ; Выход из
              ; подпрограммы
;-----
IR_Ready ; Подпрограмма подготовки к принятию
          ; 12-битных SIRC (последовательных ИК команд)
          ; в буфер IRData
;-----
          clr.w IRData                ;
          clr.w IRLength              ;
          mov.b #14,IRBit             ; Два начальных фронта
                                         ; и 12 информационных
                                         ; бит
SetupC1 mov.w #CM1+SCS+CAP+CCIE,&CCTL1 ; CAP CCI1A,
                                         ; спадающий
                                         ; фронт, прерывание
          ret                         ; Выход из
                                         ; подпрограммы
;-----
TXIR_2_PC ; Подпрограмма передачи в ПК символов CR, LF
          ; и данных IRData в виде четырех ASCII бай-
          ; тов R15 используется в качестве рабочего
          ; регистра, но его состояние не сохраняется
;-----

```

```

        mov    #CR,RXTXData      ; пересылка символа CR
                                   ; в буфер UART
        call   #TX_Byte          ; символ CR → ПК/поль-
                                   ; зователю
        mov    #LF,RXTXData      ; пересылка символа LF
                                   ; в буфер UART
        call   #TX_Byte          ; символ → ПК/пользо-
                                   ; вателю
                                   ;
TX_Word_ASCII ; Передача слова из регистра IRData
               ; в виде четырех ASCII байт
        swpb   IRData            ; IRData = 3412
        call   #TX_Byte_ASCII    ;
        swpb   IRData            ; IRData = 1234
                                   ;
TX_Byte_ASCII ; Передача байта из регистра IRData
               ; в виде двух ASCII байт
        mov.b  IRData,R15        ; передача значения
                                   ; ..x.
        call   #NUM_ASCIR        ;
        mov.b  IRData,R15        ; передача значения
                                   ; ...x
        jmp    NUM_ASCIA         ;
                                   ;
NUM_ASCIR rrc.b R15              ; путь 1. и 3.
        rrc.b  R15               ;
        rrc.b  R15               ;
        rrc.b  R15               ;
                                   ;
NUM_ASCIA and.b #0fh,R15         ; путь 2. и 4.
        add.b  #030h,R15         ;
        cmp.b  #03ah,R15         ;
        jlo    NUM_End           ;
        add.b  #039h,R15         ;
NUM_End mov.b  R15,RXTXData      ; загрузка буфера
                                   ; передачи
                                   ;
;-----
TX_Byte ; Подпрограмма передачи байта из буфера
        ; RXTXData при помощи CCR0 UART
;-----
        mov.w  &TAR,&CCR0        ; Текущее состояние
                                   ; счетчика TA

```



```

        add.w #Bitime,&CCR0      ; Временной интервал
                                   ; до первого бита
        bis.w #0100h, RXTXData  ; Добавление единично-
                                   ; го стопового бита в
                                   ; регистр RXTXData
        rla.w RXTXData          ; Добавление нулевого
                                   ; стартового бита
        mov.w #10, BitCnt       ; Загрузка счетчика
                                   ; битов, 8 бит дан-
                                   ; ных + SP
        mov.w #OUTMOD0+CCIE,&CCTL0 ; TXD = 1 = idle
TX_Wait tst.w BitCnt           ; Ожидание окончания
                                   ; передачи
        jnz    TX_Wait         ;
        ret                    ;
                                   ;
;-----
TA0_ISR ; Буфер RXTXData содержит данные UART
;-----
        add.w #Bitime,&CCR0      ; Время до следующего
                                   ; бита
UART_TX bic.w #OUTMOD2,&CCTL0    ; Принятие единицы
        rra.w RXTXData          ; LSB сдвигается в бит
                                   ; переноса
        jc     TX_Test          ; Переход → бит = 1
TX_Space bis.w #OUTMOD2,&CCTL0    ; Прием нуля
TX_Test dec.w BitCnt            ; Все биты переданы
                                   ; (приняты)?
        jnz    TX_Next         ; Следующий бит?
        bic.w #CCIE,&CCTL0      ; Все биты переданы/
                                   ; приняты, блокировка
                                   ; прерываний.
TX_Next reti                    ;
                                   ;
;-----
TAX_ISR ; Подпрограмма обработки прерываний
        ; от CCR1-4 и по переполнению
;-----
        add.w &TAIV, PC        ; Добавление к регистру
                                   ; ру таймера Timer_A
                                   ; вектора смещения
        reti                    ; CCR0 не является
                                   ; источником

```

```

; jmp TA1_ISR ; CCR1
; jmp TA2_ISR ; CCR2
; reti ; CCR3 не используется
; reti ; CCR4 не используется
;TA_over reti ; Переполнение
; TA - не используется
;
TA1_ISR mov.w #CM0+SCS+CAP+CCIE,&CCTL1 ; CAP CCI1A,
; спадающий фронт,
; прерывание
IR_ST_Test push.w &CCR1 ; Временно сохраняем
; в стеке содержимое
; CCR1
sub.w IRLength,0(SP) ; Время от последнего
; захвата
cmp.b #14,IRBit ; Первый спадающий
; фронт?
jeq IR_Next ; Переход → первый
; спадающий фронт
cmp.b #13,IRBit ; Стартовый бит?
jne IR_Bit ; Переход → не старто-
; вый бит
; cmp.w #IR_Start2,0(SP) ; Стартовый бит>2.5 мс
; jge IR_error ; Переход → IRLength >
; 2.5 мс
cmp.w #IR_Start,0(SP) ; Минимальный старто-
; вый бит ~ 2.3 мс
jge IR_Next ; Переход → IRLength >
; 2.3 мс
IR_error incd.w SP ; Очистка вершины
; стека
call #IR_Ready ; Ошибка - перезапуск
; принятой последова-
; тельности
reti ; Выход из прерывания
;
IR_Bit cmp.w #IR_Mid,0(SP) ; С=1 если принятый
; бит ИК данных = 1
IR_Shift rrc.w IRData ; Бит переноса →
; регистр IRData
IR_Next mov.w &CCR1,IRLength ; Сохранение захвачен-
; ного фронта

```

```

incd.w SP          ; Очистка верхушки стека
dec.b IRBit        ;
jnz IR_Cont        ; Переход → не последний бит
IR_Cont clr.w &CTL1  ; Отключение CCR1
rrc.w IRData        ; 12-битные IRData выровнены
                    ; по правому краю

rrc.w IRData        ;
rrc.w IRData        ;
rrc.w IRData        ;
and.w #0FFh,IRData  ; Изолирование 12-бит-
                    ; ного пакет
mov.w #GIE,0(SP)    ; Декодированный байт = ак-
                    ; тивному в основной
                    ; программе
IR_Cont reti        ;
                    ;

;-----
LED_Dispatch ; Включение светодиода (P1.0) если принятые
            ; ИК данные соответствуют коду команды
            ; «Channel+» (16)
;-----
and.w #07Fh,IRData  ; Изолирование 7-бит-
                    ; ного кода команды
LED_off bic.b #01h,&P1OUT ; Выключение светодиода
                    ; да
LED0_tst cmp.w #Ch_up,IRData ; Проверка на соот-
                    ; ветствие коду коман-
                    ; ды «Channel+» (32)
jne LED_exit        ;
bis.b #01h,&P1OUT    ; Включение светодиода
LED_exit ret        ; Выход из подпрограм-
                    ; мы
                    ;

;-----
; Используемые вектора прерываний
;-----
ORG 0FFFEh          ; Вектор сброса MSP430
DW RESET            ;
ORG 0FFF2h          ; Вектор прерывания
                    ; от таймера Timer_A0
DW TAO_ISR          ;

```

```

ORG      0FFF0h          ; Вектор прерываний
                               ; от таймеров Timer_AX
DW       TAX_ISR          ;
END

```

Приложение В: программа 11x1_RC5.s43.txt

```

; *****
; ЭТА ПРОГРАММА ПОСТАВЛЯЕТСЯ В ВИДЕ «КАК ЕСТЬ».
; КОМПАНИЯ TI НЕ ДАЕТ НИКАКИХ ГАРАНТИЙ И НЕ НЕСЕТ
; НИКАКОЙ ОТВЕТСТВЕННОСТИ ЗА ВОЗМОЖНЫЕ НЕТОЧНОСТИ
; РАБОТЫ ИЛИ НЕПРАВИЛЬНОСТЬ РАБОТЫ ПРОГРАММЫ.
; ПРИ КОММЕРЧЕСКОМ ИСПОЛЬЗОВАНИИ ДАННОЙ ПРОГРАММЫ
; КОМПАНИЯ TI ОТКАЗЫВАЕТСЯ ОТ АВТОРСТВА И СНИМАЕТ
; С СЕБЯ ВСЯКУЮ ОТВЕТСТВЕННОСТЬ.
;
; КОМПАНИЯ TI ОТКАЗЫВАЕТСЯ ОТ ЛЮБЫХ ГАРАНТИЙ В ОТНОШЕ-
; НИ ТОГО, ЧТО ПРОГРАММА НЕ МОЖЕТ ПРИНИСТИ ПРЯМЫЕ ИЛИ
; КОСВЕННЫЕ УБЫТКИ ИЛИ ПОТЕРЮ ДОВЕРИЯ ИЛИ ПРИВЫЛИ ПРИ
; ИСПОЛЬЗОВАНИИ ЕЕ ТЕОРИИ ИЛИ АЛГОРИТМОВ.
; ОТВЕТСТВЕННОСТЬ ПРИ КОММЕРЧЕСКОМ ИСПОЛЬЗОВАНИИ
; ДАННОЙ ПРОГРАММЫ ПОЛНОСТЬЮ ЛЕЖИТ НА ЛИЦЕ, ИСПОЛЬЗО-
; ВАВШЕМ ДАННУЮ ПРОГРАММУ
;
; Если не указано иное, программа, созданная компанией
; Texas Instruments поставляется на условиях «свободно
; распространяемого программного обеспечения».
; Вы можете свободно изменять программу без всяких
; ограничений
; Передавать программу третьим лицам можно только при
; доведении до их сведений этих условий и при его
; согласии с этими условиями.
; Вы должны приводить лицензию в каждой копии или
; частичной копии.
;
; Вы признаете и соглашаетесь, что Программа содержит
; защищенный авторским правом материал, коммерческую
; тайну и другую частную информацию компании TI
; и защищена в соответствии с законами об авторском
; праве, международным соглашением об авторском праве
; и законом о коммерческой тайне, а так же законом

```

; об охране интеллектуальной собственности. Для защиты;
 ; авторских прав компании TI Вы соглашаетесь не
 ; декомпилировать, перерабатывать, или преобразовывать
 ; программу к удобочитаемому виду. Вы соглашаетесь
 ; ни в коем случае не изменять и не удалять внесенную
 ; в программу информацию об авторском праве. Компания
 ; TI оставляет за собой все права, не оговоренные
 ; в данной лицензии. Кроме того, что оговорено в дан-
 ; ной лицензии, не может рассматриваться при возможном
 ; судебном разбирательстве.

;
 ; Вы не можете использовать программу для микроконт-
 ; роллеров, которые не производит компания TI.
 ;

```
#include <msp430x14x.h>
```

```
;  

  ; *****
```

```
;  

  ; MSP-FET430X110 Demo – Декодер команд ИК системы ДУ  

  ; RC5, передающий их в ПК на скорости 2400 бод
```

```
;  

  ;
```

```
;  

  ; Описание: Декодирование 12-битных ИК пакетов  

  ; бифазного формата RC5 при помощи модуля таймера  

  ; Timer_A.
```

```
;  

  ; CCR1 модуля таймера Timer_A используется для деко-  

  ; дирования ИК пакетов, режим захвата – для детектиро-  

  ; вания межбитного фронта, а режим сравнения – для  

  ; защелкивания бита данных. CCR2 используется для  

  ; декодирования перезапуска. Принятый пакет передается  

  ; в ПК при помощи CCR0 модуля Timer_A, выполняющего  

  ; функцию UART. Пакет передается в виде четырех ASCII  

  ; байтов, содержащих символы CR и LF.
```

```
;  

  ; Если принята команда «Chanel+», то на выходе P1.0  

  ; устанавливается высокий уровень, а если любая  

  ; другая – то низкий.
```

```
;  

  ; ИК данные передаются начиная со старшего значащего  

  ; бита. Пакет состоит из 2-ух стартовых бит, бита C  

  ; и 11 бит данных: S1-S2-C-A4-A3-A2-A1-A0-C5-C4-C3-C2-  

  ; C1-C0
```

```
;  

  ;
```

```
;  

  ;
```

```
;  

  ; Демонстрируется работа при помощи ИК монитора,  

  ; в который передаются данные символы CR, LF и 4 ASCII  

  ; байта
```

```

;                                     MSP430F1121
;                                     -----
;                                     /\ \ | XIN | -
;                                     |   |   | 32 кГц
;                                     --| RST   XOUT | -
;                                     |   |   |
;      IR Receiver->| P1.2/CCR1  P1.0 |--> Светодиод
;                                     |   |   | P1.1 |--> 2400 8N1
; Длительности битов
;
; 1.78 мс
; +--- +--- +--- +--- +--- +---
; |   |   |   |   |   |   |   |   |   |
; ---+ ---+ +---+--- +---+ +-----+ +---
; ^Старт^Старт^ 1 ^ 0 ^ 0 ^
;
; Используемые регистры вычислительного ядра
#define RXTXData    R4
#define BitCnt      R5
#define IRData      R6
#define IRBit       R7
; Для того, чтобы скорость обмена данными по UART
; равнялась 2400 бод необходимо, чтобы ACLK = 32768 Гц
Bitime_5 equ 06 ; Длительность 5 бит + небольшая
; подстройка
Bitime equ 014 ; длительность бита 427 мк ~ 2341
; бод
;
LED0 equ 001h ; Нулевой светодиод подключен
; к порту P1.0
TXD equ 002h ; Выход передатчика UART подключен
; к порту P1.1
IRIN equ 004h ; ИК данные поступают на порт P1.2
Bit_50 equ 29 ; 890 мкс при ACLK = 32768 Гц
Bit_75 equ 44 ; 1348 мкс при ACLK = 32768 Гц
Ch_up equ 32 ;
Ch_dwn equ 33 ;
LF equ 0ah ; ASCII код символа перевода строки
; (LF)
CR equ 0dh ; ASCII код символа возврата
; каретки (CR)
;

```

```

; Автор: M. Buccini
; Компания Texas Instruments, Inc
; июль 2001
;*****
;-----
                ORG    0F000h    ; Начало программы
;-----
RESET    mov.w    #0300h,SP    ; Инициализация указателя
                                ; стека микроконтроллера
                                ; семейства'x112x
                call    #Init_Sys    ; Инициализация системных
                                ; периферийных модулей
                                ;
Mainloop call    #IR_Ready    ; Подготовка ИК декодера
                bis.w    #LPM3,SR    ; Переход в режим LPMx,
                                ; остановка, снижение
                                ; потребления
                call    #TXIR_2_PC    ; Передача принятой команды
                call    #LED_Disb    ; Проверка, являются ли
                                ; принятые данные командой
                                ; «Channel +» или нет
                jmp     Mainloop    ;
;-----
Init_Sys    ; Инициализация системных периферийных
            ; устройств
;-----
StopWDT mov.w    #WDTPW+WDTHOLD,&WDTCTL    ; Остановка
                                ; сторожевого
                                ; таймера
SetupTA mov.w    #TASSEL0+MC1,&TACTL    ; ACLK непрерывны
SetupC0 mov.w    #OUT,&CCTL0    ; Установка на
                                ; выходе пере-
                                ; датчика единицы
SetupP1 bis.b    #IRIN+TXD,&P1SEL    ; P1.2 - CCR1,
                                ; P1.1 - CCR0
                bis.b    #LED0+TXD,&P1DIR    ; P1.0 - выход
                                ; передатчика
                bic.b    #LED0,&P1OUT    ; на P1.0 низкий
                                ; уровень - све-
                                ; тодиод выключен
                eint    ;
                ret     ; Выход из под-
                                ; программы

```

```

;-----
IR_Ready ; Подпрограмма подготовки к принятию
          ; 12-битных данных в формате RC5 в регистр
          ; IRData
;-----

      clr.w IRData      ;
      mov.w #014,IRBit ; 12 бит данных + 1 стартовый
                        ; бит + 1 окончательный
SetupC1 mov.w #CM1+SCS+CAP+CCIE,&CCTL1
                        ; CAP CCI1A, спадающий фронт,
                        ; прерывание
      ret              ;
;
;-----
TXIR_2_PC ; Подпрограмма передачи символов CR, LF
          ; и содержимого регистра IRData в виде четы-
          ; рех ASCII байт в ПК R15 используется
          ; в качестве рабочего регистра и не сохраня-
          ; ет своего состояния
;-----

      mov    #CR,RXTXData ; символ CR пересыла-
                        ; ется в буфер UART
      call   #TX_Byte     ; CR → ПК/пользователю
      mov    #LF,RXTXData ; символ LF пересыла-
                        ; ется в буфер UART
      call   #TX_Byte     ; LF → ПК/пользователю
;
TX_Word_ASCII ; передача слова из IRData в виде
              ; четырех ASCII байт
      swpb   IRData       ; IRData = 3412
      call   #TX_Byte_ASCII ;
      swpb   IRData       ; IRData = 1234
;
TX_Byte_ASCII ; Передача байта из IRData в виде двух
              ; ASCII байт
      mov.b  IRData,R15   ; передача значения
                        ; ..x
      call   #NUM_ASCIR   ;
      mov.b  IRData,R15   ; передача значения
                        ; ..x
      jmp    NUM_ASCIA    ;
;

```



```

NUM_ASCIR    rrc.b      R15    ; 1. и 3. пропускаются
              rrc.b      R15    ;
              rrc.b      R15    ;
              rrc.b      R15    ;
;
NUM_ASCIA    and.b      #0fh,R15    ; 2. и 4.
              ; пропускаются
              add.b      #030h,R15   ;
              cmp.b      #03ah,R15   ;
              jlo        NUM_End     ;
              add.b      #039,R15    ;
NUM_End      mov.b      R15,RXTXData ; загрузка
              ; буфера пере-
              ; датчика
;
;-----
TX_Byte ; Подпрограмма передачи байта из буфера
        ; RXTXData при помощи CCR0 UART
;-----
        mov.w    &TAR,&CCR0          ; Текущее состояние
              ; счетчика ТА
        add.w    #Bitime,&CCR0        ; Некоторая пауза
              ; перед первым битом
        bis.w    #0100h, RXTXData    ; Добавление единично-
              ; го стопового бита в
              ; RXTXData
        rla.w    RXTXData            ; Добавление нулевого
              ; стартового бита
        mov.w    #10,BitCnt          ; Загрузка счетчика
              ; битов, 8 бит
              ; данных + SP
        mov.w    #OUTMOD0+CCIE,&CCTL0 ; TXD = 1 = idle
TX_Wait    tst.w    BitCnt            ; Ожидание окончания
              ; передачи
        jnz      TX_Wait             ;
        ret                                ;
;
;-----
TA0_ISR ; Буфер RXTXData содержит данные для UART.
;-----
        add.w    #Bitime,&CCR0        ; Интервал до следую-
              ; щего бита

```

```

UART_TX  bic.w  #OUTMOD2,&CCTL0 ; Передача единицы
          rra.w  RXTXData        ; МЗБ (LSB) сдвигается
                                   ; в бит переноса
          jc     TX_Test         ; Переход --> бит = 1
TX_Space  bis.w  #OUTMOD2,&CCTL0 ; Передача нуля
TX_Test   dec.w  BitCnt          ; Все биты переданы
                                   ; (или приняты)?
          jnz    TX_Next        ; Следующий бит?
          bic.w  #CCIE,&CCTL0    ; Все биты переданы/
                                   ; приняты, блокировка
                                   ; прерываний
TX_Next   reti                  ;
;-----
TAX_ISR   ; Общая подпрограмма обработки прерываний
          ; CCR1-4 и переполнения
;-----
          add.w  &TAIV,PC        ; Добавление смещения
                                   ; в вектор прерывания
                                   ; от Timer_A
          reti                  ; CCR0 - не является
                                   ; источником
          jmp    TA1_ISR         ; CCR1
          jmp    TA2_ISR         ; CCR2
;          reti                  ; CCR3
;          reti                  ; CCR4
;TA_over  reti                  ; Выход по переполне-
                                   ; нию
;
TA1_ISR   bit.w  #CAP,&CCTL1      ;
          jc     RX_edge         ; Переход → захвачен
                                   ; фронт
;
RX_Bit    dec.w  IRBit           ;
          jz     RX_Comp         ; Проверка окончания
                                   ; пакета
RX_Cont    bit.w  #SCCI,&CCTL1    ; Бит переноса = бит
                                   ; данных в SCCI
          rlc.w  IRData          ; Бит переноса →
                                   ; IRData
          mov.w  #CM1+CM0+CAP+CCIE+SCS,&CCTL1

```

```

; CAP CCI1A, оба фрон-
; та, прерывание
push.w &CCR1 ; Максимальное время
; до следующего фронта
add.w #Bit_50,0(SP) ;
pop.w &CCR2 ;
mov.w #CCIE,&CCTL2 ; Разрешение прерыва-
; ния по таймауту
reti ;
;
RX_Comp clr.w &CCTL1 ; Отключение CCR1
and.w #0FFFh,IRData ; Изолирование
; 12-битного пакета
mov.w #GIE,0(SP) ;
reti ;
;
RX_edge clr.w &CCTL2 ; Отключение таймаута
; CCR2
mov.w #CCIE,&CCTL1 ; режим сравнения
add.w #Bit_75,&CCR1 ; Время до сеердины
; бита данных
reti ;
;
TA2_ISR clr.w &CCTL2 ; Блокировка таймаута
; CCR2
call #IR_Ready ; Ошибка - перезапуск
; приемника
reti ; Выход из подпрограм-
; мы
;-----
LED_Displ ; Включение светодиода (установка P1.0)
; при принятии команды «Channel+» (код 32)
;-----
and.w #03Fh,IRData ; Изолирование
; 6-битного кода
; команды
LED_off bic.b #01h,&P1OUT ; Выключение
; светодиода
LED0_tst cmp.w #Ch_up,IRData ; Принята команда
; «Channel+» (код 32)

```

```

        jne     LED_exit      ;
        bis.b   #01h,&P1OUT   ; Включение светодиода
LED_exit ret                 ; Выход из подпрограм-
                               ; мы
;-----
; Используемые вектора прерываний
;-----
        ORG     0FFFEh       ; Вектор сброса MSP430
        DW      RESET        ;
        ORG     0FFF2h       ; Вектор таймера Timer_A0
        DW      TAO_ISR      ;
        ORG     0FFF0h       ; Вектор таймеров Timer_AX
        DW      TAX_ISR      ;
        END

```

IV. Примеры конкретных применений

Глава 4.

Компенсация фазового сдвига и калибровка токового трансформатора

Компенсация фазового сдвига и калибровка токового трансформатора

Kes Tam [slaa122.pdf (Англ.)]

В этом примере применения описывается цифровая технология компенсации фазового сдвига и калибровки трансформаторов тока и напряжения, используемых для измерения электрической мощности и энергии. Традиционная аналоговая компенсационная схема заменяется цифровым корректирующим фильтром с конечной импульсной характеристикой (КИХ). Технология эмуляции нагрузки с неединичным фактором мощности (non-UPF – несогласованной нагрузки) позволяет сделать калибровку полностью автоматической. Время калибровки при этом существенно снижается, а точность и повторяемость повышаются. Использование эмуляции позволяет исключить дорогостоящую non-UPF нагрузку. 16-разрядный смешанно-сигнальный RISC процессор MSP430 производства компании Texas Instruments является эффективным решением для выполнения вычислений, необходимых для проведения компенсации и калибровки.

1. Введение

Для измерения тока и обеспечения развязки по постоянному току в электросчетчиках обычно используются трансформаторы тока (СТ). Однако, все СТ вносят фазовый сдвиг, который изменяет величину мощности на их выходе. Индуктивные и емкостные нагрузки вызывают взвешенную ошибку измерения мощности переменного тока, которая значительно увеличивается и достигает недопустимой величины при снижении измеряемой мощности.

Традиционным решением преодоления проблемы фазового сдвига является применение аналогового RC фильтра; однако, RC фильтры не способны удовлетворить современным требованиям, из-за высокой стоимости, возникновения проблем с устойчивостью и необходимости проведения трудоемкой ручной калибровки. Применение цифровой коррекции и автоматизированной калибровки позволяет избавиться от этих недостатков.

В этом примере применения рассматриваются два вопроса: коррекция и калибровка. Раздел по коррекции описывает разработку одиночного фильтра с нулевой конечной импульсной характеристикой (zero-FIR – нулевой КИХ). Этот фильтр имеет групповое время задержки, позволяющее компенсировать фазовый сдвиг токового трансформатора. К сожалению, он также изменяет коэффициент передачи по постоянному току, коррекция которого также должна быть произведена впоследствии.

В разделе по калибровке описывается расчет параметров фильтра с КИХ для различных времен задержки, затем рассматриваются вопросы эмуляции поп-UPF нагрузки, определение ошибки поп-UPF и локализация используемых актуальных параметров КИХ фильтров.

2. Цифровая коррекция фазового сдвига

2.1. Описание задачи

Из-за наличия индуктивной и емкостной составляющих нагрузки, переменные напряжение и ток в сети имеют расхождения по фазе, таким образом, при использовании токового трансформатора измеряемый ток опережает (отстает) по фазе от реального тока.

На рисунке 1 входной ток $I_{(in)}$ опережает напряжение V на угол θ , а выходной ток $I_{(out)}$ опережает напряжение на угол ϕ . Реальная мощность в этом случае равняется

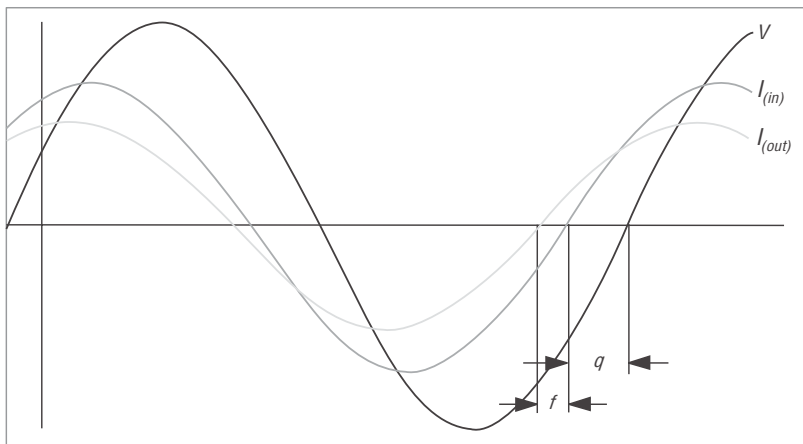


Рис. 1. Фазовая характеристика СТ

$$P = V_0 I_0 \cos \theta$$

где V_0 – амплитуда переменного напряжения, I_0 – амплитуда переменного тока, $\cos \theta$ – фактор мощности. Измеренная мощность будет равняться

$$P' = K_{(v)} V'_0 K_{(i)} I'_0 \cos(\theta + \phi)$$

где $K_{(v)}$ – коэффициент затухания напряжения ($V_0 = K_{(v)} V'_0$), $K_{(i)}$ – коэффициент трансформации тока ($I_0 = K_{(i)} I'_0$), а $\cos(\theta + \phi)$ – измеренный фактор мощности. Таким образом, ошибка измерения будет равняться

$$E_{(m)} = 1 - \frac{P'}{P} = 1 - \frac{\cos(\theta + \phi)}{\cos \theta}$$

Ошибка является нелинейной функцией от фактора мощности. Для UPF (согласованной) нагрузки величина угла ϕ обычно невелика и ошибкой можно пренебречь, однако с уменьшением фактора мощности, ошибка становится уже заметной. Например, при величине фактора мощности, равной 0.5, и фазовом сдвиге, равном 1° , ошибка составляет 3 %.

2.2. FIR фильтр

Для решения проблемы фазового сдвига в СТ, необходимо обеспечить такую задержку выходного тока, чтобы величины реального и измеренного факторов мощности были равны. Однако, эта задержка вероятнее всего не будет кратна периоду дискретизации, что не позволит решить эту задачу простой задержкой на один или несколько тактов.

Самый простой способ обеспечения задержки на дробное количество тактов является использования фильтра с нулевой КИХ:

$$y[n] = x[n] + \beta x[n-1]$$

Здесь x – входной ток, y – задержка выходного тока, n – дискретная переменная, а β – коэффициент усиления задержки.

Частотная характеристика может быть получена в результате z-преобразования характеристики задержки. Функция D является производной фазовой характеристики:

$$D = \frac{\beta(\beta + \cos \omega)}{1 + \beta^2 + 2\beta \cos \omega}$$

где ω – круговая частота после дискретизации, эквивалентная значению $2\pi x$ (основная частота/частота дискретизации).

Решением уравнения (5) относительно β является следующее выражение:

$$\beta = -\frac{(1-2D)\cos \omega \pm \sqrt{(1-2D)^2 \cos^2 \omega + 4D(1-D)}}{2(1-D)}$$

Фазовый сдвиг является проекцией группового времени задержки на угловую плоскость. Они связаны следующим выражением:

$$D = \frac{\phi}{\omega}$$

где фазовый сдвиг ϕ измеряется в радианах. Для компенсации неединичного коэффициента усиления фильтра выходной сигнал умножается на величину, обратно пропорциональную модулю коэффициента передачи:

$$A^{-1} = [(\cos \omega + \beta)^2 + \sin^2 \omega]^{-1/2}$$

Например, для электросчетчика, работающего с частотой дискретизации 995.025 Гц, измеряющего входной сигнал с частотой 50 Гц и содержащего СТ, вносящий фазовый сдвиг 1° , $\beta=0.0611975$, а $A^{-1}=0.944873$.

3. Калибровка СТ

Сначала должна быть выполнена калибровка линейной части, так как она влияет на калибровку СТ. С другой стороны, фазовый сдвиг СТ не имеет почти никакого влияния на калибровку линейной части.

Фактически калибровка СТ начинается с формирования внутренней задержки основного входного тока с целью эмуляции поп-UPF нагрузки. Это основная технология, используемая во всем процессе калибровки. Небольшой фазовый сдвиг СТ создает относительно большую ошибку в этом случае. Ошибки для диапазона времен задержки и параметров токовых трансформаторов для соответствующих корректирующих КИХ рассчитаны и сведены в таблицы. Ошибка, возникающая при эмуляции поп-UPF нагрузки измеряется эталонным измерителем. После этого истинные коэффициенты КИХ могут быть найдены по таблице.

3.1. Эмуляция поп-UPF нагрузки

Поскольку фазовый сдвиг СТ обычно небольшой, то из уравнения (3) видно, что ошибка тоже будет небольшой при UPF нагрузке. Поэтому для обеспечения большой ошибки при точной калибровке потребовались бы очень большой конденсатор или индуктивность.

Использование внутренней задержки для эмуляции поп-UPF нагрузки не только устраняет необходимость использования таких дорогих и больших нагрузок, но это также позволяет производить автоматическую калибровку при помощи только одной UPF нагрузки.

Задержка входного тока или напряжения эквивалентна изменению фактора мощности, что позволяет эмулировать поп-UPF нагрузку. Нет никаких строгих требований к величине эмулируемой поп-UPF нагрузке, требуется только чтобы она была достаточно большая для обеспечения высокой точности калибровки (но не слишком большой, чтобы дать небольшой фактор мощности). Для простоты эмуляции время задержки может быть выбрано кратным периоду дискретизации. Задержка по фазе ξ :

$$\xi = \frac{f_{(m)}}{f_{(s)}} \times N$$

где $f(m)$ – частота основного сигнала, $f(s)$ – частота дискретизации, а N – количество циклов задержки.

Фактор $\cos\xi$ – фактор эмулируемой мощности. Например, при частоте основного сигнала 50 Гц и частоте дискретизации 995.025 Гц задержка на три такта эквивалентна фазовому сдвигу на 54.27° , что вполне достаточно для калибровки.

3.2. Вычисление ошибки

Есть множество способов получить эталонный входной сигнал от внешнего эталонного измерителя. Метод, используемый в нашем случае очень прост: эталонный измерительный прибор подает на светодиод импульсы, частота которых пропорциональна измеряемой энергии. Например, производительность 1600 импульсов/час для каждого киловатт-часа означает, что 1600 импульсов будут сформированы за час при работе на нагрузку, потребляющую один киловатт. Для калибровки измерителя может использоваться простейший оптический преобразователь. Этот метод обеспечивает очень простой интерфейс и требует только одной входной линии. Предполагается, что измеритель при калибровке формирует сигналы для управления светодиодом тем же методом, что и эталонный измеритель.

Ошибка, вызванная фазовым сдвигом СТ, получается в результате нормализации разницы времен, необходимых обоим приборам для формирования одинакового количества импульсов. Мы получаем импульсно-временную ошибку $E(p)$, которая не является ошибкой измерения энергии или мощности, определяемой по формуле (3):

$$E_{(p)} = (t'_{(int)} - t_{(ref)}) / t_{(ref)}$$

где $t'_{(int)}$ – приведенная длительность импульсной последовательности, равная $t_{(int)}/\cos\xi$, $t_{(int)}$ – измеряемая длительность импульсной последовательности, а $t_{(ref)}$ – длительность импульсной последовательности эталонного измерителя. Обратите внимание, что для внешнего эталонного измерителя не формируется внутренняя задержка, поэтому необходимо преобразовать внутренний временной интервал в соответствии с условиями согласования, разделив его на эмулируемый фактор мощности.

3.3. Установка таблицы параметров КИХ

Параметры КИХ для данного фазового сдвига могут быть рассчитаны используя уравнения (6) – (8). Вычисления лучше всего делать в электронной таблице, что позволит полностью определить таблицу предварительных расчетов. Для определения максимального и минимального фазового сдвига должен быть известен допуск фазового сдвига СТ. Шаг изменения фазового сдвига выбирается таким, чтобы неточность определения ошибки оставалась в допустимых пределах. Например, если размер шага выбран 0.1° , а минимальный фактор мощности – $\cos(60^\circ)=0.5$, то максимальная ошибка составит $1 - [\cos(60^\circ+0.1^\circ)/\cos(60^\circ)]=0.3\%$.

После преобразования уравнения (10) можно получить следующее выражение:

$$E_{(p)} = \frac{\cos \xi}{\cos(\xi - \phi)} - 1$$

С целью минимизации размера прикладной программы и времени поиска для $E(p)$ может использоваться неявная таблица. Ошибки $E(\max)$ и $E(\min)$, соответствующие максимальному и минимальному фазовому сдвигу должны быть рассчитаны первыми. Тогда размер шага фазового сдвига может быть преобразован в размер шага ошибки:

$$E(\text{step}) = (E(\max) - E(\min)) / (\text{общее количество шагов}).$$

Для каждого значения $E(p)$ фактический фазовый сдвиг рассчитывается по формуле (10), а результат используется в уравнениях (6) – (8) для вычисления окончательных параметров КИХ. Поскольку $E(p)$ может быть легко рассчитано по известным $E(\min)$ и $E(\text{step})$, то нет необходимости его сохранять – следует хранить только значения $E(\min)$, $E(\max)$ и $E(\text{step})$.

3.4. СТ с большим фазовым сдвигом

В некоторых случаях СТ могут иметь большой фазовый сдвиг. Например, высокие промышленные частоты или низкая частота дискретизации приведут к увеличению фазового сдвига даже у токовых трансформаторов, имеющих небольшой собственный фазовый сдвиг. Это вытекает из выражения (7).

СТ с большим фазовым сдвигом можно также компенсировать и калибровать если среднее значение фазового сдвига находится в приемлемом диапазоне отклонения. СТ этого типа сначала компенсируются при помощи фильтров с КИХ, рассчитанной для среднего фазового сдвига h , откалиброванного при UPF нагрузке. Если h больше периода дискретизации, то необходимо использовать задержки, сформированные в результате умножения и деления периода дискретизации при помощи КИХ. В этом случае:

$$\eta = N_{(d)} \psi + \zeta$$

где ψ – фазовая задержка одного периода дискретизации, равная отношению частоты основного сигнала к частоте дискретизации, а $N(d)$ – количество задерживаемых выборок с $\zeta < \psi$. Тогда величина фазового сдвига токового трансформатора будет равняться:

$$\phi' = \phi - \eta$$

где ϕ – собственный фазовый сдвиг СТ. Так как ϕ' очень мало, то оно может игнорироваться при калибровке с UPF нагрузкой.

После калибровки с согласованной нагрузкой выполняется калибровка с несогласованной нагрузкой. В этом случае эквивалентный фазовый сдвиг ξ' используется вместо реального фазового сдвига ϕ токового трансформатора. Таким образом эквивалентная фазовая задержка будет равняться:

$$\xi' = \xi - \eta$$

где ξ – фактическая эмулируемая задержка по фазе, а η – средняя величина фазового сдвига СТ. После этого для калибровки $\rho h i'$ можно использовать описанную выше методику.

3.5. Изменение частоты измеряемого сигнала

Очень важно, чтобы фактическая частота измеряемого сигнала соответствовала частоте калибровки. Любое отклонение вызывает ошибку калибровки фазового сдвига СТ. Однако, после окончания калибровки корректирующая КИХ обеспечивает необходимую точность измерения даже при изменении частоты измеряемого сигнала.

4. Заключение

Вообще говоря, цифровая обработка имеет неоспоримое преимущество перед аналоговыми интегральными схемами обработки сигналов в отношении точности и повторяемости; однако, часто имеет более высокую стоимость. Метод, описанный в этом примере применения использует цифровые технологии для компенсации и калибровки СТ, при этом он достаточно прост, чтобы быть реализованным дешевым микроконтроллером. Данное решение было с успехом реализовано на микроконтроллере MSP430 производства компании Texas Instruments и получило одобрение для использования электросчетчиках первого класса.

Ссылки:

1. A Low-Cost Single-Phase Electricity Meter Using MSP430C11x, Application Report, Texas Instruments Literature Number SLAA075
2. MSP430x1xx Family User's Guide, Texas Instruments Literature Number SLAU049
3. MSP430 Family Mixed-Signal Microcontroller Application Reports, Application Book, Texas Instruments Literature Number SLAA024
4. Bureau of Indian Standards , AC Static Watt-Hour Meters, Class-1 and -2 Specification, Reference Number ET13 (1379)

IV. Примеры конкретных применений

Глава 5. *Твердотельный диктофон с использованием MSP430*

Твердотельный диктофон с использованием MSP430

Murugavel Raju [slaa123.pdf (Англ.)]

С введением встроенной программируемой в системе (ISP) Flash памяти реализовалась обещанная рентабельность перепрограммируемых микроконтроллеров MSP430. Встроенное программное обеспечение, зашиваемое в микроконтроллер на этапе производства, позволяет обновлять программу в установленных в устройство микроконтроллерах без использования дополнительных внешних элементов. В данном примере применения демонстрируется гибкость внутрисистемного программирования Flash памяти на примере твердотельного диктофона. Кроме того, что MSP430 осуществляет аналого-цифровое и цифро-аналоговое преобразования звукового сигнала, так он еще и записывает его во встроенную Flash память микроконтроллера в реальном масштабе времени. В этом примере применения описывается возможность использования памяти программы для хранения динамически изменяющихся данных.

1. Введение

Введение микроконтроллеров MSP430 с Flash памятью принесло большую гибкость различным устройствам. Возможность внутрисистемного программирования и срок хранения данных при отключенном питании не менее 10 лет идеально удовлетворяют требованиям самых различных устройств. В данном примере применения использован микроконтроллер MSP430F149, являющийся членом семейства 1xx микроконтроллеров MSP430 с Flash памятью. Этот прибор был выбран потому, что он имеет 60 КБ встроенной Flash памяти, способной хранить в себе до 10 секунд речевых сообщений, и 12 битный АЦП, производящий оцифровку сигнала. В данном примере применения рассматриваются следующие вопросы:

- Внутрисистемное программирование Flash памяти микроконтроллеров семейства MSP430
- Программирование Flash памяти MSP430 в реальном масштабе времени
- Работа микроконтроллера MSP430x13x/14x с высокочастотным кварцевым резонатором
- Использование встроенного 12 битного аналого-цифрового преобразователя в реальном масштабе времени
- Обмен данными MSP430 с TLV5616
- Применение операционного усилителя TLV2252 и усилителя мощности ТРА721 производства компании TI совместно с MSP430
- Работа MSP430 в смешанно-сигнальном режиме от 3 В питания

2. Аппаратная часть

2.1. Краткий обзор системы

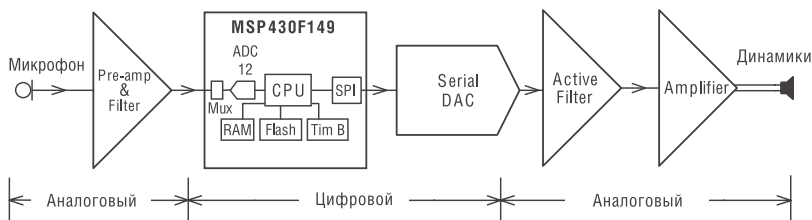


Рис. 1. Структурная схема

На рисунке 1 показана структурная схема описываемого устройства. На ней отдельно показаны цифровые и аналоговые узлы. Стрелками показано направление передачи сигналов от микрофона до динамика. Используемые периферийные модули микроконтроллера показаны внутри блока MSP430F149. Обратите внимание на встроенный 12 битный аналого-цифровой преобразователь ADC12. Встроенный в MSP430F149 аналоговый мультиплексор позволяет коммутировать ко входу ADC12 восемь аналоговых каналов. В данном примере применения оцифровывается только аналоговый сигнал на входе A0. Предварительно усиленный и отфильтрованный аналоговый сигнал с выхода микрофона подается напрямую на вход A0 микроконтроллера MSP430. При записи активны только эти два блока. При проигрывании сохраненные данные поступают из MSP430 в ЦАП по последовательному USART SPI порту. Активный фильтр сглаживает выходной сигнал ЦАП. Далее отфильтрованный сигнал подается на усилитель мощности, к выходу которого подключен динамик.

2.2. Аналоговая аппаратная часть

На рисунке 2 показан предварительный микрофонный усилитель и фильтр. Электретный микрофон преобразует акустический речевой сигнал в аналоговый электрический сигнал, который затем усиливается предварительным усилителем на базе операционного усилителя TLV2252. TLV2252 – низковольтный низкопотребляющий сдвоенный операционный усилитель, одна часть которого используется для усиления микрофонного сигнала, а другая – для построения активного фильтра низких частот (ФНЧ), который используется для сглаживания выходного сигнала ЦАП. TLV2252 выбран из-за того, что он способен работать от однополярного 3 В источника питания и имеет низкий ток покоя. Перед

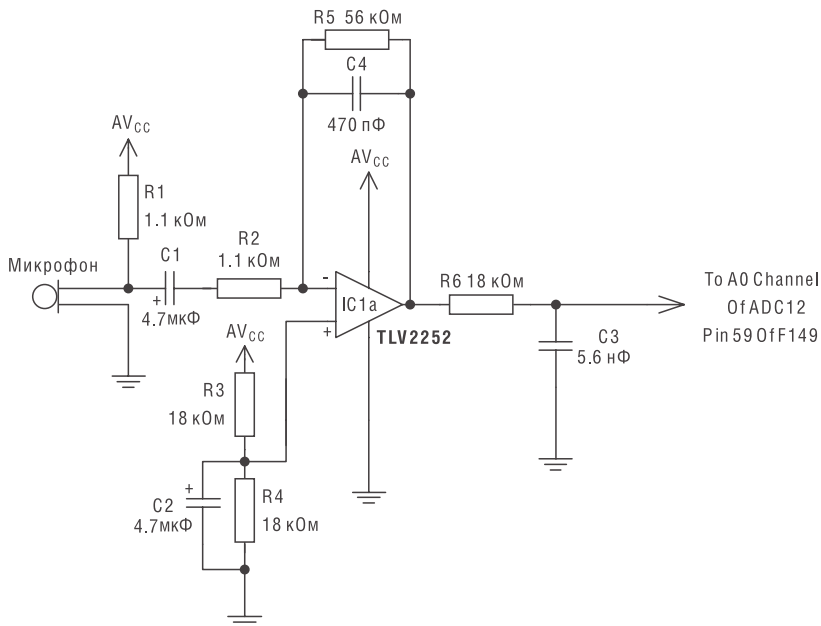


Рис. 2. Предварительный микрофонный усилитель и фильтр

подачей аналогового сигнала на вход встроенного АЦП необходимо ограничить его полосу. С этой задачей вполне справляется простой аналоговый RC фильтр, который в нашем случае включен на выходе предварительного микрофонного усилителя и имеет полосу пропускания 2.7 кГц. Конденсатор C4 в цепи обратной связи операционного усилителя также обеспечивает некоторый спад коэффициента передачи на верхних частотах. Фактически этот фильтр предназначен для исключения процесса передискретизации после осуществления выборки сигнала. Частота среза фильтра, равная 2.7 кГц, удовлетворяет требованиям теоремы Котельникова так как дискретизация в нашем случае осуществляется с частотой 5.5 кГц. Частота дискретизации 5.5 кГц выбрана в качестве компромиссного решения между качеством записываемой речи и максимально возможной длительностью записи. При упомянутых выше параметрах во встроенную Flash память можно записать около 6 секунд речи. Речевой сигнал после 12 битного преобразования без всякого сжатия записывается во встроенную Flash память. Сжатие голоса по 8- битному A- или μ - закону позволит увеличить длительность записываемого голоса до 12 секунд.

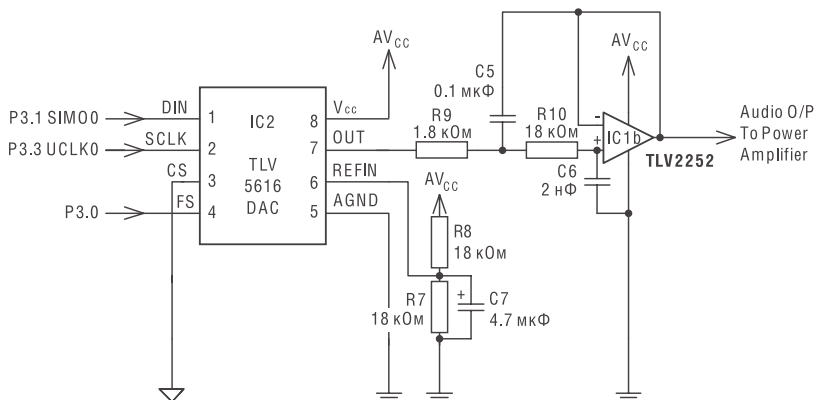


Рис. 3. Последовательный ЦАП и фильтр

На рисунке 3 показаны последовательный ЦАП и сглаживающий фильтр. Используемый в данном примере применения последовательный ЦАП TLV5616 производства компании TI способен работать от однополярного 3 В источника питания и имеет $DNL = 0.5 \text{ LSB}$. ЦАП получает данные от встроенного в MSP430 USART по SPI интерфейсу. MSP430 передает в ЦАП по SPI интерфейсу 16-разрядное за счет использования функции двойной буферизации встроенного USART модуля. ЦАП TLV5616 формирует потенциальный сигнал, который непосредственно с выхода подается на сглаживающий фильтр, изготовленный на основе операционного усилителя TLV2252. Этот узел является активным фильтром второго порядка. Перед подачей полученного сигнала на динамик, его необходимо еще усилить.

На рисунке 4 показан усилитель мощности звуковой частоты (УМЗЧ). Он реализован на основе усилителя мощности TPA721 производства компании TI. TPA721 имеет диапазон напряжения питания от 2.5 В до 5 В. Выходной мостовой усилитель (BTL) TPA721 при 3 В питании обеспечивает на 8 Ом динамике сигнал с размахом приблизительно 6 В. Также BTL устраняет необходимость использования разделительно-накопительного конденсатора. TPA721 выпускается в 8 выводном MSOP корпусе, называемом еще PowerPAD корпусом, для поверхностного монтажа. Обратите внимание, что на рисунке 4 показан RC фильтр низких частот, состоящий из резистора R13 и конденсатора C9, который предназначен для развязки по питанию.

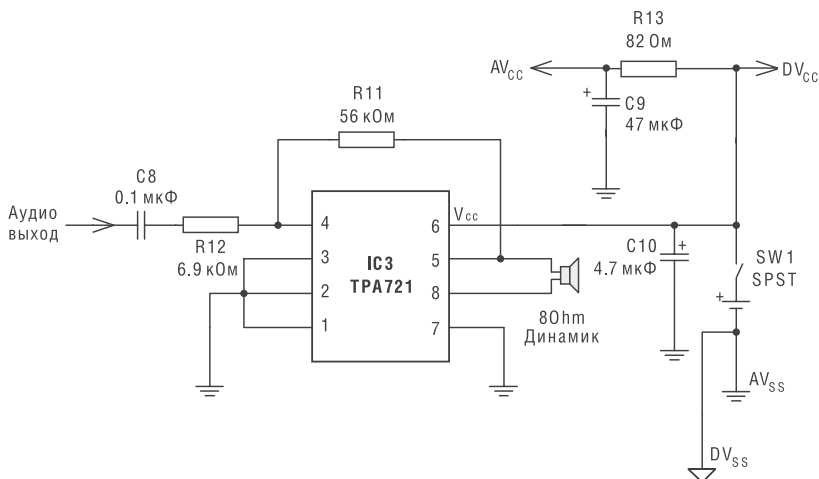


Рис. 4. Усилитель мощности звуковой частоты

2.3. Цифровая аппаратная часть

Цифровая аппаратная часть состоит из микроконтроллера MSP430F149 и пассивных элементов его обвязки. Встроенные периферийные модули микроконтроллера облегчают разработку цифровой части устройства и позволяют назвать его «системой на кристалле» («System in a Chip»). Задающий генератор MSP430 работает с внешним 3.58 МГц керамическим резонатором. Резонатор, используемый в этом примере применения, имеет встроенные нагрузочные конденсаторы. Таймер В7 используется для формирования сигнала прерывания, по которому происходит выборка аналогового сигнала. Таймер В7 тактируется от стабильных 3.58 МГц колебаний. Тактирование его сигналом со стабильной частотой вызвано тем, что нестабильность частоты дискретизации влияет на качество записи и воспроизведения речи. Выбранный аналоговый речевой сигнал оцифровывается встроенным 12 битным АЦП ADC12 микроконтроллера MSP430. Обратите внимание, что питание аналоговой и цифровой аппаратных частей разделено. Также должны быть разделены и цепи общих линий.

Оцифрованный речевой сигнал сохраняется во встроенной Flash памяти программы микроконтроллера. В режиме воспроизведения в той же самой последовательности и с той же частотой данные из памяти поступают в ЦАП через USART, работающий в режиме SPI интерфейса. ЦАП преобразует данные

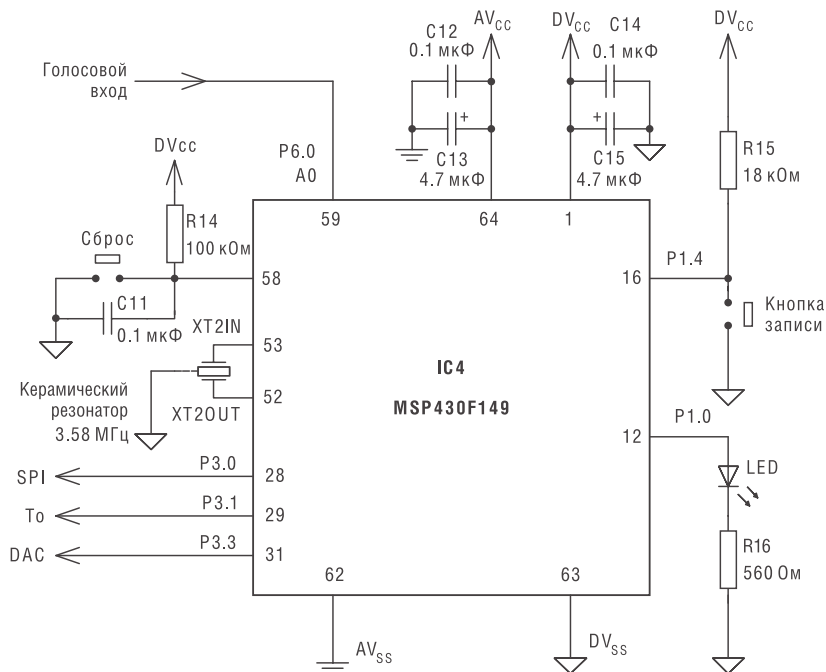


Рис. 5. Схема подключения MSP430F149

в аналоговый сигнал, сглаживающий фильтр восстанавливает его, а УМЗЧ усиливает и подает его на динамик.

3. Программное обеспечение

Прикладная программа для данного примера применения написана на ассемблере в интегрированной среде разработки IAR KickStart. Основная память MSP430F149 разделена на 120 сегментов, адреса которых лежат в пределах от 1100 до FFFFh. С нулевого по 118 сегменты имеют размер 512 байт, а 119 сегмент – 256 байт. Нулевой сегмент содержит вектора прерываний и поэтому он не должен изменяться при выполнении программы. Еще два сегмента (A и B), имеющие размер по 128 байт, используются в качестве информационной памяти прибора. Пользователь может использовать эту память для хранения идентификационного кода прибора. Также информационная память может использоваться в качестве замены EEPROM памяти или для хранения выпол-

няемого кода. Однако в данном примере применения информационная память не используется.

Код прикладной программы записан в память начиная с адреса 1100h – начального адреса основной памяти. Откомпилированная программа имеет объем 346 байт и занимает 119 (256 байт) и 118 (90 байт из 512 байт) сегменты. В нулевом сегменте записаны вектора прерываний. Остальные сегменты с первого по 117 предназначены для хранения записанной речевой информации. Эти 117 сегментов занимают объем памяти с адресами от 1400h по FDFh. При записи оцифрованный речевой сигнал записывается последовательно во Flash память, а при воспроизведении – считывается оттуда.

Программа выполняется в двух режимах – запись и воспроизведение. Выбор режима осуществляется нажатием кнопки управления. При включении система переходит в режим воспроизведения и сразу проигрывает записанное речевое сообщение. Во включенном состоянии в режиме воспроизведения диктофон непрерывно повторяет записанное в него сообщение. Для перехода в режим воспроизведения необходимо выполнить следующие действия. При проигрывании сообщения необходимо нажать и удерживать кнопку записи. После этого светодиод начнет мигать, показывая, что Flash память очищена и готова к новой записи. Отпустите кнопку и говорите в микрофон. Сообщение запишется во Flash память, а когда память заполнится полностью – светодиод погаснет, показывая, что запись окончена. После этого система вновь перейдет в режим непрерывного повтора записанного сообщения.

Обратите внимание, что Flash память может программироваться и стираться только в том случае, если напряжение питания MSP430 больше или равно 2.7 В. Это описано в техническом описании на данный микроконтроллер. Если напряжение питания падает ниже 2.7 В из-за разряда батареи, то система не сможет записать речь. Однако, воспроизведение возможно до того момента, пока напряжение источника питания не станет ниже допустимого для использованных аналоговых микросхем.

Примечание:

Устройство, описанное в данном примере применения, не предназначено специально для записи речевых сообщений. Это только пример внутрисистемного программирования (ISP) Flash памяти в реальном масштабе времени и выполнения микроконтроллером MSP430 операций, свойственных для DSP. Для упрощения реализации в данном примере применения не осуществляется сжатие речевого сигнала. Компания TI выпускает ряд приборов, специально предназначенных для записи речи. Ознакомиться с ними можно в интернете по адресу <http://www.ti.com/sc/docs/products/speech/index.htm>.

4. Демонстрационная программа F149VoiceDemo.s43

; ЭТА ПРОГРАММА ПОСТАВЛЯЕТСЯ В ВИДЕ «КАК ЕСТЬ».
; КОМПАНИЯ TI НЕ ДАЕТ НИКАКИХ ГАРАНТИЙ И НЕ НЕСЕТ
; НИКАКОЙ ОТВЕТСТВЕННОСТИ ЗА ВОЗМОЖНЫЕ НЕТОЧНОСТИ
; РАБОТЫ ИЛИ НЕПРАВИЛЬНОСТЬ РАБОТЫ ПРОГРАММЫ.
; ПРИ КОММЕРЧЕСКОМ ИСПОЛЬЗОВАНИИ ДАННОЙ ПРОГРАММЫ
; КОМПАНИЯ TI ОТКАЗЫВАЕТСЯ ОТ АВТОРСТВА И СНИМАЕТ
; С СЕБЯ ВСЯКУЮ ОТВЕТСТВЕННОСТЬ.
;
; КОМПАНИЯ TI ОТКАЗЫВАЕТСЯ ОТ ЛЮБЫХ ГАРАНТИЙ В ОТНОШЕ-
; НИИ ТОГО, ЧТО ПРОГРАММА НЕ МОЖЕТ ПРИНИСТИ ПРЯМЫЕ ИЛИ
; КОСВЕННЫЕ УБЫТКИ ИЛИ ПОТЕРЮ ДОВЕРИЯ ИЛИ ПРИВЫЛИ
; ПРИ ИСПОЛЬЗОВАНИИ ЕЕ ТЕОРИИ ИЛИ АЛГОРИТМОВ.
; ОТВЕТСТВЕННОСТЬ ПРИ КОММЕРЧЕСКОМ ИСПОЛЬЗОВАНИИ
; ДАННОЙ ПРОГРАММЫ ПОЛНОСТЬЮ ЛЕЖИТ НА ЛИЦЕ, ИСПОЛЬЗО-
; ВАВШЕМ ДАННУЮ ПРОГРАММУ
;
; Если не указано иное, программа, созданная компанией
; Texas Instruments поставляется на условиях «свободно
; распространяемого программного обеспечения».
; Вы можете свободно изменять программу без всяких
; ограничений
; Передавать программу третьим лицам можно только при
; доведении до их сведений этих условий и при его
; согласии с этими условиями.
; Вы должны приводить лицензию в каждой копии или
; частичной копии.
;
; Вы признаете и соглашаетесь, что Программа содержит
; защищенный авторским правом материал, коммерческую
; тайну и другую частную информацию компании TI
; и защищена в соответствии с законами об авторском
; праве, международным соглашением об авторском праве
; и законом о коммерческой тайне, а так же законом об
; охране интеллектуальной собственности. Для защиты
; авторских прав компании TI
; Вы соглашаетесь не декомпилировать, перерабатывать,
; или преобразовывать программу к удобочитаемому виду.
; Вы соглашаетесь ни в коем случае не изменять и не
; удалять внесенную в программу информацию
; об авторском праве. Компания TI оставляет за собой

```

; все права, не оговоренные в данной лицензии. Кроме
; того, что оговорено в данной лицензии, не может рас-
; сматриваться при возможном судебном разбирательстве.
;
; Вы не можете использовать программу для микроконт-
; роллеров, которые не производит компания TI.
;*****
NAME F149VoiceDemo ; Цифровой диктофон на микроконт-
; роллере MSP430F149 с FLASH памятью
; Автор: Murugavel Raju
; Компания Texas Instruments, Inc
;*****
#include <msp430x14x.h>
;*****
; Микроконтроллер работает на частоте 3.58 МГц
;
; Определение констант
FS      equ    001h
Memstart equ    1401h
Memend   equ    0fe01h
;*****
                RSEG   CSTACK
                DS      0
;*****
                RSEG   CODE
;*****
RESET      mov.w  #SFE(CSTACK),SP ; определение указате-
                                   ; ля стека
                call #Init_Sys    ; Инициализация
                                   ; системы
                mov.b &P1IN,R5    ; проверка состояния
                                   ; вывода P1.1 для
                                   ; определения режима
                and.b #BIT4,R5    ; Маскирование Bit4
                                   ; для проверки состоя-
                                   ; ния кнопки 'Record'
                jnz   Play        ; Переход по метке
                                   ; Play если кнопка
                                   ; не нажата
                call  #Erase      ; Вызов подпрограммы
                                   ; очистки Flash памяти

```

```

        xor.w  #FXKEY+WRT, &FCTL1; Активизация FLASH
                                   ; памяти для записи
        bis.b  #BIT0, &P1OUT      ; Включение светодиода
Play    eint                                     ; Разрешение прерыва-
                                   ; ний
        mov.w  #Memstart, R14      ; Пересылка начального
                                   ; адреса памяти
                                   ; в регистр R14
Mainloop jmp   $                   ; Зацикливание, выход
                                   ; только по прерыванию
;*****
TB7_ISR ; Подпрограмма обработки прерывания таймера
        ; Timer B7, сохранение длительности записи
        ; и пересылка этих данных для проигрывания
;*****
        bit.b  #BIT4, R5           ; Проверка нажатия
                                   ; кнопки выбора режима
        jnz    Play1              ; Если не нажата, то
                                   ; переходим к метке
                                   ; Play1
        bic.w  #ADC12SC, &ADC12CTL0 ; Начало преобразо-
                                   ; вания
Conv_tst bit.w  #ADC12BUSY, &ADC12CTL1 ; Ожидание
                                   ; окончания преобразо-
                                   ; вания
        jc     Conv_tst           ; Зацикливание
                                   ; до окончания преоб-
                                   ; разования
        bis.w  #ADC12SC, &ADC12CTL0 ; возврат в режим
                                   ; выборки
        mov.w  &ADC12MEM0, 0 (R14) ; Запись слова
                                   ; во FLASH память
        incd.w R14                ; Увеличение счетчика
        cmp.w  #Memend, R14       ; Проверяем, заполнена
                                   ; ли память
        jnz    Proceed           ; Если память не
                                   ; заполнена, то пере-
                                   ; ходим к обработке
        bic.b  #BIT0, &P1OUT      ; Если память заполне-
                                   ; на, то выключаем
                                   ; светодиод

```

	<code>xor.w #FXKEY+WRT, &FCTL1;</code>	Запрещаем запись
		; во FLASH память
	<code>xor.w #FXKEY+LOCK, &FCTL3</code>	; Блокировка FLASH
		; памяти
	<code>jmp RESET</code>	; Начинаем сначала
		; (если кнопка отжата,
		; то начнем воспроиз-
		; ведение)
Play1	<code>mov.w @R14, R15</code>	; Считывание данных из
		; указателя памяти
	<code>Incd.w R14</code>	; Дублирование увели-
		; чение указателя для
		; перехода к следующе-
		; му слову данных
	<code>cmp.w #Memend, R14</code>	; Проверка достижения
		; конца массива памяти
	<code>jnz Go_on</code>	; Переход к метке
		; Go_on если конец не
		; достигнут
	<code>jmp RESET</code>	; переходим к началу
		; если достигнут конец
		; массива памяти
Go_on		
L1	<code>bit.b #UTXIFG0, &IE1</code>	; Зацикливание до тех
		; пор, пока предыдущая
		; передача
	<code>jc L1</code>	; не закончится и буфер
		; не очистится
	<code>bis.b #FS, &P3OUT</code>	; Кадровый импульс
		; синхронизации
	<code>bic.b #FS, &P3OUT</code>	; для начала загрузки
		; в TLV5516 следующего
		; слова данных
	<code>swpb R15</code>	; меняем местами байты
		; в регистре R15
	<code>mov.b R15, &TXBUF0</code>	; пересылка старшего
		; байта в TXBUF модуля
		; SPI
	<code>swpb R15</code>	; Меняем местами байты
		; в регистре R15
	<code>mov.b R15, &TXBUF0</code>	; Пересылка младшего
		; байта в TXBUF модуля
		; SPI


```

Proceed reti ; Выход из подпрограммы
; обработки прерывания
;*****
Init_Sys ; Настройка периферийных модулей
;*****
StopWDT mov.w #WDTPW+WDTHOLD,&WDTCTL ; Остановка
; сторожевого таймера
SetupBC bic.b #XTOFF,&BCSCTL1 ; Активизация автоге-
; нератора на XT2
call #Delay ; Задержка для обеспе-
; чения выхода автоге-
; нератора в стабиль-
; ный режим
mov.b #SELM1+SELS,&BCSCTL2 ; MCLK=SMCLK=XT2CLK
bic.b #OIFG1,&IFG1 ; Очистка OIFG
SetupP1 mov.b #0h,&P1OUT ; Очистка выходного
; регистра порта P1
bis.b #0ffh,&P1DIR ; Настройка всех линий
; порта P1 для работы
; в режиме выхода
; Вывод P1.0 использу-
; ется для управления
; светодиодом
bic.b #BIT4,&P1DIR ; Для переключения
; выходов
SetupP2 mov.b #0h,&P2OUT ; очистка выходного
; регистра порта P2
bis.b #0ffh,&P2DIR ; Неиспользуемые выво-
; ды переводятся
; в режим выходов
SetupP3 bis.b #00ah,&P3SEL ; Линии порта
; P3.1 & P3.3
; используются для SPI
; модуля
bis.b #FS,&P3OUT ; Установка FS
bis.b #0feh+FS,&P3DIR ; P3.0,3.1,3.3 и неис-
; пользование выводы
; переводятся в режим
; выходов
SetupP4 mov.b #0h,&P4OUT ; Очистка выходного
; регистра порта P4

```

```

bis.b #0ffh,&P4DIR      ; Неиспользуемые выво-
                        ; ды переводятся в
                        ; режим выходов
SetupP5 mov.b #0h,&P5OUT ; Очистка выходного
                        ; регистра порта P5
bis.b #0ffh,&P5DIR      ; Неиспользуемые выво-
                        ; ды переводятся в
                        ; режим выходов
SetupP6 bis.b #BIT0,&P6SEL ; P6.0 = A0 вход
                        ; модуля ADC12
mov.b #0h,&P6OUT        ; Выходные линии P6
                        ; используются для
                        ; сброса
bis.b #0feh,&P6DIR      ; P6.1 – 6.7 выходы
                        ; (неиспользуются)
SetupADC Call #ADCset   ; Инициализация модуля
                        ; ADC12
SetupUSART bis.b #040h,&ME1 ; Активизация модуля
                        ; USART
mov.b #CHAR+SYNC+MM,&U0CTL ; 8-битный
                        ; модуль SPI в режиме
                        ; ведущего
mov.b #CKPL+SSEL1+SSEL0+STC,&U0TCTL
                        ; SMCLK для передачи,
                        ; 3-проводный интер-
                        ; фейс
mov.b #02h,&U0BR0       ; SMCLK/2 для задания
                        ; скорости обмена дан-
                        ; ными
clr.b &U0BR1           ;
clr.b &U0MCTL           ; Очистка модуляции
SetupCCR0 bis.w #CCIE,&TBCCTL0
Mov.w #649,&TBCCR0      ; Инициализация тайме-
                        ; ра TBCCR0 для выборки
                        ; с частотой 5.5 кГц
SetupTB7 bis.w #TBSSEL1+MC0,&TBCTL ; Перевод тайме-
                        ; ра TimerB в режим UP
SetupFlash xor.w #FXKEY+FN2+FN1+FN0,&FCTL2
                        ; Установка частоты
                        ; генератора FLASH
                        ; памяти 447.5 кГц
ret                    ; Выход из подпрограммы

```

```

;*****
ADCset ; Инициализация модуля ADC12, VCC используется
; в качестве опорного напряжения
; Одноканальный (A0) режим однократного преобразования
;*****
        bis.w #ADC12ON+ADC12SC+ENC,&ADC12CTL0
                                ; Включение АЦП и выборка
                                ; входного сигнала
        call #Delay             ; Задержка для стабилизации
        bis.w #ADC12SSEL_1+ADC12SSEL_2,&ADC12CTL1
                                ; ADC12 синхронизируется
                                ; от SMCLK
        ret                     ; Выход из подпрограммы

;*****
Erase   ; Инициализация FCTL и стирание FLASH памяти
        для новой записи
;*****
        dint                     ; Запрещение прерываний
        xor.w #FXKEY+LOCK,&FCTL3 ; Разблокирование
                                ; FLASH памяти для записи
Test_Busy1 bit.w #BUSY,&FCTL3    ; Проверка флага BUSY
        jnz Test_Busy1 ; Зацикливание
        mov.w #1400h,R13 ; Адрес начала массива памяти
                                ; пересылается в регистр R13
NextSeg mov.w #(FWKEY+ERASE),&FCTL1 ; Установки
        Clr.b 0(R13) ; Выполнение макет записи для
                                ; активизации стирания
                                ; сегмента
Test_Busy2 bit.w #BUSY,&FCTL3    ; Проверка флага BUSY
        jnz Test_Busy2 ; Зацикливание
        add.w #200h,R13 ; Установка указателя на
                                ; следующий сегмент
        cmp.w #0fe00h,R13 ; Все сегменты очищены?
        jnz NextSeg ; Если нет, то переходим
                                ; к очистке следующего
                                ; сегмента
        ret                     ; Выход из подпрограммы

;*****
Delay   ; Подпрограмма формирования задержки
;*****
        push.w #0FFFFh ; Задержка до TOS
DL1     dec.w 0(SP) ; Уменьшение TOS

```

```

jnz     DL1           ; Задержка окончена?
Incd.w SP             ; Очистка TOS
ret                      ; Выход из подпрограммы

;*****
COMMON INTVEC ; Векторы прерываний MSP430F14x
;*****
ORG      TIMERB0_VECTOR
WDT_VEC  DW TB7_ISR   ; Обработка прерывания
                        ; таймера Timer B7

ORG      RESET_VECTOR
RESET_VEC DW RESET    ; Обработка POR,
                        ; внешнего сброса
                        ; и сброса сторожевого
                        ; таймера

END

```

Ссылки:

1. MSP430x13x, MSP430x14x Mixed Signal Flash Microcontroller datasheet (SLAS272B)
2. MSP430x1xx Family User's Guide (SLAU049)
3. TLV225x Rail-To-Rail Low-Voltage Low-Power Operational Amplifier (SLOS185B)
4. TLV5616 Low Power 12-Bit Digital-To-Analog Converters (SLAS152B)
5. TPA721 700-mW Mono Low-Voltage Audio Power Amplifier (SLOS231B)

IV. Примеры конкретных применений

Глава 6. *ШИМ-управление двигателем постоянного тока*

ШИМ-управление двигателем постоянного тока

Brian Merrit/Murugavel Raju [slaa120.pdf (Англ.)]

В данном примере применения описывается теория управления двигателем постоянного тока при помощи широтно-импульсной модуляции (ШИМ). Для формирования управляющего сигнала с ШИМ используется модуль таймера Timer_A.

1. Введение

Многие из микроконтроллеров семейства MSP430 имеют или модуль таймера Timer_A, или модули двух таймеров Timer_A и Timer_B. Эти модули содержат 16-разрядный счетчик, который может увеличиваться от различных источников синхросигнала. Они могут работать даже в режиме пониженного потребления. Также эти таймеры имеют регистры захвата/сравнения, которые могут применяться по-разному в разных приложениях. Одним из таких приложений является автоматическое формирование сигнала с ШИМ для управления двигателем постоянного тока. В данном примере применения описывается управление при помощи модулей таймеров Timer_A и/или Timer_B одной из множества существующих схем управления двигателем постоянного тока.

2. Теория функционирования

При работе демонстрационная схема, приведенная на рисунке 1, постоянно опрашивает кнопки управления на двух входах и при нажатии на них увеличивает или уменьшает содержимое регистра захвата/сравнения CCR1 таймера Timer_A. Для того чтобы не произошла инверсия назначения кнопок, управляющая программа проверяет регистр на максимальное и минимальное значение. Timer_A настроен на увеличение по сигналу MCLK, а выход модуля Out1 в режиме 7 подключен к выводу P1.2/TA1 микроконтроллера, на котором и формируется сигнал с ШИМ. В регистр CCR0 записано значение 255, которое и определяет период таймера Timer_A. Изменение значения регистра CCR1 изменяет длительность рабочего импульса (скважность сигнала), сформированного модулем таймера Timer_A. Сигнал с ШИМ используется для коммутации составного транзистора, включенного по схеме Дарлингтона. В свою очередь этот транзистор подключает двигатель постоянного тока. Так как изменяется скважность сигнала управления, то будет изменяться и средняя мощность, подаваемая на двигатель. А средняя мощность, в свою очередь, и определяет скорость вращения двигателя. Частота вращения двигателя изменяется дискретно 256 шагами в пределах от 0 до максимального

значения. Встроенный в микроконтроллер автогенератор с цифровым управлением (DCO) работает на частоте приблизительно 5 МГц, обеспечивая частоту ШИМ сигнала приблизительно 20 кГц, что позволяет двигателю работать не производя акустических шумов.

Обратите внимание, что к MSP430 не подключен никакой кварцевый резонатор. Частота DCO использованного микроконтроллера MSP430x11x(1) зависит от температуры и напряжения питания. Поэтому изменяется и частота ШИМ сигнала. Так как длительность рабочего цикла ШИМ сигнала также зависит от частоты DCO, то скважность сигнала не изменится. Поэтому средняя мощность, подаваемая на двигатель, не изменится. Если же требуется стабильная частота сигнала с ШИМ, то к микроконтроллеру необходимо подключить кварцевый резонатор.

Показанный на схеме прибор TPS77133 производства компании TI, формирует стабилизированное напряжение VCC для питания микроконтроллера MSP430 и формирование сигнала сброса. Если необходимо формировать и сигнал установки напряжения питания (Power Good), то можно использовать прибор TPS77233, который также производится компанией TI.

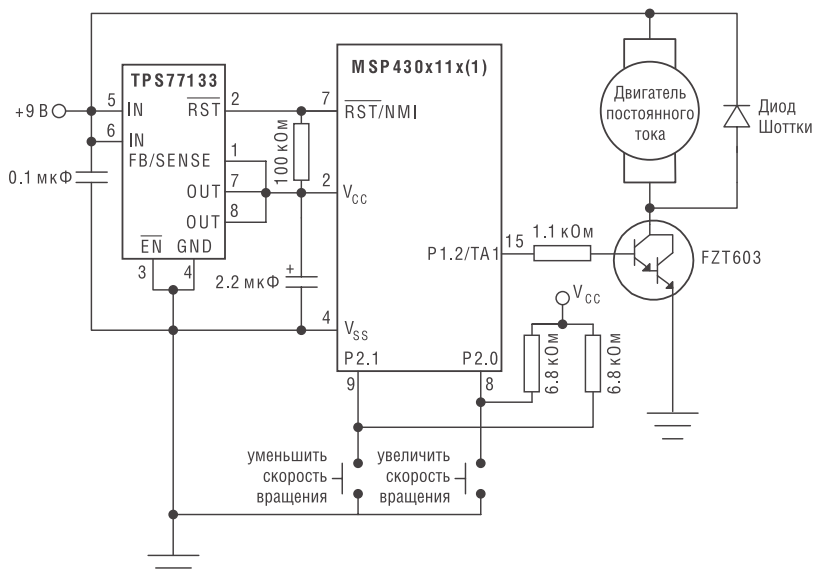


Рис. 1. Демонстрационная схема управления двигателем постоянного тока при помощи сигнала с ШИМ

3. Демонстрационная управляющая программа

Демонстрационная управляющая программа для приведенной на рисунке 1 схемы на основе микроконтроллера MSP430 доступна на *сайте* <http://www.ti.com/sc/msp430>. Программа написана на ассемблере в интегрированной среде обработки IAR Kickstart.

Ссылки:

1. MSP430x11x1 Datasheet (SLAS241C)
2. MSP430x1xx User's Guide (SLAU049)
3. TPS77xxx Datasheet (SLVS225B)

IV. Примеры конкретных применений

Глава 7. *Милливольтметр сигма-дельта типа на MSP430F11x1*

Милливольтметр сигма-дельта типа на MSP430F11x1

Mark Buccini [slaa104.pdf (Англ.)]

В данном примере применения описывается метод реализации дешевого 12 битного сигма-дельта АЦП на основе микроконтроллера MSP430F11x1 семейства 16-разрядных смешанно-сигнальных RISC процессоров. Описываемое устройство использует периферийный модуль comparator_A. Для демонстрации возможностей на микроконтроллере реализован вольтметр, передающий результат измерения в милливольтх по UART протоколу со скоростью 9600 бод. Описываемое устройство реализовано на микроконтроллере MSP430F11x1, но может быть легко адаптировано под любой микроконтроллер семейства MSP430, имеющий модуль comparator_A.

1. Введение

Используя микроконтроллер MSP430F11x1 семейства 16-разрядных смешанно-сигнальных процессоров, можно легко реализовать дешевый аналого-цифровой преобразователь с высоким разрешением на основе внутреннего компаратора `comparator_A` микроконтроллера. Описанный аналого-цифровой преобразователь использует технологию сигма-дельта АЦП. Демонстрацион-

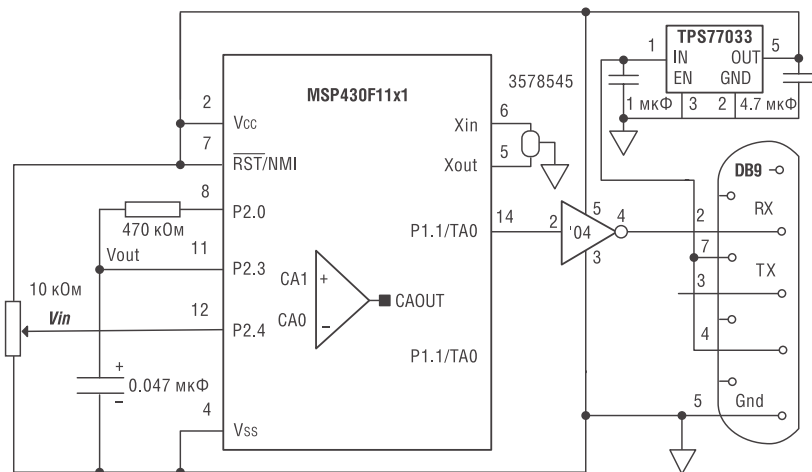


Рис. 1. Милливольтметр на базе MSP430F11x1

ная схема, приведенная на рисунке 1 имеет разрешение 1 мВ при 3.3 В однополярном питании. АЦП такого типа, имеющие высокое разрешение, идеально подходят для измерения медленно меняющихся физических величин типа температуры, давления, интенсивности освещения и напряжения. Высокая точность получаемого АЦП и сверх малое потребление MSP430 позволяют изготавливать измерительные устройства с автономным питанием. Ключевые характеристики встроенного АЦП микроконтроллера MSP430F11x1:

- 1 мВ / 12 битное разрешение
- Интегрированное решение, обеспечивающее превосходное подавление шумов
- Не требуется калибровка компаратора
- Для реализации АЦП требуется только два внешних компонента.

2. Теория функционирования

Концепция интегрированного АЦП состоит в сравнении неизвестного входного напряжения V_{in} с известным выходным напряжением V_{out} . Используя один единственный цифровой выход, MSP430F11x1 осуществляет 1 битное цифро-аналоговое преобразование. На выходе ЦАП подсчитывается сколько импульсов было сформировано для того, чтобы выходное напряжение V_{out} сравнялось с входным напряжением V_{in} . Программное обеспечение формирует и считает выходные импульсы ЦАП при помощи кольцевого регистра. На выходе ФНЧ ЦАП формируется постоянное напряжение V_{out} (см. рисунок 2). Программная следящая связь сравнивает сформированное выходное напряжение V_{out} с измеряемым входным напряжением V_{in} при помощи встроенного модуля компаратора `comparator_A`. Изменяя длину регистра следящей связи можно изменить разрешающую способность. Например, кольцевой регистр, имеющий длину 256, обеспечит 8 битное разрешение, а имеющий длину 4096 – 12 битное разрешение. В описываемой схеме выбрана длина счетчика, равная 3300, что позволяет измерять напряжение в диапазоне от 0 до 3,3 В с разрешением 1 мВ. В любом случае результат преобразования пропорционален напряжению питания. Не следует забывать, что диапазон входного сигнала ограничивается диапазоном допустимого синфазного сигнала на входе встроенного компаратора модуля `comparator_A`.

Положим, что напряжение V_{out} установлено в предыдущем цикле преобразования равным напряжению V_{in} , а выходное сопротивление открытых транзисторов порта ввода-вывода MSP430 очень низкое ($R_{DSon} < 300 \text{ мОм}$), что обеспечивает уровень логической единицы близким к напряжению питания, а уровень логического нуля – близким к нулю:

if $V_{in}(t) = V_{in}$, then solving...

$$V_{in}(t) = V_{in} + n(V_{CC} - V_{in})(1 - e^{-\frac{t}{RC}}) - (3300 - n)(V_{in})(1 - e^{-\frac{t}{RC}});$$

$$V_{in} = V_{CC} \frac{n}{3300}; \quad V_{in}(mv) = n$$

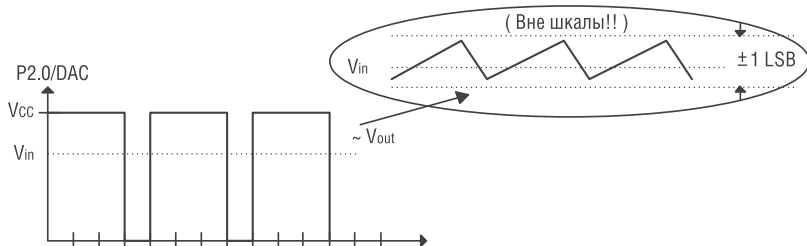


Рис. 2. График выходного напряжения встроенного ЦАП

2.1. Расчет номиналов элементов ФНЧ 1 битного ЦАП

Выбор резистора и конденсатора ФНЧ ЦАП очень важен. Этот фильтр должен обеспечивать пульсацию выходного напряжения V_{out} не более 1 LSB. Такой маленький уровень пульсации выходного напряжения и гарантирует точность преобразования ± 1 LSB. Период следования импульсов, формируемых программной обратной связью, в приведенной программе равняется 14 тактовым циклам микроконтроллера. В данном примере применения MSP430F11x1 работает от внешнего кварцевого резонатора с частотой 3.578545 МГц. Тогда период следования импульсов будет равен $14/3578545$ сек = 4.9 мкс. Размах напряжения пульсаций должен быть менее 1 LSB (1/3300). При таких условиях можно рассчитать номиналы элементов ФНЧ ЦАП:

$$V(t) = V(1 - e^{-\frac{t}{RC}}); \quad \frac{V(t)}{V} = 1 - e^{-\frac{t}{RC}}; \quad e^{-\frac{t}{RC}} = 1 - \frac{V(t)}{V};$$

$$-\frac{t}{RC} = \ln(1 - \frac{V(t)}{V}); \quad -RC = \frac{t}{\ln(1 - \frac{V(t)}{V})} = \frac{4.9 \text{ us}}{\ln(\frac{3299}{3300})} = -0.01616$$

$$\text{if } R = 470k \text{ then } C = 0.035 \text{ uf use } \approx 0.047 \text{ uf}$$

2.2. Используемые ресурсы MSP430F11x1

Для реализации встроенного АЦП требуется очень мало ресурсов MSP430F11x1:

- Модуль comparator_A и связанные с ним входы P2.3 и P2.4 компаратора.
- Один цифровой выход для реализации 1 – битного ЦАП, например выход P2.0 в примере.

Для реализации однобитного ЦАП может использоваться любой доступный цифровой выход MSP430F1101. В описываемом устройстве используется линия P2.0 порта ввода-вывода. Линия P2.0 настроена на работу в режиме цифрового выхода, использующего выходной регистр второго порта (P2OUT). Линии P2.3 и P2.4 могут использоваться в качестве линий порта ввода-вывода общего назначения или, как в нашем случае, в качестве входов аналогового компаратора модуля comparator_A. Выбор режима работы линий осуществляется путем установления нужного значения регистра второго порта (P2SEL). При помощи регистра управления направленностью линий второго порта (P2DIR) линии P2.3 и P2.4 должны быть настроены на работу в качестве входов. Компаратор модуля comparator_A, настраиваемый при помощи управляющих регистров SACTL1 и SACTL2, используется для сравнения аналоговых напряжений Vout и Vin. Результат измерения передается при помощи UART, использующего модуль таймера Timer_A, на скорости 9600 бод по линии P1.1.

2.3. Требующиеся внешние компоненты

Для реализации АЦП требуется только два внешних компонента:

- Один резистор
- Один полиэстровый пленочный конденсатор с низким током утечки

Макетная схема устройства, приведенная на рисунке 1, имеет связь с персональным компьютером и питается непосредственно от RS232 интерфейса. Для согласования уровней логических сигналов микроконтроллера и RS232 интерфейса используется микросхема интерфейса типа MAX232. Также можно использовать одно-вентильный интерфейс типа SN74AH1GC04. Кроме этого в схеме применены низкопотребляющий стабилизатор напряжения с низким падением напряжения TPS77033 и кварцевый резонатор с частотой 3.578545 МГц.

2.4. Снижение системной ошибки

Общая точность аналого-цифрового преобразователя определяется точностью однобитного ЦАП. ЦАП формирует выходное напряжение в диапазо-

не от напряжения питания (VCC) до нуля (GND). Обеспечивается это тем, что выходные полевые транзисторы цифрового порта, использующегося под ЦАП, имеют очень низкое сопротивление открытого канала. Также точность ЦАП зависит от точности стабилизации напряжения питания MSP430. В демонстрационной схеме напряжение питания VCC было установлено точно равным 3300 милливольтам. Это значение 3300 запоминается в регистре VCC_Cal и вызывается в подпрограмме следящей связи Meas_ADC. Аналого-цифровой преобразователь является масштабируемым по отношению к напряжению питания. Если необходимо производить измерение напряжения, то точность АЦП будет определяться точностью стабилизации напряжения питания.

Компаратор имеет напряжение смещения, которое вызовет ошибку. Уникальная особенность модуля comparator_A – наличие входного мультиплексора, позволяющего менять местами входы. Этот простой, но мощный мультиплексор устраняет необходимость компенсации напряжения смещения компаратора. Коррекция смещения производится путем вычисления среднего значения двух результатов, полученных в результате переключения входов. Или, как в этом примере применения, 50 % преобразующей последовательности формируется при одном состоянии мультиплексора, а 50 % – при другом. Переключение входов модуля comparator_A осуществляется путем изменения бита CAEX в управляющем регистре CACTL1. Для более полного описания см. техническое описание на микроконтроллер. Если устройство должно иметь внешнюю калибровку, то точное значение может быть сохранено во Flash памяти MSP430F11x1 или непосредственно в ОЗУ (если устройство будет иметь непрерывное питания).

Замечание: Демонстрационная схема питается непосредственно от последовательного порта персонального компьютера. Импульсные блоки питания некоторых компьютеров являются источниками сильных помех, которые могут попасть через последовательный порт непосредственно на MSP430. Кроме того, не следует упускать из внимания и вопросы правильной трассировки платы.

Программная следящая связь должна быть эквивалентна тракту и не блокироваться при выполнении аналого-цифрового преобразования. Программа написана таким образом, что период следования импульсов ЦАП равняется 14 тактам, что позволяет обеспечить симметрию длительности всех импульсов ЦАП. Если при выполнении подпрограммы измерения происходит прерывание, то до и после выполнения подпрограммы обслуживания прерывания должны анализироваться значения DINT и EINT. Это позволит избежать обслуживания ложных прерываний.

3. Описание программы fet_intADC.s43

Демонстрационная программа fet_intADC.s43 состоит из нескольких процедур. После сброса выполняется подпрограмма Init_Sys, инициализирующая порты ввода-вывода и модули микроконтроллера MSP430.

В основной программе ЦАП вызывается подпрограммой Meas_ADC. Подпрограмма Meas_ADC осуществляет выборку и преобразование аналогового сигнала, а результат преобразования помещает в регистр ADCData. Встроенная следящая связь с коэффициентом пересчета 3300, непосредственно в теле подпрограммы Meas_ADC высчитывает необходимое количество импульсов в десятичном виде для выполнения цифро-аналогового преобразования. Это позволяет MSP430, работающему от 3.3 В питания выдавать результат преобразования непосредственно в милливольттах.

```

; -----
Meas_ADC ; Подпрограмма: измерения при помощи АЦП,
; результат преобразования АЦП
; в двоично-дополнительном коде → ADCData
; R15 используется в качестве рабочего
; регистра и не сохраняет свое состояние
; -----

        mov.b #CAON,&CACTL1 ; Включение компаратора
        clr   ADCData       ; Очистка регистра
                                ; ADCData
        call  #Sample_ADC    ; результат выборки →
                                ; ADCData
        call  #Sample_ADC    ; результат выборки →
                                ; ADCData + предыдущие
                                ; данные
Meas_Over clr.b &CACTL1      ; Выключение компара-
                                ; тора
        ret                ; Выход из подпрограммы
                                ;
Sample_ADC mov &VCC_Cal,R15 ; ~3300
        rra   R15           ; VCC/2
; предварительный заряд конденсатора ;
Pre_ADC bis.b #DAC_Out,&P2OUT ; заряд конденсатора
C1      bit.b #CAOUT,&CACTL2 ; На выходе компарато-
                                ; ра сигнал высокий
                                ; или низкий?
        jz    C1            ;
; Настройка и накопление импульсов ЦАП ;

```

```

Test_DAC bit.b #CAOUT,&CACTL2 ; На выходе компарато-
                                ; ра сигнал высокий
                                ; или низкий?
                                ;
                                jnc     Low1
High    bic.b #DAC_Out,&P2OUT ; Разряд конденсатора
                                jmp     Meas_
Low1    bis.b #DAC_Out,&P2OUT ; Заряд конденсатора
                                setc
                                dadc    ADCData
                                ; увеличение ADCData
                                ; в двоично-десятичном
                                ; коде
Meas_   dec     R15
                                ; Уменьшение счетчика
                                ; цикла слежения
                                jnz     Test_DAC
                                ; цикл слежения
                                ; закончен?
                                bic.b #DAC_Out,&P2OUT ; разряд конденсатора
                                xor.b #CAEX,&CACTL1 ; инвертирование выво-
                                ; дов компаратора
                                ret
                                ; Выход из подпрограммы

```

Затем вызывается подпрограмма TX_2_PC, осуществляющая передачу данных в персональный компьютер со скоростью 9600 бод. После этого опять начинает выполняться основная программа.

3.1. Программное обеспечение для основного 12 битного АЦП

Ниже приведена программа, которая может использоваться для реализации основного пропорционального напряжению питания 12 битного АЦП. Программа почти идентична демонстрационной программе, за исключением выполнения двух двоичных выборок (при переключенных входах компаратора) и усреднения. Результат, преобразования сохраняется в регистре ADCData и лежит в пределах от 0000 до 0FFFh.

```

;-----
Meas_ADC ; Подпрограмма: измерения при помощи
          ; АЦП, результат преобразования АЦП
          ; в двоичном коде → ADCData
          ; R15 используется в качестве рабочего
          ; регистра и не сохраняет свое состоя-
          ; ние
;-----
          mov.b #CAON,&CACTL1 ; Включение компаратора

```



```

        clr    ADCData          ; Очистка регистра
                                ; ADCData
        call   #Sample_ADC      ; выборка → ADCData
        call   #Sample_ADC      ; выборка → ADCData +
                                ; предыдущее значение
        rra     ADCData         ; ADCData/2
Meas_Over clr.b &CACTL1        ; Отключение компара-
                                ; тора
        ret                     ; Выход из подпрограммы
                                ;
Sample_ADC mov #04096,R15      ; 12-бит
; предварительный заряд конденсатора ;
Pre_ADC bis.b #DAC_Out,&P2OUT   ; Заряд конденсатора
C1      bit.b #CAOUT,&CACTL2    ; На выходе компарато-
                                ; ра высокий или низ-
                                ; кий сигнал?
        jz     C1              ;
; Настройка и накопление импульсов ЦАП ;
Test_DAC bit.b #CAOUT,&CACTL2   ; На выходе компарато-
                                ; ра высокий или низ-
                                ; кий сигнал?
High    jnc     Low1           ;
        bic.b  #DAC_Out,&P2OUT   ; Разряд конденсатора
        jmp    Meas_          ;
Low1    bis.b  #DAC_Out,&P2OUT   ; Заряд конденсатора
        nop                     ;
        inc    ADCData         ; Увеличение ADCData
Meas_   dec    R15             ; Уменьшение счетчика
                                ; цикла слежения
        jnz    Test_DAC        ; Цикл слежения
                                ; закончен?
        bic.b  #DAC_Out,&P2OUT   ; Разряд конденсатора
        xor.b  #CAEX,&CACTL1    ; Инвертирование выво-
                                ; дов компаратора
        ret                     ; Выход из подпрограммы

```

3.2. Использование стандартных процедур компании TI

Код для этого примера применения был разработан при помощи MSP-FET430X11X и встроенных инструментальных средств интегрированной среды программирования. Код программы написан на ассемблере (.s43). Все примеры программ этого примера применения используют стандартные процедуры

компании TI. Приведенная ниже команда копирует зарегистрированные стандартные процедуры в исходный код:

```
*include <msp430x11x1.h>
```

Компания TI поощряет использование программистами стандартных процедур, что позволяет объединить разработчиков в единое сообщество и упростить процедуру разработки и отладку прикладных программ.

4. Заключение

Современная 16-разрядная RISC архитектура MSP430 очень хорошо подходит для устройств, манипулирующих с данными, имеющими большую разрядность. 16-разрядная архитектура MSP430 позволяет при помощи одной команды манипулировать АЦП с высоким разрешением, используемым в данном примере применения.

Технология аналого-цифрового преобразования, описанная в данном примере применения очень проста, недорога и не требует больших ресурсов MSP430. Это позволяет изготавливать дешевые АЦП с высоким разрешением. Эта технология интегрирующего АЦП не обеспечивает высокого быстродействия. После того, как входное и выходное напряжения сравниваются преобразование выполняется при работе на частоте 3.578545 МГц за 13 мс. Для снижения длительности цикла преобразования можно, если преобразование прошло успешно, оставлять накопительный конденсатор заряженным или использовать более высокую рабочую частоту микроконтроллера MSP430. Максимальная рабочая частота MSP430F11x1 намного выше чем 3.578545 МГц, использованная в данном примере применения. Быстродействия интегрирующих АЦП достаточно для измерения сигналов от телеметрических датчиков, систем сбора данных и других датчиков медленно меняющихся физических величин (температура, давление и т.д.).

Имеются и дополнительные преимущества использования интегрирующих ЦАП с коротким периодом симметричного импульса. Аналого-цифровое преобразование, выполняющееся на основе обычной ШИМ, ШИМ с одиночным большим периодом или переменного цикла, требует RC цепочки с относительно большим временем преобразования. Отклик на короткие импульсы, применяющиеся в данном примере применения, устанавливается быстрее, что позволяет использовать RC цепочку с меньшей постоянной времени. Поэтому интегрирующий АЦП фактически накапливает несколько однобитных приращений, что позволяет снизить ошибки, вызванные шумами.

```

#include <msp430x11x1.h>
;*****
NAME ;FET_intADC MSP430F11x1 Demo
; Описание: Эта программа демонстрирует как реализо-
; вать на микроконтроллере MSP430F11x1 милливольтметр,
; использующий компаратор comparator_A. Напряжение
; считывается и передается в ПК со скоростью 9600 бод
; по протоколу 8N1.
; Используемые регистры вычислительного ядра
#define ADCData      R11
#define BitCnt       R6
#define RXTXData     R5
;
; Условия для обеспечения скорости HW/SW UART 9600 бод
; при MCLK = 3.579545MHz
Bitime_5 equ      0186 ; длительность половины бита
Bitime    equ      0373 ; 104 мкс
TXD       equ      002h ; TXD подключен к порту P1.1
DAC_Out   equ      001h ; к порту P2.0 подключен
; выход ЦАП
CR        equ      0dh ; ASCII символ перевода
; каретки

;
;                               MSP430F1121
;
;
;      3.3В|-----XIN|-----
;      |   |         |   | 3.58 МГц
;      ---|RST   XOUT|---
;
;      AIN--->|P2.4   |
;
;      + -470 к-|P2.0   |
;
;      |         |         | 9600 8N1
;      +-----|P2.3   P1.1|----->
;
;      ==.047   |         |
;
;      |         |         |
;      -----|VSS     |
;
;
; Автор M.Buccini
; Компания Texas Instruments, Inc
; Сентябрь 2000 г.
;*****

```

```

;-----
                ORG    010FEh    ; Информационная память
;-----
VCC_Cal DW 03300                ; Актуальное напряжение
                                ; питания (VCC)

;-----
                ORG    0F000h
;-----
RESET  mov     #300h,SP    ; Инициализация указателя
                                ; стека
        call   #Init_Sys  ; Инициализация системных
                                ; периферийных модулей
;
Mainloop call #Meas_ADC    ; Преобразование АЦП,
        call   #TX_2_PC    ; Пересылка результата
                                ; измерения АЦП в ПК или
                                ; пользователю
Delay  push    #0FFFFh    ; Программная задержка до TOS
L1     dec     0(SP)       ; Уменьшение TOS
        jnz    L1          ; Задержка окончилась?
        incd   SP          ; Очистка TOS
        jmp    Mainloop   ; Повтор

;-----
Init_Sys; Инициализация системных периферийных модулей
;-----
        mov     #WDTWP+WDTHOLD,&WDTCTL ; Остановка
                                ; сторожевого таймера
SetupC0 mov     #OUT,&CCTL0 ; TXD освобожден для
                                ; маркировки
SetupP1_2 bis.b #TXD,&P1SEL ; P1.1/TA0 для функции TXD
        bis.b #TXD,&P1DIR ; P1.1 = выход TXD
        bis.b #DAC_Out,&P2DIR ; P2.0 = выход ЦАП
SetupCA mov.b #P2CA0+P2CA1,&CACTL2 ; P2.3+ P2.4-
SetupBC bis.b #XTS,&BCSCTL1 ; ACLK = LFXT1 HF XTAL
                                ; резонатор

```

```

SetupOsc bic.b #OFIFG,&IFG1      ; Очистка флага пов-
                                   ; реждения автогенера-
                                   ; тора
        mov    #0FFh,R15
SetupOsc1 bit.b #OFIFG,&IFG1      ; Флаг повреждения
                                   ; автогенератора
                                   ; установлен?
        jnz    SetupOsc          ;
        dec    R15               ; дополнительная
                                   ; задержка для убежде-
                                   ; ния в запуске
        jnz    SetupOsc1
        bic.b  #OFIFG,&IFG1      ; Очистка флага
                                   ; повреждения автоге-
                                   ; нератора
        bis.b  #SELM1+SELM0,&BCSCTL2 ; MCLK = LFXTL1
                                   ;
        mov    #TASSEL0+TACLR,&TACTL ; Очистка Timer_A,
                                   ; ACLK, TAR
        bis    #MC1,&TACTL       ; Запуск ТА в непре-
                                   ; рывном режиме
        eint                    ; Глобальное разреше-
                                   ; ние прерываний
        ret                     ; Выход из подпрограммы

;-----
Meas_ADC ; Подпрограмма: измерения при помощи АЦП,
        ; результат преобразования АЦП
        ; в двоично-дополнительном коде → ADCData
        ; R15 используется в качестве рабочего регис-
        ; тра и не сохраняет свое состояние
;-----
        mov.b  #CAON,&CACTL1     ; Включение компаратора
        clr    ADCData           ; Очистка регистра
                                   ; ADCData
        call   #Sample_ADC       ; результат выборки →
                                   ; ADCData

```

```

        call    #Sample_ADC      ; результат выборки →
                                   ; ADCData + предыдущие
                                   ; данные
Meas_Over clr.b &CACTL1          ; Выключение компара-
                                   ; тора
        ret                    ; Выход из подпрограммы
                                   ;
Sample_ADC mov  &VCC_Cal,R15     ; ~3300
        rra     R15              ; VCC/2
                                   ; предварительный
                                   ; заряд конденсатора;
Pre_ADC bis.b  #DAC_Out,&P2OUT    ; заряд конденсатора
C1      bit.b  #CAOUT,&CACTL2     ; На выходе компарато-
                                   ; ра сигнал высокий
                                   ; или низкий?
        jz     C1                ;
; Настройка и накопление импульсов ЦАП;
Test_DAC bit.b #CAOUT,&CACTL2     ; На выходе компарато-
                                   ; ра сигнал высокий
                                   ; или низкий?
        jnc    Low1              ;
High    bic.b  #DAC_Out,&P2OUT    ; Разряд конденсатора
        jmp    Meas_             ;
Low1    bis.b  #DAC_Out,&P2OUT    ; Заряд конденсатора
        setc                    ;
        dadc   ADCData           ; увеличение ADCData в
                                   ; двоично-десятичном
                                   ; коде
Meas_    dec   R15               ; Уменьшение счетчика
                                   ; цикла слежения
        jnz    Test_DAC          ; цикл слежения закон-
                                   ; чен?
        bic.b  #DAC_Out,&P2OUT    ; разряд конденсатора
        xor.b  #CAEX,&CACTL1     ; инвертирование
                                   ; выводов компаратора
        ret                    ; Выход из подпрограммы

```

```

;-----
TX_2_PC ; Поресылка CR и 16-битного слова ADCData
        ; (регистр R11) в ПК/пользователю
;-----
        mov    #CR,RXTXData    ; пересылка CR в буфер
                                ; UART
        call   #TX_Byte        ; CR → ПК/пользователю
;
; Переход к нижеследующей подпрограмме
;
;-----
TX_Word_ASCII ; Передача слова из ADCData в виде
              ; четырех ASCII байт
;-----
        swpb   ADCData          ; ADCData = 3412
        call   #TX_Byte_ASCII   ;
        swpb   ADCData          ; ADCData = 1234
;-----
TX_Byte_ASCII ; Передача байта из ADCData в виде двух
              ; ASCII байт
;-----
        push   ADCData          ; передача значения
                                ; ..x
        call   #NUM_ASCIR       ;
        push   ADCData          ; передача значения
                                ; ...x
        call   #NUM_ASCIA       ;
        ret                    ; Выход из подпрограммы
;
NUM_ASCIR    ; Преобразование числа от 0 до F
              ; в ASCII символ с левым
              ; выравниванием 2 (SP)
        rrc    2(SP)            ; 1. и 3. пропускаются
        rrc    2(SP)            ;
        rrc    2(SP)            ;
        rrc    2(SP)            ;
                                ;

```

```

NUM_ASCIA    and #0fh,2(SP)    ; 2. и 4. пропускаются
              add #030h,2(SP)    ;
              cmp #03ah,2(SP)    ;
              jlo NUM_End        ;
              add #039,2(SP)     ;
NUM_End mov 2(SP),RXTXData      ; загрузка буфера
                                   ; передатчика, FALL
              mov @SP+,0(SP)     ; Очистка w\ возврат
                                   ; адреса TOS

;-----
TX_Byte ; Подпрограмма, которая передает байт, находя-
        ; щийся в буфере RXTXData.
;-----
              mov #TX_Count,BitCnt ; TX_Count → Указатель
                                   ; перехода
              push &TAR           ; Текущее значение ТА
              add #Bitime,0(SP)   ; Сдвиг к следующему
                                   ; биту
              pop &CCR0           ; Время для следующего бита
                                   ; в CCR0
              mov #OUTMOD2+OUTMOD0+CCIE,&CCTL0
                                   ; TXD = пустой старт бит
TX_Wait bit #CCIE,&CCTL0 ; Ожидание окончания
                                   ; передачи
              jnz TX_Wait         ;
              ret                 ; Выход из подпрограммы

;-----
TA0_ISR ; Подпрограмма обработки прерывания CCR0/UART
ISR: буфер RXTXData содержит
;      данные для UART.
;-----
              add #Bitime,&CCR0    ; Bitime до следующего
                                   ; бита
              mov @BitCnt+,PC      ; Ветвление программы
                                   ;
TX_Bit rra.b RXTXData             ; LSB сдвигается в бит
                                   ; переноса

```



```

        jc      TX_Mark      ; Переход если бит = 1
TX_Space bis    #OUTMOD2,&CCTL0 ; Передача нуля
        reti
TX_Comp  bic     #CCIE,&CCTL0   ; Все биты приняты,
                                ; запрет прерывания
TX_Mark  bic     #OUTMOD2,&CCTL0 ; передача 1
        reti
                                ;
                                ;
        even
TX_Count DW     TX_Bit        ; Передача первого
                                ; бита
        DW     TX_Bit        ;
        DW     TX_Bit        ;
        DW     TX_Bit        ;
        DW     TX_Bit        ;
        DW     TX_Bit        ;
        DW     TX_Bit        ;
        DW     TX_Bit        ;
        DW     TX_Mark      ; передача стоп
                                ; бита = 1
TX_End   DW     TX_Comp      ; Передача завершена
                                ;

;-----
RSEG INTVEC ; вектора прерываний MSP430x11x1
;-----
        DW     RESET        ; нет источника
        DW     RESET        ; нет источника
        DW     RESET        ; P1.x
        DW     RESET        ; P2.x
        DW     RESET        ; нет источника
        DW     RESET        ; нет источника
        DW     RESET        ; нет источника
        DW     RESET        ; нет источника
        DW     RESET        ; Timer_AX
        DW     TA0_ISR      ; Timer_A0
        DW     RESET        ; Сторожевой таймер / Таймер
        DW     RESET        ; Компаратор Comparator_A

```

```

DW      RESET      ; нет источника
DW      RESET      ; нет источника
DW      RESET      ; NMI, повреждение
                        ; автогенератора
DW      RESET      ; POR, внешний сброс,
                        ; сторожевой таймер
END

```

Ссылки:

1. MSP430x11x1 Data Sheet (SLAS241C)
2. MSP430x1xx Family Users Guide (SLAU049)
3. MSP430 Application Report, Lutz Bierl (SLAA024)
4. Implementing a UART Function with Timer_A3, Mark Buccini, (SLAA078)

IV. Примеры конкретных применений

Глава 8.

*Недорогой однофазный
счетчик электроэнергии на MSP430C11x*

Недорогой однофазный счетчик электроэнергии на MSP430C11x

Kes Tam [slaa075.pdf (Англ.)]

В данном примере применения описывается построение недорогого однофазного счетчика электроэнергии на базе микроконтроллера MSP430C11x, позволяющего обеспечить аналого-цифровое преобразование, внутреннюю калибровку и защиту от внешнего вмешательства.

1. Введение

В течение многих десятилетий для измерения потребления электроэнергии использовались механические счетчики электроэнергии (электросчетчики). Новые, полностью электронные электросчетчики заменяют механические электросчетчики, но до сих пор массовое распространение электросчетчиков тормозилось из-за их высокой стоимости, что было вызвано тем, что в них использовались аналого-цифровые преобразователи (АЦП) и микропроцессор.

MSP430C11x — дешевый член семейства MSP430 микроконтроллеров производства компании Texas Instruments. Он не имеет встроенных аппаратных блоков типа АЦП и перемножителей, которые обычно применялись в электронных электросчетчиках. Однако, наличие у него трех регистров захвата данных позволяет реализовать три дешевых АЦП с высоким разрешением. А его высокопроизводительное 16-разрядное вычислительное ядро с RISC архитектурой позволяет реализовать высокоточный дешевый полностью электронный электросчетчик. Вот некоторые из основных характеристик этого электросчетчика:

- Трехканальный градиентный АЦП, использующий встроенный таймер захвата
- Двухдиапазонный токовый вход с автоматическим управлением, обеспечивающим перекрытие тока до 400 %
- Требуемая для измерителей первого класса высокая точность (ошибка менее 1%)
- Защита от внешнего вмешательства, позволяющая предотвратить воровство электроэнергии
- Простая и легкая внутренняя калибровка
- Однокристальное решение, не требующее внешнего вычислителя
- Устранение смещения по постоянному току в обоих каналах (напряжения и тока)
- ФНЧ для мгновенных значений мощности

Преобразование потребляемой энергии в частоту импульсной последовательности

2. Аналого-цифровое преобразование

В электросчетчиках должны применяться специальные каскады АЦП. Для вычисления мощности в реальном времени необходимо одновременно выбирать мгновенные значения напряжения и тока. Наиболее простой способ одновременного выбора напряжения и тока описан в данном примере применения. В описываемом устройстве используется трехканальный АЦП (назначение третьего канала будет объяснено ниже). При использовании технологии одно-градиентного АЦП становится возможным создание рентабельного трехканального АЦП с высоким разрешением (11 бит) и адекватной частотой дискретизации (1 000 kSPS на канал).

2.1. Одно-градиентный АЦП

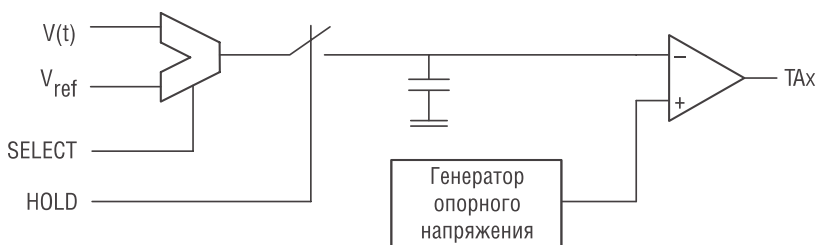


Рис. 1. Одно-градиентный АЦП

Для аналого-цифрового преобразования используется стандартная одно-градиентная технология. Аналоговый мультиплексор в первом цикле выбирает сигнальный вход, а во втором – опорный сигнал. После выборки сигнал сравнивается с опорным пилообразным сигналом. В момент, когда пилообразный сигнал достигает значения выбранного мультиплексором сигнала, происходит переключение таймера захвата. Таким образом, имея временные интервалы, пропорциональные величинам входного и опорного напряжений, мы можем рассчитать величину входного напряжения по формуле:

$$V[n] = V_{\text{ref}} \times \left(\frac{t_V}{t_{\text{ref}}} \right)$$

Где $V[n]$ – последовательность, состоящая из мгновенных значений V ($n \times t_c$).

2.2. Формирование пилообразного сигнала

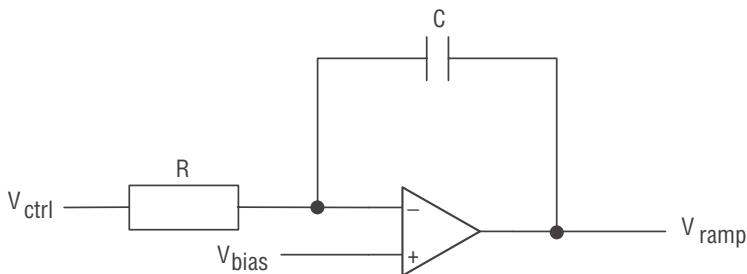


Рис. 2. Генератор пилообразного сигнала

Генератор пилообразного сигнала построен на базе интегратора на основе операционного усилителя (ОУ). При подаче на вход постоянного напряжения, через емкость C потечет постоянный ток, равный $(V_{bias} - V_{ctrl})/R$. Этот ток сформирует на выходе ОУ линейное пилообразное напряжение.

2.2.1. Период пилообразного сигнала

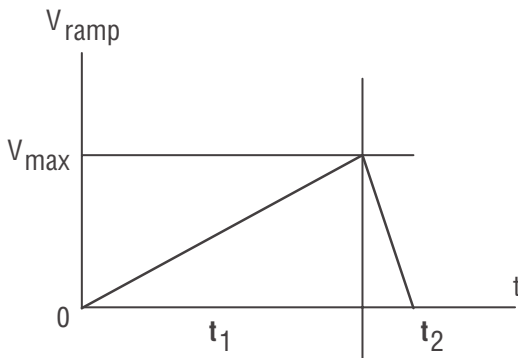


Рис. 3. Период пилообразного сигнала

Рассмотрим рисунок 3: если V_{ctrl} устанавливается в ноль в момент времени $t = 0$, то на выходе операционного усилителя V_{ramp} начинает формироваться пилообразный сигнал, начинающийся с нуля. В момент времени t_1 , на входе V_{ctrl} устанавливается напряжение питания V_{cc} , и выходное напряжение V_{ramp} , достигнув своего максимального значения V_{max} , начинает пилообразно уменьшаться. И

за период времени, равный t_2 , на выходе V_{ramp} снова установится нулевое значение. Подав на вход V_{ctrl} нулевой потенциал, мы запустим цикл заново.

Если $C = Q / V_c$,

тогда $C = I_c / (dV_c/dt)$,

или $C = (V_{\text{bias}} - V_{\text{ctrl}}) / R / (d(V_{\text{ramp}} - V_{\text{bias}})/dt)$,

или $\Delta t = RC \times \Delta V_{\text{ramp}} / (V_{\text{bias}} - V_{\text{ctrl}})$,

следовательно $t_1 = RC \times V_{\text{max}} / V_{\text{bias}}$,

а $t_2 = RC \times V_{\text{max}} / (V_{\text{cc}} - V_{\text{bias}})$

Таким образом, длительность цикла будет равняться:

$$t_c = t_1 + t_2 = RC \times V_{\text{max}} \times \left(\frac{1}{V_{\text{bias}}} + \frac{1}{(V_{\text{cc}} - V_{\text{bias}})} \right)$$

$$t_c = t_1 + t_2 = RC \times V_{\text{max}} \times ((1 / V_{\text{bias}}) + (1 / (V_{\text{cc}} - V_{\text{bias}})))$$

Это уравнение позволяет выбрать значение RC по заданной частоте дискретизации. Кроме того, следует учитывать следующие замечания:

Для максимизации входного динамического диапазона V_{max} должен быть как можно большим. Оно должно равняться выходному напряжению насыщения ОУ или максимальному входному напряжению компаратора (если оно меньше напряжения насыщения).

Из рисунка 2 видно, что V_{bias} должно быть как можно меньше. Для одно-градиентного АЦП нас интересует только отрезок нарастания пилообразного сигнала, а поэтому время спада должно быть минимальным (необходимо обеспечить $t_1 \gg t_2$). Маленькое значение V_{bias} позволяет получить большое t_1 и маленькое t_2 .

Небольшое улучшение может быть достигнуто путем выбора немного меньшего значения RC , чем было получено в результате расчета. Это позволяет гарантировать при заряде и разряде начальные условия 0 и V_{max} . Неизвестные начальные условия могут вызвать смещение пилообразного сигнала вверх или вниз.

Из-за программной регулировки задержки фактическая сумма $t_1 + t_2$ будет меньше, чем t_c . Это другая причина, вызывающая необходимость обеспечения меньшего значения RC .

2.2.2. Практический пример

$V_{\text{cc}} = 5 \text{ В}$

$V_{\text{max}} = 3.5 \text{ В}$

$V_{\text{bias}} = 0.6 \text{ В}$

Частота дискретизации = 995 Гц $\Rightarrow t_c = 1 / (2 \times 995)$

Если выбрать $C = 1000$ пФ, то получаем $R = 75.8$ кОм. Для снижения значения RC выберите $R = 68$ кОм (или даже 62 кОм).

2.2.3. Измерение временных параметров пилообразного сигнала

Таймер захвата MSP430 может быть легко настроен на фиксацию данных по выходному сигналу компаратора. Но нет никакого внешнего аппаратного управления для запуска таймера в начале периода пилообразного сигнала. В нашем случае начало пилообразного сигнала фиксируется путем сравнения пилообразного сигнала с небольшим фиксированным пороговым напряжением. Время нарастания пилообразного сигнала до порогового значения определяется при подаче на вход опорного сигнала.

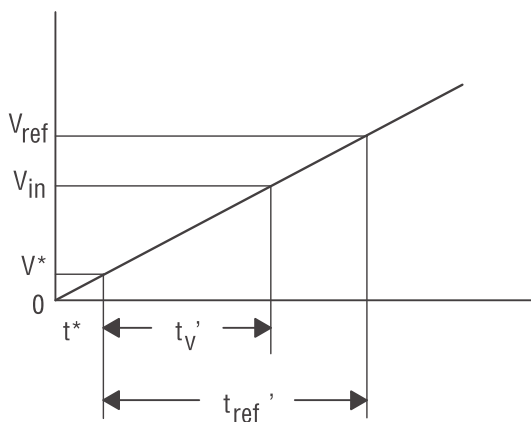


Рис. 4. Измерение временных параметров пилообразного сигнала

Из рисунка 4 получаем:

$$V_{ref} / (t_{ref}' + t_t) = V_t / t_t$$

$$\text{Отсюда следует } t_t = t_{ref}' \times (V_t / (V_{ref} - V_t))$$

$$\text{Или } t_t = t_{ref}' \times K_t$$

$$\text{Где } K_t = V_t / (V_{ref} - V_t)$$

Как видно из формулы, после измерения необходимо производить умножение, поэтому рекомендуется выбрать параметры таким образом, чтобы K_t было простым значением, что позволит заменить простым сдвигом или суммированием/вычитанием подпрограмму вычисления произведения, требующую высокой производительности микроконтроллера.

2.3. Двухдиапазонный АЦП для канала измерения тока

Среднее квадратичное значение напряжения на входе изменяется в небольших пределах. Однако среднее квадратичное значение тока на входе изменяется в широких пределах в зависимости от величины и типа нагрузки. Из этого следует, что необходимо применять АЦП с высоким разрешением или АЦП с низким разрешением, но изменяющимся входным динамическим диапазоном. Чтобы перекрыть входной диапазон от 5 % до 400 % с ошибкой измерения не более $\pm 1\%$ требуется обеспечить минимальный динамический диапазон $400/5 \times 100 / (1+1) = 4000$. Для этого требуется АЦП с разрешением не менее 12 бит. Для одно-градиентного АЦП имеется зависимость между разрешающей способностью, частотой дискретизации и шумами. Наиболее удобно работать при 10 битном разрешении. 12 битное разрешение может быть обеспечено за счет добавления внешнего четырехкратного усиления для малых сигналов. Даже с учетом того, что реальная разрешающая способность равна 10 битам, в описываемом устройстве обеспечивается эффективная разрешающая способность, равная приблизительно разрешающей способности 11 битного АЦП.

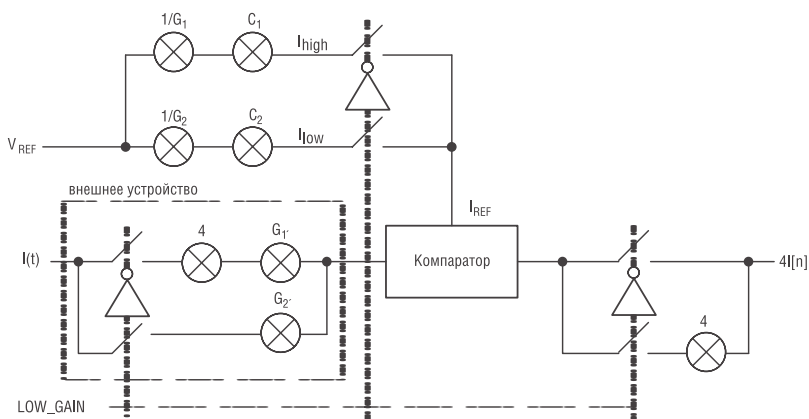


Рис. 5. Структурная схема двухдиапазонного АЦП

На рисунке 5 показана структурная схема двухдиапазонного АЦП. Он состоит из однодиапазонного АЦП и внешнего устройства, обеспечивающего четырехкратное усиление малых сигналов. Теоретически, коэффициенты усиления всех перемножителей G_1 , G_2 , $1/G_1$ и $1/G_2$ должны равняться едини-

це, однако на практике обеспечить это не удастся из-за разброса параметров внешних дискретных компонентов. Для компенсации этого расхождения вводят дополнительные узлы C1 и C2. Это позволяет увеличить согласованность каналов.

C1 и C2 компенсируют отклонение коэффициента усиления G1' и G2' от коэффициентов усиления G1 и G2. Поэтому стараются добиться истинности равенств $C1 = G1/G1'$ и $C2 = G2/G2'$. Так как коэффициент усиления узлов G1' и G2' неизвестен и изменяется от устройства к устройству, то после изготовления необходимо произвести калибровку, а вычисленные значения C1 и C2 должны быть сохранены EEPROM памяти для дальнейшего использования. В нормальном режиме (или режиме малого усиления) нижние на схеме выключатели замкнуты, а верхние – разомкнуты. При использовании в качестве опорного сигнала напряжение Vref получаем:

$$I' [n] = Vref \times (ti / tref).$$

Где:

$$I' [n] = I [n] \times G2',$$

$$\text{a } Iref = Ilow = Vref/G2 \times C2 = Vref/G2'.$$

$$\text{Следовательно } I [n] \times G2' = (Iref \times G2') \times (ti / tref),$$

$$\text{то есть } I [n] = Iref \times (ti / tref)$$

В правой части уравнения записано стандартное равенство для вычисления значения, измеренного при помощи одно-градиентного АЦП при использовании опорного тока Iref. Это означает, что на выходе мы получим $Iad [n] = I [n]$. И окончательный результат: $Iout [[n] = 4 \times Iad [n] = 4I [n]$.

В режиме большого усиления верхние по схеме выключатели замкнуты, а нижние – разомкнуты. Тогда, используя те же выражения, получаем:

$$I' [n] = Vref \times (ti / tref).$$

$$\text{Но теперь } I' [n] = 4I [n] \times G1,$$

$$\text{a } Iref = Ihigh = Vref / G1.$$

$$\text{Следовательно } 4I [n] \times G1 = (Iref \times G1) \times (ti / tref).$$

$$\text{Тогда } 4I [n] = Iref \times (ti / tref)$$

$$\text{или } 4I [n] = Iad [n]$$

$$\text{В результате чего получаем: } Iout [n] = Iad [n] = 4I [n].$$

В обоих случаях мы обеспечили на выходе одинаковые результат и точность. Дополнительное четырехкратное усиление в режиме малого усиления компенсирует внешнее усиление в режиме высокого усиления. Другой способ расширения динамического диапазона состоит в подавлении больших сигналов, при этом малые сигналы не изменяются. Но это нежелательно, так как это

уменьшает амплитуду и количество значащих цифр, что ведет к увеличению ошибки вычисления.

2.4. Автоматическое переключение коэффициента усиления

Переключение коэффициента усиления у двухдиапазонного АЦП должно выполняться автоматически. В начале раздела 2.3, говорилось, что максимальное выходное значение АЦП, равное 4000, соответствует 400 % входного тока. В режиме высокого коэффициента усиления малого входного сигнала 10 битный АЦП перекрывает диапазон, равный 1024, или примерно 100 %. В режиме малого коэффициента усиления больших входных сигналов 10 битный АЦП с коэффициентом усиления, равным 4, способен перекрыть диапазон от 4 до 4096 или приблизительно от 4 % до 400 %. Это означает, что любой сигнал в диапазоне от 4 % до 100 % может быть оцифрован при любом коэффициенте усиления.

В данном примере применения, диапазоны при работе в режиме малого и большого коэффициента усиления перекрывают друг друга. Это позволяет обеспечить гистерезис, исключающий частые переключения между двумя диапазонами. В системе реализованы два диапазона: от 5 % до 80 % для большого коэффициента усиления, и от 60 % до 400 % для малого коэффициента усиления. Выбор этих диапазонов был сделан произвольно, но довольно разумно, так как обеспечивает 20%-ое перекрытие и максимальное значение на 20 % ниже граничного значения (100 %-80 % в режиме большого коэффициента усиления).

3. Защита от внешнего вмешательства, предотвращающая воровство электроэнергии

В некоторых странах воровство электроэнергии путем незаконного вмешательства в работу электросчетчика является серьезной проблемой. Ниже показаны некоторые возможные методы вмешательства.

Основным способом вмешательства является снижение или полное закрытие протекание тока через датчик, установленный на фазе или нейтрале. Если ток взять только от одной линии (фазы или нейтрали), то электросчетчик будет измерять более низкое или вообще нулевое потребление. Самым простым способом предотвращения последствий этих действий является измерение тока в обеих линиях и использование для расчета большего тока. Однако для этого потребуется использовать дополнительный канал аналого-цифрового преобразования. Добавление еще одного канала АЦП обычно ведет к существенному удорожанию системы. К счастью, таймер захвата MSP430C11x обеспечивает три канала АЦП почти за ту же самую стоимость, что и два канала.

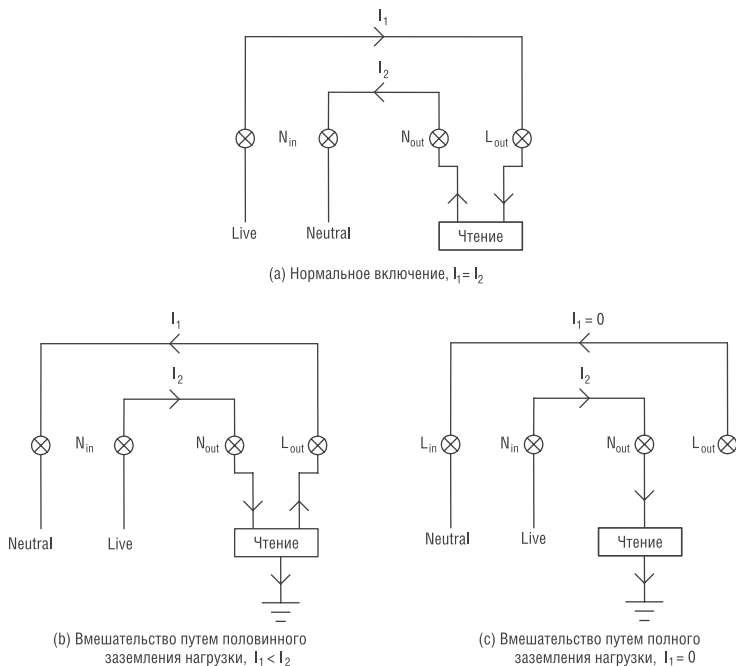


Рис. 6. Примеры вмешательства в работу электросчетчиков

4. Потoki данных в системе

Понимание направления движения и назначения потоков – ключ к пониманию работы системы. На рисунке 7 показаны все внутренние потоки данных во всех узлах за исключением высоковольтного преобразователя и преобразователя энергия-импульс. Сначала от оконечных устройств входное напряжение и оба входных тока после понижения (или повышения) поступают в трехканальный АЦП, где производится их одновременная выборка. В канале измерения напряжения используется однодиапазонный АЦП. А в обоих каналах измерения тока используются АЦП с дополнительными усилителями. Регулятор усиления обеспечивает автоматическую регулировку усиления усилителей обоих АЦП. Схема подавления постоянной составляющей позволяет выделить переменные составляющие напряжения и тока ($V[n]$, $4i1[n]$ и $4i2[n]$). Схема выбора тока обеспечивает подключение того канала, ток в котором больше. При помощи ФНЧ можно выделить среднее значение потребляемой энергии 4VA. Это значение подается на выходной преобразо-

ватель энергия-импульс, который управляет дисковым электромеханическим или другим индикатором.

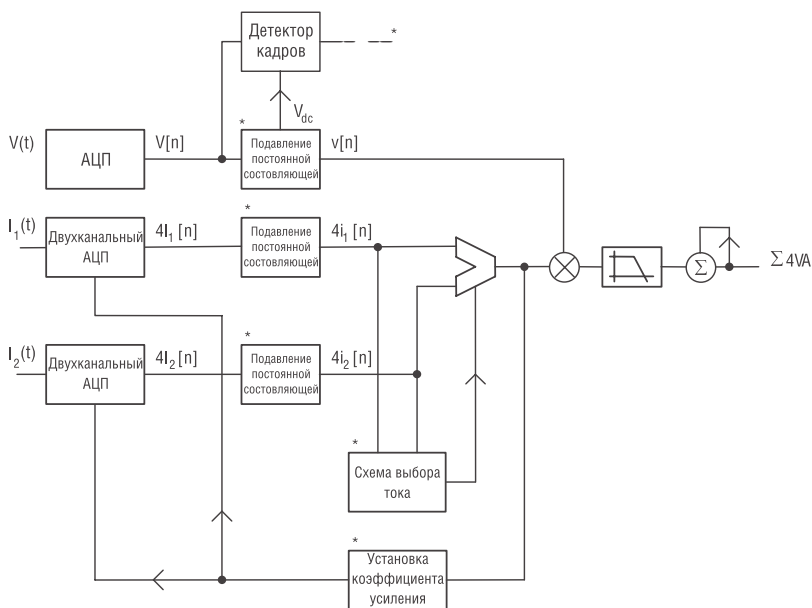


Рис. 7. Структурная схема внутренних потоков данных

4.1. Детектирование кадров

Кадром называется некоторое количество полных периодов входного напряжения. Он определяет период обновления данных в модуле. В конце каждого кадра обновляются значения о постоянной составляющей входных напряжения и тока, происходит выбор канала измерения тока и выбор коэффициента усиления токовых каналов. Количество периодов переменного напряжения определяется по переходам через ноль входного напряжения. Выбор количества периодов в кадре является произвольным, но это количество должно быть достаточно большим. В описываемом устройстве кадр состоит из 90 периодов. Это усредняет возможные ошибки при подавлении постоянной составляющей и вычислении количества периодов. В некоторых случаях уровень постоянной составляющей может превысить амплитуду колебаний. В этом случае для вычисления количества периодов используется временное ограничение.

4.2. Исключение постоянной составляющей

Постоянная составляющая входного сигнала исключается путем вычитания ее из входного сигнала. Для некоторого количества периодов постоянная составляющая равняется среднему значению.

$$\text{То есть. } V_{dc} = \Sigma V [n] / N$$

$$\text{а } I_{dc} = \Sigma I [n] / N$$

Где N – количество отсчетов в кадре.

Если количество периодов в кадре велико, то точность вычисления постоянной составляющей будет довольно велика.

Исключение постоянной составляющей в каналах измерения тока немного сложнее, чем в канале измерения напряжения. Возможно появление обратного тока при реверсивном включении основного входа и нагрузки. Это может привести к отрицательному значению вычисленной мощности. Для предотвращения этого в каждом кадре производится проверка знака мощности ($\Sigma 4VA$). Если полученное значение является отрицательным, то в следующем кадре произойдет изменение знака вычисления токов $4i1 [n]$ или $4i2 [n]$.

Вообще говоря, если имеется возможность нахождения постоянного значения напряжения, то постоянную составляющую тока исключать необязательно.

$$\text{Изначально } v(t) = V_{dc} + V_o \times \sin(\omega t)$$

$$\text{а } i(t) = I_{dc} + I_o \times \sin(\omega t + \theta)$$

$$\text{Тогда } \Sigma vi = \Sigma V_{dc} \times I_{dc} + (V_{dc} \times I_o + I_{dc} \times V_o) \times \Sigma \sin(\omega t) + V_o \times I_o \times \Sigma \sin(\omega t)$$

Так как за период

$$\Sigma \sin(\omega t) = 0$$

$$\text{Тогда } \Sigma vi = \Sigma V_{dc} \times I_{dc} + V_o \times I_o \times \Sigma \sin(\omega t) \sin(\omega t + \theta)$$

Если V_{dc} известно, то

$$\Sigma ((v - V_{dc}) \times i) = V_o \times I_o \times \sin(\omega t) \sin(\omega t + \theta)$$

не содержит постоянной составляющей I_{dc} .

Конечно, всегда имеется небольшая ошибка в вычислении V_{dc} , но она ведет только к небольшому увеличению ошибки вычисления потребляемой энергии. Если такая ошибка допустима, то исключение постоянной составляющей в каналах измерения тока можно не производить.

Однако следует рассмотреть один вопрос относительно постоянной составляющей. В рассматриваемом устройстве наблюдается изменение постоянной составляющей, которое может повлиять на точность. Уровень постоянной составляющей увеличивается в 4 раза в режиме высокого коэффициента усиления. В

результате этого происходит смещение и ограничение входного сигнала. Из-за эффекта насыщения измерение больших входных сигналов будет происходить с ошибкой. Если, например, диапазон АЦП от 0 до 3.5 В, смещение равняется 1.75 В, то 0.2 В смещение по постоянному току в режиме большого коэффициента усиления будет равняться 0.8 В. Это приведет к тому, что напряжение смещения станет равным 2.55 В, а диапазон измеряемых амплитуд уменьшится с 1.75 В до 0.95 В.

4.3. Блок выбора канала тока

Блок выбора канала тока предназначен для защиты от внешнего вмешательства. Внешнее вмешательство может состоять в уменьшении или полном исключении токов одного из каналов, но не токов обоих каналов. Большой ток по-прежнему отображает реальное потребление. Самый простой способ нахождения большего тока состоит в вычислении суммы токов обоих каналов. В конце каждого кадра производится обновление результата вычисления большего тока, после чего производится выбор канала, используемого для дальнейших вычислений.

4.4. Регулятор усиления

Как описано в разделе 2.4, автоматическая регулировка усиления требует обнаружения превышения входным током заданного порога. Рис. 8 показано как это происходит.

Предположим, что требуется обнаружить превышение сигналом некоторого порога y . Самый простой способ состоит в детектировании максимального значения сигнала. Проблема применения этого метода состоит в том, что детектирование усложняется при условии работы в среде с высоким уровнем шума. Для улучшения этого способа производятся выборки сигнала в соседних с максимумом

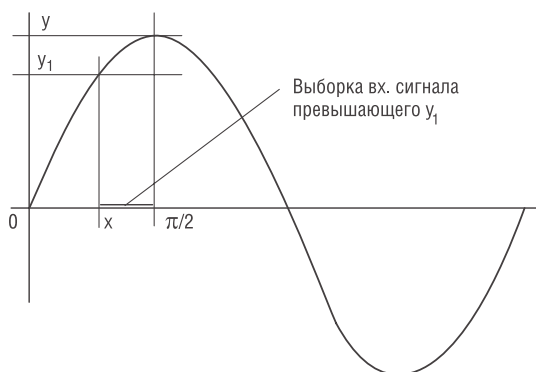


Рис. 8. Пороговое детектирование

точках. Рассматривается порог $y_1 = \sin(x)$, который немного ниже, чем y , при этом входные отсчеты, лежащие в интервале $(\pi/2 - x)$ должны быть больше y_1 . Другими словами, если номер отсчетов, больших y_1 , больше чем $(\pi/2 - x)$, то максимальное значение должно быть больше y . Хотя эта технология не позволяет найти максимальное значение, но она позволяет определить, будет ли превышен порог. В конце каждого кадра идет сравнение количества отсчетов, превысивших порог y_1 с количеством всех отсчетов. Если процент таких отсчетов выше или ниже заданного значения, то коэффициент усиления должен быть изменен.

4.5. Фильтр нижних частот

Последний элемент на рисунке 7 – фильтр нижних частот.

$$\text{Если } v(t) = V_0 \times \sin(\omega t)$$

$$\text{а } i(t) = I_0 \times \sin(\omega t)$$

$$\text{то } p(t) = V_0 \times I_0 \times \sin^2(\omega t) = (1/2 \times V_0 \times I_0) \times (1 - \cos(2\omega t))$$

Из этого выражения следует, что мгновенное значение потребляемой мощности является функцией с постоянной составляющей и переменной составляющей, имеющей удвоенную частоту колебаний. Размах колебаний в два раза превышает среднее значение мощности. Но при долгосрочном накоплении это не имеет никакого значения, так как за длительный период будет произведено усреднение. ФНЧ только повышает устойчивость системы и не влияет на общую точность.

5. Преобразователь энергия-импульс

Коэффициент усиления между окончательным каскадом и низковольтным АЦП, а также внутренние масштабирующие коэффициенты (и/или другие константы) должны быть известны при реальных вычислениях электроэнергии.

Если на выходе окончательного высоковольтного преобразователя мы имеем

$$V_h(t) = K_v \times v(t)$$

$$I_{1h}(t) = K_i \times i_1(t)$$

$$I_{2h}(t) = K_i \times i_2(t)$$

а при внутренних вычислениях производится масштабирование с коэффициентом K_s :

$$V(t) = K_s \times v(t)$$

$$I_1(t) = K_s \times i_1(t)$$

$$I_2(t) = K_s \times i_2(t)$$

$$\text{то есть } V_h(t) = K_v / K_s \times V(t)$$

$$I_{1h}(t) = K_i / K_s \times I_1(t)$$

$$I_{2h}(t) = K_i / K_s \times I_2(t)$$

тогда потребляемая мощность:

$E = \sum V_h \times A_h \times t_c$, где t_c – период выборки, а A_h – наибольший из токов I_{1h} или I_{2h}

$= \Sigma VA \times Kv \times Ki / Ks2 \times tc$, где A – наибольший из токов $I1$ или $I2$

$= \Sigma Es / 4 \times Kv \times Ki / (Ks2 \cdot fs)$, где $Es = 4VA$ и $fs = 1/tc$ – частота дискретизации

Потребляемую электроэнергию принято отображать с точностью до 0.1-кВт*ч (Киловатт-час)

Тогда $Ed = E / (0.1 \times 1000 \times 3600) = \Sigma Es / \delta$, где $\delta = 0.1 \times 1000 \times 3600 \times 4 \times Ks2 \times fs / (Kv \times Ki)$

Очевидно, что δ – коэффициент преобразования измеренной энергии $Es = 4VA$. Если каждую единицу δ представить в виде одного импульса, то вычисленную мощность можно отобразить на дисплее счетчика импульсов. Использование δ в качестве порога существенно упрощает разработку импульсного генератора. На рисунке 9 приведена блок – схема программы преобразователя энергия-импульс.

После каждой выборки сигнала происходит вычисление потребленной энергии $Es = 4VA$, это значение добавляется к вычисленной ранее сумме, и если она становится выше порога δ , то происходит вычитание из суммы значения δ и формирование импульса. По этому импульсу индикатор увеличивает свое значение на единицу.

Обычно в качестве индикатора применяется электромеханический дисковый счетчик. Однако он может быть заменен программой, отображающей значение на ЖКИ или светодиодном индикаторе.

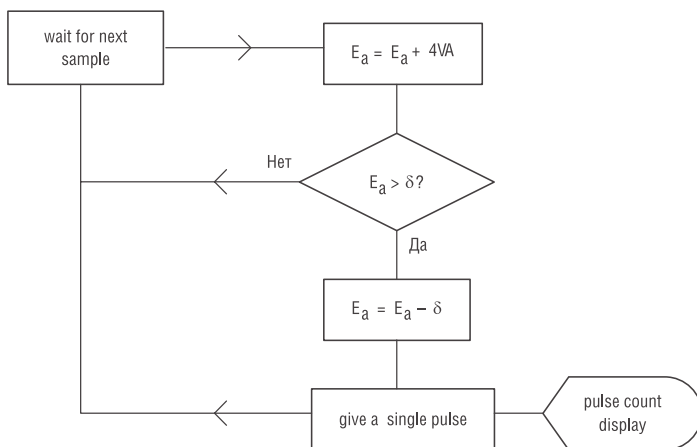


Рис. 9. Блок-схема программы управления увеличивающимся индикатором потребленной энергии

Если требуется более высокая точность измерения, то порог δ может быть уменьшен, что приведет к повышению частоты следования импульсов при том же потреблении.

6. Калибровка

Хотя большинство резисторов, номинал которых влияет на коэффициент усиления, имеют высокую точность, калибровка устройства все равно должна быть произведена. Несмотря на то, что каналы измерения напряжения и тока разделены, калибровка должна производиться при помощи измерителя мощности. Так как имеется два канала измерения тока, то получаем четыре точки калибровки. Такие операции дороги и трудоемки. Поэтому автоматический метод калибровки позволяет упростить устройство.

Идея метода калибровки состоит в измерении времени формирования заданного количества импульсов эталонным измерителем, подключенным параллельно. Отношение времен позволит скорректировать ошибку измерения. Коэффициенты коррекции могут быть записаны в EEPROM память и использованы при дальнейших вычислениях.

7. Заключение

В данном примере применения описано построение дешевого электросчетчика на базе микроконтроллера MSP430C11x. Существуют и более сложные устройства, но они более дороги, так как не имеют дешевых вычислительных микроконтроллеров и одно-градиентных АЦП. В данном примере применения дешевый АЦП на основе встроенного таймера и мощное вычислительное ядро микроконтроллера позволяют справиться с задачей сложных вычислений. MSP430C11X, примененный в описанном электросчетчике, позволяет обеспечить высоко конкурентную цену.

Ссылки:

1. Bureau of Indian Standards, AC Static Watthour Meters, Class 1 and 2 Specification, Reference number ET13 (1379)
2. MSP430 Family, Architecture Guide and Module Library User's Guide, Literature Number SLAUE10B
3. MSP430 Family, Metering Application Report, Literature Number SLAAE10C
4. MSP430 Family, Software User's Guide, Literature Number SLAUE11
5. MSP430 Family, Assembly Language Tools User's Guide, Literature Number SLAUE12

IV. Примеры конкретных применений

Глава 9. *Цифровой термометр на базе MSP430*

Цифровой термометр на базе MSP430

Brian Merritt [slaa038.pdf (Англ.)]

В этом примере описывается разработка цифрового термометра с использованием возможностей slope-аналогово-цифрового преобразования на базе модуля таймера-порта Timer Port микроконтроллеров MSP430x3xx. В общем плане можно рассматривать данную разработку как пример подключения резистивных датчиков и опорных резисторов к модулю таймера – порта Timer Port.

1. Введение

В этом примере описывается разработка цифрового термометра с использованием возможностей slope-аналогово-цифрового преобразования на базе модуля таймера-порта Timer Port микроконтроллеров MSP430x3xx. В общем плане можно рассматривать данную разработку как пример подключения резистивных датчиков и опорных резисторов к модулю таймера – порта Timer Port. Такой модуль входит в состав всех микроконтроллеров серии MSP430x3xx. Таймер – порт позволяет подключать несколько резистивных датчиков и опорных резисторов, как и сделано в данном приборе. Неиспользуемые выводы модуля могут быть использованы в качестве независимых выходов.

1.1. Краткое описание микроконтроллеров со сверхнизким энергопотреблением семейства MSP430

MSP430 это 16-разрядный микроконтроллер с RISC-архитектурой, обладающий дополнительными функциями, облегчающими разработку и ортогональной структурой, обеспечивающей ядру повышенную вычислительную мощность наряду с высокой гибкостью использования. Эти особенности позволяют микроконтроллеру MSP430 потреблять всего 400 мкА в активном режиме в типовой 3-вольтовой схеме. MSP430 потребляет только 2 мкА в спящем режиме и выходит из него в полностью синхронизированный активный режим не более, чем за 6 мкс. Подсемейства серии MSP430 включают различные наборы периферийных модулей, представляя собой высокоинтегрированные системы. На рис.1 показана блок-диаграмма микроконтроллеров MSP430x32x.

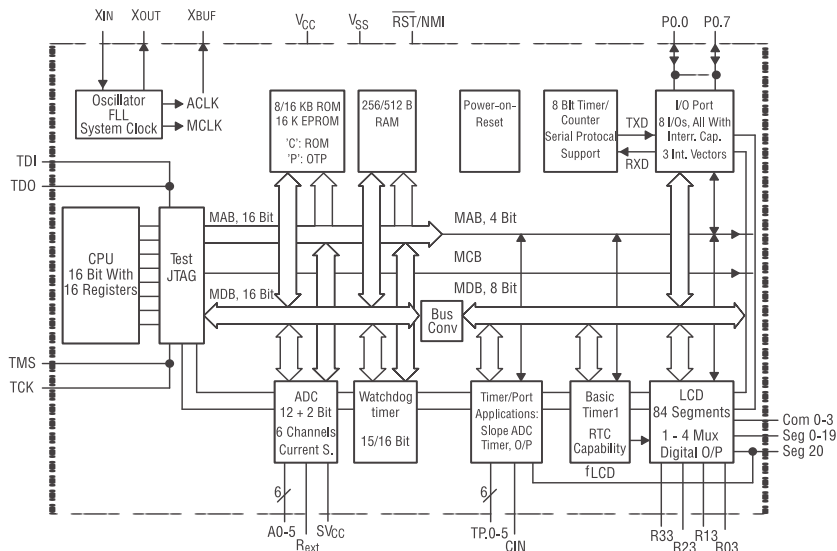


Рис. 1. Фазовая характеристика СТ

1.2. Описание схемы

Схема состоит из термистора (Radio Shack #271–110), опорного резистора сопротивлением 10кОм и конденсатора 0.1 мкФ. Детали подключаются непосредственно к MSP430, как показано на рис.2. Если требуется визуальное отображение информации, то может быть подключен ЖКИ. Прибор осуществляет измерения путём заряда конденсатора до уровня примерно равного V_{CC}, последующего разряда его через опорный резистор с одновременным подсчётом внутренних тактов до того момента, пока на входе CIN не появится лог. «0». Далее конденсатор снова заряжается до значения, близкого к V_{CC} и разряжается через термистор, при этом также подсчитываются тактовые импульсы. Неизвестное сопротивление резистора вычисляется как отношение числа тактов при разряде конденсатора термистором к числу тактов при разряде через опорный резистор и последующим домножением на известное значение сопротивления опорного резистора. Программа вычисляет сопротивление термистора, переводит это значение в температуру, переводит её в градусы Фаренгейта и отображает значение на ЖКИ. Несмотря на то, что текущее значение отображается непрерывно, MSP430 проводит основную часть времени в режиме пониженного потребления 3 (LPM3). Это время может

быть использовано для проведения дополнительных измерений, для связи с другими устройствами или для осуществления вычислений.

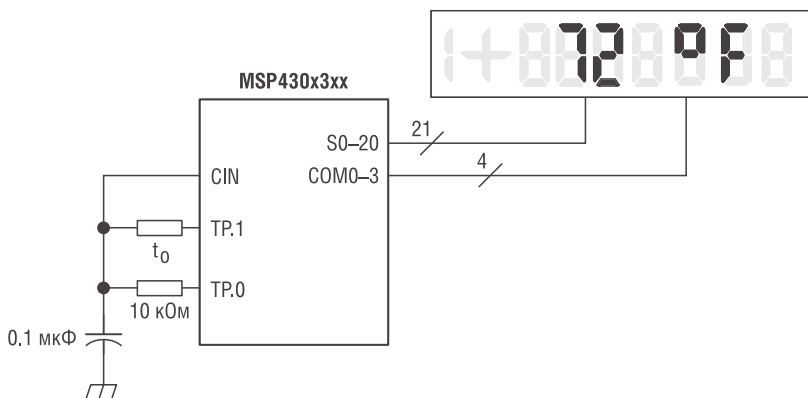


Рис. 2. Схема цифрового термометра

Все три элемента схемы могут быть подключены непосредственно к отладочной плате Texas Instruments MSP430 starter kit (STK) или evaluation kit (EVK). Остальные соединения, в том числе, требуемые для ЖКИ уже присутствуют на платах STK и EVK. Приведенный пример программы полностью помещается в 512 байт ОЗУ, имеющейся в наличии на отладочных платах STK и EVK, базирующихся на микроконтроллерах MSP430x325. Программа может быть загружена в ОЗУ через последовательный порт компьютера с использованием имеющегося на платах интерфейса.

2. Описание работы

Формула времени разряда конденсатора имеет следующий вид:

$$t = -R \times C \times \ln \frac{V_{ref}}{V_{CC}}$$

$$t = N \times t_{clock} \quad (N - \text{число тактовых импульсов})$$

$$N \times t_{clock} = -R \times C \times \ln \frac{V_{ref}}{V_{CC}}$$

$$N = -R \times C \times f_{clock} \times \ln \frac{V_{ref}}{V_{CC}}$$

Значения C , f_{clock} , и $V_{\text{ref}}/V_{\text{CC}}$ известны. Значение резистивного датчика могут быть получены из следующей формулы, с учётом того, что сопротивление опорного резистора это известная и постоянная величина:

$$\frac{N_{\text{sensor}}}{N_{\text{ref}}} = \frac{-R_{\text{sensor}} \times C \times f_{\text{clock}} \times \ln \frac{V_{\text{ref}}}{V_{\text{CC}}}}{-R_{\text{ref}} \times C \times f_{\text{clock}} \times \ln \frac{V_{\text{ref}}}{V_{\text{CC}}}}$$

$$\frac{N_{\text{sensor}}}{N_{\text{ref}}} = \frac{R_{\text{sensor}}}{R_{\text{ref}}}; \quad R_{\text{sensor}} = R_{\text{ref}} \frac{N_{\text{sensor}}}{N_{\text{ref}}}$$

Окончательная формула для схемы на рис. 2 будет иметь вид:

$$R_{\text{sensor}} = 10 \text{ кОм} \frac{N_{\text{sensor}}}{N_{\text{ref}}}$$

3. Возможности таймера – порта Timer Port

Модуль таймера – порта поддерживает различные комбинации резистивных датчиков и опорных резисторов. Если требуется проведение нескольких различных измерений в пределах одного диапазона сопротивлений, можно использовать только один опорный резистор. В противном случае, для каждого датчика нужно использовать свой опорный резистор. (см. рис 3).

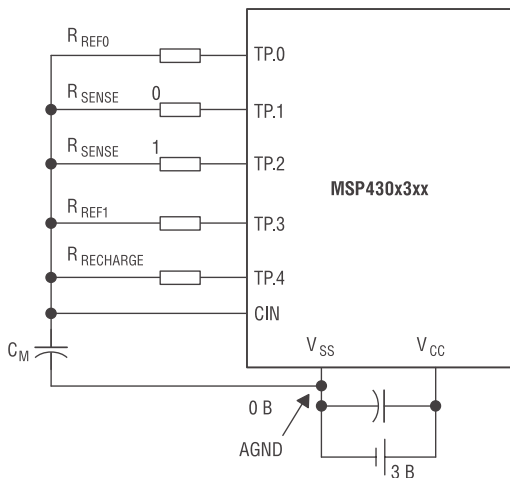


Рис. 3. Пример применения модуля таймера-порта

Все неиспользуемые выводы могут быть использованы в качестве цифровых выходов. Модуль Timer Port также включает в себя 2 8-битных счётчика с возможностью каскадирования в 1 16-битный счётчик. В случае, если эти счётчики не используются таймером – портом, они могут быть использованы для других целей. Дополнительную информацию можно найти в Metering Applications Report (файл SLAAE10C) и Architecture Guide and Module Library User's Guide (файл SLAUE10B).

4. Заключение

Timer Port, присутствующий в семействе MSP430x3xx, представляет собой весьма универсальный модуль. С его помощью можно измерять показания разнообразных резистивных датчиков в комбинациях с опорными резисторами. Детали схемы могут быть непосредственно подключены к модулю Timer Port, формируя, таким образом, полноценную измерительную систему с минимальным количеством навесных элементов. Совместно с модулем таймера-порта, 16-разрядное ЦПУ с ультранизким потреблением демонстрирует непревзойдённые характеристики MIPS на ватт.

Ссылки:

1. MSP430 Family Architecture Guide and Module Library User's Guide, SLAUE10B, Texas Instruments Incorporated, 1996
2. MSP430 Family Metering Applications Report, SLAAE10C, 1998

Приложение: Листинг программы

```
; *****
; Программа цифрового термометра
; *****
; Эта программа демонстрирует пример использования
; модуля таймера-порта для измерения значений резистив-
; ных датчиков. Программа запускается в ОЗУ отладочной
; платы STK или EVK для MSP430. Программу также можно
; запустить из ПЗУ, для этого должен быть установлен
; бит "TOOL", равный двум. При загрузке из ПЗУ таблица
; значений сопротивления может быть расширена для уве-
; личения измеряемого температурного диапазона.
; Части этой программы взяты из книги «MSP430 METERING
; APPLICATIONS REPORT», автор LUTZ BIERL, и из примеров
; программ, написанных Марком Буччини.
; TEMPDEMO Версия 1.1, 4/1998
```



```

;*****
; Системные определения для 320 STK/EVK
;*****
TOOL      .SET      0                ; 0 = STK/EVK ОЗУ
                                           ; 1 = Симулятор
                                           ; 2 = Встроенное ПЗУ

STACK     .EQU      003DEH          ; Указатель стека
RAM_ORIG  .EQU      00200H          ; Начальный адрес
                                           ; памяти ОЗУ
ROM_ORIG  .EQU      0C100H          ; Начальный адрес
                                           ; памяти ПЗУ

        .IF      TOOL = 0
I_VECTORS .EQU      003FFH          ; Вектора прерываний
                                           ; в ОЗУ
MAIN      .EQU      RAM_ORIG+20H    ; Стартовый адрес
                                           ; программы в ОЗУ
BTLOAD    .EQU      035H            ; Прерывания раз в 0,5 сек.
        .ELSEIF TOOL = 1
I_VECTORS .EQU      0FFFFH          ; Вектора прерываний в ПЗУ
MAIN      .EQU      ROM_ORIG        ; Стартовый адрес программы
                                           ; в ПЗУ
BTLOAD    .EQU      011H            ; Частые прерывания
                                           ; (не 1 сек)
        .ELSE
I_VECTORS .EQU      0FFFFH          ; Вектора прерываний в ПЗУ
MAIN      .EQU      ROM_ORIG        ; Стартовый адрес программы
                                           ; в ПЗУ
BTLOAD    .EQU      035H            ; Прерывания раз в 0,5 сек.
        .ENDIF

;*****
; Определения для АЦП на базе модуля TIMER PORT
;*****
TPCTL     .EQU      04BH            ; Регистр управления модулем
                                           ; TIMER PORT (04BH)
TPSSEL0   .EQU      040H            ; Источник тактирования:
                                           ; 0=CMR,
                                           ; 1=ACLK (Бит 6 в TPCTL)
ENB        .EQU      020H            ; Контроль сигнала EN1
                                           ; в TPCNT1
                                           ; 1 (+ENA=1)=CMR (Бит 5 в
                                           ; TPCTL)
ENA        .EQU      010H            ; Контроль сигнала EN1
                                           ; в TPCNT1
                                           ; 1 (+ENB=1)=CMR (Бит 4 в
                                           ; TPCTL)

```

```

EN1      .EQU    008H      ; Сигнал ENABLE в TPCNT1
                        ; только для чтения (Бит 3 в
                        ; TPCTL)
RC2F     .EQU    004H      ; Флаг переполнения TPCNT2
                        ; (Бит 2 в TPCTL)
EN1FG    .EQU    001H      ; Флаг EN1 (Бит 0 в TPCTL)
TP1E     .EQU    004H      ; Разрешение прерываний от
                        ; TIMER PORT (Бит 3 в IE2)
TPCNT1   .EQU    04CH      ; Младший байт счётчика
TPCNT2   .EQU    04DH      ; Старший байт счётчика
TP       .EQU    04EH      ; Регистр данных TP (0-5 =
                        ; Выход TP, 6=CPON, 7 = B16 =
                        ; 2x8 Бит или 1x16Бит счёт-
                        ; чик)
B16      .EQU    080H      ; Раздельные таймеры (0) или
                        ; один 16-битный (1)
CPO      .EQU    040H      ; Компаратор выкл (0)/вкл (1)
TPDMAX   .EQU    002H      ; Позиция бита выхода TPD.MAX
                        ; (2=Бит1=TPD.1)
TPE      .EQU    04FH      ; Регистр разрешения выходов
                        ; (0-5 = Биты разрешения TPD
                        ; 6-7=источник тактирования
                        ; TPCNT2)
MSTACK   .EQU    03D2H      ; 1-е слово стека результата
PRESET    .EQU    0E8H      ; Предустановка TPCNT2 для
                        ; заряда C счёт останавлива-
                        ; ется при переполнении
                        ; TPCNT2, значение рассчитано
                        ; на постоянную времени 6RC
; *****
; Определения управляющих регистров
; *****
IE1      .EQU    0H        ; Регистр разрешения прерыва-
                        ; ний 1
IE2      .EQU    01H      ; Регистр разрешения прерыва-
                        ; ний 2
P01IE    .EQU    08H      ; Разрешение прерываний от
                        ; P0.1 в IE1
BTIE     .EQU    080H      ; Разрешение прерываний от
                        ; BASIC TIMER в IE2
IFG1     .EQU    02H      ; Регистр флагов прерываний 1
IFG2     .EQU    03H      ; Регистр флагов прерываний 2
LCDCTL   .EQU    030H      ; Регистр управления ЖКИ
LCDM1    .EQU    031H      ; Первая ячейка памяти ЖКИ
BTCTL    .EQU    040H      ; Регистр управления BASIC
                        ; TIMER

```

```

BTCNT1 .EQU 0046H ; Счётчик 1 BASIC TIMER
BTCNT2 .EQU 0047H ; Счётчик 2 BASIC TIMER
WDTCTL .EQU 0120H ; Регистр управления WATCHDOG
WDTHOLD .EQU 080H ; Маска останова WATCHDOG
WDT_KEY .EQU 05A00H ; Ключ доступа к WATCHDOG
WDT_STOP .EQU 05A80H ; WATCHDOG маска + ключ
GIE .SET 8H ; Общее разрешение прерываний
CPUOFF .SET 10H ; Бит выключения CPU
OSCOFF .SET 20H ; Бит выключения генератора
SCG0 .SET 40H ; Контроль системного
; тактирования, бит0
SCG1 .SET 80H ; Контроль системного такти-
; рования, бит1
LPM0 .SET CPUOFF ; Биты установки режима LPM0
LPM1 .SET SCG0+CPUOFF ; Биты установки режима LPM1
LPM2 .SET SCG1+CPUOFF ; Биты установки режима LPM2
LPM3 .SET SCG1+SCG0+CPUOFF ; Биты установки режима
; LPM3
LPM4 .SET OSCOFF+CPUOFF ; Биты установки режима
; LPM4
; *****
; Служебные регистры для вычисления сопротивления
; датчика
; *****
MLTPLR_HW .EQU R5
TEN_K .EQU R6
BITTEST .EQU R7
MRESLT_HW .EQU R8
MRESLT_LW .EQU R9
LPCNTR .EQU R10
RESULT .EQU R11
; *****
; Начало программы
; *****
.SECT "MAIN", MAIN
RESET MOV #STACK, SP ; Инициализация указателя
; стека
; *****
; Конфигурация периферии
; *****
SETUP
SETUPINT MOV.B #P01IE, &IE1 ; Разрешить P0.1/UART
; для монитора RS232
MOV.B #BTIE+TPIE, &IE2 ; Разрешить прерыва-
; ния от B.TIMER, &
; TMR. PORT

```

```

CLR.B    &IFG1      ; Очистить все флаги
              ; прерываний
CLR.B    &IFG2      ;
EINT      ; Разрешить прерывания
SETUPWDT  MOV      #WDT_STOP,&WDTCTL
              ; Остановить WATCHDOG
              ; TIMER
SETUPLCD  MOV.B    #0FFH,&LCDCTL ; ЖКИ STK, все сегмен-
              ; ты, 4MUX
SETUPBT   MOV.B    #BTLOAD,&BTCTL ; Загрузить требуемую
              ; частоту прерываний
              ; в BASIC TIMER
CLR.B    &BTCNT1    ; Очистить счётчик BT 1
CLR.B    &BTCNT2    ; Очистить счётчик BT 2
CLEARLCD  MOV      #15,R6      ; Очистить 15 ячеек
              ; памяти ЖКИ
CLEAR1    MOV.B    #0,LCDM1-1(R6) ; записав туда «0»
              ; Вся памяти ЖКИ
              ; очищена?
              JNZ    CLEAR1     ; нет, чистим дальше
;*****
; Начало основной программы
;*****
BEGIN     BIS      #LPM3,SR      ; Установить SR-биты
              ; для LPM3
;*****
; Подпрограмма измерения с запрещёнными прерываниями.
; TP.2-5 не используются поэтому переписаны.
; Используются только TPD.0 & 1.
; Начальная инициализация: Указатель стека = 0,
; Начало с TPD.1
; 16-битный таймер, тактирование от MCLK, CIN разреша-
; ет счёт
;*****
MEASURE   PUSH.B   #TPDMAX      ; Сохранить в стеке
              ; для последующего
              ; использования
              CLR    R8          ; Указатель для стека
              ; результата
MEASLOP   MOV.B    #(TPSEL0*3)+ENA,&TPCTL
              ; TPCNT1 CLK=MCLK,
              ; EN1=1
;*****
; Конденсатор C заряжается на протяжении t > 5?.
; Используются выходы N-1
;*****

```

```

MOV.B    #B16+TPDMAX-1, &TPD ; один 16-битный
                                ; счётчик,
                                ; выбор выходов заряда
MOV.B    #TPDMAX-1, &TPE ; Разрешить выходы
                                ; заряда
MOV.B    #PRESET, &TPCNT2; Загрузить время
                                ; заряда
BIS      #CPUOFF, SR        ; Режим пониженного
                                ; потребления
MOV.B    @SP, &TPE          ; Разрешить только
                                ; текущий датчик
CLR.B    &TPCNT2

; *****
; Запретить все прерывания, чтобы обеспечить непрерыв-
; ную работу таймера и разряд конденсатора
; *****
DINT      ; Запрет прерываний
CLR.B    &TPCNT1            ; Очистить младший
                                ; байт таймера
BIC.B    @SP, &TPD          ; Переключить все
                                ; датчики в «0»
MOV.B    # (TPSSEL0*3) + ENA + ENB, &TPCTL
                                ; TPCNT1 CLK=MCLK,
                                ; разрешить вход CIN
EINT      ; Разрешить прерыва-
                                ; ния, общий старт
BIS      #CPUOFF, SR        ; Режим пониженного
                                ; потребления

; *****
; EN=0: Завершение преобразования: 2X8 бит результата
; в MSTACK
; Включить следующий датчик : если больше нет,
; завершение
; *****
MOV.B    &TPCNT1, MSTACK(R8)
                                ; Сохранить результат в стеке
MOV.B    &TPCNT2, MSTACK+1(R8)
                                ; Сохранить старший байт результата
L$301    INCD               R8 ; Инкремент адреса
RRA.B    @SP                ; Следующий выход TPD.X
JNC      MEASLOP             ; Если C=1 - заверше-
                                ; ние
INCD     SP                  ; Убрать старший TPD
                                ; из стека

```

```

;*****
; Вычисление сопротивления датчика
;*****
; Подпрограмма беззнакового умножения: MSTACK X TEN_K?
; MRESLT_HW/MRESLT_LW
; Исползованы регистры MSTACK, TEN_K, MLTPLR_HW,
; MRESLT_LW, MRESLT_HW, BITTEST
; Подпрограмма беззнакового умножения с накоплением:
; (MSTACK X TEN_K) + MRESLT_HW|MRESLT_LW →
; MRESLT_HW|MRESLT_LW
;*****
CALC_RES
      MOV      #10000,TEN_K    ; Загрузить 10,000
                                ; десятичное в TEN_K
MPYU      CLR      MRESLT_LW   ; 0? LSBS результата
      CLR      MRESLT_HW       ; 0? MSBS результата
MACU      CLR      MLTPLR_HW   ; 0? MSBS множителя
      MOV      #1,BITTEST      ; Регистр проверки бит
L$002     BIT      BITTEST,MSTACK ; Проверить текущий
                                ; бит
      JZ       L$01            ; Если «0» ничего не
                                ; делать
      ADD      TEN_K,MRESLT_LW ; Если «1» добавить
                                ; множитель к резуль-
                                ; тату
      ADDC     MLTPLR_HW,MRESLT_HW
L$01      RLA      TEN_K        ; Множитель X 2
      RLC      MLTPLR_HW       ;
      RLA      BITTEST         ; Проверить следующий
                                ; бит
      JNC      L$002           ; Если бит в CARRY:
                                ; завершить

;*****
; Подпрограмма беззнакового деления 32-бита на 16-бит
; Исползованы регистры (MSTACK+2), MRESLT_LW, RESULT,
; LPCNTR, MRESLT_HW MRESLT_HW MRESLT_LW/(MSTACK+2)?
; RESULT Остаток в MRESLT_HW
; По выходу: CARRY = 0: ОК CARRY = 1: Частное > 16 Бит
;*****
DIVIDE    CLR      RESULT       ; Очистить RESULT
      MOV      #17,LPCNTR      ; Инициализация
                                ; счётчика
DIV1      CMP      MSTACK+2,MRESLT_HW
      JLO      DIV2
      SUB      MSTACK+2,MRESLT_HW

```

```

DIV2      RLC      RESULT
          JC        RES_2_F      ; Ошибка: RESULT>16 Бит
          DEC       LPCNTR      ; Декремент счётчика
          JZ        DIV3        ; 0: выход без ошибки
          RLA       MRESLT_LW
          RLC       MRESLT_HW
          JNC       DIV1
          SUB       MSTACK+2,MRESLT_HW
          SETC
          JMP       DIV2
DIV3      CLRC                      ; Ошибки нет, C = 0
;*****
; Перевод сопротивления датчика в градусы Фаренгейта
; для отображения
;*****
RES_2_F
          CLR       R12          ; Указатель на значе-
                                ; ние в таблице R
          MOV       #064H,R13    ; Поместить мин.
                                ; темп.-1 в темп. ин-
                                ; дикатор
          JMP       FIRST_CMP    ; При первом сравнении
                                ; 1 не добавлять
CHECK_R   INCD      R12          ; Инкремент указателя
                                ; на значение в табли-
                                ; це сопротивлений
          DADD      #1,R13       ; Десятичный инкремент
                                ; счётчика
FIRST_CMP CMP       RESIS_TAB(R12),RESULT
                                ; Сравнить табличное
                                ; значение с вычислен-
                                ; ным сопротивлением
          JNC       CHECK_R      ; Переход, если R дат-
                                ; чика < табличного
                                ; значения по адресу
                                ; указателя
;*****
; Отобразить "F" и знак «°» на ЖКИ
;*****
DISPLAY   MOV.B     #A+E+F+G,LCDM1+1 ; "F" ? дисплей-
                                ; ная память ЖК
          MOV.B     #A+B+F+G,LCDM1+2 ; "°" (знак граду-
                                ; са)? дисплейная па-
                                ; мять ЖК

```

```

;*****
; Отобразить значение, хранящееся в R13 в формате BCD
; на ЖКИ
;*****
      MOV      R13,R12      ; Копировать число
                               ; в виде BCD в R12
      MOV      #LCDM1+4,R14 ; Младшая цифра
                               ; в памяти ЖКИ? R14
      BIC      #0FFF0H,R13  ; Погасить всё, кроме
                               ; младшей цифры
      MOV.B    LCD_TAB(R13),0(R14)
                               ; Отправить младшую
                               ; цифру в ЖКИ
      MOV      R12,R13      ; Восстановить
                               ; значение в R13
      RRA      R13           ; 4 сдвига
      RRA      R13
      RRA      R13
      RRA      R13
      BIC      #0FFF0H,R13  ; Погасить всё, кроме
                               ; младшей цифры
      MOV.B    LCD_TAB(R13),1(R14)
                               ; Отправить старшую
                               ; цифру в ЖКИ
      JMP      BEGIN        ; Переход к началу
                               ; программы
;*****
; Подпрограмма обработки прерываний базового таймера
; BASIC TIMER:
; ЦПУ по возвращении остаётся в активном режиме
; благодаря очистке битов
; LPM3 в статусном регистре SR в стеке.
;*****
BTINT      BIC      #LPM3,0(SP) ; Очистить в SR биты
                               ; LPM3, вверху стека
      RETI
;*****
; Подпрограмма обработки прерываний таймера – порта
; TIMER PORT:
; ЦПУ при возвращении по RETI остаётся в активном
; режиме
;*****
TPINT      CLR.b    &TPCTL      ; Очистить флаги
                               ; прерываний TP, оста-
                               ; новить счёт

```



```

        BIC        #LPM3,0(SP)      ; Очистить в SR биты
                                   ; LPM3, вверху стека
        RETI
;*****
; ЖКИ модуля STK
;*****
LCD_TYPE
A        .EQU      01H
B        .EQU      02H
C        .EQU      10H
D        .EQU      04H
E        .EQU      80H
F        .EQU      20H
G        .EQU      08H
H        .EQU      40H
LCD_TAB  .BYTE A+B+C+D+E+F          ; Отображает "0"
        .BYTE B+C                    ; Отображает "1"
        .BYTE A+B+D+E+G              ; Отображает "2"
        .BYTE A+B+C+D+G              ; Отображает "3"
        .BYTE B+C+F+G                ; Отображает "4"
        .BYTE A+C+D+F+G              ; Отображает "5"
        .BYTE A+C+D+E+F+G            ; Отображает "6"
        .BYTE A+B+C                  ; Отображает "7"
        .BYTE A+B+C+D+E+F+G          ; Отображает "8"
        .BYTE A+B+C+D+F+G            ; Отображает "9"
;*****
; Таблица сопротивлений 65–99 °F. Значения = КОМЫ
; X1000 – в 3 десятичных цифры
;*****
        .EVEN                        ; Выравнивание
RESIS_TAB .WORD      12953            ; по чётному адресу
        .WORD      12666            ; 65 F
        .WORD      12378
        .WORD      12090
        .WORD      11858
        .WORD      11626            ; 70 F
        .WORD      11393
        .WORD      11161
        .WORD      10929
        .WORD      10697
        .WORD      10464            ; 75 F
        .WORD      10232
        .WORD      10000
        .WORD      9813
        .WORD      9625

```

```

.WORD 9438 ; 80 F
.WORD 9250
.WORD 9063
.WORD 8875
.WORD 8688
.WORD 8500 ; 85 F
.WORD 8313
.WORD 8161
.WORD 8008
.WORD 7856
.WORD 7703 ; 90 F
.WORD 7551
.WORD 7398
.WORD 7246
.WORD 7093
.WORD 6941 ; 95 F
.WORD 6817
.WORD 6694
.WORD 6570
.WORD 6446 ; 99 F
;*****
; Вектора прерываний
;*****
.EVEN ; Выравнивание по чётному
; адресу
.SECT "INT_VECT", I_VECTORS-31
.WORD RESET ; PORT0, Биты 2...7
.WORD BTINT ; BASIC TIMER
.WORD RESET ; Нет источника
.WORD RESET ; Нет источника
.WORD RESET ; Нет источника
.WORD TPINT ; TIMER PORT
.WORD RESET ; Нет источника
.WORD RESET ; Нет источника
.WORD RESET ; Нет источника
.WORD RESET ; Нет источника
.WORD RESET ; WATCHDOG/TIMER,
; Режим таймера
.WORD RESET ; Нет источника
.WORD RESET ; UART
.WORD RESET ; P0.0
.WORD RESET ; NMI, Сбой генератора
.WORD RESET ; POR, Внеш. Reset,
; WATCHDOG

.END

```

IV. Примеры конкретных применений

Глава 10.

*FSK модуляция и демодуляция
с использованием микроконтроллера
MSP430*

FSK модуляция и демодуляция с использованием микроконтроллера MSP430

[slaa037.pdf (Англ.)]

В этом примере применения описывается программа, позволяющая микроконтроллеру MSP430 выполнять функции приемопередатчика FSK модема стандарта V.23. Здесь используется фильтр с новой архитектурой, позволяющий выполнять функции DSP при помощи обычного микропроцессора, способного только сдвигать и суммировать данные.

1. Введение

Многие измерительные устройства (например, электросчетчики и счетчики газа) должны иметь возможность установки связи с центральной станцией для передачи результатов измерения и получения новых тарифов. Такая связь может быть легко установлена по телефонной линии.

Передача данных при помощи сигнала с частотной манипуляцией (FSK) и двухтонального многочастотного (DTMF) сигнала – два наиболее популярных метода представления двоичных данных для передачи по телефонным линиям. В данном примере применения описывается программа V.23-совместимого FSK приемопередатчика.

Интеграция в одном чипе измерительных и коммуникационных функций ведет к увеличению стоимости и потреблению. Используя высокоскоростной низкопотребляющий микроконтроллер MSP430, возможно изготовить модем, питающийся от телефонной линии.

В данном примере применения приведены математические формулы для передачи и детектирования FSK сигналов. К листингу управляющей программы приложены типовые схемы телефонного интерфейса и дешевого аналого-цифрового преобразователя. Данные схемы являются типовыми и для использования их в некоторых случаях необходимо провести соответствующие доработки.

2. Теория демодуляции

Демодуляция FSK сигнала обеспечивается квадратурным демодулятором. В этом демодуляторе производится перемножение принятого и задержанного сигнала с последующей низкочастотной фильтрацией результата перемножения. Если время задержки T установить в соответствии с выражением $W_{\text{carrier}} \times T = \pi/2$, то на выходе фильтра нижних частот будет присутствовать сигнал, пропорциональный отклонению частоты, который и является информационным.

Если $w = W_{\text{carrier}} \pm W_{\text{delta}}$ и $T \times W_{\text{carrier}} = \pi/2$, где $w = 2 \times \pi \times f$ (круговая частота), то процесс демодуляции можно описать с помощью следующего математического выражения:

$$\begin{aligned}\cos[wt] \times \cos[w(t - T)] &= \cos wT + \cos(2wt - wT) \rightarrow \text{ФНЧ} \rightarrow \cos wT \\ \cos wT &= \sin[\pm W_{\text{delta}}] = \pm \sin[W_{\text{delta}}]\end{aligned}$$

2.1. Выбор частоты дискретизации

Для упрощения формирования задержки частота дискретизации выбирается равной $F_{\text{carrier}} \times 4$. Для стандарта V.23 частота несущей $F = 1700$ Гц, поэтому частота дискретизации должна быть 6800 Гц. При использовании 32768 Гц кварцевого резонатора обеспечивается частота дискретизации 6793.3 Гц, которая отличается от требуемой на 0.1 %. Частота дискретизации задается 8-битным интервальным таймером. Так как таймер имеет коэффициент пересчета, равный 256, то прерывание устанавливается дважды за один период дискретизации, при этом выборка осуществляется один раз за два периода.

2.2. Управление оконечными устройствами

Большинство аналого-цифровых преобразователей, включая аналого-цифровой преобразователь последовательного приближения микропроцессора MSP430C325, нуждается в обеспечении смещения по постоянному току; эти АЦП выдают результат преобразования в виде смещенного положительного целочисленного числа. Прежде, чем выбранный сигнал будет обработан далее необходимо подать его на фильтр, подавляющий постоянную составляющую, что приведет к преобразованию кода в целочисленное значение со знаком. Этот же фильтр обеспечивает подавление порядка 30 дБ сигнала основной частоты.

2.3. Демодуляция FSK

Принятые прямой и задержанные сигналы необходимо перемножить; в нашем случае для этого применяется мультиплицирующий 8?8 кольцевой регистр со знаком.

Цепочка, составленная двух частотно-зависимых элементов, является фильтром низких частот, предназначенным для подавления сигнала удвоенной частоты. На выходе этого устройства присутствует целочисленное значение со знаком, представляющее собой значение полученного бита.

Фильтр нижних частот использует технологию цифровой фильтрации колебаний. Эта технология обеспечивает устойчивые характеристики с очень хорошей точностью. Все перемножения выполняются при помощи операций сложения и сдвига при некоторых округлениях. Поскольку фильтр имеет высокую точность коэффициента передачи, то это округление не влияет на параметры

фильтра. Фильтр Баттерворта, используемый в данном случае, дает коэффициент ослабления приблизительно 40 дБ в полосе затухания и неравномерность АЧХ в полосе пропускания порядка 1 дБ.

2.4. Синхронизация бита

Для обработки входного потока данных значения демодулированных битов должно быть определено и синхронизировано. Эта технология также известна под названием выделения битов и восстановления синхроимпульсов. Так как частота дискретизации (6800 Гц) не кратна скорости потока данных (1200 бит/с), то необходимы дополнительные шаги для объединения этих двух скоростей. Это реализовано при помощи суммирующе-вычитающего счетчика с последовательностью предварительно загруженных значений (5,6,5). Каждые 17 отсчетов частота дискретизации и скорость потока восстанавливают синхронизацию. Синхронизация бита или регенерация синхроимпульсов выполняются за счет отслеживания изменения значения бита. После этого информация об опережении или запаздывании обрабатывается, и суммирующе-вычитающий счетчик корректирует свое значение. Из-за разности между частотой выборки и скоростью потока данных выбор бита данных никогда не осуществляется точно в середине периода потока данных, а имеет отклонение от -5 % до +13%. Экспериментально доказано, что это не оказывает влияние на точность работы системы.

3. Теории модуляции

При FSK модуляции осуществляется отклонение несущей частоты на величину, равную девиации частоты, при этом знак изменения частоты зависит от значения передаваемого информационного бита. В стандарте V.23 информационный ноль передается отклонением частоты на 400 Гц, а информационная единица – отклонением на -400 Гц.

$$\text{FSK сигнал} = \text{Амплитуда} \times \cos [t \times 2 \times \pi \times (F_{\text{carrier}} \pm F_{\text{delta}})]$$

Синусоидальный сигнал генерируется в соответствии с таблицей значений, которая содержит значения косинуса в пределах от 0 до $2 \times \pi$ радиан. Параметр, называемый PHASER соответствует следующим угловым значениям: 0 = 0 градусов, 8000h = 180 градусов, 10000h = 360 градусов. После каждой выборки этот угол перемещается к следующему значению в соответствии с величиной 16 битного параметра DELTA, который определяет частоту сигнала (наибольшее значение DELTA соответствует наивысшей частоте). Частотная модуляция осуществляется за счет изменения величины параметра DELTA, которое определяется по формуле:

$$\text{DELTA} = F_{\text{desired}} / F_{\text{sampling}} \times 65536.$$

Преимущество этого метода по сравнению с методом цифрового автогенератора состоит в том, что этот метод сохраняет фазовое соотношение, даже когда частота изменяется от выборки к выборке.

3.1. Выбор частоты дискретизации

8-битный интервальный таймер устанавливает частоту дискретизации 19200 выборок/сек. Эта скорость кратна требуемой скорости потока данных 1200 бод. Также, это позволяет упростить построение устройства выборки-хранения (УВХ) АЦП.

3.2. Построение таблицы значений

Чтобы сохранять объем памяти ПЗУ в таблице закодирован только первый квадрант в формате Q7 (от 0 до 127). Это сделано путем деления первого квадранта (90 градусов) на 128 интервалов с шагом приблизительно 0.7 градуса. Восстановление значений трех других квадрантов выполняется путем дополнительных вычислений.

3.3. FSK модуляция

Параметр PHASER изменяется на величину DELTA при каждом прерывании. Первые 9 битов параметра PHASER используются, для восстановления значения косинуса. Так как функция косинуса является симметричной, то значения ее в третьем и четвертом квадрантах такие же, как и во втором и первом квадрантах, а значит также используются только абсолютная величина, задаваемая первыми 9 битами параметра PHASER.

Затем, всю значения второго квадранта вычисляются из значений первого квадранта, содержащиеся в таблице.

8 битный результат защелкивается в порту P0.OUT.

Каждые 16 прерываний, параметр DELTA обновляется в соответствии с величиной частоты следующего посылаемого бита.

4. Преобразование данных

В этом разделе описываются необходимые аналого-цифровое и цифро-аналоговое преобразования данных.

4.1. Аналого-цифровое преобразование

Наиболее простой способ оцифровки входящего FSK сигнала состоит в том, чтобы использовать 12 битный режим встроенного 14 битного аналого-

цифрового преобразователя микроконтроллера MSP430C325. Однако, для обеспечения динамического диапазона, необходимого для демодуляции принятого FSK сигнала 12 битный преобразователь не нужен. Результаты исследований показывают, что для обработки вполне достаточно 8 битного преобразования и программно реализованной системы АРУ. При этом обеспечивается динамический диапазон 25 дБ. При использовании внешней системы АРУ динамический диапазон может быть расширен. Так как существует большое количество дешевых решений построения 8 битных АЦП, то имеется возможность применять все микроконтроллеры семейства MSP430. В прикладной программе описываемого примера применения используется АЦП на основе универсального таймера и внешнего компаратора. Это позволяет использовать данную прикладную программу в устройствах на основе всех микроконтроллеров данного семейства.

4.2. Цифро-аналоговое преобразование

Для формирования 6 битного цифро-аналогового преобразователя используется внешняя R-2R цепочка. Так как частота переключений 19200 Гц, что в девять раз больше, чем максимальная частота FSK сигнала, равная 2100 Гц, то реализовать сглаживающий фильтр очень просто. В приведенной схеме применен фильтр низких частот первого порядка на одном конденсаторе, но при необходимости количество звеньев фильтра может быть увеличено при помощи дополнительных пассивных цепочек.

5. Потребляемая мощность

Концепция FSK разработана с целью снижения потребления. Демодулятор FSK работает с производительностью менее 2 MIPS. Вместе с низко потребляющим операционным усилителем, использованным в качестве оконечного каскада, потребление системы не должно превысить 1.5 мА. Таким образом, возможно питание системы от телефонной линии. В приведенной в данном примере применения схеме имеется и интерфейс с телефонной линией. Однако, в зависимости от стандарта конкретной страны применения, возможно потребуются изменить схемотехнику данного узла.

6. Программное обеспечение

В данном разделе описывается прикладная программа FSK модема.

6.1. FSK приемник

FSK сигнал поступает от телефонной интерфейсной схемы. Этот сигнал должен иметь смещение по постоянному току 1.2 В и размах 400 мВ. Программное обеспечение декодирует этого FSK сигнал и выставляет на трех выходах логические сигналы, позволяющие пользователю отследить демодулированные данные.

На выходе TP.3 выдается восстановленный из входного FSK сигнала синхросигнал.

На выходе TP.5 присутствуют восстановленные из входного FSK сигнала данные, которые защелкиваются по каждому нарастающему фронту сигнала на выходе TP.3.

На выходах P0.2-P0.7 присутствует результат низкочастотной фильтрации. Отфильтровав этот сигнал при помощи внешней R-2R цепочки, можно отслеживать уровень демодулированного FSK сигнала. Этот сигнал формируется только из 6 старших бит 8 битных данных, загруженных в порт P0.

6.2. FSK передатчик

Программа передатчика выводит FSK сигнал в соответствии с BITMAP данными, определенными в таблице TX_DATA_TABLE. Образ начинается с начальной зоны записи (преамбулы), сопровождаемой длинным MARK периодом. После этого идет передача данных. Эта таблица использует нулевое слово в качестве окончательного маркера, а программа перезапускается после достижения нулевого значения данных битового массива.

7. Типовая схема

В этом разделе приведен и описан пример схемной реализации модема.

7.1. Использование MSP430C325 в качестве центрального процессора

На рисунке 1 приведена схема электрическая принципиальная FSK модема, использующего в качестве центрального процессора микропроцессор MSP430C325. Схема была протестирована при подаче на вход FSK сигнала с размахом 400 мВ. Чтобы получить те же самые результаты, на вход Rx должен быть подан FSK сигнал с постоянной составляющей 1.2 В и размахом 400 мВ.

7.2. Пример телефонного интерфейса

На рисунке 2 приведен пример схемной реализации телефонного интерфейса, а в таблице 1 приведены параметры FSK приемопередатчика.

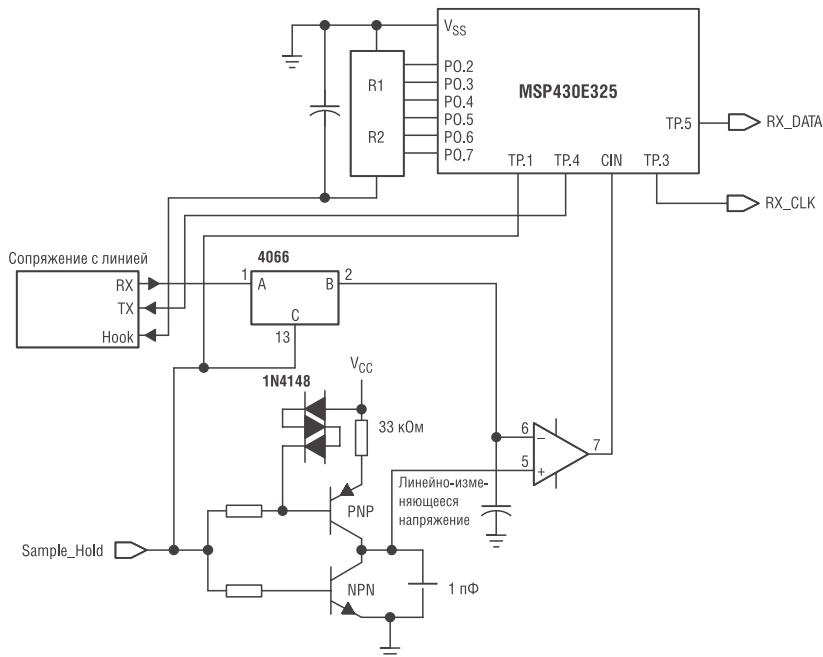


Рис. 1. Центральный процессор и аналого-цифровой преобразователь

8. Заключение

Обычно FSK приемопередатчики реализуются аналоговыми средствами или при помощи DSP процессоров и аппаратных MAC модулей. Использование RISC микропроцессора MSP430 без MAC модулей для реализации функций приемопередатчика – очень необычная технология. Способность создавать фильтры при помощи технологии цифровой фильтрации колебаний совместно с дополнительным набором команд и 16 битной архитектурой микроконтроллера MSP430 делает код прикладной программы весьма компактным и производительным. Кроме того, сверхнизкое потребление MSP430 позволяет питать модем от телефонной линии. Это позволяет сделать устройство с малым числом компонентов. Были проведены дополнительные исследования, позволившие сделать заключение, что FSK приемник может работать с 8 битным аналого-цифровым преобразователем и иметь достаточно высокую чувствительность. Поэтому FSK приемопередатчик может быть реализован на любом микроконтроллере семейства MSP430.

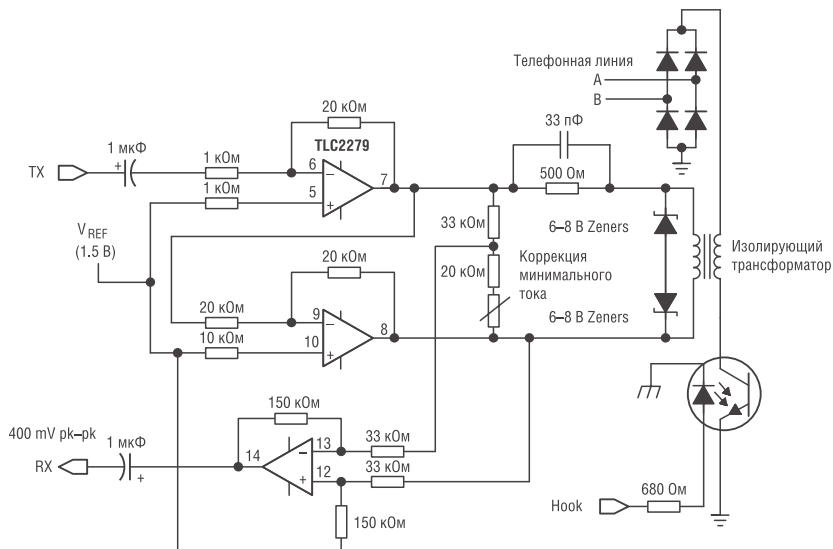


Рис. 2. Телефонный интерфейс

Ссылки:

1. Texas Instruments: MSP430 Family, Architecture User's Guide and Module Library.
2. Texas Instruments Digital Signal Processing Application with the TMS320 Family Volume 2.
3. Gaszi, L: Explicit Formulas for Lattice Wave Digital Filters; IEEE Trans. On Circuits and Systems VOL. CAS-32, NO. 1, January 1985

Приложение А: подпрограмма FSK приемника

```

CPUOFF      .equ      010h
SCG0        .equ      040h
SCG1        .equ      080h
IE1         .equ      0h
IE1_POIE1   .equ      08h
IE1_POIE0   .equ      04h
IE1_OFIE    .equ      02h
IE1_WDTIE   .equ      01h
IE2         .equ      01h
    
```

IE2_BTIE	.equ	80h	
IE2_TPIE	.equ	08h	
IE2_ADIE	.equ	04h	
IE2_UTXRIE	.equ	02h	
IE2_URXIE	.equ	01h	
IFG1	.equ	02h	
IFG1_2	.set	04h	
IFG2	.equ	03h	
ME1	.equ	04h	
ME2	.equ	05h	
P0IN	.equ	010h	
P0OUT	.equ	011h	
P0DIR	.equ	012h	
P0FLG	.equ	013h	
P0IES	.equ	014h	
P0IE	.equ	015h	
LCDCTL	.equ	030h	
LCDM	.equ	030h	; Адрес регистра ; управления и режима ; ЖКИ
BTCTL	.equ	040h	
BTCTL_SSEL	.equ	80h	
BTCTL_Hold	.equ	40h	
BTCTL_DIV	.equ	20h	
BTCTL_FREQ1	.equ	10h	
BTCTL_FREQ0	.equ	08h	
BTCTL_IP2	.equ	04h	
BTCTL_IP1	.equ	02h	
BTCTL_IP0	.equ	01h	
BTCTL1	.equ	046h	
BTCTL2	.equ	047h	
BTIFG	.equ	080h	; Флаг прерывания ВТ
TCCTL	.equ	042h	; Адрес управляющего ; регистра таймера/ ; счетчика
TCCTL_SSEL1	.equ	080h	
TCCTL_SSEL0	.equ	040h	
TCCTL_ISCTL	.equ	020h	
TCCTL_TXE	.equ	010h	
TCCTL_ENCNT	.equ	008h	
TCCTL_RXACT	.equ	004h	
TCCTL_TXD	.equ	002h	

TCCTL_RXD	.equ	001h	
TCPLD	.equ	043h	; Адрес регистра пред- ; варительной загрузки
			; таймера/счетчика
TCDAT	.equ	044h	; Адрес таймера/счет- ; чика
TPD	.equ	04eh	
TPD_B16	.equ	080h	
TPD_CPON	.equ	040h	
TPE	.equ	04fh	
TPE_0	.equ	01h	
TPE_1	.equ	02h	
TPE_2	.equ	04h	
TPE_3	.equ	08h	
TPE_4	.equ	10h	
TPE_5	.equ	20h	
TPE_TPSSEL2	.equ	40h	
TPE_TPSSEL3	.equ	80h	
TPCTL	.equ	04Bh	
TPCTL_EN1FG	.equ	01h	
TPCTL_RC1FG	.equ	02h	
TPCTL_RC2FG	.equ	04h	
TPCTL_EN1	.equ	08h	
TPCTL_ENA	.equ	10h	
TPCTL_ENB	.equ	20h	
TPCTL_TPSSEL0	.equ	40h	
TPCTL_TPSSEL1	.equ	80h	
TPCNT1	.equ	04Ch	
TPCNT2	.equ	04Dh	
SCFI0	.equ	050h	
SCFI1	.equ	051h	
SCFQCTL	.equ	052h	
CBCTL	.equ	053h	
CBE	.set	1	
AIN	.equ	0110h	
AEN	.equ	0112h	
ACTL	.equ	0114h	
ACTL_CSTART	.equ	0001h	
ACTL_SVCC_OFF	.equ	0000h	
ACTL_SVCC_ON	.equ	0002h	
ACTL_2	.equ	0004h	
ACTL_3	.equ	0008h	

```

ACTL_4      .equ    0010h
ACTL_5      .equ    0020h
ACTL_SEL_A0 .equ    0000h
ACTL_SEL_A1 .equ    0004h
ACTL_SEL_A2 .equ    0008h
ACTL_I_SRC_A0 .equ    0000h
ACTL_I_SRC_A1 .equ    0040h
ACTL_RNG_B  .equ    0200h
ACTL_RNG_AUTO .equ    0800h
ACTL_POWER_UP .equ    0000h
ACTL_POWER_DOWN .equ    1000h
ACTL_CLK_MCLK .equ    0000h
ACTL_CLK_MCLK_2 .equ    2000h
ACTL_CLK_MCLK_3 .equ    4000h
ADAT        .equ    0118h
ADIFG       .equ    04h
WDTCTL      .equ    0120h
WDTHold     .equ    80h
WDT_wrkey   .equ    05A00h
STACK       .set    300h ; Начальное значение
                        ; системного стека
                        ; 280h

; *****
; Фильтры
; *****
WDF_PARMS   .usect  "FILTMEM", 10, 0200h
IN_Z        .set    0
Z1_1        .set    2
Z1_2        .set    4
Z3_1        .set    6
Z3_2        .set    8
end_of_parms .usect  "FILTMEM", 2
data_word   .usect  "FILTMEM", 2
last_sample .usect  "FILTMEM", 2
bit_lead_lag .usect  "FILTMEM", 2
cycle_counter .usect  "FILTMEM", 1 ; Пользователь-
                                   ; ская сервисная
                                   ; программа
; *****
; Они используются для FSK демодуляции
; при прерывании от 8- битного таймера
; *****

```

```

;*****
; DYNAMIC: Регистры маркировки (используемые WDF)
; не должны использоваться для перемещения.
;*****
currenty      .set    R6      ; Используется WDF
currentx      .set    R7      ; Используется WDF
IROP1         .set    currenty ; Используется
                                   ; WDF
IROP2L        .set    R7      ; Используется WDF
IRACL         .set    R8      ; Используется WDF
IRBT          .set    R9      ; Используется WDF
lastx         .set    R10
bit_data      .set    R11     ; Используется WDF
mem_ptr       .set    R15     ; Используется WDF
;*****
; STATIC
;*****
bit_sync_timer .set    R12    ;
global_status  .set    R13    ;
bits_count     .set    R14    ;
INTERRUPT_TOGGLE .set    1
FALLING        .set    2
CLOCK          .set    4
;*****
;Инициализация системы
;*****
;RAM_NORMAL_DEMOD .sect    "RAM_CODE",02f0h
Start           .sect    "ADC",0f000h ;0214h ;0F000h
                mov      #STACK,SP      ; Инициализация сис-
                                   ; темного указателя
                                   ; стека
                mov      #(WDTHold+WDT_wrkey), &WDTCTL
                                   ; Остановка сторожево-
                                   ; го таймера
                clr.b    &IFG1          ; очистка всех флагов
                                   ; прерываний
                clr.b    &IFG2
                mov.b    #(17*5)-1, &SCFQCTL
                                   ; MCLK=32768*17*5 →
                                   ; получаем производи-
                                   ; тельность 2.8 MIPS

```

```

mov.b    #4,&SCFIO      ; Установка частоты RC
                        ; генератора, равной
                        ; 2xFreq
mov.b    #-205,&TCPLD    ; 328/2 ; сброс
                        ; нарастающих/спадаю-
                        ; щих фронтов
bis.b    #TPD_B16,&TPD   ; операционный усили-
                        ; тель
bis.b    #11111110b,&P0DIR ; установка
                        ; портов P0.7 - P0.1
                        ; на работу в режиме
                        ; выходов P0.0 исполь-
                        ; зуется в качестве
                        ; входа сигнала RING
mov.b    #(TCCTL_SSEL1+TCCTL_ISCTL +
TCCTL_ENCNT),&TCCTL
bic.b    #IE2_BTIE,&IE2  ; отключение прерыва-
                        ; ния основного таймера
bic.b    #IE1_P0IE0,&IE1 ; отключение прерыва-
                        ; ния от P0.0
bic.b    #IE2_TPIE,&IE2  ; отключение прерыва-
                        ; ния универсального
                        ; таймера
bis.b    #TPE_0+TPE_1+TPE_2+TPE_3+TPE_5,&TPE
                        ; TP_0 используется для сигнала Hook
                        ; TP_1 используется для управления
                        ; RAMP генератором
                        ; TP_2 используется для тестирования
                        ; схемы
                        ; TP_3 перекрывает биты синхрониза-
                        ; ции
                        ; TP_5 - бит данных
mov      #1,bit_sync_timer
mov.b    #3,cycle_counter
call     #clear_all_parameters

; <----- активизация демонстрации FSK!!!!-----
bis.b    #IE1_P0IE1,&IE1
eint

WAIT_LOOP:                ; неопределенное ожидание
                        ; выход из цикла по прерыванию
        jmp      WAIT_LOOP

TIM8_Int:

```



```

;*****
; Выборка входного сигнала
;*****
NORMAL_DEMOD:
    bit        #INTERRUPT_TOGGLE,global_status
                ;!!!!

    jz         filters

do_ADC
    bic        #INTERRUPT_TOGGLE,global_status

SLP_A_D:
    mov.b      #(TPCTL_TPSSEL1+TPCTL_TPSSEL0),
                &TPCTL        ; использование MCLK
                                ; в качестве входного
                                ; сигнала, останавли-
                                ; вающего счет
    bis.b      #TPE_2+TPE_1,&TPD    ; сброс RAMP
                                ; только TPE_2,
                                ; используемый для
                                ; тестирования схемы,
                                ; может прервать.
    mov.b      &TPCNT1,currenty ; обнуление MSByte
    mov.b      &TPCNT2,currentx ; обнуление MSByte
    swpb
    bis        currenty,currentx ; объединение для
                                ; формирования 16-бит-
                                ; ного результата эха
                                ; выборки на выходе

    clr.b      &TPCNT1
    clr.b      &TPCNT2        ; активизация счетчика
    mov.b      #TPCTL_ENA+TPCTL_ENB+TPCTL_TPSSEL1+
                TPCTL_TPSSEL0,&TPCTL
    bic.b      #TPE_2+TPE_1,&TPD ; перезапуск RAMP

_0DB:
    rla        currentx
    rla        currentx
    rla        currentx

;*****
; Проверка того, что входное напряжение < VCC/4
;*****
;*****
; Результаты анти-смещения
; ответ = входной сигнал + (0.875 * lasty) - lastx;

```

```

; lasty = ответ;
; lastx = входной сигнал;
; *****
                mov     currentx,currenty
                sub     lastx,currenty
                mov     currentx,lastx

; *****
; Здесь коэффициент усиления настроен на оптимальное
; значение размаха 1 В
; при более высоком значении может потребоваться его
; уменьшение при помощи внешнего резистивного делителя
; *****
;                rra     currenty
;                rra     currenty
;                rra     currenty
;                mov     &last_sample,IROP2L
;                mov     currenty,&last_sample
;                and     #0ffh,currenty
;                and     #0ffh,IROP2L

; *****
; подпрограмма перемножения восьмибитных чисел со
; знаком
; *****
                clr     IRACL
                tst.b    currenty
                jge     L$101
                swpb     IROP2L
                sub      IROP2L,IRACL
                swpb     IROP2L
L$101
                tst.b    IROP2L
                jge     MACU8
                swpb     currenty
                sub      currenty,IRACL
                swpb     currenty
MACU8
                mov     #1,IRBT
L$002          bit     IRBT,currenty
                jz      L$01
                add     IROP2L,IRACL
L$01           rla     IROP2L

```

```

        rla.b    IRBT
        jnc      L$002
        mov      IRACL,R9          ; подготовка для LP
                                   ; фильтра в следующем
                                   ; цикле

;*****
; Выполнение синхронизации фронтов
;*****
        bit      #FALLING,global_status ;!!!!!!!!!
        jz       detect_rising
detect_falling:
        tst      bit_data          ; последний фронт был
                                   ; нарастающим, поэтому
                                   ; проверяем для спада-
                                   ; ющего
        jge      update_bit_sync_timer ;не спадающий,
                                   ; выполняем следующую
                                   ; команду
        bic      #FALLING,global_status ;определен
                                   ; спадающий фронт,
                                   ; в следующем цикле
                                   ; необходимо отследить
                                   ; нарастающий
        jmp      new_edge_detected
detect_rising:
        tst      bit_data
        jn       update_bit_sync_timer
        bis      #FALLING,global_status

;*****
; Было обнаружено изменение фронта, сейчас для
; определенности синхронизируем бит synch таймера,
; опережение или отставание
;
;                               / ____ Выборка 3
;
; Опережение Отставание
;  __/|\__  /|\__
; |  |  |  |  |  |
; |  |  |  |  |  |
; _|  |  |  |__|__|
;
;
; Новый фронт был обнаружен, теперь необходимо

```

```

; синхронизировать встроенный программный генератор,
; работающий на частоте 1200 бод, с внешним фронтом.
; Внутренние данные синхронизации содержатся в бите
; BIT2 регистра TPD и используются для определения
; опережения или отставания. Так как имеется дребезг
; входных синхроимпульсов, то мы не используем инфор-
; мацию о запаздывании или опережении для немедленного
; обновления. Вместо этого мы осуществляем «низкочас-
; тотную фильтрацию» результата для добавления или
; вычитания его из счетчика только после того, как его
; абсолютное значение превысит порог, который мы уста-
; новили.
;*****
new_edge_detected
    bit.b    #CLOCK,global_status    ;!!!!!!!!!!
    jz       lagging
leading:
    add      #1,bit_lead_lag
    jmp      update_bit_sync_timer
lagging:
    sub      #1,bit_lead_lag

;*****
; Формирование бита синхронизации
;  $6/6800+5/6800+6/6800=3/1200$ 
; Это внутренний синхроимпульс, который должен опреде-
; лять скорость 1200 бод, но он имеет дребезг, так как
; формирование сигнала с частотой 1200 Гц осуществля-
; ется путем деления сигнала с частотой 6 800 Гц.
; В регистр bit_sync_timer загружается 6, а потом 5,
; а потом 6 и т.д.
;*****
update_bit_sync_timer:
    sub      #1,bit_sync_timer
    jz       load_new_timer_value ; если таймер
                                ; достиг нуля,
                                ; то загружается новое
                                ; значение фронт синх-
                                ; роимпульса сбрасыва-
                                ; ется
    cmp      #3,bit_sync_timer    ;
    jnz      done_bit_sync
    bis.b    #TPE_3,&TPD

```

```

bis.b    #CLOCK,global_status ; фронт синхро-
                                ; импульса устанавли-
                                ; вается в середине
                                ; цикла для выполнения
                                ; выборки данных, но
                                ; это не используется
                                ; в данной программе.

cmp      #20,bit_data
jge      data_is_space ; В этой точке уста-
                                ; навливается C

data_is_mark
bic.b    #TPE_5,&TPD
clrc
jmp      count_down_bits
done_bit_sync:
reti
data_is_space
bis.b    #TPE_5,&TPD
setc
count_down_bits
rrc      data_word
reti
load_new_timer_value:
bic.b    #TPE_3,&TPD
bic.b    #CLOCK,global_status ; очистка бита
                                ; внутренней синхрони-
                                ; зации.

mov      #6,bit_sync_timer
sub.b    #1,cycle_counter ; определение
                                ; положения для следу-
                                ; ющего цикла - 5
                                ; или 6

jnz      do_6_counts
do_5_counts
mov.b    #3,cycle_counter
mov      #5,bit_sync_timer ; если значение
                                ; равно 5, то нам надо
                                ; компенсировать
                                ; опережение

cmp      #-7,bit_lead_lag ;
jl       compensate_lag
reti

```

```

do_6_counts:
    cmp        #7,bit_lead_lag ; если значение
                                ; равно 6, то нам надо
                                ; компенсировать
                                ; отставание
    jge        compensate_lead
    reti
compensate_lag
    add        #1,bit_sync_timer
    mov        #0,bit_lead_lag
    reti
compensate_lead
    sub        #1,bit_sync_timer
    mov        #0,bit_lead_lag
    reti

; *****
; Выполнение алгоритма этого цикла происходит
; за 113 циклов
; *****
; *****
; Параметры нового фильтра выборки Freq_Stop (частота
; затухания): 2.5 кГц, Attenuation_Stop (подавление
; на частоте затухания): 40 дБ
; Freq_Pass (верхняя частота полосы пропускания):
; 1.4 кГц, Attenuation_Pass (подавление на верхней
; частоте): 1 дБ
; Порядок фильтра = 5
; *****
filters:
    bis        #INTERRUPT_TOGGLE,global_status
    mov        #WDF_PARMS,mem_ptr
    .word      4f16h
    .word      0000h
    .word      498fh
    .word      0000h
    .word      4f17h
    .word      0008h
    .word      8607h
    .word      4708h
    .word      1108h
    .word      4806h
    .word      1108h

```

```
.word 1108h
.word 1108h
.word 1108h
.word 1108h
.word 8806h
.word 8f16h
.word 0008h
.word 4f9fh
.word 0006h
.word 0008h
.word 468fh
.word 0006h
.word 8706h
.word 4f17h
.word 0004h
.word 8907h
.word 4708h
.word 1108h
.word 1108h
.word 1108h
.word 4809h
.word 1108h
.word 1108h
.word 1108h
.word 8809h
.word 5f19h
.word 0004h
.word 8907h
.word 4f9fh
.word 0002h
.word 0004h
.word 478fh
.word 0002h
.word 8906h
mov R6,bit_data
```

```
;*****
; Данные с выхода фильтра сохраняются в регистре R6
; Содержимое регистра R6 переводится в аналоговое
; значение после некоторых аппаратных ограничений
;*****
add #80h,R6
tst R6
```

```

        jge     non_negative
        mov     #0,R6
non_negative
        cmp     #0ffh,R6
        jlo     non_ceiling
        mov     #0ffh,R6
non_ceiling
        mov.b   R6,&P0OUT
exit_D_A
        reti

;*****
; очистка всех параметров, лишних для гребенчатого
; фильтра
;*****
clear_all_parameters:
        mov     #0,r6
        mov     #0,r7
        mov     #0,r8
        mov     #0,r9
        mov     #0,r10
        mov     #0,r11
        mov     #0,r12
        mov     #0,r13
        mov     #0,r14
        mov     #0,r15
        mov     #WDF_PARMS,r4
clear_parms_loop
        mov     #0,0(r4)
        incd    r4
        cmp     #end_of_parms,r4
        jnz     clear_parms_loop
        ret

;*****
;**** Адреса векторов прерывания:*****
        .sect   "Int_Vect",0ffe0h    ;03e0h ; 0FFE0h
        .word   Start                ;P0.27
        .word   Start                ;BT
        .word   Start                ;
        .word   Start                ;
        .word   Start                ; универсальный
                                           ; таймер

```



```
.word Start ; АЦП
.word Start ;
.word Start ;
.word Start ;
.word Start ;
.word Start ; сторожевой
; таймер
.word Start ;
.word TIM8_Int ; P0.1
.word Start ; P0.0
.word Start ; RSTI/OF
.word Start ; PUC/WDT
```

Приложение В: подпрограмма FSK передатчика

```
CPUOFF .equ 010h
SCG0 .equ 040h
SCG1 .equ 080h
IE1 .equ 0h
IE1_POIE1 .equ 08h
IE1_POIE0 .equ 04h
IE1_OFIE .equ 02h
IE1_WDTIE .equ 01h
IE2 .equ 01h
IE2_BTIE .equ 80h
IE2_TPIE .equ 08h
IE2_ADIE .equ 04h
IE2_UTXRIE .equ 02h
IE2_URXIE .equ 01h
IFG1 .equ 02h
IFG2 .equ 03h
ME1 .equ 04h
ME2 .equ 05h
P0IN .equ 010h
P0OUT .equ 011h
P0DIR .equ 012h
P0FLG .equ 013h
P0IES .equ 014h
P0IE .equ 015h
LCDCTL .equ 030h
LCDM .equ 030h ; адрес регистра
; управления и режима
; ЖКИ
```

BTCTL	.equ	040h	
BTCTL_SSEL	.equ	80h	
BTCTL_Hold	.equ	40h	
BTCTL_DIV	.equ	20h	
BTCTL_FREQ1	.equ	10h	
BTCTL_FREQ0	.equ	08h	
BTCTL_IP2	.equ	04h	
BTCTL_IP1	.equ	02h	
BTCTL_IP0	.equ	01h	
BTCNT1	.equ	046h	
BTCNT2	.equ	047h	
BTIFG	.equ	080h	; флаг прерывания ВТ
TCCTL	.equ	042h	; адрес управляющего ; регистра таймера/ ; счетчика
TCCTL_SSEL1	.equ	080h	
TCCTL_SSEL0	.equ	040h	
TCCTL_ISCTL	.equ	020h	
TCCTL_TXE	.equ	010h	
TCCTL_ENCNT	.equ	008h	
TCCTL_RXACT	.equ	004h	
TCCTL_TXD	.equ	002h	
TCCTL_RXD	.equ	001h	
TCPLD	.equ	043h	; адрес регистра пред- ; варительной загрузки ; таймера/счетчика
TCDAT	.equ	044h	; адрес таймера/счет- ; чика
TPD	.equ	04eh	
TPD_B16	.equ	080h	
TPD_CPON	.equ	040h	
TPE	.equ	04fh	
TPE_0	.equ	01h	
TPE_1	.equ	02h	
TPE_2	.equ	04h	
TPE_3	.equ	08h	
TPE_4	.equ	10h	
TPE_5	.equ	20h	
TPE_TPSSEL2	.equ	40h	
TPE_TPSSEL3	.equ	80h	
TPCTL	.equ	04Bh	
TPCTL_EN1FG	.equ	01h	

```

TPCTL_RC1FG      .equ    02h
TPCTL_RC2FG      .equ    04h
TPCTL_EN1        .equ    08h
TPCTL_ENA        .equ    10h
TPCTL_ENB        .equ    20h
TPCTL_TPSSEL0    .equ    40h
TPCTL_TPSSEL1    .equ    80h
TPCNT1           .equ    04Ch
TPCNT2           .equ    04Dh
SCFI0            .equ    050h
SCFI1            .equ    051h
SCFQCTL          .equ    052h
CBCTL            .equ    053h
AIN              .equ    0110h
AEN              .equ    0112h
ACTL             .equ    0114h
ACTL_CSTART      .equ    0001h
ACTL_SVCC_OFF    .equ    0000h
ACTL_SVCC_ON     .equ    0002h
ACTL_2           .equ    0004h
ACTL_3           .equ    0008h
ACTL_4           .equ    0010h
ACTL_5           .equ    0020h
ACTL_SEL_A0      .equ    0000h
ACTL_SEL_A1      .equ    0004h
ACTL_SEL_A2      .equ    0008h
ACTL_I_SRC_A0    .equ    0000h
ACTL_I_SRC_A1    .equ    0040h
ACTL_RNG_B       .equ    0200h
ACTL_RNG_AUTO    .equ    0800h
ACTL_POWER_UP    .equ    0000h
ACTL_CLK_MCLK    .equ    0000h
ACTL_CLK_MCLK_2  .equ    2000h
ACTL_CLK_MCLK_3  .equ    4000h
ADAT             .equ    0118h
ADIFG            .equ    04h
WDTCTL           .equ    0120h
WDTHold          .equ    80h
WDT_wrkey        .equ    05A00h
STACK            .set     3d0h ; 280h запуск систем-
                               ; ного стека

```

```
;*****
; КОНСТАНТЫ:
; delta_phase = (freq/sam_freq)*65536
;*****
_1300_Hz      .equ    01155h
_2100_Hz      .equ    01c00h
DELTA_PHASE   .equ    1

;*****
; Фильтры
;*****
sinne_value   .usect   "FILTMEM",2,200h
tx_cycle_counter .usect "FILTMEM",2
tx_cycle_ptr   .usect   "FILTMEM",2 ; подпрограмма
                                           ; обслуживания
                                           ; пользователя

delta_phase    .set     R6
phase_ptr      .set     R7
tx_data_ptr    .set     R8
tx_data_mask   .set     R9
DELAY_COUNTER  .set     R10
global_status  .set     R11
reg_1          .set     R12
reg_2          .set     R13
reg_3          .set     R14
TX_DONE        .set     1
FALLING        .set     2
CLOCK          .set     4
HUNT           .set     8
_20MS          .set     136
_1PT5S         .set     28800

;*****
; Инициализация системы
;*****
Start          .sect     "ADC",0f000h          ;0214h
                                           ;0F000h

                mov      #STACK,SP            ; инициализация
                                           ; системного указателя
                                           ; стека
                mov      #(WDTHold+WDT_wrkey), &WDTCTL
                                           ; остановка сторожево-
                                           ; го таймера
```

```

        clr.b    &IFG1                ; очистка всех флагов
                                         ; прерываний
        clr.b    &IFG2
        mov.b    #(75)-1,&SCFQCTL ; MCLK=32768*17*4
                                         ; получаем 2.45 MIPS
        mov.b    #4,&SCFIO            ; установка частоты RC
                                         ; генератора 2xFreq
        mov.b    #-128,&TCPLD        ; 32768*75/128 = 19200
                                         ; выборка/секунду
        mov.b    #(TCCTL_SSEL1+TCCTL_ISCTL +
                    TCCTL_ENCNT),&TCCTL
        mov.b    #IE1_P0IE1,&IE1 ; разрешение 8-бит-
                                         ; ного таймера
        bis.b    #11111111b,&P0DIR ; установка пор-
                                         ; тов P0.7-P0.0
                                         ; на работу в качестве
                                         ; выходов
        eint

;**** Основная программа:*****
Loop                                ; ожидание прерывания
        mov      #TX_DATA_TABLE,tx_data_ptr
        mov      #08000h,tx_data_mask
        mov      #0,phase_ptr
        mov      #1,tx_cycle_counter
        bic      #TX_DONE,global_status
        call     #fetch_new_output_bit
        call     #fsk_modulation
wait_for_tx_done
        bit      #TX_DONE,global_status
        jz       wait_for_tx_done
Loop2:
        jmp      Loop2
TIM8_Int:
NORMAL_MOD:
        call     #fsk_modulation

;*****
; Эта часть программы формирует сигнал на выходе
; P0OUT, к которому должен быть подключена 8-битная
; R-2R цепочка. Она используется для отслеживания
; отфильтрованного значения, но если необходимо
; использовать порт для формирования сигнала

```

```

; FSK передатчика, то она должна быть отключена
; *****
D_A
    mov     sinne_value, reg_1
    mov.b   reg_1, &P0OUT ; MSB содержится
                        ; в седьмом бите.
    reti

; *****
; Таблица слежения
; *****
fsk_modulation:
    add     delta_phase, phase_ptr
                        ; таблица содержит
                        ; 128 элементов,
                        ; 4*128 = 512 = ± 256
                        ; извлекаются старшие 9 бит
    mov     phase_ptr, reg_1
                        ; таблица косинусов 3-
                        ; и 4-ого квадрантов
                        ; переносятся на 1- и 2-ой
                        ; квадранты
    tst     reg_1
    jge     no_tx_abs
    xor     #0ffffh, reg_1
    add     #1, reg_1
no_tx_abs:
    swpb    reg_1 ; LSB (МЗБ) результата в MSB
                ; (СЗБ) регистра reg_1, 8 MSB
                ; бит в младшем байте регист-
                ; ра reg_1
    bit     #8000h, reg_1
    rlc     reg_1 ; С в бит bit0, 8 MSB бит
                ; в биты 1-8
    bic     #0fe00h, reg_1
                ; формат Q8
first_two_quadrant:
    cmp     #128, reg_1
    jn      first_quadrant
second_quadrant
    mov     #256, reg_2
    sub     reg_1, reg_2
    mov.b   cos_table(reg_2), reg_2

```

```

        xor        #0ffffh,reg_2
        add        #1,reg_2
        jmp        output_sample
first_quadrant                                ; 0-127 градусов
        mov.b      cos_table(reg_1),reg_2
output_sample:
        add        #80h,reg_2
        mov        reg_2,sinne_value        ; результат
                                                ; в переменной
                                                ; sinne_value

;*****
;начало сервисной подпрограммы fetch_new_out_bit
;*****
fetch_new_output_bit:
        dec        tx_cycle_counter
        jnz        NO_RESET_TX_PTR
                                ; при производительности
                                ; 19200 выборок/сек div16 =
                                ; 1200 бод
        mov        #16,tx_cycle_counter
load_next_cycle:
        mov        #_2100_Hz,delta_phase ; примите это
                                                ; сначала

        mov        tx_data_ptr,reg_1
        bit        @reg_1,tx_data_mask
        jnz        TX_BIT_IS_1
TX_BIT_IS_0:
        mov        #_1300_Hz,delta_phase
TX_BIT_IS_1:
        rra        tx_data_mask
        BIC        #8000h,tx_data_mask
        jnc        NO_TX_PTR_UPDATE
        mov        #8000h,tx_data_mask
        add        #2,tx_data_ptr
        mov        tx_data_ptr,reg_1
        tst        0(reg_1)
        jnz        NO_RESET_TX_PTR
        mov        #TX_DATA_TABLE,tx_data_ptr
        bis        #TX_DONE,global_status
NO_TX_PTR_UPDATE
NO_RESET_TX_PTR
        ret

```

```
;*****
; конец сервисной подпрограммы fetch_new_out_bit
;*****
TX_DATA_TABLE
    .word 05555h
    .word 05555h
    .word 05555h
    .word 05555h
    .word 05555h
    .word 0FFFFh
    .word 0FFFFh
    .word 0FFFFh
    .word 0FFFFh
    .word 0FC07h

;*****
; это имело множество битов меток
; содержание: 5 байт, 1 3 5 7 9
;*****
    .word 0FC0Bh
    .word 0FC03h
    .word 0FC07h
    .word 0FC0Bh
    .word 0FC0Fh
    .word 0FC13h
    .word 0

cos_table:
    .byte 07fh
    .byte 07fh
    .byte 07fh
    .byte 07fh
    .byte 07fh
    .byte 07fh
    .byte 07fh
    .byte 07fh
    .byte 07fh
    .byte 07fh
    .byte 07eh
    .byte 07eh
```



```
.byte 07eh
.byte 07eh
.byte 07dh
.byte 07dh
.byte 07dh
.byte 07ch
.byte 07ch
.byte 07ch
.byte 07bh
.byte 07bh
.byte 07ah
.byte 07ah
.byte 07ah
.byte 079h
.byte 079h
.byte 078h
.byte 077h
.byte 077h
.byte 076h
.byte 076h
.byte 075h           ;cos 23.2031
.byte 075h
.byte 074h
.byte 073h
.byte 073h
.byte 072h
.byte 071h
.byte 070h
.byte 070h
.byte 06fh
.byte 06eh
.byte 06dh
.byte 06ch
.byte 06ch
.byte 06bh
.byte 06ah
.byte 069h
.byte 068h
.byte 067h
.byte 066h
```

```

.byte 065h
.byte 064h
.byte 063h
.byte 062h
.byte 061h
.byte 060h
.byte 05fh
.byte 05eh
.byte 05dh
.byte 05ch
.byte 05bh
.byte 05ah
.byte 059h
.byte 058h
.byte 057h
.byte 055h
.byte 054h
.byte 053h
.byte 052h
.byte 051h
.byte 04fh
.byte 04eh
.byte 04dh
.byte 04ch
.byte 04ah
.byte 049h
.byte 048h
.byte 047h
.byte 045h
.byte 044h
.byte 043h
.byte 041h
.byte 040h
.byte 03fh
.byte 03dh           ;cos 61.1719
.byte 03ch
.byte 03ah
.byte 039h
.byte 038h
.byte 036h

```

```
.byte 035h
.byte 033h
.byte 032h
.byte 030h
.byte 02fh
.byte 02eh
.byte 02ch
.byte 02bh
.byte 029h
.byte 028h
.byte 026h
.byte 025h
.byte 023h
.byte 022h
.byte 020h
.byte 01fh
.byte 01dh
.byte 01ch
.byte 01ah
.byte 018h
.byte 017h
.byte 015h
.byte 014h
.byte 012h
.byte 011h
.byte 00fh
.byte 00eh
.byte 00ch
.byte 00ah
.byte 009h
.byte 007h
.byte 006h
.byte 004h
.byte 003h
.byte 001h ;cos 89.2969
```

```
;*****
;**** адреса векторов прерывания:*****
.sect "Int_Vect",0ffe0h ;03e0h ;0FFE0h
.word Start ;P0.27
.word Start ;BTIM_Int ;BT
```

```

.word    Start           ;
.word    Start           ;
.word    Start           ;UTIM_Int
                        ; универсальный
                        ; таймер

.word    Start           ;АЦП
.word    Start           ;
.word    Start           ;
.word    Start           ;
.word    Start           ;
.word    Start           ;сторожевой
                        ; таймер

.word    Start           ;
.word    TIM8_Int        ;P0.1
.word    Start           ;P0.0
.word    Start           ;RSTI/OF
.word    Start           ;PUC/WDT

```

IV. Примеры конкретных применений

Глава 11. *Программный встроенный модем на MSP430*

Программный встроенный модем на MSP430

Wolfgang Lutsch [slaa204.pdf (Англ.)]

В данном примере применения подробно описывается разработка модема стандарта V.21 на базе микроконтроллера MSP430. Алгоритмы модуляции и демодуляции стандарта V.21 реализованы программно и поддерживаются периферийными модулями MSP430. В данном документе приведены как описание аппаратной части (схема электрическая принципиальная и спецификация (BOM)), так и описание программного обеспечения (исходный текст прикладной программы на «C»).

1. Введение

Цель этого примера применения состоит в том, чтобы показать, как добавить микроконтроллерному устройству основные функциональные возможности модема. Чтобы подключить устройство к телефонной линии необходимо всего несколько внешних элементов. Такое подключение может быть реализовано при помощи демонстрационной платы DAA.

Данный программный модем предназначен для работы только с входящим соединением. Такое упрощение связано с тем, что ставилась задача только показать возможность создания программного модема. Кроме того, в данном примере применения описывается, как установить соединение между программным модемом на базе MSP430 и стандартным модемом персонального компьютера.

2. Использование модема стандарта V.21 на телефонных сетях общего пользования (ТФС ОП)

Основная функция модема состоит в преобразовании двоичных сигналов в аналоговые сигналы, которые могут передаваться в полосе телефонного канала, и наоборот — принятых аналоговых сигналов в двоичные сигналы. CCITT/ITU определяют различные стандарты для модемов. Ниже некоторые основные технические требования к модемам стандарта V.21:

- Полнодуплексное соединение «точка-точка» для передачи данных по телефонной линии
- Двоичная частотная манипуляция (FSK)
- Скорость передачи данных до 300 бит/сек (бод)

Параметры модуляции стандарта V.21:

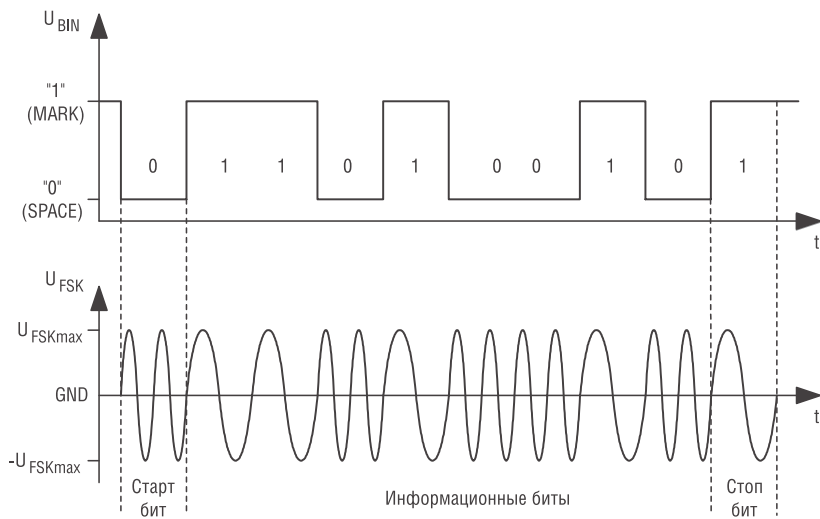
- Два частотных канала для передачи данных в обоих направлениях
 - Центральная частота первого канала $f_1 = 1080$ Гц
 - Центральная частота второго канала $f_2 = 1750$ Гц
- Частоты для передачи двоичной '1' (MARK) и двоичного '0' (SPACE) в каждом канале:
 - $f_{\text{MARK}} = f_x - 100$ Гц
 - $f_{\text{SPACE}} = f_x + 100$ Гц, где f_x – центральная частота соответствующего канала (см. Таблицу 1)
- Вызывающий модем передает данные по первому каналу, а получает их по второму каналу (режим исходящего звонка)
- Вызываемый модем передает данные по второму каналу, а принимает их по первому каналу (режим ответа)

Таблица 1. Частотные параметры FSK сигналов стандарта V.21

	Канал 1	Канал 2
Центральная частота	1080 Гц	1750 Гц
Девияция частоты	100 Гц	100 Гц
Двоичная «1»	980 Гц	1650 Гц
Двоичный «0»	1180 Гц	1850 Гц

Стандарт V.21 допускает нестабильность частоты на выходе модулятора не более ± 6 Гц. Демодулятор должен быть способен распознать частоты в пределах диапазона ± 12 Гц от ожидаемых значений. Стандарт V.21 определен для передачи данных как в синхронном, так и в асинхронном режиме. Для стандартного двухточечного соединения модемом используется асинхронные режим с одним стартовым битом, восьмью информационными битами и одним стоп битом. На рисунке 1 приведены двоичный и FSK сигналы для символа «K» в кодировке ASCII (0x4B). Данные передаются младшим значащим битом вперед (LSB).

Приведенный программный модем на базе MSP430 работает только в режиме ответа. Поэтому каналы передачи данных имеют вид, представленный на рисунке 2. Программный модем на базе MSP430 передает данные по второму каналу, а принимает – по первому.



Замечание: масштаб эпюр колебаний не соблюден

Рис. 1. Двоичный и FSK сигналы символа «К» в кодировке ASCII

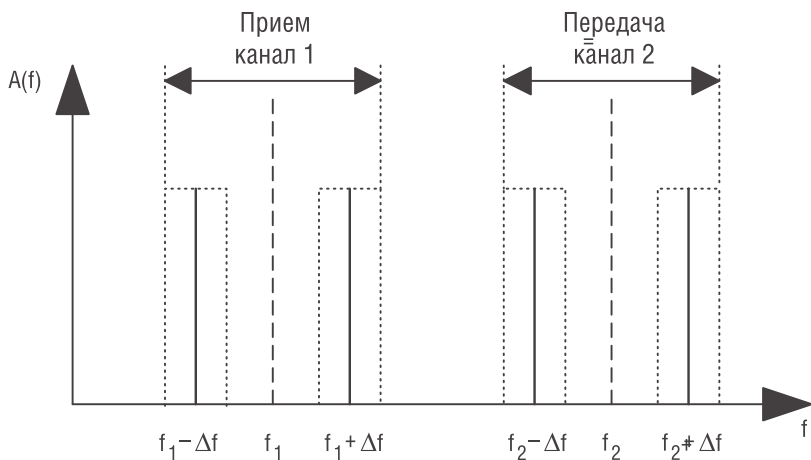


Рис. 2. Частотные спектры каналов передачи данных стандарта V.21

3. Описание аппаратной части

3.1. Описание схемы демонстрационной платы

Описываемый демонстрационный программный модем на базе MSP430 состоит из двух плат. Одна плата содержит микроконтроллер MSP430F149, который может быть заменен менее мощным микроконтроллером семейства MSP430, например – MSP430F1121. Другая плата LITELINK III содержит ИС линейного телефонного интерфейса производства компаний Clare/Ixys. Эта плата позволяет подключить программный модем к телефонной линии. Имеется две модификации этой отладочной платы DAA – CPC5621-EVAL-RDL (активное входное сопротивление по переменному току – в соответствии со стандартом США) или CPC5621-EVAL-CDL (активное входное сопротивление по переменному току – в соответствии со стандартом стран Европы).

Вместе с микроконтроллером задействованы следующие элементы:

- 3.3 В стабилизатор напряжения
- Разъем JTAG интерфейса для внутрисистемной отладки и программирования
- ОУ для обработки сигналов
- Разъем RJ11 для подключения к телефонной линии
- Три светодиода для индикации:
 - наличия требуемого напряжения питания
 - статуса соединения модема
 - трафика данных соединения
- Разъем для подключения источника питания и обеспечения доступа к портам P6.6 и P6.7

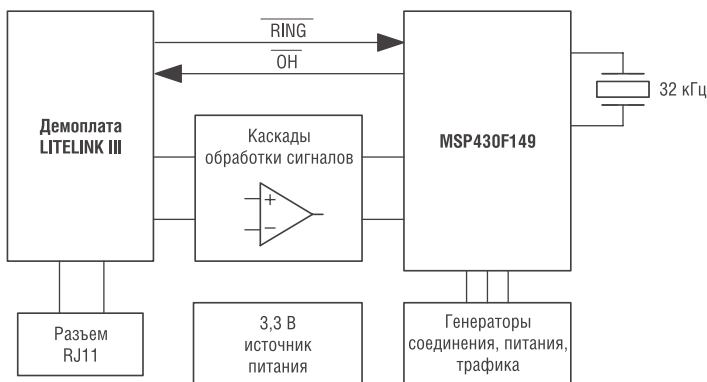


Рис. 3. Структурная схема аппаратной части программного модема

В приложении А приведена схема электрическая принципиальная описываемого программного модема. В приложении В приведена спецификация (перечень элементов) для этой платы.

3.2. Отладочная плата LITELINK III (DAA)

LITELINK III – однокристалльная DAA, используемая в устройствах передачи речи и данных для подключения к телефонной линии. Она содержит высоковольтную защиту, формирует для телефонной линии сопротивление по переменному и постоянному току, осуществляет имитацию поднятия трубки (занятие линии), а также производит детектирование вызывного сигнала и сигнала «отбой». Она может использоваться в дифференциальных и несимметричных режимах. Программный модем на базе MSP430 использует DAA в несимметричном режиме.

Так как DAA содержит высоковольтную защиту, то для формирования законченного телефонного интерфейса требуется всего несколько внешних элементов. Отладочная плата, которая представляет собой предварительно настроенный и отлаженный узел, облегчает использование и отладку, а также обеспечивает соответствие стандартам.

Так как линейная часть ИС подключена к телефонной линии, то дополнительное питание от изолированного источника ей не требуется. Остальная часть ИС может питаться от 3.3 В источника питания, что позволяет осуществлять питание от автономного источника питания и облегчает формирование интерфейса непосредственно с MSP430.

Более полную информацию о LITELINK III можно получить из технического описания ИС «CPC5620/CPC5621 LITELINK III Phones Line Interface IC (DAA)» и руководства пользователя для демонстрационной платы «CPC5621 Evaluation Board», которые можно найти на сайте производителя компании Clare Inc.

3.3. Подключение к DAA

3.3.1. Управляющие сигналы

Программный модем на базе MSP430 использует два управляющих сигнала DAA, которые непосредственно подаются на порты ввода-вывода микроконтроллера MSP430.

Первый управляющий сигнал – выходной сигнал LITELINK III – сигнал обнаружения вызова RING, который подается непосредственно на способный формировать прерывание порт P2.5 MSP430. Выходной сигнал RING обычно удерживается в высоком состоянии, а при поступлении вызова на нем появляются импульсы.

Второй управляющий сигнал, сигнал занятия линии, является для DAA входным и обозначается popON. Формируется этот сигнал на порту P2.1. В таблице 2 приведены требуемые состояния сигнала popON для двух режимов работы.

Таблица 2. Управление занятием линии

popON	Режим работы	Описание
Высокий уровень	Устройство отключено от линии	DAA готов занять линию или принять вызов, схема слежения за линией активна, а при поступлении вызова на выходе RING формируется сигнал.
Низкий уровень	Устройство подключено к линии (занятие линии)	Линейный ток протекает через линейный терминал устройства, которое осуществляет ответ или вызов другого абонента.

3.3.2. Передающий тракт

На выходе FSK модулятора MSP430 (вывод P1.7 (TA2)) присутствует прямоугольный сигнал с амплитудой 3.3 В. Далее этот сигнал ограничивается резистивным делителем (R10, R11). Дополнительный конденсатор (C11) формирует простейший ФНЧ первого порядка, который подавляет высокочастотные составляющие сигнала. Есть еще две причины, обуславливающие необходимость ограничения уровня выходного сигнала:

- Максимальный допустимый уровень входного сигнала для LITELINK III в несимметричном режиме равняется 1.1 В. Этот уровень соответствует мощности -6 dBm при работе на 600 Ом линейную нагрузку
- Уровень передаваемого модемом сигнала должен быть в пределах от -43 dBm до -9 dBm

В таблице 3 приведены уровни результирующих сигналов для частот 1650 Гц и 1850 Гц при использовании ФНЧ (см. приложение А).

$$P(U_{tx}) = 10 \times \log \left(\frac{\left(\frac{U_{tx}}{2 \times \sqrt{2}} \right)^2}{600 \text{ Ом} \times 1 \text{ мВт}} \right)$$

$$U_{tx}(f) = \frac{1}{1 + j \times 2\pi \times 9,1 \text{ кОм} \times 10 \text{ нФ} \times f}$$

Таблица 3. Уровни передаваемых сигналов

Частота (f)	Уровень сигнала P (Utx)
1650 Гц	-24 dBm
1850 Гц	-24.5 dBm

3.3.3. Приемный тракт

Сначала переменный входной сигнал подается на схему смещения на $V_{cc}/2$ (см. цепочку R12, R13 на схеме электрической принципиальной), а потом усиливается в 18 раз усилителем IC2A. Кроме того, сигнал подается на ФНЧ второго порядка, который ограничивает полосу сигнала до 1.3 кГц (поскольку частоты входного сигнала будут находиться в диапазоне от 980 Гц до 1180 Гц). Выходной сигнал подается на компаратор Comparator_A MSP430 через порт P2.3 для демодуляции.

На рисунке 4 показан принятый сигнал на входе (канал 1) и выходе (канал 2) компаратора. Сигнал RX+ несколько искажен за счет нелинейности приемного канала и за счет того, что модуляция и демодуляция осуществляется в одном и том же тракте.

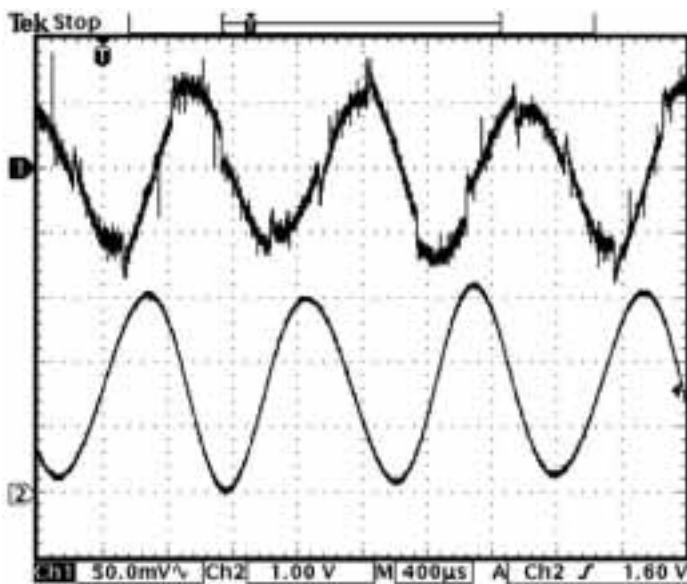


Рис. 4. Эпюры принятого сигнала на входе и выходе компаратора

4. Описание программы

В данной главе описывается алгоритм детектирования вызывного сигнала и модуль V.21. FSK модулятор/демодулятор V.21 реализован при помощи аппаратных модулей MSP430 и программного обеспечения, написанного на «С». Детектор вызывного сигнала и модули V.21 инициализируются по прерываниям и работают в фоновом режиме. Проект разделен на разные модули (см. Таблицу 4).

Таблица 4. Краткий обзор программных модулей

Модуль	Описание
Прикладная программа (softmodem.c, softmodem.h)	Реализует интерфейс пользователя при помощи модуля V.21 для передачи данных по модемному каналу.
Модуль V.21 (v_21.c, v_21.h)	Содержит все программы и подпрограммы, реализующие функциональные возможности модема V.21, включая функции, необходимые для передачи данных при помощи UART.
Детектор вызывного сигнала (ring_detection.c, ring_detection.h)	Содержит подпрограмму инициализации детектора вызывного сигнала и формирования основных временных интервалов, используемых модули V.21.

4.1. Модуль детектора вызывного сигнала

Входящий вызывной сигнал представляет собой гармонический сигнал с амплитудой от 48 В до 60 В и частотой от 16 Гц до 64 Гц. Огибающая вызывного сигнала представляет собой прямоугольный сигнал с периодом 5 с и длительностью активного импульса 1 с (см. рисунок 5). Однако в различных странах параметры вызывного сигнала могут варьироваться в зависимости от принятого в конкретной стране стандарта.

Выход RING подключен к порту P2.5. Так как входной сигнал содержит шум, то простой подсчет количества импульсов не является помехозащищенным алгоритмом. Для правильного детектирования используется специальный временной алгоритм. Этот алгоритм состоит из процедур обработки прерывания (ISR), инициализируемых портом P2.5 ISR (см. рисунок 6) и сторожевым таймером WDT (см. рисунок 7). WDT вызывает ISR каждые 250 мс.

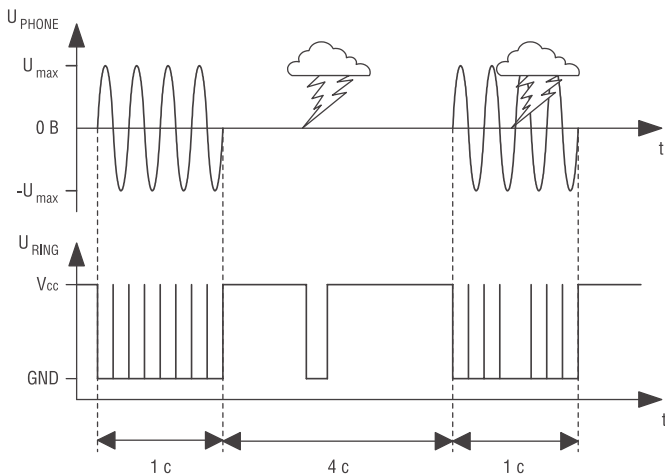


Рис. 5. Вызывной сигнал



Рис. 6. Блок-схема подпрограммы обработки прерывания по порту P2.5 алгоритма детектирования вызывного сигнала

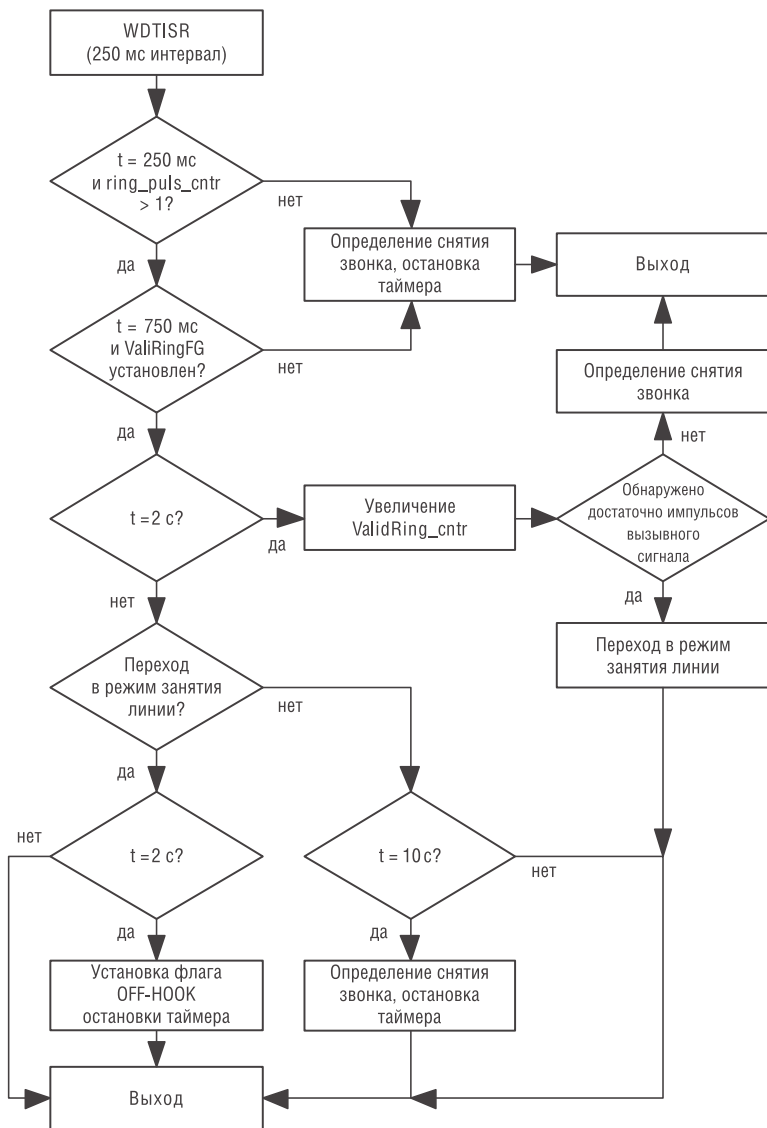


Рис. 7. Блок-схема подпрограммы обработки прерывания от WDT алгоритма детектирования вызывного сигнала

4.2. Модуль V.21

При реализации стандарта V.21 были допущены некоторые упрощения. На выходе FSK модулятора вместо гармонического сигнала формируется прямоугольный сигнал. Кроме того, демодуляция частоты входного сигнала осуществляется путем измерения периода колебаний. Для формирования исходящего и входящего потоков и упрощения реализации API используется аппаратный модуль USART.

Для формирования временных интервалов в алгоритмах модуляции/демодуляции используется встроенный в MSP430 генератор с цифровым управлением частотой (DCO). Частота DCO устанавливается и поддерживается равной 4096 Гц при помощи процедуры Set_DCO (). Частоту DCO можно изменить путем изменения константы DELTA в заголовке файла v_21.h. По умолчанию для DCO задана частота 1.024 МГц ($\text{DELTA} = 250$), что позволяет согласно требованиям стандарта V.21 формировать на выходе FSK модулятора частоты с уходом не более ± 6 Гц. Если установить частоту DCO ниже 999 кГц, то это требование выполнено не будет. Все описываемые ниже константы задаются в заголовке файла программы модуля V.21 и зависят от выбранной частоты DCO. Также DCO используется для формирования синхросигналов MCLK и SMCLK.

4.2.1. FSK модулятор

Для обеспечения интерфейса между пользовательской прикладной программой и прикладной программой микроконтроллера используется модуль USART0 (см. рисунок 8). Он настроен на работу в режиме UART в асинхрон-

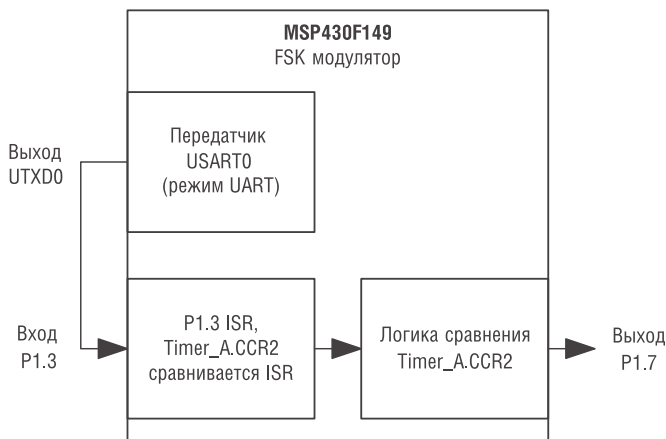


Рис. 8. Функциональная схема FSK модулятора

ном режиме со скоростью 300 бит/с. После записи байта данных в U0TXBUF, модуль USART начинает формировать на выводе UTXD0 соответствующий цифровой поток, состоящий из стартового бита, восьми информационных разрядов, передаваемых младшим значащим битом вперед, и стоп бита.

Этот вывод подключен непосредственно к выводу порта P1.3. Модуль захвата/сравнения Timer_A.CCR2 работает в режиме сравнения и используется для формирования сигнала с ШИМ. При помощи таймера Timer_A и модуля CCR2 на выходе порта P1.7 формируется сигнал с ШИМ. Значения, которые периодически добавляются к содержимому CCR2 для получения требуемой выходной частоты рассчитываются по следующей формуле:

$$\text{HalfPeriodValue} = \text{Timer_A.CLK} / (2 \times \text{SignalFrequency})$$

Цифровой порт ввода – вывода P1.3 настроен на работу в режиме входа, при этом для него разрешено прерывание. Подпрограмма обработки этого прерывания запускается при каждом изменении сигнала UTXD0. На рисунке 9 приведена блок-схема алгоритма этой ISR.

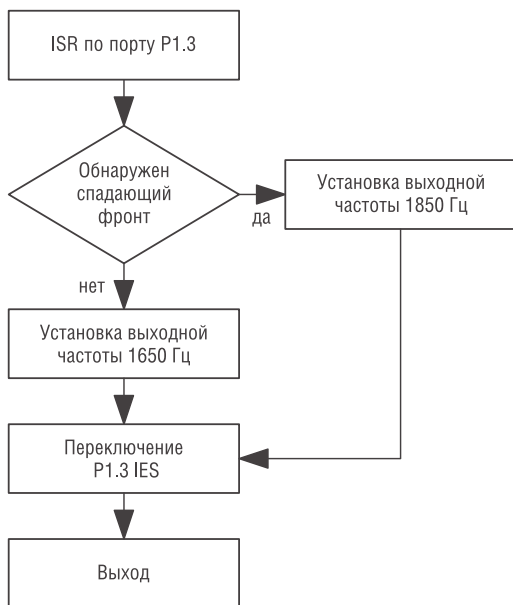


Рис. 9. Блок-схема подпрограммы обработки прерывания по порту P1.3

При чтении P1IES.3 ISR определяет, фронт сигнала (нарастающий или спадающий). В зависимости от этого обновляется переменная BitFreq, которая и определяет период формируемого сигнала с ШИМ. Вследствие этого период выходного сигнала зависит от потока данных UART (см. рисунок 10).

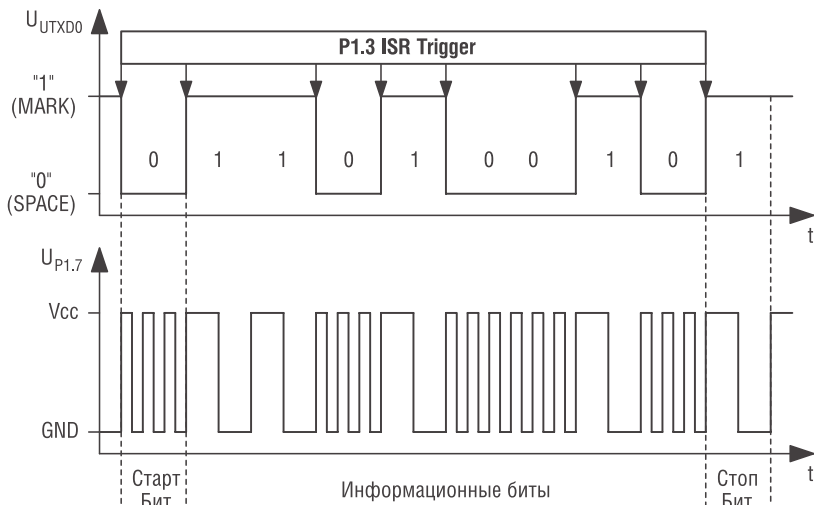


Рис. 10. Эпюры колебаний FSK модулятора

4.2.2. FSK демодулятор

После прохождения через ОУ принятый сигнал попадает на вывод P2.3 (CA0), который является входом модуля компаратора Comparator_A, который сравнивает его с опорным напряжением, равным половине питания ($V_{CC}/2$). При помощи блока захвата сравнения Timer_A.CCR1 производится измерение длительности и периода принятых импульсов. Захват осуществляется по изменению сигнала на выходе компаратора модуля Comparator_A при каждом прохождении сигнала через ноль. На рисунке 11 приведена блок-схема алгоритма подпрограммы обработки прерывания от CCR1.

ISR считывает из CCR1 захваченное значение и высчитывает длительность импульсов входного сигнала путем измерения интервала времени между переключениями. Это значение сравнивается со значениями, соответствующими значениям частот сигналов 'MARK' (980 Гц) и 'SPACE' (1180 Гц) с учетом значения допустимого возможного ухода частоты (CHN1_MARGIN). Длительность

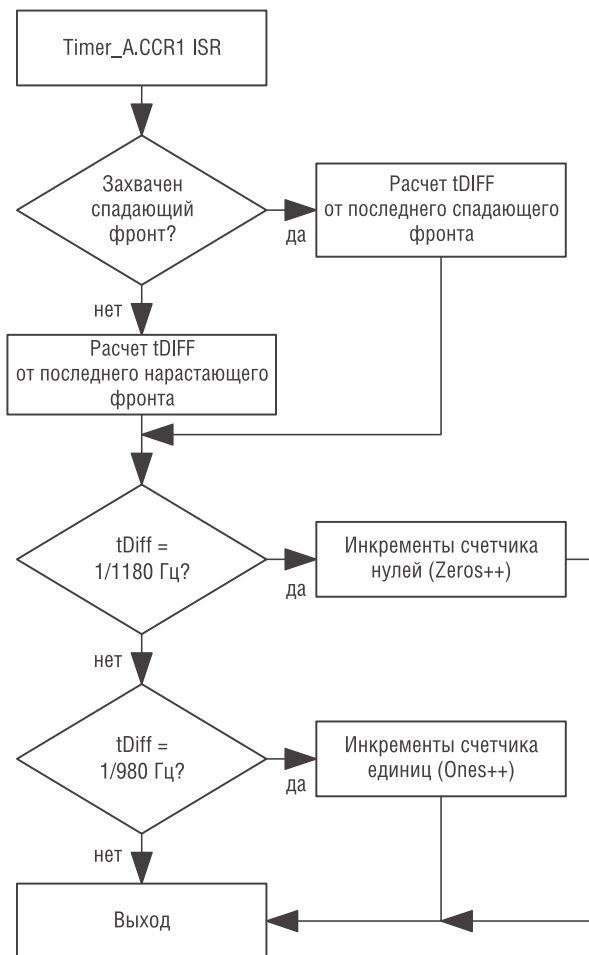


Рис. 11. Блок-схема алгоритма подпрограммы обработки прерывания от Timer_A.CCR1

рассчитывается по приведенной ниже формуле. В таблице 5 приведены значения, используемые при расчетах.

$$\text{PeriodValue} = \frac{\text{Timer_ACLK}}{\text{SignalFrequency}}$$

Таблица 5. Значения, соответствующие длительностям сигналов при демодуляции FSK сигнала

Использованные величины: Timer_A.CLK=1.024 МГц, CHN1_MARGIN=88			
Частотные параметры	Величина длительности	Интервал допустимых значений	
		Величина длительности	Частота
1180 Гц	867	955 – 779	1072 Гц – 1315 Гц
980 Гц	1044	1132 – 956	905 Гц – 1071

Если измеренная величина попадает в один из интервалов, то происходит увеличение соответствующего счетчика периодов.



Рис. 12. Блок-схема алгоритма подпрограммы обработки прерывания от модуля захвата Timer_A.CCR0

После обнаружения старт бита (распознавания шести периодов 'SPACE'), модуль захвата/сравнения Timer_A.CCR0 переходит в режим сравнения и запускается каждую 1/300 секунды (в соответствии со скоростью передачи данных). Каждый раз, когда ISR запускается, используется мажоритарное декодирование для определения какой последний бит был принят. В результате этого определяется состояние выхода P1.2, а затем счетчики очищаются, чтобы подготовиться к демодуляции следующего бита (см. рисунок 12).

Для отслеживания количества принятых битов байта используется переменная (BitCounter). После принятия восьми битов прерывания от Timer_A.CCR0 запрещаются и формируется стоп бит. Результирующий поток данных формируется на выходе P1.2 со скоростью 300 бит/сек. На рисунке 13 показаны эпюры принимаемого и декодированного сигналов.

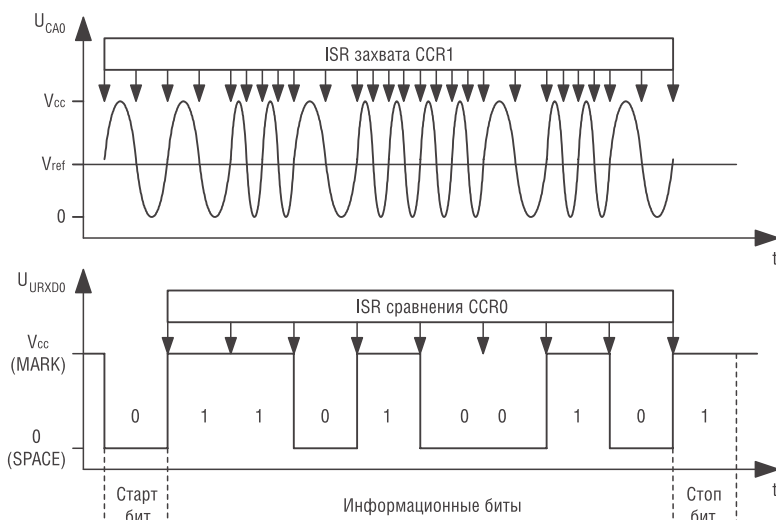


Рис. 13. Взаимодействие ISR от Timer_A.CCR1 и CCR0

Внешне выход P1.2 подключен ко входу модуля USART0 (вывод URXD0). Для формирования потока данных и обеспечения интерфейса с внешним устройством (например, ПК) модуль USART используется в режиме UART (см. рисунок 14). Соответствующее прерывание от приемника может использоваться для захвата принятых байтов.

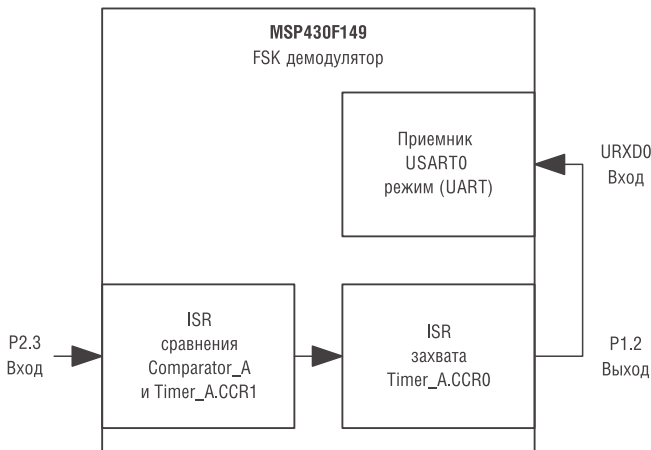


Рис. 14. Функциональная схема блока FSK демодулятора

4.2.3. Установление связи и управление занятием линии

Перед обменом данными по телефонной линии связывающиеся модемы должны договориться об общем стандарте связи. Так как этот процесс времязависимый, то для формирования временных меток используется модуль захвата/сравнения Timer_A.CCR0 работающий в режиме сравнения. Подпрограмма обработки прерывания от Timer_A.CCR0 вызывается каждую 1/300 секунды и позволяет обеспечить алгоритм установления связи (см. рисунок 15).

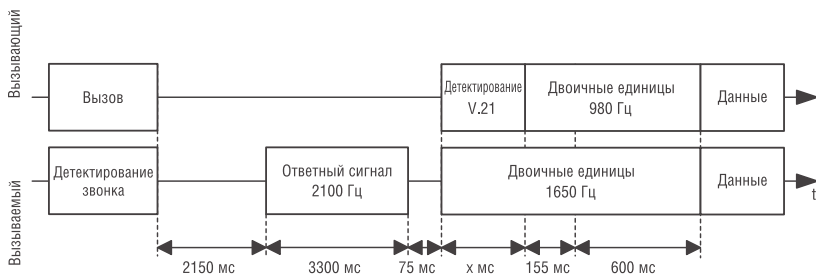


Рис. 15. Установление связи по протоколу V.21

После обнаружения вызывного сигнала программный модем на базе MSP430 подключается к телефонной линии. В этот момент тракты приема и передачи активны и MSP430 осуществляет установку связи.

После начального периода молчания длительностью 2150 мс (2 секунды из этого периода используются алгоритмом детектирования звонка), программный модем MSP430 посылает ответный тон, частота которого 2100 Гц, а длительность 3300 мс. Формируется этот сигнал при помощи модуля захвата/сравнения Timer_A.CCR2.

После дополнительного периода молчания длительностью 75 мс, программный модем MSP430 активизирует FSK модулятор и начинает посылать двоичные единицы (1650 Гц). Теперь вызывающий модем имеет время для распознавания передачи по протоколу V.21. После обнаружения стандарта V.21 сам вызывающий модем начинает передавать двоичные единицы (980 Гц). Необходимо не менее 155 мс для их распознавания программным модемом MSP430. Осуществляется это путем подсчета длительности сигналов при помощи ISR захвата Timer_A.CCR1. После того, как подсчитано достаточное количество периодов сигналов (DetectTime_Chn1), оба модема ожидают еще 600 мс, продолжая передавать единицы, а после этого переходят в режим передачи данных.

4.2.4. Работа модуля программного модема

Для использования реализованного модема V.21 пользователю надо вызывать только две функции. Конечный автомат, который реализован в функции modem(), используется для отслеживания текущего состояния (продолжается связь или нет). Вторая функция – ModemInit (). При вызове этой функции внутреннее состояние модуля изменится на CommandMode. Это начальное состояние, в котором происходит установка модуля в режим ожидания входящего вызова. Также ModemInit() выполняет основную инициацию программного модема V.21. На рисунке 16 показан упрощенный граф конечного автомата. В нем не отображены некоторые процессы (простой и переход в состояние CommandMode).

Первый вызывной сигнал переводит модем в состояние RingDetection. После обнаружения следующего вызывного сигнала алгоритм детектирования вызывного сигнала устанавливает флаг состояния OFF_HOOK. Это приводит к переходу в состояние AnsweringHandshake. После установления связи конечный автомат переходит в режим DataMode. Теперь модем полностью активен и передача данных по телефонной линии возможна путем простого обращения к модулю USART MSP430.

Окончание связи другим модемом определяется при помощи следующего алгоритма: Периодически вызываемая функция modem() сбрасывает флаг OFF_HOOK. При наличии входного сигнала флаг OFF_HOOK будет установлен снова при каждом обнаружении перехода сигналом через ноль (Timer_A.CCR1 ISR). Если входной сигнал отсутствует, то функция modem() генерирует условие простоя, останавливает работу модуля FSK, повторно инициализирует программный модем и переходит в режим CommandMode.

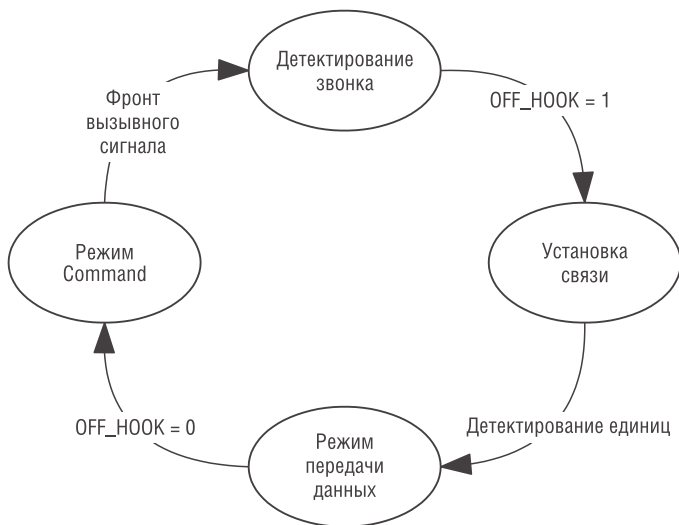


Рис. 16. Граф конечного автомата модема

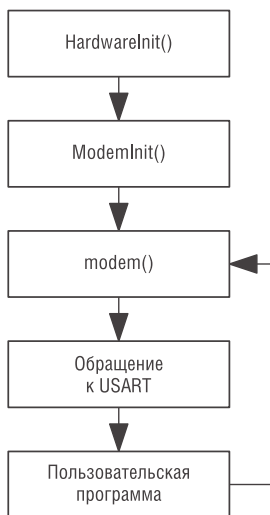


Рис. 17. Блок-схема пользовательской программы, использующей модем V.21

На рисунке 17 показана блок-схема алгоритма пользовательской программы, использующей модем V.21.

4.3. Демонстрационное применение программного модема

Пример программы, входящий в данный пример применения, демонстрирует использование модуля V.21 в комбинации с простым интерфейсом пользователя, который позволяет считывать из MSP430 измеренные данные. На рисунке 18 показан возможный вариант соединения. Персональный компьютер со стандартным модемом подключен к одному разьему симулятора телефонной линии, а программный модем на базе MSP430 – к другому.



Рис. 18. Тестовая схема подключения программного модема на базе MSP430

На ПК необходимо использовать программу Hyper Terminal, входящую в состав ОС Windows. Перед проведением теста убедитесь, что выбран правильный порт и установлены следующие параметры порта: 8 бит данных, нет бита проверки на четность и установлен один стоп бит. Скорость обмена данными может быть произвольной, так как она используется только для обмена данными между ПК и стандартным модемом. Модемы договорятся о скорости передачи данных по каналу сами.

После настройки программы Hyper Terminal можно протестировать модем при помощи следующих «AT» команд:

- ATZ <Ввод DESIGNTIMESP=27099> (выполняет сброс модема)
- ATDTx <Ввод DESIGNTIMESP=27101> (набирает последовательность цифр «х», которая должна соответствовать номеру порта, к которому подключен программный модем на базе MSP430)

После набора номера и установления соединения модем персонального компьютера сигнализирует об установлении связи с программным модемом MSP430, передавая программе Hyper Terminal строку «MSP430 Soft-Modem Demo». Посылая программному модему на MSP430 кодовые комбинации, стан-

дартный модем получает соответствующие строки, содержащие информацию о состоянии (см. Таблицу 6).

Таблица 6. Сообщения программного модема

Кодовая комбинация	Ответ программного модема
0	Программный модем MSP430 переключает порт P1.0, к которому подключен желтый светодиод. В зависимости от состояния порта светодиод загорается или тухнет.
6	Передается состояние порта P6.6. В зависимости от уровня сигнала на этом входе передается строка «P6.6 Input: HIGH» или «P6.6 Input: LOW».
7	Аналогично описанному выше, но передается состояние порта P6.7.
t	Возвращается результат аналого-цифрового преобразования встроенного температурного датчика.
x	Программный модем MSP430 возвращает строку «Good Bye!» и разрывает модемное соединение.
Любой другой символ	Полученный символ возвращается стандартному модему

5. Используемые аппаратные ресурсы MSP430

Программный модем V.21 использует следующие аппаратные ресурсы микроконтроллера:

- Таймер Timer_A3
- Компаратор Comparator_A
- Модуль USART
- Сторожевой таймер
- Два цифровых порта ввода – вывода
- 1.7 КБ Flash памяти, 100 байт ОЗУ

6. Заключение

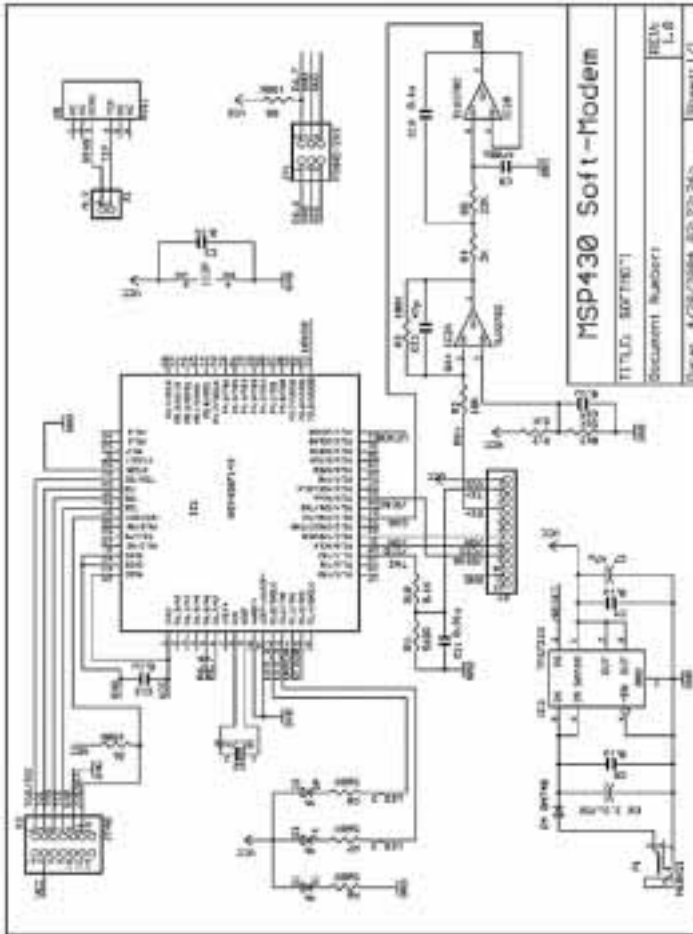
Все тестируемые стандартные модемы смогли соединиться с программным модемом на базе MSP430 без каких-либо проблем. Было показано, что функциональные возможности модема могут быть реализованы при помощи самого микроконтроллера без использования дополнительного аппаратного модема. Это позволяет поднять рентабельность многих прикладных решений, которым необходимо вести обмен данными по телефонной линии с другими устройствами.

Ссылки:

1. MSP430x13x, MSP430x14x Mixed Signal Microcontroller data sheet (SLAS272)

2. MSP430x1xx Family user's guide (SLAU049)
3. CPC5620/CPC5621 LITELINK III Phone Line Interface IC (DAA) data sheet, Clare Inc., 2002
4. CPC5621-EVAL-RDL/CPC5621-EVAL-CDL LITELINK III Evaluation-Board user's guide, Clare Inc., 2002
5. MSP430 Embedded Soft-Modem Demo, ATC Presentation by Dannenberg, A., November 2003

Приложение А. Схема электрическая принципиальная программного модема



Приложение В. Перечень элементов программного модема

Позиция	Наименование	Корпус
R1, R9	100 кОм	1206
R2	10 кОм	1206
R3	180 кОм	1206
R4	2 кОм	1206
R5	22 кОм	1206
R6, R7, R8, R11	560 Ом	1206
R10	9.1 кОм	1206
R12, R13	5.1 кОм	1206
C1, C2, C3, C4, C5, C8, C10	0.1 мкФ	1206
C6	2.2 мкФ x 50 В	D 4 мм
C7	47 мкФ x 16 В	D 6.3 мм
C9	4700 пФ	1206
C11	0.01 мкФ	1206
C13	47 пФ	1206
D1	Голубой светодиод	1206
D2	Желтый светодиод	1206
D3	Красный светодиод	1206
D4	BAT 48 SMD	1206
Q2	32 768 Гц	
IC1	MSP430F149	TQFP-64
IC2	TLV2782ID, OY Texas Instruments	S0-8
IC3	TPS7233QD, стабилизатор напряжения Texas Instruments	S0-8
P1	HEBW21/ разъем питания	
X1	Разъем 1x2	
X2	Разъем 1x12	
X3	Штырьковый разъем 2x7	
X4	Разъем RJ11	
JP1	Штырьковый разъем с перемычками 2x3	

IV. Примеры конкретных применений

Глава 12. *Генерация и распознавание DTMF-сигналов*

Генерация и распознавание DTMF-сигналов

Часть I.

Robert Siwy [slaae16.pdf (Англ.)]

1. Введение

Первая часть данного примера применения описывает генерацию DTMF сигналов с использованием микроконтроллера MSP430. Дается объяснение наиболее важных спецификаций, используемых в этом случае, приводятся теоретические и математические обоснования генерации синусоидальных сигналов при помощи прямоугольных с использованием соответствующих аналоговых фильтров. Также пример включает протестированные демонстрационные программы генерации прямоугольных сигналов на базе различных конфигураций таймеров микроконтроллеров MSP430. В заключительной части приводится принципиальная схема, позволяющая генерировать сигналы DTMF из прямоугольных сигналов.

2. Спецификация сигналов DTMF

Аббревиатура DTMF означает “Dual Tone Multi Frequency” (двухтональная мультисигнальная посылка) и представляет собой метод представления цифр различными частотами с целью передачи их по аналоговым линиям связи, например, по телефонной линии. При разработке стандарта было учтено условие – все частоты должны находиться в «голосовом» диапазоне, что позволило снизить требования к каналу передачи. В телефонных сетях сигналы DTMF используются для набора номера и передачи другой информации. Несмотря на то, что до сих пор широко используется импульсный способ набора номера, который является стандартом, например, в Германии, время набора при этом значительно увеличивается, приводя к непроизводительной загрузке линий связи. Кроме этого, многие дополнительные услуги связи возможны только с использованием тонального набора. При кодировании методом DTMF, цифры 0-9 и буквы A-D, */E and #/F представлены комбинаций двух частот:

Частота	1209 Гц	1336 Гц	1477Гц	1633Гц
697Гц	1	2	3	A
770Гц	4	5	6	B
852Гц	7	8	9	C
941Гц	*/E	0	#/F	D

В этой системе столбец представляет частоту из «верхней» частотной группы (Hi-Group: 1209-1633 Hz), а строка – частоту из «нижней» частотной группы (Lo-Group: 697-941 Hz). Тональные частоты выбраны таким образом, чтобы исключить влияние гармоник. Частоты не кратны друг другу и ни одна из частот DTMF не может быть получена суммированием или вычитанием других частот. Для формирования набора номера в сети Deutsche Telekom требуется выполнение нижеследующих спецификаций (взято из Zulassungsvorschrift des Bundesamtes für Post und Telekommunikation, BAPT 223 ZV 5 [1] (Официальная спецификация федерального министерства почты и связи):

- Отклонение частоты от номинальной – не более 1.8% в процессе набора номера
- Огибающая сигнала набора номера должна соответствовать осциллограмме, приведенной на рис. 1:

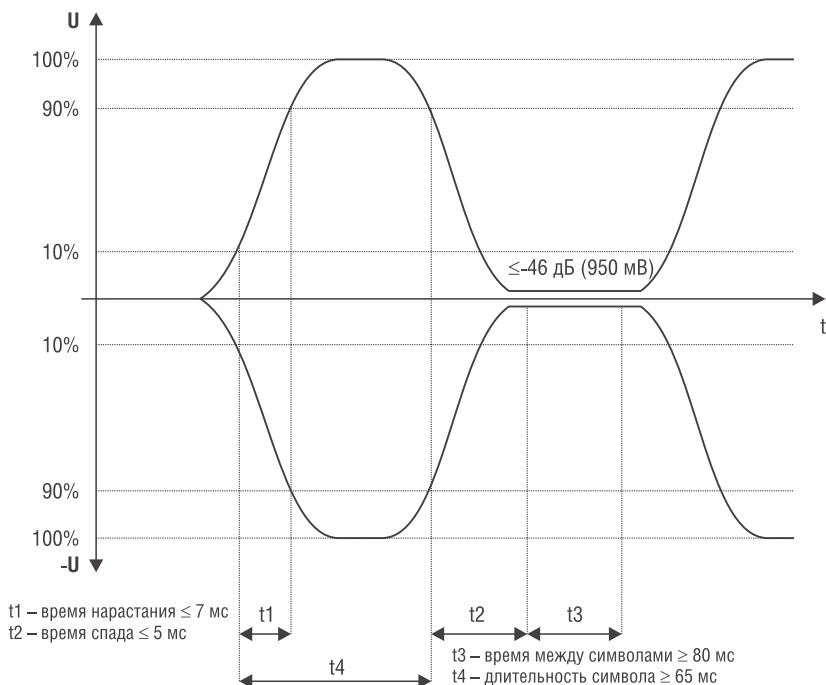


Рис. 1. Временные характеристики сигналов DTMF

- Отклонение частоты от номинальной – не более 1.8% в процессе набора номера

	Длительность символа	Пауза между символами	Мин. уровень в дБ (950мВ)		Макс. уровень в дБ (950мВ)	
			fu	fo	fu	fo
Автоматический набор или ручной набор с автоматическим ограничением длительности	$65 \text{ мс} \leq t \leq 100 \text{ мс}$	$80 \text{ мс} \leq t \leq 6500 \text{ мс}$	-16	-14	-10.5	-8.5
Ручной набор без ограничения длительности	$t \geq 65 \text{ мс}$	$t \geq 80 \text{ мс}$	-16	-14	-13	-11

- Номинальный уровень напряжения верхней из 2-х частот должен быть выше как минимум на 0.5 дБ, но не более чем на 3.5 дБ, чем номинальный уровень напряжения нижней из 2-х частот с целью компенсации потерь в длинных линиях
- В диапазоне частот 250..4600 Гц, суммарный уровень всех частот, кроме тех, которые формируют набор номера, должен быть как минимум на 23 дБ ниже суммарного уровня частот, формирующих набор номера и как минимум на 20 дБ ниже уровня каждой из таких частот.

3. Генерация сигналов DTMF

Как было описано выше, сигналы DTMF являются аналоговыми и состоят из двух независимых друг от друга синусоидальных сигналов. Таким образом, невозможно сгенерировать такие сигналы только цифровым способом. Цифровые сигналы должны быть преобразованы при помощи АЦП и/или аналоговых фильтров в требуемую синусоидальную форму.

3.1. Генерация при помощи прямоугольных сигналов

Если для формирования сигналов DTMF используются прямоугольные сигналы, требования к программному и аппаратному обеспечению минимальны. Любой непрерывный сигнал, имеющий период T может быть представлен рядом Фурье, состоящим из бесконечной суммы синусоид и косинусоид [2] следующим образом:

$$y(t) = \frac{a_0}{2} + \sum_{n=1}^{\infty} [a_n \times \cos(n\omega_0 t) + b_n \times \sin(n\omega_0 t)]$$

Где $a_0/2$ – постоянная составляющая сигнала. Элемент суммы с наименьшей угловой частотой (ω_0) называется основной (фундаментальной) гармоникой, остальные – обертонами или высшими гармониками.

Наиболее простым непрерывным сигналом, реализуемым при помощи микроконтроллера, является меандр, ряд Фурье для которого имеет вид:

$$y(t) = \frac{\hat{y}}{2} + \frac{2\hat{y}}{\pi} \left[\sin(\omega_0 t) + \frac{1}{3} \sin(3\omega_0 t) + \frac{1}{5} \sin(5\omega_0 t) + \frac{1}{7} \sin(7\omega_0 t) + \dots \right]$$

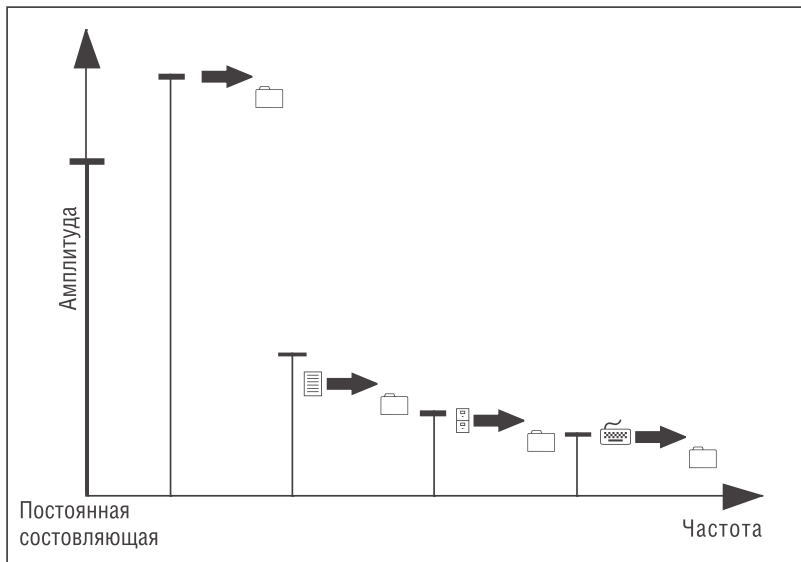


Рис. 2. Амплитудный спектр прямоугольного сигнала

При использовании аналогового фильтра постоянная составляющая и высшие гармоники эффективно подавляются, при этом на выходе имеем синусоидальный сигнал с частотой, равной частоте исходного меандра.

3.2. Программное обеспечение для генерации меандра

Программа генерации меандра должна удовлетворять следующим требованиям:

- Иметь возможность синтеза двух независимых друг от друга прямоугольных сигналов.
- В целях разделения сигналов требуются два выходных вывода, обеспечивающих генерацию сигналов из «верхнего» (Hi-Group) и «нижнего» (Lo-Group) частотных диапазонов соответственно.

- ПО должно иметь возможность установки требуемой длительности сигналов в диапазоне примерно 65 мс – 100 мс.

Микроконтроллеры серии MSP430 имеют различные встроенные таймеры, способные генерировать прямоугольные сигналы. В семействе '31x/'32x используются 8-битный таймер и таймер-порт (Timer Port) для генерации обоих прямоугольных сигналов. Данная программа проверялась при частоте MCLK равной 1.048 МГц. Таймер Timer_A в семействе '33x может самостоятельно формировать оба требуемых сигнала. Вторая программа использует этот таймер для генерации меандров и работает с любыми частотами MCLK. Далее будут подробно рассмотрены обе программы.

3.2.1. Генерация прямоугольных сигналов с использованием 8-битного таймера и таймера – порта Timer Port

На рис. 3 изображена блок-схема процедуры инициализации для генерации DTMF-сигналов. Чтобы синтезировать две частоты, используются счётчики таймера-порта Timer Port и 8-битного таймера. Каждый из них представляет собой программируемый счётный регистр, необходимый для точного синтеза требуемых частот. Если счётчики таймера-порта каскадированы в один 16-битный таймер и тактируются от системной частоты MCLK, то частоты верхнего диапазона могут быть сформированы с высокой точностью. При возникновении прерывания соответствующий выход переключается, и оба 8-битных счётных регистра перезагружаются. Загружаемые значения хранятся в двух переменных в ОЗУ с целью экономии внутренних регистров для других задач.

Частоты нижнего диапазона генерируются 8-битным таймером. Так как счётный регистр этого таймера имеет разрядность 8 бит, только каждое третье прерывание приводит к смене уровня на требуемом выходном выводе, позволяя генерировать этим же счётчиком частоты.

Два выхода таймера – порта используются для формирования двух меандров разной частоты.

Процедура инициализации выполняется только один раз. После её завершения, шестнадцатеричное значение передаваемого символа читается из глобальной переменной в ОЗУ. После того, как две частоты, формирующие верхний и нижний тона DTMF, сгенерированы из двух таблиц, требуется только инициализация и запуск обоих таймеров. Длительность посылки контролируется путём подсчёта полупериодов «нижней» частоты и считывается из дополнительной таблицы. По завершении этой процедуры производится возврат к функции поллинга. Соответствующие подпрограммы обработки

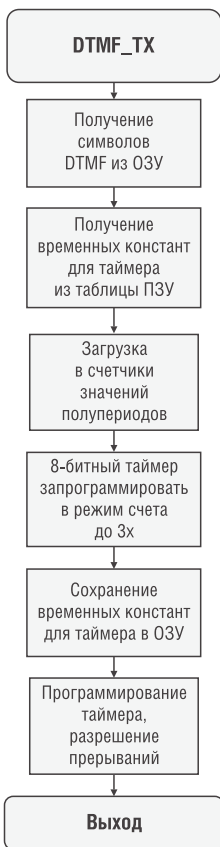


Рис. 3. Блок-схема процедуры инициализации для генерации DTMF-сигналов

прерываний осуществляют переключение выводов порта. Этот процесс продемонстрирован на рис. 4 и 5.

В задачи таймера-порта входит только фиксация лог. уровня на выходе порта и перезагрузка счётчика из ОЗУ, работа же 8-битного таймера имеет несколько более сложную структуру: каждый вход в прерывание подсчитывается счётным регистром. Выход может сменить состояние только по истечении трёх прерываний. Кроме этого, каждый полупериод также подсчитывается. Генерация прекращается по достижении определённого количества полупериодов.

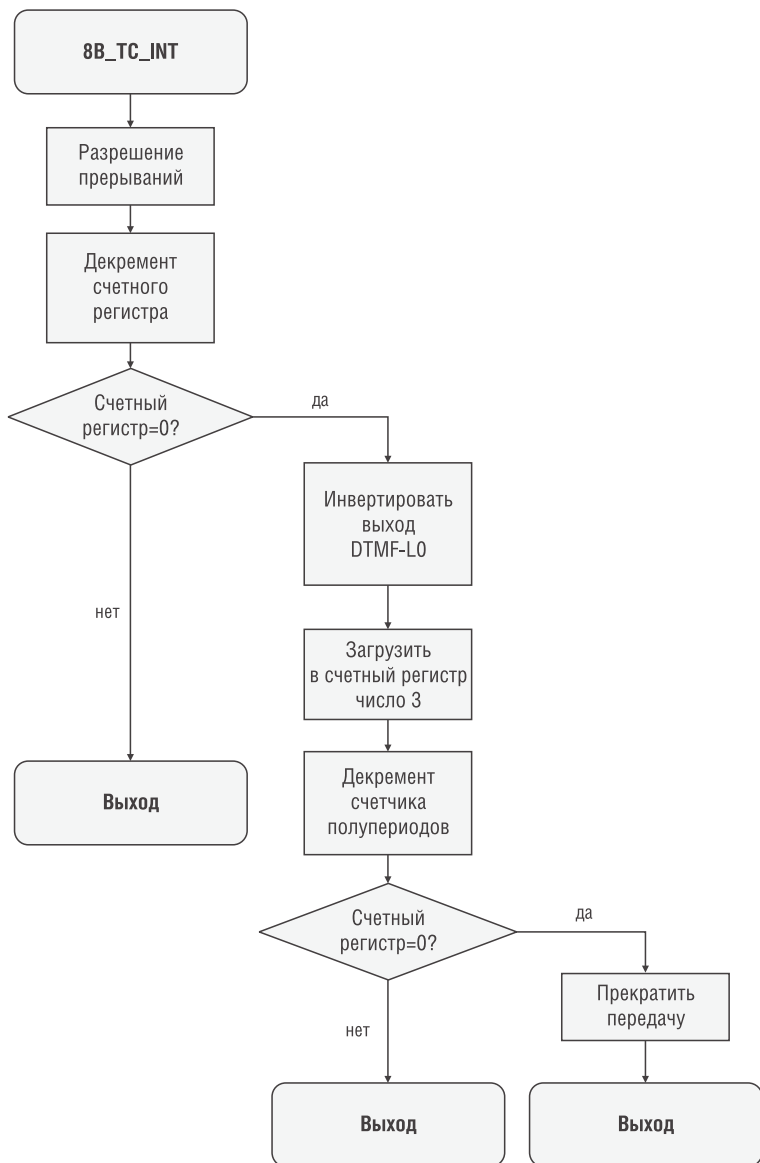


Рис. 4. Блок – схема прерываний от 8-битного таймера (Lo-Group)

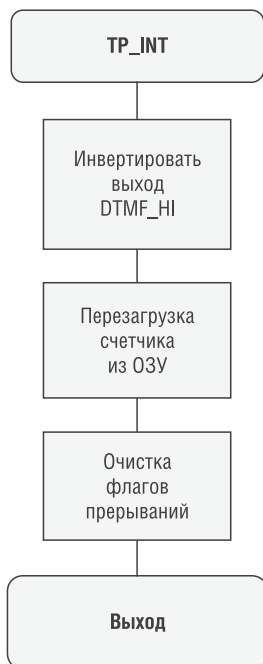


Рис. 5. Блок – схема прерываний таймера-порта (Hi-Group)

; Пользовательские определения

```

FLLMPY    .equ    32      ; Умножитель частоты FLL
                        ; на 1.048 МГц
TCLK      .equ    FLLMPY*32768 ; TCLK: FLLMPY × f
                        ; кварца
DL        .equ    85      ; Длительность сигнала DTMF
                        ; (65..100 мс)
LO_OUT    .equ    02h     ; Выход «нижней» частоты
HI_OUT    .equ    04h     ; Выход «верхней» частоты
RCOUNT    .equ    r14     ; Счётчик длины DTMF
RTEMP     .equ    r15     ; служебный регистр
                        .global DTMF_NR ; глобальная переменная в ОЗУ
                        ; для номера DTMF (0..F)
  
```

; Определения ОЗУ

```

    .even
  
```

```

        .bss      DTMF_TL ; Выравнивание по чётному
                        ; адресу
        .bss      DTMF_TH
        .bss      DTMF_NR ; глобальная переменная в ОЗУ
                        ; для номера DTMF (0..F)
        .even

; Определения для 8-битного таймера
TCCTL      .EQU      42H
TCPLD      .EQU      43H
TCDAT      .EQU      44H

; Определения для универсального таймера-порта
TPCTL      .equ       04bh ; Управление таймером-портом
TPCNT1     .equ       04ch ; Счётчик таймера-порта 1
TPCNT2     .equ       04dh ; Счётчик таймера-порта 2
TPD        .equ       04eh ; Данные таймера-порта
TPE        .equ       04fh ; Разрешение таймера-порта
        .text

; Таблицы частот DTMF: таблица содержит
; число тактов MCLK для одного полупериода.
; Таблица для «верхней» частоты
; Добавлена корректирующая поправка для учёта времени
; входа в прерывание
DTMF_HI     .word      0ffffh-(TCLK/(1336*2))+25 ; Верхняя
                        ; частота для 0
        .word      0ffffh-(TCLK/(1207*2))+28 ; Верхняя
                        ; частота для 1
        .word      0ffffh-(TCLK/(1336*2))+25 ; Верхняя
                        ; частота для 2
        .word      0ffffh-(TCLK/(1477*2))+24 ; Верхняя
                        ; частота для 3
        .word      0ffffh-(TCLK/(1207*2))+28 ; Верхняя
                        ; частота для 4
        .word      0ffffh-(TCLK/(1336*2))+25 ; Верхняя
                        ; частота для 5
        .word      0ffffh-(TCLK/(1477*2))+24 ; Верхняя
                        ; частота для 6
        .word      0ffffh-(TCLK/(1207*2))+28 ; Верхняя
                        ; частота для 7
        .word      0ffffh-(TCLK/(1336*2))+25 ; Верхняя
                        ; частота для 8

```

```
.word 0ffffh-(TCLK/(1477*2))+24 ; Верхняя
                                ; частота для 9
.word 0ffffh-(TCLK/(1633*2))+22 ; Верхняя
                                ; частота для A
.word 0ffffh-(TCLK/(1633*2))+22 ; Верхняя
                                ; частота для B
.word 0ffffh-(TCLK/(1633*2))+22 ; Верхняя
                                ; частота для C
.word 0ffffh-(TCLK/(1633*2))+22 ; Верхняя
                                ; частота для D
.word 0ffffh-(TCLK/(1207*2))+28 ; Верхняя
                                ; частота для *
.word 0ffffh-(TCLK/(1477*2))+24 ; Верхняя
                                ; частота для #
```

; Таблица для «нижней» частоты

```
DTMF_LO .byte 0ffh-(TCLK/(941*2*3)) ; Нижняя
                                ; частота для 0
.byte 0ffh-(TCLK/(697*2*3)) ; Нижняя
                                ; частота для 1
.byte 0ffh-(TCLK/(697*2*3)) ; Нижняя
                                ; частота для 2
.byte 0ffh-(TCLK/(697*2*3)) ; Нижняя
                                ; частота для 3
.byte 0ffh-(TCLK/(770*2*3)) ; Нижняя
                                ; частота для 4
.byte 0ffh-(TCLK/(770*2*3)) ; Нижняя
                                ; частота для 5
.byte 0ffh-(TCLK/(770*2*3)) ; Нижняя
                                ; частота для 6
.byte 0ffh-(TCLK/(853*2*3)) ; Нижняя
                                ; частота для 7
.byte 0ffh-(TCLK/(853*2*3)) ; Нижняя
                                ; частота для 8
.byte 0ffh-(TCLK/(853*2*3)) ; Нижняя
                                ; частота для 9
.byte 0ffh-(TCLK/(697*2*3)) ; Нижняя
                                ; частота для A
.byte 0ffh-(TCLK/(770*2*3)) ; Нижняя
                                ; частота для B
.byte 0ffh-(TCLK/(853*2*3)) ; Нижняя
                                ; частота для C
```

```

        .byte    0ffh-(TCLK/(941*2*3))      ; Нижняя
                                                ; частота для D
        .byte    0ffh-(TCLK/(941*2*3))      ; Нижняя
                                                ; частота для *
        .byte    0ffh-(TCLK/(941*2*3))      ; Нижняя
                                                ; частота для #

; Таблица длительностей сигнала
DTMF_L    .byte    2*941*DL/1000 ; Полупериоды для 0
        .byte    2*697*DL/1000 ; Полупериоды для 1
        .byte    2*697*DL/1000 ; Полупериоды для 2
        .byte    2*697*DL/1000 ; Полупериоды для 3
        .byte    2*770*DL/1000 ; Полупериоды для 4
        .byte    2*770*DL/1000 ; Полупериоды для 5
        .byte    2*770*DL/1000 ; Полупериоды для 6
        .byte    2*852*DL/1000 ; Полупериоды для 7
        .byte    2*852*DL/1000 ; Полупериоды для 8
        .byte    2*852*DL/1000 ; Полупериоды для 9
        .byte    2*697*DL/1000 ; Полупериоды для A
        .byte    2*770*DL/1000 ; Полупериоды для B
        .byte    2*852*DL/1000 ; Полупериоды для C
        .byte    2*941*DL/1000 ; Полупериоды для D
        .byte    2*941*DL/1000 ; Полупериоды для *
        .byte    2*941*DL/1000 ; Полупериоды для #

; *****
; DTMF-TX Подпрограмма DTMF
; *****
DTMF_TX
        mov.b    DTMF_NR,RTEMP ; Сохранить
                                ; номер во временном
                                ; регистре
        mov.b    DTMF_L(RTEMP),RCOUNT ; Сохранить
                                ; счётчик
                                ; длительности

; подготовка 8-битного таймера для частоты DTMF-Lo
        mov.b    #0a8h,&TCCTL ; Тактирование от MCLK
        mov.b    DTMF_LO(RTEMP),&TCPLD ; Подготовка
                                ; регистра предвари-
                                ; тельной загрузки
        mov.b    #000,&TCDAT ; Загрузка счётчика
                                ; из регистра

```



```

; предварительной
; загрузки
bis.b    #008h, &IE1    ; Разрешить прерывания
; от 8-битного таймера
; подготовка таймера-порта для частоты DTMF-Hi
rla      r15             ; * 2 для 16-битной
; таблицы
mov      DTMF_HI (RTEMP), &DTMF_TL
; сохранить слово для
; верхней частоты
mov      #003, RTEMP     ; счётчик для 8-битно-
; го таймера
bis.b    #008h, IE2      ; Разрешить прерывания
; от таймера-порта
mov.b    &DTMF_TH, &TPCNT2 ; Загрузка старшего
; байта в TC2
mov.b    &DTMF_TL, &TPCNT1 ; Загрузка младшего
; байта в TC1
bis.b    #080h, &TPD      ; Разрешить 16-битный
; таймер
bis.b    #HI_OUT+LO_OUT, &TPE ; Разрешить
; выходы DTMF-Hi/Lo
mov.b    #090h, &TPCTL    ; Разрешить таймер
ret

; *****
; Прерывание таймера-порта
; *****
TP_INT
xor.b    #HI_OUT, &TPD    ; Инверсия выхода
; DTMF-Hi
mov.b    &DTMF_TH, &TPCNT2 ; Загрузка старшего
; байта в TC2
mov.b    &DTMF_TL, &TPCNT1 ; Загрузка младшего
; байта в TC1
bic.b    #007h, &TPCTL    ; Очистка флагов
reti

; *****
; Прерывание P0.1/8-битный таймер
; *****
TIM_8B
eint      ; Разрешить прерывания

```

```

        dec    RTEMP
        jz     TOGGLE          ; переход по 3-му
                                ; прерыванию

        reti

TOGGLE
        xor.b  #LO_OUT,&TPD    ; Инверсия выхода
                                ; DTMF-Lo
        mov    #003,RTEMP      ; счётчик для 8-битно-
                                ; го таймера
        dec    RCOUNT         ; декремент счётчика
                                ; длительности
        jz     DTMF_END        ; переход, если дли-
                                ; тельность закончилась

        reti

DTMF_END
        bic.b  #037h,&TPCTL    ; Остановить счётчик
                                ; таймера-порта
        bic.b  #008h,&TCCTL    ; Остановить счётчик
                                ; 8-битного таймера
        bic.b  #008h,&IE2      ; Запретить прерывания
                                ; 8-бит. таймера
        bic.b  #003h,&TPE      ; Запретить выходы
        reti

; Адреса векторов прерываний
        .sect  <TP_VECT>, 0ffe8h
        .word  TP_INT          ; Timer-Port
        .sect  <TIM_VECT>, 0fff8h
        .word  TIM_8B          ; 8-битный таймер
                                ; (P0.0)

```

3.2.2. Генерация прямоугольных сигналов с использованием таймера Timer_A

Данная подпрограмма генерации сигналов DTMF использует только таймер Timer_A для формирования меандров обоих требуемых частот. В процессе ассемблирования вычисляются соответствующие значения для таймера с целью использования программы независимо от значения частоты MCLK. Длительность выходного сигнала задаётся константой DL в миллисекундах.

```

; Аппаратные определения
FLLMPY    .equ    32           ; Умножитель частоты
                                ; FLL на 1.048 МГц
TCLK      .equ    FLLMPY*32768 ; TCLK: FLLMPY × f
                                ; кварца

```

```

DL      .equ      82          ; длительность сигнала
                                ; DTMF в мс 65..100 мс)
STACK   .equ      600h       ; Адрес инициализации
                                ; стека

; Определения ОЗУ
STDTMF  .equ      202h       ; Статус частот Hi
                                ; и Lo
TIM32B   .equ      204        ; Расширение регистра
                                ; таймера
LENGTH  .equ      206h       ; Счётчик длительности
                                ; DTMF
;
        .text      0F000h     ; Стартовый адрес
                                ; программы
;
; Инициализация таймера Timer_A: MCLK,
; Режим Continuous, прерывания разрешены
; Подготовка выходных модулей таймера Timer_A
; MCLK = 1.048 МГц; (автоматически)
;
INIT     MOV        #STACK, SP ; Инициализация указате-
                                ; ля стека
        CALL       #INITSR     ; Инициализация умно-
                                ; жителя частоты FLL
                                ; и ОЗУ
        MOV        #ISMCLK+TAIE+CLR, &TACTL
                                ; Инициализация
                                ; таймера
        MOV.B      #TA2+TA1, &P3SEL ; Выходы TA2 и TA1
                                ; на портах P3.5/4
        CLR        TIM32B      ; Очистить расширенный
                                ; регистр таймера
        BIS        #MCONT, &TACTL ; Старт таймера Timer_A
        EINT       ; Общее разрешение
                                ; прерываний
MAINLOOP ...                  ; Основной цикл
;
; Нажатие клавиши: в SDTMF содержится смещение
; по таблице для 2-х частот (0..6, 0..6) в старшем
; и младшем байте
;
        MOV        &TAR, R5    ; Для немедленного
                                ; старта:

```

```

ADD      FDTMFLO,R5      ; Смещение для меньше-
                        ; го времени
MOV      R5,&CCR1        ; Первая смена состоя-
                        ; ния через 0.71мс
MOV      R5,&CCR2        ; 1/(2х697) = 0.71мс
MOV      #OMT+CCIE,&CCTL1 ;Инверсия выхода,
                        ; прерывание разр.
MOV      #OMT+CCIE,&CCTL2 ;Инверсия выхода,
                        ; прерывание разр.
MOV.B    STDTMF,R5       ; Счётчик на 82 мс
RRA      R5              ; число смен состояния
                        ; нижней частоты
MOV.B    DTMFL(R5),LENGTH ; для длительности
                        ; сигнала
...                ; продолжение программы
;
; Обработчик прерывания от CCR0 (здесь не используется)
;
TIMMOD0   ...
          RETI
;
; Обработчик прерываний от регистров захвата-сравнения
; 1..4
;
TIM_HND   ADD      &TAIV,PC      ; Обработка запроса
                        ; с наивысшим приори-
                        ; тетом
          RETI                  ; нет запроса прерыва-
                        ; ния: RETI
          JMP      HCCR1        ; запрос от CCR1
                        ; (нижняя частота DTMF)
          JMP      HCCR2        ; запрос от CCR2 (вер-
                        ; хняя частота DTMF)
          JMP      HCCR3        ; запрос от CCR3
          JMP      HCCR4        ; запрос от CCR4
;
TIMOVH   INC      TIM32B        ; Расширение таймера
                        ; Timer_A до 32 бит
          RETI
;
; Нижняя частота DTMF: TA1 инвертирует выход модуля
; Output Unit 1

```

```

; Каждая смена состояния подсчитывается для контроля
; длительности сигнала
;
HCCR1      PUSH      R5                ; Сохранение использу-
                                           ; емых регистров
          MOV.B      STDTMF,R5         ; Статус нижней частоты DTMF
          ADD        FDTMFLO(R5),&CCR1 ; Добавить длительность полупериода
          DEC.B      LENGTH            ; Длительность сигнала
                                           ; DL завершена?
          JNZ        TARET            ; Нет
;
; Да, прекратить выдачу сигнала DTMF: запретить прерывания
;
          BIC        #OMRS+OUT+CCIE,&CCTL1 ; Сброс TA1
          BIC        #OMRS+OUT+CCIE,&CCTL2 ; Сброс TA2
TARET      POP       R5                ; Восстановить R5
          RETI        ; Возврат из прерывания
;
; Верхняя частота DTMF: TA2 инвертирует выход модуля
; Output Unit 2
;
HCCR2      PUSH      R5                ; Сохранение использу-
                                           ; емых регистров
          MOV.B      STDTMF+1,R5       ; Статус верхней частоты DTMF
          ADD        FDTMFHI(R5),&CCR2 ; Добавить длительность полупериода
          POP       R5                ; Восстановить R5
          RETI        ; Возврат из прерывания
;
HCCR3      ...                        ; Задача, контролируемая регистром CCR3
          RETI
HCCR4      ...                        ; Задача, контролируемая регистром CCR4
          RETI
;

```

```

; Таблица частот DTMF: в таблице содержится число
; тактов MCLK на полупериод. Значения скорректированы
; для действующей частоты MCLK в процессе ассемблиро-
; вания и округлены с минимально возможной ошибкой
; частоты
;
FDTMFLO .word ((TCLK/697)+1)/2 ; Нижняя частота
; DTMF 697Hz
        .word ((TCLK/770)+1)/2 ; 770Hz
        .word ((TCLK/852)+1)/2 ; 852Hz
        .word ((TCLK/941)+1)/2 ; 941Hz
FDTMFHI .word ((TCLK/1209)+1)/2 ; Верхняя частота
; DTMF 1209Hz
        .word ((TCLK/1336)+1)/2 ; 1336Hz
        .word ((TCLK/1477)+1)/2 ; 1477Hz
        .word ((TCLK/1633)+1)/2 ; 1633Hz

; Таблица содержит число полупериодов для длительности
; сигнала DL (мс).
; Для подсчёта используется нижняя частота DTMF
;
DTMFL .byte 2*697*DL/1000 ; Число полупериодов
        .byte 2*770*DL/1000 ; для DL в мс
        .byte 2*852*DL/1000 ;
        .byte 2*941*DL/1000 ;
;
        .sect «TIMVEC», 0FFF0h ; Вектора прерываний таймера
; Timer_A
        .word TIM_HND ; Вектор модулей
; 1..4 таймера
        .word TIMMOD0 ; Вектор модуля
; 0 таймера
        .sect «INITVEC», 0FFFEh ; Вектор сброса
; Reset
        .word INIT

```

Нижне приведено несколько более быстрое решение. Однако, при этом оно требует большего объёма ОЗУ т.к. данные, получаемые из таблиц не пересчитываются каждый раз, а хранятся в двух словах в ОЗУ DTMFLO и DTMFHI. Чтение производится из подпрограмм обработки прерываний таймера Timer_A. Используемые таблицы идентичны приведенным в предыдущем примере.

```

FLLMPY    .equ    32      ; Умножитель частоты FLL
                        ; на 1.048МГц
TCLK      .equ    FLLMPY*32768 ; TCLK: FLLMPY × f
                        ; кварца
DL        .equ    82      ; Длительность сигнала DTMF
                        ; в мс (65..100 мс)
STDTMF    .equ    202h    ; Статус частот Hi и Lo
TIM32B    .equ    204      ; Расширение регистра таймера
LENGTH    .equ    206h    ; Счётчик длительности DTMF
DTMFLO    .equ    208h    ; Полупериод нижней частоты
DTMFHI    .equ    20Ah    ; Полупериод верхней частоты
STACK     .equ    600h    ; Адрес инициализации стека
        .text    0F000h ; Стартовый адрес программы

; Инициализация таймера Timer_A: MCLK, Режим
; Continuous, прерывания разрешены
; Подготовка выходных модулей таймера Timer_A
; MCLK = 1.048 МГц ; (автоматически)
;
INIT      MOV      #STACK, SP      ; Инициализация
                        ; указателя стека
        CALL      #INITSR         ; Инициализация
                        ; умножителя частоты
                        ; FLL и ОЗУ
        MOV      #ISMCLK+TAIE+CLR, &TACTL ; Старт
                        ; таймера
        MOV.B    #TA2+TA1, &P3SEL ; Выходы TA2 и TA1
                        ; на портах P3.5/4
        CLR      TIM32B           ; Очистка расширенного
                        ; регистра таймера
        BIS      #MCONT, &TACTL ; Запуск таймера Timer_A
        EINT                        ; Общее разрешение
                        ; прерываний
MAINLOOP  ...                    ; Основной цикл

; Нажатие клавиши: в SDTMF содержится смещение
; по таблице для 2-х частот (0..6, 0..6) в старшем
; и младшем байте
;
        MOV      &TAR, R5         ; Для немедленного
                        ; старта
        ADD      FDTMFLO, R5      ; Смещение для меньше-
                        ; го времени

```

```

MOV      R5,&CCR1      ; Первая смена состоя-
                        ; ния через 0.71мс
MOV      R5,&CCR2      ; 1/(2х697) = 0.71мс
;
; Fetch the two cycle counts for the DTMF frequencies
;
MOV.B    STDTMF+1,R5   ; Верхняя частота DTMF
MOV      FDTMFHI(R5),DTMFHI ; Длительность
                        ; полупериода
MOV.B    STDTMF,R5     ; Нижняя частота DTMF
MOV      DTMFLO(R5),DTMFLO ; Длительность
                        ; полупериода
;
                        ; Счётчик длительности
RRA      R5            ; Prepare byte index
MOV.B    DTMFL(R5),LENGTH ; число смен состо-
                        ; яния нижней частоты
MOV      #OMT+CCIE,&CCTL1 ;Инверсия выхода,
                        ; прерывание разр.
MOV      #OMT+CCIE,&CCTL2 ;Инверсия выхода,
                        ; прерывание разр.
...      ; Возврат в основной
                        ; цикл
;
; Обработчик прерывания от CCR0 (здесь не используется)
;
TIMMOD0  ...
          RETI
;
; Обработчик прерываний от регистров захвата-сравнения
; 1...4
;
TIM_HND  ADD      &TAIV,PC      ; Обработка запроса
                        ; с наивысшим приори-
                        ; тетом
          RETI      ; нет запроса прерыва-
                        ; ния: RETI
          JMP      HCCR1      ; запрос от CCR1 (ниж-
                        ; няя частота DTMF)
          JMP      HCCR2      ; запрос от CCR2 (вер-
                        ; хняя частота DTMF)
          JMP      HCCR3      ; запрос от CCR3

```



```

        JMP      HCCR4          ; запрос от CCR4
;
TIMOVH   INC      TIM32B        ; Расширение таймера
        ; Timer_A до 32 бит
        RETI
;
; Нижняя частота DTMF: TA1 инвертирует выход модуля
; Output Unit 1
;
HCCR1    ADD      DTMFLO,&CCR1  ; Добавить длительность полупериода
        DEC.B     LENGTH        ; Длительность сигнала
        ; DL завершена?
        JNZ      TARET          ; Нет
;
; Да, прекратить выдачу сигнала DTMF: запретить прерывания
;
        BIC      #OMRS+OUT+CCIE,&CCTL1 ; Сброс TA1
        BIC      #OMRS+OUT+CCIE,&CCTL2 ; Сброс TA2
TARET    RETI              ; Возврат из прерывания
        ;
; Верхняя частота DTMF: TA2 инвертирует выход модуля
; Output Unit 2
;
HCCR2    ADD      DTMFHI,&CCR2  ; Добавить длительность полупериода
        RETI              ; Возврат из прерывания
;
HCCR3    ...           ; Задача, контролируемая регистром CCR3
        RETI
HCCR4    ...           ; Задача, контролируемая регистром CCR4
        RETI
;
; Таблицы и вектора прерываний идентичны приведенным
; в предыдущем примере

```

3.3. Аппаратная часть для генерации сигналов DTMF

Как было указано выше, в диапазоне частот 200 Гц .. 4600 Гц уровень сигнала частоты передачи должен быть как минимум на 20 дБ выше уровня посторонних сигналов (шумов). Кроме этого, исходя из спецификации, сигналы из «верхней» и «нижней» групп должны иметь разные уровни, поэтому для каждого сигнала требуется собственный фильтр. Амплитуды частот синусоидальной формы можно получить из ряда Фурье.

Для выбора частот среза при разработке аналогового фильтра необходимо придерживаться следующих требований, основанных на [1]:

- Т.к. необходимо обеспечить возможность любых комбинаций частот «нижней» группы с частотами «верхней» группы, разность уровней между самой низкой и самой высокой частотой в группе не должна превышать 3 дБ.
- Для самой низкой частоты в группе (f_1), подавление гармоник ($3f_1$) должно быть не хуже 20 дБ. Выполнение этого условия наиболее критично для нижней частоты в группе, т.к. она находится максимально далеко от частоты среза фильтра.

Формула [3] описывает квадрат абсолютного значения на выходе фильтра высоких частот Баттерворта порядка n :

$$|A(f_1)|^2 = \frac{1}{1 + \left(\frac{f}{f_0}\right)^{2n}}$$

Данная формула представляет собой зависимость коэффициента усиления фильтра высоких частот Баттерворта от частоты. Параметры f_0 и n определяют частоту среза и порядок фильтра соответственно.

Прежде всего, необходимо вычислить требуемый порядок фильтра с учётом соответствия приведенным выше требованиям.

Для выполнения первого условия отношение квадратов абсолютных значений самой нижней и самой верхней частоты в группе должно быть не более 3 дБ или $\sqrt{2}$:

$$\frac{|A(f_1)|^2}{|A(f_4)|^2} < \sqrt{2}$$

Второе условие выполнится автоматически, если отношение квадратов абсолютных значений частот f_1 и $3f_1$ будет более 10/3, в данном случае третья гармоника в прямоугольном сигнале меньше на 1/3 (см. ряд Фурье и рис. 2):

$$\frac{|A(f_1)|^2}{|A(3f_3)|^2} < \frac{10}{3}$$

В результате вычислений для частот обеих групп имеем требуемый порядок фильтра $n=1.15$. Таким образом, требованиям будет удовлетворять фильтр 2-го порядка, который может быть построен на операционном усилителе. В случае использования фильтра 3-го порядка понадобится только два дополнительных элемента. Однако, это позволит снизить требования к разбросу параметров компонентов. Оба приведенных выше требования будут соблюдены, если частота среза находится в следующих пределах:

Нижняя группа	$f_g > 880 \text{ Гц}$	$f_g < 1418 \text{ Гц}$
Верхняя группа	$f_g > 1527 \text{ Гц}$	$f_g < 2460 \text{ Гц}$

Если частота среза минимальна, будет наблюдаться максимальное подавление гармоник. Однако, в этом случае разность уровней самой низкой и самой высокой частоты в группе будет равна 3 дБ. При максимально возможной частоте среза разность уровней минимальна, но гармонические составляющие подавляются только на 20 дБ.

При расчёте фильтра было обращено повышенное внимание на подавление гармоник, разность уровней внутри группы была зафиксирована на уровне 2 дБ. В результате этого, частоты среза имеют значения 977 Гц и 1695 Гц. Результирующее подавление гармоник значительно превосходит предъявляемые требования. Разность уровней частот в группе находится в пределах требований даже в случае девиации частоты среза, связанной с разбросом параметров применяемых компонентов. При вычислении значений элементов фильтра резисторы выбирались исходя из соображения их максимальной близости к стандартным значениям ряда E12.

На выходах фильтров в результате получаем два синусоидальных сигнала с значительно подавленными гармониками. Для объединения этих сигналов введен дополнительный сумматор.

Таким образом, используя только 3 ОУ и несколько пассивных элементов, мы имеем возможность генерировать сигналы DTMF при помощи микроконтроллера без использования значительных вычислительных ресурсов.

При помощи программ-симуляторов была проведена проверка приближенных значений. Отклик фильтров весьма точно совпадает с расчётной частотной характеристикой.

На рис. 6 приведен амплитудный спектр прямоугольного сигнала частотой 697 Гц, пропущенного через фильтр 3-го порядка. Как видно из рисунка, третья и пятая гармоники (2091 Гц и 3485 Гц) существенно ослаблены (-25.6 dB).

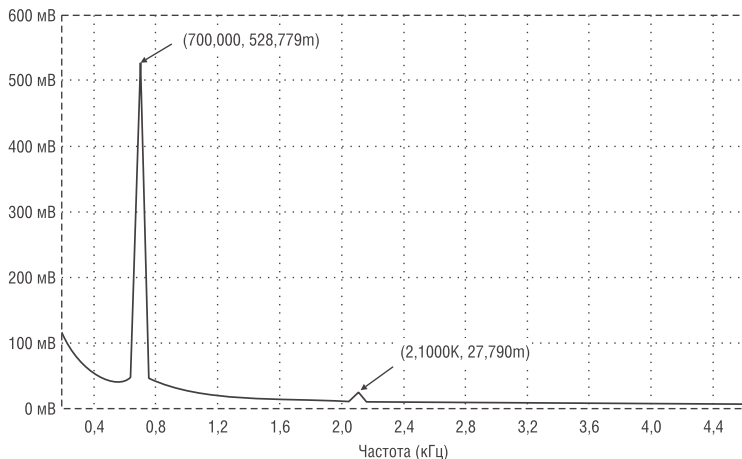


Рис. 6. Амплитудный спектр прямоугольного сигнала частотой 697 Гц на выходе фильтра 3-го порядка

На рис. 7 показан спектр прямоугольного сигнала частотой 941 Гц. В интересующей нас частотной области до 4600 Гц присутствует только одна гармоника. После прохождения фильтра, эта гармоника частотой 2823 Гц значительно ослаблена (-27.9 дБ). Разность уровней между самой низкой и самой высокой частотой в группе не превышает 1.9 дБ.

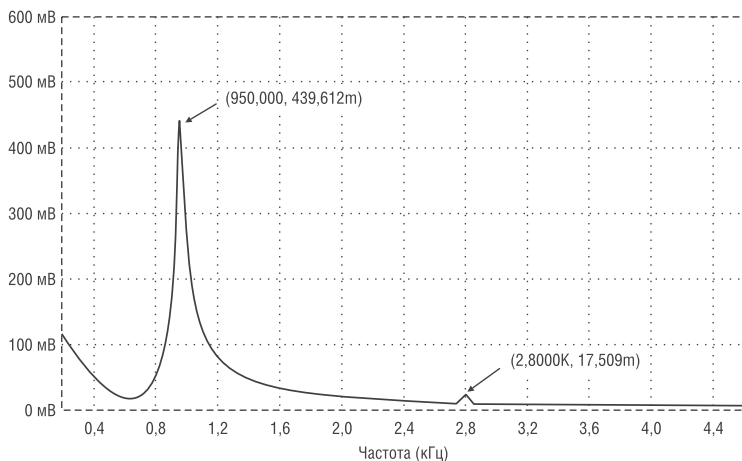


Рис. 7. Амплитудный спектр прямоугольного сигнала частотой 941 Гц на выходе фильтра 3-го порядка

С целью использования недорогих компонентов с высоким разбросом параметров было проведено дополнительное моделирование. Результаты моделирования показали, что допустимый разброс величины резисторов и конденсаторов составляет 10%.

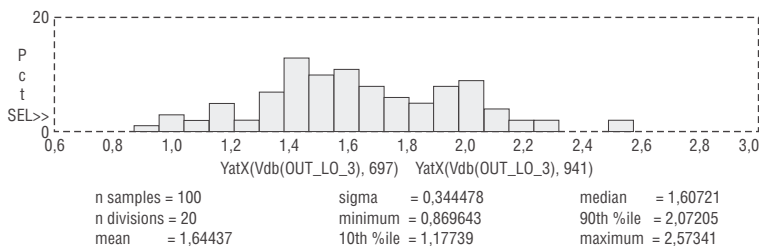


Рис. 8. Гистограмма – распределение уровней сигнала в группе

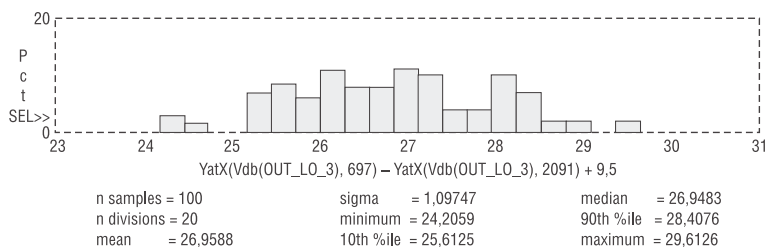


Рис. 9. Гистограмма – подавление гармоник

На рис. 8 и 9 показаны гистограммы полученные при помощи анализа по методу Монте-Карло. В данном случае, значения компонентов менялись случайным образом в пределах разброса 10%. После 100 итераций результаты для всех смоделированных фильтров помещены на гистограммы. В гистограмме на рис. 8 показана разность уровней между частотами внутри группы. Максимально допустимая разность – 3 дБ между минимальной и максимальной частотой ни в одном случае не была достигнута. Среднее значение составляет 1.6 дБ, что несколько лучше расчётного значения 2 дБ.

На рис. 9 показано ослабление гармоник для «нижней» группы частот. Требуемая величина в 20 дБ достигается во всех случаях, среднее значение примерно 27 дБ. В наихудшем случае гармоника подавлена на 24.2 дБ.

Значения, рассчитанные для фильтра «нижней» группы частот выполняются также и для фильтра «верхней» группы частот.

Оба фильтра имеют идентичную схемотехнику. Различие только в частотах среза фильтров для верхней и нижней групп частот. R1 и C1 формируют ФВЧ

1-го порядка. Т.к. входное сопротивление схемы также зависит от R_1 , значение этого элемента не должно быть слишком маленьким; в противном случае выходы микроконтроллера будут перегружены, а прямоугольная форма сигнала искажена. В таком случае к сигналу будут добавлены дополнительные частоты, являющиеся результатом интермодуляционных искажений, что отрицательно скажется на соотношении сигнал-шум.

Функция передачи фильтров более высокого порядка не может быть получена с использованием только пассивных компонентов.

Таким образом, фильтр 2-го порядка должен содержать операционный усилитель. Усиление активного фильтра установлено на уровне 0.2 при помощи резисторов R_{1-1} и R_{1-2} . Как видно, сигнал при этом несколько ослабляется. Это необходимо для исключения перегрузки ОУ, т.к. пиковая амплитуда фундаментальной синусоидальной гармоники прямоугольного сигнала превышает амплитуду собственно прямоугольного сигнала (См. ряд Фурье и Рис.2). В дополнительном сумматоре производится регулировка требуемого выходного уровня. Из-за имеющей место постоянной составляющей прямоугольного сигнала рабочая точка ОУ выставляется на уровне $V_{cc}/2$ (См. также ряд Фурье и Рис. 2). При этом, постоянная составляющая не может быть устранена входным делителем R_{1-1}/R_{1-2} . Для развязки цепи ОС по постоянному напряжению служит конденсатор C_3 .

На каждом из выходов аналогового фильтра формируется сигнал верхней и нижней групп частот соответственно. В дополнительном сумматоре данные сигналы складываются. В этом элементе схемы можно установить соотношение между «нижней» и «верхней» частотой в суммарном сигнале и уровень выходного сигнала при помощи резисторов R_4 и R_5 . Таким образом, выходная амплитуда может быть легко отрегулирована в соответствии с различными требованиями, существующими в разных странах.

При расчёте номиналов компонентов значения конденсаторов были, как обычно, зафиксированы, и в соответствии с ними рассчитывались номиналы резисторов. В данной схеме были использованы конденсаторы и резисторы стандартного ряда E12 с разбросом 10%.

На рис. 10 показана принципиальная схема аналоговых фильтров и сумматора:

Конденсаторы C_{1-1} и C_{1-2} объединяют два сигнала вместе в рабочей точке с уровнем $V_{cc}/2$. Для этих элементов не следует выбирать слишком больших значений, т.к. они являются элементами ФНЧ для устранения низкочастотных субгармоник. Фильтрующий конденсатор C_5 устраняет шумы опорного напряжения. Дополнительный конденсатор C_6 , подключенный в параллель к

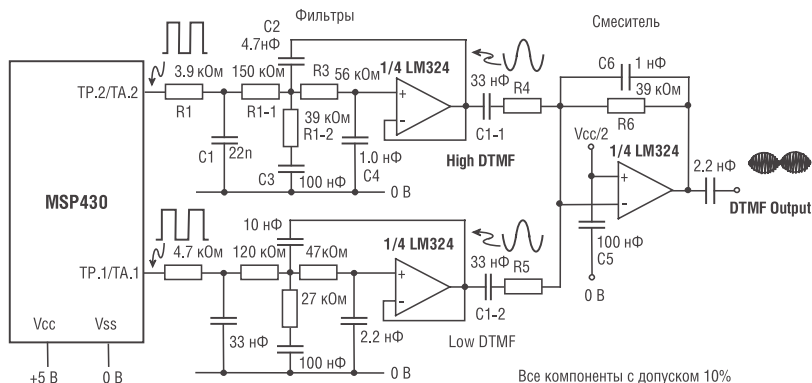


Рис. 10. Принципиальная схема аналоговых фильтров с дополнительным сумматором

резистору обратной связи R6 формирует ФВЧ первого порядка. Если выбрана наименьшая частота среза фильтра, дополнительная фильтрация высокочастотных интермодуляционных помех улучшает качественные показатели выходного сигнала, однако при этом будет наблюдаться некоторое ослабление самых высоких частот «верхней» группы. В некоторых случаях генерация самой высокой частоты DTMF 1633 Гц не требуется, т.к. она используется только для формирования служебных символов A-D, при этом можно улучшить соотношение сигнал – шум понижением частоты среза фильтра. Таким образом, повышение частоты среза ведёт к увеличению уровня высокочастотных помех, но при этом снижается негативное влияние на наиболее высокочастотные компоненты сигнала DTMF.

4. Результаты исследований сигналов DTMF

Приведенные ниже спектрограммы (Рис. 11 и 12) демонстрируют выходные сигналы DTMF – передатчика на различных частотах. На рис. 11 показан амплитудный спектр символа “1”. Необходимые для его передачи частоты – 697 и 1207 Гц находятся на уровне -10.5 дБ и -8.5 дБ соответственно. Гармоники 2091 и 3621 Гц подавлены почти на 30 дБ. Для передачи символа “D” генерируются две наиболее высокие частоты – 941 и 1633 Гц. Как видно из рис. 12, уровень нижней частоты составляет -12 дБ, уровень верхней частоты -11 дБ. Соответствующие гармоники ослаблены более, чем на 30 дБ. Таким образом, измеренные значения соответствуют результатам моделирования и требованиям спецификации [1].

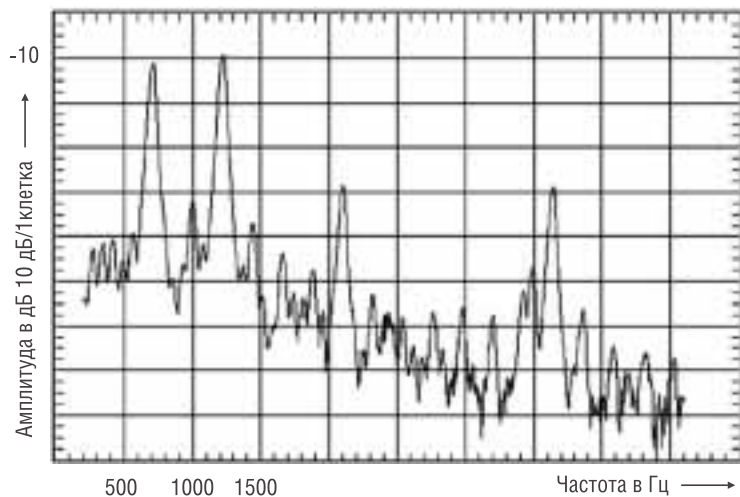


Рис. 11. Амплитудный спектр символа "1": 697 и 1207 Гц

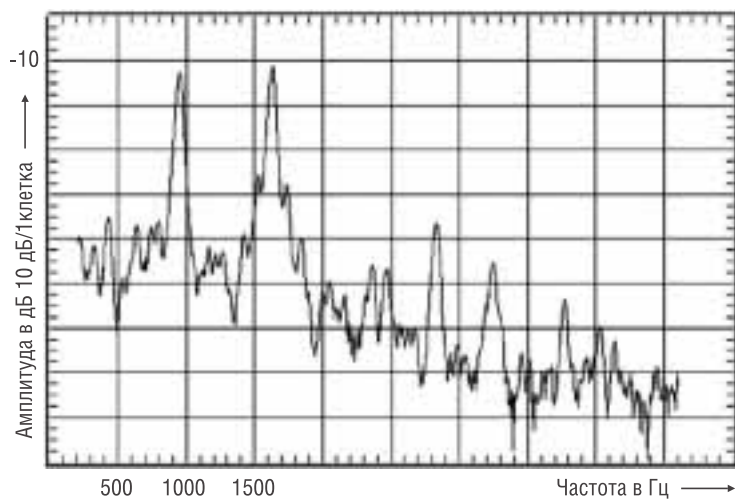


Рис. 12. Амплитудный спектр символа "D": 941 и 1633 Гц

Абсолютной точности частоты генерируемых прямоугольных сигналов при использовании двух различных таймеров достичь не удастся, результат будет зависеть от комбинации двух частот и типа используемых таймеров. При-

чиной этого является конфликт прерываний таймеров. Тем не менее, требуемая точность $\pm 1.8\%$ выполняется с большим запасом.

Если используются 8-битный таймер и таймер – порт Timer Port timer на системной частоте MCLK 1.048 МГц, то частоты «нижней» группы генерируются с точностью не хуже 0.3 %. Для частот «верхней» группы на практике получена девиация не выше 0.5 %.

Единственное исключение – символ DTMF “D”, для которого генерируются наивысшие частоты. В результате этого, в данной комбинации частота «верхней» группы 1633 Гц имеет отклонение в -0.97 %.

Без учёта этого исключения, даже самая высокая частота 1633 Гц генерируется с точностью лучше 0.5 %. Максимальные девиации для различных частот приведены в таблице:

«Нижняя» группа частот		«Верхняя» группа частот	
Частота, Гц	Макс. девиация	Частота, Гц	Макс. девиация
697	-0,28%	1207	+0,33%
770	-0,13%	1336	+0,45%
853	$\pm 0,12\%$	1477	$\pm 0,14\%$
941	-0,21%	1633	-0,97%

Если для генерации частот используется таймер Timer_A, ошибка будет зависеть от используемой частоты MCLK:

MCLK, МГц	1,048	2,096	3,144	3,800
Умножитель FLL	32	64	96	116
697 Гц	+0,027%	+0,027%	+0,027%	+0,027%
770 Гц	-0,015%	-0,016%	+0,033%	-0,016%
852 Гц	+0,059%	-0,023%	+0,005%	+0,031%
941 Гц	+0,029%	+0,029%	+0,029%	+0,035%
1209 Гц	-0,079%	+0,036%	+0,036%	-0,003%
1336 Гц	+0,109%	-0,018%	+0,025%	+0,025%
1447 Гц	-0,009%	-0,009%	-0,009%	-0,009%
1633 Гц	+0,018%	+0,018%	+0,018%	+0,018%

5. Заключение

Программное обеспечение для данного примера весьма простое и, занимая примерно 300 Байт, требует малого объёма ОЗУ и ПЗУ. Благодаря наличию встроенного модуля таймера требуемые частоты генерируются с высокой точностью без непроизводительной загрузки ЦПУ. В конфигурации, где для генерации используются 8-битный таймер и таймер-порт Timer/Port, подпро-

граммы обработки прерываний занимают примерно 12% ресурса ЦПУ. В том случае, когда частоты генерируются таймером Timer_A, загрузка ЦПУ на обработку подпрограмм прерываний снижается до 6%. В результате этого, во время передачи сигналов DTMF могут выполняться другие задачи, либо ЦПУ может быть переведен в режим пониженного потребления с целью снижения потребляемого тока.

Хорошая функциональность описанного модуля генерации сигналов DTMF при помощи прямоугольных сигналов демонстрируется схемотехникой аппаратного обеспечения. Т.к. в схеме возможно применение компонентов с большим разбросом параметров, цена такого решения весьма низкая. Все требования спецификации выполнены с большим запасом, таким образом, отдельный модуль генератора сигналов DTMF в приборах, использующих MSP430 в качестве управляющего контроллера, не требуется.

Если в конкретном случае требуется повысить соотношение сигнал – шум, используя дополнительный ОУ можно сконструировать фильтр для дальнейшего подавления интермодуляционных искажений. Такой дополнительный операционный усилитель уже присутствует в составе счетверённого ОУ в корпусе DIL14.

Ссылки:

1. Bundesamt fur Post und Telekommunikation (Federal Office for Post and Telecommunications): BAPT 223 ZV 5, Zulassungsvorschrift fur Endeinrichtungen zur Anschaltung an analoge Wahlanschlusse (ausgenommen Notruf- und Durchwahlanschlusse) des Telefonnetzes (approval specification for end equipment to be connected to analog dialling connections of the telephone network, except for emergency and in-dialling connections) / ISDN of the Deutschen Bundespost Telekom; Bundesministerium fur Post und Telekommunikation, Draft, Bonn April 1994

2. Papula: Mathematik fur Ingenieure 2 (Mathematics for Engineers); Vieweg Verlag, Braunschweig 1990

3. Tietze / Schenk: Halbleiterschaltungstechnik; (Semiconductor Circuit Design), 10th. Edition; Springer Verlag, Berlin 1993

4. Lutz Bierl / Texas Instruments: MSP430 Family, Metering Application Report, Texas Instruments, Issue 2.1, Jan 1997, SLAAE10B

5. Texas Instruments: MSP430 Family, Architecture User's Guide and Module Library, Texas Instruments, 1996, SLAUE10B

6. Texas Instruments: MSP430 Family, Software User's Guide, Texas Instruments, 1996

7. Texas Instruments: MSP430 Family, Assembly Language Tools User's Guide, Texas Instruments, 1996

8. Siwy, Robert: Systementwicklung einer Telekom-Applikation zum Senden und Empfangen von DTMF-Signalen mit dem Microcontroller MSP430 (System development of a telecom application to send and receive digital signals with the Microcontroller MSP430); Diplomarbeit, Fachhochschule Landshut, Mai 1997

Часть II.

1. Введение

Во второй части данного примера применения описан приём и распознавание сигналов DTMF при помощи микроконтроллера MSP430. Рассматриваются теоретические и математические вопросы, связанные с реализацией цифровых фильтров. Отдельно рассмотрено применение таких фильтров для выделения требуемых компонентов из аналоговых сигналов. Для аналого-цифрового преобразования используется АЦП, имеющийся в конфигурации C32х. 8 цифровых фильтров обрабатываются в режиме реального времени, из их выходных значений восстанавливаются символы DTMF. Также в данном разделе приведена принципиальная схема, поясняющая подключение аналоговых сигналов к MSP430.

2. Прием сигналов DTMF с помощью цифровых фильтров-резонаторов

С целью приёма сигналов DTMF при помощи микроконтроллера без использования специализированных дорогостоящих компонентов, входные импульсы должны быть оцифрованы с помощью аналогово-цифрового преобразователя (АЦП) и распознаны методами цифровой фильтрации. MSP430 подходит для этих целей благодаря следующим особенностям:

- Несмотря на сравнительно низкую тактовую частоту (до 3.3 МГц) (в современных моделях – до 8 МГц, прим. перев.), микроконтроллер имеет достаточно высокую вычислительную мощность благодаря наличию команд, выполняемых за один цикл, аналогичных архитектуре RISC.
- Благодаря наличию встроенного АЦП, подключение МК к аналоговым системам не представляет сложностей. Необходимость во внешнем АЦП отсутствует.

3. Основы цифровых фильтров

В аналоговой схемотехнике для фильтрации сигналов используются активные и пассивные фильтры, состоящие из резисторов, конденсаторов, катушек индуктивности и усилителей. Обрабатываемые сигналы при этом

непрерывны во времени. При использовании цифровой обработки сигналов имеется ряд преимуществ, таких, как повышенная точность и хорошая повторяемость, недостижимых в аналоговой схемотехнике из-за разброса и старения компонентов. Основными компонентами цифровых фильтров являются память и вычислительное устройство, таким образом, они имеют также преимущества с точки зрения структурной простоты. Тем не менее, для их работы также требуются аналогово-цифровые и цифроаналоговые преобразователи. В отличие от аналоговых фильтров, имеющих дело с непрерывно меняющимися сигналами, цифровые фильтры обрабатывают последовательности дискретных отсчётов.

Эти значения получаются в процессе «выборки» непрерывного сигнала через фиксированные промежутки времени при помощи АЦП, который в данном случае должен содержать устройство выборки-хранения (УВХ). По теореме о дискретизации [2] частота выборки должна быть не менее, чем в два раза выше максимальной частоты, присутствующей во входном сигнале. Исходя из этого, перед АЦП установлен аналоговый ФВЧ [3].

Если на выходе цифрового фильтра требуется получить аналоговый сигнал, вычисленные значения преобразуются цифроаналоговым преобразователем, на выходе которого сигнал фильтруется при помощи ФВЧ [3].

В данном примере внимание обращено только на обработку отфильтрованного сигнала, обратное преобразование в аналоговый сигнал не требуется.

3.1. Свойства цифровых фильтров-резонаторов

Алгоритмы цифровой фильтрации, как правило, требуют большого числа операций умножения. С целью уменьшения числа переполнений и ошибок округления, разрядность операндов должна быть достаточно высокой. По этой причине, желательно использовать для вычислений сигнальные процессоры с аппаратным умножителем. Разрядность при этом имеет диапазон от 16 до 24 бит.

Некоторые сигнальные процессоры также имеют возможность работы с числами в формате «плавающая точка», в котором обрабатываемые данные представлены с соответствующей точностью. В этом случае для цифровой фильтрации используются цифровые фильтры-резонаторы (ЦФР) (Wave Digital Filters (WDFs)). Такие фильтры {5,107} аналогичны аналоговым LC – цепям и микрополосковым фильтрам, их характеристики также идентичны характеристикам соответствующих аналоговых фильтров.

Особенностью ЦФР является пониженная чувствительность к изменению коэффициентов {5}. Это особенно важно в нашем случае, т.к. все операции

умножения из-за отсутствия аппаратного умножителя выполняются методом «сдвиг-сложение». Таким образом, укорочение и оптимизация коэффициентов экономит время вычислений. При этом, такое упрощение не наносит вреда стабильности частотных характеристик фильтра. В результате применения более коротких коэффициентов понижается ошибка округления, возникающая при умножении. Ещё одно положительное свойство ЦФР – большой динамический диапазон {12, 22}.

Стабильность ЦФР не нарушается при разрывах входного сигнала. Это его свойство позволяет обрабатывать запросы прерываний, после чего возвращаться к обработке фильтров. Благодаря «пассивной» природе таких фильтров, к их положительным свойствам можно также отнести быстрое восстановление при перегрузках и других искажениях входного сигнала. Такое поведение известно как «Стабильность ИХ фильтра» (“Response Stability”) {120}. Для обычных цифровых БИХ-фильтров эта проблема нерешаема [5, S 302; 136], в то время, как ЦФР остаются стабильными даже при «закольцевании» [19].

3.2. Структура используемых цифровых фильтров-резонаторов

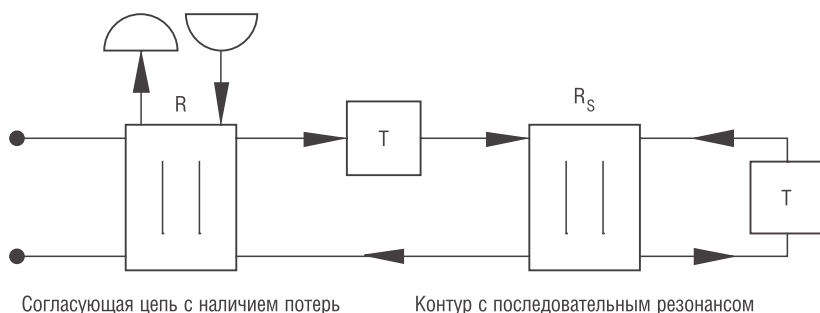


Рис. 13. Структура цифрового фильтра-резонатора

В нашей задаче требуется определение наличия конкретной частоты во входном сигнале. С этой целью используется фильтр, изображенный на рис. 13 и состоящий из контура с последовательным резонансом и «согласующей цепи» с наличием потерь [4]. Амплитудная характеристика такого фильтра показана на рис. 14.

Фильтры рассчитаны таким образом, чтобы резонансная частота в точности соответствовала искомой. Если интересующая нас частота появляется в спектре входного сигнала, она будет усилена, в то время, как остальные

частоты будут ослаблены. Коэффициент усиления резонансной частоты, либо коэффициент ослабления остальных частот определяется параметром K в виде $-20 \log K$ (дБ). Полоса пропускания фильтра определяется как полоса частот по уровню $-10 \log K$ (дБ). Превышение амплитудой на выходе фильтра определённого порога означает присутствие искомой частоты в спектре. Регулировка усиления производится при помощи согласующей цепи с наличием потерь. Структура этой цепи идентична для всех фильтров (см. рис. 15). Цепь с последовательным резонансом состоит из двух элементов задержки и адаптера. Существуют четыре различных конфигурации в зависимости от требуемой резонансной частоты (рис. 16).

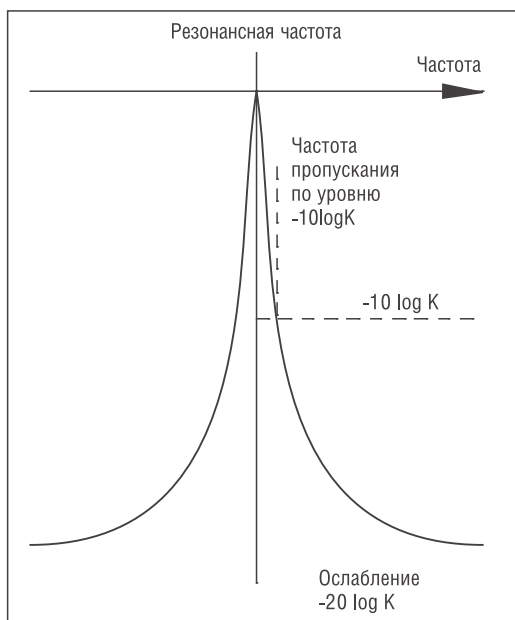


Рис. 14. Амплитудно-частотная характеристика цифрового фильтра-резонатора

Т.к. в данном примере арифметика с плавающей точкой не используется, алгоритм фильтра вычисляется с целочисленной точностью 16 бит; коэффициенты умножения присутствуют в конфигурациях А и D, что улучшает динамические характеристики фильтра.

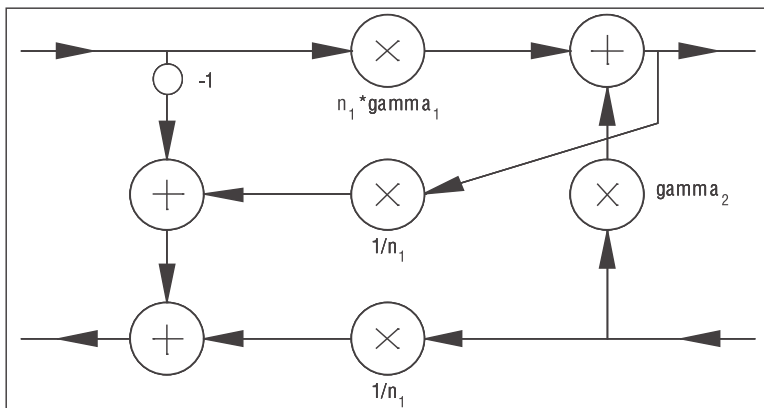


Рис. 15. Структура согласующей цепи с наличием потерь

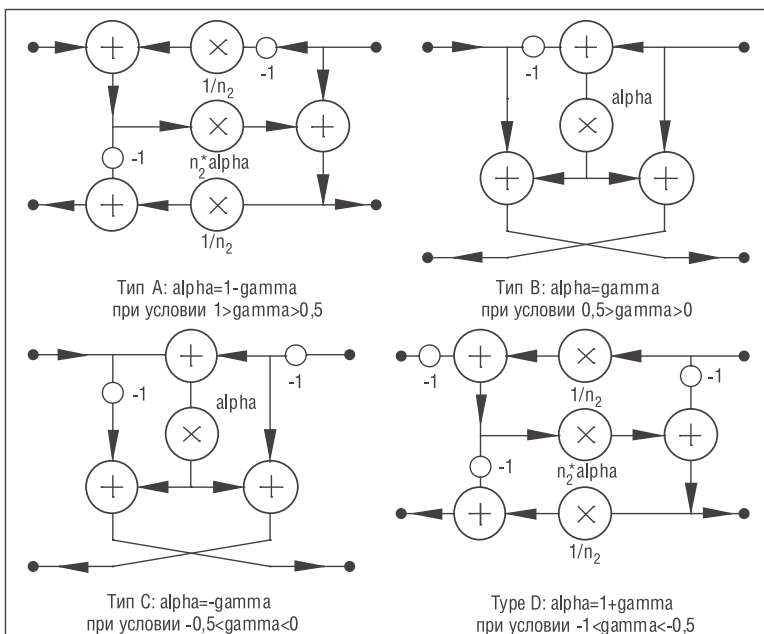


Рис. 16. Четыре различные конфигурации четырёхполюсника

4. Представление чисел и арифметика

Т.к. из ценовых соображений MSP430X325 не содержит аппаратного умножителя, все операции умножения выполняются при помощи сдвигов и сложений либо вычитаний. Любое умножение двух чисел может быть представлено как многократное прибавление множителя. Если над двоичным числом произвести n операций сдвига вправо, результат будет идентичен умножению на 2^{-n} . Т.к. представления коэффициентов в интервале 2^n как правило недостаточно, каждая '1' в двоичном числе должна быть заменена при помощи сдвига и последующего сложения. Поясним это на примере умножения с использованием метода сдвига-сложения (shift-and-add):

$ \begin{array}{r} 1 * 0.46875 \\ \\ = 2^{-2} \\ + 2^{-3} \\ + 2^{-4} \\ + 2^{-5} \\ \hline = 0.46875 \end{array} $	$ \begin{array}{r} 1b * 0.011110b \\ \\ = 0.01b \\ + 0.001b \\ + 0.0001b \\ + 0.00001b \\ \hline = 0.01111b \end{array} $
--	--

Ввиду того, что в таких числах слишком много знаков после запятой, требуется большое число сложений для того, чтобы представить число в виде дополнения до 2-х. Чтобы повысить скорость обработки в таких случаях, используется кодировка Canonically Signed Digit Code (CSD) [5]. В результате использования кода CSD, число знаков коэффициента после запятой уменьшено до минимума. Это достигнуто введением величины -1 для представления двоичных чисел. Эта величина появляется в вычислениях при вычитании. Вышеприведенный пример при использовании кода CSD будет выглядеть следующим образом:

$ \begin{array}{r} 1 * 0.46875 \\ \\ = 2^{-2} \\ - 2^{-5} \\ \hline = 0.46875 \end{array} $	$ \begin{array}{r} 1b * 0.1000 - 10 \text{ (CSD)} \\ \\ = 0.01b \\ - 0.00001b \\ \hline = 0.01111b \end{array} $
--	---

В результате использования кода CSD, процедура данного вычисления сокращена на два сложения путём замены одним вычитанием. Практическая выгода использования кода CSD имеет место только в случае, когда двоичные числа состоят из большого количества единиц.

Учитывая, что результат умножения n бит на m бит имеет разрядность $n+m$ бит, а разрядность операндов не может увеличиваться от одного умножения к другому, результат должен быть округлён или младшие биты долж-

ны быть отброшены, желательно при этом не потерять требуемую точность. С минимальной ошибкой это может быть обеспечено использованием метода Хорнера [5].

В нижеприведенном примере 12-битный результат формируется умножением 12-битного и 9-битного множителей.

Обычное умножение выглядит так:

X	*	Y		
0.11001011111b	*	0.01001001b		
0.796386719	*	0.28515625	=	0.22709465

При использовании метода сдвига-сложения:

0.00110010111	$X \cdot 2^{-2}$		
0.00000110010	$X \cdot 2^{-5}$		
0.00111001111	$X \cdot 2^{-8}$	=	0.226074219 (-2.1 LSB)

Как видно из примера, ошибка превышает 2 МЗР, из-за того, что большое количество бит было утеряно в результате сдвига вправо.

Метод Хорнера лишён этого недостатка, в нём младшие биты сохраняются до тех пор, пока это возможно:

0.00011001011	$X \cdot 2^{-3}$		
0.11001011111	+ X		
0.11100101010			
0.00011100101	$(X \cdot 2^{-3} + X) \cdot 2^{-3}$		
0.11001011111	+ X		
0.111101000100			
0.00111010001	$(X \cdot 2^{-3} + X) \cdot 2^{-3} X \cdot 2^{-3} + X) \cdot 2^{-3} =$		0.227050781 (-0.1 LSB)

Процесс начинается с младшей '1' множителя, в дальнейшем происходит накопление, после чего промежуточный результат сдвигается вправо (на разницу положений бит до следующей '1'), с последующим сложением, сдвигом и т.д. Операции абсолютного сдвига (в данном случае 2^{-8} , 2^{-5} , 2^{-2}) выполняются корректно. Вышеописанный процесс используется для введения и оптимизации коэффициентов фильтра, с целью минимизировать вычислительные ресурсы при максимальном сохранении точности. Т.к. арифметические вычисления в микроконтроллере обычно осуществляются с 16-битными целыми операндами, следует договориться о представлении значений сигнала. Для программной обработки операций с плавающей точкой потребуется слишком

много времени, таким образом, следует использовать арифметику с фиксированной точкой.

Примем следующее распределение бит для алгоритма фильтрации:

- 1 Бит знака
- 2 Бита переполнения
- 13 Бит данных с фиксированной точкой

Таким образом, двоичное число 0.01111b будет представлено в данной системе, как показано на рис. 17:



Рис. 17. Цифровое представление значений сигнала

Отрицательный коэффициент представляется при помощи разделения на две части. Первая часть равна -1, вторая представляет дополнение до требуемого отрицательного значения. Таким образом, -0,3 будет представлено как $-1 + 0,7$.

5. Расчет 8-ми цифровых фильтров-резонаторов и оптимизация коэффициентов

ЦФР используются для распознавания частоты, расчёт таких фильтров требует определения следующих параметров [4]:

- Частота выборки
- Резонансная частота фильтра
- Усиление (ослабление) фильтра в виде $-20 \log K$ (дБ)
- Полоса пропускания по уровню $-10 \log K$ (дБ)

Так как наивысшая частота DTMF равна 1633 Гц, по теореме о квантовании частота выборок в 3640 Гц будет достаточной. Это соответствует 1/9 частоты ACLK, равной 32.768 кГц присутствующей в MSP430, и по этой причине она легко может быть сгенерирована таймером. Несмотря на то, что речевые сигналы, попадая на вход АЦП оцифровываются с очень низкой частотой, это не приведёт к некорректному поведению при распознавании DTMF, т.к. сигнал распознаётся как корректный DTMF символ только в том случае, если в спек-

ре присутствуют обе частоты DTMF одновременно на протяжении определённого промежутка времени. Кроме этого, во время передачи сигнала DTMF все посторонние сигналы, включая речевые, должны быть ослаблены не менее, чем на 23 дБ [1].

Резонансные частоты фильтров соответствуют каждой индивидуальной частоте DTMF. Такие частоты должны проходить через фильтр без ослабления. Подавление для остальных частот $-20 \log K$ (дБ) выбрано на уровне -40дБ. Это соответствует коэффициенту ослабления $K=100$. Если сигнал с частотой, равной граничной частоте фильтра (по уровню $-10 \log K$) попадёт в соседний канал DTMF, то при этом гарантируется, что уровень искомой частоты будет в 10 раз выше, чем у соседней частоты. Это соответствует разности в 20 дБ. Однако, в результате укорочения и оптимизации коэффициентов, достичь коэффициента 10 не удастся, но при том реальная разница уровней является полностью достаточной для надёжного распознавания частот. При сравнении выходных значений фильтров, частота DTMF считается присутствующей в спектре, если значение на выходе соответствующего фильтра в 10 раз превышает значение на выходах остальных фильтров.

Учитывая вышеописанные требования, коэффициенты для 8 фильтров вычислим следующим образом [4]:

Частота, Гц		697	770	852	941	1209	1336	1477	1633
Цепь с посл. резонансом	Тип согл. цепи	В	В	В	С	С	Д	Д	Д
	alpha	0.3594420	0.2393157	0.0999495	0.0534851	0.4935082	0.3291608	0.1702124	0.0516459
	n2	—	—	—	—	—	1.5146077	2.4736004	5.8005043
Согла-сующая цепь с налив-нием потерь	gamma1	0.9773007	0.9774177	0.9759909	0.9749654	0.9696236	0.9702183	0.9692265	0.9694046
	gamma2	-0.9743449	-0.9745813	-0.9716988	-0.9696270	-0.9588356	-0.9600369	-0.9580333	-0.9583930
	n1	0.0250727	0.0248447	0.0276217	0.0296126	0.0399140	0.0387729	0.0406752	0.0403340

Т.к. данные коэффициенты затруднительно представить в двоичной системе и вычисления с их использованием потребуют больших ресурсозатрат, в дальнейшем произведём укорочение и оптимизацию коэффициентов. Величины $n1$ и $n2$ являются масштабирующими коэффициентами и довольно свободно могут быть изменены. Поэтому они были округлены до следующей минимальной степени двух, что позволило упростить вычисления до нескольких операций сдвига. Значения коэффициента α требуют большего внимания, т.к. они влияют на резонансную частоту. Чтобы удостовериться в корректности

проведенного укорочения данных коэффициентов, резонансная частота была пересчитана заново с учётом новых значений. В результате успешной оптимизации были получены следующие значения:

Частота, Гц	697	770	852	941	1209	1336	1477	1633
alpha (*n2)	0.359375	0.234375	0.09375	0.046875	0.5	0.328125	0.171875*2	0.046875*4
Резонансная частота	697	773	855	937	1213	1337	1475	1642
Отклонение, %	0	0.39	0.35	-0.43	0.33	0.07	-0.14	0.55
alpha (*n2) в представлении CSD	0.010111	0.010000-1	0.00011	0.000011	0.1	0.010101	0.01011	0.0011

Несмотря на значительное укорочение коэффициентов и, благодаря этому, времени вычисления, отклонение частоты от номинальной не существенно. При внимательном рассмотрении выясняется подобие коэффициентов согласующей цепи с наличием потерь. Это легко объяснимо с учётом того фактора, что данные коэффициенты зависят в основном от параметра $-10 \log K$, который выбран одинаковым для всех фильтров исходя из критерия: соседняя частота DTMF должна быть подавлена на $-10 \log K$.

В результате этого, для всех 8-ми фильтров DTMF была выбрана единственная согласующая цепь с округлённым значением. Такое решение имеет массу преимуществ:

- Требуется расчёт и программирование только одной цепи вместо 8
- Ассемблерный код лучше оптимизирован
- Существенно упрощается отладка при использовании одинаковой программы
- При наличии свободного времени на вычисления, можно сэкономить память вызывая подпрограмму

Оптимальные значения коэффициентов представлены в таблице:

диапазон	697 Гц – 1633 Гц
n1*gamma1	0.00001b
gamma2	-1,0 + 0,00001b
1/n1	32

6. Проверка расчета фильтров с помощью программ математического моделирования

Встроенный в MSP430 АЦП даёт возможность перехода от аналоговых к цифровым сигналам. Однако, для проверки характеристик фильтров, нам потребуются точные значения на выходе фильтров. Получить их непосредственно невозможно (т.к. встроенный ЦАП отсутствует), поэтому проверить амплитудные характеристики фильтра также невозможно. Чтобы получить в этом случае информацию о достигнутых характеристиках фильтров, алгоритм был смоделирован математической программой, с её помощью были получены амплитудные характеристики.

В отличие от микроконтроллера, осуществляющего вычисления с 16-битной точностью, алгоритм в данном случае оперирует с точностью плавающей запятой. При этом были использованы укороченные коэффициенты. В качестве примера приведена программа моделирования и результат – амплитудная характеристика фильтра на 770 Гц. Для сравнения также приведена характеристика фильтра с неокруглёнными коэффициентами. (Рис. 18):

```

f(alpha,gamma_1_n1,gamma_2,n1):=
    t1 ← 0
    t2 ← 0
    for i ∈ 0..N
        k1 ← (t2 - t1) × alpha
        k3 ← k1 + t2
        k2 ← k1 + t1
        t2 ← k2
        t1 ← k3 × gamma_2 + x_i · gamma_1_n
        y_i ← (k3 + t1) / n1 - x_i
    y

```

Программа для проверки фильтра на 770 Гц методом математического моделирования

$u := f(0.234375, 0.0242837, -0.9745813, 0.0248447$

Функция с неокруглёнными значениями gamma_1_n1, gamma_2 и n1

$$z := f \left[0.234375, \frac{1}{32}, \left(\frac{1}{32} - 1 \right), \frac{1}{32} \right]$$

Функция с округлёнными значениями

- Резонансная частота равна 773 Гц, поэтому предварительно полученное значение $\alpha = 0.234375$ достигается точно.
- Усиление фильтра уменьшилось до -36 дБ взамен -40 дБ. Причиной этого является округление коэффициентов γ_{11} и γ_{22} . При моделировании с использованием точных значений коэффициентов достигается расчётное значение -40 дБ.
- Соседняя частота DTMF (697 Гц) имеет усиление -18.5 дБ, что соответствует 1/8 входного значения. Такого разделения достаточно для безошибочного распознавания частоты.

Этот пример демонстрирует весьма важное свойство цифровых фильтров-резонаторов – их низкую чувствительность к округлению коэффициентов: несмотря на значительное укорочение коэффициентов и сокращение времени расчёта, характеристики фильтра существенно не изменились.

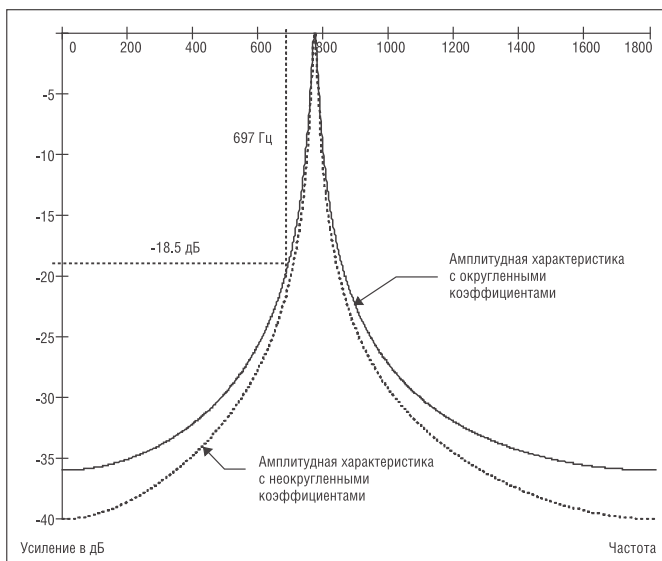


Рис. 18. Амплитудно-частотная характеристика фильтра на 770 Гц, смоделированная при помощи MathCad

7. Программное обеспечение для реализации цифровых фильтров

Алгоритмы цифровых фильтров требуют большой вычислительной мощности, ввиду большого числа умножений и сложений, требуемых при расчётах – особенно в тех случаях, когда умножение производится без помощи аппаратных умножителей, а осуществляется методом сдвигов-сложений. По этой причине для таких задач часто применяются цифровые сигнальные процессоры, т.к. они имеют быстрый умножитель с накоплением (MAC), или, как минимум, многорегистровый циклический сдвигатель (Barrel Shifter), позволяющий осуществлять операцию сдвига на несколько позиций одной командой. Таким образом, реализовать алгоритм фильтра в реальном времени с использованием медленного микроконтроллера невозможно.

Тем не менее, микроконтроллер MSP430 позволяет реализовать такой алгоритм благодаря тому, что инструкции, операнды которых находятся в регистрах, выполняются за один такт. Кроме этого, для повышения вычислительной мощности системная частота повышена с 1 МГц до 3.3 МГц (прим. перев. – на момент перевода макс. частота семейства – 8МГц).

Аналоговая часть легко сопрягается с цифровой при помощи встроенного АЦП. Его разрешение в 14 бит достаточно высоко, чтобы гарантировать в последующем хороший динамический диапазон цифрового фильтра. При этом гарантируется, что сигнал с низкой амплитудой будет обработан с такой же точностью, что и сигнал с большой амплитудой. Необходимость во внешнем АЦП соответственно отпадает.

Если алгоритм должен быть обработан в режиме реального времени, вычисление определённого «куска» данных должно быть завершено перед тем, как будут получены новые данные. Промежуточное хранение данных в ОЗУ невозможно, т.к. его объёма недостаточно для хранения большого массива данных, поступающих из АЦП с определённой частотой.

Для таймера-порта выбрана частота прерываний 3640 Гц. Это соответствует девяти тактам частоты ACLK. АЦП запускается по прерыванию таймера-порта. Т.к. частота ACLK генерируется при помощи «часового» кварца частотой 32.768 кГц, получаемая частота выборки имеет низкое дрожание фазы (джиттер) и высокую стабильность, присущую кварцевому резонатору. При этом гарантируется корректная выборка аналогового сигнала и, соответственно, верные расчётные значения [10]. По завершении преобразования, осуществляется вычисление для 8-ми цифровых фильтров и распознавание частот DTMF на основе полученных значений фильтров. На алгоритм расчёта и распознавания выделено, таким образом, 274.7 мкс. Это соответствует 915 тактам при дли-

тельности такта 300 нс. По завершении этого времени вычисление должно повторяться с использованием обновлённых данных АЦП. Реализация алгоритма представлена на рис. 19.

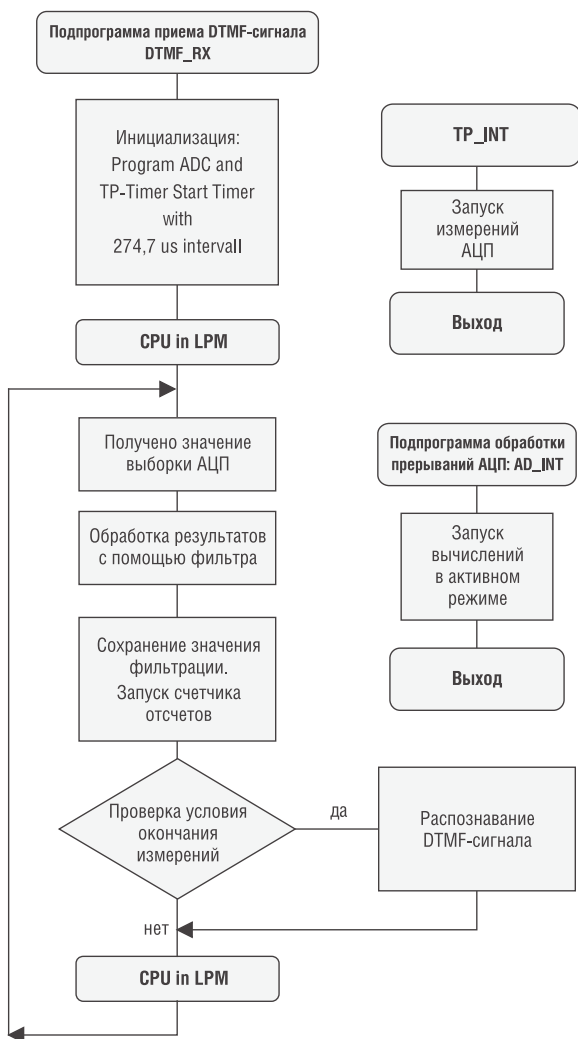


Рис. 19. Алгоритм вычисления 8-ми цифровых фильтров и распознавания сигналов DTMF в режиме реального времени

8. Программа распознавания сигналов DTMF

С целью распознавания полученного символа DTMF из вычисленных значений фильтров, после вычислений фильтров применяется алгоритм декодирования. Этот алгоритм должен сопоставить распознанные частоты символу DTMF. Кроме этого, в процессе вычисления фильтров производится сравнение их значений. В практической реализации, отдельно для «верхней» (Hi-Group) и «нижней» (Lo-Group) групп частот находится частота с максимальной амплитудой, её значение сохраняется. Таким образом, после каждого вычисления 8-ми фильтров нам известны частоты «верхней» и «нижней» группы с максимальной амплитудой. На рис. 20 представлен алгоритм декодирования:

Чтобы убедиться в том, что все фильтры находятся в установившемся режиме и максимумы соответствующих частот определены, всегда производится 20 последовательных просчётов фильтра. Когда обработано достаточное число отсчётов, два максимума из «верхней» и «нижней» группы проверяются на достоверность по двум критериям. Первый критерий – оба максимальных значения должны превышать уровень шума. Второй критерий – незначительность отличия максимальных уровней друг от друга. Конечно, эти критерии могут быть расширены и дополнены с целью улучшения качества приёма относительно слабых сигналов.

В том случае, если сигнал удовлетворяет обоим условиям, обе частоты DTMF считаются достоверными. При этом, частоты могут быть признаны сигналами DTMF только по истечении определённого времени. Это осуществляется при помощи счётчика длительности, фиксирующего длительность сигнала. По достижении требуемой длительности, символ DTMF выбирается из таблицы исходя из двух частот, определяющих столбец и строку. Чтобы исключить повторное распознавание одного сигнала, при достижении требуемой длительности сигнала устанавливается соответствующий флаг, сигнализирующий о достоверности принятого символа.

При невыполнении одного из условий счётчик длительности декрементируется. Достижение счётчиком нуля интерпретируется как пауза между символами. Флаг при этом сбрасывается.

На рис. 21 показано графическое представление временных параметров при распознавании сигнала DTMF:

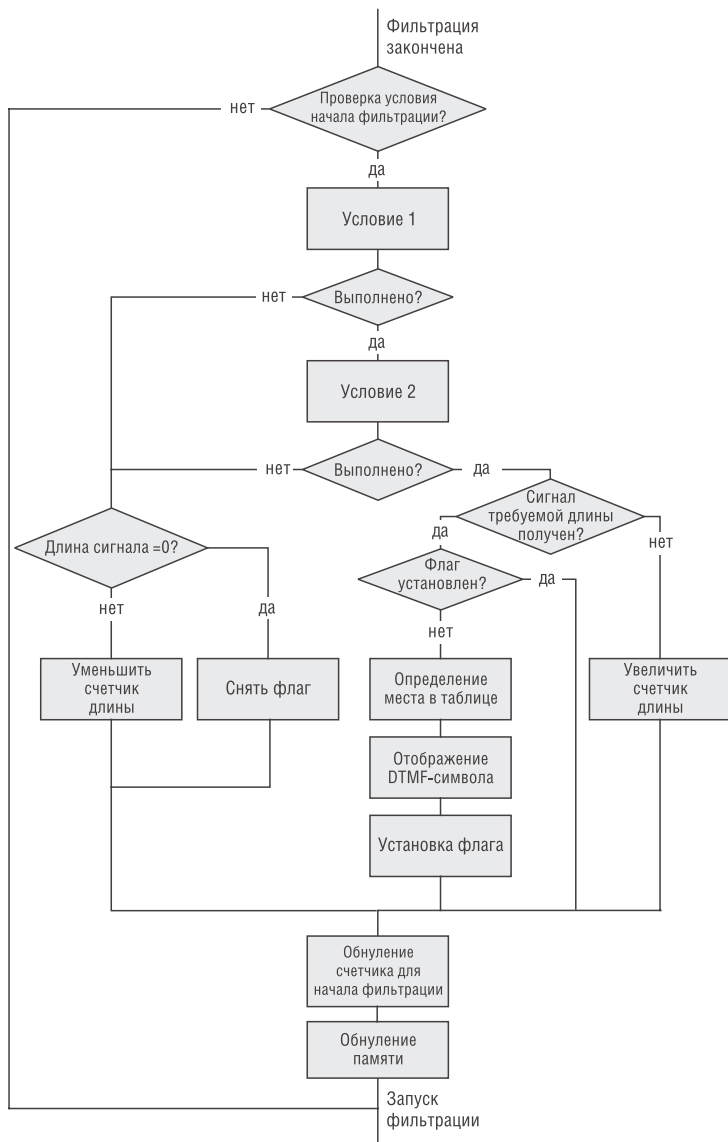


Рис. 20. Алгоритм декодирования

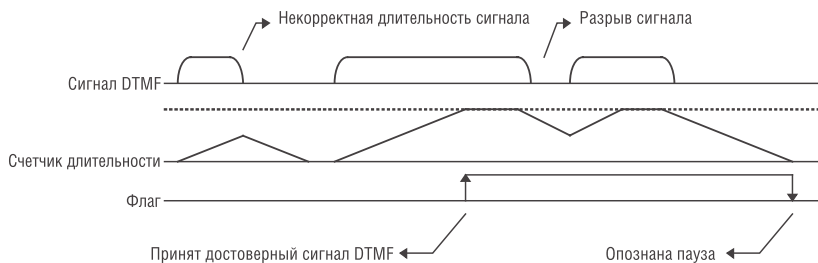


Рис. 21. Временные параметры при распознавании сигнала DTMF

```

; *****
; Программа вычисления 8-ми цифровых фильтров-резона-
; торов для определения сигналов DTMF.
; Входной аналоговый сигнал оцифровывается на входе A4
; с частотой отсчётов 3640 Гц
; Распознанные символы DTMF отображаются на ЖКИ
; Автор Robert Siwy, October 1997
; *****

```

```

FILTER_1 .equ 1 ; включить фильтр на 1633
FILTER_2 .equ 1 ; включить фильтр на 1477
FILTER_3 .equ 1 ; включить фильтр на 1336
FILTER_4 .equ 1 ; включить фильтр на 1209
FILTER_5 .equ 1 ; включить фильтр на 941
FILTER_6 .equ 1 ; включить фильтр на 852
FILTER_7 .equ 1 ; включить фильтр на 770
FILTER_8 .equ 1 ; включить фильтр на 697

```

```

; Определения для используемой аппаратной части

```

```

STACK .equ 400h
TCCTL .equ 42h
TCPLD .equ 43h
TCDAT .equ 44h
BTCTL .equ 40h
LCDCTL .equ 30h
IE1 .equ 00h
IE2 .equ 01h
SCFI .equ 50h
SCFQCTL .equ 52h
WDTCTL .equ 0120h
WDTCCL .equ 88h
WDTPW .equ 05a00h

```

```

ACTL      .equ      0114h
PD        .equ      1000h
ADAT      .equ      0118h
; Программные определения
DISPL     .equ      031h      ; Базовый адрес ЖКИ
LPM0      .equ      010h
LENGTH    .equ      7        ; (Length+1) * 5.48 мс = дли-
                                ; тельность сигнала
THRE      .equ      100      ; порог шума
FLAG_REG  .equ      r4
LCOUNT    .equ      r5
ROW       .equ      r6      ; Строка «нижней» группы
COL       .equ      r7      ; Столбец «верхней» группы
MAXLO     .equ      r8
IN        .equ      r9      ; Входной регистр
MAXHI     .equ      r10
OUT       .equ      r13
COUNT    .equ      r15
; *****
; Распределение памяти для 8-ми фильтров
; *****
        .bss      T1_1633, 2
        .bss      T2_1633, 2
        .bss      T1_1477, 2
        .bss      T2_1477, 2
        .bss      T1_1336, 2
        .bss      T2_1336, 2
        .bss      T1_1209, 2
        .bss      T2_1209, 2
        .bss      T1_941, 2
        .bss      T2_941, 2
        .bss      T1_852, 2
        .bss      T2_852, 2
        .bss      T1_770, 2
        .bss      T2_770, 2
        .bss      T1_697, 2
        .bss      T2_697, 2
; *****
; Программа
; *****
        .text
INIT
        mov      #STACK-2, SP      ; инициализация
                                    ; стека

```

```

        mov        #WDTPW+WDTCCL, &WDTCCTL ; остановить
                                                ; сторожевой
                                                ; таймер

; Инициализация MCLK
        mov.b      #102-1, &SCFQCTL          ; 102*32.768 Гц =
                                                ; 3,342 МГц
        bis.b      #008h, &SCFIO             ; Макс. частота
                                                ; 3 MHz

; Инициализация ЖКИ/очистка памяти ЖКИ
CLRSCR
        mov        #11, r5
        clr1
        clr.b      DISPL-1 (r5)
        dec        r5
        jnz        clr1
        mov.b      #017h, &BTCTL             ; Тактирование
                                                ; ЖКИ
        mov.b      #0ffh, &LCDCTL            ; Генератор ЖКИ

; Инициализация АЦП
        mov        #04912h, &ACTL             ; Инициализация
                                                ; АЦП, внутрен-
                                                ; ний ИОН
        bis.b      #004h, &IE2               ; разрешить пре-
                                                ; рывания от АЦП

; Инициализация 8-битного таймера
        mov.b      #100h-9, &TCPLD ; счёт до 9
                                                ; тактов
                                                ; ACLK=274 мкс
        mov.b      #000h, &TCDAT             ; загрузка
                                                ; счётчика
        mov.b      #068h, &TCCTL            ; Инициализация
                                                ; 8-битного тай-
                                                ; мера
        mov.b      #008h, &IE1               ; разрешить прерывания
                                                ; от таймера
        eint
        bis.b      #LPM0, SR                 ; выкл. CPU

DTMF_FILTER
        inc        COUNT
        cmp        #20, COUNT                ; 20 отсчётов принято?
        jlo        FILTER                    ; нет

```

```

; проверка 1-го условия
    cmp     #THRE,MAXLO      ; Максимум для «ниж-
                             ; ней» группы ниже
                             ; порога?
    jlo     FALSE            ; да, выход
    cmp     #THRE,MAXHI      ; Максимум для «верх-
                             ; ней» группы ниже
                             ; порога?
    jlo     FALSE            ; да, выход

; проверка 2-го условия
    rra     MAXLO             ; разделить MAXLO на 2
    cmp     MAXLO,MAXHI      ; MAXHI > MAXLO/2
    jlo     FALSE            ; да, выход
    rla     MAXLO             ; восстановить MAXLO
    rra     MAXHI            ; разделить MAXHI на 2
    cmp     MAXHI,MAXLO      ; MAXLO > MAXHI/2
    jlo     FALSE            ; да, выход
    cmp     #LENGTH,LCOUNT  ; длительность сигнала
                             ; ла достаточна?
    jeq     DISPLAY          ; да, переход к отоб-
                             ; ражению
    inc     LCOUNT          ; нет, инкремент счёт-
                             ; чика длительности
    jmp     CONTINUE

DISPLAY
    bit     #01h,FLAG_REG    ; проверка флага
    jnz     CONTINUE          ; распознавание завер-
                             ; шено
    add     ROW,COL           ; суммировать строку
                             ; и столбец таблицы
    mov.b   DTMF_Tab(COL),DISPL ; отобразить
                             ; символ DTMF
    bis     #01h,FLAG_REG    ; установить флаг
    jmp     CONTINUE

FALSE
    tst     LCOUNT
    jnz     FALSE1
    mov.b   #008h,DISPL+3    ; отобразить паузу
    mov.b   #008h,DISPL+4    ; « «
    bic     #01h,FLAG_REG    ; очистить флаг
    jmp     CONTINUE

FALSE1
    dec     LCOUNT

```

CONTINUE

```

    clr    MAXHI
    clr    MAXLO
    clr    COUNT

```

FILTER

```

    mov     &ADAT, IN      ; Отсчёт ? r9/IN
    sub     #01ffff, IN    ; сформировать значе-
                           ; ние со знаком
    rla     IN              ; *2
    rla     IN              ; *2
    .if     FILTER_1

```

FILTER_1633

```

    mov     T2_1633, r12    ; N11=T2/4-T1
    mov     r12, r13
    rra     r12
    rra     r12
    sub     T1_1633, r12    ; N11 в r12
    mov     r12, r14        ; N11*2*alpha-T2
    rra     r12
    rra     r12
    rra     r12
    mov     r12, r11
    rra     r12
    add     r12, r11
    sub     r13, r11        ; N15 в r11
    mov     r11, T2_1633    ; N3=N15/4-N11
    rra     r11
    rra     r11
    sub     r14, r11        ; N3 в r11
                           ; Общая согласующая цепь с потерями
                           ; требуется N3 в r11
                           ; 1/n1=32, gamma_2=-1+1/32)
                           ; n1*gamma_1=1/32
    mov     r11, r13        ; сохранить N3 в r13
    mov     IN, r12         ; T1=N3*gamma_2+n1*gamma_1*IN
    rra     r12
    rra     r12
    rra     r12
    rra     r12
    rra     r12            ; n1*gamma_1*IN in r12
    sub     r11, r12
    rra     r11
    rra     r11
    rra     r11

```

```

rra      r11
rra      r11
add      r11,r12 ; T1 в r12
mov      r12,T1_1633 ; сохранить T1 в ОЗУ
rla      r13      ; N3 * 32
rla      r13
rla      r13
rla      r13
rla      r13
rla      r12      ; T1 * 32
rla      r12
rla      r12
rla      r12
rla      r12
sub      IN,r12
add      r12,OUT ; В r13 выходное значение
;*****
cmp      OUT,MAXHI      ; Сравнить выходное
                        ; значение с миниму-
                        ; мом
jge      SWAP1
mov      OUT,MAXHI      ; замена максимума
mov      #03,COL      ; столбец = 3
SWAP1    ;*****
        ; окончание согласующей цепи
FILTER_1633_END
        .endif
        .if      FILTER_2
FILTER_1477
mov      T2_1477,r12      ; N11=T/2-T1
mov      r12,r13
rra      r12
sub      T1_1477,r12      ; N11 в r12
mov      r12,r14      ; N15=N11*2*alpha-T2
rra      r12
rra      r12
mov      r12,r11
rra      r12
rra      r12
add      r12,r11
rra      r12
add      r12,r11
sub      r13,r11      ; N15 в r11

```



```

mov     r11,T2_1477      ; N3=N15/2-N11
rra     r11
sub     r14,r11          ; N3 в r11
; Общая согласующая цепь с потерями
; требуется N3 в r11
; 1/n1=32, gamma_2=-1+1/32)
; n1*gamma_1=1/32
mov     r11,r13          ; сохранить N3 в r13
mov     IN,r12           ; T1=N3*gamma_2+n1*gamma_1*IN
rra     r12
rra     r12
rra     r12
rra     r12              ; n1*gamma_1*IN в r12
sub     r11,r12
rra     r11
rra     r11
rra     r11
rra     r11
add     r11,r12          ; T1 в r12
mov     r12,T1_1477      ; сохранить T1 в ОЗУ
rla     r13              ; N3 * 32
rla     r13
rla     r13
rla     r13
rla     r12              ; T1 * 32
rla     r12
rla     r12
rla     r12
rla     r12
sub     IN,r12
add     r12,OUT          ; В r13 выходное
; значение
; *****
cmp     OUT,MAXHI        ; Сравнить выходное
; значение с максимумом
; мом
jge     SWAP2
mov     OUT,MAXHI        ; замена максимума
mov     #02,COL          ; столбец = 2
SWAP2    ; *****

```

; окончание согласующей цепи

```

FILTER_1477_ENDE
    .endif
    .if      FILTER_3
FILTER_1336
    mov     T2_1336,r12    ; N11=T2-T1
    mov     r12,r13
    sub     T1_1336,r12    ; N11 в r12
    mov     r12,r14        ; N15=N11*alpha-T2
    rra     r12
    rra     r12
    mov     r12,r11
    rra     r12
    rra     r12
    add     r12,r11
    rra     r12
    rra     r12
    add     r12,r11
    sub     r13,r11        ; N15 в r11
    mov     r11,T2_1336
    sub     r14,r11        ; N3=N15-N11 в r11
    ; Общая согласующая цепь с потерями
    ; требуется N3 в r11
    ; 1/n1=32, gamma_2=-1+1/32)
    ; n1*gamma_1=1/32
    mov     r11,r13 ; сохранить N3 в r13
    mov     IN,r12    ; T1=N3*gamma_2+n1*gamma_1*IN
    rra     r12
    rra     r12
    rra     r12
    rra     r12
    rra     r12    ; n1*gamma_1*IN в r12
    sub     r11,r12
    rra     r11
    rra     r11
    rra     r11
    rra     r11
    rra     r11
    add     r11,r12 ; T1 в r12
    mov     r12,T1_1336    ; сохранить T1 в ОЗУ
    rla     r13            ; N3 * 32
    rla     r13
    rla     r13
    rla     r13

```

```

        rla      r13
        rla      r12      ; T1 * 32
        rla      r12
        rla      r12
        rla      r12
        rla      r12
        sub      IN,r12
        add      r12,OUT ; В r13 выходное значение
;*****
        cmp      OUT,MAXHI      ; Сравнить выходное
                                ; значение с миниму-
                                ; мом
        jge      SWAP3
        mov      r13,MAXHI      ; замена максимума
        mov      #01,COL      ; столбец = 1

SWAP3      ;*****
; окончание согласующей цепи
FILTER_1336_ENDE
        .endif
        .if      FILTER_4
FILTER_1209
        mov      T2_1209,r13      ; N1=(T1-T2)*alpha
        mov      T1_1209,r11
        mov      r11,r14
        sub      r13,r11
        rra      r11      ; N1 в r11
        mov      r11,r12      ; N3=N1-T2
        sub      r13,r11      ; N3 в r11
        sub      r14,r12      ; N2=N1-T1 в r12
        mov      r12,T2_1209
                                ; Общая согласующая цепь с потерями
                                ; требуется N3 в r11
                                ; 1/n1=32, gamma_2=-1+1/32)
                                ; n1*gamma_1=1/32
        mov      r11,r13      ; сохранить N3 в r13
        mov      IN,r12      ;T1=N3*gamma_2+n1*gamma_1*IN
        rra      r12
        rra      r12
        rra      r12
        rra      r12      ; n1*gamma_1*IN в r12
        sub      r11,r12
        rra      r11

```

```

rra      r11
rra      r11
rra      r11
rra      r11
add      r11,r12 ; T1 в r12
mov      r12,T1_1209 ; сохранить T1 в ОЗУ
rla      r13          ; N3 * 32
rla      r13
rla      r13
rla      r13
rla      r13
rla      r12          ; T1 * 32
rla      r12
rla      r12
rla      r12
rla      r12
sub      IN,r12
add      12,OUT ; В r13 выходное значение
;*****
cmp      OUT,MAXHI      ; Сравнить выходное
                        ; значение с миниму-
                        ; мом

jge      SWAP4
mov      r13,MAXHI      ; замена максимума
mov      #00,COL        ; столбец = 0

SWAP4      ;*****
; окончание согласующей цепи
FILTER_1209_ENDE
        .endif
        .if      FILTER_5
FILTER_941
        mov      T2_941,r13      ; N1=(T1-T2)*alpha
        mov      T1_941,r12
        mov      r12,r14
        sub      r13,r12
        rra      r12
        rra      r12
        rra      r12
        rra      r12
        rra      r12
        mov      r12,r11
        rra      r12
        add      r12,r11

```

```

mov     r11,r12          ; N1 в r11/r12
sub     r13,r11          ; N3=N1-T2 в r11
sub     r14,r12          ; N2=N1-T1
mov     r12,T2_941
        ; Общая согласующая цепь с потерями
        ; требуется N3 в r11
        ; 1/n1=32, gamma_2=-1+1/32)
        ; n1*gamma_1=1/32
mov     r11,r13          ; сохранить N3 в r13
mov     IN,r12           ; T1=N3*gamma_2+n1*gamma_1*IN
rra     r12
rra     r12
rra     r12
rra     r12
rra     r12             ; n1*gamma_1*IN в r12
sub     r11,r12
rra     r11
rra     r11
rra     r11
rra     r11
add     r11,r12          ; T1 в r12
mov     r12,T1_941       ; сохранить T1 в ОЗУ
rla     r13              ; N3 * 32
rla     r13
rla     r13
rla     r13
rla     r12             ; T1 * 32
rla     r12
rla     r12
rla     r12
rla     r12
sub     IN,r12
add     r12,OUT          ; В r13 выходное
        ; значение
; *****

cmp     OUT,MAXLO        ; Сравнить выходное
        ; значение с максимумом
jge     SWAP5
mov     r13,MAXLO        ; замена максимума
mov     #012,ROW         ; строка = 12

```

```

SWAP5      ;*****
; окончание согласующей цепи
FILTER_941_ENDE
        .endif
        .if      FILTER_6
FILTER_852
        mov      T2_852,r11      ; N1=(T2-T1)*alpha
        mov      r11,r12
        mov      T1_852,r13
        sub      r13,r12
        rra      r12
        rra      r12
        rra      r12
        mov      r12,r14
        rra      r12
        add      r12,r14      ; N1 в r14
        add      r14,r11      ; N3=T2+N1 в r11
        add      r14,r13      ; N2=T1+N1
        mov      r13,T2_852
        ; Общая согласующая цепь с потерями
        ; требуется N3 в r11
        ; 1/n1=32, gamma_2=-1+1/32)
        ; n1*gamma_1=1/32
        mov      r11,r13      ; сохранить N3 в r13
        mov      IN,r12      ; T1=N3*gamma_2+n1*gamma_1*IN
        rra      r12
        rra      r12
        rra      r12
        rra      r12
        rra      r12      ; n1*gamma_1*IN в r12
        sub      r11,r12
        rra      r11
        rra      r11
        rra      r11
        rra      r11
        add      r11,r12      ; T1 в r12
        mov      r12,T1_852      ; сохранить T1 в ОЗУ
        rla      r13      ; N3 * 32
        rla      r13
        rla      r13
        rla      r13
        rla      r13

```

```

        rla      r12      ; T1 * 32
        rla      r12
        rla      r12
        rla      r12
        rla      r12
        sub      IN,r12
        add      r12,OUT ; В r13 выходное значение
;*****
        cmp      OUT,MAXLO      ; Сравнить выходное
                                ; значение с миниму-
                                ; мом
        jge      SWAP6
        mov      r13,MAXLO      ; замена максимума
        mov      #08,ROW        ; строка = 8
SWAP6      ;*****
; окончание согласующей цепи
FILTER_852_ENDE
        .endif
        .if      FILTER_7
FILTER_770
        mov      T2_770,r11      ; N1=(T2-T1)*alpha
        mov      r11,r12
        mov      T1_770,r13
        sub      r13,r12
        rra      r12
        rra      r12
        mov      r12,r14
        rra      r12
        rra      r12
        rra      r12
        rra      r12
        sub      r12,r14      ; N1 в r14
        add      r14,r11      ; N3=T2+N1 в r11
        add      r14,r13      ; N2=T1+N1
        mov      r13,T2_770
                                ; Общая согласующая цепь с потерями
                                ; требуется N3 в r11
                                ; 1/n1=32, gamma_2=-1+1/32)
                                ; n1*gamma_1=1/32
        mov      r11,r13      ; сохранить N3 в r13
        mov      IN,r12      ; T1=N3*gamma_2+n1*gamma_1*IN
        rra      r12
        rra      r12

```

```

rra      r12
rra      r12
rra      r12      ; n1*gamma_1*IN в r12
sub      r11,r12
rra      r11
rra      r11
rra      r11
rra      r11
rra      r11
add      r11,r12 ; T1 в r12
mov      r12,T1_770      ; сохранить T1 в ОЗУ
rla      r13      ; N3 * 32
rla      r13
rla      r13
rla      r13
rla      r13
rla      r12      ; T1 * 32
rla      r12
rla      r12
rla      r12
rla      r12
sub      IN,r12
add      r12,OUT ; В r13 выходное значение
;*****

cmp      OUT,MAXLO      ; Сравнить выходное
                        ; значение с миниму-
                        ; мом

jge      SWAP7
mov      r13,MAXLO      ; замена максимума
mov      #04,ROW        ; строка = 4

SWAP7      ;*****
; окончание согласующей цепи
FILTER_770_ENDE
        .endif
        .if      FILTER_8
FILTER_697
mov      T2_697,r11
mov      r11,r12
mov      T1_697,r13
sub      r13,r12
rra      r12
rra      r12
mov      r12,r14

```



```

rra      r12
add      r12,r14
rra      r12
rra      r12
rra      r12
sub      r12,r14      ; N1 в r14
add      r14,r11      ; N3=T2+N1 в r11
add      r14,r13
mov      r13,T2_697
          ; Общая согласующая цепь с потерями
          ; требуется N3 в r11
          ; 1/n1=32, gamma_2=-1+1/32)
          ; n1*gamma_1=1/32
mov      r11,r13      ; сохранить N3 в r13
mov      IN,r12      ; T1=N3*gamma_2+n1*gamma_1*IN
rra      r12
rra      r12
rra      r12
rra      r12
rra      r12      ; n1*gamma_1*IN в r12
sub      r11,r12
rra      r11
rra      r11
rra      r11
rra      r11
rra      r11
add      r11,r12      ; T1 в r12
mov      r12,T1_697      ; сохранить T1 в ОЗУ
rla      r13      ; N3 * 32
rla      r13
rla      r13
rla      r13
rla      r13
rla      r12      ; T1 * 32
rla      r12
rla      r12
rla      r12
rla      r12
sub      IN,r12
add      r12,OUT      ; В r13 выходное значение
;*****

cmp      OUT,MAXLO      ; Сравнить выходное
                        ; значение с миниму-
                        ; мом

```

```

    jge      SWAP8
    mov      OUT,MAXLO      ; замена максимума
    mov      #00,ROW        ; строка = 0

SWAP8      ;*****
; окончание согласующей цепи
FILTER_697_ENDE
    .endif
FILTER_ENDE
    bis.b    #LPM0,SR        ; откл. CPU
    br       #DTMF_FILTER
;*****
; Прерывание 8-битного таймера
;*****
TIM_8B
    bic      #PD,&ACTL        ; вкл. АЦП
    bis      #001h,&ACTL      ; старт преобразования
    reti
;*****
; Прерывание АЦП
;*****
AD_INT
    bis      #PD,&ACTL        ; АЦП выл.
    bic      #0f0h,0(SP)      ; активный режим
                                ; по выходу? стек
    reti
;*****
; Определения ЖКИ
;*****
LCD_TYPE
a          .equ    01h
b          .equ    02h
c          .equ    10h
d          .equ    04h
e          .equ    80h
f          .equ    20h
g          .equ    08h
h          .equ    40h
;*****
; Таблица ЖКИ для символов DTMF
;*****
DTMF_Tab
    .byte    b+c              ; отображает «1»

```

```

.byte    a+b+d+e+g      ; отображает «2»
.byte    a+b+c+d+g      ; отображает «3»
.byte    a+b+c+e+f+g    ; отображает «A»
.byte    b+c+f+g        ; отображает «4»
.byte    a+c+d+f+g      ; отображает «5»
.byte    a+c+d+e+f+g    ; отображает «6»
.byte    c+d+e+f+g      ; отображает «B» b
.byte    a+b+c          ; отображает «7»
.byte    a+b+c+d+e+f+g  ; отображает «8»
.byte    a+b+c+d+f+g    ; отображает «9»
.byte    a+d+e+f        ; отображает «C»
.byte    a+d+e+f+g      ; отображает «E»
.byte    a+b+c+d+e+f    ; отображает «0»
.byte    a+e+f+g        ; отображает «F»
.byte    b+c+d+e+g      ; отображает «D» d

; *****
; Таблица векторов прерываний
; *****
.sect     «Int_Vect»     , 0ffe0h
.word    INIT           ; Порт0, биты от 2 до 7
.word    INIT           ; Базовый таймер Basic Timer
.word    INIT           ; нет источника
.word    INIT           ; нет источника
.word    INIT           ; таймер-порт Timer Port
.word    AD_INT         ; завершение преобразования
                        ; АЦП
.word    INIT           ; нет источника
.word    INIT           ; нет источника
.word    INIT           ; нет источника
.word    INIT           ; нет источника
.word    INIT           ; Watchdog/Timer, режим Timer
.word    INIT           ; нет источника
.word    TIM_8B         ; 8-битный таймер Timer
                        ; (P0.0 Int)
.word    INIT           ; Порт 0.0
.word    INIT           ; NMI, срыв генерации
.word    INIT           ; POR, внеш. Reset, Watchdog

```

9. Аппаратная часть

Преобразование аналоговых сигналов в цифровые осуществляется при помощи АЦП, встроенного в MSP430C25x. Источник опорного напряжения выведен на ножку SVCC. Оно может быть подано снаружи, либо в качестве ИОН можно использовать напряжение питания контроллера (VDD). Таким образом, 14-битный

диапазон соответствует диапазону напряжений от 0 до SVCC. Рабочее напряжение должно быть 5В, т.к. выбрана максимальная тактовая частота процессора 3.3 МГц (прим. перев. – на момент перевода макс. частота семейства – 8 МГц, а макс. рабочее напряжение 3,6В). Если это же напряжение используется в качестве опорного для АЦП и подключено встроенным ключом к выводу SVCC, то можно обойтись без внешнего ИОН. Один МЗР, таким образом, соответствует величине 0.305 мВ. Доступные в продаже коммерческие приёмники DTMF принимают сигнал в диапазоне от +1 дБм (600) до -31 дБм (600). Это соответствует амплитуде сигнала от 1.230В до 30.8 мВ. Исходя из этого, разрешение наименьшего входного напряжения составляет не менее 100 МЗР. Если всё же требуется работа с более слабыми входными сигналами, к выводу SVCC следует подключать внешний ИОН. Конденсатор (C1) фильтрует помехи от внешнего или внутреннего ИОН. Для вычисления компонентов цифрового фильтра требуются значения со знаком, не поддерживаемые имеющимся АЦП. По этой причине, аналоговый вход смещён резистивным делителем (R1/R2) на величину SVCC/2, а входной сигнал подаётся через разделительный конденсатор C2. Численное значение SVCC/2 позже вычитается из каждого входного отсчёта. Если установившееся выходное значение АЦП численно равно 01ffff при разрешении в 14 бит, то оно будет равно напряжению SVCC/2, и остаток от вычитания значения SVCC/2 будет 01ffff-01ffff=0000h. Положительный входной сигнал, следовательно, будет соответствовать положительным численным значениям и наоборот. Развязывающий конденсатор C2 вкупе с резисторами R3 + (R1 || R2) формируют ФВЧ. С учётом того, что частота среза выбрана примерно в 200 Гц, помехи с более низкой частотой, например сетевые, будут подавлены. Вход АЦП защищён от перенапряжения резистором R3 и стабилитроном D1. Принципиальная схема приведена на рис. 22:

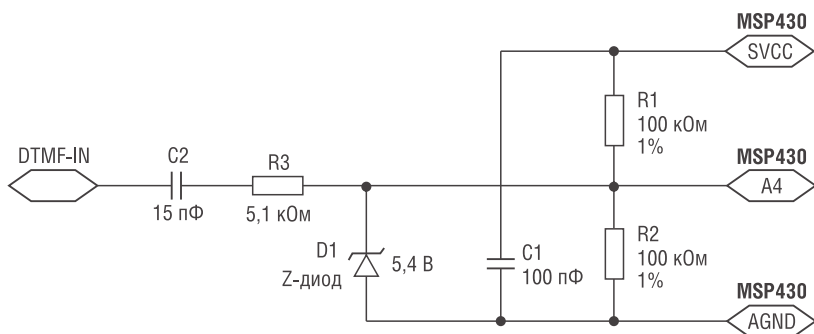


Рис. 22. Принципиальная схема подключения сигналов к АЦП

Так как разрядность измеряемого значения ограничена 14-ю битами, в то время, как обработка в ЦПУ производится с 16-ти битной разрядностью, перед вычислениями значения отсчётов сдвигаются влево дважды, что соответствует умножению на 4. Благодаря такому масштабированию, улучшается динамический диапазон и уменьшается шум, вносимый при округлении в процессе умножения.

10. Измерения и результаты

На рис. 23 показана длительность процесса фильтрации в пределах периода квантования. Канал 1 представляет прерывания АЦП; по каждому прерыванию выводится сигнал на выход порта. Канал 2 показывает длительность процесса вычисления фильтров; в начале процесса в порт выводится лог. «1», по завершению алгоритма – лог. «0».

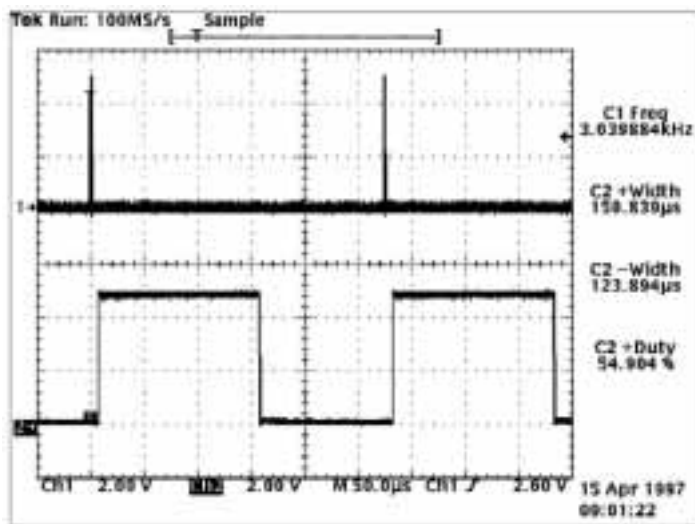


Рис. 23. Длительность вычисления фильтров

Как видно из рисунка, алгоритм занимает всего 55% доступного времени. Системная частота равна 3.3 МГц. Длительность такта при этом около 300 нс. Таким образом, доступно 915 тактов на период квантования. 474 такта требуются для выполнения 359 инструкций. Следовательно, результирующая вычислительная мощность равна 2.53 MIPS. В результате использования большого количества инструкций типа «регистр-регистр», в среднем требуется только 1.32 цикла на инструкцию.

Даже с учётом высокой вычислительной мощности, потребляемый устройством ток всего около 1.8 мА.

Программа занимает примерно 1КБ ПЗУ и требует 32 байта ОЗУ.

11. Заключение

Вычисления в системах реального времени требуют высокой вычислительной мощности, поэтому для таких целей часто используют цифровые сигнальные процессоры (DSP). Применение MSP430 в такой задаче для вычисления цифровых фильтров не является традиционным, в частности из-за того, что операции умножения осуществляются без использования аппаратного умножителя. Тем не менее, MSP430 является подходящим выбором в тех случаях, когда требуется распознавание частот в массиве цифровых отсчётов, а вычисленные значения не требуется выводить при помощи ЦАП. В MSP430 имеется в наличии высокопроизводительный АЦП. Его разрешение 14 бит гарантирует широкий динамический диапазон при вычислениях.

В нашем примере частоты с входными уровнями от 25 мВ RMS до 1.5 В RMS обрабатываются без проблем. Динамический диапазон при этом превышает 35 дБ, что позволяет избавиться от внешних приёмников DTMF. Так как 14-битный АЦП имеет теоретический динамический диапазон 84 дБ, есть возможность дальнейшего расширения динамического диапазона в обе стороны. Одним из способов является использование внешнего ИОН.

В зависимости от сложности фильтра и требуемой частоты сэмплирования, система может работать с сигналами вплоть до 5 кГц.

Расчёт и реализация восьми узкополосных полосовых фильтров, описанные в данном примере, демонстрируют:

- Возможность использования MSP430 и встроенного АЦП для задач фильтрации в режиме реального времени;
- Прекрасные свойства цифровых фильтров-резонаторов;
- Эффективность на уровне 1.32 цикла на инструкцию, возможную благодаря RISC-архитектуре процессора и ортогональности набора команд;
- Для 8-ми фильтров DTMF может быть достаточно 359 инструкций.

Ссылки:

1. Bundesamt fur Post und Telekommunikation (Federal Office for Post and Telecommunications): BAPT 223 ZV 5, Zulassungsvorschrift fur Endeinrichtungen zur Anschaltung an analoge Wahlanschlusse (ausgenommen Notruf- und Durchwahlanschlusse) des Telefonnetzes (approval specification for end equipment to be connected to analog dialing connections of the telephone network, except for

emergency and in-dialing connections) / ISDN of the Deutschen Bundespost Telekom; Bundesministerium fur Post und Telekommunikation, Draft, Bonn April 1994

2. Tietze / Schenk: Halbleiterschaltungstechnik; (Semiconductor Circuit Design), 10th. Edition; Springer Verlag, Berlin 1993

3. Marven / Ewers: A Simple Approach to Digital Signal Processing; Texas Instruments, 1994

4. Sauvagerd, Ulrich: A Ten-Channel Equalizer for Digital Audio-Applications; IEEE Transactions on Circuit and Systems, Vol. CAS-36, No 2, February 1989, pp. 276-280

5. Kaiser, Ulrich: RISP: Eine digitale Signalprozessorarchitektur mit reduziertem Befehlssatz fur Wellen-Digitalfilter (Digital Processor Architecture with Reduced Instruction Set for Wave Digital Filters); Dissertation, Fakultat fur Elektrotechnik, Ruhr-Universitat Bochum 1991

6. Lutz Bierl / Texas Instruments: MSP430 Family, Metering Application Report, Texas Instruments, Version 2.1, Jan 1996

7. Texas Instruments: MSP430 Family, Architecture User's Guide and Module Library, Texas Instruments, 1996

8. Texas Instruments: MSP430 Family, Software User's Guide, Texas Instruments, 1996

9. Texas Instruments: MSP430 Family, Assembly Language Tools User's Guide, Texas Instruments, 1996

10. Kitzberger, Holger: Aufbau einer Signalprozessorkarte mit MSP430 und Untersuchung eines Wellen-Digitalfilters; (Design of a signal processor card with the MSP430, and tests on a wave digital filter) Diplomarbeit, Fachhochschule Regensburg, April 1994

11. Siwy, Robert: Systementwicklung einer Telekom-Applikation zum Senden und Empfangen von DTMF-Signalen mit dem Microcontroller MSP430 (System development of a telecom application to send and receive digital signals with the Microcontroller MSP430); Diplomarbeit, Fachhochschule Landshut, Mai 1997



Семейство микроконтроллеров MSP430
Рекомендации по применению

Руководитель проекта
Дизайн обложки
Графика
Верстка

*Таранков И.В.
Георгадзе Е.С.
Писанко В.А.
Торочков Е.В.*

Подписано в печать 31.01.2005 г. Формат 62×90/16
Печать офсетная. Бумага ролевая. Гарнитура «HeliosCondensed»
Обложка – Бумага мел. матовая. Формат 62×64/8
Усл. печ. л. 34. Тираж 3000 экз. Зак. № 121

Отпечатано в типографии «Гран При», г. Рыбинск.
Тел. представительства в г. Москва: (095) 180-9500